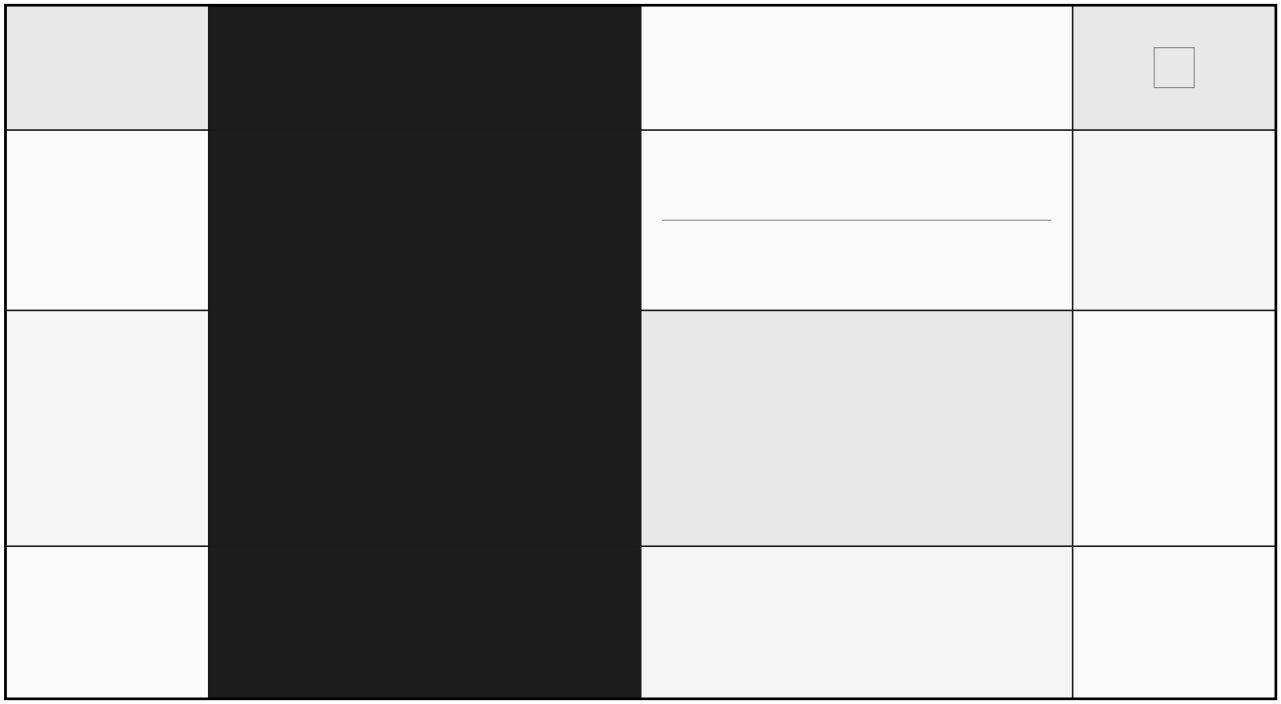
GOLANG

Giáo trình cập nhật liên tục về Kỹ nghệ phần mềm và DevOps/SRE



Đoàn Ngọc Hoàng Minh

MỤC LỤC

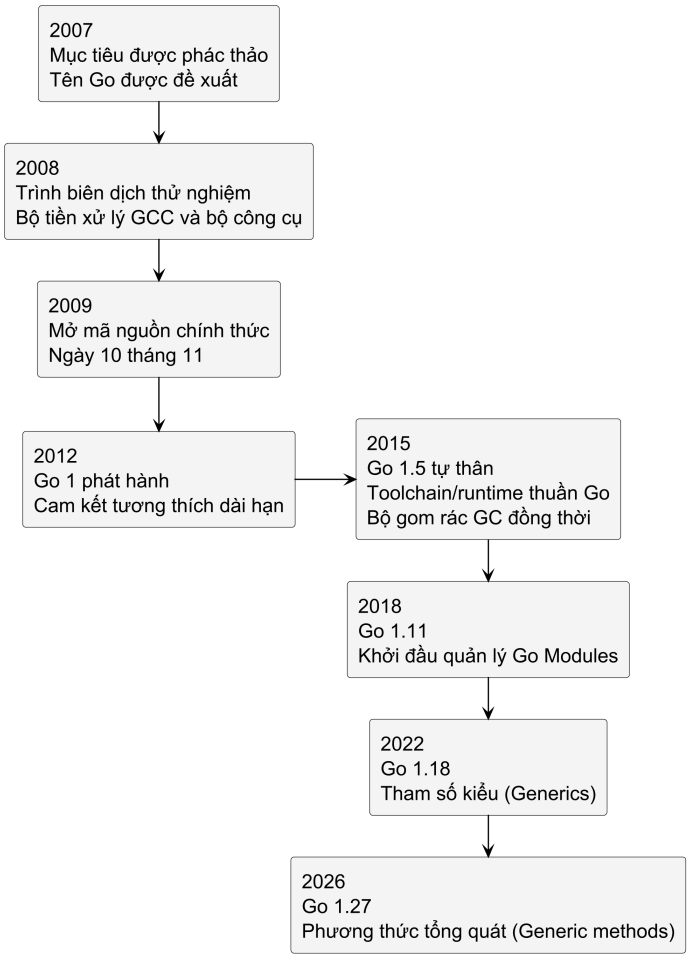
Trước khi viết dòng Go đầu tiên 3
Mở cửa vào Go 8
Chương 1 — Đọc và viết một chương trình Go 14
Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu 35
Chương 3 — Mô hình dữ liệu và trách nhiệm thay đổi 48
Chương 4 — Biên lỗi: để caller quyết định 73
Chương 5 — Package là ranh giới 86
Chương 6 — Thay đổi không sợ hãi 99
Chương 7 — Dữ liệu đi vào và đi ra 119
Chương 8 — Một race bắt đầu từ đâu 129
Chương 9 — Dòng công việc có áp suất 136
Chương 10 — Khi chương trình chậm hoặc phình 146
Chương 11 — Lần theo một request HTTP 163
Chương 12 — Một service sống và tắt thế nào 172
Chương 13 — Một thay đổi hoặc không có gì 185
Chương 14 — Khi kiểu trở thành dữ liệu 204
Chương 15 — Từ incident đến công cụ 216
Chương 16 — Thấy được hệ thống 225
Chương 17 — Đóng gói và điều phối 237
Chương 18 — Đưa thay đổi ra production 255
Chương 19 — Giữ type information khi abstraction lớn lên 273
Chương 20 — Dự án tổng kết: opsprobe từ mã nguồn đến vận hành 284
Chương 21 — Vòng lặp điều hòa và Controller Pattern 300
Chương 22 — Từ watch đến một controller Kubernetes thật 312
Chương 23 — Từ controller đến operator: API riêng và vòng đời tài nguyên 330
Chương 24 — Tự động hóa AWS bằng Go mà không biến credential thành bí mật dài hạn 350
Chương 25 — Git và GitHub dưới góc nhìn của một hệ thống tự động hóa 367
Chương 26 — Chuỗi cung ứng phần mềm có thể kiểm chứng 385
Chương 27 — Quan sát Linux từ kernel bằng eBPF và Go 409
Chương 28 — MCP và AIOps bằng Go: trao công cụ cho Agent mà không trao toàn quyền 431
Chương 29 — Kỹ sư và bằng chứng trong kỷ nguyên Agent 452
BACK MATTER — ATLAS MÃ NGUỒN 50 THƯ VIỆN GO DEVOPS & CLOUD 462
PHỤ LỤC A — ATLAS LỖI GO 517

Trước khi viết dòng Go đầu tiên

Go không bắt đầu bằng câu hỏi “cú pháp nào còn thiếu?”. Nó bắt đầu bằng một vấn đề kỹ thuật rất cụ thể: đến năm 2007, máy tính đã nhanh hơn rất nhiều, nhưng việc xây và sửa một hệ thống phần mềm lớn vẫn bị kéo chậm bởi ngôn ngữ, build system và lượng bookkeeping mà lập trình viên phải mang theo.

Ở Google khi ấy, Robert Griesemer, Rob Pike và Ken Thompson làm việc giữa những codebase rất lớn, hệ thống nối mạng và phần cứng đang chuyển mạnh sang nhiều lõi. C++ và Java là lựa chọn phổ biến cho server; chúng đem lại những năng lực quan trọng, đồng thời tạo ra một lượng phức tạp đáng kể trong quá trình build, quản lý phụ thuộc và tổ chức chương trình. Ở phía khác, các ngôn ngữ động cho cảm giác phát triển nhanh hơn, nhưng đánh đổi một phần kiểm tra kiểu và đặc tính thực thi. Bài toán không phải là chọn một phe thắng phe còn lại. Bài toán là liệu một ngôn ngữ có thể đem hiệu quả biên dịch, hiệu năng và an toàn kiểu vào cùng một quy trình làm việc nhẹ hơn hay không.

Ngày 21 tháng 9 năm 2007, ba người bắt đầu phác thảo mục tiêu của một ngôn ngữ mới trên bảng trắng. Vài ngày sau, ngày 25 tháng 9, Pike đề xuất cái tên Go. Thiết kế ban đầu diễn ra song song với công việc khác; đầu năm 2008, Thompson đã có compiler thử nghiệm sinh mã C để thử các ý tưởng. Đến giữa năm, dự án đủ rõ để hướng tới compiler dùng trong thực tế. Ian Lance Taylor bắt đầu GCC front end vào tháng 5 năm 2008; Russ Cox gia nhập cuối năm ấy và giúp đưa prototype, library cùng bộ công cụ tiến gần một ngôn ngữ có thể dùng.



Hình 1 — Dòng thời gian không kể mọi release. Nó chỉ giữ các bước làm thay đổi lời hứa của Go với người viết phần mềm: công khai, ổn định, triển khai lại nền tảng, phụ thuộc rõ ràng và generics tương thích ngược.

Go được mở mã nguồn ngày 10 tháng 11 năm 2009. Ba năm đầu là giai đoạn ngôn ngữ còn thay đổi nhanh: nhóm thiết kế và cộng đồng thử, bỏ, sửa, rồi mới chốt một nền tảng ổn định. Go 1 phát hành ngày 28 tháng 3 năm 2012. Từ đây, Go 1 Compatibility Promise trở thành một phần của văn hóa dự án: mã nguồn viết cho Go 1 được kỳ vọng tiếp tục build và chạy trong các bản Go 1 sau, với các ngoại lệ được nêu rõ. Lời hứa ấy làm thay đổi cách một feature được cân nhắc. Một bổ sung không chỉ cần hữu ích hôm nay; nó phải sống được cùng hệ sinh thái trong nhiều năm.

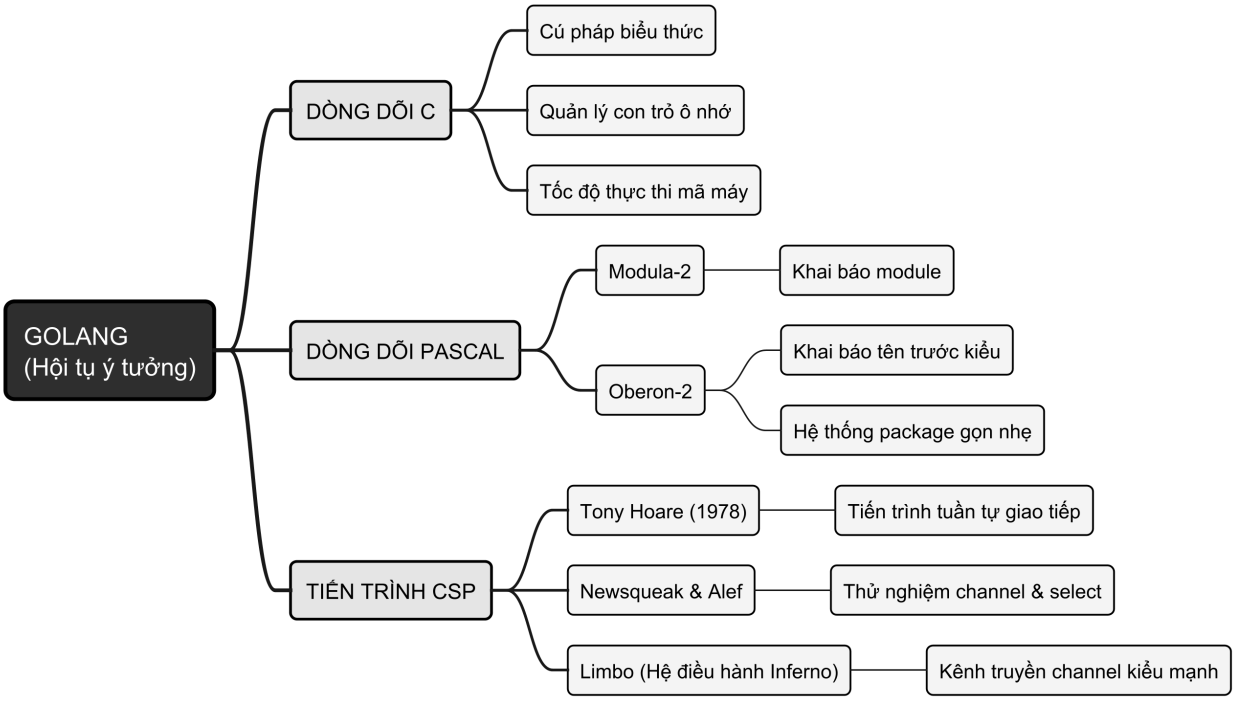
Go 1.5 là một cột mốc về implementation. Toolchain compiler chuyển từ C sang Go, runtime cũng trở thành Go kèm một phần assembly. Garbage collector được thiết kế lại để chạy phần lớn công việc đồng thời với chương trình; scheduler cho phép giá trị mặc định của GOMAXPROCS chuyển từ 1 sang số logical CPU. Đó là những thay đổi nằm dưới mặt ngôn ngữ, nhưng

chúng làm rõ mục tiêu ban đầu: một chương trình Go cần hợp với phần cứng nhiều lõi, không bắt người viết phải tự gánh mọi chi tiết của runtime.

Những thay đổi lớn sau đó cũng đi theo nhịp tiến hóa thay vì đập đi làm lại. Go 1.11 đưa modules vào như một lựa chọn thay cho GOPATH, với versioning và package distribution tích hợp vào toolchain; mô hình này dần thành cách quản lý phụ thuộc quen thuộc. Go 1.18, phát hành năm 2022, đưa tham số kiểu vào ngôn ngữ. Đến Go 1.27 năm 2026, generic methods nối phần còn thiếu vào câu chuyện đó: method có thể khai báo tham số kiểu riêng. “Go 2” vì thế không thành một nhánh rewrite tách rời; nhiều ý tưởng được thử, thảo luận và đưa dần vào dòng Go 1.x khi chứng minh được giá trị của chúng.

Một dòng họ ý tưởng, không phải một cây sao chép

Go thuộc nhiều dòng tư tưởng cùng lúc. Cú pháp cơ bản đặt nó gần họ C. Cách nhìn về khai báo, package và sự rõ ràng cấu trúc nhận ảnh hưởng đáng kể từ Pascal, Modula và Oberon. Concurrency có một lịch sử dài hơn: CSP của Tony Hoare đi qua Newsqueak, Alef và Limbo trước khi các kênh truyền trở thành một phần của Go. Những đường này giải thích vì sao một số ý tưởng nghe quen, chứ không làm Go thành bản sao của bất kỳ ngôn ngữ nào trong số đó.



Hình 2 — Nhánh CSP giải thích lịch sử của các kênh truyền; nhánh C và Pascal/Modula/Oberon giải thích những điểm xuất phát khác. Thiết kế cuối cùng vẫn được tạo cho bài toán engineering mà Go nhắm đến.

Một ngôn ngữ được sinh ra cho codebase lớn không chỉ cần syntax gọn. Nó cần một cách làm việc chung quanh syntax: build nhanh, formatter, package/phụ thuộc rõ, compiler cho phản hồi sớm và bộ công cụ có thể phân tích source. Đó là lý do Go thường được nói đến cùng gofmt, go test và go command, thay vì chỉ cùng một danh sách tính năng ngôn ngữ.

Thiết kế bằng cách từ chối

Trong giai đoạn đầu, nhóm sáng lập đặt nặng đồng thuận: một feature chỉ đi tiếp khi cả nhóm thấy nó thực sự cần. Câu hỏi không dừng ở “có thể thêm gì?”, mà còn là “có thể bỏ gì để hệ thống vẫn đủ sức làm việc?”. Kết quả là một bộ trade-off có chủ ý.

> **Một điều đáng nhớ:** sự vắng mặt của inheritance hierarchy, operator overloading, exception-style error handling hay generics trong giai đoạn đầu không phải bản tuyên ngôn rằng các cơ chế ấy luôn tệ. Mỗi cơ chế có ích trong những ngữ cảnh khác. Go ưu tiên một mô hình dễ đọc, dễ phân tích, biên dịch nhanh và dễ giữ ổn định; cái giá là một số biểu đạt quen thuộc không có mặt hoặc đến muộn hơn.

Không có type hierarchy buộc người đọc đi tìm cây kế thừa; thay vào đó, interface và composition sẽ được nói kỹ khi ta đã đọc vững mã nguồn cơ bản. Không có operator overloading giúp ý nghĩa của các toán tử ít thay đổi theo type. Error được trả như giá trị thay vì theo đường exception-style. Và generics được hoãn lại cho đến khi thiết kế có thể đem giá trị đủ lớn so với độ phức tạp nó thêm vào type system và toolchain. Đây là các lựa chọn kỹ thuật có thể tranh luận, không phải luật đạo đức.

Nhìn lịch sử theo cách này giúp giải thích cảm giác đầu tiên khi đọc Go. Bạn sẽ thấy nhiều câu ngắn, ít từ khóa, khai báo rõ và bộ công cụ hiện diện sớm. Những đặc điểm ấy là dấu vết của một mục tiêu: giảm ma sát cho người đang cùng người khác xây, đọc và sửa phần mềm lâu dài.

Sau gần hai thập kỷ tiến hóa, cách tốt nhất để gặp các quyết định ấy không phải đọc thêm lịch sử. Hãy đặt chúng trước mắt trong một tệp .go rất nhỏ.

Ghi chú kiểm chứng

1. Go Team. Frequently Asked Questions: Origins and Design. go.dev/doc/faq
2. Cox, Russ. Toward Go 2. 13 July 2017. go.dev/blog/toward-go2
3. Go Team. Go 1 and the Future of Go Programs. March 2012. go.dev/doc/go1compat
4. Gerrand, Andrew. Go 1.5 is released. 19 August 2015. go.dev/blog/go1.5
5. Bonventre, Andrew. Go 1.11 is released. 24 August 2018. go.dev/blog/go1.11
6. Go Team. Go 1.18 is released! 15 March 2022. go.dev/blog/go1.18
7. Go Team. Go 1.27 Release Notes. 2026. go.dev/doc/go1.27

Mở cửa vào Go

Một tệp nguồn Go chứa văn bản mã hóa UTF-8. Để chạy nó, bộ công cụ phải chọn các tệp thuộc chương trình, kiểm tra cú pháp và kiểu, biên dịch rồi liên kết thành tệp thực thi. Compiler có thể từ chối một phép cộng sai kiểu; nó không biết một endpoint có đúng với yêu cầu vận hành hay không. Ta sẽ dùng sự phân biệt ấy ngay từ chương trình đầu tiên.

Trước mắt, hãy đọc một tệp nhỏ và xác định ba điều: nó thuộc package nào, nó dùng tên nào từ package khác, và hàm nào sẽ chạy sau giai đoạn khởi tạo. Chưa cần biết allocator hay scheduler để trả lời đúng ba câu hỏi này.

Chương trình tối thiểu và Luồng thực thi

Lưu chương trình sau trong tệp `main.go`:

```
package main

import "fmt"

func main() {
    fmt.Println("he thong san sang")
}
```

Để chỉ thị bộ công cụ biên dịch và chạy tệp mã nguồn trên môi trường dòng lệnh, ta thực thi:

```
go run main.go
```

Chương trình ghi dòng `he thong san sang` ra đầu ra tiêu chuẩn, thường là terminal đang chạy lệnh. Dòng này xuất hiện khi `main` gọi `fmt.Println`, không phải khi compiler đọc string trong source.

Đọc từng thành phần

`package` và `import` là khai báo; `func main()` khai báo một hàm; trong thân hàm có lời gọi `fmt.Println(...)`. Dấu ngoặc nhọn giới hạn thân hàm, còn dấu ngoặc kép tạo string literal. Những vai trò này giúp anh phân biệt phần mô tả chương trình với hành động xảy ra khi chương trình chạy.

Khai báo gói và Không gian tên: `package main`

Mỗi tệp nguồn Go có một package clause trước các import và khai báo khác; comment có thể đứng trước nó. `package main` nói tệp này thuộc package tên `main`. Các khai báo cấp package được dùng chung giữa các tệp của package; tên import và biến cục bộ có phạm vi khác, sẽ được lần cụ thể ở Chương 1.

Theo đặc tả chính thức của Go, một chương trình hoàn chỉnh có thể thực thi độc lập (complete program) được tạo thành bằng cách liên kết một package không được package nào khác import, gọi là main package, cùng toàn bộ các package phụ thuộc bắc cầu của nó. Main package này bắt buộc phải có tên package là `main` và phải chứa một khai báo hàm `main` không nhận tham số và không trả về kết quả (`func main()`). Khai báo `package main` đơn thuần chưa đủ để sinh ra file thực thi nếu thiếu định nghĩa hàm `main`. Ngược lại, một package không phải `main` đại diện cho một thư viện; khi chạy lệnh `go build` trên một thư viện như vậy, bộ công cụ Go sẽ biên dịch mã nguồn để kiểm tra tính hợp lệ về cú pháp và kiểu dữ liệu nhưng mặc định không phát sinh tệp thực thi nào trên đĩa.

Nhập thư viện chuẩn: `import "fmt"`

`import "fmt"` khai báo phụ thuộc vào package thư viện chuẩn có import path `"fmt"`. Trong tệp này, tên `fmt` cho phép truy cập các định danh được export của package, chẳng hạn `fmt.Println`. Import không phải câu lệnh gọi một hàm để “nạp công cụ” mỗi khi luồng thực thi đi qua nó.

Import thông thường phải được dùng. Để thấy ranh giới, xóa lời gọi `fmt.Println` nhưng giữ `import "fmt"`: compiler báo import không được sử dụng. Đổi thành `import . "fmt"` vẫn không làm chương trình đó hợp lệ. Dot import đưa các định danh được export vào phạm vi tệp để viết

`Println(...)` không có tiền tố; nó không miễn quy tắc sử dụng import. Chỉ dạng `import _ "fmt"` trong phép thử này khai báo phụ thuộc chỉ để lấy hiệu ứng khởi tạo, không tạo tên package để gọi. Trong chương trình đầu tiên, cách viết thông thường `fmt.Println` giữ nguồn gốc tên rõ hơn.

Hàm khởi điểm và khởi tạo package: `func main()`

Từ khóa `func` định nghĩa một hàm. Tên hàm `main` đại diện cho điểm nhập cuộc thực thi logic của người dùng. Hàm `main` khởi điểm không nhận bất kỳ tham số nào và không trả về giá trị.

Theo Go specification, chương trình khởi tạo các package trước khi gọi `main`. Package được import phải khởi tạo xong trước package phụ thuộc vào nó; trong mỗi package, khởi tạo biến cấp package đi trước các hàm `init`. Đó là contract anh dùng để dự đoán code người dùng. Việc toolchain chuẩn dựng scheduler và allocator trước đó là tầng triển khai runtime, không phải danh sách thao tác mà ngôn ngữ bắt mọi implementation phải làm theo. Chương 5 sẽ đào sâu thứ tự khởi tạo; các chương runtime mới lần đường bootstrap đã pin theo phiên bản.

Truy xuất định danh công khai và Xuất dữ liệu: `fmt.Println`

Trong `fmt.Println`, `fmt` là tên package được import và `Println` là tên hàm được export của package đó. Dấu `.` nối hai tên trong một định danh có tiền tố package.

Một định danh được export khi bắt đầu bằng chữ hoa Unicode thuộc nhóm Lu và được khai báo ở cấp package, hoặc là tên field hay method. Vì vậy `Println` được dùng từ package khác; một biến cục bộ tên `Result` trong `main` không vì chữ hoa mà trở thành API. Theo contract của `fmt`, `Println` định dạng các argument, thêm newline và ghi ra standard output. Trong thư viện chuẩn Go 1.27.1, nó dùng `os.Stdout`; đường I/O tới OS phụ thuộc đối tượng đầu ra và nền tảng, không phải lời hứa về một file descriptor cụ thể của API `Println`.

Biên dịch Mã máy: Phân biệt `go run` và `go build`

Bộ công cụ Go cung cấp hai lệnh cơ bản với vai trò rõ ràng trong vòng đời phát triển:

Câu lệnh	Hợp đồng công cụ (Tool Contract)	Hành vi tệp trên đĩa	Bối cảnh sử dụng
`go run main.go`	Build rồi chạy main package được chỉ định.	Dùng executable tạm; không xuất binary vào working directory.	Thử nghiệm local.
`go build .`	Build package hiện tại; main package có thể tạo executable.	Với main package, mặc định xuất executable theo tên package/directory; `-o` chọn output khác.	Tạo artifact để phân phối.

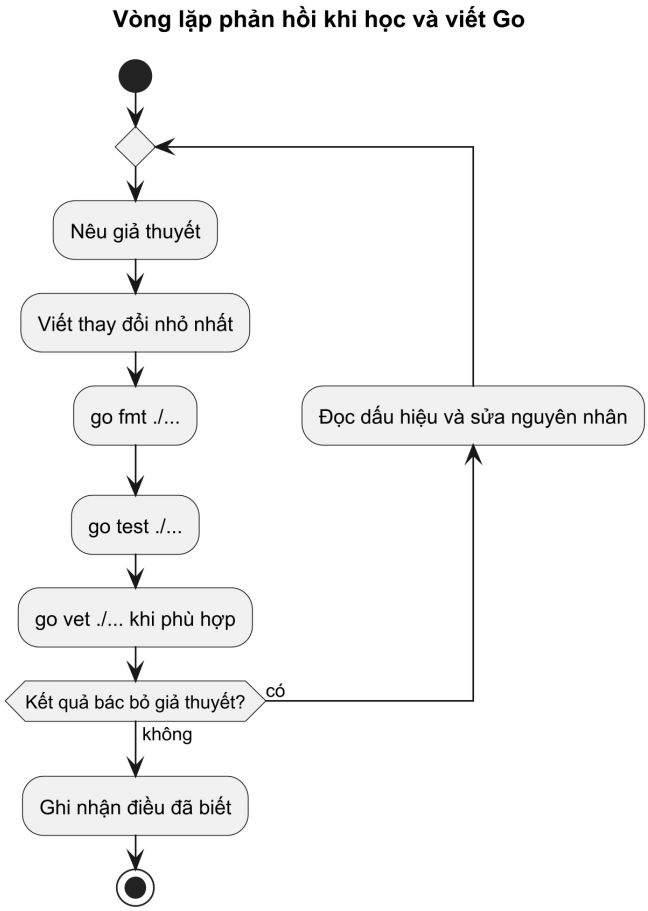
Trong lần gọi mặc định của toolchain Go 1.27.1, `go run` dùng tệp thực thi tạm và dọn work directory khi kết thúc; tùy chọn `-work` giữ lại thư mục đó để điều tra. Work directory khác `GOCACHE`, nơi toolchain giữ kết quả build có thể tái sử dụng giữa các lần gọi.

Tệp nhị phân sinh ra từ `go build` là mã máy thực thi trực tiếp trên vi kiến trúc CPU đích. Nó không chạy trên máy ảo như bytecode của Java và không dựa vào trình thông dịch như Python.

Tệp thực thi do toolchain chuẩn tạo ra có mã ứng dụng cùng phần hỗ trợ runtime cần cho chương trình, không phải toàn bộ source runtime hay mọi thư viện Go. Khả năng chạy trên máy khác còn phụ thuộc target OS, kiến trúc, cách liên kết và tài nguyên ngoài như tệp cấu hình. Chương 17 sẽ kiểm tra các phụ thuộc ấy trong container; ở đây build thành công chưa đủ để kết luận triển khai ở đâu cũng chạy.

Vòng lặp Phản hồi: Giả thuyết, Đo đạc và Giải thích

Trước khi sửa một dòng, hãy dự đoán: thay đổi này làm chương trình bị từ chối trước khi chạy, hay vẫn chạy nhưng cho kết quả khác? Sau đó chạy phép thử nhỏ để so dự đoán với bằng chứng. Cách làm này hữu ích hơn việc chỉ ghi nhớ một thông báo lỗi.



Hình 3 — Vòng lặp phản hồi giả thuyết - đo đạc - sửa mã nguồn. Trình biên dịch không phải là chướng ngại vật; trình biên dịch là công cụ kiểm định tĩnh sớm nhất cho các dự đoán của bạn.

Bảng dưới đây minh họa ba thí nghiệm nhỏ về ranh giới kiểm tra tĩnh của Go compiler:

Thí nghiệm kiểm chứng	Thao tác thay đổi mã nguồn	Phần chính của diagnostic	Cơ chế phân tích của Compiler
Đổi tên gói khởi điểm	Đổi `package main` thành `package worker`, sau đó chạy `go run main.go`.	`is not a main package`	Gói đích không thỏa mãn điều kiện tạo tệp thực thi độc lập; đây là kiểm tra của `go run`.
Import không sử dụng	Thêm dòng `import "time"` nhưng không tham chiếu định danh được export nào của `time`.	`"time" imported and not used`	Compiler kiểm tra việc sử dụng import; không phải chứng minh toàn bộ dependency graph đã tối thiểu.
Khai báo biến cục bộ thừa	Khai báo `workerID := 1` trong `main` nhưng không dùng.	`workerID declared and not used`	Kiểm tra usage của biến cục bộ; không phải chứng minh toàn bộ dead code đã được phát hiện.

Compiler bắt một lớp lỗi cú pháp, kiểu và usage trước khi chạy; nó không chứng minh phần lớn lỗi vận hành đã bị loại bỏ. Deadline, quyền, resource budget và yêu cầu nghiệp vụ vẫn cần phép kiểm tra khác. Chương tiếp theo đi vào định danh, kiểu dữ liệu và phạm vi biến để anh biết chính xác compiler đang kiểm tra điều gì.

Chương 1 — Đọc và viết một chương trình Go

Chương trình dưới đây dùng một mã trạng thái HTTP đã có sẵn để tạo nhãn rồi in kết quả. Nó chưa gọi mạng, và quy tắc `status >= 500` chỉ là policy của ví dụ, không phải định nghĩa đầy đủ về sức khỏe dịch vụ. Ta sẽ lần từ từng tên và dấu trong source tới giá trị được tạo, nhánh được chọn và dữ liệu đi ra terminal.

```
package main

import "fmt"

const service = "checkout"

func classify(status int) string {
    if status >= 500 {
        return "failed"
    }
    return "healthy"
}

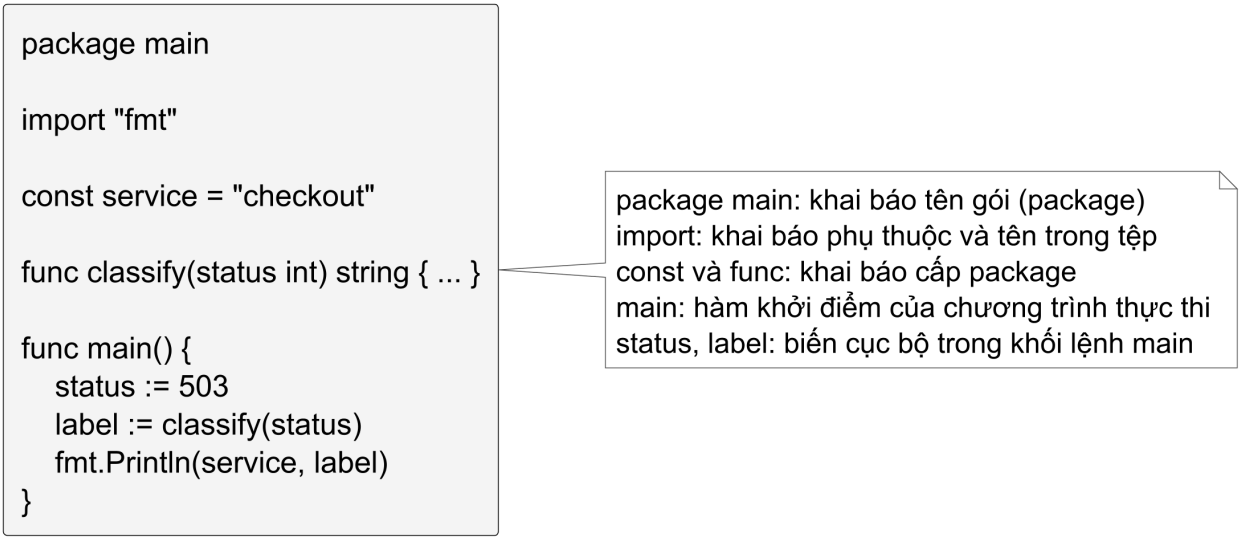
func main() {
    status := 503
    label := classify(status)
    fmt.Println(service, label)
}
```

Chạy chương trình trong một module nhỏ

Lưu đoạn mã trên thành `main.go` trong một thư mục trống. Trước khi chạy, tạo một module cục bộ cho thư mục ấy. Lệnh `go mod init` tạo tệp `go.mod` và khai báo module path; một module quản lý một hoặc nhiều package cùng các dependency của chúng. Module chưa phải là một package, cũng không có nghĩa mã phải được công bố lên mạng. Ở bài đầu này, chỉ cần biết thư mục đang thuộc một module và công cụ Go dùng `go.mod` để xác định module ấy cùng các dependency khi cần.

```
go mod init example.com/first-go
go run .
```

Lệnh đầu tạo `go.mod`; `example.com/first-go` là tên định danh minh họa, không phải một địa chỉ mà anh phải sở hữu. Lệnh sau yêu cầu Go xây package `main` trong thư mục hiện tại rồi chạy executable tạm. Ta sẽ trở lại module, đường dẫn import, đồ thị phụ thuộc và cách một package trở thành ranh giới ở Chương 5. Hiện tại chỉ cần giữ một mốc thực hành: mỗi lab nằm trong một thư mục có `go.mod` riêng, và terminal phải đứng đúng thư mục của lab trước khi chạy lệnh được ghi trong bài.



Hình 4 — Giải phẫu trực quan cấu trúc một tệp nguồn Go. Tên gói, các khai báo cấp tệp và hàm khởi điểm có ranh giới rõ ràng.

Để thấu suốt chương trình trên, ta lần theo luồng chuyển dịch trạng thái qua ba câu hỏi: chương trình bắt đầu thực thi ở đâu, giá trị nào được tạo ra, và thông điệp nào được gửi ra ngoài? Câu trả lời lần lượt là: điểm khởi đầu nằm tại hàm `main`, giá trị số nguyên `503` được gán vào biến `status` rồi truyền qua hàm `classify` theo ngữ nghĩa giá trị để nhận lại chuỗi phân loại, và cuối cùng lời gọi hàm `fmt.Println` gửi dữ liệu qua thiết bị đầu ra tiêu chuẩn.

Bản đồ Cấu trúc của một Tệp Nguồn

Từ trên xuống dưới, tệp mã nguồn được phân chia thành bốn vùng không gian cú pháp rõ rệt:

Khai báo gói (`package`): Xác định không gian tên và đơn vị biên dịch mà tệp trực thuộc.

Khai báo import (`import`): Đặt phụ thuộc và tên dùng để tham chiếu các định danh được export từ package khác.

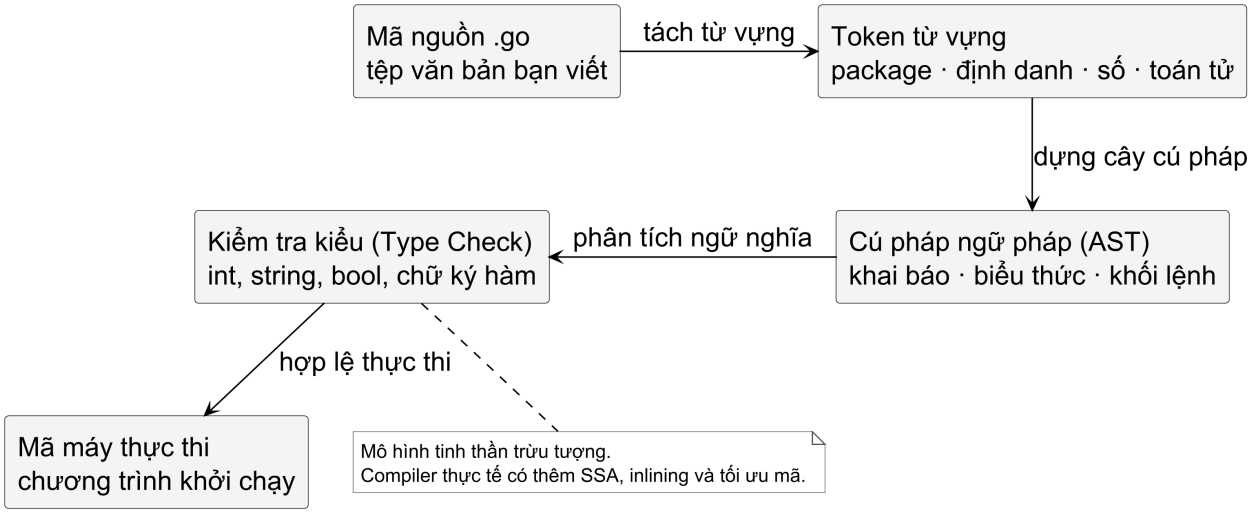
Khai báo cấp tệp: Không gian định nghĩa hằng số (`const`), biến toàn cục của gói (`var`), hoặc các kiểu dữ liệu và hàm dùng chung (`func`).

Hàm khởi điểm (`func main`): Điểm neo mà Go runtime kích hoạt để bắt đầu thực thi logic sau khi hoàn tất giai đoạn khởi tạo môi trường.

Dòng mã	Vai trò ngữ pháp	Tác động thực thi
`package main`	Khai báo gói	Báo hiệu cho compiler tạo tệp thực thi độc lập khi có hàm `main`.
`import "fmt"`	Khai báo import	Đặt tên `fmt` trong phạm vi tệp; không phải một thao tác I/O lúc chạy.
`const service = "checkout"`	Khai báo hằng	Gắn định danh `service` với chuỗi bất biến tại lúc biên dịch.
`func classify(status int) string`	Khai báo hàm	Xác lập chữ ký hàm nhận `int` và trả về `string`.
`status >= 500`	Biểu thức so sánh	Đánh giá điều kiện nhị phân, sinh giá trị `bool`.
`return "failed"`	Câu lệnh trả về	Trả giá trị chuỗi cho bên gọi và chuyển giao quyền điều khiển.
`status := 503`	Khai báo biến ngắn	Giới thiệu biến `status` mang kiểu `int` và gán giá trị 503.
`fmt.Println(service, label)`	Lời gọi hàm	Định dạng các argument rồi ghi ra standard output.

Việc định vị chính xác vai trò ngữ pháp của từng dòng giúp ta đọc thông điệp lỗi của trình biên dịch một cách bình tĩnh: lỗi có thể bắt nguồn từ một biểu

thức sai cú pháp, một phép so sánh cấn kiểu dữ liệu, hay một biến bị gọi ngoài phạm vi sống, thay vì cảm giác hoang mang rằng toàn bộ chương trình đang bị hỏng.



Hình 5 — Mô hình các lớp xử lý của Go compiler. Trình biên dịch đọc tệp văn bản từ cấp độ từ vựng (token), dựng cây cú pháp trừu tượng (AST), kiểm tra chặt chẽ tính tương thích của kiểu dữ liệu, rồi mới sinh mã máy thực thi.

Quan hệ Giữa Khai báo, Biểu thức, Câu lệnh, Kiểu và Giá trị

Để xây dựng một mô hình tư duy vững chắc, ta cần phân biệt rạch ròi các khái niệm nền tảng thường bị đánh đồng trong lập trình:

Khai báo: Giới thiệu một tên và thứ mà tên ấy chỉ tới, như biến, hằng số, kiểu hoặc hàm, trong một phạm vi xác định. Với `const service = "checkout"`, tên `service` chỉ tới một hằng số; nó không phải một câu gán lại biến mỗi lần `main` chạy.

Biểu thức (Expression): Kết hợp toán hạng và toán tử để tính giá trị. Ví dụ `status >= 500` cho một giá trị kiểu `bool`. Lời gọi hàm cũng là biểu thức: có thể trả một hoặc nhiều kết quả; lời gọi không trả kết quả được dùng như câu lệnh. Ngữ cảnh quyết định cách dùng các kết quả ấy. Đánh giá biểu thức có thể gọi hàm làm thay đổi dữ liệu hoặc gây panic, nên không đồng nghĩa với một phép tính không có tác động phụ.

Câu lệnh (Statement): Là đơn vị thực thi hoàn chỉnh chỉ dẫn máy tính thực hiện một hành động cụ thể, chẳng hạn như rẽ nhánh điều kiện (`if`), lặp

vòng (`for`), gán giá trị (`=`), hoặc trả về từ hàm (`return`). Câu lệnh cấu thành luồng chảy động của chương trình.

Kiểu dữ liệu (Type): Là định nghĩa trừu tượng quy định tập hợp các giá trị hợp lệ và tập hợp các phép toán được phép thực hiện trên các giá trị đó. Kiểu cho compiler biết các operation hợp lệ; representation vật lý, kích thước và cách mã máy dùng giá trị còn phụ thuộc implementation và kiến trúc.

Giá trị (Value): Là dữ liệu hoặc kết quả cụ thể thuộc một kiểu. Kiểu quyết định những phép toán có nghĩa đối với giá trị đó. Việc trình biên dịch có phải hiện thực giá trị trong thanh ghi, bộ nhớ hay loại bỏ chỗ lưu trữ khi tối ưu là quyết định triển khai, không phải một phần của định nghĩa giá trị ở tầng ngôn ngữ.

Lời gọi hàm (Function call): Là cơ chế chuyển giao quyền thực thi từ hàm gọi sang hàm được gọi, kèm theo việc đánh giá các biểu thức đối số và truyền các giá trị đó theo ngữ nghĩa truyền giá trị của Go. Một Go compiler có thể inline thân hàm hoặc truyền tham số qua thanh ghi/ngăn xếp theo ABI nội bộ, nhưng đó là quyết định triển khai cần được đo hay quan sát ở đúng toolchain, không phải contract của source code.

Định danh và Từ khóa

Trong ví dụ trên, `service`, `fmt`, `Println`, `classify`, `status` và `label` là các định danh (identifiers) — những cái tên do lập trình viên hoặc thư viện quy ước để đại diện cho biến, kiểu hoặc hàm. Ngược lại, `package`, `import`, `const`, `func`, `if` và `return` là các từ khóa (keywords) bất biến của ngôn ngữ Go, được dành riêng cho trình phân tích cú pháp.

Định danh bắt đầu bằng chữ cái Unicode hoặc `_`, theo sau bởi chữ cái Unicode, chữ số Unicode hoặc `_`. Quy ước Go thường dùng `MixedCaps` hay `mixedCaps` cho tên nhiều từ. `snake_case` vẫn hợp lệ về cú pháp; chọn cách đặt tên nhất quán với API và code xung quanh, không coi convention là lệnh cấm của compiler.

Export cần hai điều kiện: tên bắt đầu bằng chữ hoa Unicode thuộc nhóm Lu, và tên được khai báo ở cấp package hoặc là tên field hay method. `Println`

thỏa cả hai; `classify` không được export. Biến cục bộ `Label` trong thân hàm vẫn chỉ là biến cục bộ, không trở thành API vì viết hoa.

Khai báo Biến và Khởi tạo Bộ nhớ

Biến giữ một giá trị thuộc kiểu xác định. Bốn cách viết dưới đây khác nhau ở việc ghi kiểu và giá trị khởi tạo, không quyết định biến nằm trên stack hay heap:

Khai báo tường minh bằng `var`: `var name Type = value` chọn kiểu cho biến; biểu thức khởi tạo phải gán được cho kiểu ấy. Ví dụ `var status int = 503` tạo biến kiểu `int` với giá trị ban đầu 503.

Khai báo suy luận kiểu: Cú pháp `var name = value` cho phép compiler tự động suy diễn kiểu từ giá trị khởi tạo ở vế phải, giúp loại bỏ sự lặp lại thừa thãi.

Khai báo không khởi tạo và Quy tắc Zero Value: Nếu một biến được khai báo dưới dạng `var name Type` mà không gán giá trị khởi tạo tường minh, đặc tả ngôn ngữ Go bảo đảm biến đó nhận giá trị zero value theo đúng kiểu dữ liệu.

Khai báo ngắn trong thân hàm bằng `:=`: Cú pháp `name := value` kết hợp đồng thời việc khai báo biến mới, suy luận kiểu và gán giá trị. Cú pháp này chỉ hợp lệ bên trong thân hàm, không được phép dùng ở cấp độ tệp.

Kiểu dữ liệu	Giá trị Zero Value theo Đặc tả Ngôn ngữ	Ghi chú ngữ nghĩa
Số nguyên (`int`, `int64`, `byte`...)	`0`	Giá trị số không.
Số thực (`float32`, `float64`)	`0.0`	Số không dấu phẩy động.
Logic (`bool`)	`false`	Trạng thái sai mặc định.
Chuỗi ký tự (`string`)	`""`	Chuỗi rỗng với độ dài bằng 0.
Con trỏ, slice, map, channel, func, interface	`nil`	Zero value có ý nghĩa và giới hạn thao tác riêng theo từng kiểu; Chương 2–3 lần rõ các trường hợp dữ liệu.

Với biến được khai báo theo ngữ nghĩa Go thông thường, không có giá trị khởi tạo tường minh thì biến nhận zero value, không nhận byte rác từ chỗ lưu trữ trước đó. Đây không phải lời hứa cho bộ nhớ ngoài được đưa vào qua `unsafe` hay cgo. Zero value cũng chưa chắc hợp lệ với nghiệp vụ: `var port int` đọc được giá trị 0, nhưng API yêu cầu port `1..65535` vẫn phải từ chối nó.

Cam kết ngữ nghĩa này không đồng nghĩa với việc mỗi biến đều bắt buộc phải chiếm một ô nhớ vật lý trong RAM và trình biên dịch luôn phải thực hiện lệnh memset xóa sạch các byte về số 0. Tùy thuộc vào chiến lược tối ưu hóa, trình biên dịch có thể xóa giá trị trên thanh ghi CPU, gấp hằng số (constant-fold), hoặc thậm chí loại bỏ hoàn toàn việc cấp phát lưu trữ nếu biến không còn được dùng đến. Việc xóa các byte bộ nhớ về 0 chỉ là một cơ chế triển khai (implementation mechanism) trong các trường hợp vùng nhớ lưu trữ thực sự được hiện thực hóa (materialized) trên stack frame hoặc heap.

Khai báo ngắn cần ít nhất một biến mới không phải `_`. Nó có thể dùng lại biến đã khai báo trong cùng block nếu kiểu không đổi; khi ấy biến cũ được gán giá trị, không tạo bản thứ hai. Còn khai báo tên trùng ở block con tạo một biến khác.

> **Dừng để dự đoán.** Đọc đoạn mã sau và dự đoán chính xác hai dòng output: biến `x` trong block con nhận giá trị nào? Sau khi thoát khỏi block con, `x` có giá trị là 1, 2 hay 4?

```
x := 1
x, y := 2, 3
{
    x := 4
    fmt.Println(x, y)
}
fmt.Println(x, y)
```

Đáp án — chỉ đọc sau khi đã tự làm Kết quả in ra lần lượt là `4 3` rồi `2 3`. Tại câu lệnh `x, y := 2, 3`, do xuất hiện trong cùng block và có thêm biến mới `y`, Go áp dụng quy tắc redeclaration: biến `x` cũ nhận giá trị mới là 2 mà không tạo ra bản sao thứ hai. Khi bước vào block con, khai báo ngắn `x := 4` tạo ra một biến hoàn toàn mới nằm trong phạm vi từ vựng hẹp hơn, che khuất biến ngoài và khiến lệnh in đầu tiên xuất ra `4 3`. Sau khi thoát khỏi block con, biến nội bộ này hết phạm vi từ vựng; lệnh in cuối cùng đọc lại biến `x` của block cha (đang giữ giá trị 2) và biến `y` (3), cho kết quả `2 3`.

Phản ví dụ biên giới làm rõ bản chất: nếu bỏ `y` để viết tiếp `x := 2` trong cùng block, compiler lập tức báo lỗi `no new variables on left side of :=`. Ngược lại, phép gán `x = 2` thì hợp lệ vì nó chỉ cập nhật giá trị của biến sẵn có. Phạm vi từ vựng, không phải hình thức của cú pháp `:=`, quyết định biến nào được dùng lại.

Hằng số: giá trị chưa cần một ô nhớ

Trước khi đọc tiếp, thử dự đoán vì sao `const small = 500` gán được cho `int16`, nhưng một variable `n := 500` không tự gán được cho variable `int16`. Hai bên cùng viết số 500; khác biệt nằm ở kiểu và thời điểm kiểm tra, không ở độ lớn của RAM.

```
const small = 500
var code int16 = small
n := small
// var other int16 = n // cần conversion tường minh
_ = code
_ = n
```

`small` là hằng số nguyên chưa có kiểu. Khi gán vào `code`, giá trị phải biểu diễn được bằng `int16`; khi dùng `:=` không có kiểu đích, kiểu mặc định của hằng số nguyên là `int`. Vì vậy `n` đã là variable kiểu `int`. Nếu thay 500 bằng 70000, phép gán vào `int16` bị compiler từ chối. Nếu viết `const small int = 500`, anh đã chọn kiểu ngay trong khai báo; hằng số có kiểu không còn được đối xử như hằng số chưa có kiểu ở phép gán ấy. Hằng số biểu diễn giá trị ở thời điểm biên dịch, không phải một variable bất biến có địa chỉ để lấy bằng &.

`iota` giúp diễn đạt một chuỗi hằng số trong cùng một nhóm khai báo. Nó bắt đầu từ 0, tăng theo từng mục khai báo hằng số trong nhóm, không theo số dòng văn bản; một mục khai báo nhiều tên vẫn dùng cùng giá trị `iota`. Mục bỏ biểu thức dùng lại biểu thức và kiểu của mục trước. Hãy tính ba giá trị rồi mới chạy thử:

```
const (
    Pending = iota
    Running
    Stopped
)
const Restarted = iota // nhóm mới, trở lại 0
```

Đây là tên cho các số nguyên, chưa phải một cơ chế enum tự kiểm tra giá trị hợp lệ. Variable `int` vẫn nhận được số không thuộc ba trạng thái. Nếu số được lưu vào database hay truyền qua giao thức, chèn một dòng giữa nhóm có thể đổi ý nghĩa các giá trị về sau; khi ấy giá trị tường minh và chính sách tương thích quan trọng hơn việc gõ ít ký tự. Chương 3 sẽ thêm một kiểu có tên để phân biệt ý nghĩa, nhưng kiểu ấy cũng không tự kiểm tra miền giá trị.

Trong `labs/edition-contracts`, `TestConstantsAndIdentity` giữ trace này thành thí nghiệm nhỏ. Hãy đổi hằng số chưa có kiểu thành hằng số kiểu `int`, rồi giải thích lỗi gán trước khi sửa. Quy tắc nguồn là các mục Constants, Constant declarations và Representability của Go specification, phiên bản ngôn ngữ Go 1.27; test chỉ kiểm tra những trường hợp đã chọn.

Phạm vi Từ vựng và Vòng đời Lưu trữ: Scope không đồng nghĩa với Lifetime

Một trong những nhầm lẫn phổ biến nhất của người mới học là đồng nhất phạm vi từ vựng của tên biến (scope) với vòng đời lưu trữ của giá trị trong bộ nhớ (lifetime). Đây là hai khái niệm độc lập:

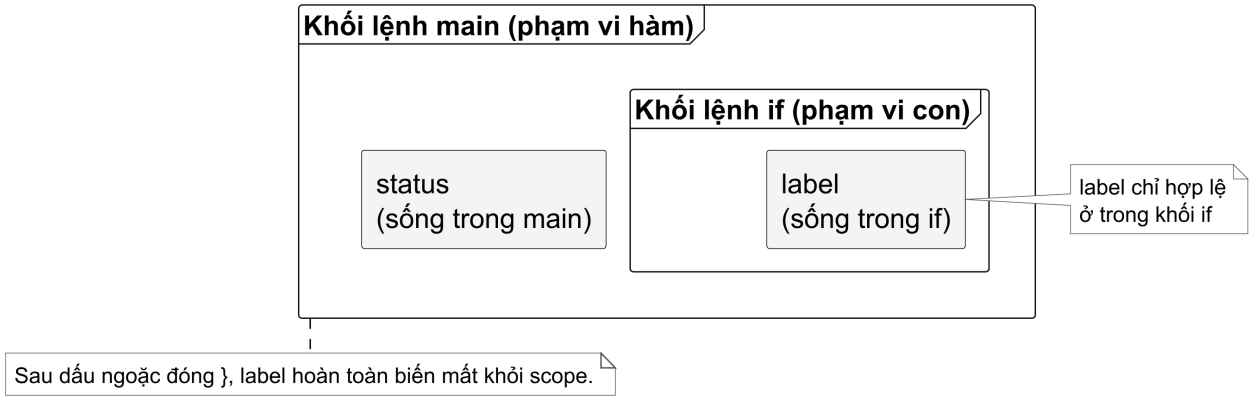
Phạm vi từ vựng: Vùng source mà một tên có thể được tham chiếu hợp lệ. Block có ngoặc nhọn là trường hợp dễ thấy; Go còn có các block ngầm của tệp, `if`, `for`, `switch` và các cấu trúc khác. Khi gặp tên trùng, phải tìm khai báo thuộc phạm vi nào, không chỉ đếm ngoặc.

Vòng đời lưu trữ (Storage Lifetime): Là khoảng thời gian mà vùng nhớ lưu trữ giá trị của biến thực sự tồn tại trong không gian bộ nhớ của tiến trình.

```
func main() {
    status := 503
    if status >= 500 {
        label := "failed"
        fmt.Println("Bên trong if:", label)
    }
    // Lệnh fmt.Println(label) ở đây sẽ gây lỗi biên dịch:
    // undefined: label
}
```

Biến `label` được khai báo bên trong khối lệnh `if`. Khi luồng đọc của trình biên dịch vượt ra ngoài dấu ngoặc nhọn đóng `}`, phạm vi từ vựng của tên `label` kết thúc; định danh này nằm ngoài lexical scope tại vị trí đó, nên quá trình phân giải tên (name resolution) của compiler không tìm thấy khai báo hợp lệ, từ chối câu lệnh với thông báo: `undefined: label`.

Tuy nhiên, việc định danh hết phạm vi từ vựng không đồng nghĩa với việc giá trị bị hủy ngay lập tức. Nếu một con trỏ hoặc closure vẫn làm giá trị có thể quan sát được, compiler sẽ chọn representation đủ dài cho lifetime đó; trong một lần build nó có thể là heap. Nếu không, compiler có thể dùng thanh ghi, stack hoặc bỏ hẳn storage. Scope là quy tắc tên của ngôn ngữ; vị trí lưu trữ là chi tiết tối ưu hóa, nên đừng suy ra cái sau chỉ từ cái trước.



Hình 6 — Trực quan hóa ranh giới phạm vi từ vựng (lexical scope) của biến. Tên biến khai báo trong khối con không thể được tham chiếu từ khối cha bên ngoài.

Một cạm bẫy kỹ thuật nguy hiểm là hiện tượng che khuất biến (variable shadowing). Khi lập trình viên sử dụng toán tử `:=` bên trong một khối lệnh con với một tên biến trùng với biến đã có ở khối lệnh cha, Go sẽ tạo ra một biến hoàn toàn mới, che khuất biến bên ngoài trong suốt phạm vi của khối con:

```
func main() {
    retries := 0
    if retries == 0 {
        retries := 3
        fmt.Println("Gia tri trong if:", retries)
    }
    fmt.Println("Gia tri ngoai if:", retries)
}
```

Chương trình in ra giá trị 3 bên trong khối `if`, nhưng biến `retries` ban đầu ở ngoài vẫn giữ nguyên giá trị 0. Để cập nhật biến của khối cha, ta phải sử dụng phép gán `=` thay vì toán tử khai báo ngắn `:=`.

Hệ thống Kiểu Dữ liệu Cơ bản

Go kiểm tra kiểu trước khi chạy. Với hai biến số thuộc các kiểu khác nhau như `int` và `int64`, cùng giá trị hoặc cùng kích thước lưu trữ không làm phép cộng hay phép gán trực tiếp hợp lệ. Ta cần conversion tường minh trong ví dụ dưới đây. Quy tắc gán được rộng hơn riêng trường hợp số; Chương 3 sẽ phân biệt identity của kiểu, assignability và interface satisfaction.

Specification quy định `int` và `uint` cùng kích thước, 32 hoặc 64 bit; toolchain Go 1.27.1 trên target amd64 đang dùng chọn 64 bit. Các kiểu `int8`, `int16`, `int32`, `int64` và bản không dấu tương ứng có width cố định. `byte` là alias của `uint8`; `rune` là alias của `int32`, thường dùng để biểu diễn điểm mã Unicode, nhưng kiểu `int32` không tự kiểm tra một số có phải điểm mã hợp lệ hay không.

Kiểu số thực gồm `float32` và `float64` tuân theo chuẩn IEEE-754. Toán tử so sánh bằng `==` hoàn toàn hợp lệ với số thực khi tính bằng nhau tuyệt đối (exact equality) chính là hợp đồng bài toán đòi hỏi (chẳng hạn so sánh với `0.0` hoặc hằng số cố định). Tuy nhiên, đối với các giá trị sinh ra từ chuỗi phép tính có sai số làm tròn (rounding error), ta thường cần tiêu chí gần bằng phù hợp với từng miền nghiệp vụ cụ thể, thay vì áp đặt máy móc một giá trị epsilon cố định cho mọi bài toán.

Kiểu logic `bool` chỉ nhận một trong hai giá trị `true` hoặc `false`, đồng hành cùng các toán tử logic gồm phép và &&, phép hoặc `||`, và phép phủ định `!`.

`string` là chuỗi byte bất biến: anh có thể gán một string khác cho biến, nhưng không gán vào `text[0]` để sửa byte của string đang có. `len` đếm byte, không đếm ký tự hiển thị. Chương 2 sẽ dùng UTF-8 và phép chuyển sang byte slice để kiểm tra hai điều này; không cần giả định layout bộ nhớ của string để dùng đúng contract.

Phép chuyển đổi kiểu bắt buộc phải thực hiện tường minh theo cú pháp `T(v)`, trong đó `T` là kiểu đích và `v` là giá trị cần chuyển:

```
var a int = 10
var b int64 = 20
var c int64 = int64(a) + b
```

Hằng số chưa có kiểu như `500` được xử lý theo quy tắc representability đã thử ở phần hằng số: `var code int16 = 500` hợp lệ vì giá trị biểu diễn được trong `int16`. Điều này khác với gán một biến đã có kiểu `int` sang `int16`; đừng gọi mọi phép gán hợp lệ là conversion ngầm hay xem hằng số như ngoại lệ duy nhất của hệ thống kiểu.

Toán tử và Biểu thức

Toán tử số học bao gồm cộng `+`, trừ `-`, nhân `*`, chia `/`, và chia lấy dư `%`. Phép chia giữa hai số nguyên `5 / 2` luôn cho kết quả nguyên `2` do phần thập phân bị cắt cụt; nếu muốn nhận kết quả số thực, ít nhất một toán hạng phải mang kiểu số thực như `5.0 / 2`.

Toán tử so sánh gồm `==`, `!=`, `<`, `<=`, `>` và `>=`. Kết quả là giá trị logic; theo spec, kết quả so sánh là boolean chưa có kiểu và nhận kiểu phù hợp theo ngữ cảnh. Chẳng hạn `failed := status >= 500` tạo biến `failed` kiểu `bool`.

Về mặt cú pháp, `count++` và `count--` là câu lệnh, không phải biểu thức. Vì vậy `x = count++` không hợp lệ; Go cũng không có dạng prefix `++count`. Quy tắc này không cấm mọi biểu thức có side effect: một lời gọi hàm vẫn có thể thay đổi state.

Xuất Dữ liệu Định dạng với Gói `fmt`

Gói thư viện chuẩn `fmt` cung cấp ba cơ chế xuất dữ liệu chủ đạo với các đặc tính riêng biệt:

`fmt.Print`: Xuất các đối số nối tiếp nhau ra stdout mà không tự động chèn khoảng trắng phân cách (trừ khi hai đối số liên tiếp không phải là chuỗi) và không tự ngắt dòng.

`fmt.Println`: Xuất các đối số ra stdout, tự động thêm dấu cách giữa các đối số và luôn thêm ký tự xuống dòng ở cuối.

`fmt.Printf`: Định dạng chuỗi theo mẫu (format string) thông qua các ký hiệu động từ định dạng (verbs) bắt đầu bằng ký tự %.

Ký hiệu Verb	Ý nghĩa định dạng	Ví dụ biểu diễn
`%v`	Biểu diễn giá trị mặc định theo kiểu dữ liệu tự nhiên.	`fmt.Printf("%v", 503)` xuất `503`.
`%+v`	Mở rộng biểu diễn struct kèm theo tên từng trường dữ liệu.	`fmt.Printf("%+v", req)` xuất `{Path:/ ID:1}`.
`%#v`	Biểu diễn theo cú pháp Go của giá trị (Go-syntax representation); không phải mọi kết quả đều là biểu thức compile trực tiếp được.	`fmt.Printf("%#v", "ok")` xuất `"ok"`.
`%T`	Xuất tên định danh của kiểu dữ liệu.	`fmt.Printf("%T", 503)` xuất `int`.
`%t`	Xuất giá trị logic (`true` hoặc `false`).	`fmt.Printf("%t", true)` xuất `true`.
`%d`	Xuất số nguyên theo hệ thập phân cơ số 10.	`fmt.Printf("%d", 255)` xuất `255`.
`%x` / `%X`	Xuất số nguyên hoặc mảng byte dưới dạng hệ thập lục phân.	`fmt.Printf("%x", 255)` xuất `ff`.
`%f`	Xuất số thực dấu phẩy động (có thể định dạng độ rộng như `%.2f`).	`fmt.Printf("%.2f", 3.14159)` xuất `3.14`.
`%s`	Xuất chuỗi ký tự hoặc mảng byte dưới dạng văn bản.	`fmt.Printf("%s", "ready")` xuất `ready`.
`%q`	Xuất chuỗi có dấu ngoặc kép và escape theo cú pháp Go; không thay validation hay lọc secret.	`fmt.Printf("%q", "ready")` xuất `"ready"`.
`%p`	Xuất giá trị con trỏ ở dạng thập lục phân có tiền tố 0x; không phải địa chỉ RAM vật lý.	`0xc000014088` là địa chỉ minh họa; `fmt.Printf("%p", &status)` có thể cho địa chỉ khác theo lần chạy.

Điều khiển Luồng: Rẽ nhánh và Lặp

Go tối giản hóa cấu trúc điều khiển luồng, loại bỏ các biến thể cú pháp dư thừa để giữ mã nguồn đồng nhất trên diện rộng.

Rẽ nhánh Điều kiện với `if / else`

Cú pháp `if` trong Go không yêu cầu cặp ngoặc đơn bọc điều kiện, nhưng bắt buộc phải có cặp ngoặc nhọn `{}` bao quanh khối lệnh, bất kể khối lệnh chỉ có một dòng đơn.

Go hỗ trợ câu lệnh khởi tạo ngắn đặt ngay trước mệnh đề điều kiện, phân cách bởi dấu chấm phẩy `;`:

```
if err := executeCheck(); err != nil {
    fmt.Println("Kiem tra that bai:", err)
}
```

Tên `err` dùng được trong điều kiện `err != nil`, thân `if` và các nhánh `else` của cùng câu lệnh. Nó không dùng được sau khi câu lệnh `if` kết thúc. Đây là phạm vi tên, không phải cơ chế tự đóng tài nguyên mà error có thể liên quan.

Lựa chọn Rời rạc với `switch`

Cấu trúc `switch` trong Go tự động kết thúc khi một nhánh `case` thỏa mãn, không đòi hỏi lập trình viên phải chèn lệnh `break` thủ công như các ngôn ngữ họ C. Nếu chủ đích muốn luồng chạy tiếp xuống nhánh kế tiếp, từ khóa `fallthrough` phải được khai báo tường minh.

Go cũng hỗ trợ `switch` không điều kiện (`switch {}`). Các `case` được kiểm tra theo thứ tự và nhánh đầu tiên đúng được chọn; dạng này có thể làm một chuỗi điều kiện dễ đọc hơn, không phải lúc nào cũng nên thay `if / else if`:

```
switch {
case status >= 500:
    return "failed"
case status >= 400:
    return "warning"
default:
    return "healthy"
}
```

Vòng lặp Duy nhất: `for`

Go chỉ có một từ khóa lặp duy nhất là `for`. Mọi nhu cầu lặp từ ba thành phần truyền thống, lặp theo điều kiện (tương đương `while`), lặp vô tận, cho đến duyệt tập hợp dữ liệu (`for range`) đều được biểu đạt thông qua `for`.

```
// Dang 1: Lap co bien dem truyen thong
for i := 0; i < 3; i++ {
    fmt.Println("Lan thu:", i)
}

// Dang 2: Lap theo dieu kien (tuong duong while)
for retries < 3 {
    retries++
}

// Dang 3: Lap vo han (cho den khi break hoac return)
for {
    if isDone() {
        break
    }
}

// Dang 4: Duyet mang hoac slice voi range
statuses := []int{200, 404, 500}
for index, code := range statuses {
    fmt.Printf("Chi so %d mang gia tri %d\n", index, code)
}
```

Trong ví dụ `[]int` này, `code` nhận bản sao giá trị phần tử; gán lại `code` không sửa phần tử của `statuses`. Muốn sửa phần tử gốc, dùng index rồi

gán `statuses[index]`. Nếu phần tử về sau chứa pointer, slice hay map, bản sao vẫn có thể dẫn tới dữ liệu dùng chung; Chương 2–3 sẽ lần sự khác biệt đó. _ bỏ kết quả index khi không cần dùng.

Hàm và Ngữ nghĩa Truyền Giá trị

Hàm là đơn vị đóng gói logic độc lập, nhận vào các tham số hình thức và trả về kết quả cho bên gọi.

Go truyền đối số theo giá trị (pass by value): tham số nhận giá trị của đối số, không trở thành chính biến của bên gọi. Đây là ngữ nghĩa của lời gọi, không phải cam kết về một lệnh sao chép bit hay vị trí stack/thanh ghi; compiler có thể tối ưu nếu giữ nguyên hành vi quan sát được. Với số nguyên, gán lại tham số không đổi biến gốc. Với con trỏ hoặc cấu trúc chứa con trỏ, giá trị được truyền vẫn có thể dẫn tới vùng dữ liệu chung.

Go cho phép hàm trả về cùng lúc nhiều giá trị độc lập, tạo tiền đề cho khuôn mẫu xử lý lỗi tiêu chuẩn của ngôn ngữ bằng cách trả về cặp giá trị gồm kết quả dữ liệu và đối tượng lỗi:

```
func parsePort(raw string) (int, error) {
    if raw == "" {
        return 0, fmt.Errorf("cong mang khong duoc de trong")
    }
    return 8080, nil
}
```

`parsePort` ở đây chỉ là stub để đọc chữ ký hai kết quả: nó chưa phân tích số và trả 8080 cho mọi input không rỗng. Không dùng nó để validation port; parser có kiểm tra range sẽ xuất hiện ở Chương 5. Nơi gọi nhận cả hai kết quả và kiểm tra lỗi trước khi dùng số:

```
port, err := parsePort("8080")
if err != nil {
    fmt.Println("Loi phan tich:", err)
    return
}
fmt.Println("Cong mang:", port)
```

Nhập môn Con trỏ: Địa chỉ Ô nhớ và Phép Giải Tham chiếu

Biến là tên gọi đại diện cho một vị trí lưu trữ giá trị. Con trỏ (pointer) là một giá trị chứa địa chỉ trong không gian địa chỉ của tiến trình trỏ tới một vị trí lưu trữ khác.

Toán tử lấy địa chỉ & trích xuất giá trị con trỏ trỏ tới biến đứng sau nó. Toán tử giải tham chiếu * đặt trước một biến con trỏ để đọc hoặc ghi đè giá trị tại vị trí mà con trỏ đang tham chiếu. Giá trị con trỏ in ra qua %p đại diện cho địa chỉ trong không gian địa chỉ ảo của tiến trình theo quy ước của nền tảng, không phải địa chỉ RAM vật lý.

```
func main() {
    x := 100
    var p *int = &x

    fmt.Printf("Gia tri goc: %d\n", x)     // 100
    fmt.Printf("Dia chi &x:  %p\n", &x)    // Vi du 0xc000014088
    fmt.Printf("Gia tri *p:  %d\n", *p)    // 100

    *p = 200
    fmt.Printf("Gia tri moi: %d\n", x)     // 200
}
```

Con trỏ cho phép các hàm khác nhau cùng thao tác và biến đổi một vùng dữ liệu mà không phải sao chép toàn bộ khối dữ liệu đó qua từng lời gọi hàm.

Thí nghiệm Nhỏ: Hợp ngữ Compiler Plan 9 đối chiếu với Mã máy objdump

Để hiểu rõ đường đi từ mã nguồn xuống phần cứng, ta cần phân biệt rạch ròi hai tầng biểu diễn thường bị nhầm lẫn: bản danh sách hợp ngữ trung gian của compiler (`-S`) và mã máy thực thi sau liên kết (`objdump`).

Thí nghiệm dùng Go 1.27.1, target `windows/amd64`, với source ở `labs/part1-reading-go/testdata/classify/main.go` đúng như ví dụ mở chương. Chạy từ thư mục `labs/part1-reading-go`. Cờ `-l` tắt inlining cho package đang build để hàm nhỏ `classify` còn hiện ra riêng; đây là điều kiện quan sát, không phải cấu hình tối ưu production. Một lệnh tạo binary đồng thời in listing compiler:

```
go build -gcflags="-l -S" -o classify.exe ./testdata/classify
```

Biểu diễn trung gian Plan 9 từ `cmd/compile/internal/ssagen`:

```
main.classify STEXT nosplit size=34 align=0x0 args=0x8
    locals=0x0 funcid=0x0
    TEXT    main.classify(SB), NOSPLIT|NOFRAME|ABIInternal, $0-8
    CMPQ    AX, $500
    JLT 21
    LEAQ    go:string."failed"(SB), AX
    MOVL    $6, BX
    RET
    LEAQ    go:string."healthy"(SB), AX
    MOVL    $7, BX
    RET
```

Đoạn trích giữ các lệnh liên quan tới điều kiện và kết quả, bỏ các dòng metadata; header được ngắt dòng cho vừa khung. Plan 9 dùng symbol như `go:string."healthy"(SB)` chưa có địa chỉ liên kết cuối cùng. Đọc chính binary vừa tạo bằng `objdump`, thay vì build lại một source hay bộ cờ khác rồi so hai kết quả:

```
go tool objdump -s "main\.classify" classify.exe
```

Các cột địa chỉ, byte lệnh và instruction dưới đây được trích từ lần build local ấy; cột tên tệp/dòng và padding bị lược bỏ. Địa chỉ tuyệt đối có thể đổi khi môi trường hay artifact thay đổi, không phải giá trị cần học thuộc:

```
  0x1400a5260: 483df4010000    CMPQ AX, $0x1f4
  0x1400a5266: 7c0d            JL 0x1400a5275
  0x1400a5268: 488d053e100000  LEAQ runtime.rodata+685(SB), AX
  0x1400a526f: bb06000000      MOVL $0x6, BX
  0x1400a5274: c3              RET
  0x1400a5275: 488d057b110000  LEAQ runtime.rodata+1015(SB), AX
  0x1400a527c: bb07000000      MOVL $0x7, BX
  0x1400a5281: c3              RET
```

Sự đối chiếu giữa hai đầu ra làm nổi bật bản chất của toolchain:

Trong hàm `classify` của build này, ABI nội bộ dùng `AX` cho argument `status`; hàm không dựng stack frame riêng. Điều đó không có nghĩa mọi hàm Go đều không dùng stack, hay tên thanh ghi là contract của source.

Giá trị `$500` trở thành `$0x1f4` trong cách hiển thị của objdump, cùng giá trị số. Nhánh `JLT 21` của listing tương ứng `JL 0x1400a5275`, được mã hóa bằng `7c 0d`. Cả hai đều đi tới đường trả `"healthy"` khi `status < 500`; không có bằng chứng từ cặp đầu ra này rằng linker đã đảo điều kiện.

Các symbol string được giải quyết thành địa chỉ trong vùng mà objdump hiển thị là `runtime.rodata`. `LEAQ` đặt địa chỉ dữ liệu vào `AX`, `MOVL` đặt độ dài byte 6 hoặc 7 vào `BX`, rồi `RET` trả kết quả theo ABI của build này. Đây là biểu diễn cụ thể của string result, không thay định nghĩa string bất biến ở tầng ngôn ngữ.

Đọc Thông điệp Biên dịch: Phân tích Tĩnh Thay vì Phỏng đoán

Khi gặp lỗi biên dịch, hãy tìm tên, biểu thức hoặc khai báo mà diagnostic đang chỉ tới. Bảng này tách các loại lỗi để anh chọn một phép thử nhỏ:

Tình huống kiểm chứng	Thao tác tạo lỗi cú pháp	Thông báo thực tế từ Go 1.27.1 Compiler	Tầng phân tích của Compiler
Định danh chưa khai báo	Gán `status = 503` mà chưa từng khai báo biến.	`undefined: status`	Tầng phân giải tên (Symbol table lookup): Định danh không tồn tại trong bảng ký hiệu hiện hành.
Vượt ngoài tầm vực từ vựng	Gọi `fmt.Println(label)` ngoài khối `if`.	`undefined: label`	Tầng phân tích tầm vực (Lexical scoping): Tên biến nằm ngoài lexical scope tại điểm gọi.
Khai báo lại không có biến mới	Dùng `x := 10; x := 20` liên tiếp.	`no new variables on left side of :=`	Kiểm tra cú pháp khai báo ngắn: Vế trái của `:=` bắt buộc phải có ít nhất một biến mới.
Sai lệch kiểu tham số hàm	Truyền `int` vào hàm nhận `string`.	`cannot use status (variable of type int) as string value in argument to takesString`	Tầng kiểm tra kiểu tĩnh (`types2`): Chữ ký hàm từ chối đối số không tương thích kiểu.
Phép toán lệch kiểu số học	Thực hiện `a + b` giữa `int` và `int64`.	`invalid operation: a + b (mismatched types int and int64)`	Tầng kiểm tra kiểu tĩnh (`types2`): Cấm phép toán hai ngôi ngầm định giữa hai kiểu khác nhau.

Thông điệp biên dịch chỉ ra lỗi mà compiler phát hiện tại vị trí đang xét; nó không chứng minh chương trình đã đúng với yêu cầu hay an toàn khi chạy. Phân biệt lỗi phân giải tên, cú pháp và kiểu dữ liệu giúp bạn đặt giả thuyết cụ thể rồi kiểm chứng bằng một thay đổi nhỏ. Chương 2 tiếp tục cách điều tra đó với slice: vì sao một bản sao giá trị vẫn có thể chia sẻ bộ nhớ nền.

Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Chạy đoạn này trước khi cố giải thích nó. Nếu bạn đoán a và b khác nhau sau dòng b[0] = 99, đó là một dự đoán rất hợp lý - và cũng là cánh cửa vào phần này.

```
a := []int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a)
fmt.Println(b)
```

Kết quả từ lab part2-values là:

```
[99 20 30]
[99 20 30]
```

Đừng ghi nhớ riêng một câu “slice nguy hiểm”. Thay vào đó, ta sẽ lần ngược từ kết quả: Go đã copy cái gì khi chạy b := a, và dữ liệu nào không được copy?

Một baseline để dự đoán

Với một int, assignment tạo ra một giá trị độc lập. Cả a và b đều có giá trị riêng của chính chúng.

```
a := 10
b := a
b = 99

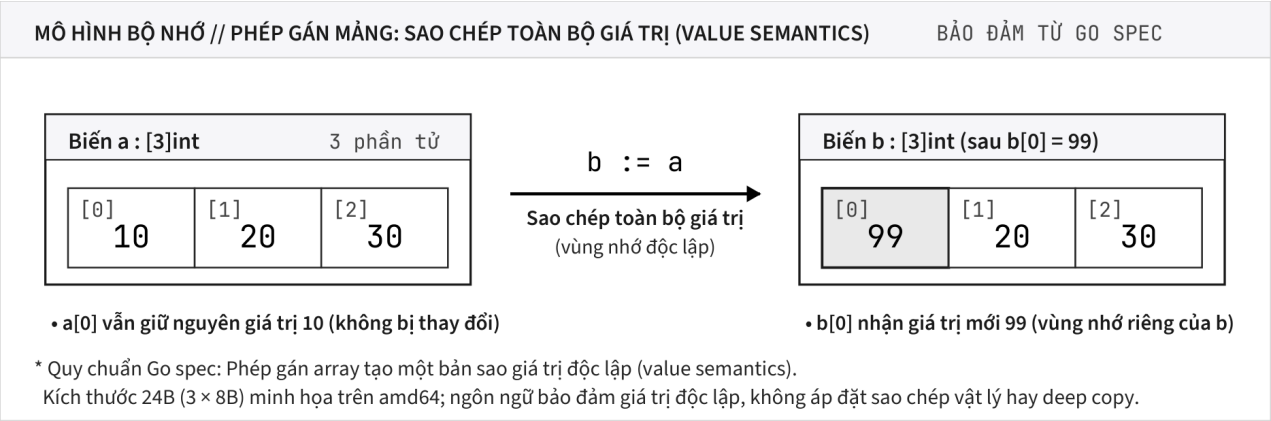
fmt.Println(a, b) // 10 99
```

Go đã copy value của a vào b. Sau đó b nhận value mới; a không có lý do gì để đổi. Đây là quy tắc nền: assignment copy value. Điều còn thiếu là một value có thể chứa gì.

Array giúp ta kiểm tra quy tắc ấy với nhiều phần tử. Độ dài là một phần của type, nên [3]int và [4]int là hai array type khác nhau. Khi array được gán, toàn bộ array value được copy.

```
a := [3]int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a) // [10 20 30]
fmt.Println(b) // [99 20 30]
```



Hình 7 — Assignment của array copy các phần tử. Việc thay b[0] chỉ chạm vào array b.

Đến đây, trực giác "copy nghĩa là độc lập" vẫn đúng. Sự thay đổi xuất hiện chỉ khi ta bỏ một ký tự trong type: [3]int trở thành []int.

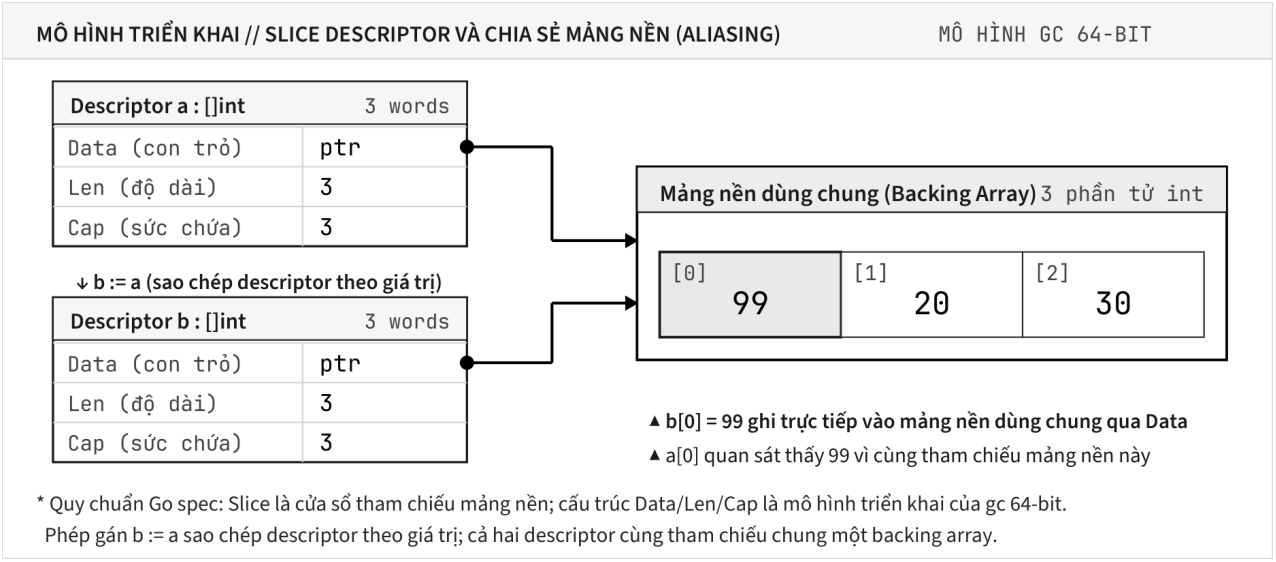
Khoảnh khắc slice đổi kết quả

```
a := []int{10, 20, 30}
b := a
b[0] = 99
```

[]int không phải array không ghi độ dài. Nó là slice type. Một slice value mô tả một đoạn liên tiếp của underlying array.

Trong source runtime Go 1.27.1 (`src/runtime/slice.go`), mô hình descriptor gồm con trỏ tới dữ liệu, `len int` và `cap int`. Với target amd64 đang dùng, ba trường này tương ứng 24 byte. Đây là biểu diễn đã pin, không phải kích thước mà specification bắt mọi implementation phải giữ. Để dự đoán mutation ở đây, chỉ cần biết slice nhìn vào array nền và có length, capacity riêng.

Khi phép gán `b := a` diễn ra, ngôn ngữ thực hiện sao chép giá trị theo ngữ nghĩa (copy by value). Trong mô hình ba từ máy, các thành phần của descriptor được sao chép sang `b` với cùng con trỏ dữ liệu, cùng `len` và cùng `cap`. Trình tối ưu hóa có thể giữ các trường này trên thanh ghi mà không bắt buộc phải phát sinh một lệnh chép 24 byte vật lý ở bộ nhớ. Biến `b` sở hữu một bản sao giá trị riêng biệt về cửa sổ nhìn (`len` và `cap`), nhưng con trỏ bên trong vẫn dẫn về cùng mảng nền ban đầu.



Hình 8 — b := a copy slice value, gồm cửa sổ nhìn vào dữ liệu, len và cap. Cả hai descriptor vẫn dẫn tới cùng underlying array, nên b[0] thay đổi phần tử mà a cũng nhìn thấy.

Vì vậy hai câu sau cùng đúng, và không mâu thuẫn:

Bảng 1 — Copy value và chia sẻ storage là hai tầng khác nhau của cùng một phép gán.

Câu nói	Đúng ở đâu?
Assignment copy value.	b nhận một slice descriptor riêng: đổi len của b bằng reslice không tự đổi len của a.
Hai slice có thể cùng đổi dữ liệu.	Slice descriptor cùng tham chiếu một backing array; mutation một element hiện ra qua mọi slice cùng nhìn element đó.

Từ aliasing dùng cho tình huống nhiều đường tham chiếu đến cùng dữ liệu. Nó không phải lỗi tự thân. Aliasing giúp cắt một cửa sổ trên buffer mà không copy cả buffer. Nó thành bug khi một người đọc tưởng mình có dữ liệu riêng nhưng thực tế lại đang sửa vùng dùng chung.

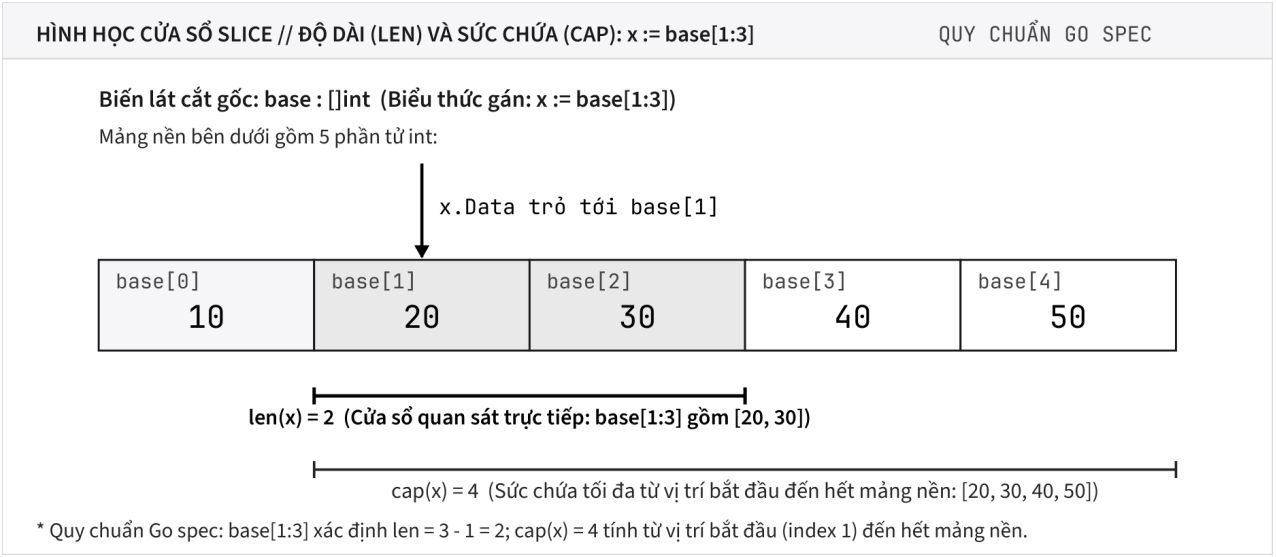
Cửa sổ, length và capacity

Đặt một slice lên một slice khác để thấy descriptor thực sự đang mô tả gì.

```
base := []int{10, 20, 30, 40, 50}
x := base[1:3]

fmt.Println(x)      // [20 30]
fmt.Println(len(x)) // 2
fmt.Println(cap(x)) // 4
```

x bắt đầu ở phần tử 20 của base và kết thúc trước 40. Vì thế length là 2. Capacity đếm số phần tử từ điểm bắt đầu của x đến cuối backing array mà x hiện có thể mở rộng tới; ở đây là 20, 30, 40, 50, nên là 4.



Hình 9 — x nhìn thấy hai phần tử đầu của cửa sổ. cap(x) không nói x đang có bốn phần tử; nó nói x có thể được reslice hoặc append trong phạm vi backing array hiện tại.

Hãy tự dự đoán trước khi chạy:

```
base := []int{1, 2, 3, 4}
a := base[:2]
b := base[1:3]
a[1] = 99

fmt.Println(base)
fmt.Println(a)
fmt.Println(b)
```

Kết quả là [1 99 3 4], [1 99], [99 3]. a[1] và b[0] đều chỉ vào phần tử base[1]. Hình vẽ không thay thế được việc lần index: nó chỉ cho biết vì sao có thể có một phần tử chung để lần.

append là một phép thử về capacity

append trả về slice kết quả. Go specification phân biệt hai trường hợp: nếu capacity hiện tại đủ cho các phần tử được thêm, append dùng lại underlying array; nếu không đủ, append cấp underlying array mới đủ lớn cho kết quả. Vì slice trả về có thể đổi độ dài hoặc mảng nền, dạng thông thường là `s = append(s, value)`. Quy tắc dùng lại mảng nền là ngữ nghĩa của ngôn ngữ; cách chọn capacity mới khi phải tăng mảng là chi tiết triển khai.

```
s := make([]int, 2, 4)
s[0], s[1] = 10, 20
old := s

s = append(s, 30)
s[0] = 99

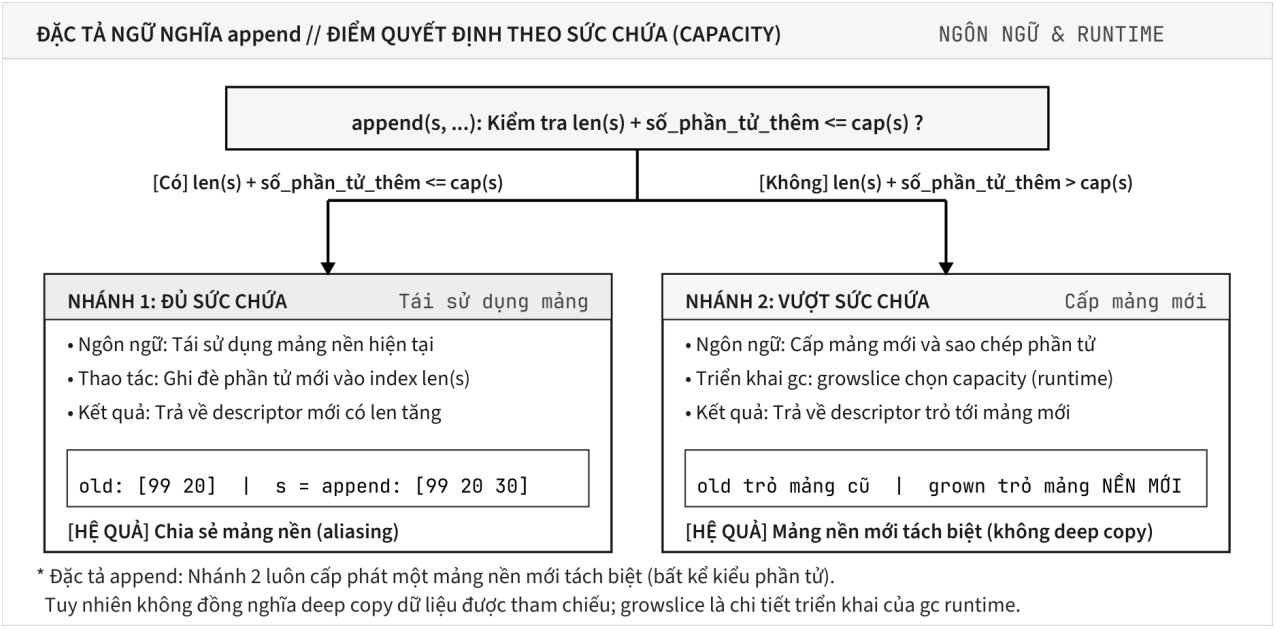
fmt.Println(old) // [99 20]
fmt.Println(s)   // [99 20 30]
```

Ở đây `s` còn capacity. `old` và `s` có length khác nhau, nhưng vẫn cùng nhìn hai element đầu của backing array. Không dùng chuỗi capacity tình cờ đo được để thiết kế ownership. Trong Go 1.27.1, `nextslicecap` ở `runtime/slice.go` chọn capacity mới theo cả length cần có và capacity cũ; nếu length mới vượt hai lần capacity cũ, nó có thể chọn thẳng length mới. Đường thông thường tăng gấp đôi dưới ngưỡng 256, rồi chuyển dần về mức tăng nhỏ hơn; `growslice` còn tính kích thước element và làm tròn allocation. Đây là thuật toán triển khai, không phải luật để dự đoán mọi lần `append` hay kết luận dữ liệu nhất thiết nằm trên heap.

Để chủ động phòng vệ, Go cung cấp cú pháp lát cắt ba chỉ số đầy đủ `s[low:high:max]` (Full Slice Expression). Cú pháp này giới hạn sức chứa của lát cắt mới ở mức `max - low`. Khi ta đặt `max = high` (như `s[:n:n]`), capacity của lát cắt mới bị giới hạn đúng bằng độ dài nhìn thấy; thao tác `append` thêm phần tử vào lát cắt này sẽ không thể dùng tiếp phần capacity nằm ngoài phạm vi quan sát của lát cắt cũ, loại bỏ nguy cơ vô tình ghi đè lên các phần tử phía sau của mảng nền chung:

```
old := []int{10, 20}
s := old[:2:2]
grown := append(s, 30)
grown[0] = 99

fmt.Println(old)   // [10 20]
fmt.Println(grown) // [99 20 30]
```



Hình 10 — Hình mô tả hai semantics mà code phải chịu được. Đừng suy luận allocation từ một capacity tình cờ; nếu code cần một kết quả riêng, hãy tạo bản copy có chủ ý.

Một aliasing bug có người dùng thật

Một function “chỉ chuẩn bị preview” đôi khi phá dữ liệu của caller.

```
func redactPreview(fields []string) {
    fields[0] = "***"
}

requestFields := []string{"api-key", "region"}
redactPreview(requestFields)
fmt.Println(requestFields) // [*** region]
```

Parameter fields nhận một slice value mới, nhưng descriptor ấy vẫn reference storage của requestFields. Việc đổi element là mutation shared storage. Nếu mục tiêu thật sự là tạo preview độc lập, copy phần tử sang một slice mới trước:

```
func redactedCopy(fields []string) []string {
    preview := make([]string, len(fields))
    copy(preview, fields)
    preview[0] = "***"
    return preview
}
```

copy sao chép tối đa số phần tử vừa với destination và trả về số phần tử đã copy. Ở đây make tạo backing array mới, nên preview độc lập ở tầng slice elements. Đổi lấy independence là allocation và chi phí copy; với buffer lớn hoặc đường xử lý nóng, đó là một quyết định cần có lý do. Với một preview cần an toàn khỏi mutation, nó diễn đạt đúng intent.

Thực hành - bug hunt có kiểm chứng. Mở `labs/part2-values` trong VS Code và chạy riêng `TestRedactedCopyDoesNotMutateInput` trước khi sửa source. Sau đó thay phần tạo `preview` bằng `preview := fields`. Đừng đoán bằng mắt: chạy lại test để thấy nó báo input đã bị mutate. Khôi phục code, xóa thân `redactedCopy`, rồi tự viết lại từ contract của test mà không nhìn implementation cũ. Đây là một bước nhỏ, nhưng nó buộc mental model "copy slice value vẫn có thể chia sẻ storage" đi qua một failure thật.

Đáp án — chỉ đọc sau khi đã tự làm. `preview := fields` chỉ copy slice descriptor, nên `preview[0]` vẫn sửa phần tử chung với `requestFields`. Bản đúng cần một backing array mới, chẳng hạn `make` rồi `copy`. Test phải giữ cả hai assertion: preview bị redacted và input không đổi; bỏ assertion thứ hai là bỏ mất contract ownership.

Một cửa sổ nhỏ giữ cả căn phòng

Độc lập khi sửa dữ liệu chưa phải câu hỏi cuối cùng về ownership. Giả sử service nhận một buffer lớn, lấy 16 byte đầu làm mã nhận diện rồi giữ mã ấy trong lịch sử. Buffer không còn cần cho nghiệp vụ. Anh dự đoán: giới hạn cả length lẫn capacity xuống 16 có cho phép GC thu hồi phần còn lại không?

```
buf := make([]byte, 4<<20)
preview := buf[:16:16]
fmt.Println(len(preview), cap(preview)) // 16 16
```

Hãy phân biệt ba câu hỏi. `len` cho biết bao nhiêu phần tử đang nằm trong cửa sổ; `cap` giới hạn cửa sổ có thể mở rộng tới đâu bằng reslice hoặc dùng lại storage khi append. Còn GC phải hỏi object nào vẫn được tham chiếu. `preview` vẫn dẫn tới underlying array của `buf`. Đối với allocation lớn trong thí nghiệm này, còn giữ cửa sổ là còn giữ object ấy, không phải chỉ 16 byte đầu. Biểu thức ba index thay quyền mở rộng, không chuyển quyền sở hữu sang một allocation độc lập.

Ngay cả khi đặt `buf = nil`, nếu chương trình vẫn sử dụng `preview`, đường tham chiếu tới array còn tồn tại. Ngược lại, biến còn nằm trong phạm vi từ vựng không bảo đảm compiler coi nó còn sống đến cuối scope. Khi đo, ta cần giữ kết quả thực sự được dùng; lab ở Chương 10 dùng `runtime.KeepAlive` để bảo vệ ý đồ của thí nghiệm. Không suy ra số byte heap chính xác chỉ từ cú pháp `make`: vị trí cấp phát và cách tối ưu là chuyện phải quan sát trên đúng toolchain.

Anh hãy mở `labs/part10-resource-retention/baseline/retention.go`, đọc `Prefix`, nhưng chưa đọc bản sửa. Viết test với hai yêu cầu: sửa prefix không đổi input, và prefix không dùng chung byte đầu với input khi độ dài khác zero. Bản ba index sẽ thất bại ở cả hai yêu cầu dù `cap(prefix) == len(prefix)`. Test ấy kiểm chứng sharing; profile sau nhiều chu kỳ mới cho thấy hệ quả về lượng bộ nhớ giữ lại.

Nếu contract là trả về dữ liệu độc lập, lời giải trực tiếp đã nằm trong thao tác copy mà ta vừa học:

```
func Prefix(buf []byte, n int) []byte {
    out := make([]byte, n)
    copy(out, buf[:n])
    return out
}
```

Tiền điều kiện ở đây là `0 <= n && n <= len(buf)`. Với prefix 16 byte, ta giữ 16 phần tử byte trong storage riêng thay vì kéo dài tuổi thọ của toàn bộ buffer đầu vào. Đây là copy các byte, không phải lời hứa giải phóng tức thì buffer cũ: nếu một alias khác còn giữ buffer, nó vẫn sống; nếu không, GC còn cần chạy. Chi phí cấp phát cũng đổi, nên chỉ copy ở boundary thật sự cần sở hữu độc lập, không rải copy để che mọi vấn đề.

`bytes.Clone(buf[:n])` cũng phù hợp khi API làm việc với byte slice. `slices.Clone` dùng được với slice tổng quát nhưng là shallow copy: clone một `[]*Record` tạo storage mới cho các pointer, không clone những `Record` mà chúng trỏ tới. Hai hàm có contract về bản sao; không dùng capacity quan sát được của chúng làm bằng chứng về kích thước allocation. Bản `make` và `copy` trong lab cố ý đơn giản để ta kiểm tra ownership mà không phải học thêm generic API ở đây.

Một view lớn sống lâu có thể hoàn toàn hợp lệ nếu caller vẫn cần toàn bộ dữ liệu. Trường hợp đáng sửa là tuổi thọ thực tế vượt nhu cầu của API: nghiệp vụ chỉ giữ một mã nhỏ nhưng implementation vô tình giữ cả buffer. Ta gọi đó là giữ bộ nhớ ngoài ý muốn, **memory retention**. GC không bị hỏng; contract về dữ liệu cần sống bao lâu đang bị implementation làm rộng hơn.

String: bytes trước, text sau

String cũng là value, nhưng nó có luật riêng: string là immutable sequence of bytes. len đếm byte; index trả về byte. Text thường là UTF-8, song Go không yêu cầu mọi string phải chứa UTF-8 hợp lệ.

```
text := "Việt"
fmt.Println(len(text)) // 6

for index, value := range text {
    fmt.Println(index, value)
}
```

Lab chạy đoạn này với kết quả:

```
6
0 86
1 105
2 7879
5 116
```

V và i mỗi chiếm một byte. ệ được mã hóa bằng ba byte UTF-8, vì vậy rune tiếp theo bắt đầu ở byte index 5. range trên string decode UTF-8: giá trị thứ hai là rune, thường dùng để biểu diễn Unicode code point. byte là alias của uint8; rune là alias của int32. Code point vẫn không đồng nghĩa với "một ký tự người dùng nhìn thấy": một grapheme có thể gồm nhiều code point. Ta chưa cần giải quyết segmentation ở đây; chỉ cần không nhầm len với số ký tự hiển thị.

String không cho phép gán trực tiếp `text[0] = 'v'`. Với thao tác theo byte trong ví dụ này, chuyển sang `[]byte`, sửa slice rồi tạo string kết quả là một cách rõ ràng. Đó không phải con đường duy nhất tạo text mới: thao tác theo điểm mã có thể dùng rune, còn ghép chuỗi có thể dùng builder. Chọn đơn vị dữ liệu trước khi chọn công cụ sửa nó.

Conversion giữa string và byte slice phải giữ semantics của giá trị: sửa byte slice tạo từ string không được làm đổi string gốc; string tạo từ slice giữ giá trị dù slice về sau bị sửa. Specification không bắt một lần cấp phát hay physical copy ở mọi biểu thức. Compiler có thể bỏ allocation/copy khi chứng minh behavior quan sát không đổi. Nếu cần biết một biểu thức có allocate không, đo và xem output compiler của đúng version thay vì suy ra từ syntax.

Pointer là câu hỏi khác

Sau slice, pointer dễ bị giải thích sai như "slice nhưng rõ hơn". Pointer là một value trực tiếp reference một variable khác; slice là một descriptor cho segment của underlying array. Chúng có thể cùng dẫn tới sharing, nhưng chúng trả lời hai câu hỏi khác nhau.

```
count := 10
p := &count
*p = 99
fmt.Println(count) // 99
```

Ở đây &count tạo pointer tới variable count, còn *p truy cập variable đó. Pointer sẽ có chương riêng khi ta cần thiết kế API quanh mutation trực tiếp và nil. Trước khi đến đó, hãy giữ model vừa xây: luôn hỏi value nào được copy, và value ấy có reference tới storage hay variable nào không.

Nếu câu trả lời là "không", assignment thường tạo independence như int và array. Nếu câu trả lời là "có", aliasing có thể là công cụ tốt hoặc một bug - tùy việc bạn có làm rõ ownership và mutation hay không.

Một quyết định ở ranh giới API

Khi một hàm nhận hoặc trả về slice, câu hỏi thiết kế không phải là "Go có copy không?" mà là: **ai được phép sửa storage này, và trong khoảng thời gian nào?** Một hàm lọc dữ liệu có thể trả về một view để tránh allocation; khi đó caller cần biết view ấy còn dùng chung buffer. Một hàm dựng response hoặc preview thường nên trả về dữ liệu độc lập, vì caller không có lý do để đoán một thay đổi sau đó sẽ đi ngược vào input.

Không có đáp án mặc định miễn phí. Copy tốn allocation và thời gian; chia sẻ tiết kiệm chúng nhưng làm contract về ownership quan trọng hơn. Trước khi tối ưu, hãy viết test cho hành vi mutation mà API hứa. Sau đó, khi đọc một bug khó hiểu, lần theo ba bước: value nào được gán, value đó reference storage nào, và ai còn có thể chạm storage ấy. Đó là cách biến "slice lạ quá" thành một cuộc điều tra có thể kiểm chứng.

1. Go Team. Go Specification, Slice types và Full slice expressions. go.dev/ref/spec#Slice_types; go.dev/ref/spec#Full_slice_expressions
2. Go Team. Package `bytes`, `Clone`, Go 1.27.1. pkg.go.dev/bytes@go1.27.1#Clone
3. Go Team. Package `slices`, `Clone`, Go 1.27.1. pkg.go.dev/slices@go1.27.1#Clone
4. Go Team. Go Garbage Collector Guide, Tracing Garbage Collection. go.dev/doc/gc-guide#Tracing_Garbage_Collection

Chương 3 — Mô hình dữ liệu và trách nhiệm thay đổi

`opsprobe` sẽ dần trở thành một CLI kiểm tra trạng thái dependency. Trước khi nó gọi HTTP hay đọc file config, nó đã có một bài toán nhỏ hơn: làm sao để biểu diễn trạng thái của một service mà người đọc không phải tự ghép bốn biến rời rạc trong đầu?

```
serviceName := "billing"
port := 8080
healthy := true
retries := 0
```

Bốn biến này đi cùng nhau. Chúng mô tả một service, nhưng compiler chưa biết điều đó. Càng nhiều hàm nhận chúng rời rạc, càng dễ đổi nhầm thứ tự, quên truyền một field, hoặc cập nhật một bản sao mà tưởng đã cập nhật trạng thái chung. Ta sẽ refactor dần mô hình này. Mỗi lần thay đổi, ta dừng ở cùng một câu hỏi: **function nhận value nào, và value ấy mở đường tới dữ liệu nào?**

Bug đầu tiên: bonus đi đâu mất?

Hãy tạm quên service. Một ví dụ nhỏ hơn cho thấy vấn đề rõ nhất:

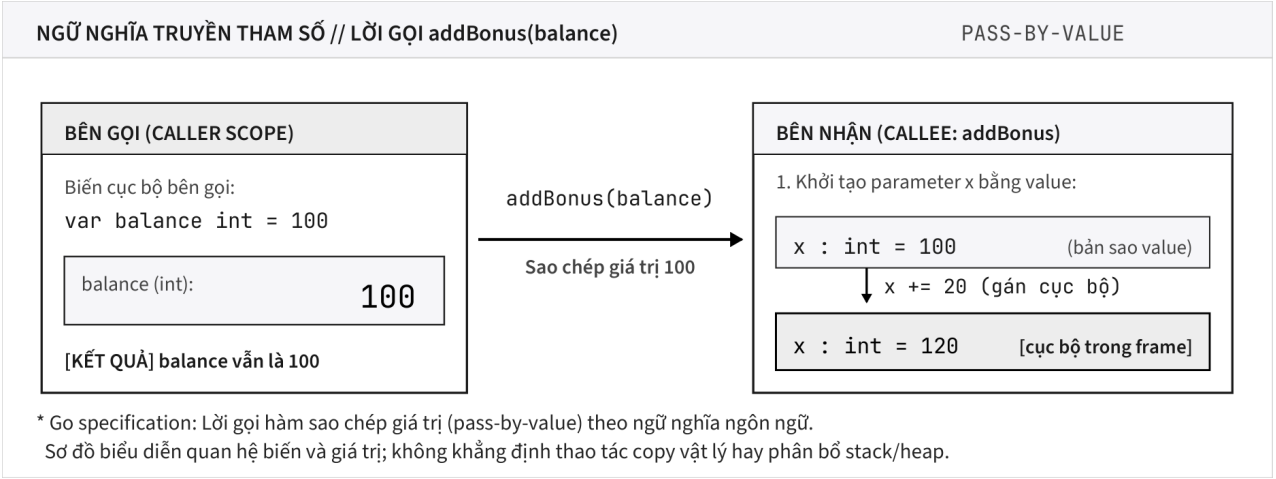
```
balance := 100

func addBonus(x int) {
    x += 20
}

addBonus(balance)
fmt.Println(balance) // 100
```

`balance` không đổi vì lời gọi hàm cũng là một phép truyền value. Khi gọi `addBonus(balance)`, function nhận một parameter `x` có value `100`. Câu `x`

`+= 20` thay value trong variable `x`; nó không quay lại gán cho variable `balance` của caller. Đây không phải một ngoại lệ của `int`, mà là nền cho mọi signature trong chương này.



Hình 11 — Lời gọi `addBonus(balance)` copy value 100 vào parameter `x`. Sơ đồ biểu diễn semantics của variable và value, không khẳng định vị trí vật lý của chúng trong RAM.

Nếu function cần tạo một value mới, cách trên rất tốt: giá trị đi vào, kết quả đi ra bằng `return`. Ví dụ, `withBonus(balance)` có thể trả `120`, và caller chọn có gán kết quả đó cho `balance` hay không. Nhưng một số thao tác có ý nghĩa là thay đổi chính variable caller đã đưa vào. Lúc ấy cần biểu đạt đường đi tới variable đó một cách rõ ràng.

Pointer: con đường tới một variable

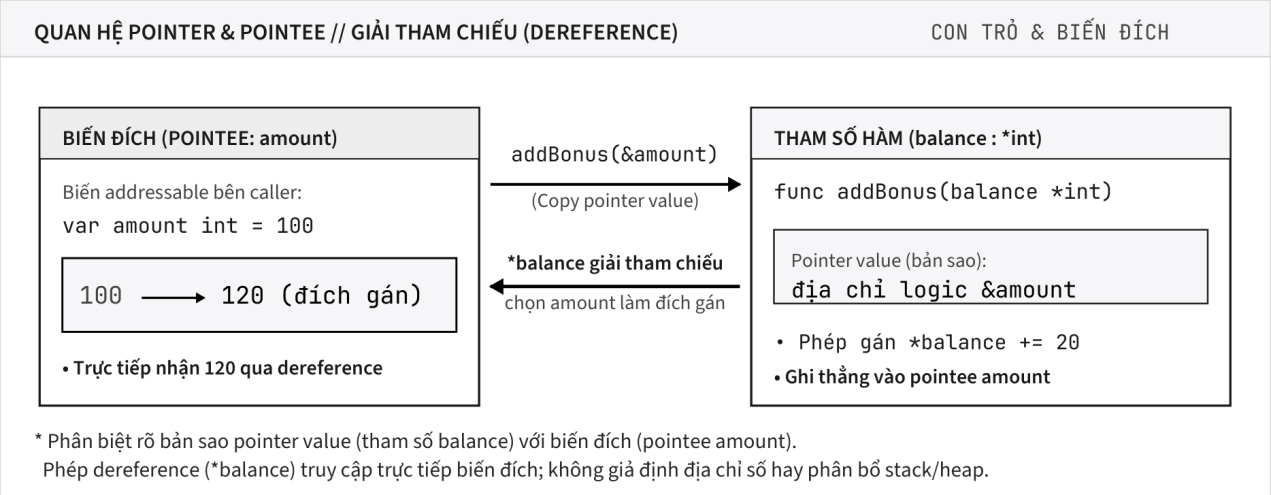
Khái niệm trong Go Semantics	Định nghĩa kỹ thuật chuẩn xác	Ví dụ tương ứng
Variable	Vị trí lưu trữ dữ liệu được cấp phát, có thể đọc hoặc gán giá trị mới.	Biến `balance`
Value	Dữ liệu thuộc một kiểu; chưa cần giả định representation vật lý.	Giá trị số nguyên `100`
Address	Kết quả của phép toán trích xuất địa chỉ ô nhớ bằng toán tử &.	Biểu thức `&balance`
Pointer value	Giá trị mang kiểu con trỏ (như `*int`), chứa địa chỉ của một biến hoặc `nil`.	Biến con trỏ `p`
Pointee	Biến đích tại ô nhớ mà con trỏ đang trỏ tới.	Ô nhớ chứa `balance`

Ta không cần giả định variable ấy nằm cố định trên stack hay heap. Đó là quyết định implementation có thể thay đổi. Điều cần dùng khi đọc code là quan hệ ngôn ngữ: pointer value có đang trỏ tới một variable hợp lệ hay không.

```
func addBonus(balance *int) {
    *balance += 20
}

amount := 100
addBonus(&amount)
fmt.Println(amount) // 120
```

Ở caller, `&amount` tạo pointer tới variable `amount`. Parameter `balance` vẫn là một value được copy vào function - lần này value đó là pointer. Khi `*balance += 20` chạy, dereference `*balance` chọn chính variable `amount` làm đích gán. Bản sao của pointer không tạo bản sao của pointee.



Hình 12 — `balance` trong function là pointer value được truyền theo value. `*balance` truy cập pointee; sơ đồ cố ý không dùng địa chỉ số hay stack/heap để tránh biến mental model thành chi tiết runtime.

Phép & không dùng được với mọi expression. Thông thường, operand của & phải addressable: variable, dereference pointer, index của slice, field của struct addressable, hoặc index của array addressable là những trường hợp quan trọng ở giai đoạn này. Go specification có một ngoại lệ hữu ích: được phép lấy địa chỉ của composite literal, như `&Service{Name: "billing"}`. Không cần gọi composite literal là “addressable” để dùng

đúng quy tắc; hãy nhớ hai ý riêng: `&x` thường cần một nơi có thể gán, còn literal có ngoại lệ được phép tạo value rồi lấy địa chỉ của nó. Map index không addressable - ta sẽ thấy lý do thực hành của giới hạn đó khi đưa map vào mô hình service.

Nil là thiếu pointee, không phải "pointer rỗng vô hại"

Zero value của pointer type là `nil`. `nil` không trỏ tới variable nào, nên dereference nó gây run-time panic.

```
var bonus *int

if bonus == nil {
    fmt.Println("chưa có bonus")
    return
}
*bonus += 20
```

Dereference `*bonus` cần một pointee để chọn làm đích gán; `nil` không có pointee, nên dereference nó panic. Kiểm tra `nil` là một phần của contract: function này có chấp nhận "không có giá trị" hay caller phải luôn truyền pointer hợp lệ? Đừng thêm `nil` check theo thói quen. Hãy quyết định và thể hiện điều đó qua API, test, hoặc lỗi trả về khi có lý do.

Gom state thành một value có tên

Quay lại bốn biến rời rạc. Một struct gom chúng thành một value mà domain có thể gọi tên:

```
type Service struct {
    Name    string
    Port    int
    Healthy bool
    Retries int
}

billing := Service{
    Name:    "billing",
    Port:    8080,
    Healthy: true,
}
```

`type Service` khai báo một named type; phần thân struct khai báo các field và type của chúng. Struct literal đặt value cho các field được nêu. `Retries` không xuất hiện trong literal nên nhận zero value của `int`, là `0`. Zero value không thay thế validation: một `Service{}` hợp lệ về type nhưng chưa chắc hợp lệ cho một service thật. Trong domain này, port `0` có thể nghĩa là "chưa cấu hình".

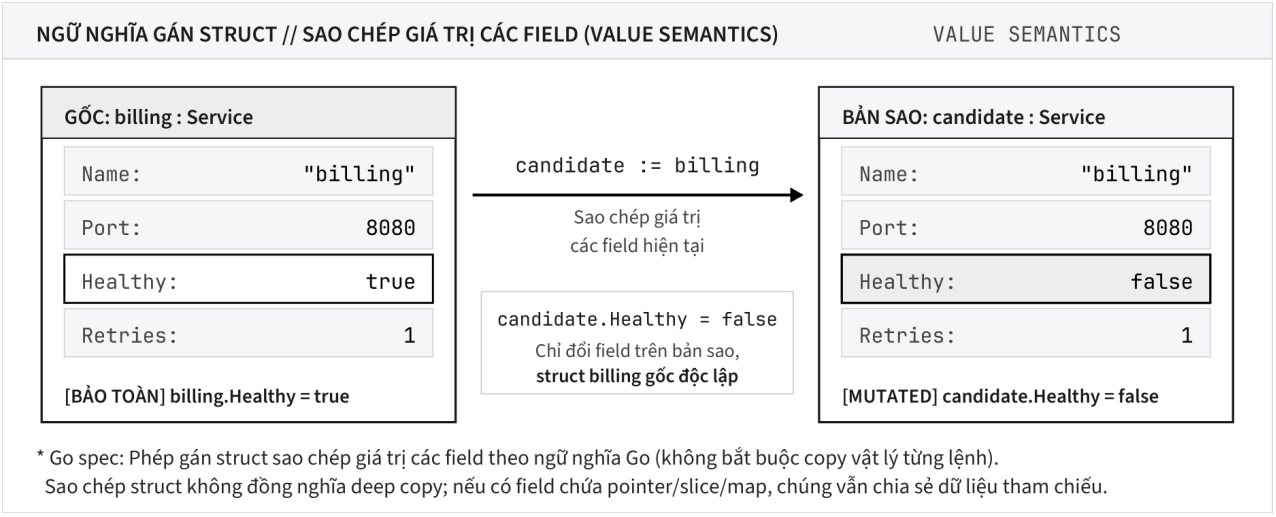
Struct không phải class thiếu method. Trước hết nó là một value có nhiều phần mang cùng một ý nghĩa. Field access chỉ đọc hoặc gán một phần của value:

```
billing.Retries++
fmt.Println(billing.Name, billing.Retries) // billing 1
```

Một struct cũng được copy khi gán hoặc truyền làm argument. Với các field trong `Service` hiện tại, copy tạo các field độc lập:

```
candidate := billing
candidate.Healthy = false

fmt.Println(billing.Healthy)   // true
fmt.Println(candidate.Healthy) // false
```



Hình 13 — Assignment copy struct value hiện tại theo từng field. Nếu một field về sau là slice, map hoặc pointer, struct vẫn được copy nhưng field đó có thể mang theo một reference; lúc ấy phải phân tích tiếp value của field thay vì gắn nhãn "struct luôn độc lập".

Nested struct hữu ích khi một phần của domain tự nó đã có ý nghĩa. Ví dụ, host và port thường đi cùng nhau hơn là hai field phẳng:

```
type Endpoint struct {
    Host string
    Port int
}

type Service struct {
    Name     string
    Endpoint Endpoint
    Healthy  bool
    Retries  int
}
```

Việc nesting không tự động làm thiết kế tốt hơn. Nó có ích khi tạo ra một khái niệm đọc được, như `Endpoint`; gom chỉ để tránh thấy nhiều field thường làm model khó tìm hơn.

Khi function sửa một bản copy

Ta muốn `opsprobe` đánh dấu service không khỏe sau một lần probe thất bại. Phiên bản đầu tiên trông rất có lý:

```
func markUnhealthy(service Service) {
    service.Healthy = false
    service.Retries++
}

markUnhealthy(billing)
fmt.Println(billing.Healthy, billing.Retries) // true 0
```

Không có bug bí ẩn ở đây. `billing` được copy vào parameter `service`; function thay đổi struct copy rồi return. Caller còn giữ value cũ. Nếu đây là một transform thuần, ta có thể trả `Service` mới. Nếu ý định là mutate state caller đang quản lý, signature cần nói điều đó:

```
func markUnhealthy(service *Service) {
    service.Healthy = false
    service.Retries++
}

markUnhealthy(&billing)
fmt.Println(billing.Healthy, billing.Retries) // false 1
```

`T` và `*T` không có nghĩa "immutable" và "mutable". Một local value `Service` vẫn có thể bị gán field; một pointer parameter có thể chỉ được đọc. Khác biệt là một `T` parameter mang struct value riêng, còn một `*T` parameter mang khả năng truy cập pointee của caller. Chọn pointer khi function phải thay đổi pointee, khi biểu diễn sự vắng mặt bằng `nil` là một phần hữu ích của domain, hoặc khi API có lý do rõ ràng khác. Chọn value khi function cần snapshot cục bộ, transform rồi return, hoặc sharing sẽ làm contract khó hiểu hơn.

Cùng một lời gọi, bảy value khác nhau

Đến đây có thể đọc nhiều signature bằng một quy tắc duy nhất: Go truyền argument theo value. "Theo value" không trả lời xong câu hỏi; ta còn phải nhìn value đó chứa dữ liệu nào hoặc reference nào.

Bảng 2 — Mỗi lời gọi đều copy argument value vào parameter; khả năng chia sẻ nằm trong value được copy.

Parameter	Value được copy khi gọi	Thay đổi từ function có hiện ra ở caller?
`x int`	số nguyên	Không, trừ khi return và caller gán kết quả.
`a [3]int`	toàn bộ array value	Không, các element của bản copy bị sửa riêng.
`s []int`	slice value, gồm reference tới array nền	Sửa element có thể hiện ra; reslice parameter không tự đổi len caller.
`m map[string]int`	map value, có reference tới map data	Ghi/xóa entry hiện ra qua map value cùng map data.
`p *int`	pointer value	`*p = ...` sửa pointee của caller nếu pointer hợp lệ.
`v Service`	các field của struct value	Không với các field value; xem từng field nếu chúng chứa reference.
`p *Service`	pointer value	Sửa field qua `p` sửa pointee của caller nếu pointer hợp lệ.

Bảng này không phải bảng để học thuộc type. Nó là cách lần một lời gọi: parameter mới nhận bản copy nào; bản copy ấy có dẫn đến dữ liệu chung hay variable chung không; expression nào là đích mutation? Đó cũng là lý do value receiver và pointer receiver sẽ được học sau cùng methods, thay vì biến dấu `*` thành một nghi thức.

> **Bài tập - sửa đúng contract:** Viết `recordFailure(service Service)` sao cho caller nhận được `Healthy == false` và `Retries` tăng một. Trước khi nhìn đáp án, chọn một trong hai contract: function return một `Service` mới, hoặc function nhận `*Service` và mutate pointee.

Đáp án. Nếu caller quản lý cùng một state và function được đặt tên như một command, pointer làm ý định trực tiếp:

```
func recordFailure(service *Service) {
    service.Healthy = false
    service.Retries++
}

recordFailure(&billing)
```

Một biến thể value vẫn đúng khi API diễn đạt transform:

```
func failedCopy(service Service) Service {
    service.Healthy = false
    service.Retries++
    return service
}

billing = failedCopy(billing)
```

Hai phiên bản khác nhau ở trách nhiệm của caller. Phiên bản pointer cho phép function tác động lên state đã có; phiên bản return khiến việc thay state nằm rõ ở câu gán của caller. Không có "bản nhanh hơn" mặc định quan trọng hơn contract ở đây.

Thực hành - một bug vẫn compile. Trong `labs/part3-data-models`, mở `main_test.go` trước. Ở `recordProbe`, tạm bỏ câu `registry[name] = service`, rồi chạy riêng `TestRecordProbeUpdatesExistingMapEntry`. Code vẫn compile vì local struct copy được sửa hợp lệ; test mới chỉ ra entry trong map không hề nhận value mới. Khôi phục bằng đường mutation mà anh đã lần, không bằng cách sửa expectation. Sau đó xóa thân `recordFailure` và tự dựng lại nó từ `TestRecordFailureMutatesPointee`.

Đáp án — chỉ đọc sau khi đã tự làm. Map lookup trả một `Service` value. Sửa local `service` chưa chạm map data; phép gán ngược vào `registry[name]` mới là mutation của registry. `recordFailure` cần nhận `*Service` và sửa pointee, vì test đòi caller quan sát cùng state thay đổi.

Nhiều service cần lookup, không cần thêm biến

Một `Service` model một service. Khi `opsprobe` giữ nhiều service theo tên, ta không muốn `billing`, `search`, `checkout` thành các variable độc lập. Map diễn đạt một quan hệ key/value:

```
services := map[string]Service{
    "billing": {
        Name:    "billing",
        Port:    8080,
        Healthy: true,
    },
}

billing, found := services["billing"]
fmt.Println(found, billing.Port) // true 8080
```

`string` là key type; `Service` là element type. `found` là kết quả thứ hai của map lookup, thường gọi là comma-ok. Nó phân biệt key không có mặt với element có zero value. Một lookup một-result vẫn hoàn toàn hợp lệ: `services["missing"].Retries` đọc `Retries` của zero value `Service{}`, nên cho `0`. Nhưng `0` khi ấy có thể là "chưa có service" hoặc "service có `Retries == 0`"; đó là lý do phải dùng comma-ok khi sự hiện diện của entry mang ý nghĩa.

```
attempts := map[string]int{"billing": 0}

count, found := attempts["billing"]
fmt.Println(count, found) // 0 true

count, found = attempts["search"]
fmt.Println(count, found) // 0 false
```

Map literal tạo map đã sẵn sàng ghi. `make(map[string]Service)` cũng tạo map sẵn sàng ghi. Ngược lại, zero value của map là `nil`: đọc từ nil map cho zero value, `range` qua nó có zero iteration, nhưng gán entry vào nil map sẽ panic. Một map nhận từ caller vì thế cần contract rõ như pointer: caller

có luôn khởi tạo nó không, hay function chịu trách nhiệm tạo nó trước khi ghi?

```
var services map[string]Service
fmt.Println(services["billing"].Name) // ""

// services["billing"] = billing
// panic: assignment to entry in nil map
services = make(map[string]Service)
services["billing"] = billing
```

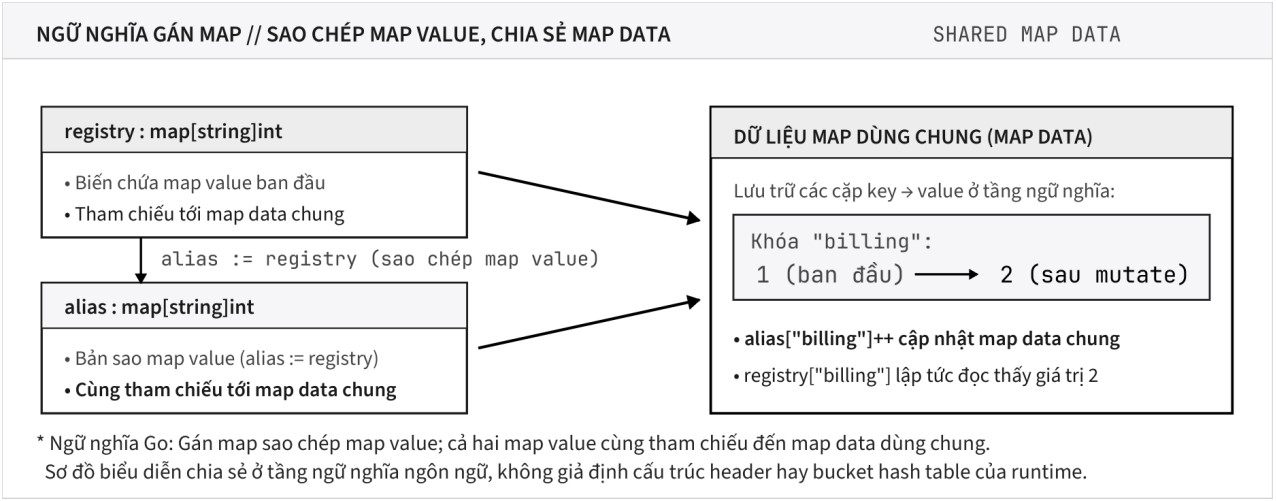
`delete(services, "billing")` xóa một entry; xóa key không có mặt là an toàn. Nhưng `range` map không có thứ tự được chỉ định. Nếu CLI cần output ổn định, hãy lấy key, sort chúng, rồi in theo thứ tự đã chọn - đừng coi thứ tự đang tình cờ xuất hiện là contract của map.

Map copy value, chia sẻ map data

Map không phải pointer và ta không cần mô tả hash table internals để dùng nó đúng. Assignment và call copy map value. Nếu hai map value cùng reference map data, ghi qua một value hiện ra qua value kia:

```
registry := map[string]int{"billing": 1}
alias := registry
alias["billing"]++

fmt.Println(registry["billing"]) // 2
```



Hình 14 — Assignment copy map value. Khi value ấy reference cùng map data, mutation entry qua `alias` hiện ra khi đọc qua `registry`; hình là conceptual diagram, không mô tả hash table implementation.

Một chi tiết làm map khác slice và array khi update struct value: map index không addressable. Vì vậy đoạn này không hợp lệ:

```
// services["billing"].Healthy = false // không hợp lệ
```

`services["billing"]` cho một `Service` value để đọc; nó không cho một variable mà ta có thể lấy địa chỉ field hay gán field trực tiếp. Cách value-oriented là lấy value ra, đổi, rồi gán lại:

```
service := services["billing"]
service.Healthy = false
services["billing"] = service
```

Nếu domain muốn registry quản lý các `*Service`, thì mutation qua pointee có thể trực tiếp hơn, nhưng đổi lại registry có thể chứa nil pointer và ownership cần được làm rõ:

```
services := map[string]*Service{
    "billing": {
        Name:    "billing",
        Port:    8080,
        Healthy: true,
    },
}

services["billing"].Healthy = false
```

Đây không phải lời mời chọn `map[string]*Service` mặc định. Với value nhỏ như `Service` hiện tại, `map[string]Service` và "lấy - sửa - gán lại" thường bộc lộ điểm mutation rõ hơn. Pointer trong map chỉ đáng dùng khi nó giải quyết một contract thực: object identity dùng chung, mutation phải đi qua nhiều nơi, hoặc value lớn và đã có lý do đo đạc/thiết kế.

> **Bài tập - đọc zero value:** Một probe trả về `latencyByService map[string]int`. Viết code phân biệt `billing` có latency `0` với `billing` chưa có kết quả. Sau đó xóa kết quả của `billing` mà không cần kiểm tra key có tồn tại.

Đáp án. Comma-ok mang thông tin hiện diện; `delete` tự xử lý key thiếu:

```
latency, found := latencyByService["billing"]
if !found {
    fmt.Println("chưa có kết quả")
} else {
    fmt.Println(latency)
}

delete(latencyByService, "billing")
```

Ai có quyền giữ một entry mãi mãi?

Registry vừa giúp tìm service theo tên. Nhưng nếu tên không còn là tập hữu hạn mà mỗi request đưa vào một key mới, quyền sửa map còn bao gồm một trách nhiệm khác: quyết định khi nào bỏ dữ liệu. Hãy xét một cache sống cùng tiến trình. Mỗi lần miss, nó thêm `cache[key] = payload`; không có `delete`, không giới hạn key và không có ngày hết hạn. Vì owner vẫn giữ map, các value còn trong map vẫn được tham chiếu. GC không có quyền đoán rằng kết quả của request hôm qua đã hết giá trị nghiệp vụ rồi xóa hộ.

Chưa thể kết luận "global map là leak". Một bảng cấu hình hữu hạn sống suốt process là thiết kế bình thường. Một kho lịch sử có retention policy, dự báo tăng trưởng và ngân sách tương ứng cũng có thể tăng hợp lệ. Cache hứa tiết kiệm tài nguyên nhưng giữ mọi key từng thấy lại không có điểm dừng. Lỗi ở contract giữ dữ liệu, không ở vị trí khai báo map. Memory leak theo nghĩa vận hành là tài nguyên tiếp tục bị giữ sau khi nhu cầu đã hết; trong Go nó thường vẫn reachable, chứ không nhất thiết là allocation mất mọi pointer như cách người ta mô tả leak trong ngôn ngữ quản lý bộ nhớ thủ công.

Trước khi đọc `fixed` của lab `part10-resource-retention`, anh hãy viết contract cho một cache đơn chủ sở hữu: tối đa hai entry, mỗi value tối đa 4 MiB, key tối đa 128 byte; khi thêm key thứ ba thì bỏ key được thêm sớm nhất. Cập nhật key cũ không làm mới tuổi của nó. Input vượt giới hạn phải bị từ chối trước khi thay cache. Caller được sửa byte slice của mình sau `Put` mà không đổi bản cache đã nhận. Đây là FIFO, không phải LRU, và không có TTL ngầm.

Chính sách ấy dẫn tới những test không cần đo RAM. Sau chuỗi `a, b, a, c`, cache phải còn `b` và `c`; cập nhật `a` không tạo entry thứ ba và không cứu nó khỏi bị loại. Gửi value quá lớn phải trả error, không được đẩy một entry hợp lệ ra ngoài. Chạy nhiều vòng key mới, `Len()` không vượt hai. Sửa input sau `Put` không được đổi value đã lưu. Các test nằm trong `fixed/retention_test.go`; hãy làm bản `exercise` chạy được trước khi đối chiếu implementation.

Giới hạn entry riêng lẻ chưa phải giới hạn byte: hai entry mỗi cái hàng gigabyte vẫn quá lớn. Ngược lại, kiểm `len(value)` rồi lưu thẳng một slice nhỏ cũng chưa đủ, vì nó có thể giữ backing array lớn như Chương 2. Bản sửa copy các byte thực sự được nhận; clone key string để một substring nhỏ không vô tình giữ storage của string lớn. Với chính sách lab, tổng độ dài payload được cache giữ không vượt `maxEntries * maxValueBytes`; key và metadata có chi phí riêng. Đây không phải trần RSS hay lời hứa rằng allocator trả bộ nhớ ngay sau eviction.

Map mất entry thì đường tham chiếu từ map tới value cũ bị bỏ; alias ở nơi khác vẫn có thể giữ value ấy. Các slot trong hàng đợi FIFO cũng phải bỏ reference sau khi dịch, thay vì chỉ giảm length và để một string cũ nằm ở vùng capacity chưa dùng. Lab có đúng một owner nên không cần mutex; nếu cho nhiều goroutine gọi `Put`, ta phải thiết kế đồng bộ hóa trước, không suy ra map an toàn từ việc đã có capacity limit.

Trong ca đo ở Chương 10, cùng chuỗi key mới làm bản không giới hạn tăng từ hai lên tám entry, trong khi bản sửa dừng ở hai. Bằng chứng kết thúc không phải "RAM trông đẹp hơn", mà là contract hữu hạn được test giữ đúng, cộng profile xác nhận payload bị loại không còn bị cache giữ. Nếu product thật sự cần lưu tất cả key, đổi requirement và nơi lưu dữ liệu; đừng gọi việc mất lịch sử vì eviction là một bản sửa thành công.

Một update có trách nhiệm trong registry

`opsprobe` chưa cần network để hưởng lợi từ model này. Một update value-oriented cho registry không cần pointer: lấy struct value ra khỏi map, sửa bản local, rồi gán nó trở lại entry. Map parameter được truyền theo value, nhưng map value ấy vẫn mở đường tới map data của caller; chính phép gán cuối là mutation có chủ ý lên registry chung.

```
func recordProbe(
    registry map[string]Service,
    name string,
    healthy bool,
) bool {
    service, found := registry[name]
    if !found {
        return false
    }

    service.Healthy = healthy
    if !healthy {
        service.Retries++
    }
    registry[name] = service
    return true
}
```

Signature này nói rõ ba vai: `registry` là map value để lookup và gán entry; `name` là snapshot string; `healthy` là snapshot bool. `service` là struct copy lấy từ entry. Nếu key không có, function không bịa ra service mới mà trả `false`; nếu có, nó ghi value đã sửa về map. Khi công cụ bắt đầu đọc config và probe endpoint ở phần sau, cách đọc signature này sẽ giúp ta biết dữ liệu nào được chia sẻ, dữ liệu nào được snapshot, và mutation nào có thể lần lại.

Sau chương này, một signature không còn chỉ là tên type. Nó cho biết ta đang đưa vào function một snapshot, một cửa sổ tới storage dùng chung, một map value dẫn tới map data, hay một pointer mở đường tới variable caller. Khi cần debug mutation, hãy viết ra trước/sau lời gọi: variable nào tồn tại ở caller, parameter nào xuất hiện ở function, và expression nào thực sự là đích gán.

Kiểu là ranh giới ý nghĩa, không chỉ cách chứa byte

Một port và một số lần thử đều có thể nằm trong `uint16`, nhưng đổi chỗ chúng là bug nghiệp vụ. Khai báo kiểu có tên đặt một ranh giới compiler có thể kiểm tra:

```
type Port uint16
type Attempts uint16
type PortAlias = uint16

var port Port = 80
// var attempts Attempts = port // không gán trực tiếp
var raw uint16 = uint16(port)
var alias PortAlias = raw
```

`Port` và `Attempts` là hai defined type khác nhau; cùng underlying type `uint16` không làm chúng có cùng identity. `PortAlias` lại là tên khác của chính `uint16`, không tạo một kiểu mới. Alias hữu ích khi chuyển API sang package khác mà vẫn giữ identity cho caller, không hữu ích nếu mục tiêu là ngăn nhầm hai đại lượng. Đừng đồng nhất assignability với convertibility: conversion tường minh có thể hợp lệ khi phép gán trực tiếp không hợp lệ; interface satisfaction còn dựa trên method set chứ không chỉ representation.

Conversion không phải validation. Với variable `wide := uint32(65537)`, `Port(wide)` cho giá trị 1 do quy tắc chuyển số nguyên sang kiểu hẹp hơn. Ngược lại, `Port(65537)` với hằng số bị từ chối vì hằng số không biểu diễn được trong kiểu đích. Constructor nhận dữ liệu ngoài phải kiểm tra range trước conversion nếu contract là port `1..65535`. Kiểu có tên giúp phát hiện nhầm vai trò; nó không tự cấm zero value hoặc giá trị ngoài miền nghiệp vụ.

Để tự kiểm chứng, mở `TestConstantsAndIdentity` và `testdata/incompatible` trong `labs/edition-contracts`. Test compiler yêu cầu phép gán giữa hai defined type bị từ chối; đừng sửa fixture chỉ vì nó không build. Sau đó tự thiết kế `NewPort(n uint32) (Port, error)` và viết test cho 0, 1, 65535, 65536 trước implementation. Đây là một bài về contract tại boundary, không phải bài thuộc lòng syntax `type`.

Khi hành vi thuộc về service

`recordProbe(registry, name, healthy)` đã làm đúng việc, nhưng khi `opsprobe` có thêm nhiều thao tác quanh một service, chữ ký bắt đầu lặp lại cùng một chủ thể. Một hàm cần tạo nhãn cho service, một hàm khác ghi kết quả probe, hàm nữa kiểm tra cấu hình. Lúc này method giúp đặt hành vi cạnh type mà nó đọc hoặc thay đổi; nó không thay đổi quy tắc value semantics đã xây.

```
func (service Service) Summary() string {
    return service.Name
}

func (service *Service) Record(healthy bool) {
    service.Healthy = healthy
    if !healthy {
        service.Retries++
    }
}
```

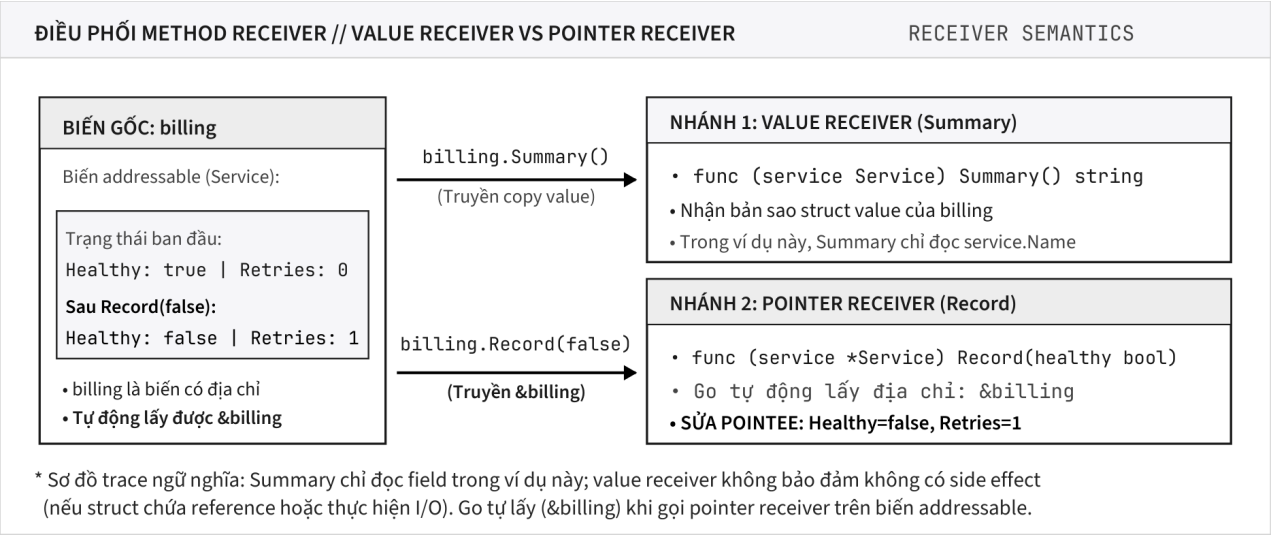
Một method là function có receiver: phần parameter đặc biệt đứng trước tên method. `Summary` nhận `Service` value, nên receiver là một struct value được dùng để tạo kết quả. `Record` nhận `*Service`, vì hành vi này có ý định sửa pointee. Tên receiver không phải từ khóa và không phải một biến toàn cục bí mật; nó chỉ là parameter được viết ở vị trí riêng để gắn method với `Service`.

```
billing := Service{Name: "billing", Healthy: true}

fmt.Println(billing.Summary()) // billing
billing.Record(false)
fmt.Println(billing.Healthy, billing.Retries) // false 1
```

`billing.Record(false)` hợp lệ vì `billing` là variable addressable. Trong trường hợp này, Go cho phép cách viết ngắn của `(&billing).Record(false)`. Sự tiện lợi ấy không biến value receiver thành pointer receiver: method set của `Service` chỉ có methods khai báo

với receiver `Service`; method set của `*Service` có cả methods receiver `Service` lẫn `*Service`.



Hình 15 — Method vẫn dựa trên receiver value. `Summary` dùng Service value; `Record` cần `*Service` để chọn pointee làm đích mutation. Mũi tên là trace semantics, không mô tả bộ nhớ vật lý.

Map index không addressable theo spec, nên cách viết ngắn lấy địa chỉ để gọi pointer-receiver method không áp dụng cho nó. Đây là giới hạn ở tầng source, không phải kết luận rằng một địa chỉ vật lý "không ổn định" trong mọi implementation:

```
// services["billing"].Record(false) // không hợp lệ

service := services["billing"]
service.Record(false)
services["billing"] = service
```

Đoạn sửa dùng đúng mô hình cũ: map lookup cho struct value, local variable `service` addressable, pointer-receiver method mutate local pointee, rồi caller gán struct value đã đổi về map. Nếu code thật cần nhiều nơi cùng giữ identity của một service, `map[string]*Service` có thể hợp lý; nhưng không dùng pointer chỉ để né phép gán lại mà chưa hiểu ownership.

Composition trước, embedding sau

Một service có endpoint và state; hai phần đó là các khái niệm riêng. Composition đơn giản nhất là đặt chúng vào những field có tên, để người đọc thấy quan hệ "có một":

```
type Endpoint struct {
    Host string
    Port int
}

type ProbeState struct {
    Healthy bool
    Retries int
}

type Service struct {
    Name     string
    Endpoint Endpoint
    State    ProbeState
}
```

`Service` có `Endpoint` và `ProbeState`; không có quan hệ kế thừa nào được ngụ ý. Đây là cách composition thường rõ nhất khi domain có các phần độc lập. Embedding thay đường viết selector, không biến object ngoài thành subtype của object trong:

```
type Named struct { Name string }
func (n Named) Label() string { return n.Name }
type Tagged struct { Named }

tag := Tagged{Named: Named{Name: "api"}}
fmt.Println(tag.Label()) // selector được promote
// var n Named = tag // Tagged không phải Named
```

`Label` vẫn nhận receiver `Named`; lời gọi ngắn đi qua field được embed. Hai field embed có cùng selector ở cùng độ sâu có thể làm selector mơ hồ và bị compiler từ chối. Method được promote cũng có thể làm type ngoài thỏa một interface ngoài dự kiến. Vì vậy embedding là quyết định về public API,

không phải mẹo tiết kiệm tên field. Khi caller cần nhìn rõ ownership, field có tên vẫn là lựa chọn dễ review hơn. Những quy tắc này thuộc Go specification về struct types, selectors và method sets, không phụ thuộc layout vật lý của struct.

Interface xuất hiện ở nơi cần một hành vi

Một interface không phải bản thiết kế cha để mọi struct đi theo. Nó là một contract nhỏ: code tiêu dùng nói mình cần method nào, còn type nào có method set phù hợp thì dùng được. Với `opsprobe`, phần in kết quả chưa cần toàn bộ `Service`; nó chỉ cần một dòng summary.

```
type SummarySource interface {
    Summary() string
}

func renderSummary(source SummarySource) string {
    return "target: " + source.Summary()
}
```

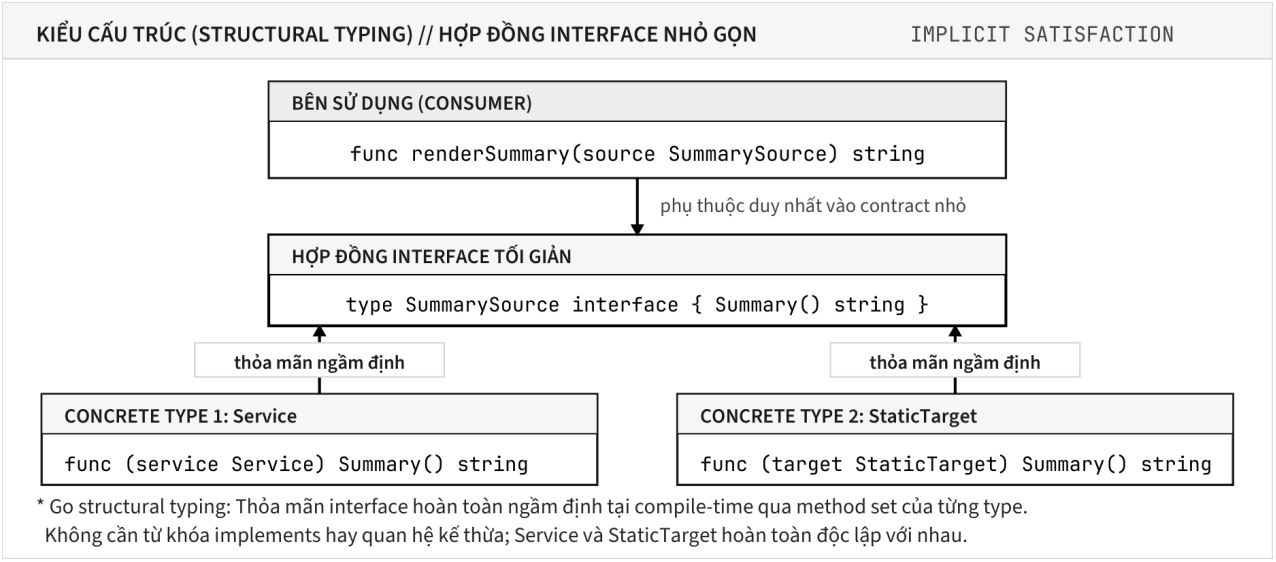
`renderSummary` không hỏi caller là `Service`, `StaticTarget`, hay một type tương lai. Nó chỉ gọi `Summary`. Không có từ khóa `implements`: assignment hoặc argument hợp lệ khi method set của type thỏa interface. Vì `Summary` có value receiver, cả `Service` và `*Service` đều có method đó trong method set.

```
type StaticTarget string

func (target StaticTarget) Summary() string {
    return string(target)
}

fmt.Println(renderSummary(Service{Name: "billing"}))
fmt.Println(renderSummary(
    StaticTarget("search (maintenance)"),
))
```

Kết quả là hai dòng có cùng format dù hai type không có ancestor chung. Interface ở đây được rút ra sau khi có consumer và hai provider có ý nghĩa. Nếu chỉ có một `Service` duy nhất và không có ranh giới cần thay thế, nhận trực tiếp `Service` thường dễ đọc hơn là thêm interface vì dự đoán tương lai.



Hình 16 — `renderSummary` chỉ cần `Summary() string`. Sơ đồ cho thấy dependency đi từ consumer tới contract nhỏ; Service và StaticTarget không cần khai báo một quan hệ kế thừa hay đăng ký implements.

Pointer receiver thay đổi interface satisfaction theo đúng rule method set. Nếu contract yêu cầu `Record(bool)`, `Service` value không thỏa vì method đó chỉ thuộc method set của `*Service`; `*Service` mới thỏa.

```
type ProbeRecorder interface {
    Record(bool)
}

func markFailed(recorder ProbeRecorder) {
    recorder.Record(false)
}

billing := Service{Name: "billing", Healthy: true}
markFailed(&billing) // *Service thỏa ProbeRecorder

// markFailed(billing) // sai: Service thiếu Record
```

Typed nil: có kiểu động nhưng giá trị pointer là nil

Để dự đoán một interface có bằng `nil` hay không, hãy tách kiểu động khỏi giá trị động. Kiểu động là kiểu của giá trị được gán vào interface; giá trị động là giá trị cụ thể ấy. Interface nil chưa có cả hai. Đây là mô hình ngữ nghĩa đủ để giải thích ví dụ dưới đây, không cần biết địa chỉ hay bảng con trỏ hàm.

Nếu muốn đối chiếu tầng triển khai, source Go 1.27.1 ở `runtime/runtime2.go` định nghĩa `iface` với `tab *itab` và `data unsafe.Pointer`, còn `eface` với `_type *_type` và `data`. Trên amd64, đó là hai trường con trỏ 64 bit. Cách concrete value được đưa vào trường data còn phụ thuộc kiểu; không coi nó như một object luôn được cấp phát riêng trên heap. Biểu diễn này không phải contract của specification và không cần dùng để quyết định nhánh `src == nil`.

Hãy dự đoán phép so sánh trước khi nhìn representation:

```
var s *Service = nil
var src SummarySource = s

if src == nil {
    fmt.Println("Interface la nil")
} else {
    fmt.Println("Interface KHONG nil") // Ket qua thuc te
}
```

Một giá trị interface chỉ bằng `nil` khi chưa có kiểu động và giá trị động. Khi gán con trỏ `s` kiểu `*Service` có giá trị `nil` vào `src`, interface vẫn có kiểu động `*Service`, nên `src == nil` trả về `false`. Trong ví dụ này, `Summary` có value receiver `Service`: gọi `src.Summary()` cần giải tham chiếu con trỏ nil để tạo receiver value, nên panic xảy ra **trước khi vào thân method**. Với một method có pointer receiver, receiver nil có thể đi vào thân method; method ấy chỉ an toàn nếu contract và implementation chủ động xử lý nil. Đừng suy từ "interface không nil" thành "lời gọi method chắc chắn an toàn".

So sánh được chưa chắc so sánh an toàn

Map key cần kiểu comparable; slice, map và function không có phép so sánh bằng giữa hai giá trị cùng loại, dù chúng có thể so với `nil`. Array comparable khi element comparable; struct comparable khi các field comparable. Điều này giải thích vì sao thêm field slice vào một struct có thể làm một API dùng struct ấy làm map key không còn compile.

Interface cần thêm một bước suy luận. Kiểu interface cho phép viết `==`, nhưng giá trị động bên trong có thể không comparable:

```
var a any = []int{1}
var b any = []int{1}
// a == b sẽ panic: kiểu động là []int
```

Đây không phải bug của garbage collector. Compiler chỉ biết static type `any`; khi hai dynamic type giống nhau, phép so sánh phải áp dụng cho dynamic value và slice không có contract ấy. `TestDynamicComparabilityPanics` kiểm tra đúng phản ví dụ này. Muốn so nội dung slice, hãy định nghĩa nghĩa của "bằng nhau" rồi so phần tử; không ép nó vào interface để né lỗi compiler. Chương 19 sẽ quay lại giới hạn này khi thiết kế generic API dùng `comparable`.

Một API nhỏ, Đủ để Đọc bằng Semantics

Ở cuối mạch thiết kế này, công cụ `opsprobe` đã có thể diễn đạt ba ý định kiến trúc khác nhau một cách mạch lạc mà không cần dựa vào những danh xưng mơ hồ:

`Summary` nhận bản copy receiver `Service`; với các field value đang có, nó chỉ đọc dữ liệu để tạo chuỗi. Nếu type được thêm field pointer, map hoặc slice, copy receiver vẫn có thể chia sẻ dữ liệu nền. Value receiver không bảo đảm mọi method tương lai không có side effect.

Hàm `func (service *Service) Record(bool)` tiếp nhận receiver dưới dạng giá trị con trỏ, cho phép cập nhật trực tiếp biến trạng thái của bên gọi mà không cần sao chép toàn bộ cấu trúc dữ liệu.

Hàm `func renderSummary(SummarySource) string` nhận interface với hành vi mà consumer cần. Nó gọi `Summary` thay vì đọc field của một struct cụ thể; đây là boundary có thể thay provider mà không đổi code hiển thị.

Mô hình cấu thành (composition) giúp dữ liệu có đường đi rõ ràng; method đặt hành vi cạnh kiểu dữ liệu; interface xác lập ranh giới lỏng lẻo khi xuất hiện nhu cầu hoán đổi nhà cung cấp. Từ nền tảng này, chương tiếp theo sẽ trang bị cho các API này một cơ chế báo cáo thất bại tường minh: lỗi kiểm tra probe không được phép chỉ âm thầm đổi trường `Healthy` rồi biến mất, mà phải vượt qua ranh giới hàm với đầy đủ ngữ cảnh để bên gọi chủ động ra quyết định xử lý.

1. Go Team. Go Specification, Map types và Deletion of map elements. go.dev/ref/spec#Map_types; go.dev/ref/spec#Deletion_of_map_elements
2. Go Team. Package `strings`, `Clone`, Go 1.27.1. pkg.go.dev/strings@go1.27.1#Clone
3. Go Team. Go Garbage Collector Guide, Tracing Garbage Collection. go.dev/doc/gc-guide#Tracing_Garbage_Collection

Chương 4 — Biên lỗi: để caller quyết định

`opsprobe` vừa có thêm `Service`, state và một hàm ghi kết quả probe. Phiên bản đầu của hàm trả `bool`: `true` nếu tên service tồn tại, `false` nếu không. Nó là một API hợp lệ khi caller chỉ cần biết có ghi được entry hay không.

Nhưng hãy đặt nó vào một tình huống vận hành. Màn hình báo `billing` không healthy. Có ít nhất hai câu hỏi khác nhau: service có được cấu hình không, hay service đã cấu hình nhưng probe tới endpoint thất bại? Một `false` không mang câu trả lời nào đi qua ranh giới function. Caller buộc phải đoán, hoặc đọc state đã bị đổi sau sự kiện.

Chương này không bắt đầu bằng một danh sách hàm trong package `errors`. Nó bắt đầu bằng quyền quyết định: nơi gọi cần nhận được failure nào để chọn thông báo, retry, hay dừng công việc; nơi phát hiện failure cần thêm context nào mà không phá mất identity của nguyên nhân.

Một failure chưa phải là một quyết định

Ta tách hành vi probe thành một function value. Nó chưa mở network; điều đó sẽ đến ở chương I/O. Hiện tại nó là một boundary có thể thành công hoặc trả về nguyên nhân thất bại:

```
type Endpoint struct {
    Host string
    Port int
}

type ProbeFunc func(Endpoint) error
```

`ProbeFunc` nhận endpoint value và trả về `error`. Nếu không có lỗi, giá trị trả về là `nil`. Nếu có lỗi, caller nhận một error value. `error` là interface của Go; điều đáng chú ý ở đây không phải interface đó có một method, mà là failure đi ra trên đường trả về với một shape thống nhất. Không dùng `panic`

để báo endpoint unreachable, và cũng không in log bên trong `ProbeFunc` rồi giả vờ thành công.

API ghi kết quả của `opsprobe` đổi từ câu hỏi "có entry không?" sang "tôi đã probe và cập nhật state được chưa?"

```
func applyProbe(
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error
```

Signature nói một điều quan trọng trước khi ta đọc implementation: caller không nhận state mới hay một boolean mơ hồ. Nó nhận `nil` khi operation hoàn tất, hoặc một error value để quyết định nhánh kế tiếp. `registry` vẫn là map value dẫn tới shared map data; vì map đang lưu struct value, hàm vẫn lấy entry ra, sửa local `Service`, rồi gán lại entry như ở Chương 3.

Phân loại để quyết định, thêm context để điều tra

Không phải mọi string lỗi đều là contract. Một CLI có thể muốn đối xử riêng với "tên service không có trong config", nhưng không nên parse câu tiếng Anh từ `err.Error()` để làm điều đó. Ta tạo một sentinel cho điều kiện mà caller có hành động riêng:

```
var ErrUnknownService = errors.New("service is not configured")
```

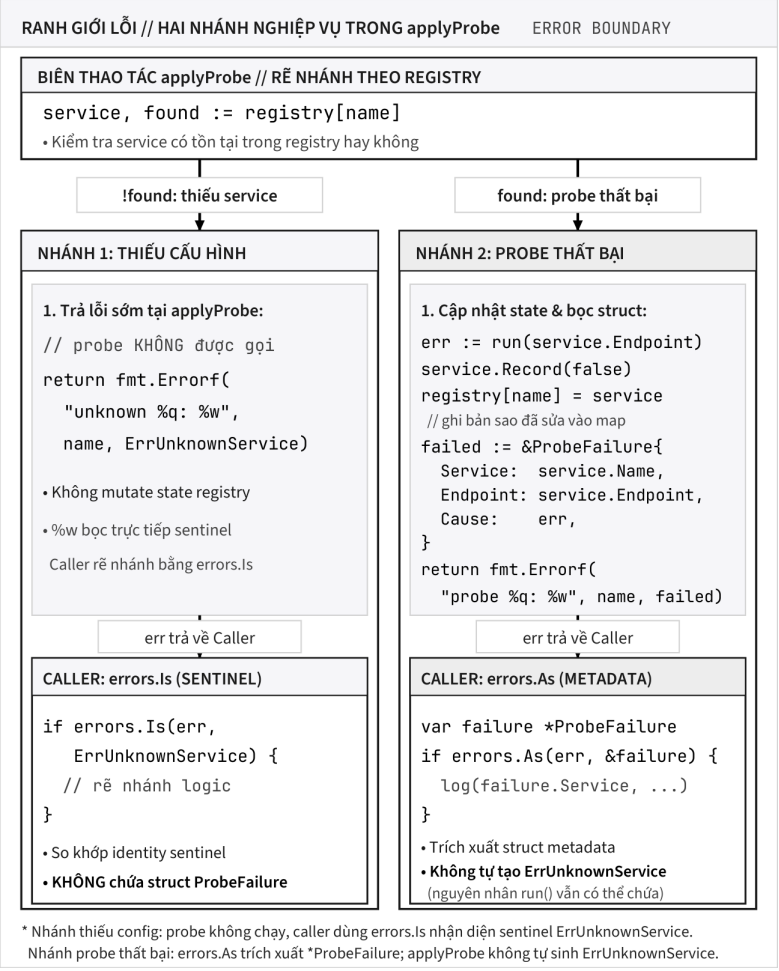
Khi tên không tồn tại, `applyProbe` không trả sentinel trần. Nó thêm thông tin operation và vẫn giữ nguyên identity bằng `%w`:

```
return fmt.Errorf("unknown %q: %w", name, ErrUnknownService)
```

`%w` không chỉ nối string. Nó tạo một error có thể unwrap về nguyên nhân. Vì vậy caller có thể kiểm tra ý nghĩa bằng `errors.Is` thay vì so câu chữ:

```
if errors.Is(err, ErrUnknownService) {
    fmt.Println("thêm service vào config trước khi probe")
    return
}
```

Trong câu lệnh trên, caller không cần biết hiện có bao nhiêu lớp context bên ngoài sentinel. `errors.Is` đi theo chuỗi wrapping. Nếu đổi `%w` thành `%v`, thông báo có thể nhìn giống nhau nhưng identity bị cắt; một điều kiện có thể phân loại được đã vô tình thành text chỉ dành cho người đọc.



Hình 17 — Biên lỗi không nuốt nguyên nhân. Mỗi lớp chỉ thêm context mà nó thực sự biết; caller chọn `errors.Is` khi cần một quyết định theo loại lỗi và `errors.As` khi cần dữ liệu từ một error cụ thể.

Một error có dữ liệu mà CLI thực sự cần

Sentinel trả lời "đây có phải loại lỗi đó không?". Khi probe tới một endpoint đã có cấu hình nhưng thất bại, CLI cần thêm service nào và endpoint nào liên quan. Ta dùng custom error cho context có cấu trúc:

```
type ProbeFailure struct {
    Service  string
    Endpoint Endpoint
    Cause    error
}

func (failure *ProbeFailure) Error() string {
    return fmt.Sprintf(
        "probe %s at %s:%d: %v",
        failure.Service,
        failure.Endpoint.Host,
        failure.Endpoint.Port,
        failure.Cause,
    )
}

func (failure *ProbeFailure) Unwrap() error {
    return failure.Cause
}
```

`Error` tạo message cho người đọc. `Unwrap` nói rõ `Cause` vẫn là nguyên nhân nằm dưới lớp context này. Error value cần tìm trong chain là `*ProbeFailure`, vì `ProbeFailure` dùng pointer receiver. Còn argument `target` thực tế truyền cho `errors.As` là `&failure`: pointer tới variable nhận kết quả. Hàm sẽ gán error tìm được vào variable đó:

```
var failure *ProbeFailure
if errors.As(err, &failure) {
    fmt.Printf("%s đang lỗi tại %s:%d\n",
        failure.Service,
        failure.Endpoint.Host,
        failure.Endpoint.Port,
    )
}
```

`errors.As` không hỏi "message có chứa host không?". Nó tìm một error assignable vào type mà caller yêu cầu. Đây là lý do custom error nên có dữ liệu mà caller thật sự dùng, không phải chỉ là một struct bọc mỗi string.

Failure analysis: state nào phải được ghi, lỗi nào phải được trả?

Đây là implementation của boundary. Hãy đọc nó như một review: mỗi return mang thông tin nào, và state nào còn tồn tại sau return?

```
func applyProbe(
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error {
    service, found := registry[name]
    if !found {
        return fmt.Errorf(
            "unknown %q: %w",
            name,
            ErrUnknownService,
        )
    }

    if err := run(service.Endpoint); err != nil {
        service.Record(false)
        registry[name] = service
        failed := &ProbeFailure{
            Service:  service.Name,
            Endpoint: service.Endpoint,
            Cause:    err,
        }
        return fmt.Errorf("probe %q: %w", name, failed)
    }

    service.Record(true)
    registry[name] = service
    return nil
}
```

Nhánh thiếu config trả sớm: không có `Service` để mutate, nên registry không đổi. Nhánh probe thất bại vẫn ghi state `Healthy=false` và tăng retry, rồi trả failure có nguyên nhân. Đây không phải hai cơ chế báo lỗi cạnh tranh; state trả lời "lần quan sát gần nhất ra sao", còn error trả lời "operation vừa gọi không hoàn tất vì sao". Nhánh thành công ghi state rồi trả `nil`.

Khi kiểm thử mã nguồn này, hệ thống tập trung chứng minh ba cam kết hợp đồng cụ thể thay vì chỉ so khớp chuỗi thông báo:

Tình huống kiểm thử	Điều kiện xác minh trạng thái	Điều kiện xác minh lỗi (Error Contract)
Dịch vụ không tồn tại trong cấu hình	Hàm kiểm tra probe tuyệt đối không được gọi.	`errors.Is(err, ErrUnknownService)` trả về `true`.
Thao tác probe thất bại	Trạng thái dịch vụ cập nhật `Healthy=false` và tăng số lần thử lại.	`errors.As` trích xuất thành công `*ProbeFailure`, đồng thời `errors.Is` bảo toàn nguyên nhân gốc.
Thao tác probe thành công	Trạng thái dịch vụ cập nhật `Healthy=true` tại đúng endpoint cấu hình.	Giá trị lỗi trả về là `nil`.

Lab `part4-error-boundaries` biến ba contract này thành test. `ProbeFunc` được thay bằng function nhỏ trong test; không cần mở socket thật để chứng minh error boundary. Hãy mở `main_test.go` trước `main.go`, đổi riêng `%w` thành `%v` ở return cuối của nhánh probe lỗi, rồi chạy `TestApplyProbePreservesFailureCauseAndContext`. Message vẫn gần như cũ, nhưng test phải đỏ vì `errors.Is` không còn thấy nguyên nhân. Khôi phục `%w`, sau đó tạm bỏ nhánh cancellation đặc biệt và chạy `TestApplyProbeCancellationDoesNotChangeHealthState`: lúc này feedback của test cho thấy cancellation đã bị viết nhầm thành health failure. Hai failure injection này đáng làm hơn là chép lại error chain đã có sẵn.

> **Bài tập - giữ lại điều gì?** Một `run` trả `errors.New("connection refused")`. Nếu `applyProbe` đổi `%w` ở return cuối thành `%v`, message của CLI có thể vẫn hiển thị "connection refused". Contract nào trong ba contract trên đã mất? Viết một assertion thể hiện điều đó.

Đáp án. Contract nhận biết nguyên nhân gốc mất: `errors.Is(err, errConnectionRefused)` sẽ false vì chuỗi error không còn unwrap tới error ban đầu. Context cho người đọc không đủ để thay thế identity cho

code. Assertion cần tạo `errConnectionRefused`, gọi `applyProbe`, rồi yêu cầu `errors.Is(got, errConnectionRefused)` là true.

Điểm dừng của boundary này

Ta chưa retry, chưa timeout, và chưa cleanup tài nguyên; mọi thứ đó sẽ chỉ làm mạch này khó đọc nếu đưa vào trước. Hiện tại API đã có ranh giới rõ: `nil` nghĩa là operation hoàn tất, error giữ được nguyên nhân và context, còn caller dùng `Is` hoặc `As` đúng theo quyết định mình cần.

Phần tiếp theo của Chương 4 sẽ đặt error boundary này dưới áp lực của tài nguyên và cancellation: `defer` phải chạy ở đâu, `context` đi qua API nào, và vì sao `panic` không phải đường tắt thay cho một failure có thể dự đoán.

Cancellation không phải health signal

Một probe có thể bị hủy vì người dùng dừng CLI, vì request cha hết thời gian, hoặc vì service quản lý gọi đang shutdown. Không tình huống nào trong số đó chứng minh endpoint của `billing` đang down. Nếu `applyProbe` gộp mọi error vào nhánh `Record(false)`, một cancellation của caller sẽ tạo ra một health signal giả.

`context.Context` mang deadline và tín hiệu cancellation qua ranh giới API. Nó đi vào đầu signature, không được cất trong `Service`, và được truyền xuống function có thể chặn:

```
type ProbeFunc func(context.Context, Endpoint) error

func applyProbe(
    ctx context.Context,
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error
```

Trước khi chạy probe, boundary kiểm tra cancellation để không bắt đầu I/O vô ích. Nếu cancellation đến trong lúc `run` đang làm việc, `run` phải tôn trọng `ctx` và trả một error vẫn nhận diện được bằng `errors.Is(err,`

`context.Canceled)` hoặc `errors.Is(err, context.DeadlineExceeded)`. Trả thẳng `ctx.Err()` là cách đơn giản; wrapping để thêm context cũng đúng nếu không làm mất identity. `applyProbe` nhận diện hai nguyên nhân chuẩn ấy, giữ chúng trong error chain, nhưng không thay health state:

```
if err := ctx.Err(); err != nil {
    return fmt.Errorf("probe canceled: %w", err)
}

if err := run(ctx, service.Endpoint); err != nil {
    if errors.Is(err, context.Canceled) ||
        errors.Is(err, context.DeadlineExceeded) {
        return fmt.Errorf("probe incomplete: %w", err)
    }

    service.Record(false)
    registry[name] = service
    failed := &ProbeFailure{
        Service: service.Name,
        Endpoint: service.Endpoint,
        Cause: err,
    }
    return fmt.Errorf("probe %q: %w", name, failed)
}
```

Đây là một policy của `opsprobe`, không phải luật rằng cancellation không bao giờ có ý nghĩa vận hành. Tại boundary này, state `Healthy` có nghĩa là kết quả của lần quan sát hoàn tất. Một operation bị hủy chưa đưa ra quan sát đó. Caller vẫn có thể dùng `errors.Is(err, context.Canceled)` để dừng yên lặng, hoặc `errors.Is(err, context.DeadlineExceeded)` để báo timeout.

`defer` đứng cạnh acquisition

Cancellation không tự đóng mọi resource mà một function đã mở. Ownership nằm ở nơi acquisition thành công: function nào có session thì function đó phải sắp cleanup trước khi gọi phần có thể return. Ta tách phần này khỏi health state để đọc được policy cleanup mà không lẫn nó với policy endpoint:

```
type ProbeSession interface {
    Close() error
}

type OpenSession func(Endpoint) (ProbeSession, error)
type SessionProbe func(
    context.Context,
    Endpoint,
    ProbeSession,
) error

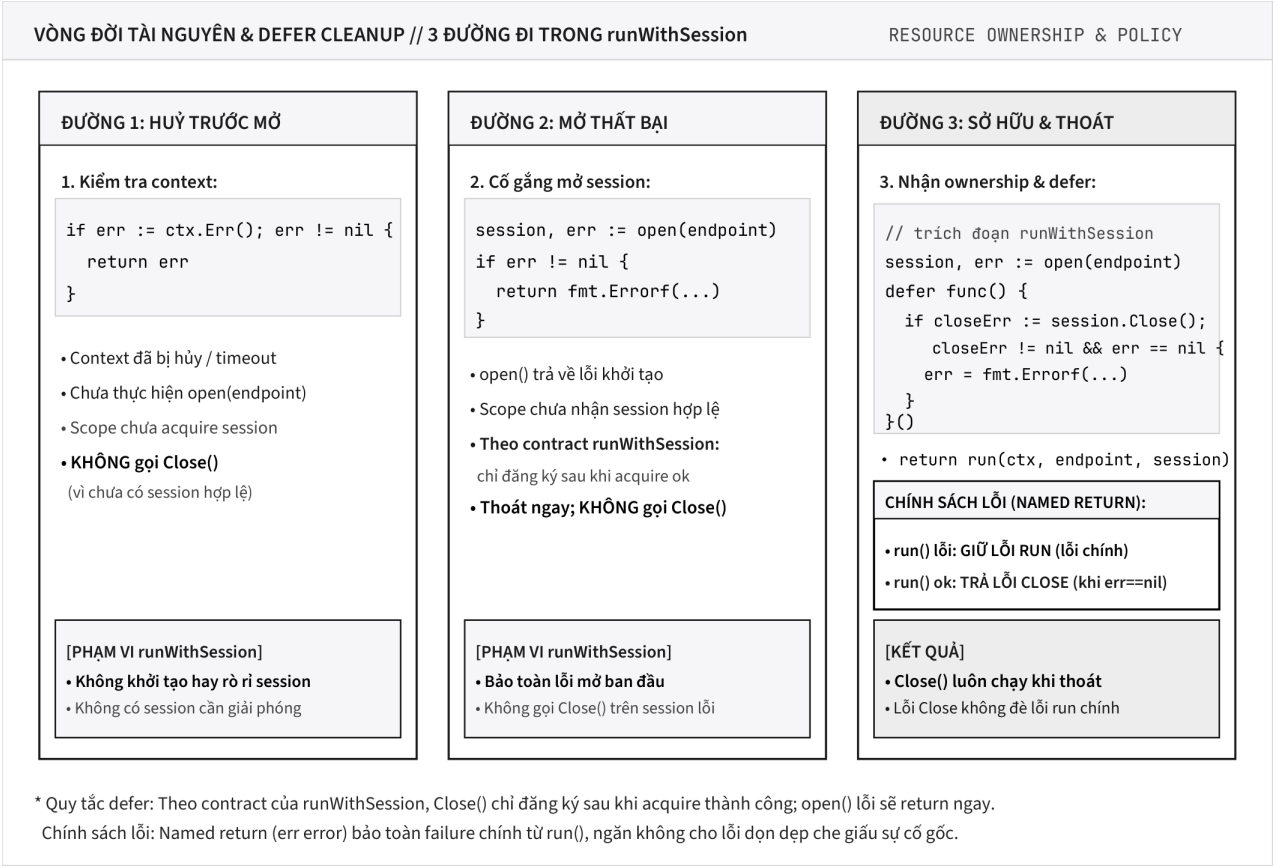
func runWithSession(
    ctx context.Context,
    endpoint Endpoint,
    open OpenSession,
    run SessionProbe,
) (err error) {
    if err := ctx.Err(); err != nil {
        return err
    }

    session, err := open(endpoint)
    if err != nil {
        return fmt.Errorf("open session: %w", err)
    }
    defer func() {
        if closeErr := session.Close(); closeErr != nil {
            if err == nil {
                err = fmt.Errorf("close session: %w", closeErr)
            }
        }
    }()

    return run(ctx, endpoint, session)
}
```

`defer` được đăng ký ngay sau khi `open` thành công. Nếu `open` thất bại, chưa có session để đóng. Nếu `run` return theo bất kỳ nhánh nào, `Close` chạy trước khi `runWithSession` trả result cuối cùng. Named result `err` ở đây có lý do cụ thể: cleanup error chỉ trở thành result khi operation chính đã thành công. Nếu `run` đã có error, code giữ primary failure thay vì thay thế

nó bằng một lỗi close thứ cấp. Một hệ thống có yêu cầu lưu cả hai có thể dùng policy khác, nhưng phải làm điều đó có chủ ý.



Hình 18 — `defer` không phải một câu thần chú dọn dẹp toàn chương trình. Nó ràng buộc cleanup với scope đã nhận ownership. Error từ `Close` chỉ thay result khi `run` chưa có failure để bảo toàn nguyên nhân chính.

Lưu ý một ranh giới khác: caller tạo context dẫn xuất phải gọi `cancel`, kể cả khi deadline có thể tự hết. Trong code ngắn, pattern là `ctx, cancel := context.WithTimeout(parent, timeout)` ngay sau đó `defer cancel()`. `cancel` giải phóng resource liên quan tới context; còn `session.Close` giải phóng resource mà `runWithSession` đã acquire. Hai cleanup này có owner khác nhau.

Code review: đừng dùng `recover` để che failure dự đoán được

Một review có thể gặp đoạn sau và thấy nó “tránh crash”:

```
defer func() {
    if recovered := recover(); recovered != nil {
        err = fmt.Errorf("probe failed: %v", recovered)
    }
}()
```

Đoạn này không phải cách thay thế cho `return error`. Cancellation, timeout, config thiếu và connection refused đều là outcome mà API đã dự kiến; chúng phải đi qua `error` chain để caller phân loại được. Dùng `recover` ở đây biến bug lập trình như nil dereference hoặc invariant hỏng thành một string mơ hồ, trong khi state có thể đã bị thay đổi dở dang.

`panic` thường phù hợp với lỗi lập trình, invariant nội bộ không giữ được, hoặc boundary recovery được thiết kế rõ để cách ly code không tin cậy. `recover` chỉ có chỗ khi boundary đó có kế hoạch phục hồi state, ghi nhận đầy đủ và có contract rõ ràng sau recovery. `opsprobe` hiện không có boundary như vậy; để panic lộ ra trong test thường trung thực hơn là giả nó thành "probe failed".

> **Bài tập - trace cleanup:** `ctx` bị cancel sau khi `open` trả một session, còn `run` trả `context.Canceled`. `Close` có được gọi không? Error cuối giữ identity nào? `registry["billing"]` có nên đổi `Healthy` hay `Retries` không?

Đáp án. `Close` được gọi đúng một lần vì `defer` đã được đăng ký sau acquisition. `runWithSession` giữ `context.Canceled` là primary error nếu `Close` cũng lỗi; `applyProbe` bọc nguyên nhân đó để `errors.Is(err, context.Canceled)` vẫn true. Vì operation không hoàn tất một quan sát endpoint, policy hiện tại giữ nguyên `Healthy` và `Retries`.

Kết chương: failure có đường đi

Một error boundary tốt không cố làm mọi failure giống nhau. `ErrUnknownService` cho caller một loại lỗi để cấu hình; `ProbeFailure` giữ context của quan sát thật; wrapping giữ identity; cancellation đi qua `Context` nhưng không bị viết nhầm thành health signal; `defer` đóng resource mà scope đã nhận ownership. `panic` vẫn là tín hiệu khác, không phải lối tắt để khỏi thiết kế result.

`opsprobe` nay nói rõ điều nó sở hữu, quan sát và để caller quyết định. Tiếp theo: package boundary, export và dependency kín.

Bốn câu hỏi trước khi merge một error boundary

Tiêu chí kiểm định ranh giới lỗi	Yêu cầu kỹ thuật bắt buộc
Nhận diện định danh lỗi (Error Identity)	Bên gọi có cần rẽ nhánh quyết định theo từng loại lỗi cụ thể không? Nếu có, phải bảo toàn identity bằng sentinel error hoặc custom type có cấu trúc; cấm bắt caller parse chuỗi văn bản.
Bản chất quan sát miền nghiệp vụ	Lỗi này có phản ánh trạng thái thực của dịch vụ đích không? Timeout hoặc cancellation từ Context của caller không tự động đồng nghĩa với việc endpoint bị unhealthy.
Quyền sở hữu tài nguyên (Resource Ownership)	Tài nguyên được cấp phát ở đâu, và lệnh dọn dẹp qua defer có được đặt ngay sau đó không? Mỗi defer phải gắn trực tiếp với scope chịu trách nhiệm giải phóng.
Chính sách bảo toàn lỗi gốc (Primary Error Policy)	Nếu thao tác dọn dẹp trong defer cũng phát sinh lỗi, lỗi ban đầu có bị nuốt mất không? Cần bảo toàn lỗi gốc để không làm gián đoạn việc điều tra sự cố.

Khi bốn câu trả lời hiện ngay trong chữ ký hàm, chuỗi bọc lỗi, chuyển dịch trạng thái và kiểm thử tự động, người đọc sau không cần phỏng đoán lỗi có bị che lấp, số lần thử lại có bị tăng sai, hay tài nguyên hệ thống có bị rò rỉ hay không. Đó là sự chuẩn bị cần thiết trước khi tách mã nguồn thành các package độc lập.

Chương 5 — Package là ranh giới

Sau Chương 4, `opsprobe` có một API cho một lần probe. Nhưng lab vẫn là một package `main` chứa model, policy error, config giả và cách in ra terminal. Khi mọi thứ còn ngắn, một tệp là lựa chọn tốt. Vấn đề xuất hiện không phải vì số tệp tăng, mà vì một thay đổi ở cách in CLI có thể nhìn thấy mọi chi tiết của probe, và một thay đổi config có thể kéo domain model đi cùng.

Ta sẽ không mở đầu bằng cây thư mục "chuẩn". Ta bắt đầu bằng một review: code nào phải biết `fmt.Println`, code nào phải biết endpoint, và code nào có quyền quyết định config được đọc từ đâu? Khi câu trả lời khác nhau, boundary cũng phải khác nhau.

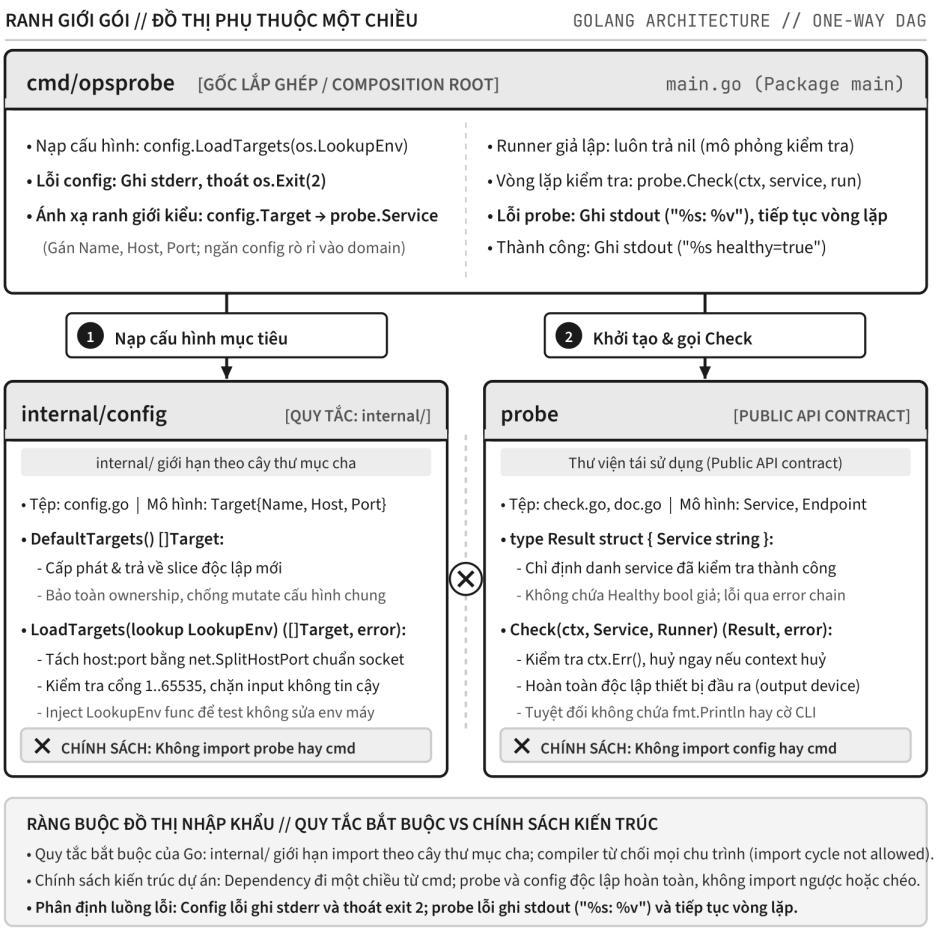
Một dependency graph có thể đọc từ ngoài vào trong

Refactor đầu tiên của `opsprobe` phân định ba vai trò kiến trúc rõ rệt:

Package `cmd/opsprobe` đóng vai trò điểm khởi đầu (command entrypoint): tiếp nhận các phụ thuộc, điều phối các ca sử dụng và trình bày kết quả ra console.

Package `probe` là thư viện có thể tái sử dụng (importable package): sở hữu mô hình nghiệp vụ cho một lần kiểm tra và hoàn toàn độc lập với thiết bị đầu ra.

Package `internal/config` là chi tiết triển khai nội bộ của ứng dụng: quản lý cấu hình mặc định, với phạm vi import bị `go` command giới hạn theo cây thư mục cha của `internal`, không phải một quy tắc chung "cùng module".



Hình 19 — Dependency direction là công cụ review. `cmd/opsprobe` là composition root; nó biết các package cụ thể. `probe` không biết CLI hay config source nên có thể được đọc, test và thay thế runner mà không kéo presentation theo.

`go.mod` đặt module path, còn mỗi directory Go là một package. Vì lab dùng module `example.com/golang-master/part5-package-design`, import path của package `probe` là `example.com/golang-master/part5-package-design/probe`. Một package có thể gồm nhiều file trong cùng directory; chia file không tạo thêm boundary nếu chúng vẫn khai báo cùng `package probe`.

```
part5-package-design/
├── go.mod
├── cmd/opsprobe/main.go
├── probe/doc.go
├── probe/check.go
└── internal/config/config.go
```

Tên `cmd` là convention hữu ích khi repository có thể chứa nhiều executable; bản thân compiler không trao ý nghĩa đặc biệt cho nó. `main.go` cũng chỉ là tên file theo convention. Boundary thực sự đến từ package clause, import path và hướng dependency.

Export là lời hứa, không phải cách sửa compile error

Trong `probe`, caller cần tạo service, truyền một runner và đọc result. Những khái niệm ấy mới cần export:

```
package probe

type Endpoint struct { Host string; Port int }
type Service struct { Name string; Endpoint Endpoint }

type Runner func(context.Context, Endpoint) error
type Result struct { Service string }

func Check(
    ctx context.Context,
    service Service,
    run Runner,
) (Result, error)
```

Chữ cái đầu viết hoa không làm type "quan trọng hơn"; nó mở một contract cho package khác. `Check` được export vì command thực sự gọi nó. Helper thêm context error ở bên trong có thể giữ unexported vì chưa có caller nào ngoài `probe` cần nó. Export trước rồi hy vọng một consumer xuất hiện là cách tạo API phải mang gánh nặng tương thích không cần thiết.

Package documentation cũng là một phần của contract. File `probe/doc.go` bắt đầu bằng comment `Package probe ...`, nói package chịu trách nhiệm gì và không chịu trách nhiệm gì. Comment của export nên mô tả behavior mà caller có thể dựa vào, không chỉ lặp lại tên type.

`internal` là boundary do go command kiểm tra

Target mặc định là policy của application hiện tại. Ta để nó trong `internal/config`:

```
package config

type Target struct {
    Name string
    Host string
    Port int
}

func DefaultTargets() []Target {
    return []Target{
        {
            Name: "billing",
            Host: "billing.internal",
            Port: 8443,
        },
    }
}
```

Package ở `internal` không phải "private bằng thỏa thuận". Go chỉ cho code nằm trong cây thư mục của parent import nó. Vì `cmd/opsprobe` nằm dưới module root — parent của `internal` — nó import được `internal/config`. Code nằm ngoài cây cha ấy bị `go` command từ chối. Không nên dùng số module làm phép thử: một repository nhiều module có thể vẫn có các package nằm trong cây được phép.

Đây không phải lý do để nhét mọi package vào `internal` theo phản xạ. Với binary tự chứa, đó thường là default tốt vì không hứa API ngoài. Với `probe` trong lab này, ta cố tình để nó importable để boundary giữa domain operation và CLI được nhìn thấy. Nếu mai kia không còn consumer ngoài application, nó cũng có thể trở thành internal; vị trí phải theo consumer thật, không theo cây thư mục đẹp mắt.

Composition root chuyển dữ liệu, không giấu dependency

`config.Target` và `probe.Service` không phải cùng type. Chúng thuộc hai boundary khác nhau: config nói dữ liệu được khai báo thế nào; probe nói một operation cần gì. `cmd/opsprobe` là nơi phù hợp để map giữa chúng:

```
for _, target := range config.DefaultTargets() {
    service := probe.Service{
        Name: target.Name,
        Endpoint: probe.Endpoint{
            Host: target.Host,
            Port: target.Port,
        },
    }

    result, err := probe.Check(ctx, service, run)
    if err != nil {
        fmt.Printf("%s: %v\n", service.Name, err)
        continue
    }
    fmt.Printf("%s healthy=true\n", result.Service)
}
```

Mapping này trông như boilerplate nhỏ, nhưng nó giữ dependency direction trung thực: `probe` không cần import config, config không cần biết Result, và code in terminal không rò vào domain package. Khi source config chuyển từ default sang file hay environment ở chương sau, `probe.Check` không đổi chỉ vì đường đi của config đã đổi.

Một refactor có kiểm chứng, không phải một lần dời file

Lab kiểm tra ba cam kết kỹ thuật trọng yếu khi phân tách package nhằm bảo vệ ranh giới kiến trúc:

Thành phần kiểm thử	Hành vi kỳ vọng của hệ thống	Rủi ro nếu thiết kế sai ranh giới
Điều phối hủy bỏ (`probe.Check`)	Chuyển `ctx.Err()` thành lỗi nhận diện được bằng `errors.Is`, không kích hoạt runner khi context đã bị hủy.	Tiếp tục tiêu tốn tài nguyên mạng dù bên gọi đã phát tín hiệu dừng.
Toàn vẹn kết quả	Đối tượng kết quả mang chính xác tên định danh dịch vụ đã cấu hình.	Trộn lẫn trạng thái giữa các dịch vụ trong quá trình duyệt danh sách.
Độc lập dữ liệu (`internal/config`)	Trả về một slice độc lập theo từng lần gọi factory.	Caller sửa slice nhận được làm đột biến giá trị cấu hình mặc định dùng chung.

Test cuối không phải để khẳng định "mọi slice đều dangerous". Nó chứng minh ownership của config factory: `DefaultTargets` tạo dữ liệu mới cho caller thay vì phát một slice dùng chung. Đây là cùng câu hỏi value semantics của Chương 2, nay được đặt vào một package boundary.

Khởi tạo Gói và Ràng buộc Đồ thị Phụ thuộc (Initialization DAG)

Go specification không cho phép package import chính nó, trực tiếp hay gián tiếp. Vì vậy đồ thị import không có chu trình; build có import cycle bị từ chối với diagnostic `import cycle not allowed`. Quy tắc này giúp xác định thứ tự khởi tạo package, không chứng minh các dependency ngoài process đã sẵn sàng.

Nếu package `A` import `B`, thì `B` phải được khởi tạo hoàn tất trước khi `A` bắt đầu. Trong phạm vi từng package, các biến cấp gói được khởi tạo theo thứ tự phụ thuộc tĩnh, tiếp theo là các hàm `init()` chạy tuần tự trong goroutine khởi tạo trước khi `main` chạy. Tuy nhiên `init` có thể tạo goroutine khác; import graph không chứng minh các goroutine ấy đã hoàn tất hoặc config ngoài đã sẵn sàng. Công việc có thể lỗi, cần deadline hoặc cần dependency ngoài nên được gọi tường minh từ điểm khởi tạo ứng dụng, nơi caller quan sát được error.

Lần này không bắt đầu từ implementation đã tách. Mở `labs/part5-package-refactor` trong VS Code: điểm xuất phát là một `main.go` monolithic đang chạy. Chạy `go run .` để giữ behavior làm mốc,

rồi chạy `go test -tags exercise ./...`. Lỗi compiler vì package `probe` chưa tồn tại là tín hiệu bắt đầu, không phải lỗi để lờ đi: test đang đòi một public boundary mà code chưa có. Từ requirement và test, anh tự quyết định file nào đi vào `probe`, file nào là `internal/config`, và command map hai model ở đâu. Khi xong, `go run ./cmd/opsprobe` phải giữ output, còn `go test -tags exercise ./...` và `go vet ./...` phải xanh.

Đáp án — chỉ đọc sau khi đã tự làm. Reference implementation nằm ở `labs/part5-package-design`, nhưng chỉ nên mở sau khi acceptance test đã cho phản hồi đầu tiên. Ranh giới cần giữ là: `probe` export operation mà test gọi, `internal/config` giữ policy default và trả slice mới, còn `cmd/opsprobe` là nơi map data rồi in kết quả. Không có lý do để `probe` import ngược config hoặc terminal.

> **Bài tập - đặt code ở đâu?** Một bạn muốn thêm `fmt.Println` vào `probe.Check` để in progress, và muốn `probe` gọi `config.DefaultTargets` để đỡ mapping ở `main`. Với dependency graph trên, hai thay đổi đó tạo ra rủi ro gì? Đề xuất nơi thay thế cho mỗi việc.

Đáp án. `fmt.Println` kéo presentation vào package có thể import; command nên quyết định format và nơi ghi output. Để `probe` gọi config đảo dependency từ domain operation sang application policy, khiến test probe phải mang config theo. Progress có thể là result/event do caller trình bày; mapping nên ở command hoặc một application service nằm phía ngoài cả `probe` lẫn config.

Một config source là dữ liệu không tin cậy, không phải endpoint

Requirement tiếp theo đến từ người vận hành: trong một lần chạy, họ muốn đổi endpoint mặc định mà không sửa source. Ta chọn một biến môi trường có shape rõ ràng: `OPS_PROBE_TARGET=payments.internal:9443`. Đây là capability của application, không phải capability của `probe`. `probe.Check` chỉ nhận `Endpoint` đã hợp lệ; nó không có lý do để biết chuỗi đó đến từ môi trường, file hay một flag tương lai.

Điều này giữ một ranh giới quan trọng. Environment luôn đưa vào string; endpoint cần host và port có ý nghĩa. Việc tách, chuyển kiểu và kiểm tra range là công việc của `internal/config`. Command đưa implementation thật `os.LookupEnv` vào boundary ấy, rồi chỉ nhận `[]Target` hoặc error. Shape của API đủ nhỏ để test mà không đổi environment của máy:

```
type LookupEnv func(string) (string, bool)

func LoadTargets(lookup LookupEnv) ([]Target, error)
```

Đây không phải một dependency-injection framework. `LookupEnv` chỉ là function type nói đúng dependency mà config cần: với một key, trả value và cho biết key có tồn tại hay không. Trong test, một closure nhỏ thay `os.LookupEnv`; trong command, `os.LookupEnv` thỏa đúng signature. Nhờ vậy test mô tả input rõ ràng mà không có test nào phải sửa environment process chung.

`host:port` cũng không nên được cắt bằng `strings.Split`. Dấu `:` xuất hiện trong IPv6, nên `net.SplitHostPort` là parser phù hợp cho grammar socket. `payments.internal:9443` là input hợp lệ; IPv6 cần bracket như `[2001:db8::9]:9443`. Sau khi tách, port vẫn là text: `strconv.Atoi` chuyển nó thành `int`, rồi config kiểm tra range `1..65535` trước khi tạo target. Mọi input lỗi trả error có context để CLI in ra và dừng với exit code khác 0.

Ở đây không cần sentinel mới. Caller hiện chỉ có một policy cho config lỗi: báo người dùng và dừng trước khi probe bắt đầu. Nếu sau này CLI cần phân

biệt lỗi syntax với secret bị thiếu để đưa remediation khác nhau, đó mới là lúc error identity đáng được đưa vào contract. Chương 4 không dạy "mọi error phải có type"; nó dạy chỉ giữ identity khi caller có quyết định khác nhau.

Thực hành - requirement thay đổi sau refactor. Sau khi hoàn thành lần tách package đầu tiên trong `labs/part5-package-refactor`, chạy `go test -tags configexercise ./...`. Test đòi `internal/config.LoadTargets` nhận một function đọc environment và đổi endpoint khi có `OPS_PROBE_TARGET`. Tự chọn file, API và error message; constraints là dùng `net.SplitHostPort`, từ chối host rỗng và port ngoài `1..65535`, còn `cmd/opsprobe` mới được gọi `os.LookupEnv`. Khi test xanh, chạy command với `OPS_PROBE_TARGET=payments.internal:9443` để kiểm tra đường đi runtime.

Đáp án — chỉ đọc sau khi đã tự làm. Reference implementation giữ `LoadTargets` trong `internal/config`, bắt đầu từ `DefaultTargets` để ownership của slice không đổi, rồi chỉ override host và port của target đầu. Command xử lý error ở process boundary bằng stderr và exit code 2; `probe` không thay đổi. Đây là dấu hiệu refactor tốt: requirement về source config đi qua một package, trong khi operation kiểm tra endpoint không cần biết nguồn dữ liệu vừa đổi.

Code review: một field không có trạng thái thứ hai

Sau config change, ta đọc lại public API thay vì vội thêm feature khác. `Check` có hai outcome: return `error` khi operation không hoàn tất, hoặc return `Result` khi runner thành công. Trong contract này, `Result.Healthy` luôn là `true`. Field đó trông vô hại, nhưng nó gợi cho caller một trạng thái thứ hai mà function không bao giờ trả về.

Đây là một bug thiết kế, không phải bug compiler. Một caller mới có thể viết `if !result.Healthy { retry() }`, rồi tin rằng branch ấy có ý nghĩa. Thực tế, failure đã đi qua `error`; `result` zero value đi cùng error không

phải một observation “unhealthy”. Giữ bool chỉ vì câu in terminal cần chữ `true` làm biên API rộng hơn dữ liệu mà operation thực sự cung cấp.

Ta thu hẹp `Result` còn service đã hoàn tất. `cmd/opsprobe` đã có nhánh `err == nil`, nên chính command - nơi sở hữu presentation - in `healthy=true`. Không có information bị mất: success đã là bằng chứng của `true`; failure vẫn là error với context. Đây là một lần refactor đặc biệt đáng làm vì test không đỏ trước khi đổi. Test cũ xanh, nhưng contract cũ làm người đọc có thể suy luận sai.

```
// Result identifies the service whose check
// completed successfully.
type Result struct {
    Service string
}
```

Một comment tốt giờ cũng có thể viết chính xác hơn: Result không “chứa health status”, mà định danh service có check thành công. Khi API thật sự cần diễn đạt nhiều trạng thái quan sát - ví dụ healthy, degraded, unreachable với timestamp và latency - đó sẽ là một model khác, được thiết kế cùng requirement và test riêng. Đừng giữ placeholder cho một feature chưa tồn tại chỉ để API trông linh hoạt.

Thực hành - review dưới ràng buộc. Trong `labs/part5-package-design`, tạm giữ `Healthy bool` trong `Result` rồi viết ra hai outcome mà `Check` có thể trả. Sau đó bỏ field, sửa test và chỉ sửa presentation ở command cho đến khi `go test ./...` cùng `go run ./cmd/opsprobe` đều xanh. Không được chuyển failure thành `Result{Healthy: false}`: điều đó làm mất error chain mà Chương 4 đã xây. Trong bản refactor độc lập, `public_boundary_test.go` là contract tối thiểu; hãy sửa implementation theo test, không thêm field chỉ để đoán tương lai.

Đáp án — chỉ đọc sau khi đã tự làm. `Result{Service: service.Name}` là đủ cho nhánh thành công. Sau `err == nil`, command tự in literal `healthy=true`. Nếu một ngày cần health state mà vẫn giữ operation failure riêng, API sẽ cần một contract mới nói rõ result nào có thể xuất hiện cùng error nào; đó không phải việc của bool hiện tại.

Module chọn source nào để build

Một package boundary đúng vẫn có thể build với source khác điều anh tưởng. Package là đơn vị tổ chức code; module là tập package có version và dependency requirements. `go.mod` đặt module path, phiên bản ngôn ngữ và các requirement; `go.sum` lưu checksum để kiểm tra nội dung dependency đã tải. Nó không phải lockfile liệt kê duy nhất mọi version sẽ dùng trong mọi chế độ build.

Go dùng minimal version selection cho các requirement trong module graph: với mỗi module path, chọn version cao nhất được yêu cầu trong graph liên quan, không tự chọn bản mới nhất trên Internet. “Minimal” nói về tập version thỏa requirement, không có nghĩa chọn version thấp nhất trong mọi dependency. `replace` có thể thay source local hoặc version; một build dùng replace local chỉ chứng minh bản source ấy. Replace trong dependency không tự trở thành policy của main module tiêu dùng nó. Khi điều tra, đọc `go list -m all`, `go mod graph` và `go env GOMOD GOWORK` trước khi kết luận test đang chạy đúng dependency.

Workspace thêm một lựa chọn local: `go.work` có thể đưa nhiều module vào vai trò main module để phát triển cùng nhau. Nó không gộp chúng thành một module để phát hành. Một service trong workspace có thể dùng library vừa sửa trên máy, trong khi downstream vẫn tải version đã phát hành. Vì vậy kiểm tra ngoài workspace là một phép thử khác, không phải sự lặp thừa:

```
go env GOMOD GOWORK
go list -m all
# Chạy trong module muốn kiểm tra độc lập
$env:GOWORK = "off"
go test ./...
Remove-Item Env:GOWORK
```

Nếu đã có `GOWORK` trong môi trường, hãy lưu và khôi phục giá trị thay vì xóa nó; đoạn trên minh họa phiên shell chưa đặt biến ấy. Bài thực hành không cần dependency thật: tạo hai module local, cho module A import B, dùng workspace để test, rồi tắt workspace. Giải thích vì sao kết quả đổi, và điều gì phải xuất hiện trong `go.mod` hoặc version đã phát hành để consumer độc lập build được. Đừng chữa lỗi bằng một replace trỏ vào đường dẫn laptop rồi gọi artifact đó là tái tạo được.

Toolchain đang chạy và phiên bản ngôn ngữ của module là hai điều khác nhau. Trong Go 1.27, `go test` mặc định chạy vet check `stdversion`, phát hiện symbol thư viện chuẩn quá mới so với version có hiệu lực cho file, xét cả `go.mod` và build tags. Cài toolchain mới không tự hợp thức hóa việc dùng API mới trong module tuyên bố hỗ trợ bản cũ. `go mod tidy` của module `go 1.27` còn gộp các require block trùng thành nhóm trực tiếp/gián tiếp; diff đó là thay đổi biểu diễn dependency, không phải bằng chứng đã nâng hay triển khai dependency thành công.

Build là phép chọn tệp trước khi compile

Hai tệp trong cùng thư mục chưa chắc cùng đi vào một binary. Tên như `probe_linux.go`, suffix kiến trúc và dòng `//go:build` điều khiển việc chọn source. Constraint phải ở đầu file, trước package clause và có dòng trống phân cách. Đây là bước chọn file của toolchain; compiler vẫn kiểm tra những file đã được chọn theo cùng language semantics.

Với một helper thuần Go, anh có thể đặt hai implementation cùng signature trong các tệp chọn theo OS. `go list -json .` cho biết `GoFiles`, `IgnoredGoFiles` và `CgoFiles`; nó hữu ích hơn suy đoán từ cây thư mục. Cross-compilation đổi target build, không làm máy Windows thực thi hay kiểm tra syscall Linux:

```
$env:GOOS = "linux"
$env:GOARCH = "amd64"
$env:CGO_ENABLED = "0"
go build -o probe-linux ./cmd/opsprobe
```

Chạy trong shell tạm hoặc khôi phục ba biến về giá trị trước đó. Build thành công mới chứng minh compiler/linker chấp nhận tập source cho target, chưa chứng minh binary chạy đúng trên target. Với cgo, còn cần C compiler và library phù hợp target; tắt cgo không phải một bản cross-build tương đương nếu chức năng C bị loại khỏi source. Chương 14 dạy hợp đồng ở biên giới C, Chương 17 kiểm tra process trong môi trường triển khai, còn Chương 26 kiểm tra provenance và vulnerability của artifact. Các bước ấy trả lời những câu hỏi khác nhau.

Điểm dừng: package đủ nhỏ để thay đổi

Chưa có một "architecture hoàn chỉnh" nào ở đây: chưa parse file, chưa dùng generic, chưa tạo package cho từng noun. Ta chỉ có một graph mà mỗi mũi tên trả lời được "vì sao package này cần biết package kia?". Đó là mức cấu trúc đủ để tiếp tục thay đổi code mà không biến mọi thay đổi thành sửa `main`.

Chương 6 — Thay đổi không sợ hãi

Ở cuối Chương 5, `opsprobe` đã biết lấy một endpoint từ config và gọi `probe.Check`. Nhưng một yêu cầu rất bình thường vẫn khó chứng minh: khi operator đặt `OPS_PROBE_TARGET=payments.internal:9443`, endpoint nào thực sự đi tới runner? Chạy CLI và nhìn dòng `billing healthy=true` không trả lời được. Dòng đó không in host hay port; kể cả có in, một lần chạy tay vẫn không phải contract mà code giữ được khi nó đổi tiếp.

Đây là lúc nhiều codebase đi sai hướng. Có người viết test gọi `main`, sửa environment process chung, chụp stdout, rồi hy vọng không đụng `os.Exit`. Có người khác bỏ test vì "CLI chỉ vài dòng". Cả hai đều né câu hỏi thiết kế: phần nào là orchestration có thể kiểm chứng, phần nào là process boundary chỉ nên mỏng và rõ?

Một smoke test không biết điều ta cần biết

`main` hiện ghép nhiều trách nhiệm: lấy environment thật, gọi config, gọi probe, quyết định exit code và in terminal. Một test gọi nó chỉ quan sát đường chạy được thử. Với thiết kế hiện tại, test khó đưa environment giả vào an toàn, quan sát endpoint runner nhận được hoặc tách config error khỏi probe error mà không đụng side effect của process.

Vấn đề không phải `main` là xấu. `os.LookupEnv`, stdout, stderr và `os.Exit` đều có chỗ đúng: chúng là ranh giới nơi chương trình gặp process. Vấn đề là logic cần quyết định trước khi chạm ranh giới ấy chưa có một đơn vị để test gọi.

Ta tạo package nội bộ `internal/app`. Nó không thay thế `probe` hay `config`; nó phối hợp hai package đó. `Run` nhận đúng hai dependency biến đổi được trong test: function đọc environment và runner thực hiện probe. Nó trả outcome thay vì in:

```
type Outcome struct {
    Service string
    Err     error
}

func Run(
    ctx context.Context,
    lookup config.LookupEnv,
    runner probe.Runner,
) ([]Outcome, error)
```

`Outcome.Err` là failure của một target đã được cấu hình; CLI có thể in nó rồi tiếp tục target khác. `error` return riêng là failure không tạo được danh sách outcome có nghĩa, như config không hợp lệ hoặc cancellation của toàn bộ run. Việc tách hai đường này không phải để tạo type cho đẹp: caller có policy khác nhau. Chương 4 từng đặt câu hỏi "caller cần quyết định gì?"; ở đây câu trả lời quyết định shape của result.

Refactor đến khi test có thứ để nắm

`Run` load target, map `config.Target` sang `probe.Service`, gọi `probe.Check`, rồi giữ mỗi kết quả trong một `Outcome`. Nó không đọc `os.LookupEnv`, không gọi `fmt.Printf` và không được gọi `os.Exit`. Những side effect ấy còn ở `cmd/opsprobe/main.go`:

```
outcomes, err := app.Run(
    context.Background(),
    os.LookupEnv,
    runner,
)
if err != nil {
    fmt.Fprintln(os.Stderr, err)
    os.Exit(2)
}

for _, outcome := range outcomes {
    if outcome.Err != nil {
        fmt.Printf("%s: %v\n", outcome.Service, outcome.Err)
        continue
    }
    fmt.Printf("%s healthy=true\n", outcome.Service)
}
```

Đoạn `main` vẫn có logic tối thiểu: policy trình bày và process exit nằm lộ ra, dễ review. Nhưng test không cần gọi nó. Test nằm cạnh `Run`, đưa một closure làm `LookupEnv` và một `probe.Runner` ghi lại endpoint được gọi. Không có socket, environment toàn cục hay terminal nào bị đụng tới.

Trước khi viết assertion, hãy tách ba thứ bằng mắt. Chúng ngăn test biến thành bản sao của implementation:

Bảng 3 — Phạm vi của unit test quanh `Run`

Test kiểm soát	Test quan sát	Test không cần chạm
`LookupEnv` trả target nào; runner thành công hay lỗi	endpoint runner nhận; `Outcome` hoặc error trả về	environment process, stdout, stderr, `os.Exit`

Nếu một requirement chỉ hiện ra ở cột cuối, `Run` chưa phải đơn vị đúng để test; đó là việc của integration test ở một thời điểm có lý do. Nếu nó nằm ở cột đầu và cột giữa, unit test có thể cho feedback nhanh mà không giả lập cả process.

```
func TestRunMapsEnvironmentTargetToRunner(t *testing.T) {
    var got probe.Endpoint

    outcomes, err := Run(context.Background(), lookup(
        "OPS_PROBE_TARGET", "payments.internal:9443",
    ), func(_ context.Context, endpoint probe.Endpoint) error {
        got = endpoint
        return nil
    })

    if err != nil {
        t.Fatalf("Run() error = %v", err)
    }
    if got != (probe.Endpoint{
        Host: "payments.internal",
        Port: 9443,
    }) {
        t.Fatalf("runner endpoint = %+v", got)
    }
    if outcomes[0].Err != nil ||
        outcomes[0].Service != "billing" {
        t.Fatalf("outcome = %+v", outcomes[0])
    }
}
```

Đây là unit test không vì nó ngắn, mà vì nó kiểm chứng một đơn vị có boundary hẹp. Test hỏi một điều quan sát được: input config có đi tới runner đúng không, và result có được gán với service đúng không. Nó không kiểm tra implementation detail như `Run` dùng bao nhiêu local variable hay loop viết thế nào.

Hai error, hai policy

Thử thêm hai test trước khi đọc hết implementation. Với `OPS_PROBE_TARGET=payments.internal:0`, config phải error trước khi runner chạy. Đây là invalid input của cả run, nên `Run` return error và không có outcome nửa vời. Với runner trả `errors.New("connection refused")`, target đã tồn tại và operation đã bắt đầu; `Run` giữ `Outcome{Service: "billing", Err: ...}` để command có thể báo một dòng lỗi rồi tiếp tục khi danh sách target sau này dài hơn.

Cancellation khác với cả hai. `probe.Check` giữ identity `context.Canceled`, còn `Run` dừng thay vì biến cancellation của toàn bộ run thành một outcome per-service. Từ bên ngoài, ba policy hiện rõ:

Bảng 4 — Policy cho lỗi cấu hình, lỗi probe và cancellation

Tình huống	`Run` trả gì	Command nên làm gì?
Config không hợp lệ	`error`	In stderr, exit 2, không gọi runner.
Một probe thất bại	`Outcome.Err`	Báo service đó; policy hiện tại cho phép đi tiếp.
Context bị cancel	`error` giữ identity	Dừng run; không ghi health signal giả.

Không phải test nào cũng cần fake. Config parser đã có test input/output riêng ở Chương 5; `probe.Check` đã có test cancellation và wrapping. `Run` chỉ test phần nối ba contract ấy. Nếu một test chỉ lặp lại assertion của package dưới nó, nó làm refactor khó hơn mà không tăng niềm tin.

Thực hành - remove and rebuild. Mở `labs/part6-testable-command`, đọc `internal/app/run_test.go` trước. Đừng chạy command trước; hãy dự đoán mỗi test đang bảo vệ policy nào. Sau đó tạm đổi tên `Run` để compiler báo các điểm mà test đang đòi contract, rồi tự dựng lại function từ test và các type đã có. Khi unit test xanh, chạy `go run ./cmd/opsprobe`. Cuối cùng đổi runner trong `main` thành một runner trả error và xác minh CLI in service name thay vì nuốt failure. Khôi phục runner thành công trước khi rời lab.

Đáp án — chỉ đọc sau khi đã tự làm. `Run` cần gọi `config.LoadTargets` đúng một lần trước loop, map từng `Target` sang `probe.Service`, rồi append outcome thành công hoặc failure của từng probe. Chỉ config error và cancellation thoát khỏi `Run` qua return error. `main` giữ environment thật, output và exit code. Lời giải không phải "dùng mock framework"; hai function value đang có đã là seams vừa đủ.

Khi một test case che mất các policy khác

Test đầu tiên của `Run` cố ý chỉ dùng một override hợp lệ. Nó trả lời câu hỏi của refactor: endpoint có thật sự đi tới runner không? Nhưng nếu cứ nhân bản kiểu test đó cho từng input xấu, test suite sẽ dần thành một hành lang dài của những hàm gần giống nhau. Lúc ấy người đọc không còn nhìn thấy các rule của config; họ chỉ thấy rất nhiều lần gọi `LoadTargets`.

Một reviewer tốt sẽ hỏi khác đi: parser này có bao nhiêu policy độc lập? Với `OPS_PROBE_TARGET`, ta cần giữ ít nhất hai. Variable vắng mặt dùng default đã công bố. Variable có mặt phải là một `host:port` hợp lệ, có host và port trong khoảng TCP. Mỗi hàng test là một ví dụ cho một policy, không phải một input ngẫu nhiên để tăng coverage.

Đây là lúc bảng test có ích. Không phải vì Go community có một nghi thức tên là "table-driven test", mà vì bảng làm lộ phần thay đổi và phần giữ nguyên. `raw`, `found` và `want` thay đổi theo case; cách gọi parser và assertion về một target duy nhất được giữ chung:

```
for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        targets, err := LoadTargets(
            func(string) (string, bool) {
                return tt.raw, tt.found
            },
        )
        if err != nil {
            t.Fatalf("LoadTargets() error = %v", err)
        }
        if len(targets) != 1 || targets[0] != tt.want {
            t.Fatalf("LoadTargets() = %+v", targets)
        }
    })
}
```

`t.Run` không chỉ để kết quả trông có tổ chức. Khi một case vỡ, `go test -v` sẽ nói case nào vỡ, thay vì để anh đếm hàng trong slice. Vì vậy `name` phải mô tả behavior: “port zero” tốt hơn “case 4”, còn “uses the documented default” nhắc cả người bảo trì rằng default là một phần contract.

Review một bảng test trước khi tin nó

Thêm các input bị từ chối vào một table thứ hai trong `internal/config/config_test.go`:

```
tests := []struct {
    name string
    raw  string
}{
    {name: "missing port", raw: "payments.internal"},
    {name: "empty host", raw: ":8443"},
    {name: "non-numeric port", raw: "payments.internal:https"},
    {name: "port zero", raw: "payments.internal:0"},
    {
        name: "port above TCP range",
        raw:  "payments.internal:65536",
    },
}
```

Điểm dễ bỏ sót không nằm ở vòng `for`, mà ở assertion. Test này chỉ hứa rằng từng input không hợp lệ bị từ chối; nó không hứa nguyên văn error message. Message là chi tiết có thể sửa để tốt hơn cho operator. Rule "input này không được phép tạo target" mới là contract. Nếu CLI hoặc API cần phân loại lỗi bằng máy, khi đó mới có lý do để thiết kế error type hay sentinel rõ ràng; đừng dùng string comparison như một cách thay thế cho thiết kế đó.

Với edition Go 1.27.1 này, range variable được khai báo bằng `:=` có một variable riêng ở mỗi iteration (semantics từ Go 1.22), nên closure không còn vô tình cùng giữ một variable `tt` như folklore Go cũ. Nhưng một bẫy pointer khác vẫn còn: `&tt` trỏ tới bản copy của iteration, không phải phần tử thật trong slice `tests`. Nếu mục tiêu là mutate phần tử gốc, hãy range lấy index rồi dùng `&tests[i]`. Ở đây `t.Run` chạy đồng bộ và closure chỉ đọc `tt`, nên pointer lẫn lifetime goroutine đều không phải điều policy parser cần nghĩ.

Bài review ngắn. Trước khi chạy test, hãy tự phân loại năm hàng trên: hàng nào bị `net.SplitHostPort` từ chối, hàng nào đi qua parser nhưng bị validation của sách từ chối? Sau đó thêm một case hợp lệ cho IPv6 theo dạng `[2001:db8::10]:443`. Assertion của nó phải kiểm tra `Host` không còn dấu ngoặc và `Port == 443`, thay vì chỉ kiểm tra `err == nil`.

Đáp án — chỉ đọc sau khi đã tự làm. Thiếu port và `:8443` không tạo được target hợp lệ ở bước parse/host check. `https`, `0` và `65536` đi xa hơn: chúng cần conversion hoặc range validation. Với IPv6, `net.SplitHostPort` tách dạng có ngoặc thành host `2001:db8::10` và port `443`; chính vì vậy test nên ghi điều đó thành contract. Khi bảng bắt đầu chứa các cột không cùng một rule, hoặc mỗi row cần setup rất khác, tách test ra sẽ rõ hơn là nhồi thêm cột.

Chạy đúng bằng chứng

Khi suite xanh, `go test ./...` trả lời câu hỏi có giá trị nhưng rộng: có package nào vi phạm contract không? Lúc điều tra một report cụ thể, chạy tất cả test nhiều lần lại làm mất ngữ cảnh. Nếu operator báo rằng port `0` từng lọt qua validation, ta cần nhìn đúng rule đó trước, rồi mới mở rộng phạm vi.

Từ thư mục `labs/part6-testable-command`, lệnh sau chọn một subtest bằng đường dẫn tên của nó:

```
$case = 'TestLoadTargetsRejectsMalformedOverrides/port_zero'
go test -run $case -v ./internal/config
```

Đây không phải shortcut để né test đầy đủ trước khi commit. Nó là một kính lúp trong lúc debug: output giữ tên parent test và case `port_zero`, nên anh biết mình đang xem policy nào. Sau khi sửa bug, chạy lại package rồi chạy toàn bộ suite; một lỗi parser có thể làm app thất bại trước cả khi runner được gọi.

Hãy thử thay đổi tạm thời điều kiện range trong `LoadTargets` để port `0` đi qua, chỉ để xem subtest vỡ như thế nào. Đừng commit thay đổi đó. Bài học không phải thuộc câu lệnh `-run`; nó là phân biệt vòng lặp điều tra ngắn với bằng chứng đủ rộng để nhận một thay đổi vào codebase.

Từ đây Chương 6 đã có hai loại test khác nhau: `Run` kiểm chứng việc nối các boundary, còn config subtest giữ một ma trận input nhỏ nhưng có ý nghĩa. Phần tiếp theo sẽ tạo một failure mà unit test xanh vẫn chưa bắt được - hai goroutine cùng chạm vào một vùng dữ liệu.

Một test xanh vẫn có thể bỏ sót race

`opsprobe` hiện chưa chạy probe song song; đó là lựa chọn có chủ đích khi ta còn đang học boundary và policy lỗi. Vì vậy sẽ là sai nếu em giả vờ vừa phát hiện một race trong code của nó. Thay vào đó, hãy tách đúng failure cần học: nhiều worker cùng ghi vào một counter hoàn thành. Đây là loại bug có thể trông vô hại khi chạy một lần, vì không có compiler error và unit test tuần tự vẫn xanh.

Data race xảy ra khi nhiều goroutine truy cập cùng một biến đồng thời, có ít nhất một lần ghi, và không có synchronization phù hợp. Đây là định nghĩa về hành vi, không phải một lời phán đoán dựa trên việc count cuối cùng "có vẻ đúng". `successes++` không phải một hành động nguyên tử ở cấp source; nó đọc giá trị cũ, tạo giá trị mới rồi ghi lại. Hai goroutine có thể xen vào giữa các bước ấy.

Lab `labs/part6-race-detector/broken` giữ một counter cố ý sai. Test tạo 128 goroutine, mỗi goroutine gọi đúng một lần `RecordSuccess`:

```
type Counter struct {
    successes int
}

func (c *Counter) RecordSuccess() {
    c.successes++
}
```

Chạy test thường có thể không nói gì đáng ngờ. Để đưa memory access vào quan sát, hãy chạy fixture với race detector:

```
go test -race -tags raceexercise ./broken
```

Lệnh này được mong đợi là thất bại và in `WARNING: DATA RACE`. Đó là một failure có chủ ý của lab, không phải một check được phép xanh cho milestone. Race detector ghi stack trace ở các access xung đột và nơi các goroutine liên quan được tạo. Khi đọc report, hãy lần theo ba câu hỏi: biến

nào được dùng chung, access nào là write, và synchronization nào đáng lẽ phải tạo thứ tự giữa chúng?

Bảng 5 — Ba phần hữu ích nhất của một race report

Phần report	Nó trả lời gì	Đừng kết luận quá mức
`Read by goroutine` hoặc `Write by goroutine`	Access hiện tại nằm ở dòng nào?	Dòng đó chưa chắc là nguồn gốc policy sai.
`Previous ... by goroutine`	Access xung đột nào chồng lên nó?	Thứ tự in ra không phải timeline đầy đủ của chương trình.
`created at`	Goroutine nào mở đường cho access này?	Một report không chứng minh mọi đường chạy đều racy.

Detector là công cụ tìm bằng chứng khi code đã chạy, không phải chứng minh tuyệt đối rằng chương trình không có race. Một đường chưa được test chưa tạo memory access để detector quan sát. Vì thế `go test -race` đứng cạnh test tốt và workload có ý nghĩa; nó không thay thế chúng.

Sửa invariant, không chỉ làm detector im lặng

Trước khi thêm `sync.Mutex`, hãy nói invariant bằng một câu: `Successes` phải trả về số lần `RecordSuccess` đã hoàn tất, kể cả khi nhiều worker đang chạy. `Counter` vì thế mang một field `mu sync.Mutex` cạnh `successes`. Lock không còn là bùa chú; nó là ranh giới bảo vệ hai access vào state dùng chung:

```
func (c *Counter) RecordSuccess() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.successes++
}

func (c *Counter) Successes() int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.successes
}
```

Phiên bản này nằm trong `labs/part6-race-detector/fixed`. Test của nó chờ mọi worker bằng `sync.WaitGroup`, sau đó kiểm tra count là 128; `go test -race ./fixed` phải xanh. `WaitGroup` ở đây chỉ giúp test biết khi nào worker xong. Nó không tự bảo vệ `successes`; mutex mới làm việc đó. Phân biệt hai trách nhiệm này ngay từ đầu sẽ tránh rất nhiều "đã Wait rồi, sao vẫn race?" sau này.

`Mutex` và memory ordering sẽ được đào sâu ở phần concurrency. Ở Chương 6, bài học hẹp hơn: một test có assertion đúng vẫn có thể chưa quan sát được cách state bị chạm. Nếu requirement đưa goroutine vào code, thêm `-race` vào bằng chứng chấp nhận thay đổi là một quyết định testability, không phải một nghi thức trang trí.

Bài điều tra. Chạy fixture `broken` một lần với `-race`, rồi mở `fixed/counter_test.go`. Trước khi chạy phiên bản fixed, khoanh hai method cùng chạm `successes` và dự đoán vì sao cả getter cũng lock. Sau đó thử bỏ lock riêng trong `Successes`; test hiện tại vẫn có thể xanh vì nó đọc sau `wg.Wait`, nhưng public method đã mất contract an toàn cho caller đồng thời.

Đáp án — chỉ đọc sau khi đã tự làm. `RecordSuccess` ghi còn `Successes` đọc cùng field, nên cả hai cần đi qua cùng mutex nếu API hứa dùng được đồng thời. `wg.Wait` chỉ tạo điểm chờ trong test hiện tại; nó không thay thế synchronization trong API. Fixture broken được tag riêng để repository vẫn có suite mặc định xanh, còn lệnh race được chạy như một thí nghiệm mà failure là kết quả đúng.

Fuzz khi property đã rõ

Các subtest của config đã có tên cho những biên mà ta chủ động chọn: port `0`, host rỗng, port quá lớn. Nhưng parser nhận một `string`; không ai có thể liệt kê trước mọi tổ hợp dấu ngoặc, Unicode, dấu hai chấm hay byte lạ mà input từ environment có thể mang theo. Fuzzing giúp tạo thêm input, nhưng chỉ có giá trị khi target biết phải đòi điều gì ở mọi input đó.

Property của `LoadTargets` không phải "mọi string đều được parse". Phần lớn string phải bị từ chối. Property đúng là: nếu `LoadTargets` trả thành công, nó trả đúng một target có host không rỗng và port trong `1..65535`. Điều này ghép tốt với table test: table nói các case quan trọng nào phải bị từ chối hay được map đúng; fuzz tìm xem còn input nào làm implementation tự mâu thuẫn với invariant đó.

Đừng thay property ấy bằng "function không panic". Không panic là một hygiene check đáng có, nhưng một parser có thể bình tĩnh trả `Target{Host: "", Port: 0}` và vẫn phá command ở chỗ xa hơn. Property mạnh hơn buộc test nhìn vào kết quả thành công. Ngược lại, đừng bắt fuzz phải đoán error message hay bắt mọi input xấu rơi đúng vào một nhánh implementation; những điều đó làm target giòn mà không tăng sự hiểu biết về contract.

Trong thực tế có nhiều họ property: kết quả luôn giữ invariant, encode rồi decode trả về dữ liệu tương đương, hay hai cách biểu diễn phải đồng thuận. Ta chỉ dùng họ đầu tiên ở đây vì parser này chưa có format output để round-trip. Chọn property theo API đang có giúp fuzzing phục vụ thiết kế, thay vì ép mọi function phải có một bài test ngẫu nhiên.

```
func FuzzLoadTargetsNeverReturnsInvalidTarget(f *testing.F) {
    for _, seed := range []string{
        "",
        "billing.internal:8443",
        "[2001:db8::10]:443",
        "payments.internal:0",
        "not a target",
    } {
        f.Add(seed)
    }

    f.Fuzz(func(t *testing.T, raw string) {
        targets, err := LoadTargets(lookup(raw))
        if err != nil {
            return
        }
        if len(targets) != 1 {
            t.Fatalf("got %d targets", len(targets))
        }
        target := targets[0]
        if target.Host == "" ||
            target.Port < 1 ||
            target.Port > 65535 {
            t.Fatalf("invalid target %+v", target)
        }
    })
}
```

`f.Add` không phải danh sách đầy đủ. Nó là seed corpus có chủ đích: một default, một hostname thường, một IPv6, một range error và một chuỗi không có hình dạng target. Khi chạy `go test`, seed corpus này vẫn được chạy như test bình thường. Khi chạy với `-fuzz`, toolchain mutate corpus để tìm input tạo coverage mới. Nếu tìm được failure, Go lưu input tái lập được để lần `go test` sau không làm bug biến mất theo may rủi.

Từ `labs/part6-testable-command`, chạy một lượt có giới hạn thời gian:

```
go test -run=^$ `
  -fuzz=FuzzLoadTargetsNeverReturnsInvalidTarget `
  -fuzztime=2s ./internal/config
```

`-run=^$` bỏ qua ordinary test trong lượt fuzz riêng này; chúng vẫn phải chạy trong suite bình thường. `-fuzztime=2s` là ngân sách cho thí nghiệm local, không phải một con số chứng nhận parser an toàn. Fuzz target xanh chỉ nói rằng các input đã chạy chưa phá property; nó không thay thế review, test case có ý nghĩa hay validation giới hạn tài nguyên.

Bài thiết kế property. Một đề xuất nghe có vẻ hợp lý là "nếu `err == nil`, `target.Name` luôn là `billing`". Hãy quyết định có nên đưa nó vào fuzz target này không. Nếu ngày mai config hỗ trợ nhiều service, assertion đó sẽ làm test cản refactor dù parser vẫn đúng. Hãy giữ property gần với contract hiện tại nhất: shape hợp lệ của target, không phải một chi tiết của demo.

Đáp án — chỉ đọc sau khi đã tự làm. `Name == "billing"` là contract hiện tại của fixture nhỏ, nhưng không phải property parser cần giữ khi input model lớn hơn. Fuzz property nên đủ mạnh để bắt target lỗi và đủ hẹp để không đóng băng một thiết kế chưa được hứa. Khi fuzz tìm được failure thật, trước hết biến input đó thành một regression test dễ đọc; corpus là bằng chứng, test có tên là lời giải thích cho người đến sau.

Đo câu hỏi trước khi tối ưu câu trả lời

Sau khi thấy parser có nhiều validation, một phản xạ quen thuộc là đề nghị cache kết quả: "`net.SplitHostPort` có thể chậm". Nhưng `LoadTargets` hiện chỉ chạy khi command khởi tạo. Kể cả một benchmark cho thấy override tốn allocation, điều đó chưa biến nó thành bottleneck của một CLI chạy vài probe. Benchmark tốt bắt đầu bằng câu hỏi đủ cụ thể để kết quả có thể thay đổi một quyết định.

Ở đây câu hỏi hẹp là: một lần load default, hostname override và IPv6 override có chi phí tương đối ra sao trên máy đang chạy? Test setup nằm trước `b.Loop()`, nên timer của benchmark hiện đại không tính phần đó; `b.ReportAllocs` yêu cầu output thêm allocation:

```
func BenchmarkLoadTargets(b *testing.B) {
    tests := []struct {
        name string
        raw  string
    }{
        {name: "default"},
        {
            name: "hostname override",
            raw:  "payments.internal:9443",
        },
        {name: "IPv6 override", raw: "[2001:db8::10]:443"},
    }

    for _, tt := range tests {
        b.Run(tt.name, func(b *testing.B) {
            lookup := func(string) (string, bool) {
                return tt.raw, tt.raw != ""
            }
            b.ReportAllocs()
            for b.Loop() {
                _, _ = LoadTargets(lookup)
            }
        })
    }
}
```

Chạy từ lab với `go test -run=^$ -bench=BenchmarkLoadTargets -benchmem ./internal/config`. `-run=^$` tránh trộn ordinary test vào lượt đo; `-benchmem` cho `allocs/op` và `B/op`. Đừng copy số `ns/op` của máy này vào một README như một sự thật phổ quát. CPU, Go version, power mode và noise của hệ thống đều làm số đổi. Nếu một thay đổi performance thật sự sắp được nhận, hãy đo trước/sau trên cùng môi trường, lặp lại và ghi đúng workload mà quyết định đang phục vụ.

Trong case này, kết luận có thể là “không tối ưu”. Đó là một kết quả kỹ thuật hoàn toàn hợp lệ: config parse một lần chưa đáng đổi API, thêm cache hay làm code kém đọc. Benchmark đã làm việc của nó khi giúp ta từ chối một tối ưu không có pressure, chứ không chỉ khi nó dẫn tới một patch nhanh hơn.

Bài review. Chạy benchmark hai lần. Chỉ đề xuất tối ưu khi có workload như config reload trong hot loop hoặc profile thật; "số lớn hơn" chưa đủ.

Contract ở ranh giới process

Unit test của `app.Run` cố ý không biết terminal hay exit code. Điều đó giúp nó giữ boundary hẹp, nhưng cũng tạo một khoảng trống cần kiểm tra ở đúng nơi: command vẫn phải in success vào stdout, đưa config error sang stderr và trả mã exit 2. Đây là contract nhìn thấy được của process; đổi nó có thể làm script của operator hỏng dù các package phía trong vẫn xanh.

Gọi `main` trong test thường làm khoảng trống ấy khó kiểm chứng, vì `os.Exit` kết thúc process test. Thay vì fake cả process, ta rút phần presentation thành một function nhỏ nhận writer và trả exit code. `main` chỉ nối dependency thật vào function đó:

```
func main() {
    runner := func(context.Context, probe.Endpoint) error {
        return nil
    }
    os.Exit(run(
        context.Background(),
        os.Stdout,
        os.Stderr,
        os.LookupEnv,
        runner,
    ))
}
```

`run` vẫn gọi `app.Run`; nó chỉ thêm policy process rất nhỏ. Test vì thế có thể dùng `bytes.Buffer` làm stdout và stderr, rồi quan sát đúng ba thứ mà shell nhìn thấy: code, output thành công và output lỗi.

```
func TestRunWritesSuccessContract(t *testing.T) {
    var out, errOut bytes.Buffer
    code := run(
        context.Background(),
        &out,
        &errOut,
        func(string) (string, bool) { return "", false },
        func(
            context.Context,
            probe.Endpoint,
        ) error { return nil },
    )

    if code != 0 {
        t.Fatalf("code = %d", code)
    }
    if out.String() != "billing healthy=true\n" {
        t.Fatalf("stdout = %q", out.String())
    }
    if errOut.Len() != 0 {
        t.Fatalf("stderr = %q", errOut.String())
    }
}
```

Test này không assert `run` loop outcome thế nào hay `app.Run` parse config ra sao. Những contract đó đã thuộc package riêng. Nó chỉ giữ lời hứa của command: success đọc được bằng máy trên stdout và không rò error message.

Test phía failure cũng phải hẹp như vậy. Đưa vào một port không hợp lệ, để runner làm test fail nếu bị gọi, rồi assert exit 2, stdout rỗng và diagnostic ở stderr. Đừng so toàn bộ message: wording validation có thể được cải thiện, còn nơi xuất hiện và exit policy mới là contract vận hành ổn định.

Bài code review. Chuyển một `fmt.Fprintf` từ `out` sang `errOut` rồi quyết định đó là chỉnh presentation vô hại hay breaking change cho shell pipeline. Sau đó tưởng tượng một mode `--json`: contract test nên có case riêng cho mode được hứa rõ ràng này, không được làm contract text mode yếu đi đến mức nhận bất cứ output nào.

Đáp án — chỉ đọc sau khi đã tự làm. Trong shell pipeline, stdout thường mang dữ liệu command sau sẽ tiêu thụ; chuyển healthy result sang stderr có thể là breaking change. Test không cố đóng băng dấu câu mãi mãi. Nó bảo vệ boundary mà con người và automation dựa vào: success exit `0` và có result trên stdout; configuration failure exit 2, không in success data, đồng thời có diagnostic ở stderr.

Contract test này không thay thế một end-to-end test khởi động binary thật. Khi sau này command nhận flag, file config, signal hoặc network thật, một số luồng cần được kiểm chứng qua process thật. Nhưng dùng binary test để assert mọi dấu cách và mọi nhánh validation sẽ chậm, khó đọc và trùng với unit test. Ranh giới tốt là: logic có dependency thay thế được kiểm tra ở package bên trong; vài lời hứa mà process công bố được giữ ở `run`; chỉ một số đường đi quan trọng mới cần vượt ra integration test.

Đây là điểm dừng của mốc hiện tại: testability không đồng nghĩa phủ một lớp mock lên mọi package. Nó là khả năng đặt từng policy vào một boundary đủ nhỏ để ta tạo input, quan sát kết quả và biết chính xác thay đổi nào đang được bảo vệ.

Kết quả xanh chưa trả lời câu hỏi nào?

Với các lab vừa chạy, kết quả xanh nghĩa là những assertion đã được thỏa trên các input và lịch chạy được thực thi. Assertion thiếu vẫn có thể bỏ lọt lỗi trên chính input ấy. Fuzzing mở rộng tập input theo ngân sách và feedback coverage; số lượt chạy không cố định, và fuzzer không tự đặt thêm một property mà tác giả chưa viết. Khi có crasher, giữ corpus cùng regression test có tên để người sửa sau biết contract đã từng bị phá ở đâu.

Benchmark lại trả lời câu hỏi khác. `B.Loop` có từ Go 1.24; trong Go 1.27.1, lần gọi đầu reset timer và lần trả false dừng timer. Với đúng dạng `for b.Loop() { ... }`, compiler giữ sống các argument, kết quả lời gọi và biến được gán trong thân vòng theo cơ chế mô tả ở `testing/benchmark.go`. Không suy rộng cơ chế ấy sang một wrapper tùy ý hoặc xem nó là bảo đảm workload đại diện cho production. Mẫu `b.N` vẫn hợp lệ; nếu dùng nó, loại setup khỏi timer khi cần và bảo đảm phép tính cần

đo không bị bỏ vì kết quả không được dùng. Không trộn cả hai kiểu vòng lặp trong một benchmark.

Race detector cũng có chi phí riêng. Tài liệu chính thức nêu mức thường gặp khoảng 5–10 lần bộ nhớ và 2–20 lần thời gian chạy, tùy chương trình; đây không phải số đo của các lab trong sách. Detector quan sát access thực sự chạy, nên một đường chưa thực thi không thể tạo report cho đường ấy. Độ trễ mạng có thể làm lộ lịch chạy mới, nhưng không có nghĩa detector không dùng được với workload mạng. Muốn tăng khả năng phát hiện, chọn scenario có ý nghĩa và chạy đủ đường liên quan, không chỉ lặp một happy path.

`go test -cover` báo statement coverage theo instrumentation của Go, không phải tỷ lệ mọi dòng source và cũng không phải branch hay path coverage. Dù đạt 100%, nó chưa chứng minh mọi tổ hợp điều kiện, input hay tương tác đồng thời đều đúng. Ví dụ một `if` đã được đi qua với điều kiện true vẫn có thể chưa thử nhánh false. Khi đọc report, hãy hỏi assertion nào bảo vệ policy, đường nào chưa chạy và công cụ nào có thể quan sát loại lỗi đó. Đó là giới hạn bằng chứng cụ thể, không phải lý do bỏ kiểm thử.

Chương 7 — Dữ liệu đi vào và đi ra

Một lệnh vừa in ra một dòng `billing healthy=true` có vẻ như đã giao tiếp được với thế giới bên ngoài. Nhưng terminal chỉ là một `io.Writer` rất đặc biệt. Khi dữ liệu vào chuyển thành tệp cấu hình, pipe từ lệnh khác, body HTTP hay một stream nén, trực giác "đọc file rồi có dữ liệu" bắt đầu che một sự thật quan trọng: dữ liệu không nhất thiết đến cùng lúc, và tài nguyên không tự hết hạn đúng lúc ta muốn.

Mô hình tinh thần của chương này là **stream cung cấp byte theo từng lượt, không phải một slice đã có đủ dữ liệu**. Caller phải xử lý byte nhận được, xác nhận điểm kết thúc và biết ai chịu trách nhiệm đóng tài nguyên. `io.Reader` không tự hứa một deadline hay tiến độ trong thời gian hữu hạn; những chính sách ấy thuộc nguồn dữ liệu và application.

Một lần `Read` không có nghĩa là toàn bộ input

Hãy bắt đầu với một lỗi nhỏ hơn `opsprobe`. Một cấu hình JSON trông như thế này:

```
[
  {
    "name": "billing",
    "endpoint": "https://billing.internal/health"
  }
]
```

Một cách nghĩ sai nhưng rất tự nhiên là cấp buffer đủ lớn cho mẫu test, gọi `Read` một lần, rồi phân tích số byte nhận được. Nó thường xanh với `strings.NewReader` và input ngắn. Nhưng `io.Reader` không hứa lấp đầy buffer. Reader có thể là file, socket, pipe hay decoder khác; mỗi lần gọi chỉ nhận được một phần byte hiện có.

Contract tối thiểu của `Read(p)` là: nó trả `n` byte nằm trong `p[:n]`, và có thể trả một error. `n` có thể nhỏ hơn `len(p)`. Đặc biệt, một reader được phép trả byte hữu ích cùng `io.EOF` trong một lượt; caller phải tiêu thụ `p[:n]` trước

khi xử lý error. `io.EOF` nghĩa là không còn byte nào cho stream đó, không phải là "lần gọi trước không có dữ liệu".

Bảng 6 — Cách đọc kết quả `Read`

Kết quả	Caller phải làm gì	Diễn giải đúng
`n > 0`, `err == nil`	Xử lý `p[:n]`, rồi đọc tiếp	Stream đang tiến lên nhưng chưa kết thúc.
`n > 0`, `err == io.EOF`	Vẫn xử lý `p[:n]`, rồi kết thúc	Lượt cuối vừa có dữ liệu vừa báo hết stream.
`n == 0`, `err == io.EOF`	Kết thúc	Không còn byte để tiêu thụ.
`n == 0`, error khác `nil`	Trả hoặc phân loại lỗi	Lượt này không trả byte và có lỗi.
`n == 0`, `err == nil`	Không coi là EOF; tránh vòng busy-loop vô hạn	Không có tiến độ trong lượt này; contract không khuyến khích kết quả ấy với buffer không rỗng nhưng caller vẫn phải phân biệt nó.

Đừng tự viết vòng `Read` chỉ để chứng minh đã hiểu contract nếu standard library đã có primitive đúng. `io.ReadAll` là lựa chọn hợp lý khi input có giới hạn kích thước đáng tin và việc giữ toàn bộ trong memory là chấp nhận được. Khi input có thể lớn hoặc vô hạn, thiết kế phải chuyển sang xử lý dần từng phần. Khác biệt này không phải micro-optimization: nó quyết định chương trình có thể bị ép allocate bao nhiêu memory bởi input không tin cậy.

Giới hạn byte trước khi gọi parser

"Tệp cấu hình chỉ nhỏ" không phải là một giới hạn. Nếu chương trình nhận một đường dẫn hoặc reader từ bên ngoài, nó cần đặt giới hạn ngay tại ranh giới mình kiểm soát. Chỉ đặt giới hạn sau `json.Decode` là quá muộn: bộ phân tích đã có quyền đọc bất cứ lượng byte nào trước khi chính sách được áp dụng.

Trước khi mở `bounded.go`, chỉ đọc hai test `TestReadBounded...`. Chúng đòi `ReadBounded` nhận tối đa `max` byte; input lớn hơn đúng một byte phải trả error mà caller nhận diện được bằng `errors.Is`. Hãy tự viết phần thân

theo ba bước: bọc reader để chỉ cho phép tối đa `max + 1` byte, đọc phần đã bọc, rồi so độ dài với `max`. Byte thứ `max + 1` không phải dữ liệu cần giữ; nó là bằng chứng phân biệt tệp vừa chạm ngưỡng với tệp thật sự vượt ngưỡng.

Lab đầy đủ còn từ chối `max < 0` và thêm ngữ cảnh cho error. Những chi tiết đó cần có trong code chạy thật, nhưng chúng không được che câu hỏi đang học: vì sao phải xin thêm đúng một byte.

`io.LimitReader` chỉ giới hạn số byte mà reader bọc ngoài sẽ trả; nó không tự nói JSON có hợp lệ hay không, và cũng không làm input nhỏ trở nên đáng tin. Vì thế giới hạn byte là một chính sách tài nguyên riêng; `Decode` vẫn chịu trách nhiệm cho cấu trúc JSON và số tài liệu. Hai test ngắn bảo vệ hai câu hỏi khác nhau, không gộp chúng thành một assertion mơ hồ kiểu "cấu hình lỗi".

Đây cũng là điểm cần tránh một lời hứa sai về cancellation. `io.Reader` chỉ công bố `Read`; interface không mang `context.Context`. Một wrapper như `ReadBounded` có thể dừng sau khi nhận đủ byte, nhưng không tự làm một underlying reader đang block thức dậy. Với file local, điều đó thường không phải policy cần thêm ở đây. Với body HTTP, deadline và cancellation thuộc lifecycle request sẽ được dạy lại ở chương networking, nơi ta nhìn được owner của connection và cách transport phản ứng.

Failing test trước parser

Lab `labs/part7-stream-boundaries` không gắn ngay tệp JSON vào `opsprobe`. Project xuyên suốt hiện chưa cần tệp cấu hình; nhét nó vào lúc này chỉ khiến trực giác mới bị lẫn với chính sách endpoint cũ. Ta dùng một chương trình tái hiện tối thiểu để thấy ranh giới stream trước.

Mở `targets/targets_test.go` và chỉ đọc `TestDecodeConsumesChunkedStream`. `chunkReader` cố tình chỉ nhả ba byte trong mỗi lần gọi. Trước khi xem `Decode`, hãy trả lời: nếu implementation giả định một lượt `Read` là đủ, test sẽ mất đoạn nào của document? Sau đó tạm đổi tên `Decode`, chạy test và tự dựng lại function theo contract mà test đòi.

Function kết thúc có thể nhỏ:

```
func Decode(r io.Reader) ([]Target, error) {
    dec := json.NewDecoder(r)
    var targets []Target
    if err := dec.Decode(&targets); err != nil {
        return nil, fmt.Errorf(
            "decode target document: %w", err,
        )
    }

    var extra any
    if err := dec.Decode(&extra); !errors.Is(err, io.EOF) {
        if err == nil {
            return nil, errors.New(
                "target config must contain one JSON value",
            )
        }
        return nil, fmt.Errorf(
            "read after target document: %w", err,
        )
    }
    return targets, nil
}
```

`json.Decoder` nhận một `io.Reader`, nên nó tự tiếp tục đọc qua nhiều chunk; caller không cần giả vờ socket hay file là slice. Test thứ hai đặt thêm một JSON value sau document đầu. Chỉ `Decode` một lần là chưa đủ contract: program sẽ im lặng bỏ qua dữ liệu còn lại. Lần `Decode` thứ hai không phải để lấy config mới, mà để chứng minh sau document hợp lệ chỉ còn whitespace và EOF.

Điều này không biến JSON decoder thành lời giải cho mọi format. CSV có record và quoting riêng; line-oriented log có boundary theo newline; body HTTP có lifecycle mạng và limit khác. Chúng sẽ được dạy ở context phù hợp. Ở đây chỉ giữ một câu hỏi trung tâm: input kết thúc ở đâu, và code nào đã xác nhận điều đó?

Đủ một document chưa có nghĩa mọi decoder hiểu giống nhau

Input `{"port":80,"port":443}` là một document nhưng có tên trùng. API `encoding/json` mà lab dùng vẫn giữ semantics v1: với field số trong ví dụ này, giá trị sau ghi đè giá trị trước. Go 1.27 đưa `encoding/json/v2` và `encoding/json/jsontext` thành API thông thường; mặc định v2 từ chối tên trùng và UTF-8 không hợp lệ. API v1 tiếp tục được hỗ trợ, dù implementation phía dưới đổi và wording lỗi có thể khác. Không đổi import rồi coi đó là migration tương đương.

Trước khi chạy `TestJSONVersionBoundary` trong `labs/edition-contracts`, hãy dự đoán decoder nào chấp nhận document trên và `port` cuối cùng là bao nhiêu. Test chỉ kiểm tra hai trường hợp tên trùng và UTF-8 sai; không chứng minh decoder là bộ kiểm định toàn bộ policy cấu hình. Giới hạn byte, đúng một document, schema được phép và semantics chọn API là bốn câu hỏi riêng.

Đóng output cũng là một phần của result

Khi đọc file, `defer file.Close()` gần acquisition giúp không rò file descriptor. Khi ghi file, `Close` còn có thể là nơi buffer được flush và failure cuối cùng lộ ra. Nếu function ghi JSON chỉ trả error của `Encode`, caller có thể báo thành công trước khi biết output có được đóng hoàn chỉnh hay không.

Lab đầy đủ mở resource rồi đặt `defer Close` ngay sau acquisition. Để nhìn riêng policy chọn result, phần lõi có thể viết ngắn hơn:

```
func save(w io.WriteCloser, targets []Target) error {
    writeErr := Encode(w, targets)
    closeErr := w.Close()
    if writeErr != nil {
        return writeErr
    }
    return closeErr
}
```

Lab đầy đủ vẫn đóng tài nguyên trên mọi đường return. Đoạn trên chỉ làm policy lộ ra: giữ write error trước, rồi mới trả close error. Writer giả giữ contract ấy mà không cần filesystem thật.

Ranh giới Dữ liệu: Short Write, Bộ đệm và Ngộ nhận về Syscall

Khi làm việc với các dòng nhập xuất dữ liệu, trực giác bề mặt thường dẫn tới hai ngộ nhận tai hại: đánh đồng hành vi của Reader với Writer, và cho rằng mỗi thao tác đọc ghi đều kích hoạt một lời gọi hệ thống xuống nhân hệ điều hành.

Ràng buộc Khắt khe của `io.Writer` và Khái niệm Short Write

Nếu như `io.Reader` cho phép trả về số byte đọc được `n < len(p)` kèm theo `err == nil` trong các tình huống bình thường (chẳng hạn khi đường ống mạng chưa nhận đủ dữ liệu), thì giao diện `io.Writer` lại áp đặt một hợp đồng nghiêm ngặt hơn nhiều. Theo đặc tả của thư viện chuẩn Go, một hàm `Write(p)` bắt buộc phải trả về một lỗi khác `nil` nếu nó không thể ghi trọn vẹn toàn bộ lát cắt dữ liệu (`n < len(p)`).

`io.ErrShortWrite` là sentinel mô tả một lượt ghi ngắn không có error thích hợp. Writer có thể trả error cụ thể khác; contract không bắt mọi short write mang chính sentinel này. Nếu writer trả `n < len(p)` và error `nil`, nó vi phạm contract API của `io.Writer`, không phải một quy tắc ngôn ngữ. Caller dùng API ấy cần kiểm tra kết quả, không suy ra rằng byte đã tới đích cuối chỉ vì một lượt ghi ở tầng trung gian thành công.

Bộ đệm User-space với `bufio`

Một syscall đưa việc thực thi qua ranh giới user/kernel. Nó có chi phí, nhưng không đồng nghĩa với context switch của scheduler sang thread khác; việc block và chuyển lịch còn phụ thuộc operation và trạng thái tài nguyên. Ghi từng byte qua file chưa đệm thường tăng số lượt I/O so với gom nhiều byte. Không thể suy ra một số syscall cố định hay mức giảm throughput chỉ từ số lần gọi interface `Write`; phải xác định concrete writer và đo đường chạy trên OS cụ thể.

`bufio.Reader` và `bufio.Writer` gom dữ liệu trong đệm user-space. Trong source Go 1.27.1, constructor mặc định dùng buffer 4096 byte; con số ấy không phải quy tắc của `io.Reader` hay cam kết mọi buffer đều cùng kích thước. Writer chuyển byte xuống underlying writer khi cần chỗ hoặc khi Flush; caller phải kiểm tra lỗi Flush. Buffer có thể giảm số lượt gọi tầng dưới, không bảo đảm một tỷ lệ syscall hay throughput cố định.

Lời gọi `Read` không đồng nghĩa với Syscall

Một ngộ nhận kỹ thuật phổ biến là mặc định rằng mọi lệnh `r.Read(p)` trong mã nguồn Go đều tương ứng với một chỉ thị CPU `SYSCALL` trực tiếp xuống kernel.

Đường đi qua user-space hay syscall phụ thuộc implementation cụ thể, OS và trạng thái operation. Bảng dưới tách những trường hợp thường gặp:

Kiểu cụ thể của Reader	Bản chất cơ chế khi gọi `Read(p)`	Có phát sinh Syscall xuống OS không?
`*bytes.Buffer` hoặc `*strings.Reader`	Sao chép byte vào buffer của caller từ dữ liệu trong bộ nhớ.	Không cần syscall I/O cho phần sao chép; cách compiler hiện thực `copy` không phải contract của Reader.
`*bufio.Reader` (khi còn dữ liệu trong đệm)	`Read` sao chép byte từ đệm vào buffer của caller; khác với các API trả một slice nhìn vào đệm.	Không cần gọi underlying Reader trong lượt được phục vụ đủ từ đệm.
`*bufio.Reader` (khi bộ đệm đã cạn)	Gọi `Read` lên đối tượng `io.Reader` bên dưới để nạp lại bộ đệm nội bộ.	Phụ thuộc underlying Reader. Nếu bọc in-memory buffer thì không có syscall; nếu bọc file hoặc socket thì do reader bên dưới quyết định.
`*os.File` chưa đệm	Gửi yêu cầu I/O trực tiếp tới kernel qua bảng mô tả tệp (OS file I/O path).	Có. Thực hiện syscall đọc tệp của hệ điều hành (`read` trên Linux hoặc `ReadFile` trên Windows).
`*net.TCPConn` trong đường đọc Linux đã pin bên dưới	Thử non-blocking I/O; kết hợp poller nếu chưa sẵn sàng.	Có OS I/O; không có quan hệ một-một giữa số lần gọi `Read` và syscall. Windows dùng đường completion khác.

Với Go 1.27.1 trên Linux, đường đọc socket trong `internal/poll/fd_unix.go` có thể thử non-blocking read, gặp `EAGAIN`,

chờ poller rồi thử lại. Đây là mô hình triển khai được pin, không phải lời hứa của `net.Conn`. Không áp nguyên trace readiness Linux cho Windows: backend IOCP dùng completion và đường triển khai khác. API chung vẫn là đọc byte, error và deadline; poller giúp điều phối goroutine chứ không loại bỏ I/O xuống OS. Chương 11 sẽ đặt cơ chế đó vào một request thật.

Một filesystem là một khả năng, không phải một đường dẫn

Parser ở trên chỉ cần byte, không cần quyền mở mọi file trên máy. Khi một application có nhiều nguồn file, `fs.FS` mô tả khả năng `Open(name)` với tên logic dùng dấu `/`. Consumer có thể nhận `embed.FS` chứa asset lúc build, `os.DirFS` nhìn vào directory lúc chạy, hoặc filesystem giả trong test. Consumer không cần đổi parser theo nguồn.

```
// Đặt trong package có thư mục fixtures tại build time.
//go:embed fixtures/*.txt
var files embed.FS

data, err := fs.ReadFile(files, "fixtures/message.txt")
```

Imports của đoạn này là `embed` và `io/fs`; bản chạy được nằm trong `labs/edition-contracts`. Directive phải ở package scope, gắn với variable phù hợp. Byte được đưa vào binary lúc build, không tự cập nhật khi file ngoài máy đổi. `embed.FS` có API đọc, không phải nơi lưu runtime secret hoặc config cần sửa sau triển khai. Những byte được bundle có thể bị trích xuất từ binary; “không còn file rời” không phải bảo mật.

`fs.ValidPath` không cho component `..` hoặc path bắt đầu bằng `/`; tên như `a..b` vẫn có thể hợp lệ, còn `.` là trường hợp đặc biệt chỉ root. Path hợp lệ không tự tạo sandbox. `os.DirFS` và `fs.Sub` không ngăn symlink bên dưới trỏ ra ngoài directory. Nếu nhận tên từ bên không tin cậy và cần confinement, đọc contract `os.Root` cùng giới hạn theo OS trong Go 1.27.1 thay vì tin `Clean` chặn mọi lối thoát. Quyền mở file và quyền parse dữ liệu là hai boundary khác nhau.

Thực hành. Viết một function nhận `fs.FS` và tên logic, đọc một tệp rồi đưa byte cho parser đã có. Test với filesystem giả; test riêng tên không hợp lệ và file thiếu. Sau đó thay bằng `embed.FS` mà không đổi parser. Đừng để test filesystem giả được diễn giải thành bằng chứng rằng symlink hoặc quyền truy cập thật trên OS đã an toàn.

Parser cũng có ngân sách

Một log "mỗi dòng là một record" khiến `bufio.Scanner` tiện hơn vòng đọc thủ công. Nhưng Scanner có giới hạn token: một dòng dài có thể khiến `Scan()` trả false trước EOF. Caller phải đọc `Err()`, và nếu domain cho phép dòng lớn hơn thì gọi `Buffer` trước khi scan, với mức trần phù hợp. Không nên tăng trần vô hạn để chữa một input hỏng. `TestScannerLimitAndValidation` dùng cùng input với hai giới hạn để phân biệt kết thúc bình thường với token vượt ngân sách.

Byte limit cũng không thay kiểm tra miền số. `strconv.ParseUint(text, 10, 16)` trả error nếu chuỗi không phải số nguyên unsigned biểu diễn được trong 16 bit; sau đó business rule vẫn phải quyết định port 0 có được phép không. `Atoi` chọn `int` của kiến trúc, không định nghĩa một wire format ổn định. Khi format yêu cầu bit width, nói rõ width trong parser và test biên.

Với pattern do người dùng nhập, dùng `regexp.Compile` và xử lý error. `MustCompile` phù hợp với pattern cố định trong source mà lỗi là lỗi lập trình, không phải cách báo lỗi cho người vận hành. Package `regexp` cam kết thời gian match tuyến tính theo độ dài input; điều đó không cam kết chi phí compile, lượng memory hoặc số lượt match không cần giới hạn. Parser, số record, pattern và lifetime của stream vẫn phải có ngân sách của application. Chọn `strings` hay `bytes` khi thao tác chỉ là tìm delimiter hoặc prefix rõ ràng; thêm regexp chỉ khi grammar cần nó.

Tự kiểm tra trước khi tích hợp

Chạy từ `labs/part7-stream-boundaries`:

```
go test ./...
go vet ./...
```

Thử lần lượt: bỏ lượt `Decode` thứ hai; cho `chunkReader` nhả một byte; rồi cho writer giả fail ở `Write` trước `Close`. Dự đoán test nào vỡ.

Đáp án — chỉ đọc sau khi đã tự làm. Bỏ lượt decode thứ hai làm `TestDecodeRejectsAdditionalDocument` thất bại; chunk nhỏ hơn không đổi contract. Khi write và close cùng fail, phải giữ write error.

Với limit, input dài đúng `max` byte được trả nguyên vẹn; input dài `max + 1` trả error giữ identity `ErrDocumentTooLarge`. Đó là lý do lab không dùng một `if len(data) == max` mơ hồ: bằng chứng cần phân biệt đúng ngưỡng với vượt ngưỡng.

Chương này không thêm file config vào `opsprobe`: chưa có yêu cầu vận hành nào bắt nó phải có. Đó là giữ teaching vehicle phục vụ bài học. Khi một yêu cầu thật cần import/export target, stream boundary và close policy ở lab này sẽ là nền để tích hợp có chủ đích.

Trước khi chạm một API I/O mới, hãy tự hỏi ba câu. Byte đến theo từng lượt nào, đâu là dấu kết thúc được chấp nhận, và ai quan sát failure của resource sau cùng? Ba câu này không thay thế tài liệu của format hay protocol cụ thể, nhưng chúng ngăn một sai lầm rất phổ biến: coi input/output là vài dòng plumbing nằm ngoài contract của chương trình.

Phần tiếp theo sẽ đặt một áp lực khác lên chương trình: nhiều goroutine cùng sống trong một process. Trước khi chọn channel hay mutex, ta cần nhìn một race không phải như một câu thần chú về thread safety, mà như hai access không có thứ tự an toàn.

1. Go Team. Go 1.27 Release Notes, encoding/json/v2 và encoding/json/jsontext. go.dev/doc/go1.27
2. Go Team. Package encoding/json: khác biệt semantics v1/v2. pkg.go.dev/encoding/json

Chương 8 — Một race bắt đầu từ đâu

Một chương trình kiểm tra nhiều endpoint thường có một người thu thập kết quả và một người trình bày. Bản chạy thử có thể luôn in đúng trên laptop, nên ta rất dễ gọi nó là "chạy song song được". Nhưng kết quả đúng trong một lượt không tạo ra một lời hứa đúng cho lượt kế tiếp.

Mô hình tinh thần của chương này là **race không phải là hai goroutine cùng tồn tại; race là các lần chạm vào cùng dữ liệu mà không có thứ tự an toàn giữa chúng.** Trước khi chọn channel, mutex hay atomic, ta phải lần được những lần truy cập đó và hỏi: thứ tự nào thực sự được thiết lập?

Một lá cờ không tự công bố kết quả

Hãy tưởng tượng collector chuẩn bị một báo cáo, rồi đặt cờ `ready` để người in biết đã xong:

```
type summary struct {
    completed int
    last      string
}

var latest summary
var ready bool

func collect() {
    latest = summary{completed: 1, last: "billing"}
    ready = true
}

func show() summary {
    for !ready {
    }
    return latest
}
```

Nguồn trông có một thứ tự hiển nhiên: gán `latest`, rồi mới gán `ready`. Nhưng đó chỉ là thứ tự **bên trong** `collect`. `show` là một luồng thực thi khác.

Nó vừa đọc `ready` khi `collect` có thể ghi `ready`, vừa đọc `latest` khi `collect` có thể ghi `latest`. Không có thao tác đồng bộ nào nối hai bên.

Vì vậy ở đây không chỉ có một vấn đề "đôi khi in cũ". Có hai cặp truy cập tranh chấp: đọc/ghi `ready` và đọc/ghi `latest`. Việc vòng lặp có tình cờ kết thúc ở một máy không biến các cặp ấy thành có thứ tự. Đặc biệt, nhìn thấy `ready == true` không là bằng chứng ngôn ngữ rằng người đọc cũng nhìn thấy bản `latest` đã chuẩn bị xong.

Bốn tầng không được trộn lẫn

Bảng 7 — Bốn loại bằng chứng khi điều tra race

Tầng	Câu hỏi nó trả lời	Không được suy ra thêm
Nghĩa của ngôn ngữ	Hai lần truy cập thường, cùng vị trí, có ít nhất một lần ghi và không có quan hệ happens-before là race.	Bộ chạy của máy này sẽ xen kẽ ở đúng chỗ nào.
Hành vi được tài liệu cam kết	Lệnh `go` khởi động goroutine mới; nhưng goroutine kết thúc không tự tạo quan hệ đồng bộ cho phần còn lại của chương trình.	Một cờ bool tự công bố mọi lần ghi xảy ra trước nó.
Chi tiết hiện thực	Bộ lập lịch, số P và thời điểm ngắt để chuyển việc quyết định một lần chạy thực tế xen kẽ ra sao.	Đó là hợp đồng ổn định của Go hay của code.
Hành vi đo được	`go test -race` báo những lần truy cập mà lượt chạy đã thực sự đi qua.	Không có báo cáo chứng minh mọi đường chạy đều không có race.

Hai tầng đầu là lời hứa để thiết kế. Tầng thứ ba chỉ hữu ích khi giải thích một vết chạy hoặc một vấn đề hiệu năng; đừng dùng nó để biện minh rằng race "khó xảy ra". Tầng cuối là bằng chứng thực nghiệm mạnh cho đường đã chạy, nhưng nó không thay thế lập luận về thứ tự. Một test xanh không cho phép ta đổi "chưa quan sát thấy" thành "được bảo đảm".

Vẽ chứng minh thay vì đoán lịch chạy

Chương 6 đã dùng race detector để tìm một lần truy cập thực sự xung đột. Ở đây bước kế tiếp không phải chạy lại cùng báo cáo, mà là viết được lời giải thích cho phiên bản đúng trước khi chạy. Ta phân tích trật tự bộ nhớ thông qua ba quan hệ định chuẩn được xác định trong Go Memory Model:

Quan hệ thứ tự	Bản chất ngữ nghĩa trong Go	Ví dụ áp dụng thực tế
Sequenced-before	Trật tự thực thi tuần tự của các câu lệnh bên trong cùng một goroutine đơn lẻ.	Thao tác `append` vào slice bắt buộc phải hoàn tất trước khi gọi `wg.Done()`.
Synchronized-before	Cạnh đồng bộ hóa liên goroutine do ngôn ngữ hoặc thư viện runtime công bố.	Lệnh `mu.Unlock()` được đồng bộ hóa trước lệnh `mu.Lock()` diễn ra kế tiếp trên cùng một Mutex; hoặc `wg.Done()` trước lệnh `wg.Wait()` mở khóa.
Happens-before	Quan hệ bao đóng bắc cầu giữa sequenced-before và synchronized-before.	Khi các access được sắp thứ tự và không có race trên vị trí ấy, một read quan sát write gần nhất đứng trước nó trong thứ tự này; cần xét cả những write khác, không chỉ vẽ một cạnh từ write mong muốn.

Cần phân biệt rạch ròi hai cặp khái niệm thường bị nhầm lẫn trong môi trường đồng thời:

Concurrency là cấu trúc nhiều phần việc có thể tiến triển trong các khoảng thời gian chồng nhau; parallelism là thực thi cùng lúc. Parallelism không bắt buộc mỗi dòng chạy trên một core vật lý riêng, vì logical CPU cũng có thể là hardware thread. Trong Go, số P qua GOMAXPROCS giới hạn thực thi Go code đồng thời; thread và việc OS lập lịch còn ảnh hưởng hành vi quan sát.

Data race là các access vào cùng memory location không được sắp thứ tự bởi happens-before, có ít nhất một access ghi và không được bao phủ bởi contract atomic tương ứng. Không gọi Go data race là undefined behavior không giới hạn kiểu C/C++: Go memory model đặt các hạn chế cho implementation, dù race vẫn là lỗi và race trên dữ liệu nhiều word có thể gây memory corruption. Race condition là lỗi thứ tự ở mức logic; nó có thể tồn tại dù mỗi access đã được bảo vệ bằng mutex.

Đừng biến ba tên thành câu thần chú. Với mỗi kết quả cần đọc, hãy vẽ đường cụ thể. Trong ví dụ `ready`, không có cạnh nào từ `latest = ...` sang việc `show` đọc `latest`; vì vậy lời giải không thể chỉ là thêm một vòng lặp chờ.

Một khóa bảo vệ một bất biến, không bảo vệ một dòng

Một `Mutex` hợp với dữ liệu thật sự được sở hữu chung, khi nhiều goroutine cần lần lượt sửa hoặc chụp lại cùng một trạng thái. Bất biến của lab mới là: **sau khi người điều phối đã chờ tất cả worker hoàn tất, snapshot chứa đúng một report của mỗi worker.** Mutex bảo vệ slice `reports` trong lúc thêm và sao chép; `WaitGroup` cho người điều phối biết khi nào có thể đánh giá bất biến.

Trước khi mở `fixed/registry.go`, chỉ đọc `exercise/registry_test.go`. Test đang đòi một `Registry`, một `Add(Report)` và một `Snapshot() []Report`. Hãy tự quyết định field nào cần có, method nào dùng pointer receiver, và vùng lock cần bao trùm đến đâu. Lệnh khởi đầu cố ý đỏ vì phần hiện thực chưa tồn tại:

```
go test -tags exercise ./exercise
```

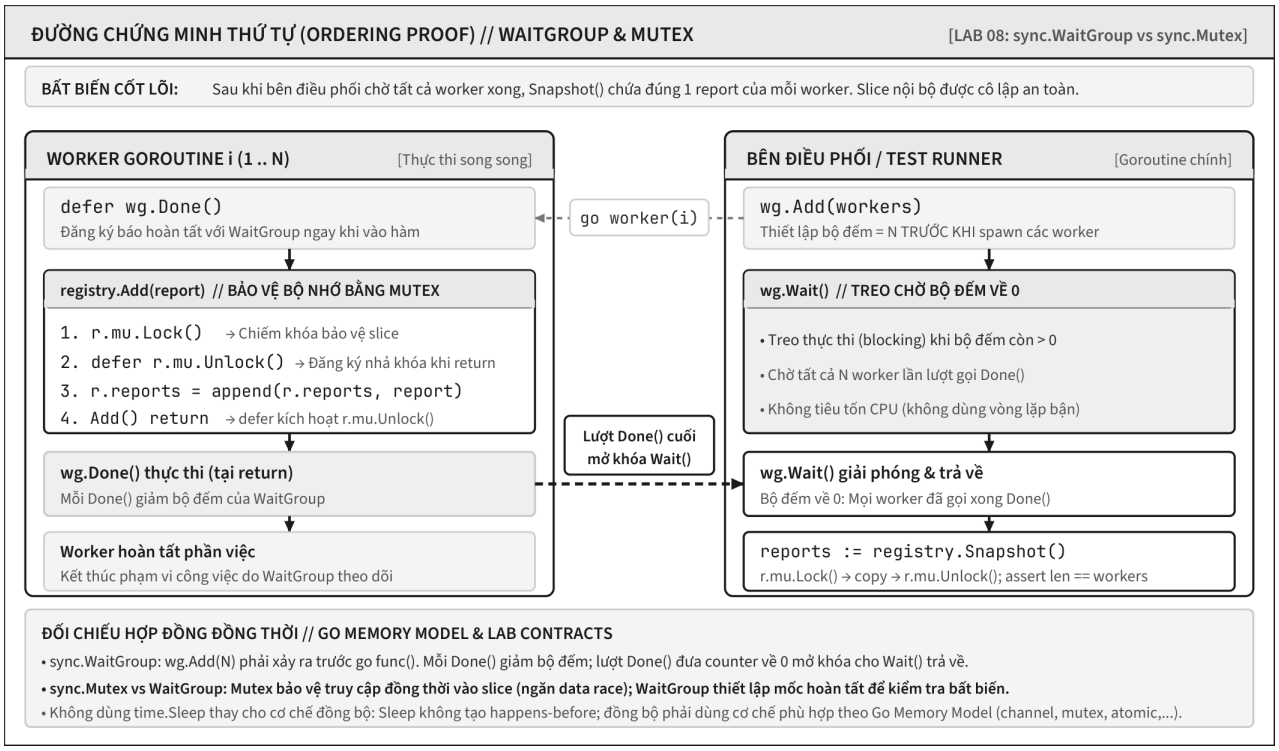
Đừng đáp bằng cách trả slice nội bộ. Bài cũ về aliasing đã cho ta lý do: người gọi giữ slice trả về có thể thay phần tử của nó. `Snapshot` ở đây hứa một ảnh chụp của container, nên lab dùng `copy` để người gọi không ghi đè phần tử bên trong registry. `Report` chỉ có string value để câu hỏi của bài vẫn là thứ tự; quyền sở hữu sâu hơn của object lồng nhau sẽ quay lại khi nó có nhu cầu thật.

Một phần hiện thực được chấp nhận không cần biết bộ lập lịch sẽ chạy worker nào trước. Nó chỉ cần khiến mỗi `Add` lock trước khi append, unlock sau đó, và `Snapshot` lock trong lúc tạo bản sao. Tài liệu `sync` cam kết `Unlock` xảy ra trước một `Lock` về sau trên cùng mutex. Đây là lời hứa API, không phải suy luận từ số core hay một lần test xanh.

WaitGroup tạo mốc hoàn tất, không thay Mutex

Test đặt `wg.Add(workers)` trước khi tạo goroutine. Mỗi worker gọi `registry.Add(report)` rồi mới `wg.Done()` qua `defer`. Khi `wg.Wait()` trả về, mỗi `Done` đã mở khóa nó; do đó mọi `Add` trước `Done` đã nằm trước thời điểm assertion gọi `Snapshot`.

Đường chứng minh cho một worker có thể đọc thành lời:



Hình 20 — Đường chứng minh cho một worker: các bước trong từng goroutine giữ thứ tự code; cạnh đứt là quan hệ Done mở khóa lượt Wait này. Snapshot chỉ chạy sau khi Wait trả về.

Các mũi tên liền là thứ tự trong code; `Done → Wait returns` là điều `sync.WaitGroup` công bố. Mutex giữ các access vào slice không chồng lên nhau. `WaitGroup` không thay thế lock: nếu các worker cùng append vào một slice mà không khóa, chúng vẫn race dù main có chờ đến khi tất cả xong mới đếm kết quả.

Sau khi tự làm, chạy bản tham chiếu độc lập:

```
go test -race ./fixed
```

Kết quả xanh ở đây là hai loại bằng chứng cùng lúc: test kiểm tra bất biến cụ thể, còn race detector không thấy lần truy cập racy trên đường chạy của test. Nó vẫn không phải lời khẳng định rằng mọi API tương lai thêm vào `Registry` đều an toàn. Bất cứ method mới nào đọc hoặc ghi `reports` đều phải quay lại bất biến và phạm vi lock.

Điều `go` cho phép, điều nó không hứa

Lệnh `go f()` thiết lập một quan hệ từ nơi khởi động đến lúc `f` bắt đầu. Điều đó đủ để goroutine mới thấy dữ liệu đã được chuẩn bị trước lệnh `go`; nó không cho chiều ngược lại. Việc `f` return hay goroutine kết thúc không tự báo cho goroutine khác rằng công việc đã xong. Đó là lý do `WaitGroup` trong lab là một phần của chứng minh, không phải nghi thức để test bớt flake.

Không dùng `time.Sleep` để thay mũi tên còn thiếu. Sleep chỉ đoán rằng worker có đủ thời gian trên một lần chạy; nó không công bố thứ tự nào và làm test vừa chậm vừa không chắc. Channel ở Chương 9 giải quyết bàn giao công việc và vòng đời; trước đó, ta cần phân biệt công bố state với việc bảo vệ state đang tiếp tục bị sửa.

Công bố state và chờ điều kiện

`sync.Once` giữ một hợp đồng "thực hiện một lần", không phải "thử cho tới thành công". Khi nhiều caller gọi `Do(f)` trên cùng Once, return của `f` xảy ra trước return của các lời gọi `Do` liên quan. Nhưng nếu `f` panic, Once vẫn coi lần ấy đã được thực hiện; caller recover rồi gọi `Do` lại không khiến `f` chạy lại. `TestOncePanicIsNotRetry` trong `labs/edition-contracts` giữ phản ví dụ này. Khởi tạo phụ thuộc có thể lỗi tạm thời cần policy retry tường minh; không giấu nó trong Once rồi hy vọng lần sau tự chữa.

`sync.Cond` lại dùng cho một câu hỏi khác: chờ cho bất biến được bảo vệ bởi lock trở thành đúng. `Wait` nhả lock, chờ rồi lấy lock lại trước khi return. Notification không phải một token trạng thái để cất cho caller tương lai; predicate mới là state cần đọc dưới lock:

```
mu.Lock()
for !ready {
    cond.Wait()
}
// Đọc state được bảo vệ khi vẫn giữ lock.
mu.Unlock()
```

Vòng `for` không nói rằng Go cho phép Wait tự tỉnh vô cớ: tài liệu yêu cầu Signal hoặc Broadcast để Wait return. Nó cần thiết vì khi goroutine lấy lại lock, goroutine khác có thể đã làm predicate không còn đúng. Producer sửa predicate với cùng lock, rồi thông báo theo policy. Nếu producer đã làm `ready=true` trước khi consumer tới, consumer đọc predicate và không cần một notification lịch sử. Đây là nền để đọc queue ở Chương 21, không phải một lý do dùng Cond thay channel cho mọi công việc.

Atomic publication phù hợp khi reader lấy một snapshot đã hoàn tất, còn writer thay toàn bộ snapshot thay vì sửa object được công bố. Một `atomic.Pointer[Config]` có thể Store pointer mới và Load pointer hiện hành; operation atomic có thứ tự sequentially consistent theo API `sync/atomic`. Nhưng truy cập atomic vào pointer không tự làm các field sau pointer an toàn. Nếu Store xong rồi sửa map, slice hay field của cùng object trong khi reader đọc, race vẫn có thể tồn tại.

Trong `TestCondPredicateAndAtomicPublication`, mỗi Config được dựng xong trước Store và không bị sửa về sau. Test kiểm tra giá trị quan sát trên các đường chạy đã chọn, race detector kiểm tra access trên lần chạy đó; chứng minh thiết kế vẫn là policy snapshot bất biến sau publication. Atomic không thay một transaction nhiều field, không giải quyết ownership của object lồng nhau và không có lời hứa luôn nhanh hơn mutex. Hãy dùng cơ chế có proof nhỏ nhất cho bất biến, rồi mới đo khi workload thực sự đòi tối ưu.

Chương này kết thúc khi anh nhìn hai goroutine không còn hỏi “có chạy cùng lúc không?”, mà hỏi “lần truy cập nào cùng dữ liệu, bất biến nào cần giữ, và đường happens-before nào làm điều đó đúng?”. Câu hỏi ấy đi cùng anh qua channel, HTTP handler, cache và code runtime; cơ chế có thể đổi, nhưng trách nhiệm chứng minh thứ tự không đổi.

Ghi chú kiểm chứng

1. Go Team. The Go Memory Model, phiên bản 6 June 2022. go.dev/ref/mem
2. Go Team. Data Race Detector. go.dev/doc/articles/race_detector
3. Go Team. Package sync. pkg.go.dev/sync

Chương 9 — Dòng công việc có áp suất

Mô hình tinh thần của chương này là **một send trên channel là một lần bàn giao còn dang dở cho tới khi bên nhận nhận nó, buffer giữ nó, hoặc người gửi chọn dừng.** Channel không biến công việc thành "song song an toàn" chỉ bằng một dấu `<-`. Nó buộc ta viết rõ ai còn nợ ai một giá trị, ai báo không còn giá trị nào nữa, và điều gì xảy ra khi người nhận không còn chờ.

Hãy bắt đầu bằng một tiện ích có vẻ hợp lý. `firstSuccess` chạy nhiều probe, nhận kết quả đầu tiên rồi return:

```
results := make(chan Result)

for _, probe := range probes {
    go func(probe Probe) {
        result := probe(ctx)
        results <- result
    }(probe)
}

select {
case result := <-results:
    return result, nil
case <-ctx.Done():
    return Result{}, ctx.Err()
}
```

Nếu một probe xong trước, caller nhận một `Result` và rời hàm. Những probe xong sau vẫn cố gửi. Vì `results` là unbuffered và không còn receiver, chúng block ở `results <- result`. Đây là goroutine leak: work đã xong nhưng goroutine không thể kết thúc vì nó đang giữ một lần bàn giao không còn ai nhận. `context` không tự cứu đoạn send này; nó chỉ là một tín hiệu mà code phải chọn lắng nghe.

Dừng đúng chỗ send

Một worker gửi kết quả phải nhìn đồng thời hai khả năng: consumer sẵn sàng nhận, hoặc request đã hết lý do để tiếp tục. `select` đặt hai quyết định ấy cạnh nhau:

```
select {
case out <- result:
    // Consumer đã nhận quyền sở hữu result.
case <-ctx.Done():
    // Không có lần bàn giao nào nữa.
    // Worker được phép kết thúc.
}
```

Đây không phải timeout chung chung. Nếu `out` chưa có receiver và context chưa bị hủy, worker chờ có chủ đích: áp suất của consumer đã truyền ngược về worker. Nếu context bị hủy, worker bỏ kết quả chưa bàn giao để không bị kẹt. Còn `Work` đang chạy bên trong worker có dừng ngay hay không lại là contract khác: `ctx` là yêu cầu hủy, không thể cưỡng bức một function hoặc system call bất kỳ dừng tức thì.

`select` chỉ chọn một communication đang có thể tiến lên. Khi nhiều case cùng sẵn sàng, specification yêu cầu chọn một case theo cách pseudo-random đồng đều; nó không hứa ưu tiên, fairness dài hạn hay thứ tự business. `default` khiến `select` không block, vì thế rất dễ vô tình biến "consumer đang chậm" thành "bỏ qua dữ liệu". Chỉ dùng `default` khi bỏ qua hoặc chuyển sang cơ chế khác là policy được viết rõ.

Buffer là khoảng nợ hữu hạn

Với channel không buffer, sender và receiver phải gặp nhau tại cùng một lần communication. Với `make(chan Result, 8)`, sender có thể đi trước tối đa tám result chưa được consumer lấy. Số 8 không làm work nhanh hơn, không phải số worker, và không phải bằng chứng hệ thống chịu tải tám request. Nó là lượng backlog chương trình cho phép giữ trong bộ nhớ trước khi áp suất quay lại người gửi.

Bảng 8 — Buffer là backlog hữu hạn, không phải số worker

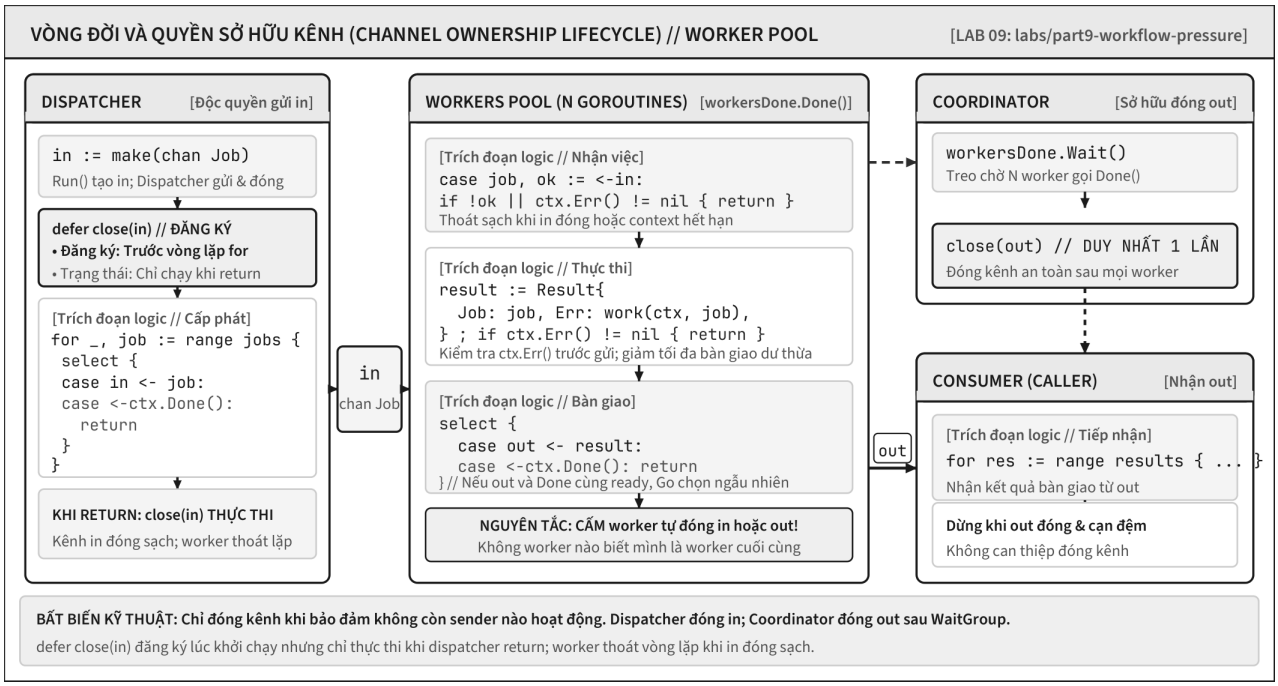
Thiết kế	Sender được đi trước bao xa?	Câu hỏi phải trả lời
`make(chan T)`	Không có chỗ đệm; cần receiver sẵn sàng.	Người gửi có được phép chờ consumer không?
`make(chan T, n)`	Tối đa `n` value chưa nhận.	`n` đại diện quota bộ nhớ hay burst nào? Khi đầy thì policy là gì?
`select` có `default`	Có thể không gửi dù channel chưa bị đóng.	Bỏ, log, retry hay trả lỗi có được hứa không?

Đây là semantics ngôn ngữ, không phải mô hình scheduler: buffered channel chỉ thay đổi lúc send/receive có thể tiếp tục. Memory model còn cam kết send xảy ra trước khi receive tương ứng hoàn tất. Cam kết ấy giúp truyền một value và các việc đã được chuẩn bị trước send; nó không cấp quyền cho nhiều goroutine cùng sửa một state tùy ý.

Close nói về người gửi, không nói "mọi thứ ổn"

`close(results)` có nghĩa là sẽ không còn send nào lên `results`. Nó không có nghĩa consumer đã xử lý hết result, không giết worker, và không làm send đang treo tự biến mất. Các value đã buffer vẫn được nhận trước; sau đó receive trả zero value với `ok == false`. Vì vậy `for result := range results` là contract dễ đọc: consumer tiếp tục lấy mọi result đã bàn giao, rồi dừng khi owner đóng kênh.

Chỉ owner của phía gửi mới nên close channel. Trong một worker pool, không worker nào tự biết mình là worker cuối; cho worker nào xong trước `close(results)` sẽ khiến worker khác send tiếp panic. Ta cần một coordinator biết tất cả worker đã return rồi mới đóng output đúng một lần.



Hình 21 — Sơ đồ này là mental model về ownership, không phải timeline scheduler. Mỗi channel có một owner của tín hiệu kết thúc, và mọi đường worker phải có lối thoát khi consumer không còn nhận.

Đọc một worker pool như một hợp đồng đóng kênh

Lab `labs/part9-workflow-pressure` không nối thẳng vào `opsprobe`. Ta chưa có requirement production nào buộc CLI ấy phải chạy nhiều endpoint cùng lúc. Dùng reproducer độc lập giữ một câu hỏi đủ sắc: một producer, nhiều worker và một consumer kết thúc với bao nhiêu result, ở thời điểm nào?

Mở `exercise/pool_test.go` trước. Test không cho sẵn hiện thực; nó chỉ đòi các type `Job`, `Result`, `Work` và function `Run`. Bốn cam kết kỹ thuật cấu thành hợp đồng thực thi của `Run` bao gồm:

Điều khoản hợp đồng	Mục tiêu kiểm chứng	Rủi ro hệ thống nếu vi phạm
Bảo toàn kết quả đơn trị	Mỗi job hoàn tất bình thường sinh ra đúng một `Result`.	Mất mát dữ liệu hoặc lặp kết quả xử lý.
Giới hạn tài nguyên song song	Số lượng lời gọi hàm `Work` thực thi đồng thời không bao giờ vượt quá ngưỡng `workers`.	Tràn bộ đệm, quá tải kết nối cơ sở dữ liệu hoặc cạn kiệt CPU.
Cơ chế giải phóng khi hủy bỏ	Với `Work` của fixture có thể return, context bị hủy cho worker thoát lệnh gửi dù output không được đọc; coordinator đóng output sau khi worker return.	Worker giữ một lần bàn giao không còn người nhận. Nếu `Work` không return, pool không có quyền cưỡng bức nó dừng.
Chu kỳ đóng kênh duy nhất	Khi dispatcher hết job và toàn bộ worker đã thoát, kênh output đóng đúng một lần.	Panic do đóng kênh nhiều lần hoặc deadlock vòng lặp `for range`.

Có một biên nhỏ nhưng đáng giữ ngay từ đầu: nếu `ctx` đã bị hủy trước khi dispatcher kịp bàn giao job, `Work` không được bắt đầu chỉ vì một worker vừa được tạo. Cancellation là policy dừng nhận việc mới; đưa một `Job` vào `Work` với context đã hết hạn chỉ tạo thêm một kết quả không còn người sở hữu.

Chạy lệnh khởi đầu; nó phải đỏ vì API chưa tồn tại:

```
go test -tags exercise ./exercise
```

Đừng thêm buffer lớn để làm test im. Test cancellation cố ý không consume output; một implementation chỉ có `out <- result` sẽ lộ goroutine bị kẹt. Cũng đừng đóng `results` từ worker. Hãy phác trên giấy ba owner trước khi code: dispatcher sở hữu send và `close(jobs)`; các worker chỉ receive job rồi send result; coordinator sở hữu `close(results)` sau `WaitGroup.Wait`.

Một khung đủ nhỏ cho điểm send quan trọng là:

```
result := Result{Job: job, Err: work(ctx, job)}

if ctx.Err() != nil {
    return
}

select {
case out <- result:
case <-ctx.Done():
    return
}
```

`ctx.Err()` trước `select` là policy có chủ đích của lab: nếu cancellation đã được quan sát sau `Work`, result đó không còn được bàn giao, kể cả khi consumer vừa quay lại. Bên trong `select`, cancellation vẫn là đường thoát cho trường hợp consumer biến mất trong lúc worker đang chờ. Phần còn lại là thiết kế của anh: dispatcher đưa `jobs` vào channel thế nào để cũng nghe `ctx.Done`; worker nhận đến khi `jobs` đóng ra sao; goroutine nào chờ `WaitGroup`; và trường hợp `workers <= 0` trả output đóng ngay hay bị từ chối ở API boundary. Test hiện tại chốt policy thứ nhất để câu hỏi vẫn tập trung vào lifecycle.

Đừng chỉ kiểm tra cancellation ở dispatcher. Một job có thể đã được bàn giao đúng lúc cancellation xảy ra; worker cần kiểm tra lại trước khi gọi `Work`. Điều này không cưỡng bức một `Work` đang chạy phải dừng — chính `Work` còn phải tôn trọng context của nó — nhưng nó giữ được lời hứa hẹp hơn và kiểm chứng được: sau khi cancellation đã được quan sát, pool không khởi động một job mới.

Sau khi tự làm, chạy bản tham chiếu với race detector:

```
go test -race ./fixed
```

Hướng của channel cho thấy quyền hạn

Trong function nội bộ, `chan Job` có cả quyền send và receive. Nhưng biên API có thể thu hẹp quyền đó:

```
func dispatch(ctx context.Context, jobs []Job) <-chan Job
func consume(in <-chan Job, out chan<- Result)
```

`<-chan Job` nói caller chỉ được receive; `chan<- Result` nói caller chỉ được send. Direction không thay thế ownership document - hai function vẫn phải thỏa thuận ai close - nhưng compiler đã ngăn một số nhầm lẫn lộ liễu. Bên nhận `in` không thể vô tình send; consumer của output không thể `close` một receive-only channel.

Khi đọc code channel, đừng bắt đầu bằng câu "channel này buffered hay không?". Hỏi theo thứ tự khó hơn nhưng hữu ích hơn: value nào đang được bàn giao, bên nào phải còn sống để receive nó, ai có quyền nói không còn send nữa, và cancellation có mở đường thoát cho mọi lần block không. Trả lời được bốn câu ấy trước khi chọn capacity thường ngăn được cả leak, deadlock và queue vô hạn.

Đã hủy công việc, vì sao goroutine vẫn còn?

Ví dụ đầu chương kẹt ở send; đổi thành receive không tự làm vấn đề biến mất. Trong `baseline.Wait` của lab `part10-resource-retention`, caller truyền context rồi hủy nó, nhưng goroutine chỉ thực hiện `<-in`. Không có sender thì nó vẫn chờ. Context là một tín hiệu, không phải quyền của caller để runtime giết một goroutine khác. Bản sửa đặt receive và `ctx.Done()` trong cùng `select`, rồi đóng `done` khi goroutine thực sự trả về. Nhận `done` là bằng chứng hoàn tất; vừa gọi `cancel()` chỉ là đã phát yêu cầu.

Test có tín hiệu `started` để chắc rằng goroutine đã bắt đầu, sau đó hủy context và chờ `done`. Bản lỗi có thêm kênh cứu hộ do test sở hữu để mở đường thoát sau khi quan sát lỗi, không để chính bộ test làm rò goroutine.

Đếm số goroutine hỗ trợ điều tra, nhưng test hồi quy nên chờ đúng công việc mình sở hữu thay vì đo toàn bộ process rồi đòi một con số cố định.

Vòng lặp định kỳ đặt ra một bẫy khác. Anh đọc đoạn dưới và thử chọn thao tác có thể kết thúc goroutine: hủy context bên ngoài, gọi `ticker.Stop()`, hay tạo một đường return riêng?

```
for range ticker.C {
    refresh()
}
```

`Ticker.Stop` ngừng phát tick nhưng không đóng `ticker.C`. Vì thế vòng `range` không được đánh thức để kết thúc chỉ nhờ Stop; nó có thể chuyển từ chạy mãi sang chờ mãi. Hủy một context mà loop không đọc cũng không giúp. Bản lab sửa vòng đời theo dạng sau; khi tick đến, hàm `work` vẫn phải có contract tôn trọng context nếu caller cần shutdown hữu hạn:

```
for {
    select {
    case <-ctx.Done():
        return
    case <-ticker.C:
        if ctx.Err() != nil {
            return
        }
        work(ctx)
    }
}
```

Owner gọi `defer ticker.Stop()` và đóng `done` khi loop kết thúc. Kiểm tra `ctx.Err()` trước `work` tránh bắt đầu việc khi đã quan sát cancellation; nó không bảo đảm cancellation không thể xảy ra ngay sau kiểm tra. Nếu callback đang kẹt trong một receive không đọc context, loop chưa quay lại `select`. Test `TestCancellationDoesNotKillNoncooperativeCallback` cố ý dùng callback như vậy: hủy context chưa đóng `done`, phải cứu callback trước. Test kế tiếp thay callback bằng tác vụ chọn cả `release` lẫn `ctx.Done()` và

chờ được kết thúc. Truyền tham số context không đủ; operation phải dùng nó tại điểm có thể block.

Đừng biến lỗi vòng đời này thành câu "quên Stop luôn gây memory leak". Tài liệu `time` từ Go 1.23 cho phép GC thu hồi ticker không còn được tham chiếu dù chưa Stop. Trong giai đoạn Go 1.23–1.26, phiên bản module và `asynctimerchan` còn có thể chọn cơ chế cũ. Go 1.27 đã bỏ setting này và luôn dùng timer channel đồng bộ; lab chạy Go 1.27.1, không dùng cờ ấy để suy ra behavior cũ. Đó là điều kiện version cần ghi cùng phép đo. Ticker được giải phóng và goroutine đọc ticker kết thúc là hai câu hỏi khác nhau; khi một worker còn sống và giữ ticker, tiền đề "ticker không còn tham chiếu" cũng chưa được đáp ứng.

Một worker nền có thể là công việc hợp lệ sống suốt service. Nó trở thành goroutine leak khi owner đã hết nhu cầu mà vẫn không có đường kết thúc, dù worker đang block hay thỉnh thoảng vẫn chạy. Resource leak rộng hơn: file, connection hoặc đăng ký callback không được trả đúng vòng đời có thể làm cạn tài nguyên dù heap không tăng đáng kể. Vì thế contract worker pool đã có cần giữ nguyên: ai nhận việc, ai phát hủy, ai close và ai xác nhận tất cả child đã kết thúc.

Mô hình Điều phối G/M/P và Giới hạn Thực tế của Concurrency

Khi trace cho thấy worker chờ chạy hoặc memory tăng theo số goroutine, mô hình điều phối runtime giúp đặt câu hỏi. Nó bổ sung cho contract worker pool, không phải điều kiện để người mới viết đúng channel.

Trong runtime Go 1.27.1, `src/runtime/proc.go` mô tả điều phối M:N bằng `G`, `M` và `P`: goroutine, luồng OS và tài nguyên điều phối gắn với `GOMAXPROCS`. `P` có hàng đợi chạy cục bộ; nó không phải một core vật lý được dành riêng. Đây là mô hình đọc trace/source, không phải contract của ngôn ngữ. `runtime/stack.go` đặt `stackMin = 2048` byte cho phần Go, nhưng kích thước cấp phát còn cộng phần theo OS rồi làm tròn; không suy ra goroutine trên Windows trong lab chỉ tốn 2 KiB tổng bộ nhớ hay luôn bắt đầu với allocation ấy.

Goroutine chia sẻ tài nguyên điều phối nhưng vẫn giữ stack, metadata và có thể giữ object sống; thêm goroutine còn thêm công việc lập lịch và root GC. Giới hạn concurrency phải xuất phát từ tài nguyên của workload — connection, file descriptor, RAM, CPU hoặc downstream — không từ một con số stack hay số goroutine truyền miệng.

Bản runtime này hiện thực channel bằng `hchan`, có khóa và hàng đợi chờ nội bộ; đó cũng không phải một API hay lời hứa về chi phí của ngôn ngữ. Dưới contention, với capacity khác nhau, hoặc khi goroutine phải ngủ rồi thức dậy, chi phí channel có thể đáng kể. Nhưng không có quy tắc phổ quát rằng `Mutex`, `atomic` hay channel luôn nhanh hơn: chúng mô tả những contract đồng bộ khác nhau. Hãy dùng channel khi cần bàn giao work hoặc phối hợp vòng đời; nếu một hot path trở thành vấn đề, hãy benchmark đúng workload và kiểm tra race trước khi đổi primitive.

Ghi chú kiểm chứng

1. Go Team. The Go Programming Language Specification, mục Channel types, Send statements, Receive operator, Close và Select statements. go.dev/ref/spec
2. Go Team. The Go Memory Model, mục Channel communication; và Package context. go.dev/ref/mem, pkg.go.dev/context
3. Go Team. Package `time`, `NewTicker` và `Ticker.Stop`, Go 1.27.1; Go 1.23 Timer Channel Changes, điều kiện module và `GODEBUG`. pkg.go.dev/time@go1.27.1#NewTicker; pkg.go.dev/time@go1.27.1#Ticker.Stop; go.dev/wiki/Go123Timer
4. Go Team. Go 1.27 Release Notes, Runtime: bỏ `asynctimerchan`. go.dev/doc/go1.27#runtime

Chương 10 — Khi chương trình chậm hoặc phình

Một chương trình bị phán xét là "chậm" hoặc "tốn bộ nhớ" thường kéo theo những nỗ lực tối ưu hóa mù quáng: thay thế gói `fmt` bằng phép cộng chuỗi thủ công, mở thêm hàng loạt goroutine không kiểm soát, hoặc viết lại thuật toán theo những mẹo vặt phức tạp. Những can thiệp này thường làm mã nguồn suy giảm độ trong sáng nghiêm trọng mà không giải quyết được nguyên nhân gốc rễ khiến người dùng hoặc hệ thống phải chờ đợi.

Mô hình tư duy nền tảng của chương này xác lập: tối ưu hóa là việc kiểm chứng một giả thuyết khoa học dưới một khối lượng công việc (workload) mang tính đại diện; một phép đo chỉ trả lời câu hỏi mà nó được thiết kế để đo lường. Benchmark không bảo đảm một dịch vụ mạng sẽ phản hồi nhanh trong mọi tình huống thực tế. Bản ghi CPU profile không phản ánh việc bộ nhớ đang phình to. Dấu vết thực thi (Execution trace) không phải là bản chụp toàn cảnh ngữ nghĩa của mã nguồn. Mỗi công cụ là một dụng cụ đo lường chuyên biệt nhằm thu hẹp không gian suy đoán.

Giả sử một tiến trình kiểm tra dịch vụ định kỳ (`opsprobe`) cần kết xuất báo cáo trạng thái cho hàng nghìn endpoint mạng. Khi chạy trên môi trường tích hợp liên tục (CI), thao tác này tiêu tốn nhiều giây và dung lượng vùng nhớ động (heap) tăng vọt. Thay vì phỏng đoán cảm tính, kỹ sư chuyển hóa vấn đề thành câu hỏi kỹ thuật có thể đo đạc: với một danh sách đầu vào xác định, hàm `Render` tiêu tốn bao nhiêu thời gian CPU và bao nhiêu lần cấp phát bộ nhớ; chu kỳ xử lý đắt đỏ tập trung ở đâu; và một thiết kế thay thế có bảo toàn từng byte đầu ra trong khi giảm thiểu chi phí tài nguyên hay không?

Bản chất Chuỗi Tối ưu hóa: Từ Source đến Mã máy

Để đặt câu hỏi về hiệu năng, tách quyết định lúc compile khỏi công việc lúc chạy:

Tầng	Cơ chế cần điều tra	Bằng chứng phù hợp
Compiler của toolchain đã pin	Inlining, escape analysis, SSA và loại bỏ kiểm tra biên có thể đổi cách thực hiện cùng semantics.	Diagnostic compiler; assembly của cùng source và cờ build.
Mã máy của artifact	Instruction, thanh ghi và lời gọi còn lại sau build.	Objdump của đúng binary, không trộn với listing từ build khác.
Runtime khi workload chạy	Allocation, GC, lập lịch và I/O tiêu tốn tài nguyên theo đường thực thi.	Benchmark, profile và trace, kèm môi trường và giới hạn phép đo.

Các bước này là mô hình để đặt câu hỏi, không phải pipeline hay ngưỡng ổn định của ngôn ngữ. Inlining có thể làm thay đổi thông tin mà escape analysis nhìn thấy; compiler cũng có thể đặt một value vào thanh ghi, stack, heap hoặc bỏ hẳn nó. Chỉ khi chi phí đã được đo, output của compiler và profiler mới cho biết điều gì xảy ra với phiên bản, kiến trúc và workload đang xét.

Phân tích Thoát Bộ nhớ: Ngăn xếp đối đầu Vùng nhớ Động

Trình biên dịch Go dùng escape analysis để suy luận liệu một value có cần tồn tại sau scope mà compiler đang xét hay không. Đây là heuristic triển khai, không phải tính chất cú pháp của một biến hay lời hứa rằng một dòng code luôn cấp phát ở một nơi.

Một value không escape có thể ở stack, thanh ghi hoặc bị tối ưu hóa mất; nó không mặc nhiên có một allocation đo được. Ngược lại, trả con trỏ, lưu vào global hoặc đóng gói vào interface là tín hiệu để compiler phân tích lifetime, không phải luật "chắc chắn heap". Kết quả còn phụ thuộc thân hàm, inlining, kiến trúc và version. Vì vậy, `-gcflags=all=-m=2` là bằng chứng cho một lần biên dịch cụ thể; benchmark `allocs/op` mới cho biết contract đo có allocation quan sát được hay không.

Chạy `go build -gcflags=all=-m=2 ./fixed` từ `labs/part10-measure-first` với Go 1.27.1. Trích đoạn dưới rút gọn

prefix đường dẫn `fixed/`, lược dòng không liên quan và ngắt dòng cho vừa khung; không phải toàn bộ diagnostic:

```
./render.go:13:6: cannot inline Render:
    function too complex: cost 231 exceeds budget 80
./render.go:16:18: inlining call to
    strings.(*Builder).WriteString
./render.go:18:36: inlining call to strconv.FormatInt
./render.go:13:13: readings does not escape
./render.go:16:18: append(strings.b.buf, strings.s...)
    escapes to heap in Render:
  flow: {heap} ← &{storage for append(...)}:
    from append(strings.b.buf, strings.s...) (spill)
    from strings.b.buf = append(...) (assign)
```

Output này ghi nhận hai quan sát cho source artifact và toolchain của thí nghiệm:

Thứ nhất, tham số `readings []Reading` được báo `does not escape`. Slice không tự quyết định nơi dữ liệu hay descriptor nằm; kết quả chỉ nói rằng compiler ở lần này không cần kéo value đó ra heap vì việc dùng trong hàm.

Thứ hai, storage tạo bởi `append` trong `strings.Builder` được báo escape. Đó là dữ liệu thực nghiệm hữu ích để giải thích `allocs/op`, không phải quy tắc rằng mọi `append` hoặc mọi `Builder` đều heap-allocate.

Tối ưu hóa SSA và Loại bỏ Kiểm tra Biên (Bounds Check Elimination)

Một tối ưu hóa quan trọng của backend SSA là loại bỏ kiểm tra biên khi compiler chứng minh được chỉ số hợp lệ. Ở ngữ nghĩa Go, truy cập ngoài biên phải panic; instruction cụ thể dùng để bảo vệ điều đó phụ thuộc compiler và kiến trúc. Output `ssa/prove` dưới đây là quan sát của thí nghiệm Go 1.27.1, không phải hình dạng mã máy mà mọi bản build phải có.

Chạy cùng lệnh build với `-gcflags=all=-d=ssa/prove/debug=1`. Cờ `all` còn phát diagnostic cho thư viện chuẩn; prefix GOROOT được rút gọn ở

trích đoạn dưới, không biến các dòng `strings.go` thành output của thân `Render`:

```
./render.go:15:20: Induction variable: limits [0,?), increment 1
./render.go:15:20: Inverted loop iteration
./render.go:21:19: Disproved Less64
strings/strings.go:987:27: Proved IsInBounds
strings/strings.go:988:38: Proved IsSliceInBounds
```

Các dòng `render.go` cho thấy nhận diện biến quy nạp và đảo chiều vòng lặp; các dòng `strings.go` báo điều kiện đã được chứng minh trong thư viện chuẩn của lượt build ấy. Muốn kết luận một bounds check cụ thể của `Render` đã bị bỏ, phải tìm đúng source vị trí và đối chiếu diagnostic/assembly tương ứng. Giữ invariant dễ đọc rồi đo, không viết code vòng vèo chỉ để ép một hình dạng tối ưu hóa.

Cơ chế Thu gom Rác và Đánh đổi Không gian - Thời gian

GC trong runtime chuẩn Go 1.27.1 dùng tracing mark-sweep đồng thời, với write barrier và các pha dừng thế giới mô tả ở `runtime/mgc.go`. Nó không phải lựa chọn thuật toán mà specification bắt mọi implementation phải dùng. GC Guide cung cấp mô hình đánh đổi CPU và bộ nhớ; để giải thích chi tiết của edition này, cần đối chiếu thêm source và release notes, không dùng guide như một benchmark dịch vụ.

Trong `runtime/mgcpacer.go` của Go 1.27.1, `gcBackgroundUtilization = 0.25` đặt mục tiêu background marking theo một phần của `GOMAXPROCS` trong pha mark. Không có nghĩa GC luôn chiếm 25% CPU toàn process hay giữ riêng một P trong bốn P: dedicated, fractional và idle workers cùng tham gia, còn assist có chi phí riêng. Muốn biết workload thực sự trả bao nhiêu CPU cho GC, đọc metric/profile của lượt chạy đó.

Runtime có điểm dừng ở sweep termination và mark termination; thời gian dừng cần đọc từ lượt chạy thật, không lấy mục tiêu thiết kế làm latency guarantee. Trong pha mark, allocation có thể tạo khoản nợ công việc đánh

dấu theo tỷ lệ mà pacer tính. Khi goroutine còn nợ, đường mark assist có thể bắt nó làm thêm công việc hoặc chờ credit. Đó là một hướng điều tra khi allocation và latency tăng cùng nhau, không phải phép so sánh đơn giản "tốc độ allocation lớn hơn tốc độ mark" hay bằng chứng tail latency chắc chắn tăng. Đối chiếu trace, `runtime/metrics` và đường assist trong source đã pin.

Green Tea GC là bối cảnh triển khai, không phải contract

Go 1.27.1 có Green Tea GC trong runtime. Ý tưởng chính của thiết kế này là ưu tiên xử lý theo page/span thay vì coi từng object là đơn vị work duy nhất; mục tiêu là cải thiện locality của một số heap. Nó là implementation detail có thể tiếp tục thay đổi giữa các bản Go và không dự báo kết quả cho một service cụ thể. Khi GC xuất hiện trong profile, câu hỏi vận hành vẫn là: allocation rate, live heap, root set, pointer density và latency của workload đang là bao nhiêu? Đo trước, rồi mới suy luận liệu upgrade Go version, giảm allocation hay đổi cấu trúc dữ liệu có ích.

Hai biến số môi trường chi phối trực tiếp hành vi đánh đổi không gian và thời gian này:

Tham số `GOGC`: Chọn một điểm trên trade-off CPU/bộ nhớ. Với Go hiện đại, target heap còn tính cả GC roots: gần đúng là `live heap + (live heap + roots) * GOGC / 100`. `GOGC=100` không luôn có nghĩa tổng heap bằng đúng hai lần live heap. Tăng nó thường giảm tần suất GC và tăng memory overhead; đó là xu hướng phải xác minh bằng workload thật.

`GOMEMLIMIT`, từ Go 1.19, là soft limit cho đại lượng bộ nhớ runtime quản lý, không chỉ live heap. GC có thể làm việc thường xuyên hơn khi tới limit, nhưng runtime cho phép vượt nó để tránh thrashing. Limit này không phải trần RSS hay trần cgroup. Khi đặt trong container, cần đo cả phần charge ngoài đại lượng runtime tính và chừa headroom tương ứng, không chọn một tỷ lệ "an toàn" chung. Nó không bảo đảm tránh OOM; một workload giữ live data lớn hơn ngân sách vẫn cần thay contract hoặc capacity.

Đo lường Hiệu năng: Benchmark là Hợp đồng Thực nghiệm

Contract benchmark quyết định việc nào phải tính giờ. Nếu đang đo riêng `Render`, dựng readings trước vòng lặp; nếu đang đo cả pipeline nhận và xử lý input, chi phí chuẩn bị tương ứng có thể chính là phần cần đo. Không loại setup theo nghi thức rồi vô tình làm mất workload mà người dùng phải chịu.

`B.Loop` có từ Go 1.24 và là cách viết phù hợp cho benchmark mới ở edition này; vòng `b.N` vẫn được hỗ trợ. Lần gọi đầu reset timer, lần trả false dừng timer, nên setup đặt trước vòng không được tính vào kết quả:

```
func BenchmarkRender(b *testing.B) {
    readings := representativeReadings(1_000)

    for b.Loop() {
        Render(readings)
    }
}
```

Lệnh thực thi đo đạc thống kê nhiều lần:

```
go test -bench BenchmarkRender -benchmem -count=6 ./fixed
```

Chỉ số `ns/op` là thời gian bình quân theo contract benchmark. `B/op` và `allocs/op` là các metric allocation mà benchmark runner báo cho workload đó; chúng không tự nói tổng RSS hay mọi chi phí của service. Một kết quả đo lường đơn lẻ không đại diện cho chân lý phổ quát; việc chạy 6 lượt liên tiếp (`-count=6`) giúp kỹ sư nhận diện được độ biến thiên (nhiễu đo lường) trước khi đưa ra kết luận.

Công cụ chẩn đoán	Câu hỏi kỹ thuật được giải đáp	Giới hạn phân tích
Kiểm thử đơn vị (Unit test)	Đầu ra và mã lỗi có bảo toàn đúng cam kết logic?	Không chứng minh được mã chạy nhanh hơn hay tốn ít RAM hơn.
Đo lường chuẩn (Benchmark)	Thao tác này tiêu tốn thời gian và phân bổ bộ nhớ ra sao dưới input chuẩn?	Không phản ánh trực tiếp độ trễ mạng hay tải tương tranh thực tế.
Phân tích CPU (CPU profile)	Sample CPU tập trung ở stack/hàm nào?	Không đo trực tiếp thời gian chờ khi goroutine đã block; CPU dùng cho syscall, spin hay đường khóa vẫn có thể xuất hiện.
Phân tích Bộ nhớ (Heap profile)	Đường dẫn mã nguồn nào chịu trách nhiệm cho các khối cấp phát trên heap?	Dữ liệu mang tính lấy mẫu thống kê, không ghi nhận từng biến riêng lẻ.
Dấu vết thực thi (Execution trace)	Sự kiện runtime và chuyển trạng thái goroutine nào xảy ra trong khoảng đã ghi?	Kích thước và overhead tùy workload, thời gian ghi và version; cần ngân sách, không coi trace là diễn giải mọi semantics của source.

Phân tích CPU và Bộ nhớ (pprof)

Khi benchmark phát hiện điểm nghẽn hiệu năng, công cụ trích xuất profile (`pprof`) giúp định vị chính xác vị trí tích lũy chi phí trong cây gọi hàm:

```
go test -run '^$' -bench BenchmarkRender `
  -cpuprofile cpu.out -memprofile mem.out ./fixed
go tool pprof -top cpu.out
```

Chỉ số `flat` ghi nhận chi phí tiêu tốn trực tiếp bên trong thân hàm, trong khi chỉ số `cum` (cumulative) ghi nhận tổng chi phí tích lũy của hàm đó cùng toàn bộ các hàm con mà nó triệu gọi. Một hàm có `flat` nhỏ nhưng `cum` lớn là dấu hiệu cho thấy nó đang dẫn lối vào một chuỗi thao tác tiêu tốn nhiều tài nguyên.

Nếu nghi goroutine bị giữ lại, Go 1.27 có profile `goroutineleak` ở `runtime/pprof` và endpoint tương ứng của `net/http/pprof`, không còn

cần experiment từ Go 1.26. Nó tìm một lớp chờ mà runtime xác định không thể được đánh thức, không tìm mọi leak và không tự thu hồi goroutine. Profile không có hit vẫn cần đối chiếu goroutine dump, ownership của channel và cancellation path như Chương 9. Khi mở trace UI, `go tool trace -http=:6060` từ Go 1.27 chỉ nghe localhost; muốn nghe địa chỉ khác phải chỉ định rõ. Đừng đưa profile/trace có dữ liệu nhạy cảm lên một listener công khai chỉ để tiện xem.

Ca điều tra: tăng bộ nhớ nhưng chưa biết ai đang giữ

Một tiến trình nhận payload 4 MiB cho mỗi item. Sau mỗi chu kỳ hai item, anh thấy `TotalAlloc` tăng; người trực ca muốn thêm lệnh GC hoặc kết luận "Go bị leak". Ta cần một câu hỏi chặt hơn: những allocation ấy chỉ đã từng xảy ra, hay vẫn sống sau khi công việc hoàn tất, và ai còn có quyền giữ chúng? Lab độc lập `labs/part10-resource-retention` cố ý tách đường giữ dữ liệu khỏi HTTP, database và opsprobe để câu trả lời không bị lẫn với connection pool hoặc cache của thư viện.

Chạy từng mode trong process riêng, giữ nguyên máy, toolchain và tham số. Từ thư mục lab, lệnh dưới dùng PowerShell; thêm `-fixed` rồi đổi thư mục output để đo bản sửa, không ghi đè profile bản lỗi:

```
go build -o artifacts/investigate.exe ./cmd/investigate
./artifacts/investigate.exe -mode=slice `
  -cycles=4 -items=2 -bytes=4194304 -limit=2 `
  -out=artifacts/slice-baseline
go tool pprof -inuse_space -top `
  artifacts/slice-baseline/04.heap.pprof
go tool pprof -alloc_space -top `
  artifacts/slice-baseline/04.heap.pprof
```

Đổi mode thành `cache`, `churn`, `goroutine` hoặc `ticker` để giữ cùng nhịp công việc nhưng thay cơ chế. `churn` tạo rồi bỏ payload, không giữ kết quả; cờ `-fixed` không thay đường chạy của mode đối chứng này. Mode goroutine và ticker không cấp phát payload theo cờ `-bytes`: chúng tạo

worker để điều tra vòng đời, không mô phỏng xử lý 4 MiB. README ghi cả giới hạn workload lẫn cách chạy mode context riêng.

Trước khi đọc source, hãy đối chiếu bảng đã đo ngày 09-10-2026 trên `go1.27.1 windows/amd64`. Các dãy là quan sát sau chu kỳ 1 đến 4; mỗi chu kỳ lab chủ động hoàn thành hai lượt GC trước khi ghi số liệu. Anh sẽ mở heap profile hay goroutine dump cho từng dòng, và giả thuyết nào còn thiếu bằng chứng?

Bảng 9 — Các triệu chứng dẫn tới những phép đo khác nhau

Mode	Quan sát bản lỗi hoặc đối chứng	Quan sát bản sửa
Slice	Entry 2, 4, 6, 8; `HeapAlloc` 8.35, 16.35, 24.35, 32.35 MiB	Cùng số entry; `HeapAlloc` xấp xỉ 0.35 MiB
Cache	Entry 2, 4, 6, 8; `HeapAlloc` 8.35, 16.35, 24.35, 32.36 MiB	Entry luôn 2; `HeapAlloc` xấp xỉ 8.35–8.36 MiB
Churn	Entry 0; `HeapAlloc` xấp xỉ 0.35 MiB	Không có đường sửa riêng; đây là đối chứng
Receive	`NumGoroutine`: 3, 5, 7, 9 sau khi caller cancel	Luôn 1, đã chờ `done` từng worker
Ticker	`NumGoroutine`: 3, 5, 7, 9 sau khi caller cancel	Luôn 1, đã chờ `done` từng worker

Các số làm tròn không phải threshold cho test và không phải ngân sách RAM phổ quát. Lab đặt `MemProfileRate = 1` để theo dõi từng allocation trong một workload nhỏ; ghi profile và goroutine dump cũng tự cấp phát. Vì vậy `TotalAlloc` còn gồm chi phí quan sát, không bằng tổng payload. Bình thường runtime lấy mẫu; profile có độ trễ đối với việc ghi nhận allocation/free, được tài liệu `runtime.MemProfile` lưu ý. Hai lượt GC là lựa chọn chuẩn hóa thí nghiệm này, không phải cách vận hành service để "chữa leak".

Từ vị trí cấp phát tới đường giữ dữ liệu

Ở mode slice, `inuse_space` sau chu kỳ 4 ghi 32 MiB tại `main.payload`. Cùng workload bản sửa, lọc các stack `main.payload|fixed.Prefix` chỉ còn 128 byte tại `fixed.Prefix`. Nhưng `alloc_space` vẫn ghi 32 MiB tại `main.payload` ở cả hai bản: cả hai đều đã nhận tám payload, chỉ khác dữ liệu phải giữ sau khi xử lý. Mode churn cũng có 32 MiB đã cấp phát tại site này mà không còn payload sống trong profile quan sát. `alloc_space` là chi phí tích lũy cấp phát, không phải lượng sống và không phải detector leak.

Profile dẫn tới nơi object được tạo, không tự vẽ reference nào đang giữ object. Đọc `Prefix` mới xác nhận đường `views` tới prefix, rồi tới array lớn; biểu thức ba index không copy. Copy ở boundary loại đường giữ đó. Đây là bản sửa có test aliasing ở Chương 2, không phải thay collector. Nếu một alias khác vẫn sống thì kết quả có thể khác, nên giữ workload và ownership của fixture cùng bằng chứng đo.

Ở mode cache, profile bản lỗi cũng chỉ tới `main.payload`: 32 MiB sống sau bốn chu kỳ. Bản FIFO chỉ còn 8 MiB payload sống ở `fixed.(*Cache).Put`, vì cache sở hữu bản copy của hai entry cuối. Trong `alloc_space`, riêng hàm Put đã cấp phát 32 MiB để copy; bản sửa có thể cấp phát nhiều hơn nhưng giữ ít hơn. Không chọn implementation chỉ vì nhìn thấy `B/op` nhỏ hơn. Đối chiếu map entry với contract Chương 3 mới biết tăng trưởng nào là hợp lệ: tám key trong kho lưu trữ có chủ đích và tám key trong cache hứa tối đa hai có cùng hình heap nhưng khác kết luận.

Goroutine không cần nhiều heap để làm lỗi vòng đời

Dãy 3, 5, 7, 9 đặt giả thuyết "mỗi chu kỳ giữ hai worker", chưa chứng minh chúng bị leak. Mở `04.goroutines.txt`: mode receive có tám stack trong `baseline.Wait.func1` ở trạng thái chờ channel; mode ticker có tám stack trong `baseline.RunTicker.func1` chờ `select`. Tiếp theo đọc source và contract: owner đã cancel, receive không đọc context, còn loop ticker chỉ có tick hoặc kênh cứu hộ của lab. Không có đường cancellation nghiệp vụ để trả về. Số lượng worker tăng bền qua các chu kỳ và những đường chờ ấy mới liên kết được triệu chứng với nguyên nhân.

Bản sửa chờ `done` sau cancel trước khi đo, nên worker đã kết thúc thay vì chỉ "hy vọng scheduler chạy sớm". Cuối thí nghiệm, harness cứu cả các worker lỗi rồi join; số goroutine trở về 1 trong lượt đã chạy. Đây là bằng chứng ta quản lý được đường dọn dẹp, không phải runtime tự giết goroutine. Test với callback không hợp tác vẫn cần release riêng như Chương 9: nếu dump kẹt ở callback, sửa `select` của loop bên ngoài là chưa đủ.

Mode context làm rõ giới hạn của bộ đếm này. Với 512 child có deadline một giờ, bản quên cancel vẫn chỉ có một goroutine trong lượt đo. Profile `inuse_objects` của nó còn 1024 object trực tiếp tại `context.WithDeadlineCause` ở chu kỳ 4; bản sửa không còn object trực tiếp tại site ấy trong profile đã đo. Không gọi 1024 là "1024 context": một lời gọi có thể tạo nhiều object, còn một ít site timer/parent vẫn hiện trong profile. Test lifecycle và source `context` ở Chương 12 mới giải thích vì sao parent giữ child đến lúc bị hủy hoặc deadline đến. Sau harness hủy parent, allocation trực tiếp tại site này không còn trong profile quan sát. Quên cancel ở đây là giữ thừa theo vòng đời, không phải lời khẳng định mọi context lỗi sống vĩnh viễn.

Heap đã giảm, vì sao bộ nhớ process chưa giảm tương ứng?

Ta còn phải phân biệt cái đã đo với cái đang suy ra. `HeapAlloc` là byte heap object được runtime tính đang cấp phát; `HeapInuse` là byte trong các span đang dùng, có thể gồm phần chưa chứa object. `HeapSys` là bộ nhớ heap runtime đã lấy từ OS, còn `HeapReleased` ghi phần trả lại cho OS. Chúng không phải RSS, và lab này không đo RSS. RSS hay working set còn phụ thuộc OS, stack, runtime và các vùng nhớ khác; bộ nhớ object giảm không hứa biểu đồ process giảm ngay cùng lượng ấy.

Trong mode slice lỗi, sau khi harness bỏ toàn bộ view, `HeapAlloc` giảm từ 33,924,560 xuống 369,936 byte nhưng `HeapSys` vẫn là 41,680,896 byte trong lượt đã chạy. Đây là ví dụ cụ thể không được kết luận "GC chưa thu hồi object" chỉ từ phần heap đã lấy của OS. Cũng không suy ra toàn bộ phần chênh lệch là RSS: đó là chỉ số khác chưa đo. Nếu nghi nhu cầu bộ nhớ cao hợp lệ, kiểm tra số dữ liệu đang phục vụ và ngân sách; nếu nghi allocation tạm hoặc GC chưa kịp chạy, so các chu kỳ sau khi workload lắng xuống và

GC hoàn tất; nếu object vẫn còn sống ngoài nhu cầu, tìm owner hoặc alias giữ nó; nếu file/connection cạn mà heap ổn, đo đúng resource thay vì tiếp tục nhìn heap.

Một kết luận có thể bảo vệ được cần ghép chuỗi workload, vị trí allocation hoặc stack chờ, source giải thích đường giữ, rồi test contract và phép đo lại sau sửa. Mỗi bằng chứng trả lời một câu khác nhau. Một snapshot cao, một lần RSS không giảm hay một report vet sạch không thể thay toàn bộ chuỗi ấy.

Đối chiếu G/M/P với Execution Trace: Nhìn vào nhịp đập của Runtime

Ở Chương 09, ta đã xây dựng mô hình tư duy về bộ điều phối của Go runtime qua ba thực thể: Goroutine (G), Thread hệ điều hành (M), và Logical Processor (P). Ta đã biết P sở hữu hàng đợi cục bộ (`runq`), M gắn với P để thực thi mã máy của G, và runtime sử dụng các cơ chế trộm việc (`work-stealing`), cướp quyền (`preemption`) và chuyển giao P khi gọi syscall (`entersyscall`).

Tuy nhiên, CPU profile thông qua `pprof` chỉ là một bản chụp lấy mẫu thống kê thời gian thực thi mã máy. Khi một dịch vụ phản hồi chậm chạp nhưng mức sử dụng CPU lại rất thấp, `pprof` trở nên bất lực: các goroutine không hề tiêu tốn CPU mà đang phải chờ đợi. Đây chính là ranh giới mà Dấu vết thực thi (Execution Trace) trở thành công cụ chẩn đoán quan trọng.

Bản chất của Execution Trace là ghi lại một tập rộng các sự kiện runtime phát sinh trong quá trình thực thi: thời điểm goroutine chuyển đổi trạng thái, thời điểm P được gắn kết hay tách rời khỏi M, các pha thu gom rác GC, và hoạt động của các điểm dừng đồng bộ hay lời gọi hệ thống. Trace không phải là một bản ghi toàn tri ghi lại mọi hành vi phần cứng. Trong mô hình trace của Go (đối chiếu `internal/trace.GoState`), runtime quản lý các trạng thái như `GoUndetermined`, `GoNotExist`, `GoRunnable`, `GoRunning`, `GoWaiting`, và `GoSyscall`. Để phục vụ việc phân tích triệu chứng vận hành, ta thường tập trung quan sát một tập con các trạng thái chủ đạo:

Một là, `GoRunning`: Goroutine đang trực tiếp chiếm giữ một Thread M trên một Processor P để tính toán mã máy.

Hai là, `GoRunnable`: Goroutine đã sẵn sàng chạy (vừa được tạo ra, vừa nhận được dữ liệu, hoặc vừa được đánh thức) nhưng đang nằm trong hàng đợi chờ P tiếp nhận. Thời gian `Runnable` cao (scheduler latency) là tín hiệu chỉ báo để điều tra áp lực lập lịch hoặc tranh chấp tài nguyên khi lượng công việc sẵn sàng lớn hơn số Logical Processor (`GOMAXPROCS`) hiện có, chứ không tự nó kết luận nguyên nhân gốc rễ.

Ba là, `GoWaiting`: Goroutine chủ động nhường quyền thực thi và chờ đợi một điều kiện đồng bộ hóa: chờ channel, chờ khóa `sync.Mutex`, chờ `sync.WaitGroup`, hoặc chờ sự kiện mạng từ network poller. Khi chờ channel hay mutex, runtime chuyển trạng thái goroutine sang chờ trong bộ điều phối người dùng mà không nhất thiết phải kéo theo việc chuyển ngữ cảnh luồng hệ điều hành ngay lập tức.

Bốn là, `GoSyscall`: Goroutine đang thực hiện một lời gọi hệ thống chặn (blocking syscall) ở tầng hệ điều hành. Cần phân biệt rõ `GoSyscall` với `GoWaiting`: ở `GoSyscall`, luồng M thực sự bị ràng buộc vào lời gọi của nhân hệ điều hành. Nếu thao tác bị kéo dài, runtime có thể tách P khỏi M (`handoffp`) để luồng khác tiếp tục phục vụ các goroutine đang chờ trong hàng đợi. Trong các pha GC dồn dập, nếu goroutine cấp phát bộ nhớ quá nhanh, runtime có thể yêu cầu goroutine đó tham gia hỗ trợ công việc đánh dấu (`GCMarkAssist`), tạo ra các khoảng gián đoạn có thể nhận diện trên dòng thời gian.

Để phân tích định lượng các điểm nghẽn mà không suy diễn cảm tính từ một trạng thái đơn lẻ, công cụ `go tool trace` hỗ trợ trích xuất bốn hồ sơ pprof chuyên biệt từ dữ liệu trace: `sync` (độ trễ do rào cản đồng bộ như mutex, waitgroup, channel), `sched` (độ trễ điều phối từ lúc goroutine sẵn sàng đến lúc được chạy), `syscall` (độ trễ chặn tại các lời gọi hệ thống), và `net` (độ trễ chờ I/O mạng từ network poller).

Trong `labs/part10-measure-first`, ta có thể thu thập trace thực tế khi chạy kiểm thử xử lý theo lô đồng thời (`TestRenderConcurrentBatchWorkload`):

```
go test -run TestRenderConcurrentBatchWorkload `
  -trace trace.out ./fixed
go tool trace trace.out
```

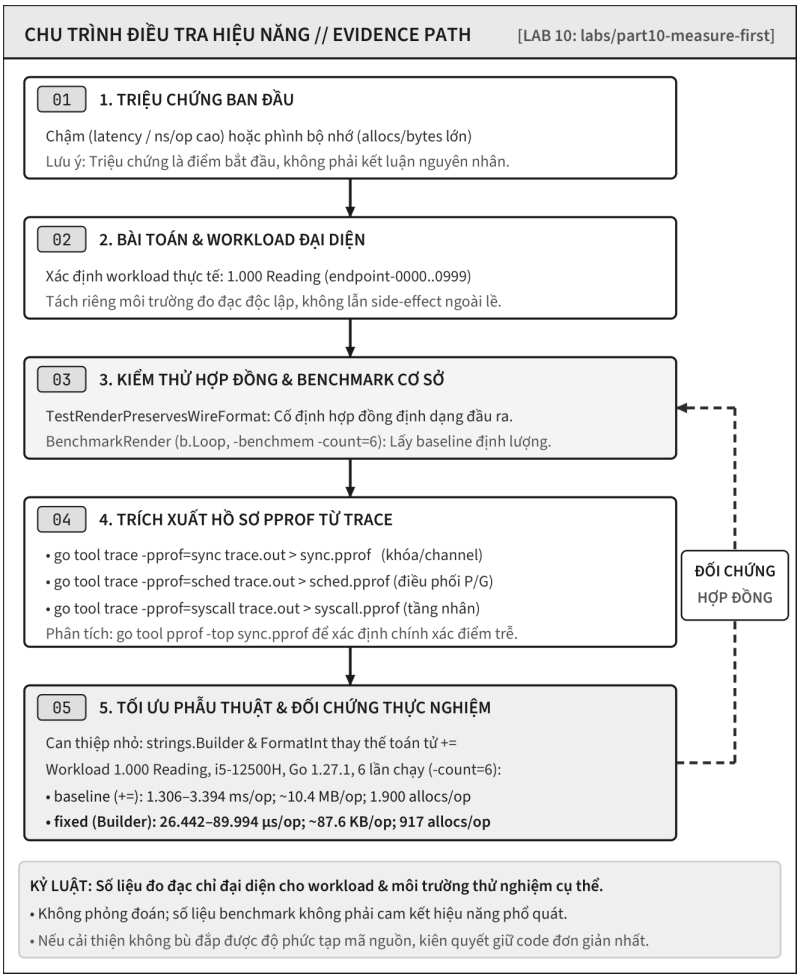
Giao diện đồ họa trên trình duyệt (mở qua lệnh `go tool trace trace.out`, mặc định chỉ lắng nghe trên localhost) hiển thị các dòng thời gian của từng Processor P, cho thấy thời điểm các worker goroutine chạy song song trên các processor và các khoảng dừng của hệ thống.

Để trích xuất các hồ sơ định lượng phục vụ phân tích, ta sử dụng cờ `-pprof` của `go tool trace`:

```
go tool trace -pprof=sync trace.out > sync.pprof
go tool trace -pprof=sched trace.out > sched.pprof
go tool trace -pprof=syscall trace.out > syscall.pprof
go tool pprof -top sync.pprof
```

Dữ liệu thực tế quan sát được trong fixture của lab phản ánh đúng bản chất của bài toán: hồ sơ `sync.pprof` ghi nhận độ trễ tại `sync.(*WaitGroup).Wait` khi goroutine chính của test chờ bốn worker hoàn thành đợt render lô, cùng các điểm nhận channel nội bộ của framework kiểm thử (`runtime.chanrecv1`). Hồ sơ `sched.pprof` ghi nhận độ trễ điều phối khi các worker goroutine được khởi tạo và chờ Processor tiếp nhận. Hồ sơ `syscall.pprof` ghi nhận thời gian dừng ở tầng nhân khi thực hiện ghi dữ liệu trace ra tập tin.

Dấu vết thực thi không phải là bằng chứng xác nhận ngữ nghĩa đúng đắn của logic nguồn hay cam kết về độ trễ trên môi trường sản xuất. Trace là công cụ phân loại triệu chứng: khi gặp vấn đề về hiệu năng, các hồ sơ pprof trích xuất từ trace giúp kỹ sư định hướng bước can thiệp tiếp theo dựa trên dữ liệu đo đạc thực nghiệm thay vì phỏng đoán.



Hình 22 — Chu trình điều tra hiệu năng hoàn chỉnh. Dữ liệu bắt đầu từ một bài toán đo lường có khối lượng công việc đại diện, kiểm chứng qua profiler, sau đó đối chiếu mã nguồn và tối ưu hóa có đo đạc đối chứng.

Điển cứu thực tế: Phân tích bài toán nối chuỗi

Thư mục `labs/part10-measure-first` duy trì hai cách hiện thực cho cùng một yêu cầu định dạng văn bản: phương án cơ sở (`baseline`) nối chuỗi bằng toán tử `+=` lặp lại; phương án tối ưu (`fixed`) sử dụng `strings.Builder` và `strconv.FormatInt`.

Workload gồm 1.000 `Reading`, từ `endpoint-0000` đến `endpoint-0999`, dựng ngoài vòng đo. Lượt kiểm chứng local ngày 01-10-2026 dùng Go 1.27.1, `windows/amd64`, CPU 12th Gen Intel Core i5-12500H, benchmark mặc định báo suffix `-16`, chạy sáu lần trên cùng máy/môi trường, Go version, workload và benchmark contract. Khoảng dưới đây là min–max của sáu kết quả `ns/op`, không phải confidence interval. Độ dao động lớn cho thấy cần cẩn trọng khi suy tỷ lệ speedup; đây không phải phép đo latency của service:

Phương án hiện thực	Khoảng thời gian mỗi operation	Allocation runner báo
`baseline` (Toán tử +=)	1.306–3.394 ms/op	10,443,876–10,443,996 B/op; 1,900–1,901 allocs/op
`fixed` (`strings.Builder`)	26.442–89.994 µs/op	87,600–87,608 B/op; 917 allocs/op

Trong source `baseline`, mỗi lần nối thêm giữ lại kết quả string dài hơn cho lượt kế tiếp. Source runtime Go 1.27.1 ở `runtime/string.go`, cùng allocation benchmark, giúp điều tra chi phí dựng lại dữ liệu chuỗi; `fixed` gom output trong Builder. Không suy từ đó rằng mọi phép + đều allocate hay gán một tỷ lệ throughput cố định. Khi lấy CPU profile bằng lệnh ở trên, đối chiếu các sample của chính lượt chạy với đường nối/copy, không tái dùng phần trăm từ một profile không còn artifact để kiểm tra.

Test output và benchmark hỗ trợ lựa chọn Builder cho workload này: format được giữ và allocation quan sát giảm. Nó chưa xác nhận service production nhanh hơn, vì chưa đo phần network, tải đồng thời hay trải nghiệm người dùng.

Tối ưu hóa Dựa trên Hồ sơ Vận hành (PGO)

Từ Go 1.20 và hoàn thiện ở các phiên bản gần đây, Go hỗ trợ kỹ thuật Tối ưu hóa Dựa trên Hồ sơ (Profile-Guided Optimization - PGO).

Trong quy trình biên dịch thông thường, compiler chỉ có cái nhìn tĩnh về mã nguồn, không biết nhánh rẽ nào trong câu lệnh `if` hay phương thức nào của `interface` được gọi thường xuyên nhất trong thực tế. Với PGO, kỹ sư thu thập một tệp CPU profile đại diện (`default.pgo`) từ hệ thống production đang chịu tải thực. Trình biên dịch nạp hồ sơ này vào pass phân tích để:

Một là, profile có thể giúp compiler ưu tiên một số cơ hội inlining ở đường nóng. Heuristic, ngưỡng và kết quả không phải API ổn định; hãy xem diagnostic hay benchmark của đúng version thay vì mặc định một hàm chắc chắn sẽ được inline.

Hai là, profile có thể mở ra cơ hội devirtualization khi compiler chứng minh được điều kiện phù hợp. Đó là tối ưu hóa có thể có, không phải lời hứa rằng mọi interface call nóng sẽ thành lời gọi tĩnh hay luôn inline xuyên package.

Tuy nhiên, PGO chỉ đáng tin khi profile đại diện cho workload muốn tối ưu. Ưu tiên profile từ production; khi không thể, staging hoặc benchmark đủ đại diện cũng có thể dùng. Microbenchmark hẹp thường mô tả quá ít chương trình để làm profile PGO tốt; hãy giữ nó để kiểm contract nhỏ. Dù dùng nguồn nào, vẫn phải đo lại kết quả end-to-end.

Kỷ luật Tối ưu hóa Kỹ thuật

Với một đề xuất tối ưu hóa, bốn bước sau giữ câu hỏi và bằng chứng đi cùng nhau:

Một là, xác định khối lượng công việc đại diện phản ánh đúng bài toán thực tế.

Hai là, thiết lập kiểm thử đơn vị để cố định hợp đồng kết quả đầu ra không bị biến dạng.

Ba là, sử dụng benchmark, escape analysis và profiler để thu thập số liệu định lượng trước khi can thiệp mã nguồn.

Bốn là, đối chiếu lại số liệu sau khi sửa đổi; nếu mức cải thiện không đủ lớn để bù đắp cho độ phức tạp gia tăng của mã nguồn, kiên quyết giữ lại giải pháp đơn giản và dễ bảo trì nhất.

1. Go Team. Package testing, phần Benchmarks và `B.Loop`. pkg.go.dev/testing
2. Go Team. Diagnostics, phần Profiling và Tracing. go.dev/doc/diagnostics
3. Go Team. Command trace. go.dev/cmd/trace
4. Go Team. Go Garbage Collector Guide. go.dev/doc/gc-guide
5. Go Team. Profile-guided optimization. go.dev/doc/pgo
6. Go Team. Package `runtime/pprof`, heap và goroutine profiles, Go 1.27.1. pkg.go.dev/runtime/pprof@go1.27.1#Profile
7. Go Team. Package `runtime`, `MemStats`, `MemProfile`, `MemProfileRate`, `NumGoroutine` và `KeepAlive`, Go 1.27.1. pkg.go.dev/runtime@go1.27.1#MemStats; pkg.go.dev/runtime@go1.27.1#MemProfile
8. Go Team. Package `context`, lifecycle của derived context, Go 1.27.1. pkg.go.dev/context@go1.27.1

Chương 11 — Lần theo một request HTTP

Ở log, một HTTP call thường chỉ còn lại một dòng: `GET https://api.example/... 1.8s`. Dòng ấy có ích, nhưng nó che gần như toàn bộ câu chuyện. 1.8 giây có thể là DNS, chờ một connection rảnh, TCP dial, TLS handshake, server xử lý, đọc response body, proxy, hoặc deadline của chính caller. Nếu chỉ có một timeout lớn và một log cuối cùng, ta biết request thất bại nhưng chưa biết cần sửa lớp nào.

Mô hình tinh thần của chương này là: **một HTTP request là chuỗi hợp đồng theo từng chặng; một chặng chỉ xuất hiện khi connection và policy khiến nó cần thiết**. Request dùng connection idle có thể không DNS, không dial, không TLS lần nữa. Một request mới có thể gặp các chặng ấy. Vì vậy sơ đồ dưới đây là bản đồ khả năng, không phải lời hứa về một thứ tự máy móc cho mọi lần gọi.

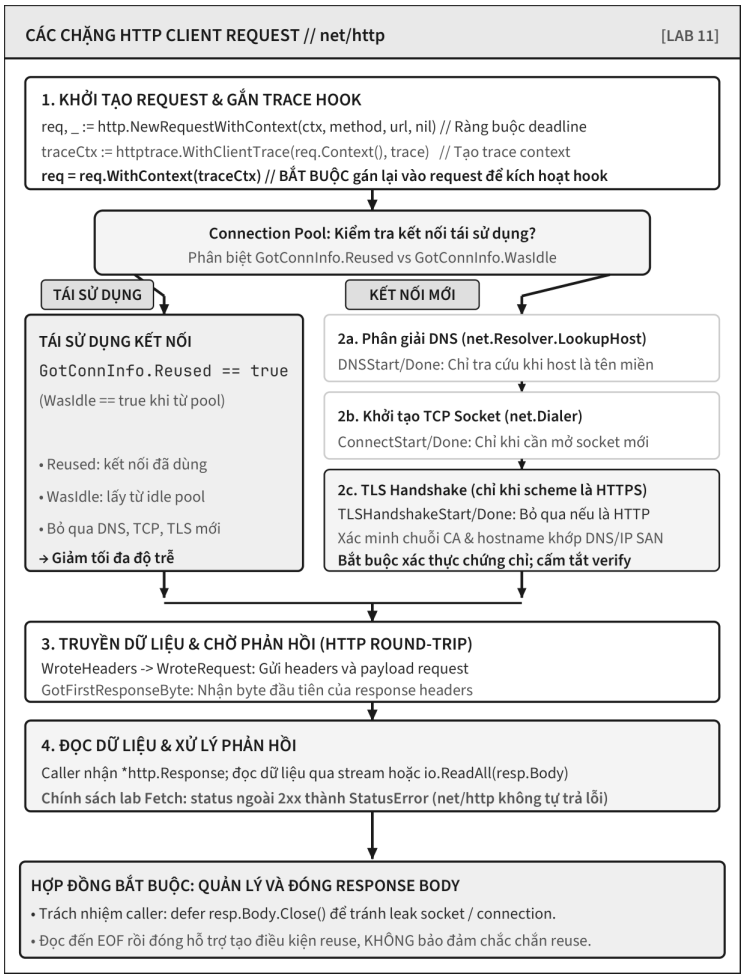
Bắt đầu bằng dấu vết thay vì API

`net/http/httptrace` gắn hook vào context của một outgoing request. Nó cho phép người điều tra đánh dấu các sự kiện như bắt đầu/kết thúc DNS, dial, TLS handshake, lấy được connection, ghi request và nhận byte response đầu tiên. Hook có thể chạy từ nhiều goroutine và một số có thể đến sau khi request đã complete hoặc failed, nên recorder phải đồng bộ nếu anh gom event vào cùng một slice.

```
type event struct {
    name string
    at   time.Duration
}

started := time.Now()
trace := &httptrace.ClientTrace{
    DNSStart: func(httptrace.DNSStartInfo) {
        record("dns start", time.Since(started))
    },
    GotConn: func(info httptrace.GotConnInfo) {
        name := fmt.Sprintf("conn reused=%t", info.Reused)
        record(name, time.Since(started))
    },
    GotFirstResponseByte: func() {
        record("first response byte", time.Since(started))
    },
}
traceCtx := httptrace.WithClientTrace(req.Context(), trace)
req = req.WithContext(traceCtx)
```

Điểm đáng chú ý là `GotConnInfo.Reused`, không phải chỉ timestamp. Nó tách một request dùng connection sẵn có khỏi request phải mở đường mới. Trace là dữ liệu quan sát của **một round trip**; nó không tự bao phủ chuỗi redirect và không thay thế log ngữ cảnh nghiệp vụ. Gắn request ID, target đã được chuẩn hóa và outcome vào log của anh; dùng trace khi một latency cần được bóc ra theo chặng.



Hình 23 — Bản đồ khái niệm của một request client. Nhánh “connection idle” bỏ qua DNS, TCP và TLS mới; response body vẫn là tài nguyên caller phải hoàn tất vòng đời.

Tên, địa chỉ, connection và danh tính không phải một thứ

Tên miền là input ở mức application. `net.Resolver.LookupHost` hỏi resolver cục bộ và trả về một danh sách địa chỉ cho host. Resolver thực tế thay đổi theo hệ điều hành, cấu hình mạng và build; trên Unix, tài liệu `net` mô tả cả pure-Go resolver lẫn đường đi qua C library trong một số điều kiện. Đừng dạy chương trình rằng “Go luôn tự gửi một DNS packet đến đâu đó”. Hãy ghi nhận lỗi DNS là lỗi của lớp resolution, cùng với host và deadline, rồi để configuration mạng là một boundary riêng.

Một địa chỉ khả dụng chưa phải identity đã được xác thực. Khi URL là HTTPS và cần một connection mới, TLS handshake thương lượng kết nối bảo mật. `tls.Config.ServerName` được dùng để xác minh hostname trên certificate, đồng thời thường được đưa vào handshake để hỗ trợ virtual hosting khi nó không phải IP. Đặt `InsecureSkipVerify=true` để “sửa

certificate error” là bỏ xác minh chain và hostname mặc định, không phải sửa network. Mặc định false giữ bước xác minh ấy; nếu dùng callback để xác minh tùy chỉnh thì phải tự giữ contract tương ứng. Với CA nội bộ, cấu hình root CA và hostname policy có chủ đích thay vì bỏ xác minh.

TCP, TLS và HTTP là các lớp khác nhau. Connection được tạo tới địa chỉ/port; TLS xác minh peer và thương lượng; HTTP chạy request/response bên trên. Với HTTP/2, nhiều request có thể cùng chia một connection. Vì thế “một request = một TCP connection” là mental model sai ngay từ đầu.

Chứng chỉ hợp lệ, vì sao thao tác vẫn phải bị từ chối?

Một caller có client certificate do CA nội bộ ký và TLS handshake thành công. Nó gọi `POST /write` kèm `X-Role: admin`. Service có được thực hiện không? Nếu câu trả lời chỉ dựa vào “đã mTLS” thì ta vừa biến quyền được vào kênh truyền thành quyền làm mọi việc trong kênh ấy.

Lab `labs/part11-request-path/identity` tạo CA, server certificate và client certificate mới trong runtime bằng `crypto/x509`. Listener chỉ ở loopback; không có private key ghi vào repository, không dùng trust store hệ điều hành và không bỏ qua verification. Client kết nối địa chỉ loopback nhưng đặt `ServerName=service.test`, đúng DNS SAN của server certificate. Địa chỉ để quay số và tên cần xác minh là hai input khác nhau; Common Name không thay SAN trong phép kiểm hostname này.

Mở `TestCertificateBoundaries` và dự đoán từng denial trước khi chạy. Client phải từ chối chain không thuộc CA đã tin và SAN không khớp. Server dùng `RequireAndVerifyClientCert`, từ chối thiếu chứng chỉ, CA client lạ, chứng chỉ hết hạn và chứng chỉ có mục đích sử dụng không phù hợp. Test đi qua handshake TLS thật của thư viện chuẩn, rồi kiểm tra không có mutation. Nó không dùng một struct giả để tự gắn nhãn authenticated.

```
cd labs/part11-request-path
go test -count=1 -v ./identity
go test -race ./identity
```

Sau handshake, `Authorize` chỉ lấy URI SAN từ leaf trong `VerifiedChains`, không lấy role từ header hoặc từ certificate chưa được xác minh. Policy của fixture cho `urn:go-book:reader` đọc `/read`; chỉ `urn:go-book:operator` được ghi `/write`. Certificate được CA tin ký nhưng mang identity lạ hoặc không có identity vẫn nhận 403. TLS không chọn policy này thay application. Chương 12 đặt nó trước handler có side effect; Chương 28 còn cần ràng buộc quyền với action, target và phê duyệt.

Tự phản nghiệm. Bật `RELIABILITY_MUTANT=trust_equals_role` rồi chạy `TestAuthorizationContract`. Biến thể lỗi cấp mọi action cho bất kỳ verified chain nào; cùng test phải đỏ vì reader ghi được và bộ đếm mutation tăng. Không chữa bằng cách tắt certificate verification: điều đó phá thêm boundary danh tính, không giải quyết authorization.

Lab chọn TLS 1.3 làm policy cục bộ, không gọi đây là yêu cầu của mọi deployment. Nó chưa kiểm tra rotation, revocation, proxy termination hay identity provider. mTLS là một lựa chọn khi có bài toán phân phối và quản lý certificate phù hợp; không phải điều kiện bắt buộc của mọi MCP transport.

Deadline có phạm vi, không phải một con số trang trí

`http.NewRequestWithContext` ràng context vào toàn bộ vòng đời outgoing request: lấy connection, gửi request, đọc header và đọc body response. Đó là deadline gần nhất với mục đích caller: caller mất kiên nhẫn thì cả request nên biết lý do dừng.

```
ctx, cancel := context.WithTimeout(parent, 2*time.Second)
defer cancel()

req, err := http.NewRequestWithContext(
    ctx,
    http.MethodGet,
    rawURL,
    nil,
)
if err != nil {
    return err
}
resp, err := client.Do(req)
```

`http.Client.Timeout` cũng là giới hạn end-to-end cho request do client đó thực hiện, kể cả đọc body. `http.Transport` có các chốt hẹp hơn: `TLSHandshakeTimeout`, `ResponseHeaderTimeout`, các giới hạn connection và dialer deadline. Chúng trả lời các câu policy khác nhau, nên không nên đồng loạt gán một con số "5 giây" vào mọi field.

Bảng 10 — Timeout phải gắn với một khả năng chờ cụ thể

Chốt	Phạm vi	Câu hỏi thiết kế
Context của request	Toàn bộ ý nghĩa của request với caller	Caller còn cần kết quả sau bao lâu?
`Client.Timeout`	End-to-end cho call qua client	Client dùng chung có một ngân sách chung hợp lý không?
`TLSHandshakeTimeout`	Handshake TLS mới	Có cần chặn handshake treo mà vẫn cho body dài?
`ResponseHeaderTimeout`	Sau khi gửi đủ request, chờ response headers	Server phải bắt đầu phản hồi trong bao lâu?
`MaxConnsPerHost`	Tổng connection theo host; dial có thể block khi đầy	Bao nhiêu concurrency là quota có chủ đích?

Một timeout không trả lời retry có an toàn hay không. `Transport` có retry nội bộ hạn chế cho một số lỗi mạng trên connection đã dùng thành công, khi request idempotent và body có thể gửi lại; đó là documented behavior của implementation library, không phải lời hứa business. Nếu code của anh

tự retry POST, anh phải định nghĩa idempotency key, retry budget, deadline chung và hậu quả khi server đã xử lý nhưng client chưa nhận response. “Network error” không đồng nghĩa operation chưa xảy ra.

Response body là khoản nợ của caller

Client.Do trả Response với Body không nil khi không có error. Caller phải đóng nó. Đọc đến EOF rồi đóng tạo điều kiện reuse persistent HTTP/1.x connection, không hứa request kế tiếp sẽ dùng nó. Từ Go 1.27, khi Close, implementation HTTP/1 tự drain phần chưa đọc đến một giới hạn bảo thủ để hỗ trợ reuse. Điều đó không có nghĩa đóng một body bất kỳ sẽ đọc hết hoặc giữ được connection; không chuyển mẹo implementation này thành ownership policy. Helper hoặc trả *http.Response và document caller phải close, hoặc đọc body, close nó rồi trả một value độc lập.

Lab của chương chọn policy thứ hai. Contract bắt đầu bằng test đỏ: Fetch phải tạo request bằng context caller, đọc toàn bộ body, đóng body, và đổi status ngoài 2xx thành StatusError. Không dùng http.DefaultClient; test đưa một transport giả vào client để quan sát request mà không hề chạm network thật.

```
cd labs/part11-request-path
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test -tags traceexercise ./exercise
go test -race -tags traceexercise ./exercise
go test ./fixed
```

Sau Fetch, có một việc độc lập thứ hai: biến các hook trace thành dữ liệu có thể đọc mà không tạo race mới. Mở exercise/trace_test.go; test không yêu cầu mở DNS hay server thật. Nó đưa các hook DNSStart, GotConn và GotFirstResponseByte vào một recorder, rồi gọi GotConn đồng thời từ nhiều goroutine. Anh tự chọn cấu trúc dữ liệu, cơ chế đồng bộ và tên event, nhưng Snapshot phải trả bản copy để caller không sửa lịch sử của recorder. Đây là chương trình tái hiện tối thiểu cho đúng lời cảnh báo ở

đầu chương: `httptrace` cho phép hook chạy đồng thời; trace dùng để điều tra request không được tự tạo một data race trong công cụ điều tra.

Thứ tự event trong một snapshot tuần tự có thể là bằng chứng tốt cho một request cụ thể. Khi hook chạy đồng thời, recorder chỉ hứa giữ event đã nhận mà không bịa một thứ tự nhân quả không tồn tại. Hãy dùng thời điểm để hỗ trợ điều tra, rồi đối chiếu với `GotConnInfo.Reused`, deadline và kết quả; đừng suy ra riêng từ danh sách event rằng DNS, dial hay server luôn là chặng chậm nhất.

Khi caller cần stream một response lớn, policy lab không còn đúng: không nên `io.ReadAll` chỉ để "cho đơn giản". Khi ấy API phải trả stream và ownership của `Close` quay về caller, giống bài về I/O ở Chương 7. Một API nhỏ nhưng mơ hồ về body thường tạo connection leak lặng lẽ hơn một API có thêm một dòng document.

Client và `Transport` an toàn khi dùng đồng thời; tài liệu `net/http` khuyên tạo rồi reuse chúng vì transport có state như connection cache. Reuse không có nghĩa dùng một global vô danh. Hãy đóng gói một client đã cấu hình đúng policy của service, truyền nó vào boundary cần gọi mạng, và giữ transport cũng như deadline có thể kiểm thử được.

Thí nghiệm local: request thứ hai có thật sự dùng connection cũ?

`labs/part11-request-path/fixed/reuse_test.go` dùng `httptest.Server`, một `http.Transport` mới và `httptrace`. Không có DNS hoặc TLS trong thí nghiệm HTTP cục bộ này; đó là chủ ý để cô lập pool idle connection. Hai request tuần tự đọc hết rồi đóng body. Test kiểm tra `GotConnInfo.Reused` là `false` cho request đầu, `true` cho request sau. Nó không chứng minh mọi request production sẽ reuse: server có thể đóng connection, body có thể chưa được trả về pool, quota hay HTTP/2 có thể đổi đường đi. Nó chỉ cho ta bằng chứng tái lập được cho mệnh đề hẹp: sau khi transport có một idle connection phù hợp, round trip kế tiếp có thể bỏ qua việc tạo connection mới.

```
cd labs/part11-request-path
go test -run TestTransportReusesIdleConnection ./fixed
go test -race ./fixed
```

Khi nhìn một request chậm, đừng hỏi "API nào chậm?". Hãy hỏi connection có được reuse không, timeout nào đã hết, DNS/TLS/first byte nằm ở đâu trên trace, body đã được trả nợ chưa, và retry có giữ nguyên nghiệp vụ không. Những câu ấy biến một dòng log 1.8 giây thành một điều tra có thể kết thúc.

1. Go Team. Package `net`, phần Name Resolution và `Resolver.LookupHost`. pkg.go.dev/net
2. Go Team. Package `crypto/tls`, phần `Config.ServerName` và `InsecureSkipVerify`. pkg.go.dev/crypto/tls
3. Go Team. Package `net/http`, phần Clients and Transports, `Request`, `Client` và `Transport`. pkg.go.dev/net/http
4. Go Team. Package `net/http/httptrace`. pkg.go.dev/net/http/httptrace
5. Go Team. Go 1.27 Release Notes, net/http: bounded drain khi đóng HTTP/1 response body. go.dev/doc/go1.27
6. Go Team. `crypto/tls` và `crypto/x509`, Go 1.27.1: `RequireAndVerifyClientCert`, `VerifiedChains`, `ServerName`, `Certificate.Verify` và `VerifyHostname`. pkg.go.dev/crypto/tls@go1.27.1; pkg.go.dev/crypto/x509@go1.27.1

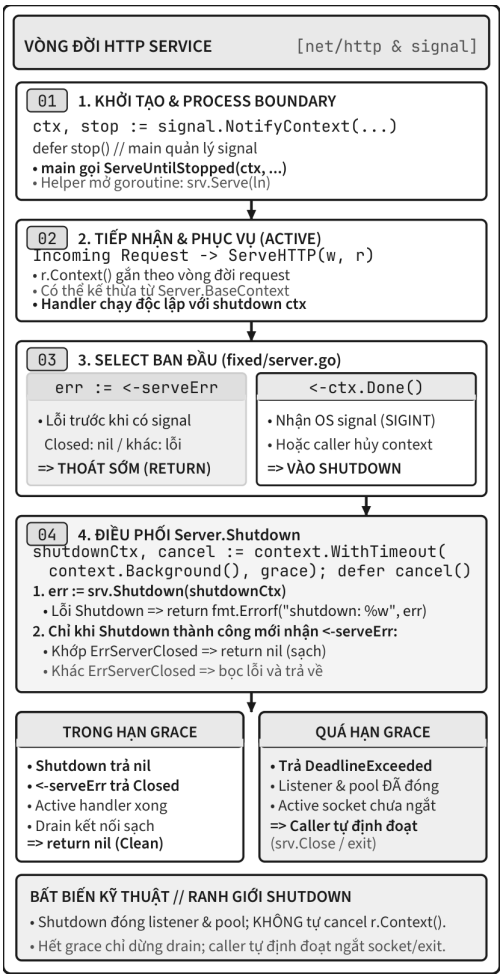
Chương 12 — Một service sống và tắt thế nào

Một service nhỏ thường bắt đầu bằng `http.ListenAndServe(":8080", handler)`. Dòng đó đủ để mở port, nhưng chưa đủ để nói service đang hứa điều gì với người gọi, với deploy system, hay với người vận hành. Khi process nhận tín hiệu dừng, nếu `main` kết thúc ngay thì request đang xử lý bị cắt. Khi một client gửi header chậm, nếu server không có policy đọc header thì connection có thể giữ tài nguyên lâu hơn dự kiến. Khi handler nhận JSON lạ và cứ cố đoán, boundary giữa input không tin cậy và state nội bộ đã biến mất.

Mô hình tinh thần của chương này là: **service là một boundary biến input không tin cậy thành công việc có giới hạn, rồi rút lui theo một lifecycle có thể quan sát.** Routing, middleware, validation và shutdown chỉ đáng đưa vào khi chúng làm boundary này rõ hơn. Chúng không phải danh sách framework feature.

Một deploy bị cắt ngang cho thấy thiếu contract nào

Hãy tưởng tượng `/v1/checks` nhận một target, lưu một yêu cầu kiểm tra, rồi trả `201`. Trong một lần deploy, load balancer chuyển traffic đi nhưng process cũ nhận tín hiệu dừng ngay giữa lúc đang ghi response. Có ba trạng thái cần phân biệt: service còn nhận connection mới, đang để request đã nhận đi tới chỗ kết thúc, hay đã dừng. `Server.Shutdown` của `net/http` đóng listener, đóng idle connection, rồi chờ active connection trở về idle và đóng. Nó không tự cancel context của handler đang active. `r.Context()` của incoming request có lifecycle riêng: nó bị cancel khi client disconnect, request HTTP/2 bị hủy, hoặc khi `ServeHTTP` trả về — không phải chỉ vì `Shutdown` vừa được gọi. Nó không tự chờ connection bị hijack, như WebSocket; những connection dài hạn cần lifecycle riêng.



Hình 24 — Lifecycle khái niệm khi service graceful shutdown. Deadline của shutdown là policy vận hành: nó giới hạn thời gian chờ, không biến mọi request thành thành công.

Một hàm chạy server có thể giữ contract ấy trong một nơi thay vì để `main` thoát vì mọi error đều trông giống nhau. Hãy đọc phần đầu và phần shutdown dưới đây như hai đoạn liên tiếp của cùng một function; tách chúng ra cũng giúp lifecycle hiện rõ hơn trên trang:

```
func ServeUntilStopped(
    ctx context.Context,
    srv *http.Server,
    ln net.Listener,
    grace time.Duration,
) error {
    if srv == nil {
        return errors.New("HTTP server is required")
    }
    if ln == nil {
        return errors.New("listener is required")
    }
    if grace <= 0 {
        return errors.New("shutdown grace must be positive")
    }

    serveErr := make(chan error, 1)
    go func() { serveErr <- srv.Serve(ln) }()

    select {
    case err := <-serveErr:
        if errors.Is(err, http.ErrServerClosed) {
            return nil
        }
        return fmt.Errorf("serve: %w", err)
    case <-ctx.Done():
    }
```

Đến đây server đã nhận yêu cầu dừng, nhưng `ctx` không nên bị dùng trực tiếp làm shutdown context: nó đã canceled. Ta tạo một ngân sách thời gian mới để `Shutdown` có cơ hội drain request đang chạy:

```
    base := context.Background()
    shutdownCtx, cancel := context.WithTimeout(base, grace)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil {
        return fmt.Errorf("shutdown: %w", err)
    }
    err := <-serveErr
    if err == nil {
        return errors.New("server stopped without an error")
    }
    if !errors.Is(err, http.ErrServerClosed) {
        return fmt.Errorf("serve after shutdown: %w", err)
    }
    return nil
}
```

Ba guard đầu không phải defensive programming vô định. `srv`, `ln` và `grace` là ba điều kiện đầu vào của lifecycle helper; nếu một trong chúng thiếu, chạy `Serve` chỉ để nhận panic hoặc treo không giúp caller sửa cấu hình. `shutdownCtx` được tạo từ `context.Background()` có chủ đích. Root context đã bị cancel để yêu cầu dừng; nếu lấy deadline shutdown trực tiếp từ nó, deadline sẽ bị cancel ngay trước khi `Shutdown` có cơ hội drain. `grace` phải đến từ SLO, thời gian deploy và loại request của service, không phải một con số copy từ ví dụ. Sau deadline, `Shutdown` trả error; quyết định tiếp theo - log, alarm, force close hay để supervisor xử lý - là policy cần được viết rõ ở boundary vận hành.

Từ signal của process sang context của server

`ServeUntilStopped` đã nhận một `context.Context`, nên `main` chỉ cần nối process lifecycle vào input đó. Trên Unix-like host, `signal.NotifyContext` biến `SIGINT` hoặc `SIGTERM` thành cancellation; `defer stop()` hủy đăng ký signal handling khi `main` rời đi. Cho đến khi `stop()` được gọi, signal vẫn bị chuyển hướng theo policy này; nếu hệ thống muốn interrupt thứ hai quay lại default hoặc force-exit, đó phải là một quyết định thiết kế có chủ đích. Sau signal đầu tiên, `ServeUntilStopped` đi vào `Shutdown` với `grace` đã chọn thay vì cắt request ngay.

```
ctx, stop := signal.NotifyContext(
    context.Background(),
    os.Interrupt,
    syscall.SIGTERM,
)
defer stop()

if err := ServeUntilStopped(
    ctx,
    srv,
    ln,
    15*time.Second,
); err != nil {
    return err
}
```

Đây là bridge ở process boundary, không phải permission cho handler bỏ qua `r.Context()`: `Shutdown` chỉ ngừng nhận request mới và chờ request đang chạy; nó không tự cancel `r.Context()` của những handler đó. Handler vẫn phải truyền `r.Context()` vào database hay outbound HTTP để phản ứng với lifecycle riêng của incoming request.

Nếu policy của service là signal cũng phải cancel handler đang chạy, việc đó cần được nối một cách tường minh, chẳng hạn bằng application context làm `Server.BaseContext` rồi cancel context ấy khi nhận signal:

```
appCtx, cancelApp := context.WithCancel(context.Background())
srv.BaseContext = func(net.Listener) context.Context {
    return appCtx
}
// Policy đã chọn khi nhận signal:
cancelApp()
```

Đó là một policy khác với graceful drain thuần túy: handler nào quan sát context có thể dừng trước khi hoàn tất công việc đang phục vụ. Chọn nó khi bounded shutdown đáng giá hơn việc để request đang chạy hoàn thành. Trên Windows, `os.Interrupt` là signal portable; service host và process

supervisor phải được kiểm chứng theo môi trường triển khai trước khi giả định `SIGTERM` có cùng đường đi.

Operation đã xong nhưng context con vẫn còn sống

Một handler đã trả kết quả không đồng nghĩa mọi tài nguyên nó tạo đều được trả đúng lúc. Trong lab `part10-resource-retention`, `DoTimeout` tạo child có deadline một giờ dưới một parent có thể hủy nhưng sống lâu. Callback trả về ngay. Anh đoán child sẽ hết hạn cùng callback, cùng handler, hay theo đường hủy của chính context? Context không biết callback đã làm xong. Nếu owner bỏ quên CancelFunc, child chưa bị hủy chỉ vì hàm return.

Theo contract của `context`, gọi cancel hủy child cùng các descendant, bỏ quan hệ giữ child từ parent và dừng timer liên quan. Trong source Go 1.27.1, `cancelCtx.propagateCancel` đăng ký child trong tập `children` của parent phù hợp; `timerCtx.cancel` tháo quan hệ đó và dừng timer. Đây là cách triển khai kiểm chứng trên version này, không phải field để application truy cập. Parent còn sống, deadline còn xa và không ai cancel là đủ để thí nghiệm giữ các child ngoài thời gian operation cần chúng. GC không thay được quyết định kết thúc này.

Đặt trách nhiệm dọn dẹp ngay tại nơi sở hữu một lần gọi:

```
func DoTimeout(
    parent context.Context,
    budget time.Duration,
    work func(context.Context) error,
) error {
    ctx, cancel := context.WithTimeout(parent, budget)
    defer cancel()
    return work(ctx)
}
```

`defer` bảo vệ cả đường thành công lẫn error. Nếu tạo nhiều child trong một vòng lặp của hàm sống suốt service rồi defer mọi cancel ở hàm ngoài, việc dọn dẹp bị đẩy tới lúc service kết thúc. Tách mỗi operation thành hàm có

scope ngắn như trên, hoặc cancel ở đúng cuối từng vòng và mọi đường thoát. Cũng đừng trả child ra cho công việc còn chạy rồi cancel ngay khi hàm tạo nó return: cần bàn giao CancelFunc và trách nhiệm chờ hoàn tất cho owner mới, không áp dụng defer một cách máy móc.

Test lab ghi nhận child mà callback đã nhận. Sau bản sửa trả về, `child.Err()` là `context.Canceled`; sau bản lỗi, child vẫn chưa có error, rồi nhận cancellation khi test hủy parent. Một test khác dùng deadline đã qua để xác nhận child có thể tự hết hạn dù owner quên cancel. Các test kiểm tra lifecycle công khai, không đọc field nội bộ hay đòi một mức giảm RAM.

`go vet` phát hiện trường hợp phổ biến như bỏ CancelFunc bằng _; fixture `testdata/lostcancel` cố ý cho diagnostic ấy. Nhưng bản lỗi chuyển cancel cho hàm `ForgetCancel` không làm gì: kiểm tra cục bộ có thể xem trách nhiệm đã được bàn giao trong khi thực tế không ai thực hiện. Vì vậy vet sạch chưa chứng minh ownership đúng. Đọc đường gọi và test thời điểm child kết thúc là hai bằng chứng bổ sung cần thiết.

Không phải mọi lần thiếu cancel đều là leak vĩnh viễn. Parent được hủy hoặc deadline đến thì child cũng kết thúc; với `WithCancel(context.Background())` không có timer hay parent đăng ký child, bỏ mọi reference tới child không tự tạo một chuỗi giữ từ parent như fixture này. Một child có value lớn, descendant, timer hoặc công việc đang chờ có chi phí khác với child nhỏ. Cần xác định đường giữ cụ thể và thời gian bị giữ thừa. CancelFunc cũng không chờ callback ngừng: câu hỏi chờ `done` của Chương 9 vẫn còn nguyên.

Incoming `r.Context()` do HTTP server sở hữu; handler không nhận CancelFunc để tự hủy context gốc ấy. Handler chịu trách nhiệm cho child nó tạo để gọi downstream. Policy hủy active request khi process shutdown vẫn là quyết định riêng đã nêu ở trên, không được dùng thiếu cancel trong một operation để sửa sai mental model về `Server.Shutdown`.

Handler là cửa kiểm tra, không phải nơi "cố hiểu" input

Trong lab, `POST /v1/checks` là một boundary nhỏ. Nó nhận JSON chỉ có `target`, chấp nhận `http` hoặc `https` với host không rỗng, chuyển value hợp lệ sang `Store`, và không tiết lộ lỗi nội bộ của store cho client. Lab chưa thực hiện probe outbound: đó sẽ là một quyết định có rủi ro SSRF và quota, nên không được lén nhét vào một handler minh họa.

> **Dừng để dự đoán.** Xét một handler HTTP đọc payload qua `json.Decoder`:
>
> ~~~go
> decoder := json.NewDecoder(r.Body)
> var input Check
> err := decoder.Decode(&input)
> ~~~
>
> Nếu client gửi body gồm một document hợp lệ rồi nối thêm dữ liệu thừa như `{"target":"https://example.com"}{"debug":true}`, lệnh `decoder.Decode(&input)` có báo lỗi không, và phần dữ liệu thừa phía sau sẽ đi về đâu? Làm thế nào để chứng minh stream đã thực sự kết thúc trước khi chuyển giao dữ liệu sang `Store`?

Đáp án — chỉ đọc sau khi đã tự làm Lệnh `decoder.Decode(&input)` đọc và giải mã thành công object đầu tiên vào biến `input` mà không trả về lỗi nào, bởi vì `json.Decoder` được thiết kế để xử lý luồng stream và dừng lại ngay khi hoàn tất một JSON value hợp lệ. Lúc này, phần dữ liệu thừa phía

sau không nhất thiết còn nguyên vẹn trong `r.Body`. Do `json.Decoder` duy trì một bộ đệm đọc trước (read buffer) nội bộ, các byte của object thứ hai có thể đã được nạp sẵn vào bộ đệm của decoder hoặc vẫn còn nằm lại trên stream của request. Nếu handler dừng lại ở lần gọi đầu tiên rồi tiếp tục xử lý nghiệp vụ, dữ liệu thừa sẽ bị bỏ qua và ranh giới kiểm soát input bị phá vỡ.

Để chứng minh stream không còn dữ liệu thừa ngoài khoảng trắng, handler thực hiện thêm một lần giải mã thứ hai vào một struct rỗng (`var extra struct{}`). Khi stream đã kết thúc đúng chuẩn, bộ giải mã bắt buộc phải trả về chính xác lỗi `io.EOF`. Mọi dữ liệu còn sót lại trong bộ đệm decoder hoặc trên stream mạng đều khiến lần gọi thứ hai trả về một kết quả khác `io.EOF`, tạo bằng chứng rõ ràng để handler từ chối request bằng mã 400 Bad Request trước khi chuyển giao việc cho `Store`.

Dưới đây là cài đặt kiểm định ranh giới hoàn chỉnh của handler trong lab:

```
func (a app) createCheck(
    w http.ResponseWriter,
    r *http.Request,
) {
    if r.Method != http.MethodPost {
        methodNotAllowed(w)
        return
    }

    r.Body = http.MaxBytesReader(w, r.Body, maxRequestBody)
    decoder := json.NewDecoder(r.Body)
    decoder.DisallowUnknownFields()

    var input Check
    err := decoder.Decode(&input)
    if err != nil || validateTarget(input.Target) != nil {
        badRequest(w)
        return
    }
    var extra struct{}
    if err := decoder.Decode(&extra); !errors.Is(err, io.EOF) {
        badRequest(w)
        return
    }
    check, err := a.store.Create(r.Context(), input)
    if err != nil {
        internalError(w)
        return
    }
    writeJSON(w, http.StatusCreated, check)
}
```

Lần `Decode` thứ hai không lấy một check khác. Nó chỉ chứng minh sau value đầu tiên còn whitespace và `io.EOF`; nếu còn JSON value, request bị từ chối trước khi chạm `Store`. Đoạn code vẫn không phải decoder hoàn chỉnh cho mọi API: endpoint nhận upload chẳng hạn không thể dùng một giới hạn 4 KiB như vậy. Ý chính là limit, schema và validation phải xuất hiện trước khi input trở thành công việc nội bộ; đừng parse vô hạn rồi mới hỏi request có hợp lệ hay không.

`http.MaxBytesReader` là một quota ở boundary body. `Decoder.DisallowUnknownFields` chọn strict schema, hữu ích khi client gửi JSON nhầm field mà ta không muốn lặng lẽ bỏ qua. Strictness là lựa chọn compatibility: một API mở rộng có thể cần versioning hoặc field policy khác. Nhưng "bỏ qua mọi field lạ" không được là phản xạ vô thức khi input điều khiển hành vi có chi phí.

Lab bắt đầu bằng các test đỏ cho request hợp lệ, method sai, JSON chứa field lạ, document thứ hai và target sai. Tự viết handler trước khi mở `fixed/`:

```
cd labs/part12-service-lifecycle
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test -tags lifecycleexercise ./exercise
go test -race -tags lifecycleexercise ./exercise
go test ./fixed
```

Test handler còn đặt hai JSON value nối nhau để tách hai failure dễ lẫn: document đầu hợp lệ không cho phép phần còn lại trở thành input bí mật bị bỏ qua. Sau khi handler xanh, test `lifecycleexercise` đưa listener cục bộ và context cancel vào `ServeUntilStopped`; contract là listener phục vụ được request trước cancellation, server return sau shutdown, và precondition sai bị trả thành error thay vì biến thành panic. Test dùng `httptest` hoặc listener cục bộ, nên kiểm chứng được contract HTTP mà không mở port ra mạng. `ResponseRecorder.Result()` là snapshot response sau khi handler đã chạy; đừng `DeepEqual` cả `http.Response`, chỉ assert status, header và body mà API đã hứa.

Routing và middleware là cách đặt luật ở đúng biên

Một router nhỏ có giá trị khi nhìn vào route table ta biết method nào vào handler nào. Với service còn ít endpoint, `http.ServeMux` và kiểm tra method rõ ràng thường tốt hơn một dependency lớn. Route pattern, authentication requirement và giới hạn body nên đọc được từ boundary; business function phía trong chỉ nhận value đã được kiểm tra và context của request.

Middleware hợp lý là luật áp dụng cho nhiều request: gắn request ID đã được server tạo, log outcome, tracing, authentication hoặc giới hạn rate. Nó không phải nơi để nhét business branching. Nếu middleware cần ghi status và số byte, wrapper `ResponseWriter` phải tôn trọng contract của `net/http`; một wrapper cẩu thả có thể làm hỏng optional interface hoặc ghi header sau body. Bắt đầu bằng middleware nhỏ mà anh có thể test, thay vì sao chép một "stack chuẩn" không ai còn đọc được.

Logging nên trả lời được câu hỏi vận hành: request ID, route/method, status, duration, outcome và lỗi đã phân loại. Nó không nên ghi password, bearer token, cookie hoặc toàn bộ body mặc định. Một error trả cho client là contract public; log nội bộ có thể giữ cause kỹ thuật với mức truy cập phù hợp. Hai thứ có cùng text thường là dấu hiệu boundary chưa rõ.

Timeout server phòng một kiểu áp suất khác

Timeout client ở chương trước bảo vệ người **gọi**. Server còn phải chọn giới hạn để bảo vệ resource của chính nó. `ReadHeaderTimeout` giới hạn thời gian đọc request header rồi reset deadline để handler tự quyết rate/deadline của body. `ReadTimeout` bao cả request body nhưng không cho handler policy riêng cho từng upload. `WriteTimeout` giới hạn response write; `IdleTimeout` giới hạn chờ request kế tiếp trên keep-alive connection. `MaxHeaderBytes` giới hạn header parser.

Bảng 11 — Server timeout là các policy khác nhau

Field	Bảo vệ điều gì	Cần quyết định cùng nó
`ReadHeaderTimeout`	Client gửi header quá chậm	Header hợp lệ cần bao lâu trong mạng thực?
`ReadTimeout`	Toàn bộ đọc request, gồm body	Endpoint upload có rate/lifetime riêng không?
`WriteTimeout`	Response write kéo dài	Streaming response có cần policy khác?
`IdleTimeout`	Keep-alive không gửi request tiếp	Bao lâu connection rảnh vẫn đáng giữ?
`MaxHeaderBytes`	Header bất thường lớn	Cookie/proxy hợp lệ chiếm bao nhiêu?

Không có bộ số chung đúng cho mọi service. Với timeout đọc/ghi/idle, zero có thể nghĩa là không đặt timeout theo documented behavior của `net/http`; `MaxHeaderBytes` có default riêng. Default vô thức vẫn là policy. Chọn workload, proxy, upload path và memory budget trước khi đặt number; rồi kiểm thử ingress thật khi có traffic.

`Server.Shutdown` không giết request active và cũng không tự cancel `r.Context()` của handler. Handler vẫn cần tôn trọng `r.Context()` khi gọi database, outbound client hoặc worker vì incoming request có lifecycle riêng. Chương 9 và 11 đã cho ta hai nửa còn lại: goroutine phải có đường thoát khi context bị hủy; client request phải nhận context của caller. Graceful shutdown chỉ thực sự graceful khi các boundary bên trong chịu trả lại quyền điều khiển.

Service đáng tin được đánh giá ở boundary: input bị giới hạn ở đâu, business code nhận gì, response/log tách public/private không, server ngừng nhận việc khi nào và request cũ được bao lâu để kết thúc. WebSocket cần một owner riêng. Trả lời được các câu ấy, service có lifecycle để vận hành thay vì chỉ `ListenAndServe`.

1. Go Team. Package `net/http`, phần Handler, Server fields, `Shutdown` và `ResponseWriter`. pkg.go.dev/net/http
2. Go Team. Package `net/http/httptest`, phần `ResponseRecorder` và test server. pkg.go.dev/net/http/httptest
3. Go Team. Package `os/signal`, phần `NotifyContext`. pkg.go.dev/os/signal
4. Go Team. Security Best Practices for Go Developers. go.dev/doc/security/best-practices
5. Go Team. Package `context`, Overview và `WithTimeout`, Go 1.27.1. pkg.go.dev/context@go1.27.1#WithTimeout
6. Go Team. Source `context/context.go`, `cancelCtx.propagateCancel` và `timerCtx.cancel`, tag go1.27.1. github.com/golang/go/blob/go1.27.1/src/context/context.go
7. Go Team. Analyzer `lostcancel`, kiểm tra CancelFunc và giới hạn phân tích tại lời gọi. pkg.go.dev/golang.org/x/tools/go/analysis/passes/lostcancel

Chương 13 — Một thay đổi hoặc không có gì

Một buổi trực, service trả `500` cho thao tác tắt một endpoint đang lỗi. Người trực tin rằng request thất bại nên thử lại. Lần đọc tiếp theo lại cho thấy endpoint đã bị tắt, nhưng audit log không có dòng nào nói ai đã làm việc đó. Đây không phải lỗi "quên ghi log" đơn thuần. Từ ngoài nhìn vào, hệ thống đang kể hai câu chuyện mâu thuẫn về cùng một thay đổi.

State bền vững khác với một value nằm trong RAM: nó sẽ được request sau, process khác, hoặc lần khởi động sau dùng làm sự thật. Vì vậy câu hỏi đầu tiên của persistence không phải là chọn ORM hay viết bao nhiêu SQL. Câu hỏi là: **sau khi một thao tác nhiều bước báo xong hoặc báo lỗi, những state nào được phép tồn tại?** Mental model của chương là một boundary: với các thay đổi cùng một ý nghĩa nghiệp vụ, người đọc state phải thấy phiên bản cũ hoặc phiên bản mới đã hoàn tất, không phải một đoạn giữa đường.

Một `if err != nil` không tự tạo atomicity

Giả sử API `Disable` cần làm hai việc: đổi `checks.enabled` thành `false`, rồi ghi event `disabled`. Đoạn code ngây thơ có vẻ cẩn thận vì đã kiểm tra error sau từng câu SQL:

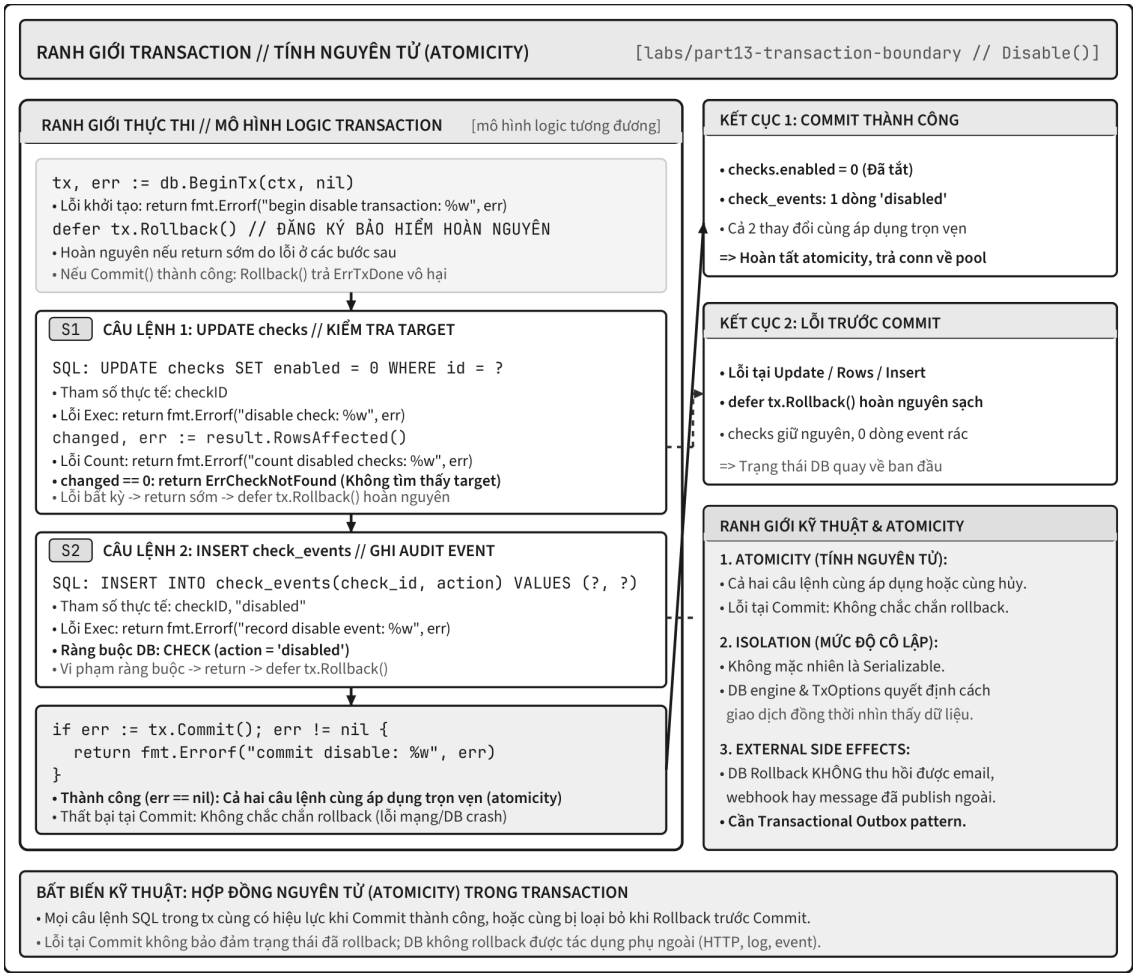
```
if _, err := db.ExecContext(ctx,
    "UPDATE checks SET enabled = 0 WHERE id = ?", checkID,
); err != nil {
    return err
}

if _, err := db.ExecContext(ctx,
    "INSERT INTO check_events(check_id, action) VALUES (?, ?)",
    checkID, "disabled",
); err != nil {
    return err
}
```

Nhưng hai lời gọi độc lập là hai boundary độc lập. Nếu `UPDATE` thành công còn `INSERT` lỗi vì constraint, hàm trả error thật, song database vẫn giữ

endpoint ở trạng thái tắt. Caller có lý do để thử lại; người điều tra có một state không có nguyên nhân. Không có dòng `if` nào quay ngược statement đầu chỉ vì statement sau thất bại.

Transaction là cách nói với database rằng các statement này thuộc về một đơn vị: hoặc tất cả được commit, hoặc không statement nào trong đơn vị được giữ lại. Nó không phải bùa chú cho mọi vấn đề consistency; nó chỉ làm lời hứa hẹp nhưng cực quan trọng ấy tại boundary database.



Hình 25 — Sơ đồ khái niệm về atomicity của thao tác `Disable`. Hình không khẳng định isolation level nào; database, driver và `TxOptions` mới quyết định chi tiết isolation hỗ trợ được.

> **Dừng để dự đoán:** nếu statement ghi event bị từ chối, test nào chứng minh endpoint vẫn `enabled`? Chỉ assert hàm trả error chưa đủ; state sau lỗi mới là điều contract phải khóa lại.

Thực hành: làm cho thất bại không để lại dấu vết nửa chừng

Lab này dùng SQLite trong memory qua `modernc.org/sqlite`, một driver thuần Go. Lý do chọn nó là để test chạy cục bộ, không cần Docker, credential hay network. `database/sql` vẫn là API chính của bài; driver chỉ là adapter nói được dialect và protocol của database cụ thể. SQLite ở đây không phải lời khuyên thay PostgreSQL cho mọi service: placeholder, concurrency và operational behavior của hai hệ khác nhau.

Mở `exercise/disable_test.go` trước. Test đã dựng schema và nêu API, nhưng chưa cho implementation. Anh cần viết `Disable` sao cho có đủ bốn contract:

Một là, `check` tồn tại được tắt và có đúng một event `disabled`.

Hai là, không có `check` thì trả `ErrCheckNotFound`, không tự tạo event.

Ba là, nếu event không thể ghi, `check` vẫn giữ `enabled = true`.

Bốn là, `nil` database là lỗi ở API boundary, không phải một panic muộn trong goroutine hay driver.

```
cd labs/part13-transaction-boundary
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test ./fixed
go test -race ./fixed
```

Đừng mở `fixed/` trước lượt tự làm đầu tiên. Contract đã nói rõ điều gì phải đúng, còn thiết kế thuộc về anh: transaction được mở ở đâu, cleanup đặt thế nào, statement nào phải chạy qua `tx` thay vì `db`, và `RowsAffected` được dùng ra sao để phân biệt không tìm thấy check với cập nhật thành công.

```
// Đáp án tham chiếu: một transaction, một commit.
func Disable(
    ctx context.Context,
    db *sql.DB,
    checkID int64,
) error {
    if db == nil {
        return errors.New("database is required")
    }
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        return fmt.Errorf("begin disable transaction: %w", err)
    }
    defer tx.Rollback()

    result, err := tx.ExecContext(ctx,
        "UPDATE checks SET enabled = 0 WHERE id = ?", checkID,
    )
    if err != nil {
        return fmt.Errorf("disable check: %w", err)
    }
    changed, err := result.RowsAffected()
```

```
    if err != nil {
        return fmt.Errorf("count disabled checks: %w", err)
    }
    if changed == 0 {
        return ErrCheckNotFound
    }

    if _, err := tx.ExecContext(
        ctx,
        "INSERT INTO check_events(check_id, action) " +
            "VALUES (?, ?)",
        checkID,
        "disabled",
    ); err != nil {
        return fmt.Errorf("record disable event: %w", err)
    }
    if err := tx.Commit(); err != nil {
        return fmt.Errorf("commit disable: %w", err)
    }
    return nil
}
```

Sau khi đã thử, đối chiếu với `fixed/disable.go`. Điểm dễ bỏ sót không phải câu SQL mà là ownership của transaction. `defer tx.Rollback()` được đặt ngay sau khi transaction đã tồn tại. Nếu bất kỳ return nào xảy ra trước `Commit`, rollback là cleanup an toàn; sau `Commit`, rollback chỉ trả về lỗi đã kết thúc và được bỏ qua. Tất cả statement thuộc unit phải gọi trên `tx`. Một `db.ExecContext` chen vào giữa sẽ chạy ngoài transaction này.

`RowsAffected` là thông tin do driver trả về; một database hoặc statement khác có thể không hỗ trợ nó. Lab dùng SQLite, nơi contract này kiểm chứng được. Trong service thật, hãy chọn cách phân biệt "không có record" dựa trên semantics của database và query đang dùng, thay vì xem một API tiện tay là luật chung của SQL.

Dấu `?` cũng không phải syntax phổ quát. SQLite driver của lab dùng placeholder đó; nhiều driver PostgreSQL dùng `$1`, `$2`. Nhưng nguyên tắc không đổi: truyền value qua argument của driver, không ghép input vào string SQL. Parameter binding tách data khỏi cấu trúc câu lệnh; nó không thay cho validation nghiệp vụ, nhưng tránh biến `target` hay `checkID` thành một mảnh cú pháp ngoài ý muốn.

`*sql.DB` là một handle dùng chung, không phải "một connection"

`database/sql` là standard library cung cấp interface chung quanh SQL hoặc SQL-like database; nó luôn cần một driver cụ thể. Theo documented behavior của package, `*sql.DB` là handle an toàn khi dùng đồng thời và quản lý một pool connection. Vì thế, giữ nó ở application boundary và đóng khi process rút lui; đừng tạo rồi đóng một `*sql.DB` cho từng HTTP request.

Một hiểu nhầm kế tiếp là xem `sql.Open` như bằng chứng database đang sống. Việc mở handle thường không thực hiện kết nối ngay. Khi startup hoặc health policy thực sự cần kiểm tra reachability, dùng `PingContext` với deadline phù hợp; đừng biến một ping thành nghi thức trước mọi query rồi gọi đó là reliability.

```
func OpenAndCheck(
    ctx context.Context,
    dsn string,
) (*sql.DB, error) {
    db, err := sql.Open("sqlite", dsn)
    if err != nil {
        return nil, fmt.Errorf("open database handle: %w", err)
    }
    if err := db.PingContext(ctx); err != nil {
        db.Close()
        return nil, fmt.Errorf("ping database: %w", err)
    }
    return db, nil
}
```

Ví dụ này chỉ minh họa boundary. Nó không chép sẵn `SetMaxOpenConns(50)` vì một con số đẹp không phải policy. Quản lý connection pool trong `database/sql` xoay quanh bốn chính sách cốt lõi và một giao diện quan sát:

Một là, `SetMaxOpenConns` đặt trần cho tổng số kết nối mở tại một thời điểm, bao gồm cả kết nối đang bận thực thi lẫn kết nối rảnh trong pool. Mặc định giá trị này bằng 0 (không giới hạn). Khi chạm trần, các truy vấn tiếp theo bị chặn và xếp hàng chờ cho đến khi có kết nối được giải phóng hoặc context bị hủy. Nếu một goroutine đang giữ một kết nối trong transaction lại cố gắng thực hiện một truy vấn độc lập từ cùng pool mà không còn kết nối trống, việc chạm trần sẽ trực tiếp dẫn đến bế tắc đồng thời (deadlock).

Hai là, `SetMaxIdleConns` khống chế số lượng kết nối rảnh tối đa được giữ lại trong pool. Giá trị mặc định trong tài liệu hiện hành của thư viện chuẩn là 2 (tài liệu lưu ý giá trị mặc định này có thể thay đổi trong các phiên bản tương lai). Nếu ứng dụng đặt `SetMaxOpenConns` rất cao nhưng giữ nguyên `SetMaxIdleConns` bằng 2, sau mỗi đợt tăng tải đột biến, toàn bộ các kết nối dư thừa sẽ bị đóng ngay lập tức, dẫn đến hiện tượng mở và đóng kết nối liên tục (connection churn), tiêu tốn chi phí bắt tay kết nối và xác thực tài khoản.

Ba là, `SetConnMaxIdleTime` xác định thời gian rảnh rỗi tối đa mà một kết nối được phép nằm chờ trong pool trước khi bị thu hồi (đóng lười khi rảnh), giúp thu nhỏ kích thước pool về mức tối thiểu trong những khung giờ thấp điểm.

Bốn là, `SetConnMaxLifetime` quy định tuổi thọ tối đa mà một kết nối được phép tiếp tục tái sử dụng (reuse) tính từ thời điểm khởi tạo; các kết nối quá hạn có thể được đóng một cách lười (lazily) trước khi tái sử dụng. Đây không phải cơ chế ngắt cưỡng chế (hard kill) khi kết nối đang bận phục vụ truy vấn. Việc chủ động đặt giới hạn tuổi thọ kết nối giúp làm mới tài nguyên kết nối định kỳ theo yêu cầu của hạ tầng hoặc máy chủ cơ sở dữ liệu.

Năm là, phương thức `DB.Stats()` cung cấp dữ liệu quan sát trực tiếp tại runtime qua struct `sql.DBStats`. Các trường `OpenConnections`, `InUse`, `Idle`, `WaitCount`, `WaitDuration`, `MaxIdleClosed`, `MaxIdleTimeClosed` và `MaxLifetimeClosed` hỗ trợ đắc lực cho việc chẩn đoán xem ứng dụng có đang gặp tình trạng thiếu kết nối hay phải chờ đợi kéo dài hay không; chúng là số liệu quan sát hỗ trợ điều tra, không phải lời khẳng định nguyên nhân gốc rễ duy nhất. Việc lựa chọn thông số pool phải dựa trên năng lực của database backend, số lượng bản sao (replicas) cùng chia sẻ database, và dữ liệu thực tế từ `DB.Stats()`, tuyệt đối không sao chép những con số cảm tính từ trên mạng.

Cancellation cũng dừng ở boundary driver. `QueryContext` và `BeginTx` nhận context, nhưng package `database/sql` ghi rõ driver không hỗ trợ context cancellation có thể chỉ return sau khi query hoàn tất. Context vẫn phải được truyền; đồng thời, deadline thực sự của query cần được kiểm chứng với đúng driver và database trước khi đưa vào SLO.

Đọc cũng mượn tài nguyên

`QueryRowContext` phù hợp khi query được kỳ vọng trả tối đa một row; error xuất hiện khi `Scan`. Nó không kiểm tra uniqueness: nếu query trả nhiều row, `Scan` lấy row đầu và bỏ phần còn lại; nếu không có row, nó trả `sql.ErrNoRows`. Invariant "chỉ có một record" phải được query/schema giữ, không được suy từ tên API. Với nhiều row, `QueryContext` trả `*Rows`, đại diện cho result stream và tài nguyên driver. Đóng nó ở mọi đường return, đọc đến hết rồi kiểm tra lỗi iteration.

```
rows, err := db.QueryContext(ctx,
    "SELECT id, target FROM checks WHERE enabled = ?", true,
)
if err != nil {
    return nil, fmt.Errorf("list enabled checks: %w", err)
}
defer rows.Close()

var checks []Check
for rows.Next() {
    var check Check
    if err := rows.Scan(&check.ID, &check.Target); err != nil {
        return nil, fmt.Errorf("scan check: %w", err)
    }
    checks = append(checks, check)
}
if err := rows.Err(); err != nil {
    return nil, fmt.Errorf("iterate checks: %w", err)
}
return checks, nil
```

Đoạn code này chưa nói gì về isolation: list có thể thấy row nào khi concurrent writer đang chạy là policy của transaction và database. Nó chỉ chốt lifecycle ở phía Go: query đã mượn resource thì function phải trả nó lại. Đây là cùng một kiểu trách nhiệm ta đã gặp với response body ở Chương 11 và listener ở Chương 12, chỉ ở một boundary khác.

Câu lệnh chuẩn bị sẵn (Prepared Statement) và ranh giới tài nguyên

Khi một câu lệnh SQL được gửi tới cơ sở dữ liệu, engine phải phân tích cú pháp (parse), lập kế hoạch thực thi (query plan), rồi mới tiến hành truy vấn. Nếu cùng một mẫu câu lệnh được thực thi hàng nghìn lần lặp lại trong một đợt xử lý, việc lặp lại các bước phân tích này có thể tạo ra chi phí phụ trội không đáng có. `database/sql` cung cấp kiểu `*sql.Stmt` thông qua phương thức `PrepareContext` để biên dịch câu lệnh một lần và thực thi nhiều lần:

```
stmt, err := tx.PrepareContext(ctx,
    "INSERT INTO check_events(check_id, action) VALUES (?, ?)",
)
if err != nil {
    return fmt.Errorf("prepare insert event: %w", err)
}
defer stmt.Close()

for _, action := range actions {
    _, err := stmt.ExecContext(ctx, checkID, action)
    if err != nil {
        return fmt.Errorf("insert event %s: %w", action, err)
    }
}
```

Quy tắc sở hữu tài nguyên ở đây rất rõ ràng: một prepared statement tạo từ `db.PrepareContext` sống cùng với vòng đời của database handle và có thể tự động chuẩn bị lại trên các kết nối vật lý khác nhau trong pool khi cần thiết. Ngược lại, statement được tạo bên trong một transaction qua `tx.PrepareContext` gắn chặt với vòng đời của chính transaction đó; khi transaction kết thúc bằng `Commit` hoặc `Rollback`, statement này sẽ tự động đóng và không thể tiếp tục sử dụng. Lệnh `stmt.Close()` tường minh vẫn hữu ích khi muốn giải phóng tài nguyên sớm trước khi transaction hoàn tất, nhưng không phải điều kiện duy nhất để ngăn ngừa rò rỉ trong transaction.

Prepared statement không mặc định đồng nghĩa với việc mọi câu lệnh đều chạy nhanh hơn. Lợi ích tối ưu của prepared statement phát huy rõ rệt khi

cùng một mẫu câu lệnh được thực thi lặp lại nhiều lần với các tham số khác nhau (chẳng hạn trong các vòng lặp chèn dữ liệu hàng loạt như dự án `opsprobe` ở Chương 20). Nếu một câu lệnh chỉ được gọi duy nhất một lần trong toàn bộ vòng đời của request, chi phí phụ trội để chuẩn bị câu lệnh cần được cân nhắc và đo đạc thực nghiệm với driver và hệ quản trị cơ sở dữ liệu cụ thể trước khi quyết định áp dụng.

Việc truyền giá trị qua tham số (parameter binding / placeholder) tách biệt rành mạch dữ liệu khỏi cấu trúc câu lệnh SQL, giúp phòng ngừa lớp lỗ hổng tấn công chèn mã SQL (SQL injection) do nối trực tiếp giá trị vào chuỗi truy vấn. Tuy nhiên, theo tài liệu hướng dẫn bảo mật chính thức của Go (`go.dev/doc/database/sql-injection`), parameter binding chỉ áp dụng cho các giá trị tham số; các thành phần động trong câu lệnh như tên bảng, tên cột, mệnh đề `ORDER BY` hoặc các đoạn SQL ghép nối vẫn cần ranh giới kiểm tra và danh sách trắng (allowlist) riêng.

Transaction giải quyết được gì, và không giải quyết được gì

Transaction giúp nhóm write cùng database. Nó không tự làm HTTP call, publish message, cache invalidation hay email trở thành atomic với commit. Nếu `Disable` vừa commit vừa gọi webhook, failure sau commit có thể để database đúng nhưng webhook chưa đi. Đó là một bài toán khác, thường cần identity, retry có kiểm soát và mô hình như outbox khi requirement đủ rõ; không nên giả vờ rằng một `defer tx.Rollback()` giải quyết nó.

Atomicity không làm retry tự nhiên an toàn

Quay lại opening incident: response có thể mất **sau** `Commit`. Caller nhìn thấy lỗi mạng nhưng database đã đổi state. Transaction đã giữ lời hứa hẹp của nó: `UPDATE` và `INSERT` cùng tồn tại hoặc cùng không tồn tại. Nó chưa trả lời lần gọi lại có phải là cùng operation hay một yêu cầu mới.

Với `Disable` trong lab, `UPDATE checks SET enabled = 0` có thể trông idempotent: chạy lại vẫn để `enabled = false`. Nhưng `INSERT` audit event có thể ghi thêm một dòng `disabled`. Vì vậy "state cuối đúng" chưa đủ để gọi retry safe; audit, notification, quota hoặc side effect khác có thể đổi mỗi

lần execution.
`TestDisableRepeatedCallRecordsAnotherAuditEvent` cố ý cho thấy contract hiện tại của lab: gọi `Disable` hai lần tạo hai event. Test xanh không phải dấu xác nhận API retry-safe; nó là bằng chứng ngược lại, để người đọc không vô tình suy ra atomicity thành idempotency.

Khi requirement nói client có thể retry cùng một operation mà chỉ được tạo một hiệu ứng, API cần identity cho operation đó: chẳng hạn idempotency key được caller giữ lại qua lần gửi lại. Service cần xác định key đã được xử lý và trả kết quả phù hợp, thường bằng unique constraint hoặc record idempotency trong cùng database transaction. Chi tiết response khi key trùng, thời gian lưu key và side effect nào thuộc cùng operation đều là phần của điều API hứa. Chúng không tự xuất hiện từ `BeginTx`.

> **Dừng để dự đoán.** Một caller gửi yêu cầu `Disable` nhưng gặp lỗi timeout mạng ở tầng truyền tải và phản hồi từ dịch vụ bị thất lạc. Tại thời điểm nhận lỗi timeout, caller chỉ biết mình chưa nhận được kết quả, trong khi phía server có thể đã commit thành công hoặc chưa từng hoàn tất transaction. Nếu caller tự ý gửi lại lệnh `Disable` mà không mang theo idempotency key, việc quan sát thấy `enabled = false` cùng số lượng bản ghi audit event tăng lên 2 sau đó nói lên điều gì về sự khác biệt giữa sự thật phía server và bằng chứng caller sở hữu, cũng như ranh giới an toàn của việc retry?

Đáp án — chỉ đọc sau khi đã tự làm

Sự cố mạng làm đứt gãy kênh truyền khiến caller chỉ sở hữu một bằng chứng duy nhất: yêu cầu đã quá hạn thời gian chờ, hoàn toàn không có bằng chứng xác nhận server đã kịp commit hay chưa. Trong thực tế, server có thể đã commit thành công trước khi phản hồi bị rơi trên đường truyền. Khi caller tự động gửi lại lệnh mà không có idempotency key, trạng thái `enabled = false` trên bảng `checks` không chứng minh được lần gọi nào đã tạo nên kết quả, bởi tài nguyên có thể đã tắt từ lần gọi đầu tiên hoặc do một tiến trình khác can thiệp.

Bằng chứng rõ ràng nhất nằm ở bảng `audit_events`: số lượng bản ghi tăng lên 2 khẳng định giao dịch thứ hai đã thực thi độc lập và ghi thêm tác dụng phụ. Tình huống này chứng minh tính nguyên tử (atomicity) của transaction chỉ bảo đảm các câu lệnh trong một phiên cùng thành công hoặc cùng thất bại, chứ không tự động mang lại tính an toàn khi gọi lại (idempotency). Để caller có thể retry an toàn sau các lỗi mạng không chắc chắn, hệ thống bắt buộc phải cung cấp hợp đồng retry tường minh thông qua idempotency key được kiểm tra và lưu vết nguyên tử cùng transaction nghiệp vụ.

Tương tự, transaction không biến mọi concurrent operation thành tuần tự. `sql.TxOptions` cho phép caller yêu cầu isolation; khi caller yêu cầu non-default isolation level mà driver không hỗ trợ, `BeginTx` trả error. Database và driver thực tế vẫn quyết định các anomaly, lock và latency có ý nghĩa gì, nên trước khi dùng isolation để bảo vệ một invariant thật, đọc tài liệu database đang chạy, viết test cạnh tranh cho invariant đó và quan sát lỗi/lock/latency. Ở chương này, ta chỉ khóa một contract nhỏ có thể chứng minh cục bộ: event lỗi thì update không được tồn tại.

Giết process ở khe giữa hai lời hứa

Một yêu cầu cộng 7 đơn vị đã commit, nhưng caller chưa nhận phản hồi thì process chết. Khi khởi động lại, service có phải cộng thêm 7 không? Nếu còn phải gửi một sự kiện, làm sao nó biết sự kiện ấy chưa được gửi? Hai câu hỏi này không có cùng câu trả lời. Database có thể nhớ operation đã hoàn tất trong khi nghĩa vụ gửi vẫn còn dang dở.

Lab `labs/part13-transaction-boundary/outbox` giữ bài toán nhỏ: một số dư, một operation key và một sự kiện. Payload là số nguyên `delta` từ 1 đến 100; cùng key và cùng delta là retry của cùng operation, còn đổi delta bị từ chối bằng `ErrConflict`. Đây là định nghĩa payload của fixture, không phải thuật toán canonicalize JSON hay chính sách retention cho mọi API. Mở `TestOutboxContract` trước khi đọc `store.go`: yêu cầu của nó là sau commit, số dư tăng đúng một lần và nghĩa vụ gửi phải tồn tại.

Bản lỗi `BrokenApply` ghi số dư cùng operation record, commit, rồi mới thêm outbox. Subprocess thoát bằng `os.Exit(77)` ngay sau commit:

deferred cleanup không chạy. Mở database bằng connection mới cho thấy số dư đã tăng nhưng không có event pending. Gửi lại cùng key không cứu được: operation record đã tồn tại, nên đường tạo event bị bỏ qua. Một job quét RAM khi startup không thể tìm lại nghĩa vụ chưa từng được lưu.

Bản sửa đưa cả ba write vào một transaction: reserve key bằng unique constraint, đổi số dư, ghi event pending. Câu lệnh đầu tiên là write; lab không đọc rồi nâng khóa trong hai transaction cạnh tranh. Hai connection cùng gửi key được kiểm thử riêng: chỉ một operation và một event tồn tại. Duplicate kiểm tra lại payload trước khi kết thúc; nó không được coi một key bất kỳ là quyền thay payload đã chấp nhận.

Bảng 12 — Trạng thái phải nhìn thấy khi mở lại database sau process death

Checkpoint chết	Dữ liệu nghiệp vụ	Nghĩa vụ gửi	Hành động khi khôi phục
Trước commit	Chưa cộng; không có operation.	Không có event.	Gửi lại cùng key để thực hiện.
Sau commit, trước phản hồi	Đã cộng đúng một lần.	Pending.	Retry không cộng thêm; dispatcher đọc pending.
Sau phía nhận commit, trước đánh dấu gửi xong	Đã cộng ở cả hai phía.	Phía gửi vẫn pending.	Gửi lại cùng identity; phía nhận khử trùng lặp.

Chạy fixture từ module Chương 13; mỗi lần chạy dùng file tạm do test sở hữu, không chạm database của dự án:

```
go test -count=1 -v ./outbox
go test -race ./outbox
```

`TestRecoveryWindows` là tên bộ test các cửa sổ lỗi, không phải test chỉ dành cho Windows. Nó chạy child tới checkpoint, đòi exit code 77, rồi mở file lại và kiểm tra state. Lần dispatch phục hồi cũng chạy ở process mới. Trường hợp sau send dùng file database phía nhận riêng: không có transaction xuyên hai file, nhưng phía nhận ghi inbox identity và tác dụng phụ của nó trong cùng transaction. Test mất acknowledgment gửi hai lần

mà chỉ có một hiệu ứng; đó là contract idempotency của phía nhận, không phải exactly-once delivery.

Dispatcher của lab có một owner, kể cả giữa các process. Nó để event pending cho tới khi sender báo thành công rồi mới đánh dấu completed. Vì vậy không có processing flag bị kẹt sau crash; đổi lại, chưa có claim/lease để nhiều dispatcher chia việc. Retry sau lỗi không rõ kết quả có thể phát lại. Nếu phía nhận không có durable dedup gắn nguyên tử với tác dụng phụ, hoặc xóa identity quá sớm, outbox không ngăn hiệu ứng lặp. Khi tích hợp provider bên ngoài, phải đọc contract idempotency và thời gian lưu key của provider, không thay bằng lời hứa của SQLite.

Dừng để tự chứng minh. Đặt `RELIABILITY_MUTANT=split_commit` rồi chạy chỉ `TestOutboxContract`; test phải đỏ với số dư 7 nhưng pending 0. Bỏ biến ấy và chạy lại. Vì sao đổi thứ tự thành "đánh dấu completed rồi send" chỉ chuyển từ nguy cơ gửi lặp sang nguy cơ mất event?

Đáp án — đọc sau khi chạy thử. Process có thể chết sau completed nhưng trước send, khiến lần khởi động lại không còn thấy việc cần làm. Pending tới acknowledgment giữ nghĩa vụ có thể khôi phục, nhưng không đóng được khe sau send. Tính nguyên tử cục bộ bảo vệ dữ liệu và nghĩa vụ; identity phía nhận mới bảo vệ hiệu ứng khi nghĩa vụ bị phát lại.

Lab ghim driver `modernc.org/sqlite v1.59.0`, dùng rollback journal `DELETE`, `synchronous=FULL` và busy timeout 5 giây trên mỗi connection. Bằng chứng đã thử là process death tại checkpoint, không phải mất điện, disk hỏng hay filesystem mạng. Các giả định khóa và flush của SQLite vẫn phải đúng; một lần `os.Exit` không chứng minh toàn bộ độ bền của storage stack.

Tiến hóa lược đồ dữ liệu (Schema Migration) như một trạng thái bền vững

Cấu trúc bảng và các ràng buộc dữ liệu (schema) cũng là trạng thái bền vững. Mã nguồn ứng dụng có thể được rollback về một commit Git cũ chỉ trong vài giây, nhưng dữ liệu trên đĩa cứng đã bị thay đổi thì không thể đảo ngược một cách tự động. Khi dịch vụ bước vào môi trường sản xuất có dữ liệu người dùng thật, việc gọi các câu lệnh tạo bảng tùy tiện trong mã khởi động không còn là giải pháp khả thi. Quản lý schema đòi hỏi sáu nguyên tắc kỷ luật:

Một là, thứ tự di chuyển có phiên bản (versioned migration ordering). Mọi thay đổi schema phải được biểu diễn thành các bước tuần tự có đánh số phiên bản hoặc nhãn thời gian tăng dần, được lưu trữ trong một bảng quản lý (như `schema_migrations`). Mỗi bước chỉ được thực thi đúng một lần và theo đúng thứ tự đã định.

Hai là, quyền sở hữu migration (migration ownership). Một phương án triển khai phổ biến là giao quyền hạn thực thi migration cho một quy trình phát hành chuyên trách (như Kubernetes Job hoặc bước chạy pipeline CI/CD trước khi cập nhật workload). Tuyệt đối không để hàng chục instance của ứng dụng cùng khởi động và cùng tranh nhau chạy các câu lệnh DDL mà không có cơ chế điều phối, vì điều này có thể dẫn đến tranh chấp khóa và gây sập ứng dụng.

Ba là, tính tương thích khi triển khai song song (deploy compatibility). Trong các chiến lược triển khai không gián đoạn (như rolling update hay blue-green), phiên bản code cũ và phiên bản code mới sẽ cùng chạy đồng thời trong một khoảng thời gian. Một chiến lược thường dùng là Mở rộng - Di chuyển - Thu hẹp (Expand - Migrate - Contract): khi cần thêm một cột mới, cột đó cho phép giá trị NULL hoặc có giá trị mặc định để code cũ chưa biết đến cột này vẫn hoạt động bình thường; sau khi toàn bộ instance mới đã nhận việc ổn định, mới tiến hành bước dọn dẹp các trường lỗi thời.

Bốn là, chiến lược xử lý sự cố migration. Sửa tiến (forward-fix) thường được ưu tiên trong thực tế vận hành để tránh rủi ro mất mát dữ liệu mới phát sinh

trong giai đoạn chuyển tiếp nếu chạy script đảo ngược (Down migration). Tuy nhiên, lựa chọn giữa forward-fix và rollback phụ thuộc vào mô hình thất bại, tính toàn vẹn của dữ liệu và chính sách vận hành của từng hệ thống.

Năm là, ranh giới transaction đối với DDL. Khả năng bọc câu lệnh DDL trong transaction phụ thuộc vào hệ quản trị cơ sở dữ liệu và phiên bản cụ thể. PostgreSQL hỗ trợ transactional DDL cho nhiều thao tác catalog (như `CREATE TABLE`, `ALTER TABLE`), tuy nhiên kỹ sư vẫn cần xem xét các yếu tố về khóa bảng và các ngoại lệ như `CREATE INDEX CONCURRENTLY`. SQLite hỗ trợ transactional DDL cho các thao tác được engine cho phép (như `ALTER TABLE ADD COLUMN` được kiểm chứng trong fixture của lab). Trong khi đó, MySQL hiện đại (từ MySQL 8.0) hỗ trợ cơ chế atomic DDL ở cấp độ từng câu lệnh riêng lẻ, nhưng các câu lệnh DDL trong MySQL vẫn tự động gây ra lệnh commit ngầm định (implicit commit), do đó không thể kết hợp tùy ý nhiều câu lệnh DDL cùng các thao tác dữ liệu khác vào trong một transaction để rollback toàn phần.

Sáu là, kiểm thử tiến hóa schema. Lab của chương chứng minh cơ chế này qua kiểm thử `TestSchemaMigrationOrderingAndRollback`. Việc nâng cấp schema từ phiên bản 1 lên phiên bản 2 được thực thi trong transaction nguyên tử. Kiểm thử chủ động tạo kịch bản migration lỗi (thêm cột `failed_col` nhưng rollback transaction trước khi hoàn tất ghi nhận phiên bản) và kiểm tra qua `PRAGMA table_info` cùng bảng `schema_migrations` để xác nhận cột hủy không tồn tại và số hiệu phiên bản giữ nguyên ở mức 1; sau đó mới thực thi bước migration v2 hợp lệ và commit thành công.

Ranh giới bộ nhớ đệm (Cache Consistency) và nguồn chân lý

Khi lưu lượng đọc tăng cao, việc đưa thêm một lớp bộ nhớ đệm (cache) vào kiến trúc là giải pháp phổ biến để giảm tải cho cơ sở dữ liệu. Tuy nhiên, cache là một ranh giới hoàn toàn tách biệt khỏi database. Trong mô hình được chọn ở chương này, cơ sở dữ liệu đóng vai trò là nguồn chân lý (single source of truth).

Mô hình đệm thông dụng nhất là Cache-Aside: Khi đọc, ứng dụng kiểm tra cache trước; nếu không có dữ liệu (cache miss), nó đọc từ database rồi lưu ngược lại vào cache. Khi cập nhật trạng thái: ứng dụng ghi thay đổi vào cơ sở dữ liệu trước, rồi mới tiến hành xóa (invalidate) bản ghi tương ứng trong cache.

Vấn đề cốt lõi của tính nhất quán nằm ở ranh giới thất bại: nếu thao tác cập nhật database thành công nhưng lời gọi mạng xóa cache phía sau gặp sự cố (đứt mạng, timeout, hoặc máy chủ cache bị crash), cơ sở dữ liệu đã mang trạng thái mới nhưng cache vẫn giữ nguyên giá trị cũ. Lượt đọc tiếp theo từ một client khác sẽ lấy dữ liệu lỗi thời từ cache (stale read), tạo ra tình huống hệ thống kể hai câu chuyện mâu thuẫn nhau.

Nguyên nhân căn bản là vì database transaction không thể bảo đảm tính nguyên tử cho một hệ thống cache bên ngoài. Transaction của `database/sql` chỉ có phạm vi hiệu lực bên trong storage engine của cơ sở dữ liệu; nó hoàn toàn không có khả năng rollback một gói tin mạng đã gửi sang Redis hay Memcached.

Cơ chế hết hạn theo thời gian (TTL - Time-To-Live) chỉ là một biện pháp giới hạn khoảng thời gian tối đa mà dữ liệu lỗi thời có thể tồn tại nếu các giả định về chính sách làm mới dữ liệu được thỏa mãn; TTL không phải một bảo đảm đúng đắn tuyệt đối. Kỹ sư phải luôn phân định rõ: với những dữ liệu cho phép eventual consistency (như số lượt xem bài viết, thông tin hiển thị hồ sơ), sự chênh lệch ngắn hạn giữa cache và database là chấp nhận được; nhưng với các bất biến nghiệp vụ quan trọng, giải pháp phụ thuộc vào yêu cầu của hệ thống: đọc trực tiếp cơ sở dữ liệu (bypass cache), sử dụng khóa có phiên bản (versioned keys), mô hình write-through/write-behind với hợp đồng phù hợp, vô hiệu hóa qua sự kiện (event-driven invalidation), hoặc cơ chế kiểm tra phiên bản (version/fencing checks).

Kiểm thử `TestCacheAsideStaleReadOnInvalidationFailure` trong lab cố ý tái lập kịch bản này: database đã cập nhật thành công cờ `enabled = false`, nhưng thao tác xóa cache gặp lỗi khiến cache vẫn trả về `true`.

Đây là bằng chứng thực nghiệm rõ ràng nhất để người học không nhầm lẫn giữa tính nguyên tử của transaction với tính nhất quán của toàn bộ hệ thống.

Biểu diễn tuần tự hóa và tương thích tiến hóa dữ liệu

Như đã phân tích ở Chương 07, việc tuần tự hóa dữ liệu (serialization) biến các cấu trúc dữ liệu trong bộ nhớ thành dòng byte để truyền qua mạng hoặc ghi xuống đĩa. Trong tầng lưu trữ, việc lưu các đối tượng phức tạp dưới dạng chuỗi JSON hoặc mảng byte nhị phân (BLOB) vào một cột của cơ sở dữ liệu là một hợp đồng biểu diễn giữa người ghi (writer) và người đọc (reader) qua các mốc thời gian khác nhau.

Tính tương thích của dữ liệu đã lưu trữ phụ thuộc vào định dạng tuần tự hóa, lược đồ phiên bản, chiến lược di chuyển dữ liệu, ngữ nghĩa giá trị mặc định, khả năng phân biệt giữa việc vắng mặt trường dữ liệu với giá trị zero tường minh, thời gian lưu giữ (retention period), và khả năng viết lại dữ liệu lịch sử nếu cần thiết. Các thẻ tag như `omitempty` (hoặc `omitzero` từ Go 1.24) là các tùy chọn định dạng tuần tự hóa, không phải cơ chế tương thích phổ quát cho mọi trường hợp tiến hóa dữ liệu. Đối chiếu Chương 07 để nắm vững cách thức xử lý dòng byte và ánh xạ dữ liệu.

Điểm dừng: Giữ vững ranh giới trạng thái bền vững

Điểm dừng của chương không phải là việc thuộc lòng các câu lệnh SQL hay cấu hình máy chủ. Đó là thói quen phân định ranh giới trạng thái một cách có kỷ luật:

Một là, gom cụm các thao tác cùng ý nghĩa nghiệp vụ vào một transaction nguyên tử trên cơ sở dữ liệu, bảo đảm tính toàn vẹn hoặc không gì cả.

Hai là, quản lý kết nối như một tài nguyên dùng chung có giới hạn qua connection pool, đo lường bằng `DB.Stats()` thay vì suy đoán thông số.

Ba là, tận dụng prepared statement cho các thao tác lặp lại theo lô, đồng thời giải phóng tài nguyên với `Close()` và bảo vệ câu lệnh bằng parameter binding.

Bốn là, coi schema như một trạng thái bền vững cần tiến hóa có phiên bản, kiểm soát quyền sở hữu và bảo đảm tương thích hai chiều khi triển khai.

Năm là, nhận thức rõ ranh giới giữa cơ sở dữ liệu và bộ nhớ đệm, luôn xác định nguồn chân lý và chấp nhận đánh đổi tính nhất quán dựa trên yêu cầu bất biến của nghiệp vụ.

Khi các ranh giới này được phân định rõ ràng, hệ thống giảm thiểu các trạng thái dữ liệu mơ hồ, cho phép kỹ sư xác lập và kiểm chứng được các ranh giới thất bại từ mã nguồn ứng dụng đến tầng lưu trữ bền vững.

1. Go Team. Package `database/sql`: `DB`, `Tx`, `Rows`, `Stmt`, `DBStats`, context cancellation và pool behavior. pkg.go.dev/database/sql
2. Go Team. Executing prepared statements. go.dev/doc/database/prepared-statements
3. Go Team. Avoiding SQL injection risk. go.dev/doc/database/sql-injection
4. SQLite. Transaction and ALTER TABLE support. sqlite.org/lang_transaction.html, sqlite.org/lang_altertable.html
5. modernc.org. Package `sqlite`, driver thuần Go dùng trong lab cục bộ. pkg.go.dev/modernc.org/sqlite
6. PostgreSQL Global Development Group. Transactional DDL. postgresql.org/docs/current/mvcc.html
7. MySQL Authors. Statements That Cause an Implicit Commit & Atomic DDL. dev.mysql.com/doc/refman/8.0/en/implicit-commit.html, dev.mysql.com/doc/refman/8.0/en/atomic-ddl.html
8. SQLite. Atomic Commit In SQLite: rollback journal, hot journal recovery và giả định khóa/flush của OS/storage. sqlite.org/atomiccommit.html
9. modernc.org. SQLite driver v1.59.0: DSN `_pragma` áp dụng khi mở mỗi connection. pkg.go.dev/modernc.org/sqlite@v1.59.0

Chương 14 — Khi kiểu trở thành dữ liệu

Một function nhận `any` thường tạo hai phản ứng trái ngược. Có người thấy nó "linh hoạt" rồi bắt đầu nhét mọi thứ vào. Có người thấy nó đáng sợ rồi cấm hoàn toàn. Cả hai phản ứng đều bỏ qua câu hỏi quan trọng hơn: khi compiler không còn biết type cụ thể tại chỗ gọi, code nào sẽ kiểm tra shape của value, và code nào được quyền thay đổi nó?

Hãy xét một loader cấu hình nhỏ. `opsprobe` có thể nhận giá trị từ environment, nhưng module đọc environment không nên phải biết trước mọi struct config của các command sau này. Nó nhận một destination và một map giá trị đã đọc; destination quyết định field nào được phép nhận value. Nếu caller truyền `Config{}` thay vì `&Config{}`, loader không được silently tạo bản copy rồi để caller tưởng config đã đổi. Nếu một field có tag nhưng type không phải `string`, loader không được đoán cách parse.

Mental model của chương là: **reflection làm type và value trở thành dữ liệu có thể kiểm tra ở runtime; nó không xóa contract, mà chuyển một phần contract từ compiler sang các guard cụ thể trong code.** `unsafe` nằm ở rìa của mô hình này: nó vượt qua guard của type system, nên chỉ có chỗ khi representation và lifetime đã được chứng minh ở một boundary hẹp.

Một value có shape mà compiler chưa biết

Trong hàm gọi bình thường, compiler biết `config.Endpoint` là `string`; `config.Endpoint = raw` được kiểm tra trước khi chương trình chạy. Với `func ApplyEnv(dst any, values map[string]string) error`, compiler chỉ biết `dst` là interface value. Dynamic type và dynamic value đi cùng `dst` đến runtime, và `reflect.ValueOf` cho ta một view để hỏi chúng.

```
config := Config{}

value := reflect.ValueOf(config)
fmt.Println(value.Kind())   // struct
fmt.Println(value.CanSet()) // false

pointer := reflect.ValueOf(&config)
target := pointer.Elem()
fmt.Println(target.Kind())   // struct
fmt.Println(target.CanSet()) // true
```

Trong source của Go 1.27.1, `reflect.Value` hiện có các trường nội bộ sau. Chúng chỉ giúp giải thích một quan sát của toolchain này; contract mà code application được dựa vào là các method document như `CanSet`, `Kind` và `Elem`, không phải tên field hay bit flag:

```
type Value struct {
    typ_ *abi.Type
    ptr_ unsafe.Pointer
    flag
}
```

Cờ nội bộ hiện giữ thông tin về `Kind`, trạng thái chỉ đọc và addressability. Ở tầng API, điều cần nhớ đơn giản hơn: `reflect.ValueOf(config)` nhận một bản sao value nên không cho phép ghi ngược vào biến của caller; `CanSet()` trả về `false`.

Ngược lại, `reflect.ValueOf(&config).Elem()` nhìn vào variable gốc qua pointer. Với struct field, `CanSet()` còn phụ thuộc việc field có thể được sửa theo quy tắc export/visibility. Đây là guard API cần dùng thay vì suy luận từ flag nội bộ.

`ApplyEnv(nil, nil)` còn có một bẫy nhỏ hơn. `reflect.ValueOf(nil)` trả zero `reflect.Value`, có `Kind` là `Invalid`; nhiều operation khác trên value không hợp lệ có thể panic. Vì vậy boundary phải kiểm tra `IsValid` trước khi làm các operation phụ thuộc shape. Đây là

guard cho một value runtime thực sự không tồn tại, không phải một trường hợp `nil` pointer thông thường.

Cơ chế thực thi	Đặc tính điều phối lời gọi	Xu hướng đóng gói và cấp phát	Khả năng Compiler Tối ưu
Lời gọi hàm hoặc phương thức tĩnh	Compiler biết target từ type tĩnh	Không cần cơ chế reflection riêng	Có thể inline hay tối ưu thêm tùy toolchain.
Lời gọi qua interface	Target phụ thuộc dynamic type	Representation và allocation là quyết định triển khai	Có thể được devirtualize khi compiler chứng minh được điều kiện; không có bảo đảm.
Lời gọi qua reflection (`Value.Call`)	Metadata và kiểm tra động là một phần thao tác	API nhận/trả `reflect.Value`	Thường đắt hơn đường tĩnh; nếu chi phí quan trọng, phải benchmark contract cụ thể.

Bảng 13 — Compiler và reflection chia việc khác nhau

Câu hỏi	Code có type tĩnh	Code reflection có `any`
Value có phải `*Config` không?	Compiler đã biết ở call site	Kiểm tra pointer, `nil` và `Elem().Kind()` khi chạy.
Field có đúng builtin `string` không?	Compiler kiểm tra assignment	So exact `StructField.Type` với `reflect.TypeFor[string]()`.
Có quyền ghi field không?	Quy tắc visibility và addressability của Go	`CanSet` và exported field vẫn giữ ranh giới đó.
Sai thì xảy ra khi nào?	Thường compile error	Phải là `error` có context, không phải panic do đoán shape.

Reflection vì thế không thay generics. Nếu function chỉ cần một operation trên tập type biết trước, type parameter cho compiler giữ contract tốt hơn. Reflection hợp lý khi operation thực sự phụ thuộc vào metadata runtime: serializer, decoder, schema tool, plugin boundary, hoặc loader đọc struct tag. Chính `encoding/json` là ví dụ quen thuộc về API nhận value với shape do runtime quyết định; điều đó không có nghĩa mọi map-to-struct helper trong application đều cần tự viết reflection.

Interface, generics, reflection và unsafe giải quyết bốn bài toán khác nhau

Interface là abstraction theo behavior: concrete type có thể thay đổi lúc runtime, nhưng compiler vẫn kiểm tra method contract. Generics hay type parameter là abstraction ở compile time trên một tập type có constraint; compiler vẫn biết type information cần thiết cho bài toán phù hợp. Reflection inspect hoặc thao tác `Type` và `Value` ở runtime khi metadata hay shape mới là input. `unsafe` là escape hatch vượt ra ngoài một phần type safety thông thường; nó chỉ có chỗ khi representation và lifetime đã có proof cụ thể. Bốn công cụ có thể cùng xuất hiện trong một codebase, nhưng không phải bốn cách thay thế cho nhau.

Đây chỉ là cầu nối để đặt reflection vào đúng ranh giới, chưa phải bài học về tham số kiểu. Chương 19 sẽ bắt đầu lại từ một phép lặp cụ thể, rồi xây type parameter, constraint và cách giữ thông tin kiểu mà không biến mọi API thành `any`. Đến lúc đó, câu hỏi không còn là "reflection có linh hoạt hơn không?", mà là contract nào compiler vẫn có thể giữ giúp ta.

Struct tag là schema nhỏ, không phải comment bí mật

Một struct tag là metadata gắn vào declaration. Compiler lưu nó trong type; `reflect.StructField.Tag.Lookup` có thể đọc nó ở runtime. Tag không tự parse environment hay validate domain. Nó chỉ cho loader biết field nào đã tham gia vào schema này.

```
type Config struct {
    Endpoint string `env:"OPS_PROBE_ENDPOINT"`
    Token    string `env:"OPS_PROBE_TOKEN"`
    Ignored  string `env:"-"`
}
```

Hai field export được chọn vì API công khai của loader chỉ được phép ghi phần state mà type đã công khai. `env:"-"` là opt-out có chủ đích. Ngược lại, một field unexported lại khai báo một tag `env` thật là schema lỗi: owner type đang vừa yêu cầu loader tham gia schema, vừa cấm nó ghi. Loader phải trả

error thay vì âm thầm bỏ qua hoặc dùng `unsafe` để biến private state thành backdoor.

Đây là implementation tham chiếu của boundary nhỏ. Nó chỉ nhận `string`; parse port, duration, URL hay secret policy thuộc các contract khác, nên không bị giấu sau một helper "tự làm mọi thứ".

```
var (
    ErrDestination = errors.New(
        "destination must be a non-nil pointer to struct",
    )
    ErrSchema      = errors.New("invalid env schema")
)

type assignment struct {
    field reflect.Value
    raw   string
}

func schemaError(field, rule string) error {
    return fmt.Errorf(
        "%w: env field %s %s",
        ErrSchema,
        field,
        rule,
    )
}
```

`ApplyEnv` dùng hai phase. Phase A chỉ đọc metadata và thu thập assignment; phase B mới gọi setter. Nhờ vậy, một field hợp lệ đứng trước field schema lỗi không thể để lại destination nửa mới nửa cũ. Đây là cùng một kỷ luật failure state của Chương 13, nhưng ở boundary runtime-generic thay vì transaction: nếu contract tránh được partial state, `error` phải trả caller về state cũ.

```
func ApplyEnv(dst any, values map[string]string) error {
    value := reflect.ValueOf(dst)
    if !value.IsValid() ||
        value.Kind() != reflect.Pointer ||
        value.IsNil() {
        return ErrDestination
    }
    target := value.Elem()
    if target.Kind() != reflect.Struct {
        return ErrDestination
    }
    typ := target.Type()
    stringType := reflect.TypeFor[string]()
    assignments := make([]assignment, 0, typ.NumField())
    for i := 0; i < typ.NumField(); i++ {
        field := typ.Field(i)
        name, tagged := field.Tag.Lookup("env")
        if !tagged || name == "-" {
            continue
        }
        if !field.IsExported() {
```

```
            return schemaError(field.Name, "is unexported")
        }
        if field.Type != stringType {
            return schemaError(field.Name, "must be string")
        }
        fieldValue := target.Field(i)
        if !fieldValue.CanSet() {
            return schemaError(field.Name, "cannot be set")
        }
        if raw, found := values[name]; found {
            assignments = append(assignments, assignment{
                field: fieldValue,
                raw:   raw,
            })
        }
    }
    for _, assignment := range assignments {
        assignment.field.SetString(assignment.raw)
    }
    return nil
}
```

`Type` trả lời câu hỏi về declaration: field thứ `i` tên gì, tag gì, exported không, exact type nào. `Kind` chỉ là category underlying runtime kind; vì `type Token string` cũng có `Kind() == reflect.String`, nó không đủ cho contract chỉ nhận builtin `string`. `TypeFor[string]()` cung cấp identity của type cần nhận. `Value` trả lời câu hỏi về instance cụ thể: value hiện tại là gì, có set được không. Quy tắc đơn giản là đọc metadata từ `Type`, đọc hoặc ghi dữ liệu từ `Value`.

> **Dừng để dự đoán.** Nếu caller gọi `cfg := Config{}` rồi thực hiện `ApplyEnv(cfg, values)` bằng giá trị thay vì truyền con trỏ `&cfg`, liệu hàm có thể cập nhật các trường cấu hình không? Vì sao implementation của `ApplyEnv` chặn lỗi ngay từ guard kiểm tra con trỏ ở đầu hàm thay vì duyệt qua từng field rồi kiểm tra `CanSet()`?

Đáp án — chỉ đọc sau khi đã tự làm Khi truyền struct bằng giá trị vào đối số interface `any`, giá trị được đóng gói thành một bản sao không có địa chỉ liên kết với biến gốc (unaddressable). Đối tượng `reflect.ValueOf(dst)` sinh ra từ một giá trị như vậy đại diện cho một bản sao độc lập, và phương thức `CanSet()` trên các trường của nó luôn trả về `false`. Nếu cố tình gọi `SetString()` lên một `reflect.Value` không có quyền gán, runtime sẽ lập tức gây panic. Kể cả khi hàm cố tình bỏ qua mà không panic, biến `cfg` ban đầu của caller cũng hoàn toàn không nhận được dữ liệu mới, biến giá trị trả về `nil` thành một tín hiệu thành công giả tạo.

Trong cài đặt thực tế của `ApplyEnv`, ta không đợi đến lúc duyệt từng trường mới kiểm tra `CanSet()`. Guard clause ở ngay đầu hàm kiểm tra trực tiếp `value.Kind() != reflect.Pointer` cùng điều kiện con trỏ không nil và trả về `ErrDestination` ngay lập tức. Đây là ranh giới phòng vệ fail-fast: một khi đích đến không phải là con trỏ trỏ tới vùng nhớ có quyền đột biến, hàm từ chối dứt khoát toàn bộ thao tác trước khi tốn chi phí duyệt metadata của struct.

Thực hành: viết kiểm tra bảo vệ trước khi gán giá trị (setter)

`labs/part14-reflection-boundary` bắt đầu bằng test đỏ. Mở `exercise/apply_test.go`, chỉ đọc type `Config` và assertion trước. Tự quyết định thứ tự guard, nhưng đừng dùng `unsafe`, đừng parse tag bằng cắt string thủ công, và đừng panic cho input không đúng shape.

```
cd labs/part14-reflection-boundary
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test ./fixed
go test -race ./fixed
```

Test yêu cầu các boundary dễ nhầm: pointer tới struct được sửa; struct value, nil typed pointer và untyped `nil` bị từ chối; tag `-` không ghi; tagged `int`, named string type, và tagged unexported field đều là schema lỗi. Một schema lỗi sau field hợp lệ vẫn không được partial mutate destination. Nó không yêu cầu loader trở thành framework config.

Đáp án — chỉ đọc sau khi đã tự làm. Bắt đầu ở `reflect.ValueOf(dst)`, kiểm tra `IsValid`, `Pointer`, `IsNil` và `Elem().Kind() == Struct` trước. Sau đó iterate field declaration, bỏ tag `-`, từ chối tag thật trên field unexported, so exact type và kiểm tra `CanSet`. Chỉ thu thập assignment ở phase A; phase B mới set chúng. Nhờ vậy malformed schema thành error mà caller có thể xử lý, thay vì panic hoặc partial mutation.

`unsafe` không phải nút "mở khóa" field private

Khi reflection từ chối ghi `debug`, có thể thấy một mẹo dùng `unsafe.Pointer` để bỏ qua. Đừng làm vậy. Việc field không export là một phần của API và ownership; code generic không biết invariant nào của state private sẽ bị phá khi nó gán text tùy ý.

`unsafe` tồn tại cho một số boundary thật: interop với memory do hệ điều hành hay foreign code quản lý, implementation thấp tầng đã có representation được chỉ định, hoặc primitive mà standard library cung cấp cho việc đó. Nó cho phép operation vượt ra ngoài type safety thông thường,

nhưng không biến mọi thứ thành không được định nghĩa. Chỉ được dựa vào guarantee mà specification và package `unsafe` ghi rõ: `unsafe.Sizeof`, `unsafe.Alignof` và `unsafe.Offsetof` đều có semantics được document.

Điều còn lại vẫn đắt: assumption về representation có thể phụ thuộc platform; architecture khác có thể có layout khác; lifetime, GC và pointer rule vẫn phải được chứng minh. Một số đo trên Windows amd64 là measurement của target đó, không phải ABI phổ quát hay lời hứa cho mọi build.

```
type Header struct {
    Flag byte
    ID   uint64
}

fmt.Println(unsafe.Sizeof(Header{}))
fmt.Println(unsafe.Alignof(Header{}))
fmt.Println(unsafe.Offsetof(Header{}.ID))
// Ghi cùng Go version, GOOS và GOARCH của lần đo.
```

Đừng suy luận layout của interface, string header, map hay runtime object từ output ấy. Chúng có implementation detail và có thể thay đổi. Lab chạy experiment này bằng test để code được compiler kiểm tra; output log luôn ghi Go version, `GOOS` và `GOARCH`, còn sách không biến một con số local thành fact phổ quát.

Code review: đừng giữ địa chỉ trong `uintptr`

Đoạn dưới là mã nguồn tiêu cực để phân tích, không phải kỹ thuật nên áp dụng trong thực tế. Nó trích xuất địa chỉ mảng byte nền tảng từ kiểu cũ `reflect.StringHeader` rồi trả về `uintptr` ra ngoài phạm vi chuỗi:

```
func badDataAddress(s string) uintptr {
    header := (*reflect.StringHeader)(unsafe.Pointer(&s))
    return header.Data
}
```

Trước khi chấp nhận đoạn mã này, hãy phân tích bản chất cơ chế của Go Runtime:

Thứ nhất, phân biệt rạch ròi giữa việc di dời ngăn xếp (stack movement) và thu gom rác trên heap (heap GC). Go runtime hiện tại dùng heap non-moving và có thể copy stack khi stack goroutine lớn lên; đó là implementation detail, không phải permission để giữ địa chỉ theo ý mình. Một `unsafe.Pointer` vẫn được GC nhận diện là pointer trong những pattern mà package `unsafe` cho phép; hãy chỉ dựa vào các conversion được tài liệu hóa.

Thứ hai, `uintptr` là số nguyên đủ lớn để chứa bit của địa chỉ trên kiến trúc đang chạy, không phải một tham chiếu sống. Nếu tách địa chỉ khỏi `unsafe.Pointer` gốc và giữ nó trong `uintptr`, GC không có reason để coi số đó là giữ object sống; stack growth cũng không thể sửa một số nguyên đã chép. Vì vậy object có thể bị thu hồi khi liveness của pointer gốc kết thúc, còn địa chỉ stack có thể trở nên stale.

Thứ ba, với pattern chuyển pointer qua `uintptr` để cộng offset rồi chuyển ngược, tài liệu `unsafe.Pointer` yêu cầu conversion và phép tính nằm trong **cùng một biểu thức:**

```
p = unsafe.Pointer(uintptr(p) + offset)
```

Pattern một biểu thức còn phải giữ pointer trong cùng object đã cấp phát và trỏ tới dữ liệu hợp lệ theo contract `unsafe`; nó không cho phép tính một địa chỉ tùy ý. Tách địa chỉ qua `uintptr` trung gian không được hợp thức hóa chỉ bằng `KeepAlive`. Các mô tả runtime và ví dụ ở đây được kiểm tra trong bối cảnh Go 1.27.1; chỉ conversion pattern được package công bố mới là phần caller nên dựa vào.

Trong API Go 1.27.1, `reflect.StringHeader` và `reflect.SliceHeader` được đánh dấu deprecated; tài liệu cảnh báo trường `Data uintptr` không đủ giữ dữ liệu sống và representation không dùng an toàn hay portable. Khi thực sự cần boundary tầng thấp, đọc

contract của các helper `unsafe` thay vì tự dựng header; helper vẫn không miễn yêu cầu về lifetime và mutation:

```
// Trích xuất con trỏ byte nền tảng
ptr := unsafe.StringData(s)

// Tái tạo chuỗi từ con trỏ byte và độ dài
str := unsafe.String(ptr, len(s))

// Lấy con trỏ phần tử đầu của slice
slicePtr := unsafe.SliceData(buf)

// Tạo slice từ con trỏ và độ dài
slice := unsafe.Slice(slicePtr, count)
```

Cần giữ contract về vòng đời và mutation: pointer Go hợp lệ được GC theo dõi, khác với một địa chỉ đã biến thành số nguyên. `unsafe.String` không cho phép sửa các byte nền trong khi string còn được sử dụng. `runtime.KeepAlive` có vai trò ở những boundary như resource có finalizer mà OS chỉ còn nhìn thấy descriptor; nó không hợp thức hóa conversion ngoài các pattern của `unsafe`, không pin memory và không sửa một địa chỉ `uintptr` đã stale.

Biên giới với C là một hợp đồng khác

Reflection vẫn hoạt động trong hệ kiểu Go. Khi library chỉ có C ABI, cgo đưa chương trình qua một boundary khác: representation, lifetime, allocator và toolchain của hai phía đều cần được đọc. Một string Go có độ dài và có thể chứa byte NUL; `strlen` của C dừng ở NUL đầu tiên. Hai phép "đo độ dài" không cùng contract.

Lab `labs/edition-contracts/interop` có một hàm nhỏ dùng `C.CString`, gọi `C.strlen`, rồi `C.free` vùng nhớ đã cấp phát. Test đòi `"a\x00b"` có kết quả 1 ở phía C, không phải `len` của string Go. Đừng biến phép thử này thành khuyến nghị chuyển parser sang C: nó chỉ làm lộ một khác biệt dữ liệu. Byte không có NUL cuối, string Unicode và C string cần chính sách chuyển đổi cụ thể; "cùng là text" chưa đủ.

Memory do `C.CString` cấp phát cần được giải phóng theo allocator C; GC Go không tự thu hồi nó. Với pointer Go truyền sang C, tài liệu cgo quy định vùng memory được phép trỏ tới, pinning và thời gian C có thể giữ pointer. Không suy luận rằng thấy một địa chỉ hợp lệ lúc gọi là được lưu nó vô hạn ở C. `runtime.Pinner` và `runtime/cgo.Handle` giải quyết những trường hợp khác nhau; Handle là định danh để giữ giá trị Go, không phải permission cho C giải tham chiếu tùy ý. Chỉ dùng sau khi đã thiết kế lifecycle và đọc contract tương ứng.

cgo còn thay deployment contract: build cần C compiler phù hợp, binary có thể phụ thuộc native library, cross-compilation cần toolchain target. Memory native không nằm trọn trong heap budget Go và crash native có thể vượt khỏi error path mà caller Go kiểm soát. Không có một số nanosecond crossing phổ quát: nếu boundary là hot path, đo đúng payload, tần suất và môi trường, rồi xem có thể gom nhiều operation thành một lượt gọi không. Go thuần thường đơn giản hơn cho tooling tự chứa; C hợp lý khi cần API, driver hoặc implementation đã được kiểm chứng mà việc viết lại tạo rủi ro lớn hơn chi phí interop.

Chương này không dạy cách "né Go". Nó dạy một boundary có trách nhiệm khi type chỉ xuất hiện lúc runtime. Reflection có thể giữ code generic nhỏ và thành thật nếu contract về shape, metadata và mutation được viết lộ ra. Khi proof ấy không đủ, quay lại type tĩnh, interface nhỏ hoặc API cụ thể thường là thiết kế tốt hơn.

1. Go Team. Package `reflect`: `Value`, `CanSet`, `Type`, `TypeFor`, `StructField` và `StructTag`. pkg.go.dev/reflect
2. Go Team. Package `unsafe`: `Pointer`, `Sizeof`, `Alignof`, `Offsetof`, `String`, `StringData`, `Slice`, `SliceData`. pkg.go.dev/unsafe
3. Go Team. The Go Programming Language Specification: struct tags, type identity và address operators. go.dev/ref/spec
4. Go Team. A Guide to the Go Garbage Collector. go.dev/doc/gc-guide; cơ chế sao chép stack đối chiếu riêng với `src/runtime/stack.go` của Go 1.27.1, không phải contract của ngôn ngữ.
5. Go Team. `cmd/cgo`, Passing pointers; `runtime.Pinner`; `runtime/cgo.Handle`. Contract kiểm tra với Go 1.27.1. pkg.go.dev/cmd/cgo

Chương 15 — Từ incident đến công cụ

Một incident nhỏ thường không thiếu lệnh để gõ. Thiếu là một lời kể đáng tin về điều lệnh đó đã làm. Một người vận hành có thể chạy `curl`, `nslookup`, `git`, hay một tool nội bộ rồi nhìn thấy một dòng lỗi. Nhưng dòng lỗi ấy không nói command nào đã thật sự được chạy, nó có nhận đủ thời gian không, có in chẩn đoán ra `stderr` không, hay process kết thúc bằng exit status nào. Gõ lại cùng một lệnh nhiều lần chỉ làm lịch sử mờ hơn.

Chương này bắt đầu từ một requirement khiêm tốn: một incident command chạy **một tool chẩn đoán đã được chọn trước**, chờ nó theo một deadline, rồi trả về output và trạng thái có thể giải thích. Đây chưa phải feature lớn cho `opsprobe`; trước hết ta dựng một minimal reproducer độc lập để học đúng boundary của process. Khi contract đã rõ, `opsprobe` mới có thể dùng nó như một building block nhỏ.

Mental model của chương là: **command runner là một supervisor nhỏ của process con**. Nó không chỉ "gọi một binary". Nó nhận intent đã được validate, sở hữu deadline, chờ process kết thúc, giữ output tách biệt, phân loại kết quả, rồi để caller quyết định cách trình bày và ghi nhận. Nếu không sở hữu những điều đó, tool có thể biến một incident rõ ràng thành một chuỗi shell command khó tái hiện.

Một câu lệnh nhìn có vẻ vô hại

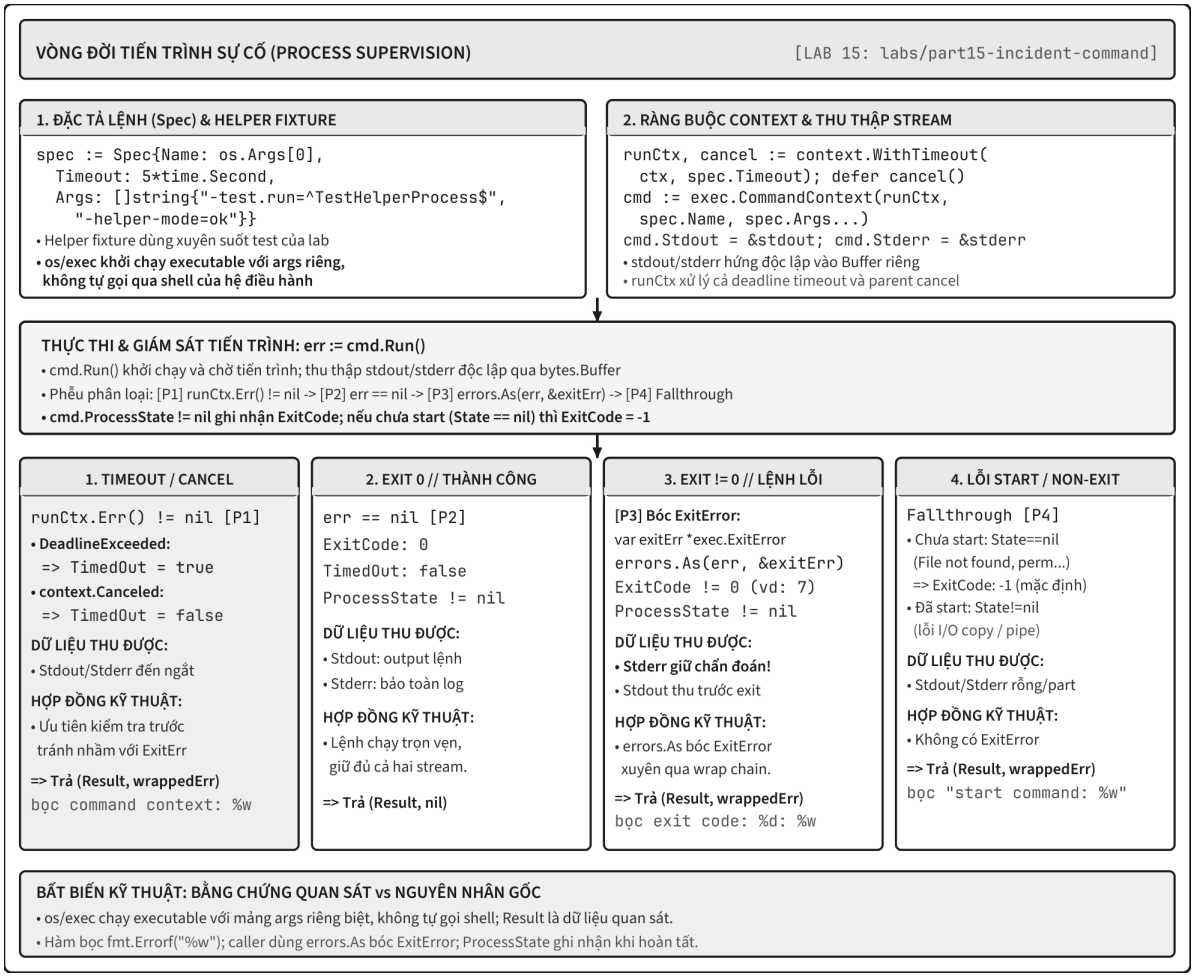
Giả sử alert báo dependency `catalog` trả lời chậm. Một command đầu tiên có thể chạy một probe nội bộ hoặc một program hệ điều hành để lấy thêm manh mối. Cách tắt nhất thường trông như sau:

```
exec.Command("sh", "-c", "probe " + target).Run()
```

Nó giấu quá nhiều quyết định. `target` là dữ liệu hay một mảnh shell program? Error có phải exit status khác 0, không tìm được executable, hay deadline đã hết? `stdout` và `stderr` đi đâu? Và nếu process cha của

`opsprobe` đang dừng theo lifecycle ở Chương 12, process con có còn chạy không?

`os/exec` cố ý không tự gọi system shell và không tự mở rộng glob, pipeline hay redirect. Đó là một ranh giới hữu ích: command name và từng argument là dữ liệu riêng. Nếu thật sự cần shell, ta phải chọn shell, quoting và threat model một cách tường minh; một incident tool không nên vô tình nhận quyền chạy shell chỉ vì ghép string.



Hình 26 — Mô hình khái niệm. `Result` là bằng chứng cục bộ về một lần chạy; nó không tự chứng minh nguyên nhân gốc của incident.

Trong chapter này, caller chọn executable từ code hoặc một allow-list. `target` chỉ là argument. Đó không phải giải pháp đầy đủ cho mọi tool runner, nhưng là boundary đúng cho yêu cầu hiện tại: người dùng không được gửi cả một command line tùy ý qua flag rồi bắt chương trình đoán ý nghĩa của nó.

Bốn kết cục, không phải một `error`

Một runner có thể trả `error`, nhưng caller cần biết loại failure. Thành công là process chạy rồi exit status bằng 0. Một command được khởi động thành công nhưng exit với status 7 lại là một kết cục khác: `stderr` của nó có thể chính là dữ liệu chẩn đoán hữu ích. Hết deadline là kết cục thứ ba. Không tìm được executable hoặc không thể start là kết cục thứ tư, thường là lỗi cấu hình hoặc môi trường trước cả khi diagnosis bắt đầu.

Bảng 14 — Kết cục của một lần chạy process

Kết cục	Process đã start?	`Result` giữ gì	Caller nên biết gì
Exit 0	Có	stdout, stderr, exit code 0	Tool hoàn tất theo contract của nó.
Exit khác 0	Có	stdout, stderr, exit code	Tool đã chạy; output có thể giải thích failure.
Deadline/ cancel	Có thể đã start	output đã thu được, cờ timeout	Parent context hay ngân sách thời gian đã kết thúc.
Không start	Không	command và exit code chưa có	Sai executable, quyền hoặc môi trường.

`Result` và `error` đi cùng nhau vì chúng trả lời hai câu khác nhau. `Result` là bằng chứng thu được; `error` nói lần chạy có thỏa contract không. Vì vậy đừng viết API chỉ trả `error` rồi làm mất `stderr` mỗi khi exit code khác 0. Và cũng đừng coi mọi non-zero exit là panic-worthy: tool chẩn đoán có thể dùng exit code để báo một điều kiện đã được dự tính.

Viết supervisor nhỏ trước

Lab dùng `Spec` rất hẹp. `Name` là executable đã được caller quyết định, `Args` là các argument riêng rẽ, còn `Timeout` là ngân sách của lần chạy. `Run` không biết `catalog` là gì, không parse JSON và không tự log. Tách những trách nhiệm này ra giữ cho test có thể kiểm soát process boundary. Hai khối `Run` dưới đây là các đoạn liên tiếp của cùng một function; source đầy đủ, có thể chạy nằm ở `fixed/run.go`.

```
type Spec struct {
    Name    string
    Args    []string
    Timeout time.Duration
}

type Result struct {
    Command  []string
    Stdout   string
    Stderr   string
    ExitCode int
    TimedOut bool
}
```

`context.Context` vẫn là input đầu tiên. Nó nối runner vào request, CLI, hoặc lifecycle của service mà không buộc runner biết signal nào đã xảy ra. Ta tạo một deadline con; parent cancellation đi qua cùng đường đó. `exec.CommandContext` dùng context để interrupt process khi context kết thúc; mặc định `Cancel` của command gọi `Kill` trên process. Đây là behavior documented của standard library, không phải lời hứa rằng toàn bộ process tree trên mọi hệ điều hành đã được dọn sạch.

```
func Run(ctx context.Context, spec Spec) (Result, error) {
    if ctx == nil || strings.TrimSpace(spec.Name) == "" {
        return Result{}, ErrInvalidSpec
    }
    if spec.Timeout <= 0 {
        return Result{}, ErrInvalidSpec
    }

    runCtx, cancel := context.WithTimeout(ctx, spec.Timeout)
    defer cancel()

    cmd := exec.CommandContext(runCtx, spec.Name, spec.Args...)
    var stdout, stderr bytes.Buffer
    cmd.Stdout = &stdout
    cmd.Stderr = &stderr
    err := cmd.Run()
```

Khối đầu kết thúc tại `Run`: nó start process, chờ process và hai stream I/O hoàn tất, rồi giữ error gốc để phân loại. Validation phải đi trước `WithTimeout`, vì `context.WithTimeout(nil, ...)` sẽ panic. Hai buffer được gán trước `Run` để caller không phải đoán một dòng chẩn đoán đến từ `stdout` hay `stderr`.

Sau khi process đã trả, phần còn lại dựng `Result` kể cả khi có error. Đọc nó như đoạn tiếp theo của function ở trên:

```
    result := Result{
        Command:  append([]string{spec.Name}, spec.Args...),
        Stdout:   stdout.String(),
        Stderr:   stderr.String(),
        ExitCode: -1,
    }
    if cmd.ProcessState != nil {
        result.ExitCode = cmd.ProcessState.ExitCode()
    }
    if contextErr := runCtx.Err(); contextErr != nil {
        result.TimedOut = errors.Is(
            contextErr,
            context.DeadlineExceeded,
        )
        return result, fmt.Errorf(
            "command context: %w",
            contextErr,
        )
    }
    if err == nil {
        return result, nil
    }
    var exitErr *exec.ExitError
    if errors.As(err, &exitErr) {
        return result, fmt.Errorf(
            "command exited with code %d: %w",
            result.ExitCode,
            err,
        )
    }
    return result, fmt.Errorf("start command: %w", err)
}
```

Sau `Run`, `ProcessState` chỉ tồn tại nếu process đã thực sự start và kết thúc; `-1` là giá trị sentinel cho case không có exit status. Cuối cùng, runner hỏi context trước khi phân loại `ExitError`: nếu deadline của chính runner đã hết, đó là failure policy quan trọng hơn việc process kịp trả một status nào trong lúc bị ngắt.

Đoạn này chỉ phù hợp với output nhỏ, có giới hạn bởi contract của tool đang gọi. `bytes.Buffer` không phải policy output cho một process có thể in vô hạn. Tool production phải chọn rõ một trong ba hướng: stream có backpressure, capture có giới hạn và truncation được đánh dấu, hoặc chuyển output thành artifact. Đừng lén dùng buffer không giới hạn rồi gọi nó là observability.

Cancel process không đồng nghĩa dọn cả cây process

Đây là chỗ rất dễ mang trực giác từ Chương 12 sang sai. `CommandContext` mặc định gọi `Kill` cho **command process** khi context done. Một command có thể spawn process khác, để process đó giữ pipe mở, hoặc cần một protocol graceful riêng. `os/exec` còn có `WaitDelay` để giới hạn một số trường hợp process con không chịu kết thúc hoặc pipe I/O không đóng; nhưng process-group, job object, signal sequence và cleanup của tool cụ thể là policy phụ thuộc hệ điều hành.

Vì vậy lab không hứa "cancel là không còn process nào". Với helper process của test, vốn không spawn hậu duệ và không giữ pipe mở sau khi bị hủy, test kiểm chứng `Run` trở về khi hết deadline và gắn nhãn kết quả. Với tool có process hậu duệ giữ pipe mở, `Run` vẫn có thể chờ I/O dù command chính đã bị kill; khi đó phải thiết kế `WaitDelay` và cleanup phù hợp. Nếu sau này product có tool spawn worker tree, requirement mới phải gọi tên target OS và lifecycle mong muốn trước khi thiết kế `SysProcAttr` hay process group.

Từ runner sang CLI có thể kể lại

CLI là boundary với con người. Nó không nên nhét parsing flag, validation target, subprocess, formatting và logging vào một `main` dài. `flag.FlagSet` đủ cho command đầu tiên: parse `-target` và `-timeout`, kiểm tra target theo contract của probe, rồi gọi runner với executable đã chọn. Khi `Run` trả về, CLI có thể in report thân thiện; log có cấu trúc chỉ ghi field hữu ích để tái dựng lần chạy.

```
logger.Info(
    "diagnostic completed",
    "tool", result.Command[0],
    "exit_code", result.ExitCode,
    "timed_out", result.TimedOut,
)
```

Đừng log token, header authorization, toàn bộ environment hay raw command line có secret. `slog` không tự làm dữ liệu nhạy cảm biến mất; chính boundary chọn field nào được đi vào log. Chương 16 sẽ biến một lần chẩn đoán đơn lẻ thành tín hiệu có thể quan sát bằng health, metric, trace và SLO. Hiện tại, điều quan trọng hơn là report không đánh tráo "command không chạy được", "command chạy nhưng fail", và "ta đã chủ động ngắt command vì hết ngân sách".

> **Dừng để dự đoán:** Nếu `Run` chỉ trả `error`, incident report nào sẽ mất khi một probe exit status 2 nhưng đã in lời giải thích hữu ích vào `stderr`? Nếu `Run` chỉ trả `Result` và không trả `error`, caller có còn phân biệt được run thành công với run bị deadline không?

Thực hành: chứng minh trạng thái thoát và thời hạn

Mở `labs/part15-incident-command/exercise/run_test.go` trước. Test không gọi `ping`, `curl` hay binary có sẵn của máy. Nó dùng test binary hiện tại làm helper process, nên case success, exit khác 0 và sleep đều chạy như nhau trên Windows, Linux và macOS. Đó là một cách thiết kế test quan trọng: thay vì phụ thuộc vào công cụ có sẵn của máy, ta kiểm soát process đang được quan sát.

```
cd labs/part15-incident-command
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test ./fixed
go vet ./fixed
go test -race ./fixed
```

Contract yêu cầu: spec rỗng hoặc timeout không dương bị từ chối trước khi start; success giữ hai stream riêng và exit code 0; non-zero exit giữ `stderr`, exit code và vẫn cho `errors.As` thấy `*exec.ExitError`; deadline trả error khớp `context.DeadlineExceeded`, đánh dấu `TimedOut`, và không treo. Parent cancellation vẫn trả `context.Canceled`, không bị gọi nhầm là timeout. Test exercise đỏ vì behavior chưa được implement, không phải vì package hỏng.

Đáp án — chỉ đọc sau khi đã tự làm. Tạo child context sau validation, dùng `exec.CommandContext` với name và args tách riêng, rồi gán hai `bytes.Buffer` trước `Run`. Dựng `Result` bất kể `Run` trả lỗi, vì output/exit status có thể vẫn cần cho incident. Kiểm tra `runCtx.Err()` trước `ExitError` để deadline không bị che bởi status của process vừa bị hủy. Bản fixed không dùng shell và không hứa cleanup process tree.

Khi nào runner nhỏ này chưa đủ

Runner này chưa nhận stdin, chưa đặt working directory hay environment riêng, chưa stream output, chưa redact secret, chưa retry, và chưa được xem là executor tổng quát cho input người dùng. Mỗi thứ đó thay đổi attack surface hoặc lifecycle. Ví dụ, `Cmd.Env` thay environment của child chứ không tự merge với environment hiện tại; `Cmd.Dir` làm working directory trở thành một input khác cần validate. Không thêm chúng chỉ vì struct còn chỗ trống.

Điểm dừng của chapter là một tiến bộ nhỏ nhưng có thật: từ việc "gõ lệnh khi hoảng" sang một command có contract về process. `opsprobe` có thể sau này dùng runner này để gọi một diagnostic đã được chọn, nhưng chỉ khi incident requirement thật cần nó. Trước khi thêm metric, dashboard hay Kubernetes, ta phải biết một lần chạy đơn lẻ đã bắt đầu, kết thúc và thất bại như thế nào.

1. Go Team. Package `os/exec`: `Command`, `CommandContext`, `Cmd`, `ExitError`, `WaitDelay` và quy tắc tìm executable. pkg.go.dev/os/exec
2. Go Team. Package `context`: cancellation, deadline và propagation. pkg.go.dev/context
3. Go Team. Package `flag`: `FlagSet` và parse command-line flags. pkg.go.dev/flag
4. Go Team. Package `log/slog`: structured logging và handler. pkg.go.dev/log/slog

Chương 16 — Thấy được hệ thống

Sau Chương 15, `opsprobe` có thể kể lại một lần chạy: command nào được gọi, exit code nào quay về, deadline có hết không. Nhưng một incident hiếm khi chỉ hỏi về một lần chạy. "Từ 9 giờ đến giờ dependency có đang tệ dần không?", "lỗi nằm ở một target hay mọi target?", "request chậm đi qua những bước nào?" là những câu hỏi khác hẳn. Nếu chỉ giữ một dòng log cuối cùng, ta có chi tiết mà không có xu hướng. Nếu chỉ giữ một biểu đồ tổng, ta có xu hướng mà mất câu chuyện của lần fail cụ thể.

Mental model của chương là: **observability tạo ra các phép chiếu có chủ đích từ hành vi hệ thống; mỗi phép chiếu giữ lại một loại bằng chứng và đánh đổi phần còn lại.** Log giữ identity và chi tiết event. Metric aggregate nhiều event để trả lời câu hỏi theo thời gian. Trace giữ đường đi nhân quả của một lần làm việc qua boundary. Health trả lời một câu hỏi hẹp về instance ngay lúc bị hỏi. Không có signal nào là "bản ghi đầy đủ của hệ thống".

Một biểu đồ xanh vẫn có thể nói dối

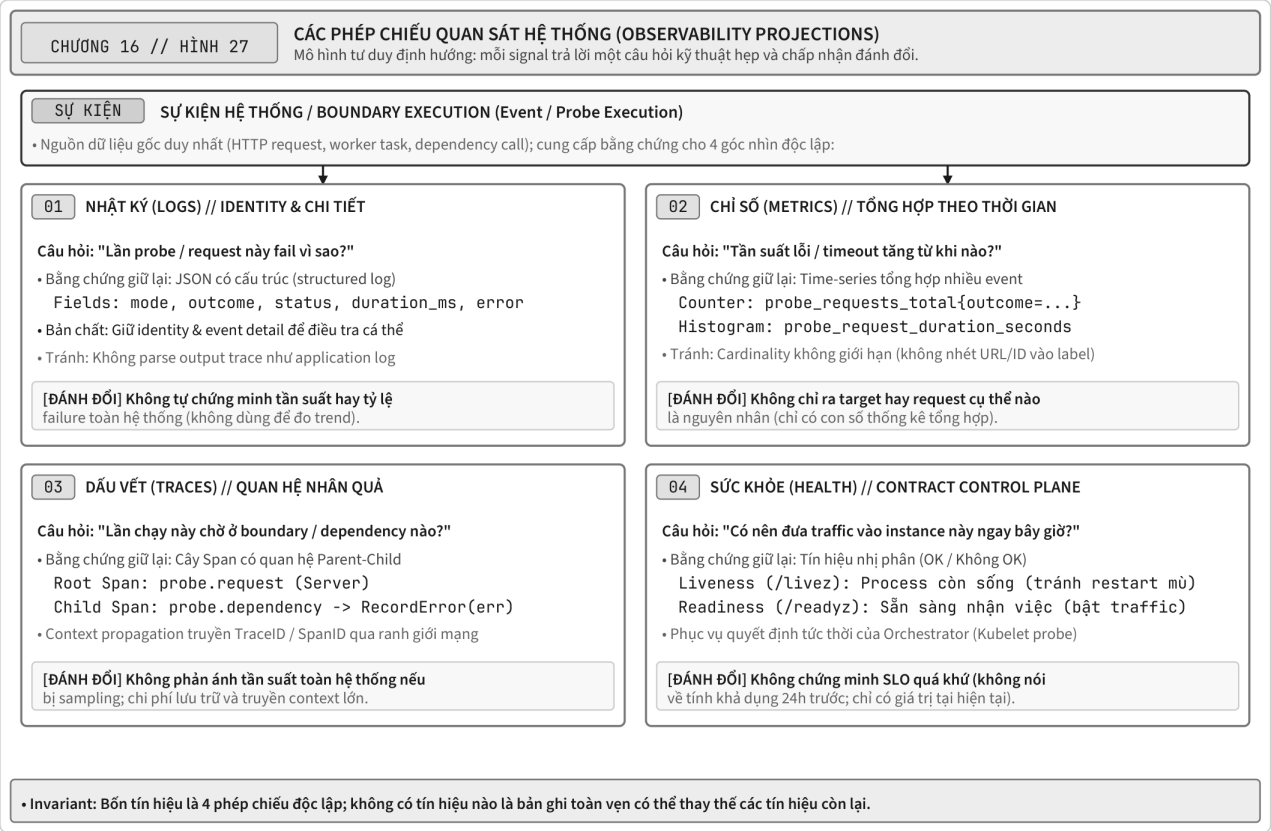
Giả sử một dashboard ghi `probe_success=1` sau lần probe mới nhất. Khi target fail rồi success trở lại, biểu đồ có thể xanh. Nhưng nó không nói trong mười phút vừa rồi có bao nhiêu lần fail, timeout có bị tính là failure không, hay probe có hề chạy lần nào chưa. Đó không phải vấn đề chọn dashboard đẹp; đó là vấn đề semantic của số liệu.

Một counter có ý nghĩa khi ta biết rõ event nào làm nó tăng. Với probe nhỏ ở đây, policy được chọn là: mỗi lần probe hoàn tất tạo đúng một outcome thuộc tập hữu hạn `success`, `failure`, hoặc `timeout`. Availability SLI của lab là số `success` chia cho tất cả outcome đã hoàn tất. Vì thế timeout nằm ở mẫu số và không được làm biến mất denominator. Đây là một policy của probe availability, không phải định nghĩa phổ quát cho mọi product.

```
availability = successes / completed probes
```

Nếu chưa có probe hoàn tất, phép chia không có nghĩa. Đừng trả `1.0` để làm dashboard xanh khi hệ thống chưa có dữ liệu. API lab trả thêm một boolean để caller phân biệt "0% thành công" với "chưa có sample". Một SLI chỉ là cách đo đã định nghĩa; SLO mới là mục tiêu và time window mà team cam kết cho SLI đó. Câu "99.9%" không có ý nghĩa vận hành trước khi ta biết đo cái gì, trong khoảng nào, và action nào sẽ xảy ra khi tiêu error budget.

Bốn câu hỏi, bốn loại bằng chứng



Hình 27 — Các signal trả lời các câu khác nhau. Diagram là mental model, không phải kiến trúc bắt buộc hay pipeline OpenTelemetry hoàn chỉnh.

Bảng 15 — Chọn signal từ câu hỏi trước

Câu hỏi	Signal chính	Điều nó không tự chứng minh
“Lần probe này fail vì sao?”	Log có field và error an toàn	Tần suất hay tỷ lệ failure toàn hệ thống.
“Timeout tăng từ khi nào?”	Metric theo thời gian	Target hay request cụ thể nào là nguyên nhân.
“Lần chạy này chờ ở boundary nào?”	Trace của một execution	Tần suất của mọi execution nếu trace bị sample.
“Có nên đưa traffic vào instance này ngay bây giờ?”	Readiness	SLO của một giờ vừa qua.

Log không phải metric với string dài hơn. Một log event có thể chứa target đã được redaction, exit code, duration, request ID hay trace ID để người điều tra tìm một execution cụ thể. Metric không nên lấy URL đầy đủ, user ID, raw error, trace ID hay một target biến đổi tùy ý làm label mặc định: mỗi tổ hợp label là một time series khác, nên cardinality không giới hạn biến chi phí và thời gian truy vấn thành một incident mới.

Trace cũng không phải log có mũi tên. Nó biểu diễn quan hệ giữa các operation trong một execution; context propagation là thứ nối các span qua HTTP, queue hay subprocess boundary khi hệ thống đã có instrumentation phù hợp. Chương 11 đã dùng `httptrace` để quan sát một client cục bộ, nhưng nó không tự biến thành distributed tracing. Chương này chỉ chuẩn bị semantic để khi dùng OpenTelemetry sau này, ta biết một attribute, span hay sampling decision đang phục vụ câu hỏi nào.

Health là signal dễ bị lạm dụng nhất. Liveness thường trả lời process còn có thể chạy để supervisor không cần restart mù quáng; readiness trả lời instance có nên nhận traffic không. Cả hai là contract cho control plane ở hiện tại. Một `/readyz` trả `200` bây giờ không chứng minh availability 24 giờ trước, cũng không nên gọi một dependency xa cho mọi liveness probe chỉ để endpoint trông “thông minh”. Dependency nào thuộc readiness là một policy dựa trên khả năng phục vụ thực tế, không phải một checklist copy từ framework.

Gắn metric vào outcome, không gắn vào cảm giác

Minimal reproducer dưới đây không export Prometheus format và không thay collector. Nó là core semantic mà exporter sau này có thể đọc: outcome là closed set, snapshot là bản copy nhất quán, và SLI không có giá trị khi chưa có sample. Việc giữ lab trong standard library giúp mình kiểm tra contract trước khi thảo luận SDK hoặc backend.

```
type Outcome string

const (
    OutcomeSuccess Outcome = "success"
    OutcomeFailure Outcome = "failure"
    OutcomeTimeout Outcome = "timeout"
)

type Snapshot struct {
    Completed uint64
    Succeeded uint64
    Failed    uint64
    TimedOut  uint64
}

func (s Snapshot) SuccessRatio() (float64, bool) {
    if s.Completed == 0 {
        return 0, false
    }
    return float64(s.Succeeded) / float64(s.Completed), true
}
```

`Completed` không phải field thừa. Nó là denominator được ghi độc lập để invariant có thể kiểm tra: `Completed == Succeeded + Failed + TimedOut`. Trong lab, một call `Observe` thành công tăng đúng hai thứ: `Completed` và bucket outcome tương ứng. Unknown outcome phải return error trước khi mutate; nếu code nhận raw string tùy ý rồi tạo bucket mới, closed set biến thành cardinality leak ngay tại source.

```
type Recorder struct {
    mu       sync.Mutex
    snapshot Snapshot
}

func (r *Recorder) Observe(outcome Outcome) error {
    r.mu.Lock()
    defer r.mu.Unlock()

    switch outcome {
    case OutcomeSuccess:
        r.snapshot.Succeeded++
    case OutcomeFailure:
        r.snapshot.Failed++
    case OutcomeTimeout:
        r.snapshot.TimedOut++
    default:
        return ErrUnknownOutcome
    }
    r.snapshot.Completed++
    return nil
}

func (r *Recorder) Snapshot() Snapshot {
    r.mu.Lock()
    defer r.mu.Unlock()
    return r.snapshot
}
```

Mutex ở đây không phải "metrics library tự viết". Nó chỉ giữ invariant của một teaching recorder khi nhiều worker từ Chương 9 cùng báo outcome. `Snapshot` trả value copy, vì caller không được sửa state nội bộ để làm counter quay ngược. Nếu exporter sau này đọc snapshot định kỳ, nó vẫn phải có contract riêng về reset khi process restart, timestamp, cumulative counter hay delta. Bản ghi in-memory này không có retention, không có window và không phải evidence lịch sử sau khi process chết.

> **Dừng để dự đoán.** Một probe kết thúc với lỗi `context.DeadlineExceeded` (timeout). Sự kiện này có đồng nghĩa với việc máy chủ đích đã sập hoàn toàn không? Trong SLI đo độ khả dụng, kết quả timeout được phân loại vào đâu?

Đáp án — chỉ đọc sau khi đã tự làm Lỗi timeout chỉ chứng minh caller không nhận được phản hồi trong ngân sách thời gian đã định, hoàn toàn không đủ để suy ra dịch vụ đích đã chết. Trong thực tế, dịch vụ đích có thể vẫn đang chạy nhưng bị nghẽn hàng đợi xử lý, hoặc phản hồi đã được gửi đi nhưng bị chậm trễ trên mạng, thậm chí một tác vụ ghi dữ liệu đã được commit thành công ở phía sau trước khi caller hết giờ chờ.

Theo hợp đồng SLI cụ thể được chọn trong lab này, timeout được tính vào mẫu số thất bại (denominator failure) bởi vì dưới góc nhìn của caller, lượt probe đó đã không thu được kết quả hợp lệ trong khung thời gian cam kết. Tuy nhiên, đây là một lựa chọn thiết kế chính sách, không phải quy luật bất biến cho mọi hệ thống. Một số dịch vụ có thể tách timeout thành SLI riêng về độ trễ, phân loại theo tác vụ idempotent, hoặc ghi nhận nhãn riêng `outcome="timeout"` để đội ngũ trực vận hành không đánh đồng nghẽn mạng với sự cố sập ứng dụng. Khi sản phẩm thay đổi chính sách phân loại, tên metric, công thức SLI và ngưỡng cảnh báo đều phải được cập nhật đồng bộ.

SLO không phải một alert thật to

Một SLO đặt expectation có thể đo; alert là policy buộc con người chú ý khi rủi ro với expectation ấy đủ lớn. Không nên alert cho mỗi `failure` event: một target test bị tắt có thể tạo noise, còn một error rate thấp nhưng tăng nhanh trên traffic thật có thể đáng điều tra. Khi có SLI, window và objective, team mới có thể chọn alert dựa trên tốc độ tiêu error budget và runbook phù hợp.

Điều này cũng giải thích vì sao `Completed=0` không được gọi là 100% availability. Không có sample thì không thể suy ra rate; trạng thái hợp lý có thể là `unknown`, dashboard annotation, hoặc alert về telemetry gap tùy

policy. Đừng sửa bằng cách chia cho `max(1, completed)`: công thức sẽ chạy, nhưng evidence đã bị thay đổi.

Một alert tốt dẫn tới thao tác tiếp theo. Nó nên chỉ ra service/target, SLI bị ảnh hưởng, time window, owner và nơi xem detail an toàn. Metric đưa người điều tra tới cohort đang xấu; log hay trace giúp trả lời "cái gì khác thường trong cohort đó?". Đó là một vòng điều tra, không phải bốn sản phẩm quan sát được đặt cạnh nhau.

Thực hành: giữ mẫu số và ranh giới nhãn quan sát

Mở `labs/part16-observability-evidence/exercise/recorder_test.go` trước. Bài không yêu cầu cài Prometheus, OpenTelemetry hay dashboard. Test bắt contract nhỏ nhưng hay bị làm sai: no data không được giả là 100%; unknown outcome không được làm state thay đổi; và nhiều goroutine ghi outcome không làm vỡ tổng số. Đây là nơi race detector từ Chương 8 trở thành một phần của observability correctness, không phải chỉ là một lệnh "chạy cho yên tâm".

```
cd labs/part16-observability-evidence
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test ./fixed
go vet ./fixed
go test -race ./fixed
```

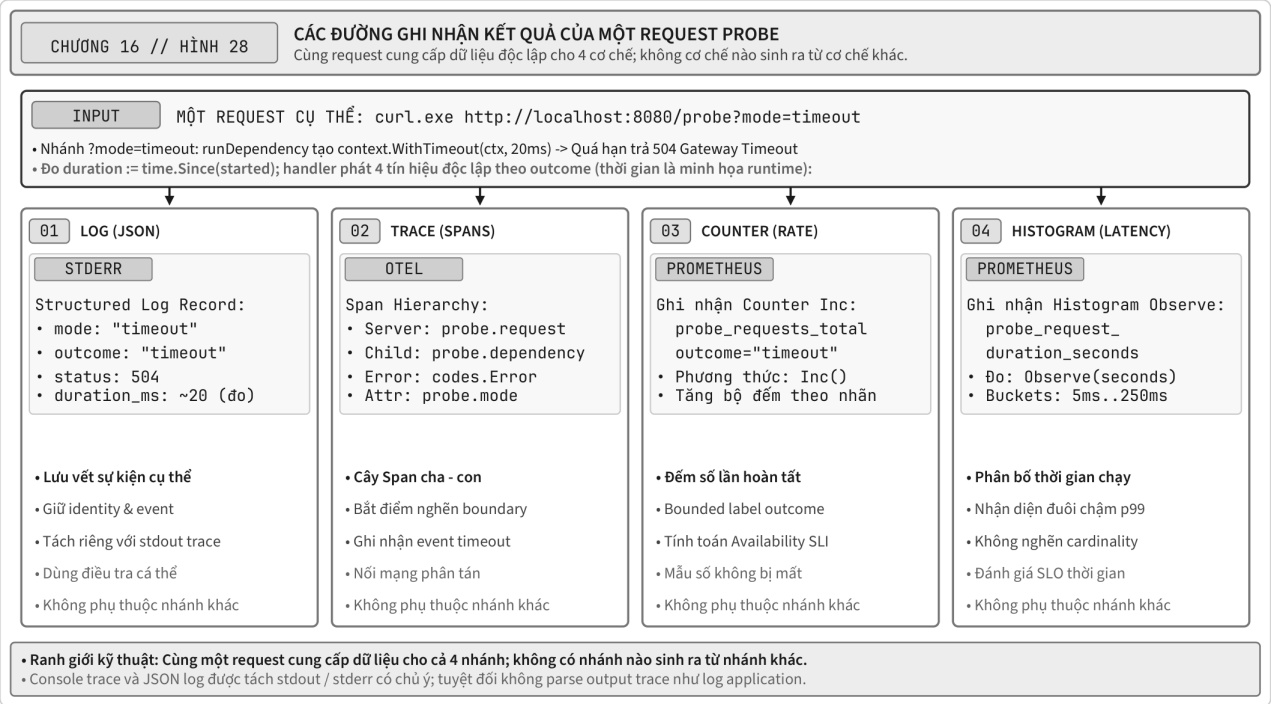
Contract cố ý không nhận `target string`, URL hay error text. Những dữ liệu đó thuộc log/trace hoặc một data model khác có retention và redaction. `Recorder` chỉ giữ một metric aggregate bounded. Khi tests xanh, hãy nhìn `fixed/` và giải thích vì sao `Snapshot` trả struct value thay vì `*Snapshot`, và vì sao một `sync.Mutex` chung là đủ cho correctness trước khi có lý do đo contention.

Đáp án — chỉ đọc sau khi đã tự làm. Khai báo `Outcome` là closed set rồi `switch` trước khi tăng `Completed`. Đặt lock bao quanh cả validation và mutation để invalid outcome không thể race với snapshot hay tạo

half-update. `SuccessRatio` trả `(float64, bool)`; boolean false là semantic cho no data, không phải error transport. Bản fixed ưu tiên một invariant dễ kiểm chứng hơn một API nhìn "linh hoạt".

Giai đoạn hai: ba tín hiệu đi qua một service thật

Recorder vừa rồi cố ý không biết Prometheus hay OpenTelemetry. Bây giờ, khi câu hỏi đã rõ, ta nối nó với tool thật mà không xây một platform quan sát khổng lồ. `labs/part16-real-signals` là một HTTP service nhỏ. Một request vào `/probe` tạo một server span `probe.request`; việc giả lập dependency tạo child span `probe.dependency`; kết quả cuối cùng được ghi thành JSON log, Prometheus counter và histogram. Console exporter in trace ra stdout để thấy quan hệ parent-child ngay trên máy local.



Hình 28 — Cùng request cung cấp dữ liệu cho log, trace, counter và histogram. Các đường ghi nhận thể hiện những góc nhìn khác nhau, không phải log sinh counter hay trace sinh histogram.

Đây là một evidence path, không phải ba cách đặt tên cho cùng một dòng. Counter trả lời **bao nhiêu lần đã hoàn tất theo outcome**. Histogram trả lời **duration phân bố thế nào so với bucket đã chọn**. Trace giữ quan hệ của **một** request với dependency nó gọi. Log giữ status và duration của event để người điều tra không phải suy từ aggregate về một request cụ thể.

Mở `labs/part16-real-signals/README.md`, đọc nhiệm vụ tự điều tra, rồi chạy service. Console trace và JSON log được tách stdout/stderr có chủ ý: đừng parse output trace như log application trong production.

```
cd labs/part16-real-signals
go test ./...
go vet ./...
go test -race ./...
go run ./cmd/probe-api
```

Ở terminal khác, chạy từng failure mode. `curl.exe -i` giữ nguyên HTTP status thay vì để shell biến response 503/504 thành lỗi khó đọc.

```
curl.exe -i http://localhost:8080/probe?mode=ok
curl.exe -i http://localhost:8080/probe?mode=slow
curl.exe -i http://localhost:8080/probe?mode=fail
curl.exe -i http://localhost:8080/probe?mode=timeout
curl.exe -s http://localhost:8080/metrics
```

`ok` và `slow` đều là `success`; chậm không tự động đồng nghĩa với failure. `fail` là dependency trả lỗi ngay và service trả `503`. `timeout` là caller hết 20 ms ngân sách trước dependency giả lập hoàn tất và service trả `504`. Đây là failure injection có boundary cụ thể, không phải latency ngẫu nhiên. So sánh JSON log, counter, histogram và trace trước khi mở source: evidence nào phân biệt `fail` với `timeout` nhanh nhất? evidence nào cho thấy `slow` vẫn thành công nhưng đã đi qua bucket khác?

Bảng 16 — Instrument của lab và contract của nó

Instrument	Event boundary	Tập dữ liệu cố ý giữ hẹp	Điều không được suy ra
`probe_requests_total`	Handler hoàn tất.	`outcome`: `success`, `failure`, `timeout`.	Counter không giữ request nào fail.
`probe_request_duration_seconds`	Cùng handler.	Cùng label; bucket 5–250 ms.	Bucket không phải SLO hay percentile chính xác.
JSON log	Cùng handler.	mode, outcome, HTTP status, duration.	Một log không là rate theo thời gian.
Hai spans	Request và dependency.	mode/outcome; error status khi fail.	Một trace không là thống kê cho mọi request.

Histogram bucket là một quyết định đo lường. Lab chọn 5, 10, 25, 50, 100 và 250 ms để phân biệt timeout 20 ms với slow 40 ms trên laptop; nó không được chép thành threshold production. Muốn trả lời câu hỏi latency ở 2 giây, bucket cao nhất 250 ms đã làm mất độ phân giải cần thiết. Muốn để mỗi route, raw URL, user ID, trace ID hay error message làm label thì một metric aggregate lại bị đẩy thành kho event vô hạn. Lab chỉ đặt `outcome` vào label; `mode` có ít giá trị ở đây, nhưng vẫn không cần để trả lời câu hỏi vận hành đã chọn.

Dừng để dự đoán. Sau khi chạy `ok`, `slow`, `fail`, `timeout` một lần, counter `success` là bao nhiêu? Restart process rồi scrape lại `/metrics`: vì sao con số reset không chứng minh probe cũ chưa từng diễn ra? Prometheus server lưu sample theo thời gian là boundary khác với client process đang export số cumulative hiện tại.

Test của lab không giả vờ thay local collector hay dashboard. Nó kiểm tra ba contract có thể kiểm tra tự động: metric chỉ có label bounded, JSON log có `outcome`, và child span thực sự có parent là request span. Source dùng Prometheus Go client để tạo exposition `/metrics`, và OpenTelemetry Go SDK với console exporter dành cho debugging local. Khi đưa trace sang collector hay backend, endpoint, sampling, retention, privacy và quyền truy

cập là policy mới phải được thiết kế; không ghi secret vào attribute chỉ để trace dễ tìm hơn.

Có sample chưa có nghĩa tín hiệu đã trả lời sự cố

Trong `part16-real-signals`, các mode local giữ instrumentation dễ quan sát. Giai đoạn tiếp theo ở `projects/opsprobe/failurelab` vẫn dùng `slog`, registry Prometheus riêng và console exporter OpenTelemetry, nhưng thay dependency giả bằng HTTP loopback có checkpoint. Mục đích không phải thêm dashboard: nó buộc ba tín hiệu nói về cùng một lượt probe khi timeout hoặc body bị cắt.

Histogram `opsprobe_probe_duration_seconds` đo thời gian trong probe; `failurelab_job_seconds` đo từ admission tới probe completion, gồm queue delay nhưng không gồm chờ consumer lấy result. Gauge `failurelab_running` chỉ bao quanh probe đang chạy, còn `failurelab_queue_depth` là job chưa được worker nhận. Chúng không thay gauge worker sống của dự án. Worker chờ send có thể còn sống dù running bằng 0; chỉ join `Done` mới xác nhận nó đã thoát.

Đọc Chương 20 để tự chọn bằng chứng phân biệt trước khi chạy. Test xác nhận có log/outcome, histogram sample và span, không xác nhận latency production hay một backend đã lưu trace bền vững. Khi chạy workload hữu hạn, ghi cả số bị từ chối: throughput trên việc được nhận mà bỏ mất rejection có thể làm một service đang shed phần lớn tải trông như khỏe. Không dùng target ID tùy ý làm metric label; lab giữ các outcome hữu hạn, còn ID nằm trong log/span theo policy dữ liệu của fixture.

Khi dùng thư viện thật

Khi requirement đã cần `/metrics`, histogram duration, exemplars, trace propagation, collector hoặc backend query, hãy dùng client library và semantic convention được duy trì thay vì tự phát minh wire format. Nhưng SDK không thể chọn SLI cho anh. Trước khi thêm một instrument, hãy viết câu hỏi vận hành, event boundary, unit, label cardinality, reset/restart behavior, ownership và action khi signal xấu. Nếu không trả lời được, thêm metric chỉ tạo thêm dữ liệu chứ không thêm khả năng thấy hệ thống.

Điểm dừng của chương là một thay đổi trong cách nhìn: outcome của Chương 15 không còn là một `Result` bị in ra rồi quên. Nó có thể thành evidence theo thời gian, nhưng chỉ sau khi ta nói thật outcome nào được đếm, signal nào giữ chi tiết, và inference nào số liệu chưa đủ sức làm. Chương 17 sẽ chuyển từ "instance có thể quan sát" sang "artefact được đóng gói và điều phối"; readiness khi ấy sẽ gặp một control plane thật, chứ không chỉ là một endpoint đẹp.

1. OpenTelemetry Authors. Signals: traces, metrics, logs và profiles. opentelemetry.io/docs/concepts/signals/
2. Prometheus Authors. Instrumentation: naming, labels, counters và metric design. prometheus.io/docs/practices/instrumentation/
3. Google SRE. Alerting on SLOs: SLI, objective, error budget và alert policy. sre.google/workbook/alerting-on-slos/
4. Go Team. Package `sync`: `Mutex` và synchronization primitives. pkg.go.dev/sync
5. Prometheus Authors. Prometheus Go client: metric primitives và HTTP exposition. pkg.go.dev/github.com/prometheus/client_golang/prometheus
6. OpenTelemetry Authors. Go exporters: console exporter cho local debugging; collector/backend cho telemetry path thực tế. opentelemetry.io/docs/languages/go/exporters/

Chương 17 — Đóng gói và điều phối

Một chương trình Go chạy được bằng `go run` thường tạo một ảo giác dễ chịu: source, compiler, shell, file cấu hình và process đang ở gần nhau, nên "chạy được" trông như một sự thật đơn giản. Nó không còn đơn giản khi program phải được build một lần, chạy trên máy khác, nhận cấu hình khác, bị dừng giữa chừng, rồi được thay thế khi node biến mất. Lúc ấy, câu hỏi không phải chỉ là "làm sao chạy Docker?" mà là: **ai sở hữu trạng thái mà ta muốn hệ thống duy trì?**

Mental model của chương là: **image đóng gói contract khởi động của một process; orchestrator duy trì desired state bằng vòng reconciliation, không lặp lại một câu lệnh start.** Image nói process mặc định nào, filesystem nào và default argument nào có mặt. Một workload declaration nói bao nhiêu bản sao của artefact ấy cần tồn tại và policy nào áp dụng. Controller quan sát trạng thái hiện tại, chọn một action nhỏ để đưa nó gần desired state hơn, rồi lại quan sát. Không bước nào trong số này tự chứng minh application đã phục vụ đúng business request.

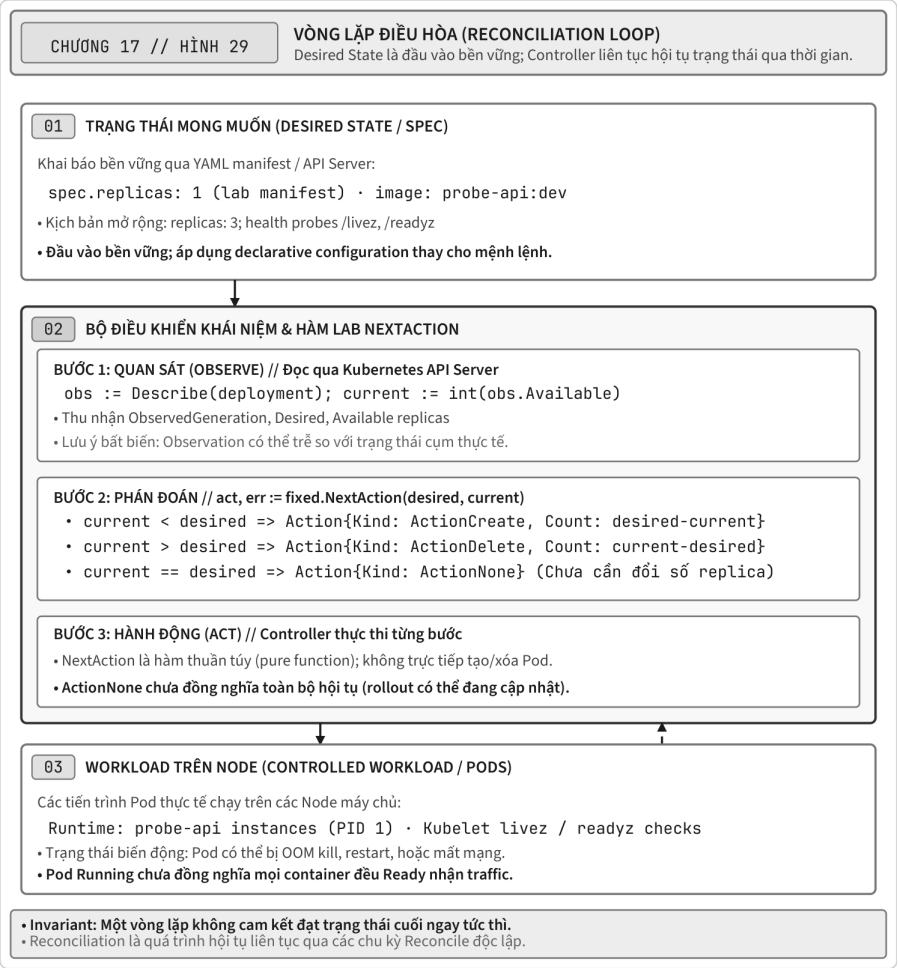
Một image chạy được chưa phải một deployment

Giả sử team đã build một image và chạy thử một container. Binary in log, cổng mở, request đầu tiên trả `200`. Khi đưa cùng image vào cluster, một instance bị restart vì process kết thúc, instance khác chưa nhận traffic vì chưa ready, còn deployment vẫn đang thay Pod cũ bằng Pod mới. Nếu gọi tất cả là "Docker bị lỗi", ta đánh mất ba boundary khác nhau:

Bảng 17 — Ba contract không nên trộn làm một

Boundary	Câu hỏi chính	Bằng chứng phù hợp
Artifact	Process nào, default args nào, file nào được mang theo?	Image config, digest, Dockerfile, SBOM nếu có.
Runtime	Process có start, nhận signal, mở listener và thoát theo contract không?	Exit status, log, health/readiness, resource observation.
Control plane	Desired state có đang được đưa gần current state không?	Spec, status, event và action của controller.

Container không phải một máy ảo thu nhỏ, cũng không biến binary thành service đúng nghĩa. Nó là cách đóng gói filesystem và execution configuration cho một process. Còn `docker run` là một thao tác mệnh lệnh: nó yêu cầu runtime tạo một container ngay lúc này. Kubernetes làm việc ở lớp khác: object có `spec` diễn tả desired state, `status` báo điều đã quan sát, và các controller liên tục cố đưa hai thứ gần nhau hơn. Vì thế, “đã apply YAML” không đồng nghĩa “đã có traffic phục vụ”, cũng như “Pod Running” không đồng nghĩa “mọi container đều ready”.



Hình 29 — Desired state là input bền hơn một lệnh start. Controller chạy nhiều vòng; trạng thái cluster có thể luôn thay đổi.

Image là process contract, không phải bản sao của laptop

OCI image configuration có những execution default như platform, working directory, user, `Entrypoint` và `Cmd`. Với image chạy một binary Go, cách đọc hữu ích là: `ENTRYPOINT` chọn executable bền vững; `CMD` là default argument mà runtime có thể override. Cả hai ở exec form giữ executable và từng argument tách riêng, tránh một shell parent làm mờ quoting và đường đi của signal.

```
FROM golang:1.27.1 AS build
WORKDIR /src

COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath \
  -o /out/probe-api ./cmd/probe-api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/probe-api /probe-api
USER nonroot:nonroot
ENTRYPOINT ["/probe-api"]
CMD ["--config", "/etc/probe-api/config.json"]
```

Đây là một **mẫu cấu trúc**, không phải Dockerfile có thể copy nguyên xi cho lab hiện có: repository của sách không có `cmd/probe-api` hay `go.sum` ở root. Chi tiết ấy quan trọng hơn vẻ "production-ready" của snippet. Một Dockerfile chỉ có ý nghĩa khi đường dẫn package, dependency, target platform và runtime assets đều khớp source thật. Đừng thêm `CGO_ENABLED=0`, distroless hay `nonroot` như một câu thần chú; hãy kiểm tra binary có cần C library, CA certificates, timezone data, shell hay write path nào không.

`RUN` chạy ở **build time** và tạo layer; `CMD` chỉ định default khi **container start**. Nhầm hai điều này có thể khiến test hoặc migration bị chạy lúc build, còn image khi start lại không có main process đúng. Tương tự, `COPY` source rồi build trong image không tự làm source reproducible: base image tag có thể trôi, module download cần dependency graph được kiểm soát, và final image phải được nhận diện bằng digest nếu identity chính xác quan trọng hơn một tag dễ đọc.

> **Dừng để dự đoán:** Nếu runtime override `CMD` thành `--config /run/config.json`, executable nào vẫn được chạy? Nếu `ENTRYPOINT` là shell form, process nào thực sự là PID 1 và signal termination sẽ đi qua đâu? Đừng trả lời bằng thói quen: mở image config hay Dockerfile rule của runtime đang dùng.

Container vẫn sống trên máy thật



Hình 30 — Pleiades tại NASA Ames, ảnh ngày 8-10-2008 của Marco Librero/NASA Ames Research Center, qua Wikimedia Commons. Asset thuộc public domain theo PD-USGov-NASA; bản sách chỉ chuyển grayscale. Ảnh cho thấy hạ tầng vật lý, không là bằng chứng NASA dùng Go hay Kubernetes trên hệ thống này.

Ảnh phòng máy giúp đặt lại một giới hạn dễ bị YAML che khuất: process cuối cùng vẫn cạnh tranh CPU, RAM, I/O và mạng trên máy thật. Namespace trên Linux tạo những góc nhìn tách biệt cho một số resource; cgroup tổ chức và giới hạn tài nguyên theo policy. Chúng không biến CPU quota thành thêm core vật lý, cũng không biến memory limit thành một lượng heap Go được bảo đảm cấp phát.

Hãy điều tra một tình huống giả định: Pod có memory limit, service không tăng số request, nhưng vẫn bị OOM kill sau khi thêm native library. Heap profile Go chưa giải thích toàn bộ RSS; ngoài Go-managed memory còn có native allocations, mappings và các thành phần OS kế toán theo policy của môi trường. `GOMEMLIMIT` là soft limit cho phần memory runtime Go quản lý, không phải RSS cap hay quyền yêu cầu kernel bỏ qua cgroup limit. Tài liệu `runtime/debug.SetMemoryLimit` nêu cả phần bị loại khỏi budget, như memory do code ngoài runtime quản lý.

Vì vậy đọc limit trong declaration, lượng dùng thực tế và profile là ba bước khác nhau. Chương 10 cho công cụ đo allocation; Chương 20 phân biệt CPU parallelism với quota; ở đây nhiệm vụ của deployment là đặt resource policy có headroom phù hợp workload và quan sát throttle/OOM/latency. Không chọn một phần trăm memory cố định cho mọi service. Bài điều tra kết thúc khi anh có thể nói thành phần nào được đo, thành phần nào chưa đo và ai thực thi ngưỡng dừng, không phải khi đã thêm một biến môi trường vào manifest.

Một process làm gì khi control plane muốn dừng nó?

Chương 12 đã tách lifecycle HTTP server khỏi `main`: signal dẫn tới context của application, server ngừng nhận connection mới, rồi `Shutdown` chờ request đang hoạt động trong một grace period. Contract ấy vẫn thuộc về program. Container runtime và kubelet chỉ tạo thêm một upstream event: tới lúc termination, process nhận signal theo runtime policy; Kubernetes cũng có thể chờ một grace period trước khi buộc dừng. Không có lời hứa chung rằng mọi handler active tự bị cancel, mọi subprocess tự biến mất, hay mọi request sẽ hoàn thành.

Vì vậy `ENTRYPOINT ["/probe-api"]` có ý nghĩa vận hành hơn `ENTRYPOINT /probe-api --config ...` khi intent là để binary trở thành process chính. Exec form không giải quyết graceful shutdown thay chương trình, nhưng nó không chèn một command shell giữa runtime và binary. `SIGTERM` vẫn chỉ là một tín hiệu: application phải quyết định cách

chuyển nó thành drain, deadline và exit code; hệ điều phối phải được cấu hình với thời gian đủ cho policy đó.

Đây cũng là lúc đọc lại readiness của Chương 16 cho đúng scope. Readiness trả lời "có nên gửi traffic mới vào instance này ngay bây giờ?". Nó không phải một lời hứa image build đúng, không phải liveness, không phải lịch sử SLO, và không phải cái cớ để restart process vì dependency xa vừa chậm. Khi readiness false, traffic routing có thể dừng hướng request mới tới instance; controller vẫn có thể giữ instance đó tồn tại để nó tự hồi phục hoặc để người vận hành điều tra.

Desired state không phải một script dài

Một manifest minh họa dưới đây chỉ đặt ba ý rõ ràng: identity của artefact, số bản sao mong muốn và readiness contract. Nó không tuyên bố rằng đây là toàn bộ configuration của một production service.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: probe-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: probe-api
  template:
    metadata:
      labels:
        app: probe-api
    spec:
      containers:
        - name: probe-api
          image: registry.example/probe-api@sha256:REPLACE_ME
          args: ["--config", "/etc/probe-api/config.json"]
          readinessProbe:
            httpGet:
              path: /readyz
              port: 8080
```

`replicas: 3` không có nghĩa "chạy lệnh start đúng ba lần rồi xong". Nó là desired count. Controller có thể tạo Pod khi thiếu, tạo replacement khi một Pod bị mất, hoặc từng bước thay replica khi template thay đổi. Pod là đơn vị tương đối ephemeral; một Pod đã được schedule không tự nhảy sang node khác. Higher-level controller mới là thứ tạo Pod khác để workload tiếp tục tiến về desired state.

`image` dùng digest trong ví dụ vì digest là identity bất biến của artefact cụ thể. Tag như `:latest` là một tên có thể trỏ đến image khác ở thời điểm khác; nó có thể hữu ích cho workflow phát triển, nhưng không nên được diễn giải như bằng chứng deployment đang chạy đúng build nào. Placeholder `REPLACE_ME` cố ý làm manifest chưa apply được: một digest bịa ra sẽ tạo configuration trông hoàn chỉnh nhưng không có artefact để kéo.

Một pod spec cũng không phải nơi để nhét mọi config trong code. Environment variable, Secret, ConfigMap, volume, service account, resource request/limit và network policy đều có semantics, quyền truy cập và lifecycle riêng. Khi một field ảnh hưởng tới khả năng start hay readiness, nó là một phần của runtime contract phải test và quan sát; nó không phải "YAML glue" để bỏ qua code review.

Đọc reconciliation bằng một hàm nhỏ

Kubernetes controller thật quan sát API object, có cache, retry, error handling và nhiều resource khác nhau. Ta không mô phỏng cluster trong lab. Thay vào đó, minimal reproducer giữ lại phần tư duy cốt lõi: một lần quan sát trả về **action kế tiếp**, không hứa system đã hội tụ ngay sau action đó.

```
type ActionKind string

const (
    ActionNone   ActionKind = "none"
    ActionCreate ActionKind = "create"
    ActionDelete ActionKind = "delete"
)

type Action struct {
    Kind  ActionKind
    Count int
}

func NextAction(desired, current int) (Action, error) {
    if desired < 0 || current < 0 {
        return Action{}, ErrNegativeReplicaCount
    }
    switch {
    case current < desired:
        return Action{
            Kind:  ActionCreate,
            Count: desired - current,
        }, nil
    case current > desired:
        return Action{
            Kind:  ActionDelete,
            Count: current - desired,
        }, nil
    default:
        return Action{Kind: ActionNone}, nil
    }
}
```

Hàm cố tình không `for` cho đến khi count bằng nhau. Nếu nó tự gọi “start” ba lần, nó sẽ che những event thật giữa các action: create có thể fail, instance có thể chưa ready, current state có thể thay đổi bởi một actor khác, hoặc controller có thể cần rate limit. `ActionNone` không chứng minh application healthy; nó chỉ nói snapshot count được đưa vào hàm đang bằng desired count. Sự khiêm tốn này là một property quan trọng của code điều phối.

Trong controller thật, desired state có thể đổi, cache có thể cũ, còn action có thể thành công trước khi caller nhận được response. `NextAction` trả cùng action cho cùng input chỉ chứng minh tính tất định của phép tính, không chứng minh side effect an toàn khi lặp. Với snapshot desired 3/current 1, thực thi "tạo 2" hai lần có thể tạo thành 5 instance. Muốn lặp an toàn, controller cần đọc lại trạng thái và thiết kế identity/ownership cho thao tác tạo; timeout không cho biết thao tác trước đã xảy ra hay chưa. Chương 21 và 23 sẽ kiểm tra boundary này với queue, observation cũ và cửa sổ crash. Đây là bài toán của control loop và API, không phải ngữ nghĩa ngôn ngữ Go.

Bảng 18 — Điều một vòng reconciliation được phép kết luận

Observation	Action hợp lý của toy reconciler	Điều vẫn chưa biết
Desired 3, current 1	Tạo thêm 2 instance.	Hai instance đó có start hay ready không.
Desired 3, current 5	Dừng/bỏ 2 instance.	Instance nào nên bị chọn, connection có cần drain không.
Desired 3, current 3	Không đổi replica count.	Image revision, traffic, latency, SLO và resource health.

Thực hành: tự hoàn thành vòng lặp điều hòa một bước

Mở `labs/part17-reconciliation-contract/exercise/reconcile_test.go` trước. Bài không cần Docker daemon, cluster, credential hay network. Test đưa desired/current count vào và đòi action bounded, tất định, không biến input âm thành một action "lạ". Test không thực thi side effect, nên không chứng minh idempotency của một API tạo instance. Đây là cách tách phép tính chênh lệch khỏi hệ thống điều phối thật.

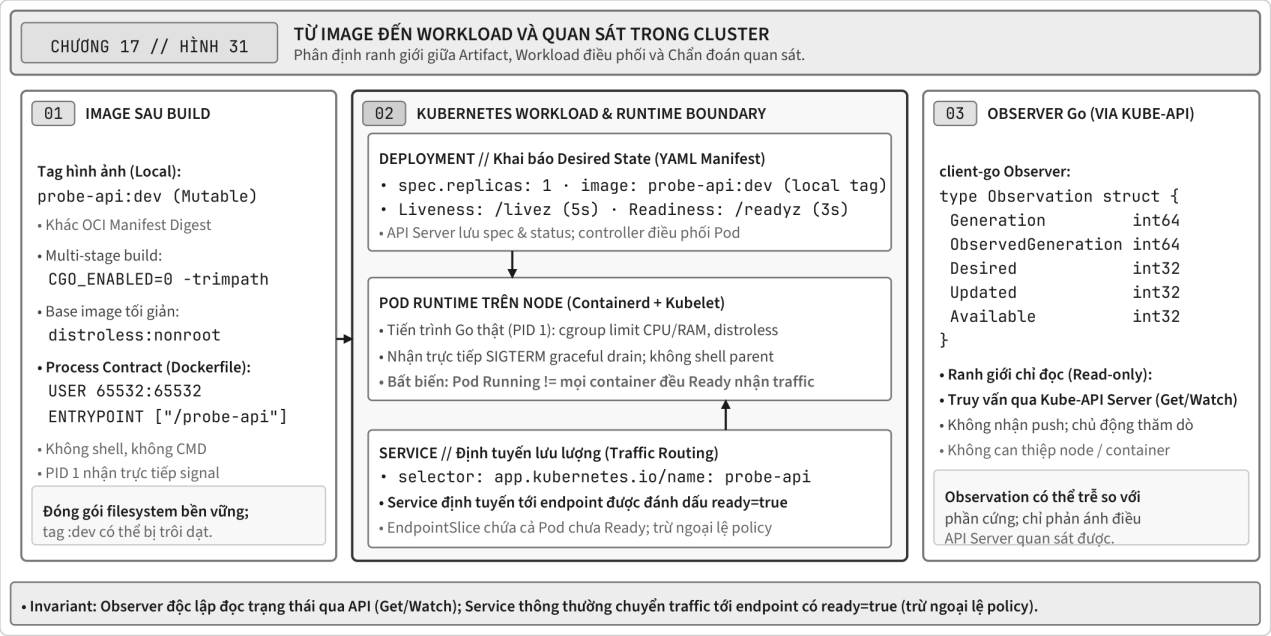
```
cd labs/part17-reconciliation-contract
go test -tags exercise ./exercise
go test ./fixed
go vet ./fixed
go test -race ./fixed
```

Hãy viết `NextAction` từ contract, không mở `fixed/` trước. Câu hỏi trước khi code là: action nào mang **chênh lệch** giữa desired và current, và case equal cần trả gì để caller không phải đoán? Sau khi tests xanh, tự thay current bằng kết quả giả định của action rồi gọi lại. Điều gì xảy ra ở vòng thứ hai? Đó là phép thử hội tụ khi observation được cập nhật, chưa phải bằng chứng lặp side effect an toàn.

Đáp án — chỉ đọc sau khi đã tự làm. Validate cả hai count trước, vì `desired - current` với số âm có thể biến một configuration lỗi thành action hợp lệ giả. Sau đó trả `create` hay `delete` với `Count` đúng bằng độ chênh; equal trả `ActionNone` với `Count` zero. Không mutate input, không loop, không sleep và không cố "chờ ready": mỗi thứ đó thuộc boundary khác.

Giai đoạn hai: đưa service thật vào container và cụm máy chủ

Vòng lặp `NextAction` ở trên giúp ta hiểu bản chất của reconciliation mà chưa cần tới cluster thật. Nhưng khi đưa một service Go vào vận hành, code không còn chạy trực tiếp trên máy lập trình. Nó phải được đóng gói vào một OCI image và giao phó cho một hệ điều phối. `labs/part17-container-kubernetes` dùng chính HTTP service `probe-api` từ `labs/part16-real-signals` để thực hành các ranh giới này trên môi trường cục bộ có Docker và Kubernetes.



Hình 31 — Deployment khai báo Pod template cùng probes, Service chọn đường traffic, runtime trên node chạy image. Observer chỉ đọc qua API server; sơ đồ lược bỏ các component điều phối trung gian.

Trước khi bắt đầu, lab kiểm tra các công cụ bắt buộc bằng script `scripts/verify-prereqs.ps1`. Nó đòi hỏi `docker`, `kubectl` và `kind` có mặt trên máy local. Nếu môi trường hiện tại chưa cài đặt những công cụ này, script sẽ dừng lại với thông báo rõ ràng; khi đó, các lệnh dưới đây là đường dẫn thực thi minh họa được chuẩn hóa để anh tái lập khi có đủ công cụ, tuyệt đối không tự ý thay thế bằng một cluster cloud có phí.

```
cd labs/part17-container-kubernetes
./scripts/verify-prereqs.ps1
```

Image là contract của process

Dockerfile của lab áp dụng mô hình build nhiều giai đoạn (multi-stage) để tách biệt môi trường biên dịch với môi trường thực thi. Giai đoạn đầu tiên dùng image Go chính thức để tải module và biên dịch binary với `CGO_ENABLED=0` và cờ `-trimpath` nhằm loại bỏ đường dẫn file hệ thống cục bộ khỏi binary. Giai đoạn cuối cùng chỉ sao chép duy nhất file binary sang base image tối giản `distroless/static-debian12:nonroot`.

```
FROM golang:1.27.1 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath \
  -o /out/probe-api ./cmd/probe-api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/probe-api /probe-api
USER 65532:65532
ENTRYPOINT ["/probe-api"]
```

Với lệnh chạy dưới đây, `ENTRYPOINT` ở exec form chạy binary trực tiếp làm PID 1 trong container; nếu thêm `--init`, tiến trình init sẽ giữ vị trí ấy. Server đọc cổng từ `PORT` như ở Chương 12 và 16. `--read-only` đặt root filesystem ở chế độ chỉ đọc, không cấm ghi vào volume hay tmpfs được mount riêng. `--cap-drop ALL` bỏ Linux capabilities, còn `USER 65532:65532` chọn tài khoản không phải root. Các cấu hình này thu hẹp quyền, không thay thế toàn bộ chính sách cô lập container.

```
# Chạy từ thư mục gốc của repository (minh họa)
docker build `
  -f labs/part17-container-kubernetes/Dockerfile `
  -t probe-api:dev `
  labs/part16-real-signals

docker run --rm --name probe-api `
  --read-only --cap-drop ALL `
  -p 18080:18080 -e PORT=18080 probe-api:dev
```

Ở một terminal khác, ta kiểm tra các contract vận hành:

```
curl.exe -i http://localhost:18080/readyz
curl.exe -i http://localhost:18080/probe?mode=slow
docker inspect --format '{{.Config.User}}' probe-api
docker stop -t 5 probe-api
```

Trong image của lab, `docker stop -t 5` gửi `SIGTERM` tới process chính và chờ tối đa 5 giây; nếu process chưa thoát, Docker mới dùng `SIGKILL`. Khoảng chờ không bảo đảm mọi request hoàn tất: cần quan sát drain và exit của chương trình để kiểm tra contract graceful shutdown. Đánh đổi của distroless và read-only filesystem là gì? Container sẽ không có shell (`/bin/sh`), không có trình quản lý gói, và binary không thể tùy tiện ghi file tạm nếu không được mount một thư mục riêng biệt. Ít công cụ và package hơn giúp giảm phần mềm cần quản lý, nhưng không chứng minh binary, dependency hay cấu hình không có lỗ hổng.

Cần lưu ý: nhãn `probe-api:dev` chỉ là định danh cục bộ trên máy phát triển. Nó không phải là một định danh bất biến dùng cho môi trường production; Chương 18 sẽ giải quyết bài toán định danh bằng digest.

Desired state và điều tra sự cố trong cluster

Khi chuyển từ container đơn lẻ sang Kubernetes, ta không còn ra lệnh chạy một process mà khai báo desired state thông qua các manifest: `namespace.yaml`, `deployment.yaml` và `service.yaml`.

```
kind create cluster --name go-book
kind load docker-image probe-api:dev --name go-book
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl -n go-book rollout status deployment/probe-api
kubectl -n go-book get deploy,pod,service
kubectl -n go-book port-forward service/probe-api 18080:80
```

Trong `deployment.yaml`, hai probe dẫn tới hai quyết định khác nhau:

Thứ nhất là Readiness Probe (`/readyz`): Trả lời câu hỏi “Pod này có sẵn sàng nhận traffic từ Service ngay lúc này không?”. Sau khi đạt ngưỡng failure, kubelet đặt điều kiện Ready thành false. Với Service thông thường, endpoint không ready không được chọn để nhận traffic; EndpointSlice vẫn có thể giữ địa chỉ với điều kiện ready=false. Service có

`publishNotReadyAddresses` là ngoại lệ cần đọc riêng. Readiness failure không tự restart container.

Thứ hai là Liveness Probe (`/livez`): kiểm tra điều kiện mà policy coi là cần restart container. Kubelet áp dụng threshold và termination policy, không restart chỉ vì một mẫu fail. Tránh dùng lỗi dependency xa làm liveness failure nếu restart không chữa được nó; điều đó có thể tạo cascading restart. Readiness và liveness phục vụ quyết định khác nhau.

Phần tài nguyên cũng phân định rõ ràng giữa `requests` (con số scheduler dùng để tìm node phù hợp cho Pod) và `limits` (giới hạn tài nguyên được thực thi theo cơ chế của môi trường chạy). Với Linux cgroup, CPU bị throttle khi dùng hết ngân sách quota; giới hạn memory được thực thi theo cơ chế phản ứng và có thể dẫn tới OOM kill. Đây không phải cam kết rằng mọi lần vượt ngưỡng đều bị kill ngay lập tức, cũng không phải giới hạn riêng cho heap Go.

Để hiểu cách điều tra sự cố bằng bằng chứng thay vì suy đoán, lab cung cấp fixture `k8s/failure-wrong-image.yaml` với tên image không tồn tại:

```
kubectl apply -f k8s/failure-wrong-image.yaml
kubectl -n go-book rollout status `
  deployment/probe-api --timeout=45s
kubectl -n go-book get pods
kubectl -n go-book get events --sort-by=.lastTimestamp
kubectl -n go-book describe pod `
  -l app.kubernetes.io/name=probe-api
kubectl apply -f k8s/deployment.yaml
kubectl -n go-book rollout status deployment/probe-api
```

Nếu image vẫn không pull được và chưa có ai sửa Deployment, lệnh `rollout status` dự kiến timeout sau 45 giây. Người mới thường vội kết luận "Kubernetes bị treo". Nhưng khi kiểm tra `get events` và `describe pod`, control plane cho ta bằng chứng cụ thể: `ErrImagePull` và `ImagePullBackOff`. Sau khi xác định đúng nguyên nhân, ta khôi phục bằng cách apply lại `deployment.yaml` gốc. Khi hoàn tất, ta dọn dẹp cluster cục bộ bằng lệnh: `kind delete cluster --name go-book`.

Quan sát API bằng client-go

Thay vì phụ thuộc hoàn toàn vào lệnh CLI `kubectl`, các kỹ sư Go thường cần viết các công cụ chẩn đoán hoặc tự động hóa bằng chính ngôn ngữ Go. Thư mục `labs/part17-container-kubernetes/client-observer` minh họa một chương trình chẩn đoán chỉ đọc (read-only) dùng thư viện `k8s.io/client-go`.

Chương trình nạp cấu hình cluster từ kubeconfig cục bộ, gọi API `Get` để lấy snapshot của Deployment `probe-api`, và mở một kênh `Watch` có giới hạn 15 giây để lắng nghe các thay đổi:

```
cd labs/part17-container-kubernetes/client-observer
go test ./...
go vet ./...
go test -race ./...
go run ./cmd/observe-deployment `
  --namespace go-book --name probe-api
```

Kết quả in ra tách biệt rõ giữa desired state và current observation:

```
get: generation=1 observed=1 desired=1 updated=1 available=1
```

Chương trình này cố ý giữ phạm vi hẹp: nó không tạo, sửa hay xóa bất kỳ tài nguyên nào. Nó minh họa sự khác biệt giữa `Generation` (số phiên bản spec mà người dùng muốn) và `ObservedGeneration` (generation mà controller báo đã quan sát). Hai số lệch nhau cho biết status chưa theo kịp generation hiện tại, không chứng minh controller đang tiến triển: nó có thể đang lỗi hoặc chưa chạy. Hai số bằng nhau cũng không chứng minh workload healthy; còn phải đọc conditions và replica status.

Bảng 19 — Các lớp kiểm soát từ binary đến cluster

Tầng kiểm soát	Nhiệm vụ chính	Bằng chứng kiểm chứng	Giới hạn không được suy diễn
Binary	Xử lý request, signal, HTTP ports.	Unit/race test, JSON log.	Chưa biết môi trường filesystem hay cgroup.
Container	Đóng gói filesystem, user, PID 1.	`docker inspect`, exit code.	Chạy được 1 container không đảm bảo tính sẵn sàng.
Workload	Điều phối replica, rolling update, probe.	Rollout status, Pod events.	Pod Available không chứng minh logic app không lỗi.
Client-go	Đọc và theo dõi trạng thái qua API.	Generation, ObservedGeneration.	Snapshot tại một thời điểm không thay thế controller.

Dừng để dự đoán. Nếu một Pod có liveness probe thành công nhưng readiness probe thất bại, Service có chuyển tiếp request vào Pod đó không? Kubelet có restart Pod không? Hãy đối chiếu câu trả lời với bảng trên trước khi xem tiếp.

Khi nào phải dùng Docker, Kubernetes và client-go thật

Khi requirement cần build/push image, chạy integration test trong container, deploy workload, đọc status cluster hoặc viết controller, lúc đó tool thật là bắt buộc. Docker CLI/BuildKit và Kubernetes API đều có version, quyền và cluster policy của chúng; client-go không phải một gói tiện ích để import chỉ vì cần parse YAML. Trước khi chạm API, hãy viết rõ resource nào là source of truth, identity nào được ownership, retry có thể lặp action nào, status nào caller được tin, và credential nào agent được phép dùng.

Điểm dừng của chương là một cách nhìn không còn “deploy = chạy command”. Artefact identity, process lifecycle và desired state là ba contract liên tiếp. Chương 16 cho ta signal để biết instance hiện ra sao; chương này cho ta controller để hiểu vì sao instance có thể bị tạo, thay hoặc rút khỏi traffic.

`NextAction` chỉ là một lần quan sát và một hành động kế tiếp; nó cố ý chưa có hàng đợi, thử lại hay API server. Sau dự án tổng kết ở Chương 20, Chương 21 sẽ xây các cơ chế điều phối đó trên một mô hình tái lập độc lập. Chương

22 mới gắn cùng mental model vào `client-go`, cache và List/Watch của Kubernetes. Trình tự này giữ cho Dockerfile, Kubernetes YAML và framework không che mất câu hỏi nền: tại lần quan sát này, hành động nào là hợp lệ và còn điều gì chưa thể kết luận?

1. Open Container Initiative. Image configuration: platform, `Entrypoint`, `Cmd`, `WorkingDir`, `User` và default execution parameters. github.com/opencontainers/image-spec/blob/main/config.md
2. Docker Authors. Dockerfile reference: `RUN`, `CMD`, `ENTRYPOINT`, exec form và shell form. docs.docker.com/reference/dockerfile/
3. Kubernetes Authors. Controllers: desired state, current state và control loops. kubernetes.io/docs/concepts/architecture/controller/
4. Kubernetes Authors. Pod lifecycle: Pod spec/status, restart policy, readiness và container lifecycle. kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/
5. Kubernetes Authors. Package `client-go`: programmatic interface cho Kubernetes API. pkg.go.dev/k8s.io/client-go
6. GoogleContainerTools. Distroless: language focused docker images, non-root user và minimal runtime footprint. github.com/GoogleContainerTools/distroless

Chương 18 — Đưa thay đổi ra production

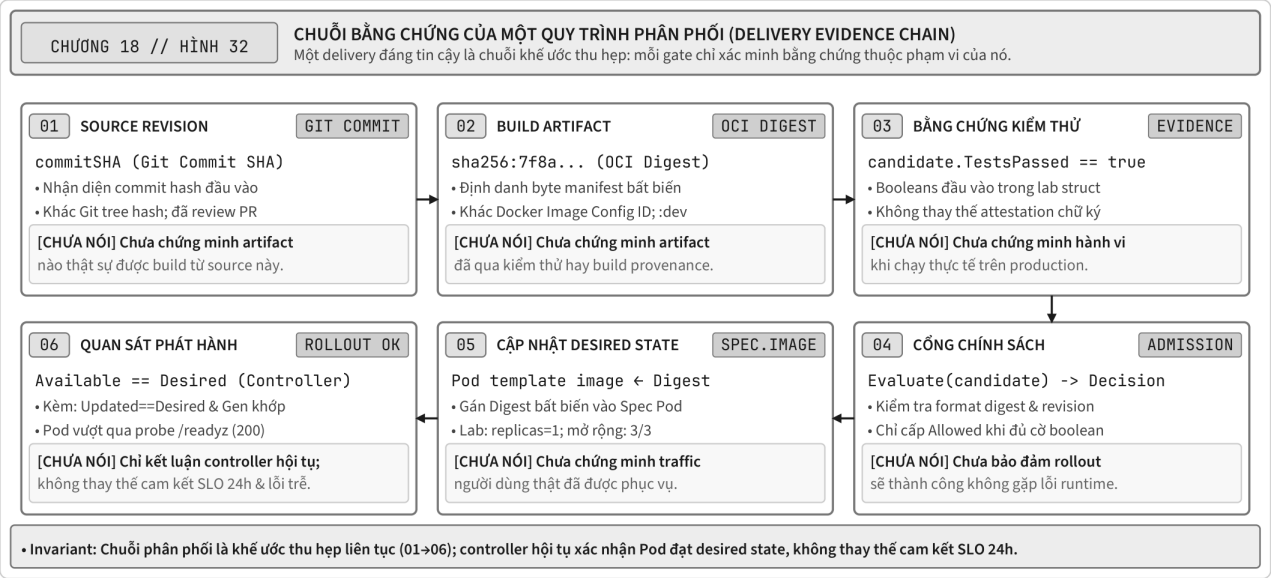
Sau Chương 17, ta có thể đặt một artifact có digest vào desired state. Nhưng một digest không tự trả lời artifact ấy đến từ source nào, đã đi qua test nào, job nào có quyền deploy, hay rollout xong có nghĩa gì với người dùng. Một pipeline xanh chỉ là tín hiệu về những bước nó đã chạy; nếu các bước không nói rõ contract, màu xanh tạo cảm giác an toàn mạnh hơn bằng chứng thực tế.

Mental model của chương là: **delivery là chuỗi bằng chứng gắn một source revision với một artifact bất biến, rồi cho phép promotion theo policy hẹp.** Pipeline không "đưa code lên production". Nó build một artifact, ghi identity của artifact, thu evidence cho các gate đã định nghĩa, và chỉ một job có quyền phù hợp mới cập nhật desired state bằng chính digest ấy. Rollback cũng là một promotion có chủ đích tới artifact đã biết, không phải nút hoàn tác cho mọi side effect của hệ thống.

Một commit xanh vẫn chưa là release

Giả sử một PR pass unit test. Sau đó runner build image, job deploy dùng tag `:main`, cluster pull image, và rollout hiện `3/3 Available`. Chuỗi này còn nhiều câu hỏi chưa được trả lời: image đang chạy có đúng là image vừa build không; source revision nào đã tạo image; credential deploy có quyền rộng đến đâu; `3/3` chỉ ra replica available hay chứng minh request business thành công; và nếu data migration đã chạy, rollback image có đảo dữ liệu lại không?

Đây không phải lý do để bỏ CI/CD vì "quá phức tạp". Nó là lý do để mỗi bước hứa một điều nhỏ, kiểm chứng được. Source revision là identity của đầu vào source. Artifact digest là identity của output build. Test là evidence về behavior mà test bao phủ. Provenance là evidence về quan hệ build mà verifier đã xác nhận. Promotion policy quyết định evidence nào đủ để thay desired state. Rollout status là observation về controller và replica, không phải SLO.



Hình 32 — Một delivery đáng tin là chuỗi contract hẹp. Mũi tên không biến test pass thành lời hứa về mọi failure mode.

Bảng 20 — Mỗi tín hiệu delivery cho phép kết luận đến đâu

Evidence	Điều có thể nói	Điều chưa thể nói
Commit/revision	Source input đã được chọn.	Artifact nào thật sự được build.
Image digest	Một artifact content-addressed cụ thể.	Artifact đó qua test hay provenance nào.
Test pass	Contract của test hiện có đã pass.	Toàn bộ behavior production, data hay dependency.
Rollout complete	Controller đã đạt điều kiện rollout của workload.	SLO, chi phí, hay business transaction của user.

Tag dễ đọc vẫn có ích cho con người, nhưng tag là một tên có thể trỏ sang artifact khác theo thời gian. Promotion production nên mang digest mà gate đã xét, không rebuild “cùng commit” ở job sau rồi hy vọng hai output giống nhau. Nếu cần reproducibility cao hơn, build policy phải pin input phù hợp, ghi revision/toolchain và quản lý base image/dependency; digest chỉ nhận diện output, không tự giải thích output được tạo thế nào.

Gate không phải nghi thức, mà là policy có người chịu trách nhiệm

Một pipeline tốt không bắt mọi thay đổi chạy toàn bộ suite đắt tiền chỉ vì dashboard thích nhiều ô xanh. Nó đặt gate theo risk và feedback loop. Static check hoặc unit test nhanh có thể chạy sớm trên mọi thay đổi. Integration test có dependency thật có thể chạy ở stage khác, với timeout và teardown rõ ràng. Build/push chỉ chạy khi source đã qua gate cần thiết. Promotion production cần identity bất biến, evidence được xác minh, permission hẹp và một đường rollback đã được diễn tập.

Cost guardrail cũng là một contract. Một job integration có thể tạo cluster tạm, registry storage hay egress; phải có owner, timeout, concurrency limit, cleanup và giới hạn phạm vi. “Tốn tiền” không tự làm một job sai, còn “CI xanh” không hợp thức hóa resource không có lifecycle. Hãy đo chi phí của stage nặng trước khi tối ưu, giống cách Chương 10 đặt benchmark trước khi đổi code.

> **Dừng để dự đoán:** Nếu build job pass nhưng không ghi digest vào promotion record, deploy job đang chứng minh điều gì khi nó pull `:main`? Nếu security scan báo sạch nhưng image deploy là output của một build khác, scan đã xét đúng artefact chưa?

Không có gate chung cho mọi sản phẩm. Một payment service có thể cần review/approval và contract test khác một CLI nội bộ. Điều cần giữ chung là policy không fail-open: evidence thiếu hoặc không hợp lệ phải dẫn đến từ chối promotion có giải thích, không âm thầm đổi tag hay dùng credential mạnh hơn để “chạy cho xong”.

Quyền deploy là capability, không phải biến môi trường tiện tay

CI runner có thể chạy source và action mà workflow cho phép, nên credential deploy không nên xuất hiện ở mọi job. Với GitHub Actions, `GITHUB_TOKEN` nên bắt đầu bằng quyền đọc tối thiểu rồi tăng quyền tại job thực sự cần. Cloud credential dài hạn được chép vào secret là một boundary rủi ro khác: secret có thể bị dùng nhầm, log redaction không phải một bằng chứng tuyệt đối, và policy ở cloud mới quyết định token có làm được gì.

OIDC là một cách đưa capability về đúng job cần nó. Workflow xin identity token; cloud provider kiểm tra claim/trust condition rồi cấp access token ngắn hạn cho job. Nó không tự cấp quyền deploy: trust policy vẫn phải giới hạn repository, branch/tag hoặc environment, role và resource. `id-token: write` chỉ cho phép job xin token OIDC, không tự cho phép sửa cloud resource. Đó là lý do "bật OIDC" không thể thay thế review policy IAM.

```
permissions:
  contents: read

jobs:
  deploy-production:
    permissions:
      contents: read
      id-token: write
    environment: production
```

Đoạn YAML chỉ mô tả **ý định phân quyền**, chưa phải workflow deploy chạy được: nó không định nghĩa cloud role, subject claim, approval rule, digest, action version hay command. Đừng copy snippet rồi thêm secret quyền admin để vượt qua bước trust. Thiết kế đúng phải viết từ resource mà job được phép thay đổi, identity của workflow được cloud tin, và điều kiện nào khiến token bị từ chối.

Promotion phải giữ identity của artifact

Lab không ký image và không gọi registry. Nó giữ lại boundary quyết định: candidate có digest hợp lệ, revision để truy vết, test evidence và provenance evidence; thiếu gate trả một `Decision` từ chối, còn digest sai là input lỗi. Phân biệt hai case này giúp caller không biến configuration sai thành "release bị từ chối bình thường".

```
type Candidate struct {
    Digest             string
    Revision           string
    TestsPassed        bool
    ProvenanceVerified bool
}

type Decision struct {
    Allowed bool
    Digest  string
    Reason  string
}

func Evaluate(candidate Candidate) (Decision, error) {
    if !validDigest(candidate.Digest) {
        return Decision{}, ErrInvalidDigest
    }
    if strings.TrimSpace(candidate.Revision) == "" {
        return Decision{}, ErrMissingRevision
    }
    if !candidate.TestsPassed {
        return Decision{Reason: ReasonTestsNotPassed}, nil
    }
    if !candidate.ProvenanceVerified {
        return Decision{Reason: ReasonProvenanceMissing}, nil
    }
    return Decision{
        Allowed: true,
        Digest:  candidate.Digest,
    }, nil
}
```

`ProvenanceVerified` ở đây là input đã được verifier khác kiểm tra; bool này **không** mô hình hóa chữ ký, trust root, identity của builder hay policy

admission. Đây là chủ ý sư phạm: trước khi dùng framework attestation hay API registry, ta cần thấy promotion decision phải giữ đúng digest và fail closed khi evidence vắng mặt. Một struct nhỏ không được phép giả vờ thay supply-chain system thật.

Rollback là một release khác, không phải máy quay thời gian

Kubernetes Deployment có revision cho thay đổi Pod template và có thể rollout quay về revision trước. Nhưng revision không bao phủ mọi state ngoài Pod template: thay đổi `replicas` không tạo revision, history có thể bị dọn theo `revisionHistoryLimit`, external config có thể đã đổi, còn database migration và message đã gửi không tự đảo. Vì thế "rollback được" phải được nói thành câu cụ thể hơn: **rollback artifact/template nào, trong môi trường nào, với data contract và side effect nào?**

Một runbook rollback cần ít nhất: tiêu chí kích hoạt, digest/revision đã biết tốt, owner được phép promote, lệnh hay automation đã thử trong môi trường phù hợp, evidence sau rollout, và quyết định cho data. Nếu release mới chỉ đổi binary stateless, rollback có thể là promotion digest cũ rồi quan sát rollout. Nếu release đổi schema theo hướng không tương thích, rollback có thể cần feature flag, expand/contract migration hoặc dừng phát hành thay vì chạy `undo` ngay.

```
rollback request
  -> chọn artifact/revision đã biết
  -> kiểm tra data và blast radius policy
  -> promote digest cũ
  -> quan sát rollout + signal
  -> ghi lại quyết định và kết quả
```

Đừng tự động rollback chỉ vì một metric chớp đỏ nếu policy chưa định nghĩa window, ownership và false-positive cost. Nhưng cũng đừng để "manual rollback" nghĩa là một người gõ tag tùy ý lúc incident. Cùng identity, gate và permission boundary của forward delivery phải được giữ khi quay lại.

Điển cứu: triển khai báo xanh nhưng người dùng vẫn lỗi

Một release `sha256:abc...` pass unit và integration test, provenance đã verify, rồi được deploy bằng digest. Deployment báo complete: Pod mới available. Vài phút sau, availability SLI tụt vì config ở production trỏ tới endpoint dependency đã retire. Không evidence nào ở đây mâu thuẫn nhau:

Một là, build/promotion proof nói artifact và gate nào đã được dùng.

Hai là, deployment status nói controller đã thay replica theo policy.

Ba là, SLI/log/trace của Chương 16 nói user path đang fail và giúp tìm cohort/config lệch.

Action đúng không phải kết luận "CI vô dụng" hay "Kubernetes đánh lừa mình". Team kiểm tra blast radius, candidate config và data contract; nếu rollback digest cũ khôi phục khả năng phục vụ mà không phá migration, họ promote digest cũ theo runbook. Sau incident, gate cần được sửa ở boundary đã thiếu: configuration validation hoặc integration environment gần production hơn, không nhất thiết là thêm một security scan vô liên quan.

Thực hành: tự viết quyết định tiếp nhận đóng khi lỗi (fail-closed)

Mở `labs/part18-promotion-evidence/exercise/admission_test.go` trước. Test không dùng network, registry, secret hay crypto library. Nó yêu cầu anh tự định nghĩa đúng sự khác nhau giữa candidate malformed và candidate hợp lệ nhưng chưa đủ evidence. Exercise đỏ vì `Evaluate` chưa được viết; fixed giữ implementation nhỏ để test các invariant trước khi bất kỳ CI vendor nào xuất hiện.

```
cd labs/part18-promotion-evidence
go test -tags exercise ./exercise
go test ./fixed
go vet ./fixed
go test -race ./fixed
```

Sau khi làm xong, thử viết một test mới: nếu `TestsPassed` là false nhưng provenance true, `Decision` có được giữ digest hay không? Hãy chọn policy

và diễn đạt lý do. Lab hiện chọn không giữ digest trong decision bị từ chối để caller không vô tình lấy identity đó đi deploy; policy khác chỉ hợp lệ khi được ghi rõ và có test riêng.

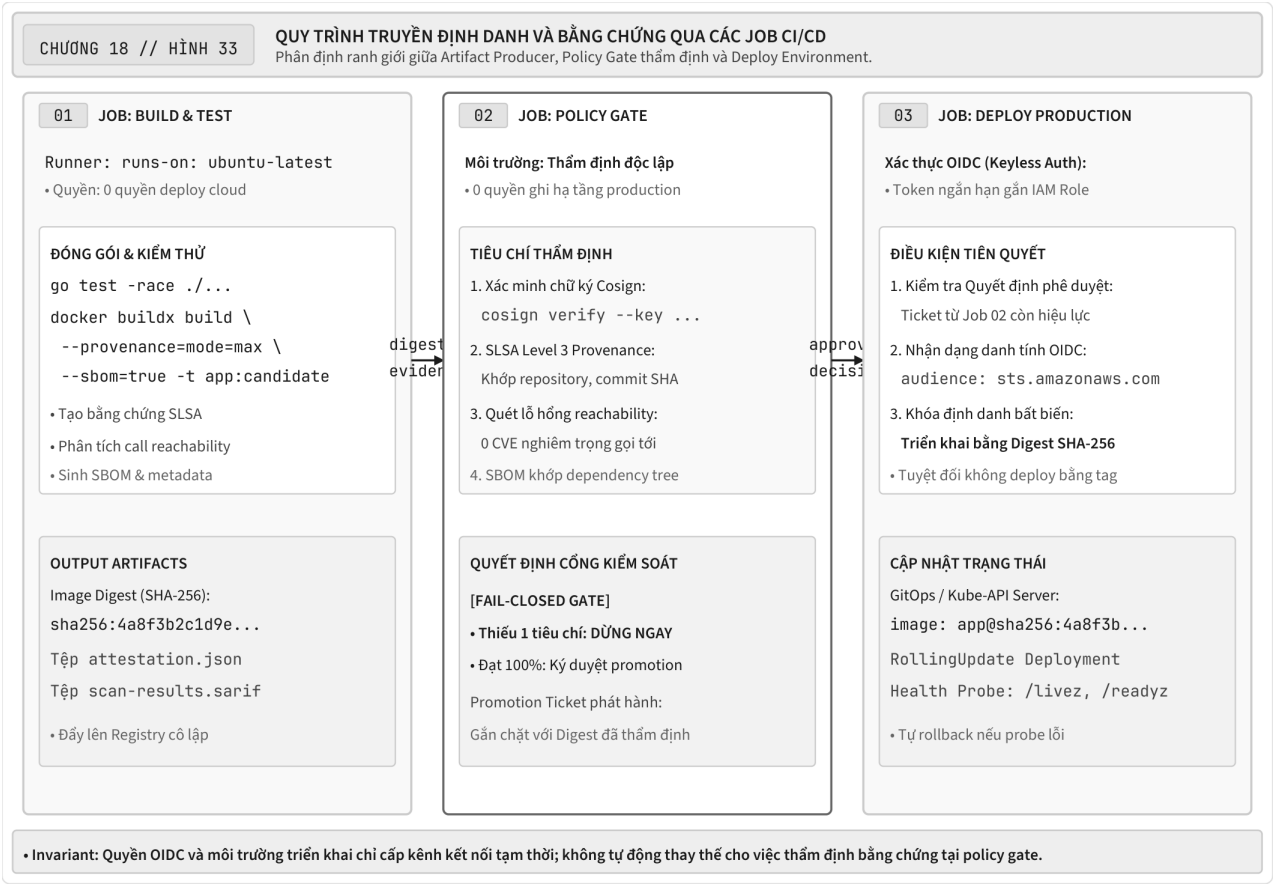
Đáp án — chỉ đọc sau khi đã tự làm. Validate digest rồi revision trước vì chúng là input contract. Sau đó kiểm tra từng gate và return decision từ chối không error; `error` dành cho candidate malformed. Chỉ decision allowed mới mang digest. `validDigest` chấp nhận đúng prefix `sha256:` và 64 chữ số hexadecimal thường; nó là validation định dạng, không phải xác thực content tồn tại hay chữ ký.

Giai đoạn hai: chuỗi chuyển giao từ commit đến cổng phê duyệt

Logic xét duyệt `Evaluate` ở trên cho ta thấy một quyết định promotion cần những bằng chứng gì. Nhưng trong thực tế, các bằng chứng ấy không xuất hiện cùng lúc trong một hàm Go đơn lẻ; chúng được tạo ra qua từng chặng của một pipeline phân tán. `labs/part18-workflow-delivery` kết nối lý thuyết này vào một quy trình hoàn chỉnh gồm ba phần: một workflow GitHub Actions mẫu (reviewable specimen), một công cụ `promote-gate` bằng Go có thể chạy trong CI, và một cấu hình Terraform thiết lập cầu nối OIDC với hạ tầng đám mây.

Để tránh những ngộ nhận tai hại trong vận hành, chuỗi delivery phải phân định rõ bảy lớp thông tin kỹ thuật:

Lớp thông tin	Bản chất kỹ thuật	Vai trò trong Delivery Pipeline
`Source revision`	Mã băm commit Git của mã nguồn đầu vào (ví dụ: `bcb3fe0`).	Xác định chính xác phiên bản mã nguồn cần đóng gói.
`Local image identity`	Tên/tag cục bộ hoặc Image Config ID (`.Id`) trên máy build.	Tạm thời cục bộ, không dùng để ra quyết định production.
`Manifest digest`	Mã băm nội dung OCI Image Manifest (`sha256:...`).	Ghim nội dung manifest thay vì tag có thể đổi. Image nhiều platform còn có index digest; phải biết pipeline đang ghim index hay manifest của một platform.
`Provenance`	Bản chứng thực ghi nhận nguồn gốc commit, builder và công thức.	Cung cấp bằng chứng chuỗi cung ứng cho gate kiểm tra.
`Verified evidence`	Kết quả xác minh chữ ký số của provenance từ công cụ chuyên trách.	Căn cứ kỹ thuật xác thực, loại bỏ cờ boolean giả mạo.
`Promotion decision`	Quyết định phê duyệt hoặc từ chối fail-closed.	Cổng chặn ngăn chặn việc triển khai khi thiếu bằng chứng.
`Actual deployment`	Cập nhật desired state của workload bằng manifest digest đã duyệt.	Thực thi đưa phiên bản đã duyệt vào vận hành.



Hình 33 — Workflow truyền định danh và bằng chứng qua các job; quyền OIDC và environment không tự thay thế việc gate xác minh candidate. Chỉ quyết định được duyệt mới cập nhật desired state bằng digest.

Workflow GitHub Actions mẫu và kiểm tra repo multi-module

Tệp `labs/part18-workflow-delivery/workflows/delivery.yaml` là tài liệu nghiên cứu và kiểm tra mẫu (runnable specimen), không phải workflow đang kích hoạt trong `.github/workflows/`. Nó minh họa các nguyên tắc thiết kế cốt lõi:

Một là, kiểm tra đúng từng Go module: Repository của chúng ta gồm nhiều module độc lập và không có `go.work` ở root. Chạy `go test ./...` tại root sẽ thất bại hoặc vô nghĩa. Job `verify` dùng `working-directory` để chạy test, vet, và race detector riêng cho từng module thuộc delivery path: `labs/part16-real-signals` (service) và `labs/part18-workflow-delivery` (gate).

Hai là, quyền mặc định chỉ đọc: Khai báo `permissions: { contents: read }` ở cấp cao nhất. Quyền `id-token: write` chỉ được mở duy nhất ở job deploy.

Ba là, ghim action bằng commit SHA bất biến: Thay vì tag phiên bản có thể bị trôi, workflow ghim mã băm commit 40 ký tự đầy đủ của từng Action.

Bốn là, không đánh đồng Local Image ID với Manifest Digest: `.Id` mà lệnh inspect của lab lấy là config digest, không phải manifest digest để pull từ registry. OCI manifest và digest của nó cũng có thể được tạo cục bộ trong OCI layout; push không phải điều kiện để digest tồn tại. Pipeline này lấy identity cho promotion từ Buildx metadata/registry và phải phân biệt manifest của một platform với image index nhiều platform.

```
# Trích đoạn từ workflows/delivery.yaml
permissions:
  contents: read

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      # Ghim commit SHA bất biến (v4.2.2 và v5.3.0)
      - uses: actions/checkout@11bd719... # v4.2.2
      - uses: actions/setup-go@3041d5d... # v5.3.0
        with:
          go-version: '1.27.1'
      # Kiểm tra từng module thuộc delivery path
      - name: Verify Service Module
        working-directory: labs/part16-real-signals
        run: |
          go test -v ./...
          go vet ./...
          go test -race ./...
      - name: Verify Gate Module
        working-directory: labs/part18-workflow-delivery
        run: |
          go test -v ./...
          go vet ./...
          go test -race ./...
```

Thực thi cổng phê duyệt bằng Go và nguyên tắc đóng khi lỗi

Để biến policy admission thành một chốt chặn tự động trong pipeline, thư mục `labs/part18-workflow-delivery/cmd/promote-gate` cung cấp một công cụ dòng lệnh nhỏ gọn viết bằng Go. Công cụ này nhận các tham số đầu vào và trả về mã thoát (exit code) chuẩn của hệ điều hành:

```
cd labs/part18-workflow-delivery
go test -v ./...
go vet ./...
go test -race ./...
```

Trong pipeline CI, `tests-passed=true` được suy ra hợp lệ vì job `verify` phía trước đã hoàn thành thành công trong đồ thị phụ thuộc (`needs: verify`). Tuy nhiên, `provenance-verified` chỉ có thể là `true` khi đã có một bước xác thực chữ ký số chuyên trách. Nếu stage này chưa có bước xác thực attestation thật, ta **tuyệt đối không truyền cờ giả mạo** `--provenance-verified=true` (evidence theater). Gate phải từ chối theo nguyên tắc fail-closed:

```
# Định danh digest mẫu 64 ký tự hex
$DIGEST = "sha256:0123456789abcdef0123456789abcdef" + `
          "0123456789abcdef0123456789abcdef"

# 1. Thiếu provenance: gate từ chối fail-closed (Exit 1)
go run ./cmd/promote-gate `
  --digest $DIGEST `
  --revision "bcb3fe0" `
  --tests-passed=true `
  --provenance-verified=false

# 2. Đạt chuẩn khi có đủ cả test và provenance (Exit 0)
go run ./cmd/promote-gate `
  --digest $DIGEST `
  --revision "bcb3fe0" `
  --tests-passed=true `
  --provenance-verified=true

# 3. Trượt test: gate từ chối (Exit 1)
go run ./cmd/promote-gate `
  --digest $DIGEST `
  --revision "bcb3fe0" `
  --tests-passed=false `
  --provenance-verified=true

# 4. Định danh trôi nổi :latest: lỗi đầu vào (Exit 2)
go run ./cmd/promote-gate `
  --digest ":latest" `
  --revision "bcb3fe0"
```

Khi truyền `--digest ":latest"`, chương trình dừng ngay với thông báo: `invalid candidate: digest must be a lowercase sha256 digest` và trả mã lỗi 2.

Đặc biệt, trong pipeline CI, khi `promote-gate` trả về mã lỗi (khác 0) do thiếu provenance hoặc trượt test, job quảng bá phải dừng ngay lập tức. Mọi cơ chế nuốt lỗi (như `|| echo` hay `continue-on-error`) để bước deploy tiếp tục chạy đều là "evidence theater": dựng gate cho có nhưng vẫn âm thầm phát hành code không kiểm chứng. Candidate bị gate từ chối đồng nghĩa với việc không có bất kỳ lệnh deploy nào được phép kích hoạt.

Cầu nối AWS OIDC và Terraform: phân quyền tối thiểu thực chất

Mục `labs/part18-workflow-delivery/terraform/` cung cấp một cấu hình Terraform minh họa mô hình liên kết danh tính OpenID Connect (OIDC) giữa GitHub Actions và AWS IAM trong pipeline này để dùng credential tạm thời thay cho access key dài hạn được lưu trong CI.

Cần hiểu đúng ranh giới của các cơ chế phân quyền trong cấu hình này:

Thứ nhất, khóa chặt danh tính: trust policy dùng `StringEquals` với subject thực tế của job deploy, truyền qua biến Terraform bắt buộc `github_oidc_subject`. Với subject mặc định dạng `repo:OWNER/REPO:environment:production`, điều kiện `sub` kiểm tra repo và environment; nó **không tự khóa branch**. Lab thêm điều kiện AWS riêng trên claim `ref`, mặc định khớp `refs/heads/main`; workflow cũng chặn job deploy ngoài main. Đây là hai gate khác nhau. GitHub Environment vẫn cần deployment branch/tag rule và approval phù hợp. Nếu backend không hỗ trợ claim `ref` riêng, subject tùy biến có ref là một phương án khác nhưng phải xác minh claim thực tế. Subject của repo dùng định danh bất biến có thêm owner/repo ID; repo cũ có thể chọn dùng định dạng ấy, nên lab không đoán subject từ tên repo.

Thứ hai, xét resource theo từng action: trong policy của lab, statement `ecr:GetAuthorizationToken` cần `Resource = ["*"]` vì action này không hỗ trợ resource-level ARN. Đó không phải ngoại lệ duy nhất trên toàn AWS IAM. Các action ECR còn lại của policy hỗ trợ repository ARN; các action App Runner được giới hạn bằng service ARN tương ứng. Tra Service Authorization Reference cho từng action trước khi chọn resource và condition; ARN hẹp của một ví dụ không thành luật chung cho mọi permission.

Thứ ba, tên ECR Repository tuân thủ chuẩn: Biến `ecr_repository_name` được tách riêng và áp dụng validation bắt buộc viết thường (`^[a-z0-9][a-z0-9-_/]*$`), tránh xung đột với quy tắc đặt tên viết hoa của GitHub repository.

```
# Trích đoạn từ terraform/main.tf
data "aws_caller_identity" "current" {}

condition {
  test     = "StringEquals"
  variable = "token.actions.githubusercontent.com:sub"
  values   = [var.github_oidc_subject]
}

condition {
  test     = "StringEquals"
  variable = "token.actions.githubusercontent.com:ref"
  values   = [var.github_deploy_ref]
}
```

Lưu ý: cấu hình Terraform này là tài liệu và mã nguồn kiểm tra cú pháp (reviewable code). Nó không chứng minh rằng role đã assume thành công, không chứng minh tài nguyên ECR/App Runner thực tế tồn tại, và không chứng minh lệnh deploy đã hoàn tất. Máy hiện tại không cài `terraform` hay `aws`; ta giữ blocker này rõ ràng, không tự cài và không tạo tài nguyên cloud thật.

Bảng 21 — Bảy tầng bằng chứng trong quy trình delivery có trách nhiệm

Tầng bằng chứng	Bản chất kỹ thuật	Trách nhiệm kiểm chứng	Giới hạn không được suy diễn
Source Revision	Git commit SHA.	Git commit graph.	Chưa chứng minh code biên dịch được.
Verification	`go test`, `go vet`, `go test -race`.	Runner CI dependency graph.	Chỉ bao phủ các ca kiểm thử hiện có.
Local Image ID	Config JSON hash (`.Id`).	Docker daemon cục bộ.	Không dùng làm định danh kéo ảnh từ xa.
Manifest Digest	OCI Manifest Hash (`sha256:`).	Buildx metadata / Registry.	Chưa chứng minh container chạy đúng logic.
Provenance	Build attestation metadata.	Verifier chuyên trách (Cosign/GH).	Thiếu verifier thì không được coi là verified.
Cổng phê duyệt	CLI Go kiểm tra fail-closed.	Admission policy.	Không thay thế được môi trường production thật.
Deployment	Cập nhật desired state cụm.	Orchestrator controller.	Replica Available không bảo đảm business SLO.

Dừng để dự đoán. Nếu một pipeline CI tự động gán cờ `--provenance-verified=true` mà không chạy bất kỳ lệnh xác thực chữ ký nào, điều gì sẽ xảy ra nếu một kẻ tấn công thay thế image trong registry bằng một image độc hại có cùng tag? Tại sao đây lại được gọi là "evidence theater"?

Điểm dừng: evidence không thay thế trách nhiệm

Pipeline, OIDC, attestation, Deployment và rollback không làm software tự an toàn. Chúng làm đường đi của một thay đổi có thể kiểm tra: source nào, artifact nào, gate nào, quyền nào, rollout nào và hành động nào khi xấu. Đó là nền để một Go service có thể được phát hành nhiều lần mà không biến mỗi lần phát hành thành một niềm tin mơ hồ.

Từ đây, sách có thể quay vào case study lớn hơn: một service nhận input, chạy work có deadline, phát signal, được đóng gói, rồi được đưa qua delivery

policy và phục hồi có evidence. Mỗi công cụ chỉ được thêm khi câu hỏi vận hành đã tồn tại.

1. GitHub Docs. Secure use reference: least privilege, `GITHUB_TOKEN`, secret risks và OIDC. docs.github.com/en/actions/reference/security/secure-use
2. GitHub Docs. OpenID Connect: token ngắn hạn, trust relationship và cloud authorization. docs.github.com/en/actions/concepts/security/openid-connect
3. GitHub Docs. Security for GitHub Actions: artifact attestations và deployment hardening. docs.github.com/en/actions/how-tos/secure-your-work
4. Kubernetes Authors. Deployments: rollout status, revision, rollback và giới hạn của revision history. kubernetes.io/docs/concepts/workloads/controllers/deployment/
5. AWS Documentation. Creating OpenID Connect (OIDC) identity providers và IAM role trust policies. docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers_create_oidc.html
6. HashiCorp Terraform. AWS Provider: `aws_iam_openid_connect_provider` và `aws_iam_role`. registry.terraform.io/providers/hashicorp/aws/latest/docs
7. AWS Documentation. Actions, resources, and condition keys for Amazon Elastic Container Registry. docs.aws.amazon.com/service-authorization/latest/reference/list_ecr.html
8. OCI. Image Format Specification: image manifest, image index và OCI image layout. github.com/opencontainers/image-spec

Chương 19 — Giữ type information khi abstraction lớn lên

Một service có hai danh sách target: tên miền và port. Cả hai đều cần cùng một thao tác: giữ lại phần tử đầu tiên của mỗi giá trị, theo đúng thứ tự ban đầu. Nếu viết thẳng, ta có hai function gần như giống nhau. Sự lặp này chưa nguy hiểm; nó làm câu hỏi thiết kế hiện ra rõ: phần nào trong operation thực sự khác theo type, và phần nào là quy luật chung?

```
func uniqueNames(values []string) []string {
    seen := make(map[string]struct{}, len(values))
    result := make([]string, 0, len(values))
    for _, value := range values {
        if _, exists := seen[value]; !exists {
            seen[value] = struct{}{}
            result = append(result, value)
        }
    }
    return result
}

func uniquePorts(values []int) []int {
    // cùng vòng lặp, chỉ khác type của value và map
}
```

Một phản ứng thường thấy là đổi mọi thứ thành `any`. Function sẽ bớt lặp trên giấy, nhưng caller mất thông tin mà compiler có thể dùng để bắt lỗi. Phản ứng ngược lại là tạo một abstraction generic cho mọi mẩu code trùng nhau. Nó cũng tệ: một type parameter chỉ có ý nghĩa khi operation chung thật sự rõ hơn sau khi tách nó ra.

Mental model của chương là: **generic giữ type information cho compiler khi cùng một operation áp dụng lên một tập type; interface giữ contract hành vi khi concrete type có thể thay đổi; reflection chỉ phù hợp khi shape hay metadata thực sự là input lúc runtime.** Đây là ba boundary khác nhau, không phải ba mức "linh hoạt" để tăng dần.



Hình 34 — Chọn abstraction theo câu hỏi cần trả lời. Sơ đồ là mental model, không mô tả representation runtime của interface hay cách compiler sinh code.

Bắt đầu bằng duplication, không bằng dấu ngoặc vuông

`uniqueNames` và `uniquePorts` đều chỉ cần một phép: so sánh hai value bằng `==`. Điều đó loại slice, map và function, nhưng đủ cho `string`, `int`, pointer, struct có field comparable, và named type có underlying type tương ứng. `comparable` ghi đúng boundary ấy.

```
func Unique[T comparable](values []T) []T {
    seen := make(map[T]struct{}, len(values))
    result := make([]T, 0, len(values))
    for _, value := range values {
        if _, exists := seen[value]; !exists {
            seen[value] = struct{}{}
            result = append(result, value)
        }
    }
    return result
}

names := Unique([]string{"api", "db", "api"})
ports := Unique([]int{443, 8080, 443})
```

`T` là type parameter, còn `comparable` là constraint: tập type argument mà `T` được phép nhận, đồng thời là giới hạn operation trong body. Vì caller đưa `[]string` hay `[]int`, compiler infer type argument; không cần viết `Unique[string](...)`. Viết explicit vẫn hợp lệ khi ngữ cảnh chưa đủ để infer, nhưng nó không phải nghi thức phải thêm vào mọi lời gọi.

Nếu thử `Unique([][]byte{{1}, {1}})`, code không compile. Đó là kết quả có ích: slice không comparable, nên map key và `==` không có semantics mà function này đã hứa. Chuyển input thành `[]any` không sửa được hợp đồng; hai interface value có dynamic type là slice còn có thể panic khi bị so sánh. Khi equality cho byte slice cần nghĩa khác, chẳng hạn `bytes.Equal`, API mới phải nói rõ nghĩa đó thay vì giả vờ `Unique` phổ quát.

`any` là alias của `interface{}`. Nó có chỗ ở boundary thực sự nhận nhiều shape, như Chương 14 nhận destination để inspect bằng reflection. Nhưng `func UniqueAny(values []any) []any` buộc implementation phải type switch, type assertion hoặc chịu lỗi runtime để biết so sánh thế nào. Generic không làm type biến mất; nó yêu cầu compiler kiểm tra type argument thỏa constraint trước khi code chạy.

Constraint là danh sách phép được phép, không phải giấy phép bao hết type

Có operation cần thứ tự thay vì equality. Ta có thể tự định nghĩa constraint nhỏ theo đúng domain của ví dụ, thay vì copy một constraint rộng chỉ để function trông "tái sử dụng được".

```
type Measurable interface {
    ~int | ~int64 | ~float64
}

func Max[T Measurable](left, right T) T {
    if left > right {
        return left
    }
    return right
}

type Milliseconds int64

slowest := Max(Milliseconds(120), Milliseconds(80))
```

Union `|` nói rằng một type argument thuộc một trong các term. Dấu `~` mở term ra cho mọi defined type có **underlying type** tương ứng. Không có `~int64`, `Milliseconds` không thỏa constraint dù `left > right` có nghĩa hoàn toàn rõ. Ngược lại, `~` không "chuyển đổi ngầm" giữa hai value đã có type: `int` và `int64` vẫn khác nhau, nên `Max(int(1), int64(2))` không có một `T` duy nhất để infer. Riêng hằng số chưa định kiểu như `1` có thể nhận type từ đối số còn lại.

Constraint có type term như `~int | ~int64` là general interface: nó dùng để giới hạn type parameter, không phải một interface value để truyền quanh chương trình. Basic interface có thể dùng làm kiểu của value: nó chỉ khai báo hoặc nhúng các method, cũng có thể rỗng như `any`. Constraint còn có thể dùng type term để giới hạn tập type và các operation compiler cho phép. Đặt cả hai dưới cùng từ "interface" dễ làm mờ hai nhiệm vụ này.

> **Dừng để dự đoán.** Xét đoạn mã sau:

> ~~~go

> slowest := Max(Milliseconds(120), Milliseconds(80))

> var raw int64 = slowest

> ~~~

> 1. Trình biên dịch thực hiện suy luận kiểu (type inference) cho `T` là `int64` hay `Milliseconds`?

> 2. Biến `slowest` có kiểu dữ liệu tĩnh là gì?

> 3. Lệnh gán `var raw int64 = slowest` có biên dịch thành công không, hay bị compiler báo lỗi?

Đáp án — chỉ đọc sau khi đã tự làm

Quá trình suy luận kiểu (type unification) chọn `T` chính xác là `Milliseconds` chứ không hề thu gọn (decay) về underlying type `int64`. Do cả hai đối số truyền vào đều mang kiểu `Milliseconds`, hàm generic bảo toàn thông tin kiểu của đối số và trả về kết quả mang đúng kiểu tĩnh `Milliseconds` thay vì ép về kiểu nền hay bọc trong interface rỗng.

Vì lý do đó, lệnh gán `var raw int64 = slowest` bị trình biên dịch từ chối ngay lập tức với lỗi: `cannot use slowest (variable of type Milliseconds) as int64 value in variable declaration`. Dù `Milliseconds` có underlying type là `int64`, hệ thống kiểu của Go không bao giờ cho phép ép kiểu ngầm định giữa hai defined type khác nhau. Muốn gán giá trị này sang một biến `int64`, lập trình viên bắt buộc phải chuyển đổi

tường minh bằng cú pháp `int64(slowest)`. Đây chính là giá trị bảo toàn thông tin kiểu (type preservation) ở thời điểm biên dịch của Generic: nó loại bỏ nguy cơ gán nhầm đơn vị đo lường mà không đòi hỏi chi phí đánh đổi qua `any` và type assertion ở runtime.

Generic named type giữ invariant cùng dữ liệu

Khi dữ liệu và operation đi cùng nhau, generic named type có thể làm API gọn hơn. Set dưới đây chỉ nhận `comparable` vì Go map key cần constraint đó. Constraint nằm ở declaration của type, nên mỗi method đều kế thừa invariant thay vì lặp lại nó.

```
type Set[T comparable] map[T]struct{}

func (set Set[T]) Has(value T) bool {
    _, found := set[value]
    return found
}

ports := Set[int]{443: {}, 8080: {}}
```

`Set[string]` và `Set[int]` là instantiated types khác nhau. Caller không thể vô tình gọi `Set[string].Has(443)`. Đây không phải lời hứa rằng generic code luôn nhanh hơn code interface; performance phụ thuộc workload, compiler và version, nên phải đo nếu nó trở thành lý do chọn thiết kế. Ở đây lợi ích đã rõ trước benchmark: API nói chính xác value nào có thể vào set.

Go 1.27 còn cho method tự khai báo type parameter của nó. Điều này hữu ích khi operation thuộc namespace của data type nhưng result đổi element type.

```
type Batch[E any] []E

func (batch Batch[E]) Map[F any](fn func(E) F) Batch[F] {
    result := make(Batch[F], len(batch))
    for i, value := range batch {
        result[i] = fn(value)
    }
    return result
}

labels := Batch[int]{2, 4}.Map(func(port int) string {
    return fmt.Sprintf("port=%d", port)
})
```

Đây là generic method của Go 1.27, khác với method thông thường trên generic receiver như `Set[T].Has`. Generic method được instantiate khi dùng. Nó không biến interface method thành generic: interface method không được tự khai báo type parameter, và một generic method không phải cách để implement một interface method generic. Nếu consumer chỉ cần gọi `Map` trên một concrete `Batch`, method giúp tổ chức code; nếu consumer cần một behavior thay thế được, một interface nhỏ với signature cụ thể thường rõ hơn.

Compiler có thể chia sẻ hoặc chuyên biệt hóa mã — đừng biến nó thành lý do chọn API

Tại ngữ nghĩa ngôn ngữ, một generic function được instantiate sau khi type argument thỏa constraint. Đặc tả không hứa cách compiler sinh mã, kích thước binary hay số allocation. Bản Go toolchain hiện tại có những tên nội bộ như `go.shape` và dictionary; chúng giúp kỹ sư đọc assembly hoặc source compiler, nhưng không phải contract cho application và có thể đổi theo version, kiến trúc, PGO và thân hàm.

Vì thế, đừng chọn generic vì tin nó luôn nhanh hơn interface, cũng đừng loại nó chỉ vì nghe rằng compiler "có dictionary". Hãy chọn generic khi operation và constraint làm API đúng hơn. Khi chi phí là lý do quyết định, hãy đo hai implementation dưới cùng Go version, cùng máy/môi trường,

workload và benchmark contract; sau đó kiểm tra allocation và profile thay vì suy ra từ representation nội bộ.

Interface thay vì generics khi câu hỏi là “ai làm được việc này?”

Một probe HTTP, một probe TCP và fake dùng trong test có thể trả outcome bằng các implementation khác nhau. Caller không cần giữ concrete type của chúng; caller cần gọi một behavior. Interface ghi contract đó ở compile time.

```
type Checker interface {
    Check(context.Context) error
}

type httpChecker struct{}

func (httpChecker) Check(context.Context) error { return nil }

func RunOne(ctx context.Context, checker Checker) error {
    return checker.Check(ctx)
}

var _ Checker = httpChecker{}
```

Dòng cuối là compile-time assertion: nếu `httpChecker` thiếu `Check`, package không compile. Khi program chạy, một value của interface có static type `Checker`, còn concrete value bên trong có thể có dynamic type khác ở từng lần gọi. Đó là lý do interface hữu ích cho polymorphism theo behavior, không phải vì nó “nhận được mọi type”.

Khi code cần hiểu concrete case nào đang ở trong interface, type assertion và type switch là công cụ trực tiếp. Chúng không cần reflection.

```
func describe(value any) string {
    switch typed := value.(type) {
    case string:
        return "name=" + typed
    case int:
        return "port"
    }
    return "unsupported"
}
```

`value.(int)` không có comma-ok sẽ panic khi assertion sai. Dạng `port, ok := value.(int)` trả zero value của `int` và `false`, nên caller có thể chọn policy error của mình. Type switch chỉ so dynamic type với các case đã biết; `reflect.Type` và struct tag của Chương 14 chỉ nên xuất hiện khi metadata runtime là dữ liệu của bài toán, chẳng hạn decoder/schema tool. Đừng dùng reflection để thay một type switch có vài case rõ ràng.

Typed nil: type động vẫn còn, dù pointer bên trong nil

Đoán output trước khi chạy đoạn này:

```
type ProbeError struct {
    Target string
}

func (err *ProbeError) Error() string {
    return "probe failed"
}

var problem *ProbeError = nil
var result error = problem

fmt.Println(problem == nil)
fmt.Println(result == nil)
```

Kết quả là `true`, rồi `false`. `problem` là nil pointer. Khi gán nó vào `error`, interface value vẫn mang dynamic type `*ProbeError`; dynamic value của type đó là nil pointer.

Không cần suy ra layout runtime để giải thích kết quả. Theo ngữ nghĩa interface, `result` giữ dynamic type `*ProbeError` và dynamic value nil của type đó, nên interface value ấy không phải nil interface. Representation hai word mà một Go runtime trên một kiến trúc có thể dùng chỉ là chi tiết triển khai; đừng dùng nó làm contract, cũng đừng dựa vào kích thước hay field nội bộ của interface trong production code.

Bug thường gặp nằm ở đường return:

```
func checkTarget(ok bool) error {
    var problem *ProbeError
    if !ok {
        problem = &ProbeError{Target: "api"}
    }
    return problem // ok == true vẫn trả error khác nil
}
```

Hãy return `nil` ở nhánh thành công, hoặc chỉ tạo concrete error khi có failure. Không cần dùng reflection để đoán nil của mọi error: caller cần contract `err == nil`, còn method của concrete error phải xác định riêng receiver nil có hợp lệ không. Lab `labs/part19-type-information/typednil` giữ bug này để người học sửa và kiểm tra bằng test.

Thực hành: ba ranh giới, ba cách tự kiểm tra

Lab `labs/part19-type-information` bắt đầu ở test/tag exercise và contract, không ở implementation có sẵn.

```
cd labs/part19-type-information
go test -tags exercise ./exercise
go test ./fixed
go test -race ./fixed
go test -tags typednilexercise ./typednil
go test ./typednil/fixed
go vet ./...
```

Ba task lần lượt yêu cầu anh viết `Unique[T comparable]` từ test, review `Load[T any]` chưa nói format/ownership/decode, và sửa typed-nil fixture để nhánh success trả `nil error` thật. Chỉ sau đó mới mở `review/answer.md` và `fixed/`; chúng là reference để so reasoning, không phải chuẩn duy nhất cho naming.

Điểm dừng của chương không phải "thấy `[]` là dùng generic". Hãy hỏi operation có cần giữ type nào, constraint nào thực sự cho phép operation ấy, behavior nào nên là interface, và metadata nào chỉ runtime mới biết. Trả lời được bốn câu đó, abstraction bớt giống một màn ảo thuật và bắt đầu trở thành một contract có thể đọc.

1. Go Team. The Go Programming Language Specification: interface values, type assertions, type switches, type constraints, instantiation và method declarations. go.dev/ref/spec
2. Go Team. Go 1.27 Release Notes: generic methods và các thay đổi về type inference. go.dev/doc/go1.27
3. Go Team. FAQ: nil interface value và nil pointer được lưu trong interface. go.dev/doc/faq
4. Go Team. Generic Methods. go.dev/blog/generic-methods

Chương 20 — Dự án tổng kết: opsprobe từ mã nguồn đến vận hành

Ở Chương 1, ta dùng một kết quả HTTP giả định để học cách đọc chương trình. Các chương sau lần lượt thêm request thật, deadline, đồng thời, lưu trữ và tín hiệu vận hành. Chương này ghép các phần ấy vào một công cụ thăm dò endpoint: khi một lượt probe thất bại, caller cần biết kết quả thuộc boundary nào và dữ liệu nào đã được lưu.

Dự án `projects/opsprobe/` là teaching vehicle có domain logic, worker pool giới hạn, SQLite, metrics, trace và HTTP API. Test tái lập những contract cụ thể, trong đó có tình huống quên đóng response body. Có implementation chạy được không đồng nghĩa công cụ đã sẵn sàng cho mọi môi trường production; phần bảo mật và triển khai dưới đây chỉ ra những giả định còn phải giữ.

Mental model của chương là: **một hệ thống Go trưởng thành không phải là một tập hợp các framework cồng kềnh, mà là sự khớp nối chính xác giữa các boundary kỹ thuật nhỏ: value semantics, error contract, áp suất đồng thời, tính nguyên tử của state, tín hiệu quan sát trung thực, và kỷ luật phát hành có trách nhiệm.**

Bản đồ kiến trúc và hướng phụ thuộc

Mã nguồn của dự án nằm tại thư mục `projects/opsprobe/`. Các package tách những trách nhiệm sau:

```
projects/opsprobe/
├── cmd/opsprobe/          # CLI oneshot và HTTP daemon
├── internal/
│   ├── probe/             # Bounded concurrency pool, Outcome
│   ├── store/             # SQLite, transaction nguyên tử
│   ├── telemetry/         # slog JSON, Prometheus, OTel
│   └── httpapi/           # REST API, backpressure, health
├── incident/              # Kịch bản sự cố rò rỉ socket TCP
├── deploy/                # Dockerfile, Kubernetes, Terraform
└── scenarios_test.go      # Kiểm thử 6 kịch bản sự cố
```

Bảng 22 — Phân tách trách nhiệm và ranh giới contract trong opsprobe

Package	Trách nhiệm vận hành	Contract bảo vệ
`probe`	Kiểm tra target, phân loại kết quả.	Giới hạn worker, outcome rõ, đóng stream.
`store`	Lưu trữ metadata và kết quả probe.	Giao dịch nguyên tử qua `database/sql`.
`telemetry`	Ghi nhận log, metrics và trace.	Không đoán mò, đo lường đúng nhãn outcome.
`httpapi`	Nhận request, điều phối, trả JSON.	Backpressure 429, readiness, graceful drain.
`cmd`	Điểm khởi đầu (Composition Root).	Parse cờ, bắt tín hiệu OS, graceful shutdown.

`probe` và `store` không import `httpapi` hay `cmd`, nên có thể kiểm thử mà không khởi động toàn bộ ứng dụng. Điều đó không biến mọi test thành phép tính thuần: test I/O của `probe` dùng HTTP server cục bộ, còn test `store` cần database. Hướng import giúp tách boundary; fixture vẫn phải tái lập đúng hành vi cần kiểm tra.

Năm trụ cột kỹ thuật của một hệ thống đáng tin

1. Phân loại kết quả và thời hạn ngữ cảnh

Trong vận hành, biết một thao tác thất bại là chưa đủ; ta cần biết nó thất bại vì lý do gì. Trả về một giá trị boolean `false` hay một chuỗi lỗi mơ hồ sẽ xóa sạch thông tin quý giá. Trong `internal/probe/probe.go`, mỗi lần probe được phân loại rành mạch thành một trong bốn `Outcome`:

```
type Outcome string

const (
    OutcomeSuccess Outcome = "success"
    OutcomeFailure Outcome = "failure"
    OutcomeTimeout Outcome = "timeout"
    OutcomeCancel  Outcome = "cancel"
)
```

Khi gửi request, ta gắn deadline chặt chẽ cho từng target:

```
targetTimeout := p.defaultTimeout
if target.Timeout > 0 {
    targetTimeout = target.Timeout
}
probeCtx, cancel := context.WithTimeout(ctx, targetTimeout)
defer cancel()

req, err := http.NewRequestWithContext(
    probeCtx, method, target.URL, nil)
```

Phân biệt `OutcomeTimeout` với `OutcomeCancel` giúp caller giữ nguyên nhân kết thúc theo contract của lab. Nó không tự phân loại được root cause mạng hay chứng minh mọi cancellation đều là shutdown; cần correlation với lifecycle, deadline và observation khác.

Mỗi kết quả probe ghi nhận thời lượng bằng mili-giây (`DurationMs`) phục vụ dashboard và nano-giây (`DurationNs`) để tránh mất thông tin do làm tròn; đơn vị nano-giây không chứng minh đồng hồ đo chính xác đến nano-giây. Thời điểm ghi nhận run trong database phân biệt `StartedAt` (bắt đầu thực thi) và `CompletedAt` (kết thúc toàn bộ worker), tránh nhầm độ trễ một request với thời gian hoàn tất cả lô công việc.

2. Giới hạn đồng thời và áp suất ngược

Khi danh sách target tăng từ 10 lên 10.000, chạy `go p.ProbeSingle(...)` cho từng target có thể tạo áp lực lên bộ nhớ, socket và dịch vụ đích. Mức ảnh hưởng phụ thuộc workload và giới hạn môi trường; không suy ra hệ điều hành chắc chắn sẽ sập từ riêng số goroutine.

`opsprobe` sử dụng mô hình worker pool với số lượng goroutine cố định:

```
numWorkers := p.concurrency
if numWorkers > n {
    numWorkers = n
}

var wg sync.WaitGroup
for w := 0; w < numWorkers; w++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for j := range jobs {
            if ctx.Err() != nil {
                results[j.index] = Result{
                    TargetID: j.target.ID,
                    Outcome:  OutcomeCancel,
                }
                continue
            }
            results[j.index] = p.ProbeSingle(ctx, j.target)
        }
    }()
}
wg.Wait()
```

Gauge `opsprobe_active_workers` đếm worker goroutine đã khởi động và chưa thoát, qua `WorkerObserver`. Nó không đếm riêng worker đang xử lý target: một worker chờ việc vẫn nằm trong gauge. Muốn đo số thao tác đang diễn ra, cần metric khác với điểm tăng/giảm quanh từng thao tác.

Ở HTTP API, buffered channel semaphore giới hạn số đợt chạy đồng thời theo cấu hình. Khi hết slot, policy của lab từ chối thêm việc bằng `429 Too`

`Many Requests`; giới hạn này là budget của ví dụ, chưa phải capacity đã được đo là an toàn cho production:

```
select {
case a.runSem <- struct{}{}:
    defer func() { <-a.runSem }()
default:
    a.tel.RunsTotal.WithLabelValues(
        "rejected_backpressure").Inc()
    a.writeJSON(
        w, http.StatusTooManyRequests,
        map[string]string{
            "error": "too many runs, backpressure applied",
        },
    )
    return
}
```

3. Giao dịch nguyên tử trên SQLite: Hoặc toàn bộ, hoặc không có gì

Dữ liệu lưu trữ chỉ có giá trị khi nó phản ánh đúng trạng thái nhất quán của thế giới thực. Nếu một đợt chạy gồm 10 target, hệ thống không được phép rơi vào trạng thái: lưu thành công 5 target đầu rồi crash, bỏ lại 5 target sau và làm lệch báo cáo tổng hợp.

Trong `internal/store/store.go`, toàn bộ bản ghi đợt chạy (`runs`) và các kết quả probe (`probe_results`) được thực thi bên trong một transaction duy nhất qua `database/sql`:

```
tx, err := s.db.BeginTx(ctx, nil)
if err != nil {
    return fmt.Errorf("begin transaction: %w", err)
}
defer tx.Rollback() // An toàn: no-op nếu Commit đã thành công

// 1. Chèn bản ghi đợt chạy (runs)
_, err = tx.ExecContext(ctx, insertRunQuery, run.ID, ...)
if err != nil {
    return fmt.Errorf("insert run: %w", err)
}

// 2. Chèn từng kết quả probe bằng prepared statement
stmt, err := tx.PrepareContext(ctx, insertResultQuery)
if err != nil {
    return err
}
defer stmt.Close()

for _, r := range results {
    _, err = stmt.ExecContext(ctx, run.ID, r.TargetID, ...)
    if err != nil {
        return err // defer tx.Rollback() sẽ hủy toàn bộ
    }
}

return tx.Commit()
```

Để kiểm chứng tính toàn vẹn của transaction một cách tất định (deterministic) mà không dựa vào thời điểm ngẫu nhiên, schema cơ sở dữ liệu được trang bị ràng buộc `CHECK (status_code >= 0)`. Trong kiểm thử tự động, ta cố tình truyền một kết quả có `status_code = -1` ở bản ghi thứ hai sau khi bản ghi cha đã được chèn. Database lập tức từ chối, transaction rollback toàn bộ, và lệnh đếm số dòng xác nhận cả hai bảng đều không lưu lại bất kỳ dữ liệu rác nào.

4. Vòng đời tài nguyên mạng: Thu hồi có giới hạn và điều kiện tái sử dụng

Caller phải luôn đóng `resp.Body`. Đọc đến EOF có thể giúp một số đường tái sử dụng kết nối, nhưng `Close` hay drain không bảo đảm reuse: Transport, protocol, server và trạng thái kết nối đều tham gia quyết định. Ngược lại, drain không giới hạn cần được cân nhắc khi target có thể trả body rất lớn hoặc không kết thúc.

`opsprobe` thiết lập chính sách bounded drain có kiểm soát:

```
const MaxDrainBytes = 16384 // 16 KiB

func drainResponseBody(body io.ReadCloser) bool {
    if body == nil {
        return false
    }
    defer body.Close()

    lr := &io.LimitedReader{R: body, N: MaxDrainBytes + 1}
    _, err := io.Copy(io.Discard, lr)
    // Body nhỏ đã chạm EOF; không phải guarantee reuse.
    return err == nil && lr.N > 0
}
```

Quy tắc kỹ thuật ở đây rất rõ ràng:

Một là, đóng body trên đường kết thúc: hoàn tất trách nhiệm với response body theo contract. `Close` không đồng nghĩa mỗi lần phải đóng socket TCP; Transport có thể giữ connection để tái sử dụng.

Hai là, chọn budget 16 KiB: Đây là chính sách của lab, không phải thống kê kích thước body của các framework. Helper đọc thêm tối đa một byte để nhận biết body vượt budget, dùng `io.Discard` thay vì giữ toàn bộ body trong RAM.

Ba là, phát hiện cạn dòng (EOF): đọc tối đa `MaxDrainBytes + 1` phân biệt body nhỏ đã đọc hết với body vượt budget. Chỉ coi drain thành công khi phép đọc không có lỗi và `lr.N > 0`; lỗi đọc trước EOF không được diễn giải là thành công. Đây là quyết định budget của helper, không phải cờ đủ điều

kiện reuse bên trong Transport. Đường tái sử dụng HTTP/1.x còn phụ thuộc body framing, server, lỗi kết nối và trạng thái Transport.

Bốn là, đánh đổi có chủ đích: Nếu body vượt quá 16 KiB, `lr.N` sẽ bằng 0. Lab ghi nhận body chưa được đọc hết và đóng body để giải phóng tài nguyên; không suy ra chắc chắn Transport sẽ hay sẽ không tái sử dụng một kết nối cụ thể.

Năm là, thời lượng probe phản ánh toàn bộ lifecycle: Đồng hồ đo thời lượng probe bắt đầu từ trước khi phát request tới sau khi quy trình cleanup/drain kết thúc. Nếu target trả về header 200 nhưng luồng body bị nghẽn (stall) vượt quá deadline, probe sẽ kết luận đúng là `OutcomeTimeout` thay vì báo nhầm `OutcomeSuccess`.

5. Ranh giới bảo mật, hợp đồng JSON và truy vết phân tán

Một công cụ vận hành chỉ an toàn khi các ranh giới ngoại vi được xác định rành mạch:

`opsprobe` giả định người vận hành và target registry được tin. Validation URL cú pháp không chặn đầy đủ SSRF. Nếu nhận input không tin cậy, cần policy target/egress, xử lý DNS và redirect theo threat model; không mặc định chặn mọi private IP vì công cụ nội bộ có thể cần truy cập đúng các endpoint ấy. Allowlist hoặc proxy cũng cần owner và cấu hình được kiểm chứng.

Về hợp đồng JSON di động: Trường `timeout_ms` trong request payload sử dụng kiểu số nguyên mili-giây (`0 <= timeout_ms <= 60000`), bảo đảm tương thích đa nền tảng thay vì parse cú pháp chuỗi duration của riêng Go.

Về phân tán ngữ cảnh (W3C TraceContext): Outbound probe request chèn header `traceparent` theo child span hiện tại. Để nối trace, dịch vụ nhận còn phải đọc context và cấu hình lấy mẫu/xuất span phù hợp; gửi header không tự chứng minh chuỗi quan sát xuyên dịch vụ đã đầy đủ.

Bài tập chẩn đoán sự cố: Bão cạn kiệt Socket

Thư mục `projects/opsprobe/incident/` mô phỏng một sự cố kinh điển trong môi trường microservices.

Kịch bản và Triệu chứng lâm sàng

> **Ghi chú phương pháp luận:** Đây là kịch bản giả định mô phỏng tình huống sự cố thực tế để đặt ra bài toán chẩn đoán cho kỹ sư.

Hệ thống probe giả định được triển khai để kiểm tra sức khỏe các microservice nội bộ. Khi số lượng request tăng lên, hệ thống giám sát ghi nhận tỷ lệ `OutcomeTimeout` tăng vọt và độ trễ probe tăng cao. Tuy nhiên, khi kỹ sư thực hiện một lệnh cURL độc lập trực tiếp tới dịch vụ đích, dịch vụ vẫn phản hồi rất nhanh trong vài phần nghìn giây. Kiểm tra tài nguyên hệ điều hành bằng `ss -s` cho thấy số lượng socket TCP mở tăng liên tục theo từng lượt gọi mà không được thu hồi, tiệm cận giới hạn file descriptors (`ulimit -n`). Đồng thời, khi kiểm tra bằng `net/http/httptrace`, hook `GotConnInfo.Reused` luôn trả về `false`, cho thấy `http.Transport` không tái sử dụng được kết nối rảnh nào giữa các lượt probe.

> **Dừng để điều tra và phản nghiệm.** Vì sao kết quả cURL nhanh chỉ là bằng chứng hỗ trợ thu hẹp giả thuyết chứ chưa đủ để bác bỏ hoàn toàn lỗi hạ tầng? Kỹ sư cần thiết kế phép thử nào với `httptrace` để xác minh vòng đời kết nối của client trước khi can thiệp vào cấu hình hệ thống?

Đáp án — chỉ đọc sau khi đã tự làm Kết quả cURL phản hồi nhanh chỉ cho thấy dịch vụ đích có khả năng phục vụ một kết nối đơn lẻ tại thời điểm thử nghiệm. Nó giúp thu hẹp nghi vấn về việc máy chủ đích bị treo hoàn toàn, nhưng chưa đủ để loại trừ các yếu tố hạ tầng phức tạp như giới hạn kết nối đồng thời trên tường lửa, chính sách cân bằng tải hay tình trạng bão hòa hàng đợi socket.

Để cô lập nguyên nhân, kỹ sư sử dụng `net/http/httptrace` đo lường tỷ lệ kết nối mới so với kết nối tái sử dụng qua trường `info.Reused`. Theo tài

liệu của gói `net/http`, một kết nối HTTP/1.x chỉ đủ điều kiện đưa trở lại pool tái sử dụng khi client đã đọc hết dữ liệu của response body và đóng nó tường minh qua `resp.Body.Close()`. Nếu client bỏ rơi body hoặc quên đóng, `http.Transport` buộc phải để kết nối ở trạng thái dở dang hoặc ngắt bỏ, khiến mỗi request tiếp theo đều phải mở một kết nối TCP mới cho đến khi cạn kiệt tài nguyên hệ thống.

Bằng chứng mã nguồn và Đo đạc thực nghiệm

Kiểm tra `incident/incident.go` (đoạn hàm `BuggyProbe`):

```
resp, err := client.Do(req)
if err != nil {
    return 0, false, err
}
// BUG: Quên gọi resp.Body.Close()!
return resp.StatusCode, false, nil
```

Trong kịch bản HTTP/1.x của lab, hàm trả về mà không đọc hết hoặc đóng body. Kết nối đang giữ body chưa hoàn tất không được tái sử dụng cho request khác; tải tiếp tục có thể làm tăng số kết nối. Không áp kết luận này nguyên xi cho cơ chế multiplexing của HTTP/2.

Bằng chứng đo đạc thực nghiệm

Trong fixture 20 request tuần tự tới server kiểm thử ở `incident/incident_test.go`, `httptrace` ghi nhận các số đếm sau. Chúng mô tả đúng fixture này, không dự đoán tỷ lệ tái sử dụng kết nối của mọi server hay workload:

Chỉ số thực nghiệm	BuggyProbe (Bỏ quên Close)	FixedProbe (Close + Drain 16 KiB)
New Conns (`Reused == false`)	**20**	**1** (trong fixture này)
Reused Conns (`Reused == true`)	**0**	**19** (trong fixture này)
Kết quả vận hành	Mỗi request mở socket mới	Tái sử dụng socket trong pool

Kiểm thử `TestIncident_BoundedDrainOversizedBody` xác nhận rằng với body 32 KiB (vượt giới hạn 16 KiB), lab trả `reusedEligible == false` và đóng body. Bounded drain giới hạn lượng dữ liệu mà client chủ động đọc bỏ; nó không phải cơ chế chống tràn bộ đệm hay bảo đảm một kết nối được tái sử dụng.

Consumer đã rời đi, worker còn nợ ai kết quả?

Các test trước bảo vệ một lô target đã biết. Bây giờ đổi câu hỏi: producer tiếp tục nhận việc trong khi dependency chậm, queue đầy, rồi consumer ngừng đọc. Thêm worker có thể làm dependency chịu tải hơn; thêm buffer chỉ kéo dài khoản nợ. Thực nghiệm `projects/opsprobe/failurelab` bọc `ProbeSingle` hiện có bằng một admission queue hữu hạn, không viết lại pool hay API production của dự án.

Mở `TestBoundedAdmission` trước source. Fixture giữ hai request tại barrier của HTTP server cục bộ. Hai việc tiếp theo nằm trong queue; 100 lần nộp thêm phải nhận `ErrFull`, không tạo thêm goroutine. Release barrier rồi consume bốn kết quả: running và queue đều về 0, peak running không vượt 2. Các số này là contract capacity của fixture, không phải benchmark máy hay khuyến nghị “production nên chạy hai worker”.

Sau đó chạy `TestConsumerStopsContract`. Probe hoàn tất, checkpoint ngay trước send xác nhận worker đang chuẩn bị bàn giao; consumer cố ý không đọc output. Cancel phải cho worker trả về và coordinator đóng `Done`. Bản lỗi bỏ nhánh cancellation ở send nên không thể join trong budget của test. Harness có đường cứu hộ riêng để thu hồi nó sau khi oracle bắt lỗi; bản lỗi không được để lại một goroutine bị kẹt trong chính suite.

```
cd projects/opsprobe
go test -count=1 -v ./failurelab
$env:RELIABILITY_MUTANT='uncancellable_send'
go test -count=1 -run TestConsumerStopsContract ./failurelab
Remove-Item Env:RELIABILITY_MUTANT
```

Lệnh thứ hai phải đỏ. Trước khi mở lời giải, chỉ ra điểm block không còn receiver và vì sao context trên HTTP request không giải phóng điểm ấy. Đây là lần tích hợp contract send của Chương 9, không phải một dạng scheduler tuning.

Đáp án. HTTP đã xong nên deadline ở `ProbeSingle` không còn quản lý worker. Worker còn chờ send kết quả; chỉ `select` nghe context của runner mới mở đường thoát. Caller phải cancel rồi join `Done` khi ngừng consume. Bản sửa không bảo đảm consumer xử lý mọi result sau cancellation; đó là policy bỏ những lần bàn giao chưa hoàn tất, cần chấp nhận trước khi dùng API streaming.

Lấy tín hiệu đủ sức phân biệt nguyên nhân

`TestLocalFaultsAndSignals` lần lượt tiêm hold/release, deadline, 503, body bị cắt dù header 200, và kết nối đóng trước response. Hold/release là một response chậm được điều khiển bằng barrier, không dựa vào sleep để đoán server đã vào handler. Deadline là failure được chờ thật. Đối với mỗi case, test đọc outcome, một log tương ứng, histogram sample và span `opsprobe.probe` từ các thành phần telemetry sẵn có. Nó không gọi collector, Internet hay service production.

Bảng 23 — Chọn phép đo phân biệt thay vì đổi cấu hình theo triệu chứng

Triệu chứng	Giả thuyết cạnh tranh	Bằng chứng cần ghép
Latency cao	Dependency đang giữ response hay worker chờ được chạy?	Barrier server và span HTTP; dùng trace `net/sched` Chương 10 để kiểm tra wait/runnable.
Queue đầy	Tải đến vượt năng lực xử lý hay worker không thể thoát?	Rejection/running/queue; ngừng nộp, cancel rồi join đúng worker.
Caller đã timeout	Dependency dừng hay chỉ client thôi chờ?	`Done` phía client và tín hiệu kết thúc riêng của handler phía nhận.
Heap tăng	Allocation tạm, backlog hợp lệ hay retention sai?	Workload và queue; profile sống sau quiescence của Chương 10, không chỉ một snapshot.
Thời gian cao, CPU ít	Chờ I/O/đồng bộ hay chi phí CPU bị nhìn thiếu?	Trace wait cùng CPU profile của đúng workload; không gọi CPU thấp là bằng chứng tuyệt đối.

`TestCancelDoesNotProveDependencyStopped` giữ một handler cố ý không đọc context. Client cancel và join được runner, nhưng handler vẫn chưa kết thúc cho tới khi fixture release nó. Một timeout trả cho caller không phải acknowledgment rằng downstream đã hủy tác dụng phụ. Với operation có mutation, quay lại identity và cửa sổ gửi lặp ở Chương 13/25 trước khi thêm retry.

Runner tính deadline từ lúc admission, nên thời gian trong queue nằm trong budget; timeout riêng của target có thể thu hẹp nó thêm. Nếu job tới worker khi budget đã hết, request không có thêm một budget mới để che latency chờ. Policy ở đây là load shedding ngay khi queue đầy, không retry. Queue chỉ giữ tối đa capacity target và worker cố định; caller vẫn phải giới hạn kích thước từng input ở boundary của ứng dụng thật.

Phép chạy `RUN_BOUNDED_LOAD=1` gửi đúng 64 attempts trong tối đa 10 giây, với bốn worker và queue 16. Nó ghi throughput, p50/p95 thời lượng probe, outcome counts, peak running/queue, goroutine và heap snapshots. Histogram `failurelab_job_seconds` còn tính thời gian chờ queue; hai định nghĩa latency này không được trộn. Output thực tế và lệnh tái lập ở README/evidence của lab. So sánh throughput giữa các máy không phải

oracle; heap chưa chuẩn hóa sau GC cũng không phải bằng chứng retention. Giữ lab memory của Chương 10 để kiểm tra giả thuyết ấy thay vì dựng thêm một detector leak theo con số tùy tiện.

Đóng gói, điều phối và chuyển giao có trách nhiệm

Để đưa `opsprobe` ra môi trường production, ta áp dụng toàn bộ các nguyên tắc đã học ở Chương 17 và 18:

Một là, Dockerfile nhiều giai đoạn (multi-stage): lab xây binary với `CGO_ENABLED=0`, rồi dùng base image `gcr.io/distroless/static-debian12:nonroot` và tài khoản không đặc quyền (`USER 65532:65532`). Với dependency thuần Go của lab, cách này không yêu cầu C runtime; không suy rộng thành bảo đảm mọi chương trình đều không có dependency ngoài binary.

Hai là, lab dùng một Pod và một file SQLite, với `replicas: 1`, PVC `ReadWriteOnce` và chiến lược `Recreate`. RWO giới hạn ghi theo node, không phải theo Pod; nhiều Pod trên cùng node vẫn có thể mount volume. SQLite có cơ chế khóa cho nhiều connection/process, nên không phải cứ hai process là dữ liệu mất nhất quán. Rủi ro tăng khi chia sẻ file qua filesystem có semantics khóa/sync không phù hợp. `Recreate` hạn chế chồng lấn rollout thông thường, không phải distributed lock hay fencing khi node lỗi. Nếu cần nhiều writer trên các node, ưu tiên database client-server như PostgreSQL, hoặc thiết kế một owner duy nhất cho SQLite cùng contract failover rõ ràng.

Ba là, cấu hình lúc khởi động qua ConfigMap: Concurrency, timeout, backpressure limit và log level được nạp từ `deploy/k8s/configmap.yaml` vào biến môi trường (`OPSPROBE_*`). Sửa ConfigMap không tự cập nhật biến môi trường của process đang chạy; cần tạo lại Pod hoặc thiết kế cơ chế đọc lại riêng.

Bốn là, ghim nội dung image trong CI/CD: policy của dự án yêu cầu manifest dùng digest (`image: ghcr.io/...@sha256:...`) đã được pipeline kiểm chứng. Cách này không phụ thuộc việc tag như `:latest` có bị trỏ sang

image khác hay không; digest tự nó không chứng minh image đáng tin, đúng platform hoặc còn được registry lưu giữ.

Lệnh kiểm thử và vận hành hệ thống

Anh có thể trực tiếp chạy và kiểm chứng toàn bộ năng lực của `opsprobe` trong terminal:

```
cd projects/opsprobe

# 1. Chạy test suite có race detector
go test -race ./...

# 2. Chạy CLI kiểm tra URL đơn lẻ
go run ./cmd/opsprobe `
  --oneshot-url=https://go.dev --timeout=2s

# 3. Khởi chạy HTTP daemon server
go run ./cmd/opsprobe --addr=127.0.0.1:8080 `
  --db=opsprobe.db --concurrency=4
```

Trong cửa sổ khác, gửi yêu cầu probe và thu thập metrics:

```
$target = '{"id":"t1","url":"http://127.0.0.1:8080/livez"}'
curl -X POST http://127.0.0.1:8080/runs `
  -H "Content-Type: application/json" `
  -d "{`"targets`":[$target]}"
curl http://127.0.0.1:8080/metrics
```

Cột mốc hoàn thành dự án tổng kết: Chốt mốc chuẩn hệ thống

Capstone ghép các proof nhỏ: value và aliasing, đồng bộ goroutine, body lifecycle, transaction và artifact identity. Mỗi proof có phạm vi riêng; transaction không giải quyết mọi external side effect, shutdown có deadline và có thể không hoàn tất mọi request, còn digest không chứng minh nội dung artifact đúng. Khi chuyển sang môi trường thật, giữ những giới hạn ấy trong test và observation.

Khi mở rộng công cụ, hãy giữ các contract này thành tiêu chí review: input nào được tin, tài nguyên nào có owner, side effect nào có thể lặp, và observation nào xác nhận kết quả. Chương 21 dùng chính các câu hỏi ấy cho một controller.

Việc hoàn thành dự án `opsprobe` chốt lại baseline vững chắc của cuốn sách sống (**living textbook**), đồng thời mở ra những bài toán hệ thống ở quy mô hạ tầng cao hơn: khi hệ thống không chỉ thăm dò thụ động mà cần liên tục tự điều hòa, dung hòa sai lệch giữa trạng thái mong muốn và thực tế để tự phục hồi.

1. Go Team. The Go Programming Language Specification: memory model, concurrency semantics, channels và types. go.dev/ref/spec
2. Go Team. Package `net/http`: Client, Transport, Request and Response lifecycle. pkg.go.dev/net/http
3. Go Team. Package `database/sql`: connection pooling, transactions, prepared statements. pkg.go.dev/database/sql
4. Go Team. Package `log/slog`: Structured Logging in Go. pkg.go.dev/log/slog
5. Prometheus Authors. Prometheus Go client library documentation. prometheus.io/docs/guides/go-application/
6. OpenTelemetry Authors. OpenTelemetry Go Documentation and Tracing API. opentelemetry.io/docs/languages/go/
7. Google Container Tools. Distroless Container Images. github.com/GoogleContainerTools/distroless

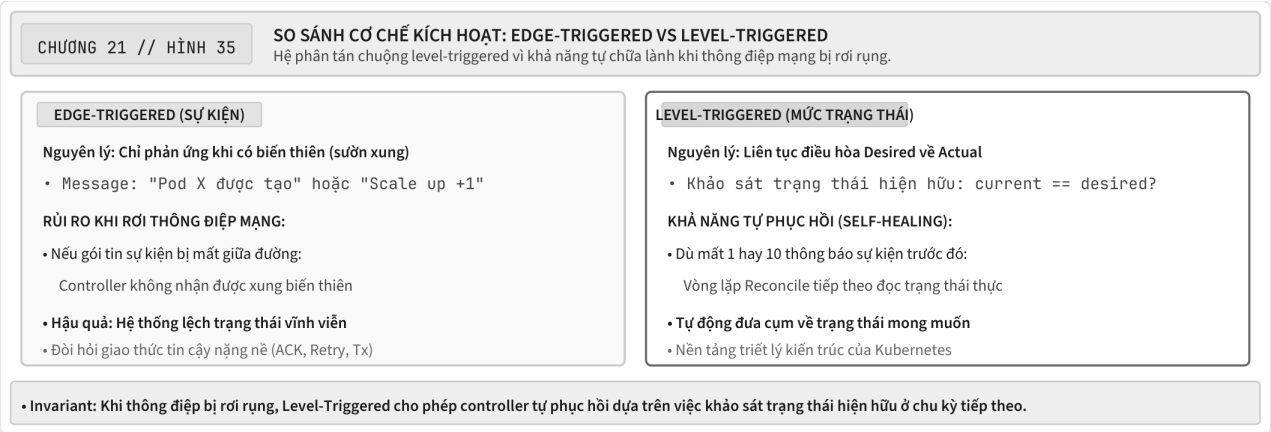
Chương 21 — Vòng lặp điều hòa và Controller Pattern

Trong Chương 20, dự án `opsprobe` đã trang bị cho ta một hệ thống thăm dò trạng thái hoàn chỉnh: kiểm tra định kỳ, phân loại kết quả rành mạch (`OutcomeSuccess`, `OutcomeFailure`, `OutcomeTimeout`), bảo toàn dữ liệu bằng giao dịch nguyên tử SQLite, và đo lường độ trễ qua Prometheus. Nhưng khi một dịch vụ phụ thuộc sập nguồn khiến dashboard chuyển màu đỏ rực, câu hỏi tiếp theo của một kỹ sư vận hành không còn là "làm sao để phát hiện?", mà là: "làm sao để hệ thống tự khôi phục trạng thái lành lặn mà không cần con người thức dậy lúc nửa đêm bấm nút restart?".

Chương này chuyển từ quan sát sang điều hòa: controller đọc trạng thái, tính sai lệch, thực hiện một thao tác rồi quan sát lại. Kubernetes dùng mẫu này cho nhiều controller; các công cụ khác có lịch chạy và mô hình quản lý state riêng, không nên gộp tất cả thành cùng một vòng lặp liên tục.

Kích hoạt theo mức và Kích hoạt theo cạnh

Để hiểu vòng lặp điều hòa trong controller của chương này, ta phân biệt hai cách thiết kế: **Edge-Triggered (Kích hoạt theo cạnh)** và **Level-Triggered (Kích hoạt theo mức)**. Đây không phải lựa chọn bắt buộc của mọi hệ thống phân tán đáng tin cậy.



Hình 35 — Khi một thông điệp bị mất, controller dựa vào trạng thái có thể đọc lại observation và tiếp tục điều hòa. Khả năng hội tụ vẫn phụ thuộc việc quan sát và điều chỉnh thành công.

Trong mô hình edge-triggered, consumer phản ứng với chuyển trạng thái. Nếu mất event mà không có replay, resync hoặc quan sát bổ sung, consumer có thể không biết state đã đổi và giữ kết luận cũ. Đây là lý do một controller cần recovery từ observation hiện tại, không chỉ một chuỗi event được giả định không bao giờ mất.

Trong mô hình hướng trạng thái của lab, controller so observation hiện tại với desired state thay vì thực thi từng event như một command. Desired 3/current 1 dẫn tới đề xuất tạo thêm 2. Observation có thể cũ hoặc thiếu, nên chưa đủ để kết luận có thể tạo ngay: actuator vẫn cần identity, precondition và quyền phù hợp.

Ngay cả khi controller bị tắt đột ngột lúc đang khởi động bản sao thứ hai, ở lần thức dậy kế tiếp, nó lại tiếp tục so sánh và nhận ra sai lệch còn tồn tại. Nó có thể yêu cầu hành động tiếp, nhưng chỉ một quan sát mới mới xác nhận được sự hội tụ. Đặc tính hướng về trạng thái mong muốn này gọi là **sự hội tụ trạng thái (State Convergence)**.

Bốn pha của Vòng lặp điều hòa

Ta dùng bốn pha dưới đây làm mô hình đọc control loop của chương, không phải quy trình bắt buộc cho mọi controller:

Bước	Tên gọi	Nhiệm vụ kỹ thuật trong Go
1	**Observe**	Đọc trạng thái thực tế từ hệ thống hoặc store.
2	**Diff**	So sánh trạng thái thực tế với trạng thái mong muốn.
3	**Act**	Thực thi hành động khắc phục để triệt tiêu sai lệch.
4	**Schedule / Observe again**	Chờ sự kiện, resync hoặc lịch thử lại để đọc một quan sát mới; không suy diễn trạng thái từ kết quả lời gọi.

Trong Go, ta đóng gói hành vi này thành một interface nhỏ gọn:

```
type Result struct {
    Requeue      bool
    RequeueAfter time.Duration
}

type Reconciler interface {
    Reconcile(
        ctx context.Context, key string,
    ) (Result, error)
}
```

Quy ước này thể hiện rõ ranh giới trách nhiệm: `Reconciler` chỉ tập trung vào logic điều hòa của một tài nguyên duy nhất được định danh bởi `key`. Việc quản lý hàng đợi, phân phối worker và kiểm soát tần suất thử lại do tầng `Controller` bên ngoài đảm nhiệm.

Hàng đợi công việc có kiểm soát tốc độ

Một sai lầm thường gặp của lập trình viên Go là sử dụng ngay một unbuffered channel (`chan string`) để làm hàng đợi cho controller. Trên môi trường production, channel thô nhanh chóng bộc lộ ba lỗ hổng nghiêm trọng:

Một là, thiếu gộp key trùng: trong scenario một consumer nhận và xử lý đủ 50 item cùng key từ channel mà không gộp, nó gọi reconcile 50 lần. Đây là cách viết queue ngây thơ, không phải lời hứa về mọi lịch chạy của channel; các lượt cũng có thể đọc observation khác nhau. Queue có dirty/processing giúp gộp lịch xử lý, không gộp mọi side effect thành exactly-once.

Hai là, xử lý cùng key song song: hai worker có thể cùng đọc observation thiếu replica rồi đều yêu cầu tạo thêm. Tuần tự hóa theo key trong một process giảm cạnh tranh đó, nhưng không phối hợp các process hay ngăn observation cũ; cần ownership và precondition ở actuator. Không gọi mọi thao tác trùng là split-brain, vốn liên quan các bên cùng cho rằng mình giữ quyền điều phối.

Ba là, retry storm: nếu dependency liên tục lỗi mà key bị đưa lại ngay không có budget/backoff, worker có thể tạo một vòng retry dày, tiêu CPU và tăng tải lên dependency. Đây là busy retry, không phải spin-lock — một cơ chế chờ lock khác. Chính sách retry cần giới hạn tốc độ và hỗ trợ cancellation.

Để nhìn rõ ba bài toán đó, lab dùng một `WorkQueue` tối giản. Nó mô phỏng các ý tưởng `dirty`, `processing` và đưa lại key sau `Done`, chứ không phải bản sao chính xác của `client-go/util/workqueue`.

```
type WorkQueue struct {
    mu           sync.Mutex
    cond         *sync.Cond
    queue        []string
    dirty        map[string]struct{}
    processing   map[string]struct{}
    failures     map[string]int
    shuttingDown bool
    config       RateLimiterConfig
}
```

Cơ chế phối hợp giữa `dirty` và `processing` tạo nên tính năng then chốt của hàng đợi:

```
func (q *WorkQueue) Add(item string) {
    q.mu.Lock()
    defer q.mu.Unlock()

    if q.shuttingDown {
        return
    }
    // 1. Deduplication: Nếu item đã nằm trong dirty, bỏ qua
    if _, exists := q.dirty[item]; exists {
        return
    }
    q.dirty[item] = struct{}{}

    // 2. Nếu item đang xử lý, không đẩy vào slice
    // để tránh hai worker cùng reconcile một key
    if _, isProcessing := q.processing[item]; isProcessing {
        return
    }

    q.queue = append(q.queue, item)
    q.cond.Signal()
}
```

Khi một worker hoàn tất lượt xử lý và gọi `Done(item)`:

```
func (q *WorkQueue) Done(item string) {
    q.mu.Lock()
    defer q.mu.Unlock()

    delete(q.processing, item)

    // Nếu trong lúc worker đang chạy, có thêm sự kiện mới
    // cho cùng key này, item lập tức được đưa lại vào queue
    if _, exists := q.dirty[item]; exists {
        q.queue = append(q.queue, item)
        q.cond.Signal()
    }
}
```

Trong mô hình này, một key không được hai worker xử lý đồng thời. Nếu `Add` xuất hiện khi key đang xử lý, `dirty` ghi nhận rằng cần có một lượt sau

`Done`. Đây là cơ chế gộp lịch xử lý, không phải cam kết exactly-once: worker có thể lỗi, tiến trình có thể dừng, và trạng thái có thẩm quyền vẫn phải được đọc lại ở lần reconcile sau.

> **Dừng để dự đoán.** Giả sử hàng đợi đang rỗng và Worker 1 vừa lấy key `"pod-1"` ra để chạy `Reconcile`. Trong lúc hàm reconcile đang thực thi dở dang, liên tiếp có hai sự kiện `Add("pod-1")` mới xuất hiện. Hàng đợi phối hợp `dirty` và `processing` như thế nào để đảm bảo worker thứ hai không nhảy vào can thiệp, và chuyện gì xảy ra khi Worker 1 gọi `Done("pod-1")`?

Đáp án — chỉ đọc sau khi đã tự làm Bảng dưới đây mô tả sự dịch chuyển trạng thái nội bộ của hàng đợi qua từng thời điểm:

Thời điểm	Thao tác	queue	dirty	processing	Ý nghĩa vận hành
T_0	Ban đầu	`[]`	`{}`	`{}`	Hàng đợi rỗng.
T_1	`Add("pod-1")`	`["pod-1"]`	`{"pod-1"}`	`{}`	Đưa vào slice và đánh dấu dirty.
T_1'	Worker 1 gọi `Get()`	`[]`	`{}`	`{"pod-1"}`	Lấy ra xử lý: xóa khỏi dirty, đưa vào processing.
T_2	`Add("pod-1")` (lần 1)	`[]`	`{"pod-1"}`	`{"pod-1"}`	Thấy đang processing: chỉ đánh dấu dirty, không thêm vào slice.
T_2'	`Add("pod-1")` (lần 2)	`[]`	`{"pod-1"}`	`{"pod-1"}`	Thấy đã trong dirty: gộp lại (deduplicate), bỏ qua.
T_3	Worker 1 gọi `Done("pod-1")`	`["pod-1"]`	`{"pod-1"}`	`{}`	Xóa khỏi processing; thấy dirty còn key: đưa lại vào slice.
T_4	Lượt `Get()` kế tiếp	`[]`	`{}`	`{"pod-1"}`	Reconcile lại để quan sát trạng thái mới nhất.

Cơ chế này hiện thực hóa mô hình điều hòa dựa trên trạng thái (level-triggered), không phải một hệ thống hàng đợi tin nhắn đảm bảo chuyển phát từng sự kiện riêng lẻ. Việc gộp sự kiện (coalescing) bảo đảm rằng dù có bao nhiêu thay đổi dồn dập trong lúc xử lý, hàng đợi chỉ lên lịch thêm đúng một lượt reconcile kế tiếp sau khi worker hiện tại hoàn tất, đồng thời ngăn chặn việc hai worker cùng xử lý một key song song. Quan trọng hơn, hàng đợi trong bộ nhớ không đưa ra cam kết exactly-once: nếu tiến trình bị dừng đột ngột, toàn bộ dữ liệu trong slice và map sẽ biến mất và

controller phụ thuộc hoàn toàn vào chu kỳ quan sát kế tiếp để tái lập bức tranh trạng thái có thẩm quyền từ nguồn chân lý.

Kiểm soát giãn cách lũy thừa khi có sự cố

Khi `Reconcile` thất bại, thay vì đưa key trở lại hàng đợi ngay lập tức, ta áp dụng thuật toán **Exponential Backoff** thông qua phương thức `AddRateLimited`:

```
func (q *WorkQueue) AddRateLimited(item string) bool {
    q.mu.Lock()
    defer q.mu.Unlock()

    if q.shuttingDown {
        return false
    }
    failures := q.failures[item] + 1
    q.failures[item] = failures

    if failures > q.config.MaxRetries {
        return false // Vượt quá số lần thử lại tối đa
    }

    // Chờ tăng gấp đôi: base * 2^(failures-1)
    multiplier := 1 << (failures - 1)
    delay := q.config.BaseDelay * time.Duration(multiplier)
    if delay > q.config.MaxDelay {
        delay = q.config.MaxDelay
    }

    time.AfterFunc(delay, func() {
        q.Add(item)
    })
    return true
}
```

Với `BaseDelay=50ms`, lịch thử lại của mô hình này tăng theo 50ms, 100ms, 200ms, 400ms, rồi chạm trần `MaxDelay`. Giãn cách giúp giới hạn nhịp retry khi hệ thống đích gặp lỗi; nó không loại bỏ mọi đợt retry dồn dập giữa nhiều key hay nhiều controller. Khi reconcile thành công, controller gọi `q.Forget(item)` để xóa bộ đếm thất bại.

Tính lũy đẳng: hành động có thể thử lại, quan sát được đọc lại

Vì một tài nguyên có thể được đưa vào hàng đợi nhiều lần do sự kiện lặp, resync định kỳ hoặc retry sau lỗi, reconcile phải được thiết kế để an toàn khi gọi lại. Công thức `f(f(x)) = f(x)` chỉ là một trực giác về trạng thái thuần; nó không mô tả đầy đủ hợp đồng production, nơi lời gọi ra bên ngoài có thể thành công nhưng phản hồi bị mất, còn cache có thể cũ.

Khi quan sát mới đã khớp trạng thái mong muốn, reconcile nên là no-op. Khi cần hành động, actuator phải có precondition hoặc idempotency key phù hợp với hệ thống đích. Sau khi `Act` trả về `nil`, controller mới biết lời gọi đã kết thúc mà không báo lỗi; nó chưa biết thế giới bên ngoài đã hội tụ. Sự xác nhận chỉ đến từ lần `Observe` kế tiếp.

Hãy xem xét một Reconciler tự chữa lành cụ thể:

```
func (r *SelfHealingReconciler) Reconcile(
    ctx context.Context, key string,
) (Result, error) {
    state, exists := r.getObservedState(key)
    if !exists {
        return Result{}, nil // Không tồn tại: No-op
    }

    // 1. Phân tích sai lệch giữa thực tế và mong muốn
    isDegraded := !state.Healthy ||
        state.Replicas < r.desiredReplicas

    // 2. Đã đạt trạng thái mong muốn thì không đổi thêm
    if !isDegraded {
        return Result{}, nil
    }

    // 3. Thực thi hành động tự chữa lành (Act)
    if err := r.healTarget(ctx, key); err != nil {
        return Result{}, fmt.Errorf("heal %s: %w", key, err)
    }

    // 4. Không tự sửa actual state.
    // Event hoặc resync sẽ đưa key quay lại
    // khi lớp quan sát thấy trạng thái bên ngoài đã đổi.
    return Result{}, nil
}
```

Nếu `payment-service` đang có 1 replica trong khi yêu cầu là 3, hàm gọi actuator với `context.Context`. Một test cố ý để actuator trả `nil` nhưng giữ quan sát cũ vẫn cho thấy target chưa khỏe; reconciler không được gán `Healthy=true` chỉ để làm bài kiểm thử xanh. Khi quan sát sau đó phản ánh 3 replica khỏe, lượt reconcile kế tiếp mới là no-op. Lab cũng chặn actuator bằng channel rồi thay quan sát trong lúc action đang chạy, để chứng minh action cũ không được ghi đè quan sát mới.

Tắt nguồn mềm mại cho Controller

Controller quản lý nhiều worker goroutine. Khi context bị hủy, controller đóng hàng đợi để đánh thức worker rồi chờ chúng thoát. Công việc đang chạy có thể trả lỗi do cancellation, không nhất thiết hoàn tất nghiệp vụ; policy drain đến khi hoàn tất cần được thiết kế riêng:

```
func (c *Controller) Run(ctx context.Context) error {
    var wg sync.WaitGroup

    // Đánh thức và đóng hàng đợi khi context bị hủy
    go func() {
        <-ctx.Done()
        c.queue.ShutDown()
    }()

    for i := 0; i < c.concurrency; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for c.processNextWorkItem(ctx) {
            }
        }()
    }

    wg.Wait()
    return ctx.Err()
}
```

Phương thức `queue.ShutDown()` giải phóng các goroutine đang bị block ở lệnh `cond.Wait()`. Vòng lặp `for c.processNextWorkItem(ctx)` nhận được tín hiệu `shutdown == true` khi hàng đợi đã cạn, kết thúc vòng lặp và thông báo qua `wg.Done()`. Việc đóng queue chỉ giúp worker thoát; nó không tự bảo đảm ghi bên ngoài hoàn tất. Actuator phải tôn trọng `ctx`, và tác dụng phụ đã gửi sang hệ thống khác cần idempotency hoặc quan sát lại riêng.

Thực hành Lab: Kiểm chứng Controller và WorkQueue

Toàn bộ kiến trúc trên được đóng gói tại thư mục thực hành `labs/part21-reconciliation-controller`:

```
cd labs/part21-reconciliation-controller
go test -v ./...     # Chạy toàn bộ test suite
go test -race ./...  # Kiểm tra race condition
go vet ./...        # Phân tích tĩnh cú pháp
```

Bộ kiểm thử dùng channel để xác nhận actuator nhận được hủy context, một action thành công không tự bịa observation, và observation mới xuất hiện khi action đang chạy không bị ghi đè. Test backoff hiện chỉ xác nhận có nhiều lượt thử trong cửa sổ kiểm thử; nó chưa đo hoặc chứng minh chính xác các khoảng giãn cách tăng theo cấp số nhân.

Bước phát triển tiếp theo

Chương này chỉ xây mô hình điều hòa tối giản: quan sát, so sánh, hành động, rồi quan sát lại. Kết hợp với tín hiệu từ `opsprobe` (Chương 20), nó cho ta một nơi để thấy rõ vì sao action result và observed state là hai bằng chứng khác nhau.

Đây là nền tảng trực tiếp cho Chương 22, nơi cùng các câu hỏi ấy gặp List/Watch, cache và workqueue của `client-go`; Chương 23 mới thêm API riêng, status, ownership và finalizer của một operator. Hàng đợi trong chương này là mô hình tái lập để đọc cơ chế, không phải bản thay thế cho implementation của Kubernetes.

1. Kubernetes Authors. Controller pattern and declarative state management. kubernetes.io/docs/concepts/architecture/controller/
2. B. Grant. Declarative application management in Kubernetes. github.com/kubernetes/community
3. Kubernetes Authors. Package `client-go/util/workqueue` reference and design. pkg.go.dev/k8s.io/client-go/util/workqueue
4. S. Newman. Building Microservices: Designing Fine-Grained Systems and Self-Healing Architecture. O'Reilly Media.
5. Go Team. Advanced Go Concurrency Patterns: Pipelines and cancellation. go.dev/blog/pipelines

Chương 22 — Từ watch đến một controller Kubernetes thật

Chương 21 dùng một queue tối giản để thấy sự khác nhau giữa action result và observed state. Khi nối tới Kubernetes API, ta phải thêm recovery của stream, cache sync, đồng bộ theo key và xử lý conflict. Một channel có thể là thành phần của thiết kế, nhưng riêng nó không cung cấp các contract ấy.

Chẳng hạn, trong scenario 500 cập nhật đến trong một giây, giữ toàn bộ payload cũ để xử lý có thể khiến worker hành động theo observation đã lỗi thời. Tải và hậu quả phải đo trong môi trường cụ thể; con số này chỉ đặt bài toán gộp key và đọc lại trạng thái.

Chương này dùng `client-go` v0.37.0 để dựng một controller nhỏ và kiểm tra các boundary đó. Lab không thay thế thiết kế RBAC, recovery hay kiểm chứng trên cluster production.

---

1. Bản chất: Tín hiệu thông báo không phải là Chân lý

Câu hỏi trung tâm của chương này là:

> **Khi nhận được một sự kiện từ Kubernetes API Server, ta có nên tin vào dữ liệu đính kèm bên trong sự kiện đó để ra quyết định điều hòa hay không?**

Với policy của controller trong chương, event là tín hiệu để xếp key, không phải snapshot được giữ tới lúc quyết định. Handler vẫn có thể dùng payload cho lọc hoặc lấy metadata; worker đọc lại observation trước khi điều hòa.

Trong các hệ thống phân tán quy mô lớn, **Sự kiện thông báo (Notification Hint)** chỉ là một lời nhắc nhở: **"Tài nguyên X tại namespace Y dường như vừa có biến đổi, hãy kiểm tra lại đi!"**. Bản thân sự kiện không phải là **Trạng thái thẩm quyền (Authoritative State)**.

API Server là boundary thẩm quyền cho Kubernetes object; informer cache là bản sao cục bộ giúp controller điều hòa hiệu quả, không phải nguồn chân lý và có thể trễ. Worker đọc cache để tránh xử lý payload event cũ, nhưng write/read-after-write hoặc quyết định cần freshness phải chọn API read hay cache bằng policy rõ ràng.

Bẫy tư duy: Xử lý theo payload của sự kiện

Hãy tưởng tượng kịch bản sau: lúc t_0, Pod `web-app` có trạng thái `Pending` và API Server phát sự kiện E_1(`Pending`). Đến lúc t_1, Kubelet khởi động xong container khiến Pod chuyển sang `Running`, và API Server phát sự kiện E_2(`Running`). Do độ trễ mạng hoặc hàng đợi bận rộn, worker trong controller nhận E_1 chậm mất 3 giây. Nếu worker dùng ngay dữ liệu bên trong E_1, nó sẽ tưởng Pod vẫn đang `Pending` và ra lệnh hủy Pod để tạo lại. Hành động này phá hủy trực tiếp Pod vừa khởi động lành lặn lúc t_1.

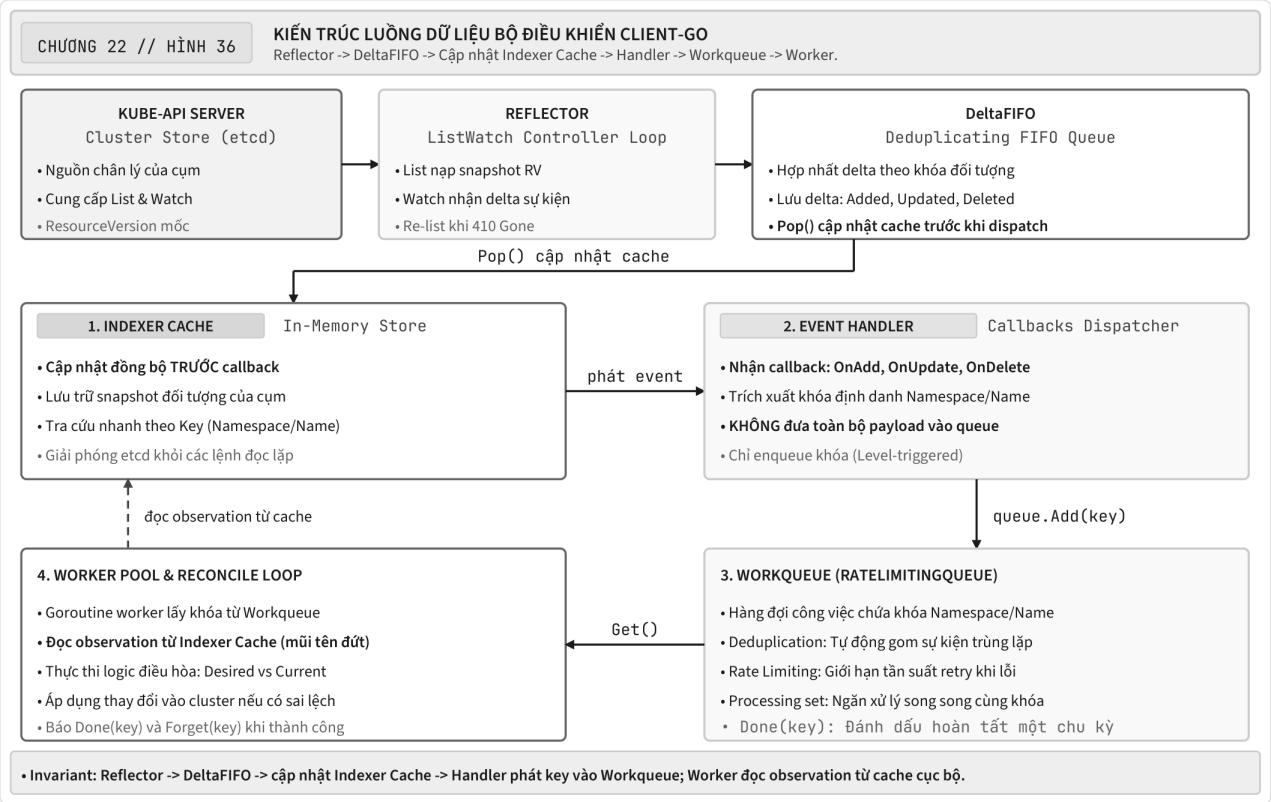
Policy được dùng trong lab:

> **Policy hàng đợi:** Đẩy key `namespace/name`, rồi đọc observation hiện hành trong Informer cache khi xử lý. Cache là observation local và có thể chậm hơn API server; không gọi nó là trạng thái mới nhất tuyệt đối của cluster.

---

2. Kiến trúc toàn cảnh của một client-go Controller

Luồng chính đi từ API Server qua `Reflector` và `DeltaFIFO` tới `SharedInformer`. Cache được cập nhật trước khi handler phát key cho hàng đợi; lúc xử lý, worker đọc lại observation cục bộ thay vì giữ payload sự kiện cũ.



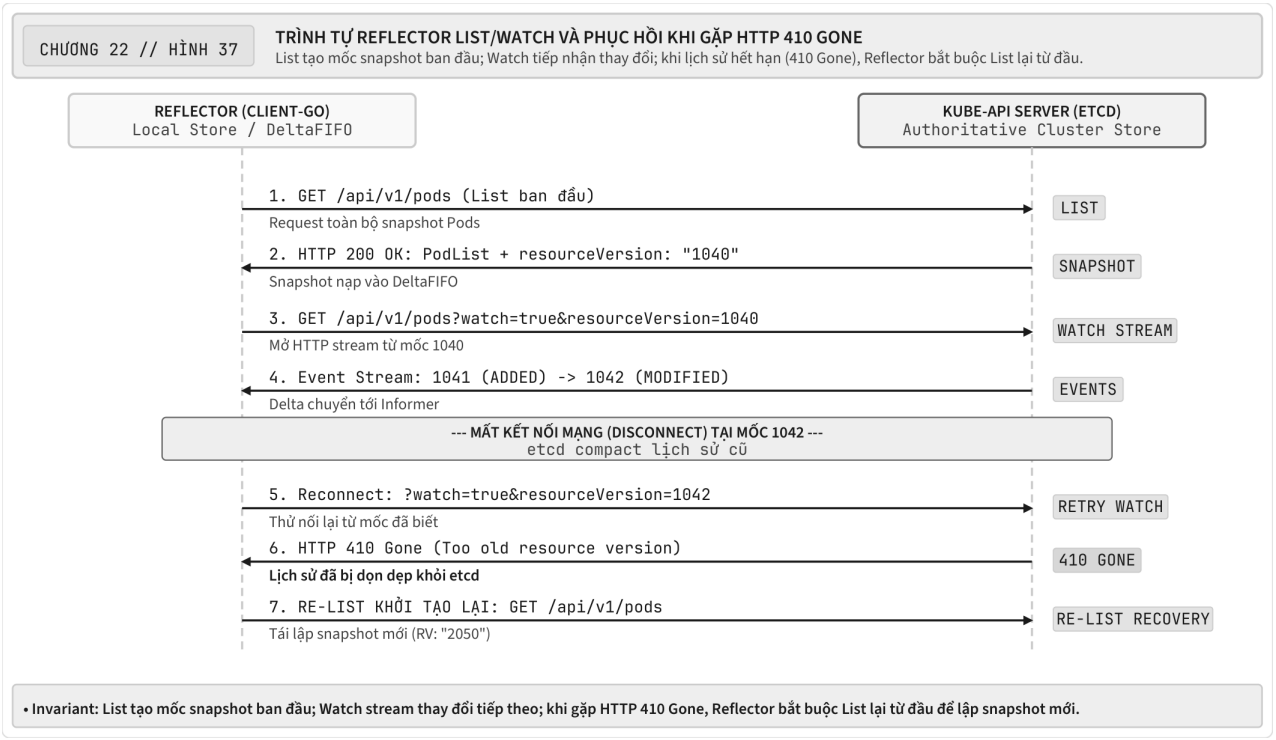
Hình 36 — Luồng sự kiện của controller. Đường nét đứt biểu thị cache cục bộ: dữ liệu này có thể trễ so với API Server.

Bốn thành phần then chốt trong chuỗi cung ứng dữ liệu

Thành phần Informer	Vai trò kỹ thuật	Tương tác trong hệ thống
`Reflector`	Mở kết nối List/Watch tới API Server	Đẩy các biến động tài nguyên vào bộ đệm `DeltaFIFO`.
`Indexer (Local Store)`	Bộ nhớ đệm in-memory có đồng bộ truy cập	Phục vụ lookup local và giảm read tới API server; chi phí cần đo theo index và workload.
`SharedInformer`	Phân phối sự kiện từ `DeltaFIFO`	Đảm bảo local cache cập nhật trước khi event handler kích hoạt.
`Typed Rate-Limited WorkQueue`	Hàng đợi công việc chuyên dụng	Gộp key trùng và kiểm soát thử lại; không thay thế đồng bộ dữ liệu dùng chung.

3. Giao thức List/Watch và vai trò của resourceVersion

List cung cấp snapshot và resourceVersion; Watch tiếp nhận các thay đổi theo mốc được API hỗ trợ. Đây là contract Kubernetes API, không phải quy tắc rằng hệ thống không được polling hay sử dụng transport khác.



Hình 37 — List tạo mốc quan sát cho Watch; khi nối lại không thể tiếp tục vì lịch sử đã hết, Reflector phải List lại.

Cơ chế hoạt động

Thứ nhất là đường List truyền thống: Reflector lấy danh sách cùng `resourceVersion` opaque. Nó đưa snapshot vào store là `DeltaFIFO`; vòng xử lý của Informer cập nhật Indexer, không phải Reflector ghi thẳng vào Indexer. Hình đang minh họa đường này. Trong client-go đã ghim còn có watch-list khi server và cấu hình hỗ trợ; không suy ra mọi lần khởi tạo đều gọi List riêng.

Thứ hai là Watch từ mốc đã nhận, chẳng hạn `?watch=true&resourceVersion=1040`. API gửi stream thay đổi sau mốc ấy khi lịch sử còn phục vụ được. HTTP/1.1 có thể dùng chunked transfer; HTTP/2 không dùng kiểu đóng khung đó. ResourceVersion không phải offset etcd mà client được phép diễn giải.

Thứ ba là tự phục hồi sau sự cố mạng: Nếu đường truyền bị đứt ở sự kiện `1042`, Reflector thử kết nối lại từ phiên bản đã biết. Đây là cơ chế bắt kịp thay đổi khi lịch sử còn giữ được, không phải lời hứa rằng mọi lần nối lại đều không cần List.

Thứ tư là xử lý lỗi HTTP 410 Gone: Nếu kết nối bị gián đoạn quá lâu khiến etcd đã dọn dẹp các bản ghi lịch sử (compact log), API Server sẽ trả về mã lỗi `410 Gone (Too old resource version)`. Khi đó, Reflector hiểu rằng khoảng trống dữ liệu không thể bù đắp, nó sẽ tự động kích hoạt một chu kỳ `List` mới từ đầu để tái lập snapshot chuẩn.

---

4. Xử lý xóa và bí ẩn của Tombstone

Trong vòng đời của một tài nguyên, sự kiện xóa (`OnDelete`) ẩn chứa một cạm bẫy kỹ thuật kinh điển mà hầu hết kỹ sư mới tiếp cận `client-go` đều vấp phải: **Tombstone** (Bia mộ dữ liệu).

Khi không nhận được delete event nhưng một lần List lại cho thấy key đã mất, DeltaFIFO có thể tạo `DeletedFinalStateUnknown`. Trường Obj là observation cuối cùng còn biết, có thể cũ, không phải snapshot tại lúc xóa. Handler cần chấp nhận cả object thông thường lẫn tombstone và không suy ra identity mới từ tên cũ.

Nếu bạn ép kiểu trực tiếp:

```
// CẢNH BÁO: Panic runtime nếu đối tượng rơi vào Tombstone!
pod := obj.(*corev1.Pod)
```

Chương trình sẽ văng lỗi `panic: interface conversion: interface {} is cache.DeletedFinalStateUnknown, not *v1.Pod`.

Giải pháp an toàn tiêu chuẩn:

Thư viện `client-go` cung cấp hàm chuẩn hóa `cache.DeletionHandlingMetaNamespaceKeyFunc`:

```
func (c *Controller) handleDelete(obj any) {
    key, err := cache.DeletionHandlingMetaNamespaceKeyFunc(obj)
    if err != nil {
        runtime.HandleError(err)
        return
    }
    c.queue.Add(key)
}
```

`DeletionHandlingMetaNamespaceKeyFunc` hỗ trợ object thông thường và tombstone `DeletedFinalStateUnknown` để lấy key. Caller vẫn phải xử lý error nếu object không có metadata phù hợp; helper không xác minh mọi dữ liệu nhận được là an toàn.

Key chỉ mang `namespace/name`, nên không phân biệt được một đối tượng cũ đã bị xóa với đối tượng mới cùng tên. Nếu cleanup ngoài Kubernetes cần danh tính bền vững, hãy ghi nhận `metadata.uid` lúc đối tượng còn tồn tại và ràng buộc tài nguyên ngoài với UID đó; đừng suy diễn UID từ một tombstone chỉ còn key.

---

5. Hàng đợi WorkQueue: Ba tập hợp và Kiểm soát tốc độ

Channel cung cấp gửi/nhận và thứ tự theo contract ngôn ngữ; nó không tự có dirty set, processing set hay policy retry. Trong implementation client-go v0.37.0, `workqueue.Typed[T]` phối hợp Queue có thể thay thế cùng hai tập key sau:

Tập hợp	Ý nghĩa kỹ thuật	Trách nhiệm
`queue []T`	Danh sách thứ tự chờ	Xác định key nào sẽ được worker lấy ra xử lý kế tiếp.
`dirty set[T]`	Tập hợp các key bị bẩn	Lưu các key có sự kiện phát sinh nhưng chưa hoàn tất điều hòa. Giúp **gộp trùng lặp (deduplication)**.
`processing set[T]`	Tập hợp key đang xử lý	Cơ chế queue để cùng key không được phân phối song song. Đây không thay thế race detector hay synchronization cho state do Reconciler sở hữu.

Quy trình điều phối của WorkQueue

`Add(key)` trước hết đánh dấu key trong `dirty`; key đã bẩn thì không được xếp thêm một bản sao. Nếu key đang ở `processing`, sự kiện mới chờ lượt xử lý hiện tại kết thúc. Nếu không, key được đưa vào `queue`. `Get()` lấy key ra, xóa dấu `dirty` và đánh dấu nó đang xử lý.

Khi worker xử lý xong, nó bắt buộc phải gọi `queue.Done(key)`. Lúc này WorkQueue sẽ xóa key khỏi tập `processing`. Nếu trong quá trình worker đang chạy mà có sự kiện mới tới (key đã được đánh dấu vào `dirty`), `Done()` sẽ tự động đưa key đó trở lại `queue` để xử lý tiếp!

Cơ chế Rate Limiting: Quên hay Phạt?

Một worker sau khi xử lý xong một lượt điều hòa (`reconcile`) sẽ đứng trước hai tình huống:

Thứ nhất, nếu thành công: Gọi `c.queue.Forget(key)` để xóa sạch lịch sử số lần thất bại, đưa bộ đếm backoff về mức 0, và gọi `c.queue.Done(key)`.

Thứ hai, nếu gặp thất bại tạm thời: Không gọi `Forget`, mà gọi `c.queue.AddRateLimited(key)`. Thời gian chờ do `RateLimiter` đã cấu hình quyết định. Lab dùng `DefaultTypedItemBasedRateLimiter` của client-go v0.37.0: lùi lũy thừa theo key, có mức trần, không tự thêm jitter hay token bucket toàn cục. Đây là cơ chế hạn chế tốc độ thử lại, không phải bảo đảm tránh mọi sự cố dây chuyền.

6. Hiện thực Controller hoàn chỉnh trong Go

Đoạn dưới trích phần lõi của `labs/part22-client-go-controller/controller.go`, dùng API generic ở client-go v0.37.0. Version được ghim để có thể đối chiếu, không phải yêu cầu luôn dùng dependency mới nhất.

```
type Reconciler interface {
    Reconcile(ctx context.Context, key string) error
}

type Controller struct {
    indexer    cache.Indexer
    queue      workqueue.TypedRateLimitingInterface[string]
    informer   cache.SharedIndexInformer
    reconciler Reconciler
}
```

Hàm khởi chạy `Run` bảo đảm khế ước đồng bộ hóa bộ nhớ đệm trước khi phân phối công việc cho các worker:

```
func (c *Controller) Run(
    ctx context.Context, workers int,
) error {
    defer runtime.HandleCrash()
    defer c.queue.ShutDown()

    // Khế ước bắt buộc: Đợi local cache đồng bộ hoàn tất
    if c.informer != nil {
        synced := cache.WaitForNamedCacheSyncWithContext(
            ctx, c.informer.HasSynced,
        )
        if !synced {
            return fmt.Errorf("hết hạn chờ sync cache")
        }
    }

    var wg sync.WaitGroup
    for i := 0; i < workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for c.processNextItem(ctx) {
            }
        }()
    }

    <-ctx.Done()
    c.queue.ShutDown()
    wg.Wait()
    return nil
}
```

Vòng lặp điều phối của từng Worker

Mỗi worker lấy một key ra khỏi hàng đợi và bọc trong khối bảo vệ:

```
func (c *Controller) processNextItem(ctx context.Context) bool {
    key, shutdown := c.queue.Get()
    if shutdown {
        return false
    }
    defer c.queue.Done(key)

    err := c.reconcileHandler(ctx, key)
    if err == nil {
        // Thành công: Xóa lịch sử retry để giải phóng bộ nhớ
        c.queue.Forget(key)
        return true
    }

    // Thất bại: Thử lại có kiểm soát tốc độ (tối đa 5 lần)
    if c.queue.NumRequeues(key) < 5 {
        c.queue.AddRateLimited(key)
        return true
    }

    // Đã vượt quá ngưỡng thử lại: Hủy bỏ và ghi log lỗi
    c.queue.Forget(key)
    runtime.HandleError(
        fmt.Errorf(
            "bỏ cuộc sau 5 lần thử key %q: %w", key, err,
        ),
    )
    return true
}
```

Logic hòa giải: Đọc từ Cache thay vì Payload sự kiện

```
func (c *Controller) reconcileHandler(
    ctx context.Context, key string,
) error {
    // 1. Truy vấn snapshot quan sát hiện có từ local indexer
    obj, exists, err := c.indexer.GetByKey(key)
    if err != nil {
        return fmt.Errorf("lỗi đọc cache key %s: %w", key, err)
    }

    if !exists {
        // Không thấy tài nguyên trong cache cục bộ.
        return c.reconciler.Reconcile(ctx, key)
    }

    pod, ok := obj.(*corev1.Pod)
    if !ok {
        return fmt.Errorf("sai kiểu *corev1.Pod: %T", obj)
    }

    // Không xử lý pod đang trong tiến trình bị xóa dở
    if pod.DeletionTimestamp != nil {
        return nil
    }

    return c.reconciler.Reconcile(ctx, key)
}
```

---

7. Bằng chứng kiểm thử: Kiểm chứng 6 hành vi then chốt

Bộ kiểm thử tại `labs/part22-client-go-controller/controller_test.go` kiểm tra sáu contract của lab dưới đây. Chúng không mô phỏng mọi điều kiện biên của cluster production:

```
=== RUN   TestEventDeduplication
--- PASS: TestEventDeduplication (0.00s)
=== RUN   TestCacheNewerThanEvent
--- PASS: TestCacheNewerThanEvent (0.00s)
=== RUN   TestRateLimitedRetryDecoupledFromDomain
--- PASS: TestRateLimitedRetryDecoupledFromDomain (0.00s)
=== RUN   TestSafeDeletionAndTombstone
--- PASS: TestSafeDeletionAndTombstone (0.00s)
=== RUN   TestCleanCancellationShutdown
--- PASS: TestCleanCancellationShutdown (0.05s)
=== RUN   TestCacheNotSyncedGuard
--- PASS: TestCacheNotSyncedGuard (0.02s)
PASS
ok      part22-client-go-controller   3.829s
```

1. Bằng chứng gộp trùng sự kiện (TestEventDeduplication)

Test thêm mười lần cùng key của Pod `production/web-proxy` vào queue trước khi worker chạy. Queue có một item và sau một lần xử lý, bộ đếm reconcile bằng một. Kết quả chứng minh gộp key đang chờ trong lịch chạy này, không phải giảm 90% tải hay bảo đảm exactly-once khi sự kiện đến trong lúc xử lý.

2. Đọc trạng thái mới từ Cache thay vì Payload cũ (TestCacheNewerThanEvent)

Test enqueue Pod có `ResourceVersion: "1"`, rồi cập nhật Indexer thành `"3"` trước khi worker xử lý. Kết quả quan sát là `"3"`, xác nhận đường xử lý đọc lại cache thay vì giữ payload cũ. Test không chứng minh cache luôn mới so với API Server.

3. Tách biệt lỗi tạm thời và Lũy thừa thử lại (TestRateLimitedRetry)

`TestRateLimitedRetryDecoupledFromDomain` giả lập hai lần reconcile thất bại rồi lần thứ ba thành công. Các assertion kiểm tra hai lần retry, ba lần gọi và không drop; nhánh thành công trong source gọi `Forget(key)` để xóa lịch sử retry của key.

4. An toàn trước Tombstone (TestSafeDeletionAndTombstone)

Test truyền `cache.DeletedFinalStateUnknown` qua `enqueue`, kiểm tra key `production/tombstone-pod` được đưa vào queue rồi ghi nhận action xóa trong kết quả. Lab không thực hiện hay chứng minh dọn dẹp tài nguyên ngoại vi.

5. Dừng sạch tài nguyên (TestCleanCancellationShutdown)

Test chạy ba worker, thêm năm key rồi gọi `cancel()`. Assertion yêu cầu `ctrl.Run` trả về trong hai giây; 50 ms là khoảng sleep trước cancellation, không phải số đo shutdown. Source đóng queue và chờ worker, nhưng test này không kiểm kê mọi goroutine hay bảo đảm độ trễ shutdown ở production.

6. Khế ước bảo vệ Cache Chưa Đồng Bộ (TestCacheNotSyncedGuard)

Trong lab, `WaitForNamedCacheSyncWithContext` trả false thì controller từ chối khởi động worker và trả lỗi cache sync. Gate ấy ngăn đường xử lý trước sync trong lifecycle này; sync không bảo đảm cache mãi không stale hay RBAC sẽ không thay đổi về sau.

---

8. Kiểm soát đồng thời lạc quan (OCC) và Lỗi 409 Conflict

Khi controller quyết định cập nhật trạng thái Pod lên API Server, nó gửi một lệnh HTTP `PUT` hoặc `PATCH`. Trong môi trường phân tán, một controller khác hoặc người dùng thông qua `kubectl` có thể đã sửa đổi Pod đó trước bạn một phần nghìn giây.

Với `Update` dưới đây, resourceVersion là precondition: version cũ khiến API server trả `409 Conflict`. Không phải mọi PATCH đều có cùng điều kiện; patch có thể bỏ resourceVersion hoặc dùng cơ chế conflict/field ownership khác. Hãy đọc contract của loại write đang dùng, không suy ra OCC chỉ từ phương thức HTTP.

```
Operation cannot be fulfilled on pods "payment-api":
the object has been modified; please apply your changes
to the latest version and try again
```

Mẫu hình chuẩn: retry.RetryOnConflict

Tuyệt đối không coi 409 là một lỗi hệ thống nghiêm trọng khiến controller sập nguồn. Đây là một hiện tượng bình thường trong hệ phân tán. Cách xử lý chuẩn xác bằng `k8s.io/client-go/util/retry`:

```
func updatePodAnnotation(
    ctx context.Context,
    client kubernetes.Interface,
    namespace, name string,
) error {
    return retry.RetryOnConflict(
        retry.DefaultRetry,
        func() error {
            // 1. Đọc lại bản ghi hiện tại từ API Server
            latest, err := client.CoreV1().Pods(namespace).Get(
                ctx, name, metav1.GetOptions{},
            )
            if err != nil {
                return err
            }

            // 2. Thực hiện sửa đổi trên bản sao mới
            if latest.Annotations == nil {
                latest.Annotations = make(map[string]string)
            }
            now := time.Now().UTC().Format(time.RFC3339)
            latest.Annotations["reconciled-at"] = now

            // 3. Ghi đè lại API Server
            _, err = client.CoreV1().Pods(namespace).Update(
                ctx, latest, metav1.UpdateOptions{},
            )
            return err
        },
    )
}
```

Mẫu hình này tự động thử lại với backoff ngắn, đọc lại snapshot hiện tại của đối tượng từ API Server, áp dụng thay đổi và lưu lại.

---

9. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Đọc API trực tiếp ở mọi lượt mà không có budget.	Tăng tải và độ trễ tùy workload.	Dùng cache khi freshness cho phép; dùng API reader có chủ đích cho precondition hay read-after-write.
Quên Done(key).	Key còn trong processing; Add mới có thể đánh dirty nhưng không được xếp lại cho tới Done.	Đặt defer Done sau Get thành công.
Gọi Forget(key) khi Reconcile gặp lỗi.	Làm mất lịch sử thử lại của key. Backoff bị xóa bỏ, dẫn đến bão thử lại liên tục.	Chỉ gọi `Forget(key)` khi điều hòa thành công hoặc khi quyết định từ bỏ sau nhiều lần thử.
Chạy worker pool trước khi đồng bộ xong cache.	Nếu diễn giải object chưa có trong cache thành đã bị xóa, worker có thể cleanup nhầm.	Gate worker bằng cache sync; ngay cả sau sync, cleanup vẫn cần bằng chứng identity/ownership riêng.

---

10. Bài tập thực hành thiết kế Controller

Thử thách 1: Tích hợp Metric đo đạc độ trễ và chiều sâu hàng đợi

Yêu cầu: Hãy thiết kế hai metric Prometheus để giám sát sức khỏe của controller: metric Gauge `controller_workqueue_depth` đo số lượng key đang chờ xử lý trong `queue`, và metric Histogram `controller_reconcile_duration_seconds` đo thời gian thực thi của một chu kỳ `Reconcile`.

Gợi ý: Đặt điểm đo `time.Now()` trước khi gọi `c.reconciler.Reconcile` và cập nhật metric trong lệnh `defer`.

Thử thách 2: Xử lý Pod bị cô lập (Orphaned Pod Cleanup)

Yêu cầu: Khi cache không còn object, key chỉ chứa namespace/name. Viết nhánh từ chối xóa file ngoài cụm nếu không có record durable ràng buộc UID và file. Chứng minh trường hợp object mới cùng tên không bị cleanup nhầm. Đây là bài tập nhận diện thiếu bằng chứng, không phải yêu cầu xóa cho bằng được.

---

11. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Đo lường quan sát controller

Khai báo các bộ đo Prometheus:

```
var (
    queueDepth = prometheus.NewGaugeVec(
        prometheus.GaugeOpts{
            Name: "controller_workqueue_depth",
            Help: "Số lượng phần tử đang chờ trong workqueue",
        },
        []string{"controller"},
    )
    reconcileDuration = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "controller_reconcile_duration_seconds",
            Help:    "Thời gian thực thi hàm Reconcile",
            Buckets: prometheus.DefBuckets,
        },
        []string{"controller", "status"},
    )
)
```

Thu thập số liệu đo lường trong vòng lặp xử lý:

```
func (c *Controller) processNextItemWithMetrics(
    ctx context.Context,
) bool {
    key, shutdown := c.queue.Get()
    if shutdown {
        return false
    }
    defer c.queue.Done(key)

    start := time.Now()
    err := c.reconcileHandler(ctx, key)
    duration := time.Since(start).Seconds()

    status := "success"
    if err != nil {
        status = "error"
    }
    reconcileDuration.WithLabelValues(
        "pod_controller", status,
    ).Observe(duration)
    // ... xử lý retry tiếp theo
    return true
}
```

Lời giải Thử thách 2: Xử lý dọn dẹp tài nguyên cô lập

```
func (r *CleanUpReconciler) Reconcile(
    ctx context.Context, key string,
) error {
    namespace, name, err := cache.SplitMetaNamespaceKey(key)
    if err != nil {
        return fmt.Errorf("key sai %q: %w", key, err)
    }

    obj, exists, err := r.indexer.GetByKey(key)
    if err != nil {
        return err
    }

    if !exists {
        // Key chỉ có namespace/name, không có UID.
        // Không xóa ngoại vi theo tên.
        // Một Pod cùng tên có thể đã được tạo lại.
        return fmt.Errorf(
            "pod %s/%s absent: cleanup needs recorded UID",
            namespace, name,
        )
    }

    // Pod vẫn tồn tại -> Điều hòa trạng thái bình thường
    _ = obj.(*corev1.Pod)
    return nil
}
```

Vì thế, đừng đặt external cleanup nguy hiểm sau nhánh `!exists` chỉ dựa vào key. Thiết kế production nên dùng finalizer khi object còn mang `metadata.uid`, hoặc một record durable đã liên kết UID với tài nguyên ngoài. Khi chỉ còn `namespace/name`, lựa chọn an toàn là không xóa và đưa tình huống vào luồng đối soát.

List/Watch, cache và workqueue cung cấp các contract khác nhau cho controller. Chương 23 thêm API riêng, ownership và finalizer để quyết định khi nào tài nguyên ngoài cụm được phép dọn dẹp.

Chương 23 — Từ controller đến operator: API riêng và vòng đời tài nguyên

Trong Chương 22, chúng ta đã nắm vững nền móng của một controller cấp thấp: dùng `client-go`, `SharedInformer` và `TypedRateLimitingInterface` để điều hòa các tài nguyên lõi như Pod. Nhưng khi hệ thống phát triển, nhu cầu quản trị hạ tầng vượt xa các khối cơ bản của Kubernetes.

Bạn không chỉ muốn quản lý Pod đơn lẻ; bạn muốn định nghĩa cả một hệ sinh thái: một cụm cơ sở dữ liệu phân tán tự động sao lưu định kỳ, một dịch vụ thanh toán tự động cấp phát tài nguyên trên đám mây, hoặc một ứng dụng nội bộ tự động cấu hình ingress, chứng chỉ SSL và tài khoản IAM.

Nếu làm theo cách thủ công, bạn phải bảo trì hàng trăm file YAML dài vô tận và phụ thuộc vào con người chạy lệnh `kubectl apply`. Đây chính là lúc khái niệm **Operator** ra đời: **đóng gói tri thức của một kỹ sư vận hành giàu kinh nghiệm (Domain Operational Knowledge) vào trong mã nguồn Go tự động.**

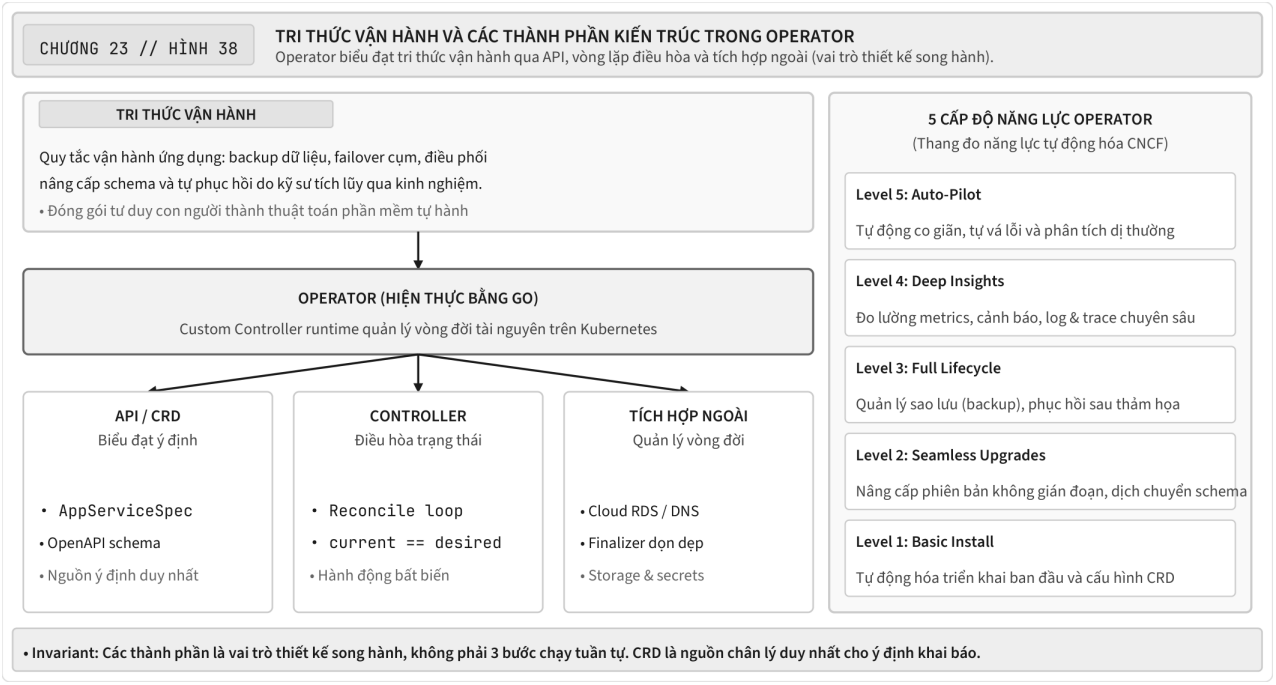
---

1. Operator là gì và Khi nào cần Operator?

Câu hỏi trung tâm của chương này là:

> **Sự khác biệt cốt lõi giữa một Kubernetes Controller thông thường và một Kubernetes Operator là gì?**

Operator dùng controller để tự động hóa vòng đời một ứng dụng hay domain cụ thể, thường cùng một CRD. Controller nói chung cũng có thể làm việc với custom resource; không có quy tắc rằng nó chỉ hiểu tài nguyên dựng sẵn. Điểm khác biệt hữu ích là tri thức vận hành mà reconciliation thể hiện, không một loại controller đặc biệt do Go hay Kubernetes compiler nhận diện.



Hình 38 — Operator biểu đạt tri thức vận hành qua API, vòng lặp điều hòa và các tích hợp quản lý tài nguyên. Các thành phần này là vai trò thiết kế, không phải ba bước chạy tuần tự.

Thay vì bắt người dùng phải tự tạo `Deployment`, gắn `ConfigMap`, tạo `Secret` và mở `Service`, Operator cung cấp một tài nguyên duy nhất gọn gàng:

```
apiVersion: apps.example.com/v1alpha1
kind: AppService
metadata:
  name: payment-api
spec:
  image: "payment:v2.4.0"
  replicas: 3
  port: 8080
```

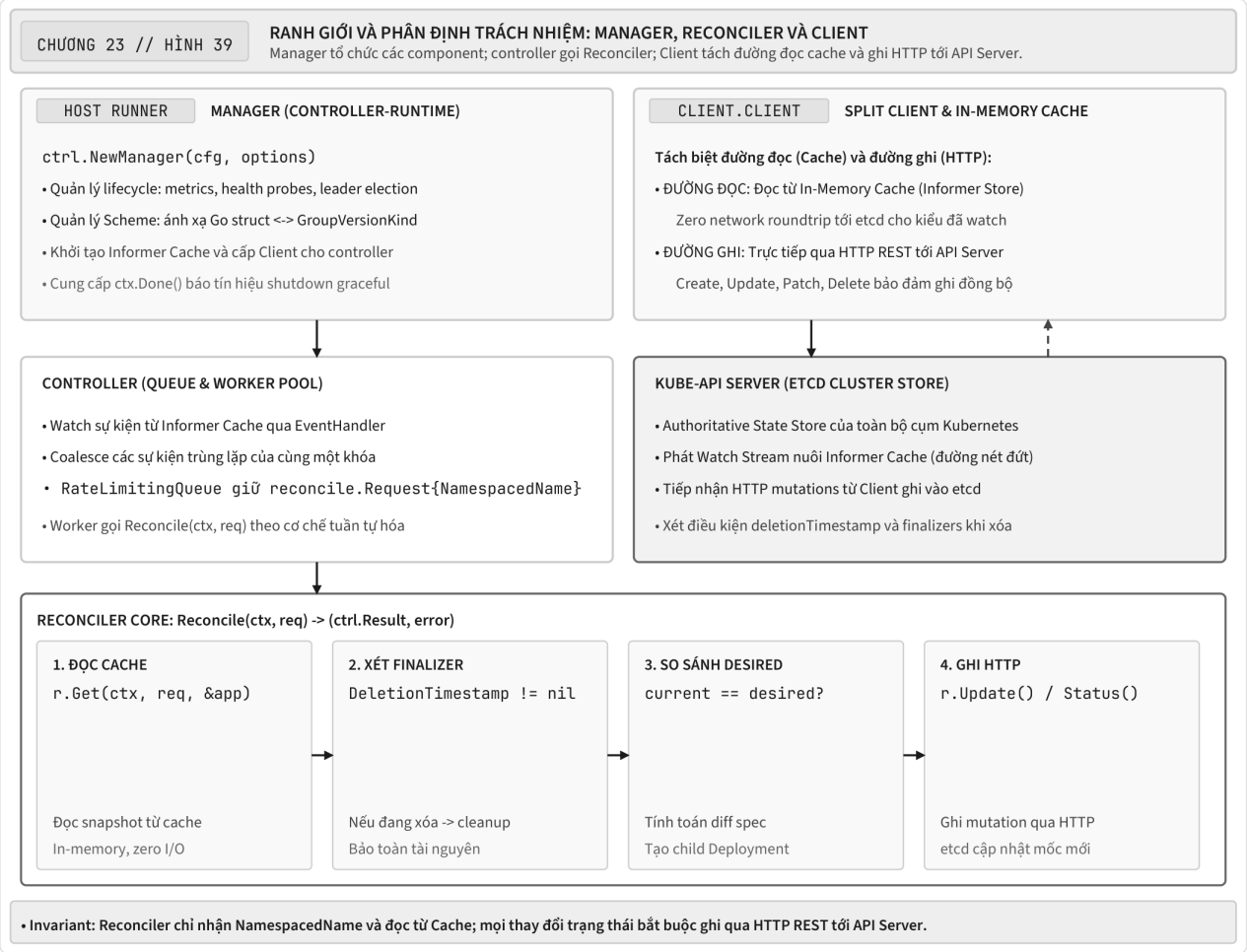
Khi bạn nộp khai báo trên, Operator sẽ tự động sinh ra Deployment tương ứng, gắn nhãn, cấu hình health check, theo dõi sức khỏe và cập nhật trạng thái thực tế về cho bạn.

---

2. Kiến trúc của controller-runtime

Viết Operator bằng `client-go` thuần túy đòi hỏi tự nối các phần quản lý cache, queue và lifecycle. Thư viện `sigs.k8s.io/controller-runtime` cung cấp lớp tổ chức chung cho các phần này; lượng code tiết kiệm phụ thuộc yêu cầu của operator, không có con số chung.

Sơ đồ dưới đây tách trách nhiệm quản lý tiến trình, xử lý khóa và truy cập tài nguyên:



Hình 39 — Manager tổ chức các component; controller gọi Reconciler cho một khóa. Client dùng cache cho đường đọc được cấu hình và HTTP cho đường ghi; sơ đồ lược bỏ các đường đọc không qua cache.

Bốn trụ cột của controller-runtime

Trụ cột `controller-runtime`	Vai trò kiến trúc	Tương tác trong hệ thống
`Manager`	Quản lý vòng đời tiến trình Operator	Quản lý bộ đệm chung, leader election, server metrics và probe.
`Scheme`	Đăng ký ánh xạ kiểu dữ liệu	Ánh xạ giữa Go struct (`v1alpha1.AppService`) và `GroupVersionKind`.
`Client` với cache	Phân tách đường đọc và ghi theo cấu hình	Get/List có thể đọc cache; kiểu bị DisableFor, metadata discovery hay cache chưa khởi tạo vẫn có đường HTTP. Writes đi tới API.
`Reconciler`	Hiện thực logic điều hòa cốt lõi	Nhận `ctrl.Request` chứa `NamespacedName`, hội tụ trạng thái.

---

3. Bản thiết kế CRD: Ranh giới rạch ròi giữa Spec và Status

Với API của lab, ta tách **Spec** — ý định người dùng — khỏi **Status** — điều controller quan sát. Đây là contract của API; quyền ghi thực tế còn phụ thuộc RBAC và subresource:

Khối dữ liệu	Ý nghĩa	Ai có quyền ghi?	Trách nhiệm
Spec	Trạng thái mong muốn (Desired State).	Người dùng, Helm, CI/CD pipeline, GitOps (ArgoCD).	Khai báo hệ thống cần đạt được điều gì.
Status	Trạng thái quan sát được.	Controller được policy giao trách nhiệm; RBAC quyết định quyền ghi thực tế.	Báo observation và generation đã quan sát.

Quy tắc chống trượt (Anti-Drift Rule)

> **Policy của lab:** Reconciler không sửa `Spec` để ghi observation hoặc tự ghi đè ý định người dùng. Observation đi vào `Status`; một controller có chức năng thay spec cần ownership và policy riêng.

Nếu người dùng cấu hình `replicas: 3`, mà Reconciler thấy tải cao rồi tự ý sửa `spec.replicas = 5`, hệ thống sẽ rơi vào cuộc chiến bất tận giữa Operator và công cụ GitOps (ArgoCD liên tục kéo về 3, Operator liên tục đẩy lên 5).

Mọi kết quả quan sát (số replica đang sẵn sàng, phiên bản thực tế đang chạy, điều kiện lỗi) bắt buộc phải ghi vào trường `Status`.

Phân lập Status Subresource

Để bảo vệ ranh giới này, Kubernetes cung cấp cơ chế **Status Subresource** (`/status` endpoint). Khi cập nhật trạng thái, bạn phải gọi:

```
// ĐÚNG: Chỉ sửa Status, không đổi metadata.generation
if err := r.Status().Update(ctx, appService); err != nil {
    return ctrl.Result{}, err
}
```

Với CRD bật status subresource, write vào resource chính bỏ qua thay đổi Status. Generation tăng khi phần dữ liệu thuộc quy tắc generation của CRD thay đổi, không phải cứ gọi Update là tăng. Một write tạo event cũng chưa chứng minh loop vô tận: predicate, nội dung thay đổi và logic reconcile đều ảnh hưởng. Đọc status qua đường đúng và xử lý conflict của write đó.

---

4. Quản lý tài nguyên con: OwnerReferences và Garbage Collection

Khi `AppService` sinh ra một `Deployment`, làm sao để Kubernetes biết hai đối tượng này có quan hệ cha - con?

Câu trả lời là **OwnerReference** (Tham chiếu chủ sở hữu). Thư viện `controller-runtime` cung cấp hàm chuẩn hóa:

```
if err := controllerutil.SetControllerReference(
    appService, newDeployment, r.Scheme,
); err != nil {
    return fmt.Errorf("không thể gán owner reference: %w", err)
}
```

OwnerReference giúp quản lý vòng đời thế nào

Thứ nhất là cascading deletion theo policy: garbage collector dùng ownerReferences hợp lệ để xét dependent, nhưng propagation policy, owner khác còn tồn tại và finalizer có thể giữ object. Lab gắn owner cho Deployment; không suy ra mọi tài nguyên con bị xóa ngay hay tài nguyên ngoài cluster được thu hồi. Về ranh giới namespace, OwnerReference là ownership metadata có quy tắc scope chặt chẽ chứ không phải liên kết cha con tùy ý. Đối với namespaced dependent, tài nguyên có thể tham chiếu owner namespaced trong cùng namespace hoặc owner cluster-scoped; nếu trỏ tới owner namespaced ở namespace khác, tham chiếu không hợp lệ và bị coi như owner không tồn tại, khiến dependent có thể bị thu hồi khi mọi owner hợp lệ khác không còn. Ngược lại, cluster-scoped dependent chỉ được phép tham chiếu owner cluster-scoped; nếu trỏ tới namespaced owner, tham chiếu trở nên không thể phân giải (unresolvable), từ Kubernetes v1.20+ garbage collector sẽ ghi nhận warning event OwnerRefInvalidNamespace và dependent không thể được thu hồi dựa trên tham chiếu đó.

Thứ hai là khả năng theo dõi sự kiện ngược dòng (Watch Events): Controller có thể cấu hình `Watches(&appsv1.Deployment{}, handler.EnqueueRequestForOwner(...))`. Bất cứ khi nào ai đó sửa đổi hoặc xóa Deployment con, sự kiện sẽ tự động ánh xạ ngược về `AppService` cha để Reconciler thức dậy sửa chữa.

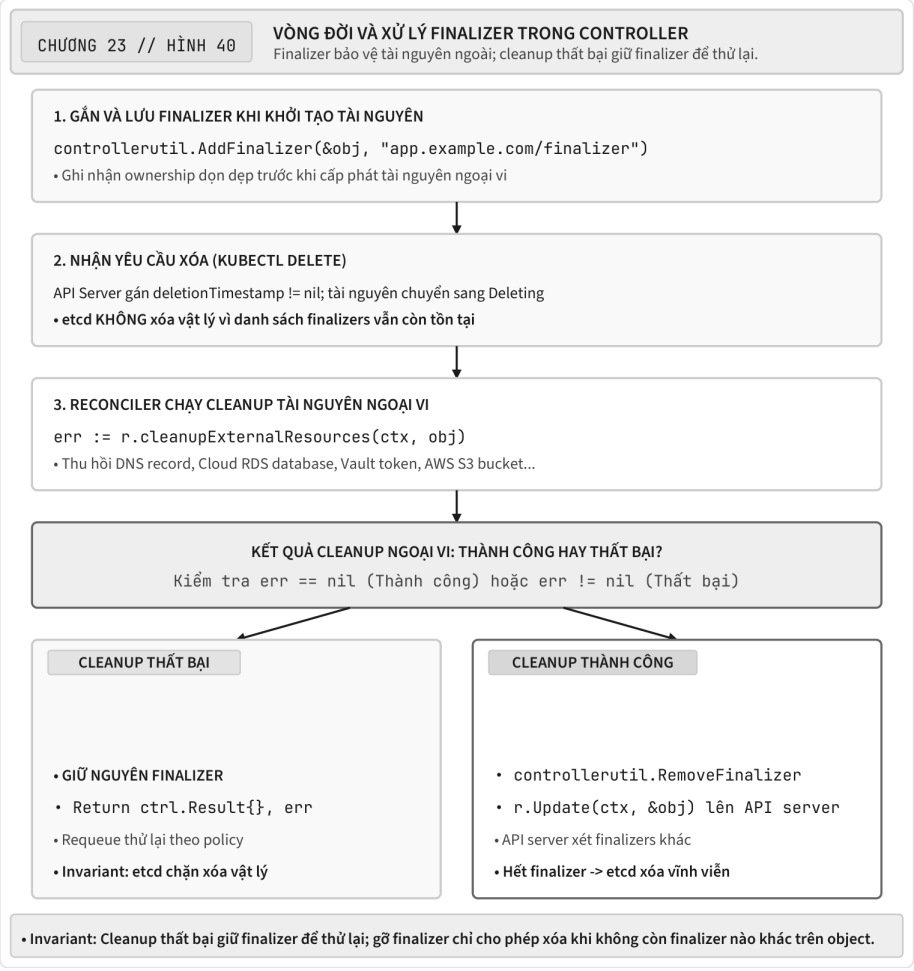
---

5. Finalizer: Cổng gác vòng đời và Dọn dẹp tài nguyên ngoại vi

Nếu `OwnerReference` giải quyết xuất sắc việc dọn dẹp các tài nguyên nội bộ trong Kubernetes, thì điều gì sẽ xảy ra nếu ứng dụng của bạn tạo ra các tài nguyên bên ngoài cụm như một cơ sở dữ liệu Amazon RDS, một bản ghi DNS trên Cloudflare, hay một hàng đợi tin nhắn AWS SQS?

Nếu đối tượng `AppService` bị xóa trước khi cleanup hoàn tất, controller có thể mất dữ liệu để xác định tài nguyên ngoài cụm. Trong scenario này, finalizer giữ object đủ lâu cho việc thu hồi; vẫn cần cơ chế điều tra hoặc đối soát để xử lý sự cố ngoài đường chạy bình thường. Không có số chi phí chung suy ra từ một object bị xóa.

Đây chính là sứ mệnh của **Finalizer** (Bộ chốt vòng đời).



Hình 40 — Cleanup thất bại giữ finalizer để có thể thử lại. Cleanup thành công chỉ cho phép gỡ finalizer của controller này; API server còn phải xét các điều kiện xóa khác.

API server chỉ có thể hoàn tất xóa khi các điều kiện lifecycle, gồm toàn bộ finalizer còn lại, cho phép. Gỡ finalizer của controller này không hứa object biến mất ngay; controller không trực tiếp gọi etcd để xóa.

Cạm bẫy Finalizer Deadlock

Cleanup cần an toàn khi gọi lại. Một response not-found có thể được coi là đã hoàn tất nếu API và identity chứng minh đúng tài nguyên đích đã mất; không bỏ qua mọi 404, vì nó cũng có thể liên quan endpoint, quyền hoặc identity sai.

Nếu coi mọi 404 khi cleanup là lỗi phải retry, controller có thể giữ finalizer dù resource ngoài đã mất. Object có thể kẹt `Terminating` cho tới khi logic được sửa hoặc có can thiệp. Chỉ coi not-found là cleanup thành công khi đã xác nhận đúng identity và contract API, không bỏ qua mọi lỗi ngoài bằng cùng một nhánh.

---

6. Hiện thực Operator hoàn chỉnh trong Go

Dưới đây là phần lõi Reconciler của `labs/part23-controller-runtime-operator/`:

Định nghĩa cấu trúc điều hòa với khả năng tiêm phụ thuộc (Dependency Injection) cho bộ dọn dẹp ngoại vi:

```
const AppServiceFinalizer = "apps.example.com/finalizer"

type ExternalCleaner interface {
    Cleanup(
        ctx context.Context, app *appsv1alpha1.AppService,
    ) error
}

type AppServiceReconciler struct {
    client.Client
    Scheme          *runtime.Scheme
    ExternalCleaner ExternalCleaner
}
```

Vòng lặp điều hòa trung tâm

Hàm `Reconcile` điều phối toàn bộ vòng đời: từ kiểm tra xóa, tạo tài nguyên con, đến sửa chữa sai lệch cấu hình:

```
func (r *AppServiceReconciler) Reconcile(
    ctx context.Context, req ctrl.Request,
) (ctrl.Result, error) {
    appService := &appsv1alpha1.AppService{}
    err := r.Get(ctx, req.NamespacedName, appService)
    if err != nil {
        if errors.IsNotFound(err) {
            return ctrl.Result{}, nil
        }
        return ctrl.Result{}, err
    }

    // 1. Xử lý Finalizer: Kiểm soát vòng đời xóa
    if !appService.DeletionTimestamp.IsZero() {
        return r.handleDeletion(ctx, appService)
    }

    // Đảm bảo đối tượng luôn có Finalizer bảo vệ
    if !controllerutil.ContainsFinalizer(
```

```
        appService, AppServiceFinalizer,
    ) {
        controllerutil.AddFinalizer(
            appService, AppServiceFinalizer,
        )
        return ctrl.Result{}, r.Update(ctx, appService)
    }

    // 2. Điều hòa Deployment con và sửa sai lệch (Drift)
    if err := r.reconcileDeployment(
        ctx, appService,
    ); err != nil {
        return ctrl.Result{}, err
    }

    // 3. Cập nhật Status Subresource
    return ctrl.Result{}, r.updateStatus(ctx, appService)
}
```

Ngữ nghĩa trả về của Result và Error trong controller-runtime

Trong `controller-runtime` v0.25.1, giá trị trả về của `Reconcile(ctx, req)` quyết định hành vi tiếp theo của hàng đợi theo bốn nhánh rành mạch:

Nhánh thứ nhất, trả về `(ctrl.Result{}, err)` với `err != nil`: Nếu lỗi không phải `reconcile.TerminalError`, controller ghi log lỗi và đưa lại request vào hàng đợi với thuật toán Rate Limiting (`AddRateLimited`). Nếu vô tình trả về đồng thời `err != nil` và cờ `Requeue` hoặc `RequeueAfter`, controller sẽ ghi warning log và bỏ qua hai cờ requeue để ưu tiên xử lý lỗi.

Nhánh thứ hai, trả về `(ctrl.Result{RequeueAfter: d}, nil)` với `d > 0`: Controller xóa lịch sử lỗi của request (`Forget`) và lên lịch đưa request trở lại hàng đợi sau khoảng thời gian `d` (`AddAfter`).

Nhánh thứ ba, trả về `(ctrl.Result{Requeue: true}, nil)`: Controller đưa request trở lại hàng đợi có kiểm soát tốc độ qua rate limiter. Trong các phiên bản controller-runtime mới, cách trả về này dần được thay thế bằng lỗi hoặc `RequeueAfter` tường minh.

Nhánh thứ tư, trả về `(ctrl.Result{}, nil)`: Điều hòa thành công. Controller xóa lịch sử lỗi (`Forget`), không xếp lại request, và worker chỉ thức dậy khi có sự kiện watch mới tác động lên tài nguyên.

Xử lý Xóa an toàn và Dọn dẹp ngoại vi

```
func (r *AppServiceReconciler) handleDeletion(
    ctx context.Context, app *appsv1alpha1.AppService,
) (ctrl.Result, error) {
    hasFin := controllerutil.ContainsFinalizer(
        app, AppServiceFinalizer,
    )
    if hasFin {
        if r.ExternalCleaner == nil {
            return ctrl.Result{}, fmt.Errorf(
                "external cleaner is required",
            )
        }
        err := r.ExternalCleaner.Cleanup(ctx, app)
        if err != nil {
            return ctrl.Result{}, fmt.Errorf(
                "cleanup lỗi: %w", err,
            )
        }
        // Dọn dẹp xong: gỡ finalizer của controller này
        controllerutil.RemoveFinalizer(
            app, AppServiceFinalizer,
        )
        if err := r.Update(ctx, app); err != nil {
            return ctrl.Result{}, err
        }
    }
    return ctrl.Result{}, nil
}
```

Tự động nhận diện và sửa chữa sai lệch (Drift Correction)

Nếu ai đó dùng `kubectl scale` để sửa số replica của Deployment con từ 3 xuống 1, Reconciler phát hiện sự không đồng nhất giữa `appService.Spec.Replicas` và `deployment.Spec.Replicas`, từ đó tự động ép trạng thái về đúng bản thiết kế ban đầu:

```
func (r *AppServiceReconciler) syncDeploymentDrift(
    app *appsv1alpha1.AppService, dep *appsv1.Deployment,
) bool {
    drifted := false
    if dep.Spec.Replicas == nil ||
        *dep.Spec.Replicas != app.Spec.Replicas {
        dep.Spec.Replicas = &app.Spec.Replicas
        drifted = true
    }
    if len(dep.Spec.Template.Spec.Containers) > 0 {
        c := &dep.Spec.Template.Spec.Containers[0]
        if c.Image != app.Spec.Image {
            c.Image = app.Spec.Image
            drifted = true
        }
    }
    return drifted
}
```

---

7. Bằng chứng kiểm thử: Chứng minh 5 quy luật Operator

Bộ kiểm thử tại `labs/part23-controller-runtime-operator/controllers/appservice_controller_test.go` dùng `fake.NewClientBuilder()` cho contract logic của reconciler. Đây không mô phỏng toàn diện API server: validation, generation/resourceVersion và nhiều semantics của subresource không giống cluster thật. Khi cần xác minh interaction với API server, dùng `envtest` hoặc cluster integration test riêng. Vì vậy lab vẫn ghi `ObservedGeneration` vào status để consumer biết status đã phản ánh spec nào, nhưng không dùng fake client để tuyên bố đã kiểm chứng đầy đủ lifecycle của generation.

```
=== RUN   TestReconcileCreateOwnedDeployment
--- PASS: TestReconcileCreateOwnedDeployment (0.51s)
=== RUN   TestReconcileDriftCorrection
--- PASS: TestReconcileDriftCorrection (0.01s)
=== RUN   TestReconcileFinalizerExecution
--- PASS: TestReconcileFinalizerExecution (0.01s)
=== RUN   TestReconcileStatusSubresource
--- PASS: TestReconcileStatusSubresource (0.01s)
=== RUN   TestNotFoundResourceNoOp
--- PASS: TestNotFoundResourceNoOp (0.01s)
PASS
ok      part23-controller-runtime-operator/controllers   1.875s
```

1. Khởi tạo tài nguyên con kèm OwnerReference (TestReconcileCreateOwnedDeployment)

Khi tạo mới một `AppService`, Reconciler tự động sinh một `Deployment` mang tên tương ứng. Bằng chứng kiểm thử xác nhận `dep.OwnerReferences[0].Name == "cache-service"`, và `app.Finalizers` lập tức được bổ sung khóa `apps.example.com/finalizer`.

2. Tự phục hồi sai lệch cấu hình (TestReconcileDriftCorrection)

Lab sửa desired deployment rồi gọi Reconcile để kiểm tra logic khôi phục những field controller sở hữu. Test không đo tốc độ hội tụ của cluster thật; cache, event delivery, retry và API error vẫn ảnh hưởng thời gian sửa drift.

3. Vòng đời Finalizer (TestReconcileFinalizerExecution)

Gán `DeletionTimestamp` vào đối tượng. Reconciler gọi `ExternalCleaner.Cleanup()` rồi mới gỡ Finalizer; thiếu cleaner là lỗi và finalizer phải giữ lại. Test chỉ chứng minh contract của lab; việc object thực sự biến mất là trách nhiệm API server/garbage collection và không được suy ra từ fake client.

4. Cập nhật Status Subresource độc lập (TestReconcileStatusSubresource)

Kiểm chứng rằng khi Deployment con đạt trạng thái sẵn sàng (`AvailableReplicas: 3`), Reconciler cập nhật `appService.Status.AvailableReplicas = 3` và gán nhãn `Phase = "Ready"` thông qua cổng `r.Status().Update()` mà không làm biến đổi bất kỳ trường nào trong `Spec`.

5. An toàn khi tài nguyên biến mất (TestNotFoundResourceNoOp)

Gửi yêu cầu điều hòa cho một tài nguyên không hề tồn tại. Reconciler trả về `(ctrl.Result{}, nil)` một cách nhẹ nhàng, không gây panic hay sinh lỗi rác trong log hệ thống.

Đặt cùng assertion trước API server thật

Fake client ở trên cho phép kiểm tra reconciler đã gọi đúng đường ghi và xử lý các object của fixture. Nó không phải một API server thu nhỏ. `TestFakeDoesNotProveGeneration` trong thư mục `integration` đổi spec mà generation vẫn là 1 trên fake của `controller-runtime v0.25.1`. Đừng sửa số ấy thành 2 bằng test helper rồi gọi đó là bằng chứng Kubernetes tự tăng generation.

Thử một contract cụ thể: status subresource phải tách đường ghi mong muốn khỏi đường ghi quan sát. Test `TestRealAPIContract` nạp CRD có structural schema và `subresources.status` vào một control plane `envtest` riêng. Nó không đọc kubeconfig của anh và không sửa cluster từ Chương 17. Sau create, status tự khai bị bỏ; root `Update` không ghi được status; `Status().Update` không đổi được spec. Spec update tăng generation từ 1 lên 2, còn observedGeneration vẫn 1 cho tới khi controller báo quan sát mới. Cuối cùng, status writer dùng resourceVersion cũ phải nhận conflict thay vì ghi đè observation mới.

```
# Từ module labs/part23-controller-runtime-operator
export KUBEBUILDER_ASSETS=/path/to/k8s-1.37-assets
go test -tags integration -count=1 -v ./integration
```

Đường thực thi này cần Linux/WSL và các binary `kube-apiserver`, `etcd` tương thích Kubernetes 1.37.x; dependency của lab đã ghim API v0.37.0. Môi trường Windows hiện tại chưa có assets đó, nên kiểm định API thật là `NOT_RUN`, không phải PASS. Test fake chạy được và việc biên dịch test cho Linux chỉ kiểm tra mã dùng đúng API thư viện. Khi tái lập trên Linux đủ prerequisite, hãy đọc output của chính test, không dùng kết quả compile làm chứng nhận integration.

Dừng trước đáp án. Nếu thay `Status().Update` bằng `Update`, test nào phải bắt được lỗi dù request không trả error? Chạy lại với `RELIABILITY_MUTANT=root_status_write` trên môi trường đủ assets; assertion của đường status phải đỏ. Đây là biến thể được chuẩn bị cho integration, chưa được ghi đã chạy ở máy thiếu control plane.

Đáp án. Root endpoint có thể chấp nhận request nhưng bỏ status, đồng thời chấp nhận spec mà code vô tình mang theo. Test đọc object lại bằng client không cache, kiểm tra cả phase, image và generation. Chỉ kiểm tra `err == nil` sẽ bỏ lọt việc ghi sai endpoint. Khi gặp conflict thật, reconciler phải đọc observation mới rồi tính lại thay đổi; ghi cùng object cũ nhiều lần không phải một cách sửa conflict.

`envtest` chạy API server và etcd, không chạy kubelet, scheduler hay garbage collector. Vì vậy test này không chứng minh Pod ready, owner reference đã thu hồi child hay external finalizer đã dọn cloud. Tách contract API khỏi vòng điều hòa và khỏi workload thật giúp ta biết chính xác bằng chứng còn thiếu ở đâu.

8. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Ghi observation vào Spec dù API quy định user sở hữu Spec.	Có thể xung đột với GitOps và gây reconcile lặp.	Giữ observation ở Status; xác định field ownership khi có chức năng sửa Spec.
Dùng `Update` cho Status khi CRD bật status subresource.	Status có thể bị bỏ qua; generation không phải cứ Update là tăng.	Dùng `Status().Update` cho status và kiểm tra conflict/observedGeneration theo API.
Cleanup coi mọi not-found là lỗi hoặc mọi 404 là thành công.	Có thể giữ finalizer không cần thiết hoặc bỏ cleanup sai đích.	Xác minh resource identity và contract API trước khi coi not-found là hoàn tất.
Không thiết kế ownership cho tài nguyên con.	Garbage collector không suy ra quan hệ cha/con từ tên; tài nguyên có thể còn lại khi cha bị xóa.	Nếu policy giao cleanup cho garbage collector, gắn owner reference hợp lệ; kiểm tra scope và deletion policy, không gắn owner tùy tiện cho tài nguyên chia sẻ.

---

9. Bài tập thực hành thiết kế Operator

Thử thách 1: Tự động khởi tạo và đồng bộ ConfigMap đi kèm Deployment

Yêu cầu: Mở rộng `AppServiceReconciler` để mỗi khi điều hòa một `AppService`, nó tự động tạo và đồng bộ một `ConfigMap` con (`<app-name>-config`) chứa file cấu hình ứng dụng `app.json` với nội dung `{"port": <Spec.Port>}`. `ConfigMap` này phải được gắn `OwnerReference` trỏ về `AppService`. Nếu `ConfigMap` đã tồn tại nhưng dữ liệu bị lệch (drift), reconciler phải cập nhật lại nội dung.

Thử thách 2: Báo cáo Condition chuẩn mực trong Status

Yêu cầu: Kubernetes khuyến nghị sử dụng slice `[]metav1.Condition` để thể hiện trạng thái chi tiết của tài nguyên. Hãy bổ sung hàm trợ giúp để cập nhật Condition `Type="DeploymentReady"` với `Status="True"` khi số lượng Pod sẵn sàng bằng số lượng replica mong muốn, hoặc `Status="False"` kèm Reason `"ReplicasUnavailable"` khi chưa đủ.

---

10. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Đồng bộ ConfigMap con và Xử lý Trôi cấu hình

```
func (r *AppServiceReconciler) reconcileConfigMap(
    ctx context.Context, app *appsv1alpha1.AppService,
) error {
    desiredJSON := fmt.Sprintf(`{"port": %d}`, app.Spec.Port)
    cm := &corev1.ConfigMap{
        ObjectMeta: metav1.ObjectMeta{
            Name:      app.Name + "-config",
            Namespace: app.Namespace,
        },
        Data: map[string]string{
            "app.json": desiredJSON,
        },
    }

    // Gán quan hệ cha con để garbage collector tự động dọn dẹp
    if err := controllerutil.SetControllerReference(
        app, cm, r.Scheme,
    ); err != nil {
        return err
    }

    found := &corev1.ConfigMap{}
    key := client.ObjectKeyFromObject(cm)
```

```
    err := r.Get(ctx, key, found)
    if errors.IsNotFound(err) {
        return r.Create(ctx, cm)
    }
    if err != nil {
        return err
    }

    // Kiểm tra quyền sở hữu, từ chối ghi đè nếu owner khác
    if !metav1.IsControlledBy(found, app) {
        return fmt.Errorf(
            "configmap %s exists but not owned by %s",
            found.Name, app.Name,
        )
    }

    // Phát hiện và sửa trôi cấu hình (configuration drift)
    if found.Data["app.json"] != cm.Data["app.json"] {
        found.Data = cm.Data
        return r.Update(ctx, found)
    }
    return nil
}
```

Phân tích ranh giới điều hòa (Reconcile Boundaries)

Trong một reconciler chuẩn mực, việc kiểm soát và bảo vệ ranh giới tài nguyên gồm ba kỷ luật cốt lõi:

Thứ nhất là kiểm tra quyền sở hữu (ownership verification). Khi `r.Get` tìm thấy ConfigMap đã tồn tại trên cụm, reconciler không được vội vàng cập nhật dữ liệu. Tài nguyên này có thể do một tiến trình khác tạo ra hoặc do người quản trị cấu hình thủ công. Hàm sử dụng `metav1.IsControlledBy(found, app)` để xác minh quyền điều khiển. Nếu ConfigMap không thuộc sở hữu của `AppService` hiện tại, chính sách an toàn nhất là từ chối can thiệp và trả về lỗi tường minh, tuyệt đối không ghi đè lên tài nguyên của chủ sở hữu khác.

Thứ hai là khắc phục trôi cấu hình (drift correction). Với ConfigMap đã xác thực đúng quyền sở hữu, việc chỉ kiểm tra `IsNotFound` khi khởi tạo ban

đầu là chưa đủ. Nếu ai đó vô tình chỉnh sửa ConfigMap trực tiếp hoặc trường `Spec.Port` của `AppService` được cập nhật sau đó, logic chỉ tạo mới sẽ bỏ sót sai lệch dữ liệu. Do đó, hàm điều hòa đối chiếu nội dung thực tế qua phép so sánh `found.Data["app.json"] != cm.Data["app.json"]`, rồi gọi `Update(ctx, found)` khi có khác biệt để bảo đảm tính nhất quán sau cùng (eventual consistency).

Thứ ba là ranh giới tác động đến tiến trình trong Pod (workload boundary). Cập nhật ConfigMap trên API server không làm cho Pod tự động khởi động lại, trừ khi ứng dụng tự thiết lập cơ chế theo dõi file trên đĩa để nạp lại. Nếu ứng dụng nạp cấu hình qua biến môi trường (`envFrom`) hoặc mount qua `subPath`, kubelet sẽ không tự đẩy dữ liệu mới vào container. Để giải quyết ranh giới này, bản thân ConfigMap thay đổi không thể tự động làm đổi Deployment; chính reconciler của Operator phải chủ động tính mã băm SHA-256 của chuỗi `app.json` mới rồi gán vào annotation của Pod template trong Deployment (chẳng hạn `app.kubernetes.io/config-hash: <sha256>`). Khi Operator cập nhật Deployment với template annotation mới, Deployment controller của Kubernetes mới nhận diện được sự thay đổi ở cấp độ Pod template và kích hoạt Rolling Update để thay thế toàn bộ Pod bằng phiên bản nạp cấu hình mới.

Lời giải Thử thách 2: Quản lý Condition bằng meta.SetStatusCondition

Sử dụng thư viện chuẩn `k8s.io/apimachinery/pkg/api/meta`:

```
import "k8s.io/apimachinery/pkg/api/meta"

func (r *AppServiceReconciler) updateConditions(
    app *appsv1alpha1.AppService, isReady bool,
) {
    condition := metav1.Condition{
        Type:               "DeploymentReady",
        Status:             metav1.ConditionFalse,
        Reason:             "ReplicasUnavailable",
        Message:            "Chưa đủ bản sao sẵn sàng",
        LastTransitionTime: metav1.Now(),
    }

    if isReady {
        condition.Status = metav1.ConditionTrue
        condition.Reason = "DeploymentAvailable"
        condition.Message = "Tất cả bản sao đều khỏe mạnh"
    }

    meta.SetStatusCondition(
        &app.Status.Conditions, condition,
    )
}
```

Hàm `meta.SetStatusCondition` tự động kiểm tra: nếu điều kiện đã tồn tại và không thay đổi trạng thái, nó sẽ giữ nguyên trường `LastTransitionTime` cũ, giúp hệ thống theo dõi chính xác thời điểm chuyển giao trạng thái.

Custom Resource, controller-runtime và finalizer cho ta các cơ chế để đọc và xây một operator nhỏ. Lab không chứng minh đã làm chủ mọi operator production; workload thật còn có schema evolution, ownership, retry, RBAC và recovery riêng. Chương 24 chuyển boundary ấy sang AWS SDK for Go v2 và credential tạm thời.

Chương 24 — Tự động hóa AWS bằng Go mà không biến credential thành bí mật dài hạn

Trong hành trình xây dựng các công cụ vận hành và nền tảng hạ tầng, giao tiếp với các dịch vụ điện toán đám mây là nhiệm vụ thiết yếu của kỹ sư DevOps/SRE. Cho dù bạn viết công cụ dọn dẹp snapshot định kỳ, sao lưu cơ sở dữ liệu lên Amazon S3, hay điều phối máy chủ qua EC2, bạn đều phải trả lời một câu hỏi bảo mật sống còn:

> **Làm thế nào để chương trình Go giao tiếp an toàn với AWS API mà không bao giờ nhúng Access Key dài hạn vào mã nguồn, file cấu hình hay biến môi trường?**

Để lộ cặp khóa `AWS_ACCESS_KEY_ID` và `AWS_SECRET_ACCESS_KEY` dài hạn trong repo hoặc log có thể cho phép sử dụng quyền của khóa cho đến khi bị thu hồi. Vì vậy chương này ưu tiên credential tạm thời và quyền tối thiểu; không gán thứ hạng nguyên nhân sự cố khi không có bộ dữ liệu tương ứng.

Chương 21 đến Chương 23 đã xây dựng mô hình điều hòa: quan sát trạng thái, tính sai lệch, hành động và quan sát lại để đảm bảo tính lũy đẳng trong Kubernetes. Khi mở rộng sang đám mây, các nguyên lý ấy vẫn giữ nguyên giá trị: việc tự động hóa AWS API (như dọn dẹp snapshot hay lưu trữ backup) cũng là một dạng điều hòa với hệ thống ngoại vi, nơi tính lũy đẳng nghiệp vụ, ranh giới lỗi và quyền hạn tạm thời quyết định độ tin cậy của hệ thống. Chương này trang bị cho bạn tư duy thiết kế hệ thống tự động hóa đám mây dựa trên thư viện chính thức AWS SDK for Go v2: từ cơ chế cấp quyền động ngắn hạn (Temporary Credentials), xác định Region, ngăn xếp middleware Smithy, ký chữ ký số SigV4, đến duyệt dữ liệu lớn qua Paginator và phân loại lỗi chuẩn mực.

---

1. Chuỗi định danh ngầm định và Quyền tạm thời

Ưu tiên production là thông tin xác thực tạm thời từ role/OIDC; điều đó không làm mọi nguồn trong default chain trở thành temporary credential. Với `config.LoadDefaultConfig`, các nguồn nền tảng gồm static credentials từ biến môi trường, shared config/credentials files, container credentials (ECS) và role credentials qua EC2 IMDS; cấu hình SSO hay web identity chọn provider tương ứng. Chain vẫn có thể nhận static credentials từ environment hoặc shared files, đặc biệt trên máy phát triển và trong test.

Các khóa tạm thời này có thời hạn hiệu lực hữu hạn: thời lượng phiên của IAM Role có thể cấu hình linh hoạt từ 15 phút tới 12 giờ tùy theo cấu hình vai trò, trong khi các cơ chế phiên liên kết (chained roles) hay `AssumeRoleWithWebIdentity` áp dụng giới hạn riêng. Mỗi bộ thông tin xác thực tạm thời luôn gắn liền với một Session Token (`AWS_SESSION_TOKEN`). Cần đặc biệt lưu ý: nếu khóa tạm thời bị rò rỉ trong thời gian còn hiệu lực, kẻ tấn công vẫn có thể sử dụng hợp lệ cho đến khi hết hạn hoặc cho đến khi quản trị viên chủ động thu hồi phiên (thông qua IAM revocation policy, cập nhật inline policy, hoặc vô hiệu hóa IAM role).



Hình 41 — Cấu hình quyết định provider được chọn; đây là bản đồ nguồn, không phải thứ tự ưu tiên bất biến. Credential có thể là static hoặc tạm thời. Cache lấy credential khi có lượt Retrieve cần dữ liệu hợp lệ.

Cơ chế bộ đệm và xác thực đồng bộ của SDK (`aws.CredentialsCache`)

Ở source SDK được ghim, credential resolution có nhánh theo cấu hình, không phải một danh sách ưu tiên bất biến: profile được chọn tường minh, environment credentials và web-identity token có các nhánh riêng; profile có thể dùng source profile, SSO hay credential process trước khi tới container/IMDS. Đọc `config/resolve_credentials.go` cùng options của caller khi cần giải thích vì sao một provider được chọn. Không suy ra credential tạm thời chỉ từ việc gọi `LoadDefaultConfig`.

Đối với các khối lượng công việc container hóa và điều phối đám mây, SDK phân định rõ ba cơ chế cấp phát định danh độc lập thay vì gộp chung mọi môi trường:

Một là, IAM Roles for Service Accounts (IRSA) trên Kubernetes: Pod gắn projected OIDC web identity token (qua biến

AWS_WEB_IDENTITY_TOKEN_FILE và AWS_ROLE_ARN), kích hoạt Web Identity provider gọi tới AWS STS thông qua API AssumeRoleWithWebIdentity.

Hai là, ECS Task Role: Container credential provider kết nối tới ECS agent credential endpoint cục bộ thông qua biến môi trường AWS_CONTAINER_CREDENTIALS_RELATIVE_URI.

Ba là, EKS Pod Identity: Container credential provider tương tác trực tiếp với EKS Pod Identity Agent trên node thông qua AWS_CONTAINER_CREDENTIALS_FULL_URI và token xác thực tại AWS_CONTAINER_AUTHORIZATION_TOKEN_FILE.

Nhằm giảm thiểu số lượt gọi mạng lặp lại trước mỗi HTTP request, SDK v2 bọc provider bên trong cấu trúc aws.CredentialsCache. Cần lưu ý rằng aws.CredentialsCache không cần một background goroutine riêng để định kỳ làm mới credentials. Lượt Retrieve(ctx) tiếp theo sẽ lấy lại credentials từ provider khi cache không còn hợp lệ. Cụ thể, mỗi khi mã nguồn gọi Retrieve(ctx), bộ đệm kiểm tra trực tiếp thời điểm hết hạn của khóa: nếu ExpiryWindow > 0, cache coi credentials hết hạn sớm hơn thời điểm hết hạn thực tế (effective expiration sớm hơn) để lượt Retrieve(ctx) tiếp theo làm mới chúng một cách đồng bộ; nếu ExpiryWindow <= 0, tùy chọn này bị bỏ qua. Nhờ cơ chế kiểm tra đồng bộ theo yêu cầu, CredentialsCache duy trì tính hợp lệ của phiên làm việc mà không cần duy trì tiến trình quét nền.

Xác định Region và Ranh giới Dịch vụ Vùng

Khi nạp cấu hình qua config.LoadDefaultConfig, việc xác định Region tuân theo chuỗi phân giải: cờ tùy chọn tường minh config.WithRegion("ap-southeast-1"), biến môi trường AWS_REGION hoặc AWS_DEFAULT_REGION, cấu hình profile trong file ~/.aws/config, và EC2 instance metadata (IMDS).

Khác với các dịch vụ mang tính toàn cầu (global services như IAM), hầu hết dịch vụ AWS (như Amazon S3, DynamoDB, EC2) đều vận hành theo từng region vật lý riêng biệt. Nếu không xác định được Region từ các nguồn cấu

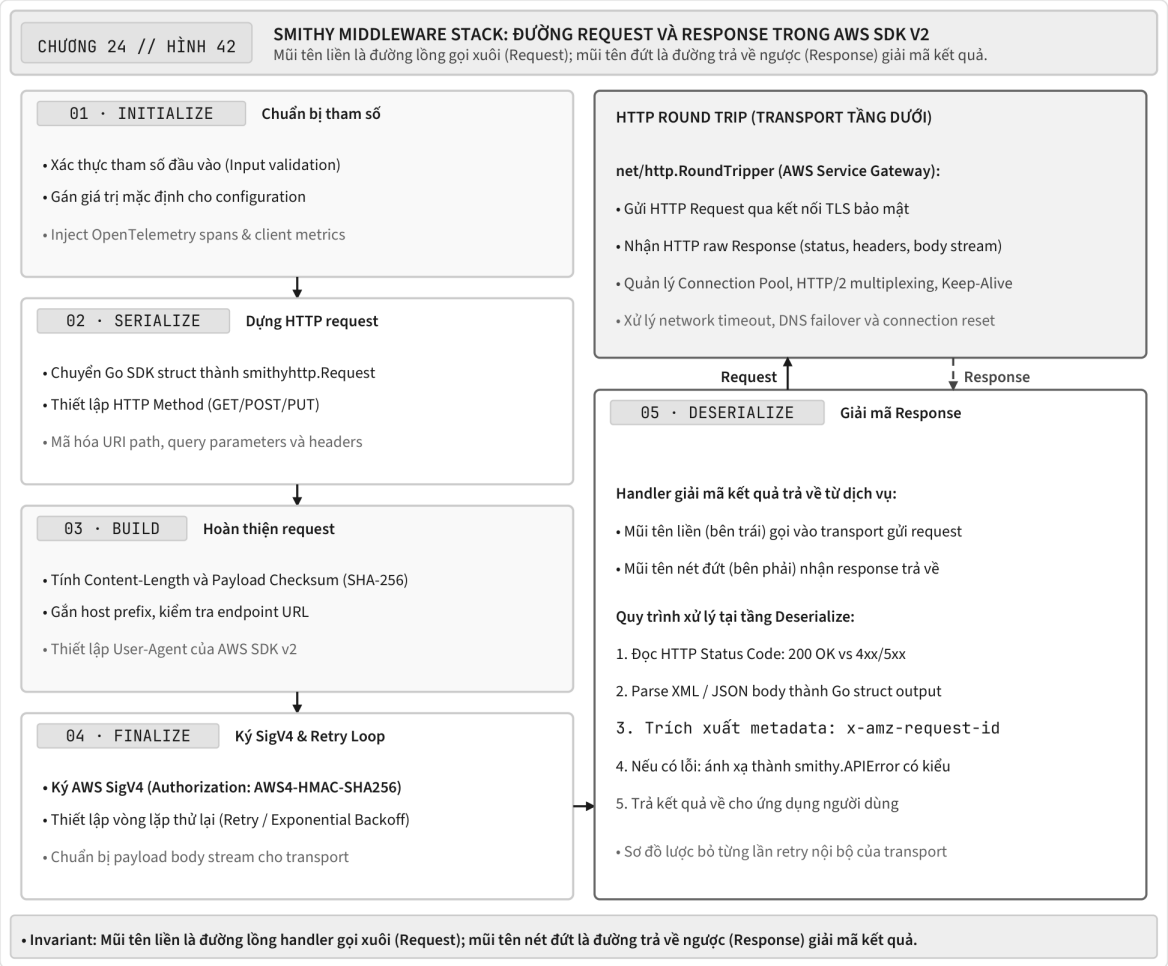
hình trên, SDK v2 sẽ trả về lỗi khi khởi tạo cuộc gọi đến dịch vụ vùng. Đặc biệt với Amazon S3, dù tên bucket là duy nhất trên phạm vi toàn cầu, mỗi bucket vẫn cư trú tại một region xác định; gửi request tới bucket ở region khác mà không cấu hình cross-region có thể dẫn đến lỗi điều hướng HTTP 301 hoặc lỗi ký chữ ký số SigV4.

---

2. Luồng thực thi của AWS SDK v2 và Smithy Middleware

AWS SDK for Go v2 dùng Smithy, một ngôn ngữ mô hình hóa service và bộ công cụ sinh SDK. Smithy không phải wire protocol thay cho HTTP hay SigV4; stack middleware là phần tổ chức đường gọi của SDK.

Một operation như `s3.PutObject` đi qua năm step của **Middleware Stack**. Các step gọi handler bên trong; response đi ngược ra ngoài. Vì vậy Deserialize gọi bước trao đổi HTTP trước khi giải mã response trả về:



Hình 42 — Mũi tên liền biểu diễn đường gọi vào handler, mũi tên đứt biểu diễn response trở về Deserialize. Middleware cụ thể và vị trí xử lý checksum phụ thuộc operation cùng cấu hình; sơ đồ không mô tả từng lần retry.

Chữ ký số SigV4 (Signature Version 4)

Trong đường gọi SigV4 đang xét, middleware ký ở pha **Finalize** xây canonical request từ phương thức HTTP, đường dẫn, query, các header được ký và giá trị payload hash theo chế độ signing. Đây là đại diện của request cho thuật toán xác thực, không phải chữ ký của từng packet mạng.

SDK dùng signing key suy ra từ secret, ngày, region và service để tạo chữ ký SigV4 cho canonical request. Server kiểm tra những phần được ký; không phải mọi header đều được đưa vào SignedHeaders và một số chế độ cho phép unsigned payload. Sửa phần được bao phủ có thể gây lỗi xác minh, nhưng không suy ra một mã lỗi HTTP cố định cho mọi service. TLS vẫn cần bảo vệ đường truyền; SigV4 không thay thế TLS.

SigV4 xác thực yêu cầu cho AWS và bảo vệ các phần được ký; nó không tự cung cấp bí mật nội dung, chống phát lại ở mọi ngữ cảnh, hay ký mọi

header/payload trong mọi lựa chọn dịch vụ. TLS vẫn bảo vệ kênh truyền, còn caller phải hiểu service và tùy chọn signing của lời gọi mình dùng.

---

3. Quản lý dữ liệu lớn bằng Paginator

Nếu chỉ gọi một trang của API liệt kê, công cụ có thể bỏ sót object ở các trang sau:

```
// CẢNH BÁO: Chỉ lấy tối đa 1.000 file đầu tiên!
resp, err := client.ListObjectsV2(ctx, &s3.ListObjectsV2Input{
    Bucket: aws.String("big-data-bucket"),
})
```

Một bucket trên S3 có thể chứa 10 triệu đối tượng. API `ListObjectsV2` mặc định chỉ trả về tối đa 1.000 file mỗi lần gọi và trả về mã thông báo tiếp tục (`NextContinuationToken`).

Nếu bạn tự viết vòng lặp `for` với token thủ công, mã nguồn sẽ trở nên cồng kềnh và dễ gặp lỗi vòng lặp vô tận nếu xử lý sai điều kiện ngắt.

Mẫu hình chuẩn: SDK Paginator

`NewListObjectsV2Paginator` quản lý continuation token và trả từng trang. Nó không hứa tổng bộ nhớ O(1) theo mọi input: kích thước trang và cách caller giữ kết quả vẫn phải có budget.

```
paginator := s3.NewListObjectsV2Paginator(
    client, &s3.ListObjectsV2Input{
        Bucket: aws.String("big-data-bucket"),
    },
)

for paginator.HasMorePages() {
    page, err := paginator.NextPage(ctx)
    if err != nil {
        return fmt.Errorf("lỗi đọc trang: %w", err)
    }
    for _, obj := range page.Contents {
        // Xử lý từng file mà không làm đầy RAM
        processFile(*obj.Key)
    }
}
```

Paginator chỉ giữ trang đang xử lý; mức nhớ của caller còn phụ thuộc xử lý của nó. `ListAllKeys` trong lab cố ý gom kết quả vào `[]string`, nên dùng O(n) theo tổng số key. Với scan lớn, xử lý từng `page.Contents` hoặc gọi callback thay vì tích lũy toàn bộ.

---

4. Phân loại lỗi và Thử lại: smithy.APIError

Khi một lệnh gọi AWS thất bại, bạn không thể chỉ so sánh chuỗi lỗi bằng `strings.Contains(err.Error(), "404")`. AWS trả về lỗi có cấu trúc chuẩn mực thông qua interface `smithy.APIError`.

Thử lại ở tầng transport của SDK (`aws.Retryer`, mặc định tối đa 3 lần với backoff lũy thừa và jitter cho lỗi tạm thời) không bảo đảm tính lũy đẳng nghiệp vụ (business idempotency). Nếu một yêu cầu ghi bị ngắt kết nối mạng sau khi máy chủ AWS đã tiếp nhận và thực thi, việc SDK tự động thử lại có thể gây ra tác dụng phụ lặp lại ngoài mong muốn. Một số API của AWS cung cấp token lũy đẳng tường minh (như `ClientToken` trong EC2) để máy chủ nhận diện yêu cầu lặp; nhưng với Amazon S3 `PutObject`, ngữ nghĩa phụ thuộc chặt chẽ vào cấu hình bucket và điều kiện tiền đề. Khi bucket bật Versioning, nhiều lượt PUT với cùng một key sẽ tạo ra các version riêng biệt

chứ không ghi đè tại chỗ, vì vậy việc dùng chung key không đảm bảo một tác dụng phụ duy nhất. Khi nghiệp vụ đòi hỏi tạo mới mà không ghi đè, hệ thống có thể cần yêu cầu có điều kiện (conditional request như `If-None-Match: *` khi dịch vụ hỗ trợ), nhưng đây không phải giải pháp vạn năng cho mọi tình huống. Tính lũy đẳng nghiệp vụ bắt buộc phải được thiết kế dựa trên đúng hợp đồng của từng thao tác và dịch vụ cụ thể. Trong policy của lab, `AccessDenied` không được retry như lỗi tạm thời. Các lỗi throttling hay service unavailable có thể cần backoff/jitter, nhưng retry còn phụ thuộc operation, budget và idempotency. Với timeout sau write, server có thể đã thực hiện side effect; không lặp mù quáng chỉ vì tên lỗi chứa `RequestTimeout`. Dùng classifier kết hợp với contract service và retryer đã cấu hình.

```
func ClassifyError(err error) (bool, string) {
    if err == nil {
        return false, ""
    }

    // 1. Kiểm tra lỗi nghiệp vụ cụ thể của dịch vụ S3
    var noSuchKey *types.NoSuchKey
    if errors.As(err, &noSuchKey) {
        return false, "NoSuchKey"
    }
    var noSuchBucket *types.NoSuchBucket
    if errors.As(err, &noSuchBucket) {
        return false, "NoSuchBucket"
    }

    // 2. Kiểm tra lỗi giao tiếp chung qua smithy.APIError
    var apiErr smithy.APIError
    if errors.As(err, &apiErr) {
        code := apiErr.ErrorCode()
        switch code {
        case "SlowDown", "ThrottlingException",
            "TooManyRequestsException", "RequestTimeout",
            "ServiceUnavailable":
            return true, code
        default:
            return false, code
        }
    }
    return false, "UnknownError"
}
```

---

5. Hiện thực Storage Client và Middleware tùy biến

Dưới đây là cấu trúc mã nguồn trích xuất từ dự án mẫu `labs/part24-aws-sdk-go-v2/`. Trong khi mã nguồn triển khai thực tế trên production sẽ sử dụng `config.LoadDefaultConfig(ctx)` để gọi trực tiếp tới các endpoint dịch vụ AWS thật, dự án lab minh họa kiến trúc bằng `httptest.Server` giả lập giao thức S3 nhằm kiểm chứng hành vi nội tại của SDK v2 (middleware, provider, paginator, phân loại lỗi) hoàn toàn cô lập, không yêu cầu tài khoản đám mây thật hay chi phí vận hành:

1. Provider cấp khóa tạm thời tự xoay vòng

Provider giả dưới đây trả credential có thời hạn để kiểm tra cache của SDK. Nó không gọi STS hay mô phỏng xác thực và phân quyền của STS:

```
type DynamicCredentialProvider struct {
    retrieveCount atomic.Int64
    ttl           time.Duration
    roleArn       string
}

func (p *DynamicCredentialProvider) Retrieve(
    ctx context.Context,
) (aws.Credentials, error) {
    count := p.retrieveCount.Add(1)
    now := time.Now()

    return aws.Credentials{
        AccessKeyID:     fmt.Sprintf("ASIA-TEMP-KEY-%d", count),
        SecretAccessKey: fmt.Sprintf("SECRET-TOKEN-%d", count),
        SessionToken:    fmt.Sprintf("SESSION-TOKEN-%d", count),
        Source:          "DynamicCredentialProvider",
        CanExpire:       true,
        Expires:         now.Add(p.ttl),
    }, nil
}
```

2. Can thiệp vào Smithy Middleware để thêm Header Audit

Để kiểm tra việc chèn một header của ứng dụng vào outbound request, ta viết middleware ở pha `Build`. Test chỉ kiểm tra header tại server cục bộ; không suy ra CloudTrail sẽ lưu custom header ấy:

```
type AuditHeaderMiddleware struct {
    Origin string
}

func (m *AuditHeaderMiddleware) ID() string {
    return "AuditHeaderMiddleware"
}

func (m *AuditHeaderMiddleware) HandleBuild(
    ctx context.Context,
    in middleware.BuildInput,
    next middleware.BuildHandler,
) (middleware.BuildOutput, middleware.Metadata, error) {
    req, ok := in.Request.(*smithyhttp.Request)
    if ok && req != nil {
        req.Header.Set("X-Audit-Origin", m.Origin)
    }
    return next.HandleBuild(ctx, in)
}
```

Khi khởi tạo client, ta đăng ký middleware này vào pipeline:

```
s3Client := s3.NewFromConfig(cfg, func(o *s3.Options) {
    o.APIOptions = append(
        o.APIOptions,
        func(stack *middleware.Stack) error {
            return stack.Build.Add(
                &AuditHeaderMiddleware{
                    Origin: "automated-backup-worker",
                },
                middleware.After,
            )
        },
    )
})
```

---

6. Bằng chứng kiểm thử: Kiểm chứng 5 hành vi then chốt

Toàn bộ bộ kiểm thử tại `labs/part24-aws-sdk-go-v2/storage_test.go` vận hành độc lập bằng `httptest.Server` (phân loại mức kiểm chứng: `MOCK_VERIFIED` đối với tầng HTTP transport và `UNIT_TESTED` đối với logic provider/paginator/middleware), mô phỏng cấu trúc phản hồi XML của S3 và kiểm chứng 5 hành vi kiến trúc then chốt mà không cần tài nguyên AWS thực tế:

```
=== RUN   TestCredentialsCacheReuseAndExpiredRefresh
--- PASS: TestCredentialsCacheReuseAndExpiredRefresh (0.11s)
=== RUN   TestCustomSmithyMiddlewareAuditHeader
--- PASS: TestCustomSmithyMiddlewareAuditHeader (0.01s)
=== RUN   TestS3OperationsPutAndGet
--- PASS: TestS3OperationsPutAndGet (0.00s)
=== RUN   TestPaginatorListAllKeys
--- PASS: TestPaginatorListAllKeys (0.00s)
=== RUN   TestErrorClassification
--- PASS: TestErrorClassification (0.00s)
PASS
ok      part24-aws-sdk-go-v2   2.054s
```

1. Tự động làm mới khóa hết hạn (TestCredentialsCacheReuseAndExpiredRefresh)

Test bọc provider giả lập có TTL 100 ms trong `aws.NewCredentialsCache` với cấu hình mặc định. Ở lần gọi thứ hai khi credential còn hạn, cache trả về ngay kết quả đã lưu mà không gọi lại provider. Chỉ sau khi credential hết hạn, lần gọi tiếp theo mới kích hoạt cache gọi lại provider để cấp phát credential mới. Đây là unit test cục bộ với fake provider nhằm kiểm chứng cơ chế bộ đệm của SDK; test không gửi request tới AWS STS thật và không chứng minh credential được refresh thành công khi mạng hay dịch vụ IAM từ xa gặp sự cố.

2. Tiêm Header qua Smithy Middleware (TestCustomSmithyMiddlewareAuditHeader)

Máy chủ HTTP cục bộ bắt gói tin `PutObject` và kiểm tra header `X-Audit-Origin`. Giá trị nhận được chính xác là `"automated-backup-worker"`, chứng minh middleware đã can thiệp thành công vào luồng gửi tin của SDK.

3. Đọc ghi luồng dữ liệu an toàn (TestS3OperationsPutAndGet)

Kiểm chứng tính toàn vẹn của chuỗi dữ liệu nhị phân khi upload và download qua S3 Client.

4. Thu thập toàn bộ danh sách file qua Paginator (TestPaginatorListAllKeys)

Máy chủ giả lập trả về trang 1 kèm `IsTruncated: true` và `NextContinuationToken`, sau đó trả về trang 2 với `IsTruncated: false`. Hàm `ListAllKeys` tự động gọi 2 lần HTTP và thu thập đầy đủ 3 file mà không cần người dùng tự quản lý token phân trang.

5. Phân loại lỗi chính xác (TestErrorClassification)

Kiểm chứng hàm phân loại bóc tách chính xác các lỗi nghiệp vụ mô hình hóa của S3 (`types.NoSuchKey` và `types.NoSuchBucket`) thông qua `errors.As` (đều xác định `isRetryable = false`), đồng thời phân biệt chính xác mã lỗi tạm thời `SlowDown` (đánh dấu `isRetryable = true`) và lỗi phân quyền `AccessDenied` (đánh dấu `isRetryable = false`) của interface `smithy.APIError`.

---

7. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Hardcode Access Key trong mã nguồn hoặc image.	Có thể lộ credential và các quyền credential cho phép.	Chọn provider credential tạm thời phù hợp môi trường; `LoadDefaultConfig` còn có thể chọn static credential nếu cấu hình để nó làm vậy.
Không đóng Response Body khi gọi `s3.GetObject`.	Body và tài nguyên Transport có thể bị giữ lại, làm tăng áp lực tài nguyên khi lặp nhiều request.	Đặt `defer resp.Body.Close()` sau khi `GetObject` thành công; việc tái sử dụng kết nối còn có điều kiện riêng.
Tự viết vòng phân trang mà không xử lý token và ngân sách.	Có thể lặp lại trang, bỏ sót dữ liệu hoặc giữ quá nhiều object.	Ưu tiên paginator của SDK; vẫn kiểm tra context, error, số trang và lượng dữ liệu giữ lại.
Thử lại mù quáng với lỗi phân quyền `AccessDenied`.	Làm tắc nghẽn hàng đợi và kích hoạt các cảnh báo bảo mật bất thường trong SIEM.	Dùng `errors.As(err, &apiErr)` để chỉ thử lại các lỗi tạm thời (`SlowDown`, `5xx`).

---

8. Bài tập thực hành thiết kế Cloud Tool

Thử thách 1: Tích hợp Context Timeout chặt chẽ cho thao tác đám mây

Yêu cầu: Viết `PutObjectWithTimeout` nhận `timeout time.Duration` và truyền context có deadline tới SDK. Test với server giả chậm để quan sát caller ngừng chờ trên đường chạy ấy. Cancellation không chứng minh tác động phía server bị rollback hoặc resource được giải phóng tức thời ở mọi tầng.

Thử thách 2: Thiết kế Waiter kiểm tra Bucket sẵn sàng

Yêu cầu: Sau `CreateBucket`, dùng `s3.NewBucketExistsWaiter` để chờ điều kiện tồn tại được quan sát qua `HeadBucket`. Đặt thời gian chờ tối đa 30 giây, khoảng lùi từ 2 đến 5 giây. Đây không phải phép kiểm chứng DNS toàn cầu hay toàn bộ độ sẵn sàng của ứng dụng.

---

9. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Quản trị thời gian chết bằng Context

```
func (s *CloudStorage) PutObjectWithTimeout(
    parentCtx context.Context,
    bucket, key string,
    body io.Reader,
    timeout time.Duration,
) error {
    ctx, cancel := context.WithTimeout(parentCtx, timeout)
    defer cancel()

    err := s.PutObject(ctx, bucket, key, body)
    if err != nil {
        if errors.Is(ctx.Err(), context.DeadlineExceeded) {
            return fmt.Errorf(
                "upload %s/%s quá hạn sau %v: %w",
                bucket, key, timeout, err,
            )
        }
        return err
    }
    return nil
}
```

Lời giải Thử thách 2: Đồng bộ hóa bằng SDK Waiter

```
func WaitForBucketReady(
    ctx context.Context,
    client *s3.Client,
    bucket string,
) error {
    waiter := s3.NewBucketExistsWaiter(
        client,
        func(o *s3.BucketExistsWaiterOptions) {
            o.MaxDelay = 5 * time.Second
            o.MinDelay = 2 * time.Second
        },
    )

    // Chờ tối đa 30 giây
    maxWait := 30 * time.Second
    return waiter.Wait(
        ctx,
        &s3.HeadBucketInput{Bucket: aws.String(bucket)},
        maxWait,
    )
}
```

`BucketExistsWaiter` lặp `HeadBucket` theo policy waiter của SDK cho đến khi đạt trạng thái chấp nhận được, lỗi hoặc hết `maxWait`. Nó chỉ chứng minh điều kiện tồn tại bucket; không chứng minh DNS, IAM policy hay ứng dụng đã sẵn sàng cho một workflow lớn hơn. Vì vậy timeout và điều kiện chấp nhận phải bám đúng operation đang chờ.

Credential provider, middleware và paginator giải quyết các boundary khác nhau. Trước khi dùng trên AWS thật, cần kiểm tra role, IAM policy, budget retry và side effect của operation. Chương 25 chuyển sang webhook và GitHub API, nơi identity của delivery cũng không đồng nghĩa identity của một thao tác nghiệp vụ.

Chương 25 — Git và GitHub dưới góc nhìn của một hệ thống tự động hóa

Trong workflow GitOps, Git có thể là nguồn khai báo desired state. Nó không chứa toàn bộ state production: database, secret ngoài repo, status của controller và observation vận hành vẫn có owner riêng. Chương này xét cách một bot đọc source revision và metadata GitHub mà không trộn hai loại dữ liệu ấy.

Một commit được merge vào nhánh `main` có thể tự động kích hoạt ArgoCD triển khai ứng dụng lên Kubernetes. Một nhãn (label) được gắn vào Pull Request có thể kích hoạt bot tự động cấp phát môi trường kiểm thử tạm thời (Ephemeral Environment) trên AWS.

Chương này xét hai lỗi thiết kế bằng ví dụ: đánh đồng object Git cục bộ với dữ liệu GitHub qua API, và xử lý webhook mà không xét authentication, duplicate delivery hay crash window. Không cần giả định mọi kỹ sư đều mắc chúng để kiểm tra boundary.

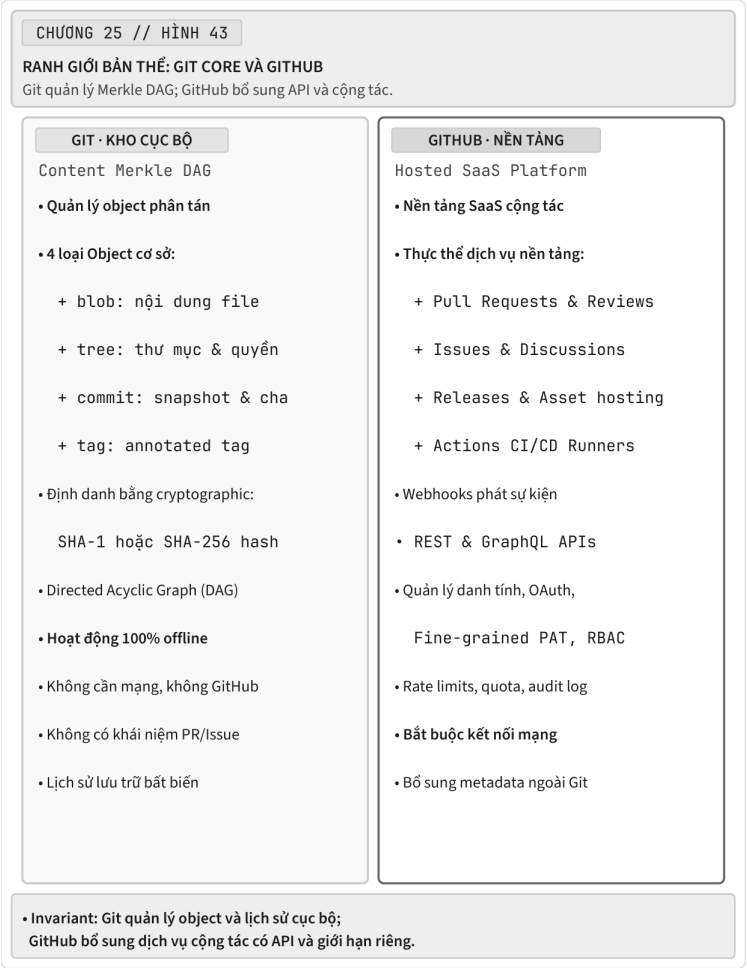
Chương này phân tích sâu bản chất kỹ thuật của Git và GitHub dưới góc nhìn của một hệ thống hướng sự kiện (Event-Driven System), từ đó hướng dẫn bạn xây dựng các bot tự động hóa chuẩn mực bằng Go.

---

1. Hai thế giới: Trạng thái Git cục bộ và Trạng thái GitHub

Câu hỏi trung tâm của chương này là:

> **Vì sao một hệ thống tự động hóa tin cậy phải tách bạch hoàn toàn giữa Trạng thái Git (Local Git State) và Trạng thái GitHub (GitHub Hosted State)?**



Hình 43 — Git quản lý object và lịch sử trong kho; GitHub bổ sung các dịch vụ cộng tác có API, quyền truy cập và giới hạn riêng.

Hai mô hình dữ liệu song hành

Git cục bộ lưu commit, tree và blob trong đồ thị định danh nội dung. Đọc object đã có có thể không cần mạng, nhưng storage có thể là disk hoặc memory, còn clone/fetch cần transport tới remote. Không có latency microsecond chung cho mọi thao tác go-git; số liệu cần workload và storage cụ thể. GitHub metadata như pull request và review là một boundary khác, không nằm trong tree source của commit.

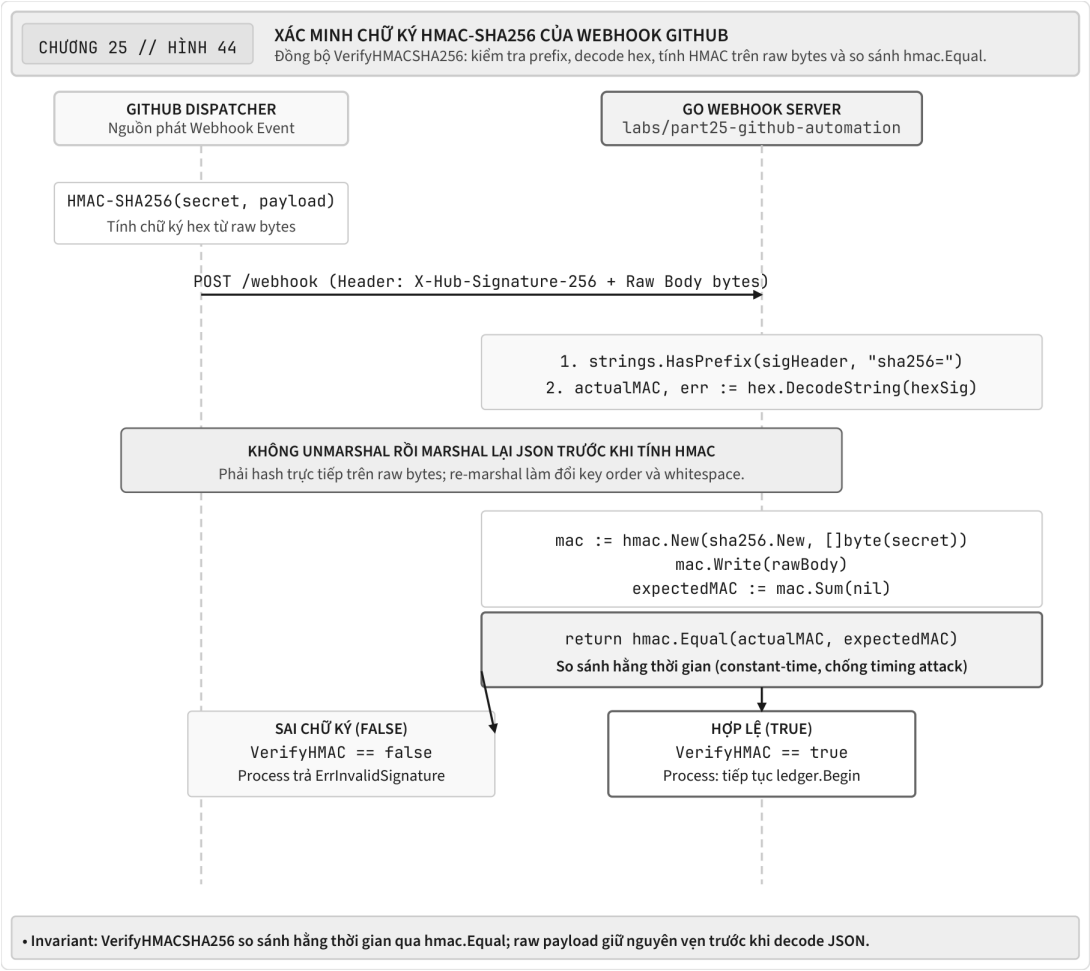
Metadata Pull Request, label và review không nằm trong Git tree của commit. Lab dùng `go-github/v92` cho REST API và `go-git/v5` cho object Git cục bộ. Một bot chỉ cần một trong hai cũng hợp lệ; chọn theo dữ liệu và thao tác thực sự cần, không theo một kiến trúc "toàn diện" bắt buộc.

---

2. Bảo mật Webhook: Phòng ngừa Timing Side-Channel bằng HMAC-SHA256

Khi GitHub gửi một sự kiện (ví dụ `pull_request.opened`) đến máy chủ của bạn qua HTTP Webhook, làm sao bạn biết chắc chắn thông điệp này thực sự do GitHub phát ra chứ không phải do một kẻ xấu giả mạo?

GitHub ký toàn bộ nội dung gói tin (raw payload) bằng mã khóa bí mật (Webhook Secret) thông qua thuật toán HMAC-SHA256, và đính kèm vào header: `X-Hub-Signature-256: sha256=<hex_digest>`



Hình 44 — Server tính chữ ký từ chính raw body đã nhận rồi so sánh với header. Không tuần tự hóa lại JSON trước khi kiểm tra chữ ký.

Rủi ro rò rỉ thông tin qua thời gian thực thi (Timing Side-Channel)

Nếu bạn dùng toán tử thông thường để so sánh chữ ký:

```
// CẢNH BÁO: Rò rỉ thông tin qua thời gian thực thi!
if actualSig == expectedSig { ... }
```

Toán tử so sánh chuỗi hoặc mảng byte mặc định trong hầu hết ngôn ngữ lập trình thường kết thúc sớm (early exit) ngay khi phát hiện byte không trùng khớp đầu tiên. Sự chênh lệch thời gian xử lý này vô tình tạo ra một kênh phụ (side-channel). Dù độ trễ biến thiên trên mạng Internet công cộng có thể làm nhiễu tín hiệu đo đạc, trong các môi trường nội bộ, mạng cục bộ tốc độ cao hoặc qua phương pháp phân tích thống kê trên lượng mẫu thử lớn, việc so sánh không hằng số thời gian vẫn có thể cung cấp manh mối để kẻ tấn công thu hẹp không gian tìm kiếm mã xác thực.

So sánh chữ ký an toàn với crypto/hmac.Equal hoặc crypto/subtle

Trong Go, bạn bắt buộc phải dùng hàm `hmac.Equal` hoặc `subtle.ConstantTimeCompare`:

```
func VerifyHMACSHA256(
    payload []byte, signatureHeader string, secret []byte,
) bool {
    if !strings.HasPrefix(signatureHeader, "sha256=") {
        return false
    }
    actualHex := strings.TrimPrefix(signatureHeader, "sha256=")
    actualSig, err := hex.DecodeString(actualHex)
    if err != nil {
        return false
    }

    mac := hmac.New(sha256.New, secret)
    mac.Write(payload)
    expectedSig := mac.Sum(nil)

    // So sánh hằng số thời gian (constant-time)
    return hmac.Equal(actualSig, expectedSig)
}
```

`hmac.Equal` so hai MAC mà không rò rỉ nội dung qua thời gian so sánh theo API contract. Nó không chứng minh toàn bộ endpoint constant-time

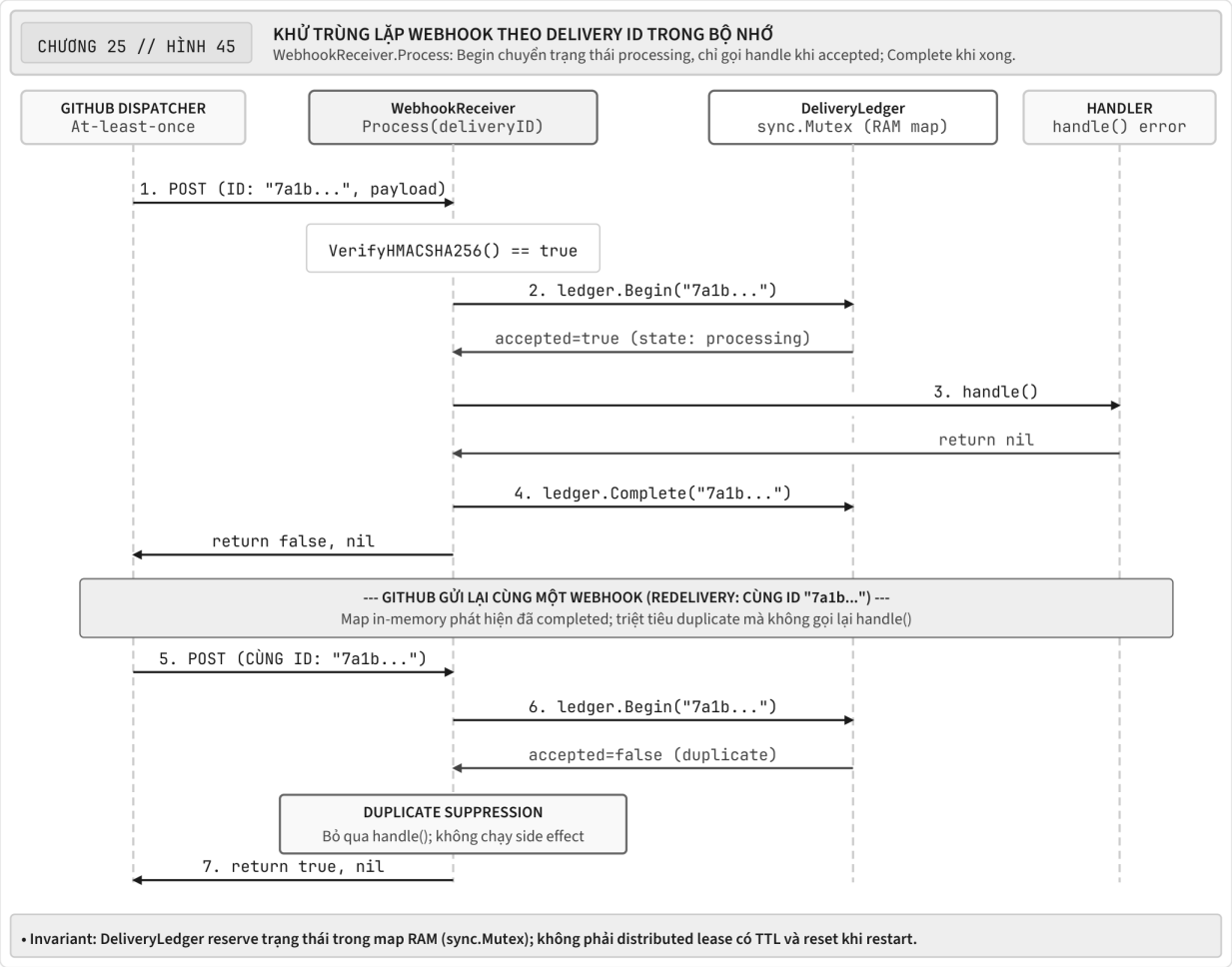
hay không có side-channel; caller vẫn cần giới hạn payload, xử lý chữ ký và bảo vệ secret.

---

3. Khế ước phân tán: Delivery ID và Tính Lũy Đẳng

Trong kiến trúc tích hợp webhook, một ngộ nhận phổ biến là cho rằng GitHub sẽ tự động thử lại (auto-retry) mỗi khi máy chủ nhận trả về mã lỗi 5xx hoặc bị timeout. Trên thực tế, trong các cấu hình webhook tiêu chuẩn, GitHub không tự động redeliver khi endpoint nhận bị lỗi. Việc redelivery chỉ xảy ra khi có thao tác kích hoạt lại từ người vận hành qua giao diện quản trị (Operator UI), lời gọi REST API redeliver từ mã nguồn tự động hóa, hoặc các công cụ nội bộ gọi API này. Khi một lần chuyển giao được kích hoạt lại (redelivered), GitHub giữ nguyên giá trị GUID trong header `X-GitHub-Delivery`.

Mỗi lần phát sự kiện, GitHub đính kèm một mã UUID duy nhất tại header: `X-GitHub-Delivery: 7a1b2c3d-4e5f-6a7b-8c9d-0e1f2a3b4c5d`



Hình 45 — Lab nhận diện cùng Delivery ID để tránh lặp side effect trong đường chạy được kiểm thử. Lưu bền, xử lý crash và delivery đồng thời cần policy cùng cơ chế riêng.

Delivery ID giúp nhận diện cùng một delivery khi redeliver. Nó không được HMAC payload ký kèm như một phần header, nên riêng việc lưu ID chưa chống replay từ bên giữ payload/chữ ký hợp lệ rồi đổi ID. Sau xác thực, cần ràng buộc event, repository và operation identity; state xử lý phải bền vững nếu muốn giữ kết quả qua crash.

Delivery ID chỉ khử trùng lặp **một lần chuyển giao**; nó không tự là khóa lũy đẳng của nghiệp vụ như “deploy commit X vào môi trường Y”. Cần có state machine tối thiểu `processing → completed`: redelivery đang xử lý bị giữ lại, attempt lỗi phải được đánh fail để có thể thử lại, còn completed mới bị suppress. Business mutation cần operation key riêng và, ở production, durable/transactional store có TTL. Lab giữ ledger trong RAM nên mọi bảo đảm chỉ tồn tại trong process lifetime.

Trên môi trường production thực tế, một hệ thống tự động hóa chịu lỗi đòi hỏi phải kết hợp một trong các cơ chế sau:

Một là, kho lưu trữ phân tán bền bỉ (Durable idempotency store): Lưu trữ delivery ID vào kho dữ liệu phân tán có TTL (như Redis hoặc DynamoDB).

Hai là, ràng buộc duy nhất trong cơ sở dữ liệu (Database uniqueness/transaction): Ghi delivery ID vào bảng sự kiện với ràng buộc khóa duy nhất (unique constraint) trong cùng transaction xử lý nghiệp vụ.

Ba là, mã định danh thao tác (Operation identity): Sinh token lũy đẳng gắn với trạng thái cụ thể của tài nguyên đích thay vì chỉ dựa vào sự kiện webhook.

Bốn là, thiết kế mutation lũy đẳng theo identity và trạng thái đã quan sát. Khai báo desired state giúp xác định đích nhưng không tự làm mọi thao tác create/delete lũy đẳng; precondition, ownership và retry sau partial failure vẫn cần được thiết kế như Ch21–23.

Thiết kế bộ tiếp nhận có kiểm tra trùng lặp trong bộ nhớ:

```
type WebhookReceiver struct {
    secret []byte
    ledger *DeliveryLedger
}

func (r *WebhookReceiver) Process(
    deliveryID, signature string,
    payload []byte,
    handle func() error,
) (bool, error) {
    if !VerifyHMACSHA256(payload, signature, r.secret) {
        return false, ErrInvalidSignature
    }
    if _, accepted := r.ledger.Begin(deliveryID); !accepted {
        return true, nil
    }
    if err := handle(); err != nil {
        r.ledger.Fail(deliveryID)
        return false, err
    }
    r.ledger.Complete(deliveryID)
    return false, nil
}
```

`Complete` xảy ra sau `handle`; lỗi giải phóng reservation để thử lại. Trong process còn sống, ledger của lab ngăn delivery đồng thời hoặc đã hoàn tất chạy lại handler. Production cần state bền, atomic claim và recovery; TTL/lease đơn lẻ không đóng crash window giữa side effect ngoài database và việc ghi Complete.

Ledger trong RAM không sống qua lần khởi động lại

Giữ nguyên lab webhook ở trên để kiểm tra HMAC và cạnh tranh trong một process. Bây giờ thử đưa nó qua khe lỗi của Chương 13: handler đã tạo một thay đổi, nhưng process chết trước `Complete`. Sau restart, ledger RAM trống; delivery giống hệt có thể đi qua lần nữa. Đây không phải lỗi của mutex hay HMAC. Mutex bảo vệ state đang sống, còn chữ ký xác nhận payload; không thứ nào lưu nghĩa vụ nghiệp vụ sau khi process biến mất.

Trong lab outbox của Chương 13, `credit-001` là operation identity nghiệp vụ, không tự lấy tên từ transport delivery ID. Khi nhận webhook, trước hết phải xác thực payload và ánh xạ nó sang operation mà application cho phép. Một delivery ID mới cho cùng operation không được tạo tác dụng phụ mới; ngược lại, hai operation khác nhau không được gộp chỉ vì tình cờ có cùng nội dung. Việc ánh xạ này là policy của bot, chưa được fixture GitHub cũ chứng minh.

Hãy tự dựng bảng ba checkpoint trong `TestRecoveryWindows`, rồi chạy test. Ở checkpoint sau send, phía nhận đã commit mà phía gửi vẫn pending. Lần dispatch mới bắt buộc có khả năng gửi lặp. `TestSendRetryAndReceiverConflict` kiểm tra hai attempts và một hiệu ứng, đồng thời từ chối payload xung đột ở phía nhận. Bằng chứng này dùng hai database cục bộ và sender gọi receiver trực tiếp; nó không xác nhận GitHub cung cấp một idempotency contract tương đương cho mọi endpoint.

Một bot thật cần chọn side effect có thể lặp an toàn, đọc lại trạng thái có identity ổn định hoặc dùng cơ chế idempotency của API thực tế. Nếu một request tạo comment đã timeout sau khi server xử lý, không được suy "thất bại nên cứ tạo lại". Outbox giữ việc chưa được xác nhận, không biến network thành transaction. Policy retry còn cần budget, delay, giới hạn số lần và đường đối soát; lab này giữ retry do người vận hành/test gọi lại, không thêm vòng retry nền vô hạn.

4. Quản trị Rate Limit: Primary và Secondary

Khi bot gọi GitHub API, rate limit là một budget cần quan sát. Nó không phải rủi ro production duy nhất hay luôn là nút thắt lớn nhất:

Tuyệt đối không xem "5.000 requests/giờ" là con số phổ quát cố định. GitHub áp dụng hạn mức phân cấp tùy theo loại định danh và ngữ cảnh ủy quyền:

Loại định danh (Authentication Context)	Hạn mức Primary Quota	Dấu hiệu nhận diện
Chưa xác thực (Unauthenticated)	60 requests/giờ (tính theo địa chỉ IP)	Dễ bị nghẽn trong môi trường NAT/CI chung
PAT / OAuth User	Mức cơ bản 5.000 request/giờ theo user; app Enterprise đủ điều kiện có mức cao hơn.	Đọc X-RateLimit-Limit của response.
GitHub App user access token	Chung quota theo user; app thuộc Enterprise Cloud đủ điều kiện có thể là 15.000/giờ.	Không coi mỗi app là một quota user độc lập.
GitHub App: Installation (Non-Enterprise)	Base 5.000 req/h; nếu repos > 20: +50/h/repo; nếu org users > 20: +50/h/user; trần tối đa 12.500 req/h	Phù hợp nhất cho bot tự động hóa cấp tổ chức
GitHub App: Enterprise Cloud Installation	Trần tối đa 15.000 requests/giờ	Áp dụng cho tổ chức trên GitHub Enterprise Cloud
GITHUB_TOKEN trong GitHub Actions	1.000 requests/giờ cho mỗi repository (hoặc Enterprise rate nếu resource thuộc Enterprise Cloud)	Áp dụng cho runner tiêu chuẩn trong GitHub Actions
GitHub Enterprise Server (On-Premises)	Cấu hình độc lập bởi quản trị viên hệ thống	Tùy biến theo chính sách triển khai tự lưu trữ

Ngoài primary rate limit, GitHub còn áp dụng secondary rate limit, chẳng hạn cho mức đồng thời hay tốc độ tạo nội dung. Khi vượt mức, response có thể là `403` hoặc `429`. Nếu có `Retry-After`, phải chờ ít nhất khoảng ấy; header này không được bảo đảm luôn có. Khi thiếu nó, đọc thông báo và các rate-limit header khác, rồi áp dụng budget chờ/thử lại theo hướng dẫn API, không loop tức thời.

Thuật toán bóc tách Rate Limit trong Go

```
func ParseRateLimit(
    header http.Header, now time.Time,
) RateLimitStatus {
    status := RateLimitStatus{}

    remStr := header.Get("X-RateLimit-Remaining")
    if remStr != "" {
        if rem, err := strconv.Atoi(remStr); err == nil {
            status.Remaining = rem
            if rem == 0 {
                status.IsExhausted = true
            }
        }
    }

    resetStr := header.Get("X-RateLimit-Reset")
    if resetStr != "" {
        epoch, err := strconv.ParseInt(resetStr, 10, 64)
        if err == nil {
            status.ResetAt = time.Unix(epoch, 0)
            if status.IsExhausted && status.ResetAt.After(now) {
                status.RetryAfter = status.ResetAt.Sub(now)
            }
        }
    }

    // Secondary rate limit dùng header Retry-After
    retryStr := header.Get("Retry-After")
    if retryStr != "" {
        if sec, err := strconv.Atoi(retryStr); err == nil {
            status.RetryAfter = time.Duration(sec) * time.Second
            status.IsExhausted = true
        }
    }
    return status
}
```

---

5. Tương tác GitHub API (`go-github/v92`) và Git trên bộ nhớ (`go-git/v5`)

Một công cụ tự động hóa toàn diện cần tương tác đồng thời với cả hai thế giới: giao tiếp với GitHub API thông qua thư viện `go-github/v92` do Google duy trì để truy xuất siêu dữ liệu, và thao tác cây thư mục Git cục bộ thông qua `go-git/v5`.

Tích hợp GitHub Client

Trong gói `labs/part25-github-automation/github_client.go`, ta đóng gói `github.Client` để hỗ trợ cấu hình linh hoạt endpoint mạng (cho phép trỏ tới máy chủ kiểm thử cục bộ `httptest.Server` hoặc máy chủ GitHub Enterprise):

```
type GitHubClient struct {
    client *github.Client
}

func NewGitHubClient(
    httpClient *http.Client, baseURL string,
) (*GitHubClient, error) {
    var opts []github.ClientOptionsFunc
    if httpClient != nil {
        opts = append(opts, github.WithHTTPClient(httpClient))
    }
    if baseURL != "" {
        if !strings.HasSuffix(baseURL, "/") {
            baseURL += "/"
        }
        opts = append(opts, github.WithURLs(&baseURL, nil))
    }
    gh, err := github.NewClient(opts...)
    if err != nil {
        return nil, fmt.Errorf("bad client options: %w", err)
```

```
    }
    return &GitHubClient{client: gh}, nil
}

func (c *GitHubClient) GetRepository(
    ctx context.Context, owner, repo string,
) (*github.Repository, *github.Rate, error) {
    repository, resp, err := c.client.Repositories.Get(
        ctx, owner, repo,
    )
    if err != nil {
        return nil, nil, err
    }
    var rate *github.Rate
    if resp != nil {
        rate = &resp.Rate
    }
    return repository, rate, nil
}
```

Thao tác Git trên bộ nhớ (In-memory Git) với go-git

Gọi Git CLI tạo dependency vào executable và lifecycle subprocess; thư mục làm việc cần ownership riêng nếu chạy đồng thời. `go-git` là lựa chọn khác: ví dụ này dùng `memory.NewStorage()` cho object store và `memfs.New()` cho working tree, không chứng minh mọi dùng CLI đều xung đột:

```
func NewInMemGitRepository() (*InMemGitRepository, error) {
    fs := memfs.New()
    storer := memory.NewStorage()

    repo, err := git.Init(storer, fs)
    if err != nil {
        return nil, fmt.Errorf("lỗi init git mem: %w", err)
    }

    return &InMemGitRepository{repo: repo, fs: fs}, nil
}
```

Ghi file và tạo commit với chữ ký tác giả trong bộ nhớ:

```
func (r *InMemGitRepository) WriteFileAndCommit(
    filePath string, content []byte, author, msg string,
) (string, error) {
    wt, err := r.repo.Worktree()
    if err != nil {
        return "", err
    }

    f, err := r.fs.Create(filePath)
    if err != nil {
        return "", err
    }
    _, _ = f.Write(content)
    _ = f.Close()

    if _, err := wt.Add(filePath); err != nil {
        return "", err
    }

    hash, err := wt.Commit(msg, &git.CommitOptions{
        Author: &object.Signature{
            Name:  author,
            Email: author + "@bot.local",
            When:  time.Now(),
        },
    })
    if err != nil {
        return "", err
    }
    return hash.String(), nil
}
```

Với `memory.NewStorage()` và `memfs` của ví dụ, Git object và working tree không được chủ động ghi ra filesystem. Hashing, allocation và GC vẫn có chi phí; hệ điều hành có thể page/swap memory, nên không suy ra "không một byte xuống đĩa vật lý" hay một mức latency từ cấu hình in-memory.

---

6. Bằng chứng kiểm thử: Chứng minh 5 quy luật tự động hóa

Test trong `labs/part25-github-automation/` kiểm tra các case HMAC, ledger, rate-limit parser và Git trong memory; HTTP fixture kiểm tra đường dùng `go-github`. Chúng không chứng minh GitHub thật luôn delivery thành công, không có replay đổi header ID, hay mọi lịch crash đều an toàn:

```
=== RUN   TestWebhookHMACVerification
--- PASS: TestWebhookHMACVerification (0.00s)
=== RUN   TestWebhookDeliveryIdempotency
--- PASS: TestWebhookDeliveryIdempotency (0.00s)
=== RUN   TestRateLimitParsingPrimaryAndSecondary
--- PASS: TestRateLimitParsingPrimaryAndSecondary (0.00s)
=== RUN   TestInMemGitCommitAndHead
--- PASS: TestInMemGitCommitAndHead (0.00s)
=== RUN   TestGitHubClientIntegrationWithMockServer
--- PASS: TestGitHubClientIntegrationWithMockServer (0.00s)
PASS
ok      part25-github-automation   0.263s
```

1. Xác thực chữ ký an toàn (TestWebhookHMACVerification)

Kiểm chứng 4 ca biên: chữ ký đúng định dạng `sha256=` với secret hợp lệ; từ chối khi sửa đổi 1 byte trong payload; từ chối khi dùng sai Webhook Secret; và từ chối khi header sai định dạng.

2. Khử trùng lặp sự kiện (TestWebhookDeliveryIdempotency)

Gửi 2 webhook mang cùng một `X-GitHub-Delivery`. Lần đầu tiên xử lý thành công (`isDuplicate = false`). Lần thứ hai được nhận diện chính xác là sự kiện gửi lặp (`isDuplicate = true`), bảo vệ hệ thống không bị kích hoạt kép khi có redelivery từ UI hoặc qua REST API.

3. Bóc tách Rate Limit hai tầng (TestRateLimitParsing)

Kiểm chứng cả hai tình huống: khi chạm trần Primary Quota (`Remaining: 0`), tính đúng thời gian cần chờ đến mốc `ResetAt`; khi chạm trần Secondary Quota (`Retry-After: 120`), chuyển đổi chính xác thành khoảng chờ 120 giây.

4. Vận hành Git trên RAM (TestInMemGitCommitAndHead)

Tạo liên tiếp 2 commit trong bộ nhớ RAM, kiểm tra mã băm SHA của commit thứ nhất và thứ hai, xác nhận con trỏ HEAD dịch chuyển chuẩn xác theo đồ thị DAG.

5. Tương tác GitHub Client (TestGitHubClientIntegrationWithMockServer)

Kiểm chứng việc khởi tạo client `google/go-github/v92`, định tuyến qua máy chủ giả lập `httptest.Server`, trích xuất chính xác cấu trúc repository và thông tin hạn mức `Rate` từ tiêu đề phản hồi.

---

7. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
So sánh chữ ký bằng == thay vì `hmac.Equal`.	Rò rỉ thông tin qua thời gian thực thi (timing side-channel).	Dùng `crypto/hmac.Equal` hoặc `crypto/subtle.ConstantTimeCompare`.
Bỏ qua X-GitHub-Delivery và xử lý mù quáng mọi webhook.	Gây trùng lặp hành vi (tạo 2 PR, merge 2 lần) khi có redelivery từ UI hoặc qua REST API.	Lưu `X-GitHub-Delivery` vào bộ đệm và bỏ qua các sự kiện trùng lặp.
Gọi API dồn mà không quan sát quota.	Có thể cạn primary hoặc secondary budget theo token/endpoint thực tế.	Đọc rate-limit headers; chọn budget chờ/retry theo workload. Mốc Remaining dưới 50 chỉ là ví dụ policy, không phải ngưỡng an toàn chung.
Dùng thư mục tạm chung không có ownership/cleanup.	Có thể xung đột hoặc để lại dữ liệu nhạy cảm.	Dùng thư mục riêng và cleanup rõ, hoặc memory storage khi kích thước và lifecycle phù hợp; RAM không tự là ranh giới bảo mật.

---

8. Bài tập thực hành thiết kế Bot Tự động hóa

Thử thách 1: Lọc sự kiện theo nhánh (Branch Filtering Middleware)

Yêu cầu: Một webhook `push` được gửi tới mỗi khi có commit mới. Hãy viết hàm lọc sự kiện chỉ cho phép xử lý nếu sự kiện nhắm vào nhánh `refs/heads/main`. Bỏ qua tất cả các sự kiện của nhánh tính năng (`feature/*`) mà không kích hoạt quy trình triển khai.

Thử thách 2: Bộ điều tốc thông minh (Smart Backoff with Jitter)

Yêu cầu: Với fixture `Retry-After: 30`, tính thời gian chờ ít nhất 30 giây rồi thêm jitter 10–25%. Nếu nhiều worker cùng chờ, jitter có thể giảm nhịp gửi đồng loạt, không bảo đảm tránh secondary limit. Các tỷ lệ này là cấu hình bài tập, không phải ngưỡng GitHub hay security policy production.

---

9. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Lọc sự kiện theo nhánh đích

```
type PushPayload struct {
    Ref string `json:"ref"`
}

func ShouldDeployFromWebhook(payload []byte) bool {
    var push PushPayload
    if err := json.Unmarshal(payload, &push); err != nil {
        return false
    }

    // Chỉ kích hoạt tự động hóa trên nhánh chính (main)
return push.Ref == "refs/heads/main"
}
```

Đây chỉ là bộ lọc tối thiểu để luyện đọc payload, không phải policy triển khai. Endpoint production phải xác thực chữ ký trước khi parse, allowlist cả tên event và action, repository/owner, installation ID (nếu dùng GitHub App) và ref; sau đó ràng buộc các giá trị đó với cấu hình deployment. Chỉ kiểm tra `ref` sẽ cho phép một kho hoặc installation ngoài phạm vi kích hoạt hành động nhạy cảm.

Lời giải Thử thách 2: Tính toán Backoff kết hợp Jitter

```
func CalculateBackoffWithJitter(
    baseDelay time.Duration,
    next func() float64,
) (time.Duration, error) {
    if next == nil {
        return 0, errors.New("jitter source is required")
    }
    value := next() // contract: 0 <= value < 1
    if value < 0 || value >= 1 {
        return 0, errors.New("invalid jitter value")
    }
    return baseDelay + time.Duration(
        float64(baseDelay)*(0.10+value*0.15),
    ), nil
}
```

Jitter chỉ giảm tương quan, không triệt tiêu hoàn toàn bão yêu cầu. Hàm nhận nguồn jitter qua dependency injection; production có thể bọc `crypto/rand` hoặc PRNG được sở hữu riêng, còn test truyền giá trị cố định để tái lập được. Không dùng global RNG như một contract ngầm.

Nắm vững cách vận hành **HMAC-SHA256 + Delivery Idempotency + Rate-Limit-Aware Client + In-Memory Git** giúp bạn tự tin xây dựng những hệ thống GitOps và CI/CD bot an toàn, vững chắc. Trong Chương 26, chúng ta sẽ mở rộng năng lực bảo vệ phần mềm lên tầm chuỗi cung ứng: xây dựng **Cổng kiểm soát chuỗi cung ứng phần mềm có thể kiểm chứng (Supply Chain Verification Gate)** bằng Go với chữ ký số Cosign không cần khóa (keyless), OCI manifest digest bất biến và quét lỗ hổng mã nguồn bằng `govulncheck`.

Chương 26 — Chuỗi cung ứng phần mềm có thể kiểm chứng

Một scenario chuỗi cung ứng có thể gồm source nghiệp vụ, nhiều module gián tiếp và base image có package OS. Số thành phần phụ thuộc cách build: image `scratch` và một base image tổng quát không có cùng tập package. Vì vậy inventory phải được lấy từ artifact thực tế, không ước lượng từ số dòng Go của application.

Điều đó mở rộng bề mặt rủi ro của **Chuỗi cung ứng phần mềm (Software Supply Chain)**.

Các sự cố an ninh trên thực tế — từ SolarWinds, Codecov cho đến các cuộc tấn công Dependency Confusion và Typosquatting — cho thấy một ranh giới bảo mật quan trọng:

> **Một kho mã nguồn sạch, được review kỹ lưỡng và vượt qua mọi bài kiểm thử unit test, vẫn có thể cho ra lò một container image độc hại nếu quy trình build, dependency hoặc kho lưu trữ artifact bị xâm phạm.**

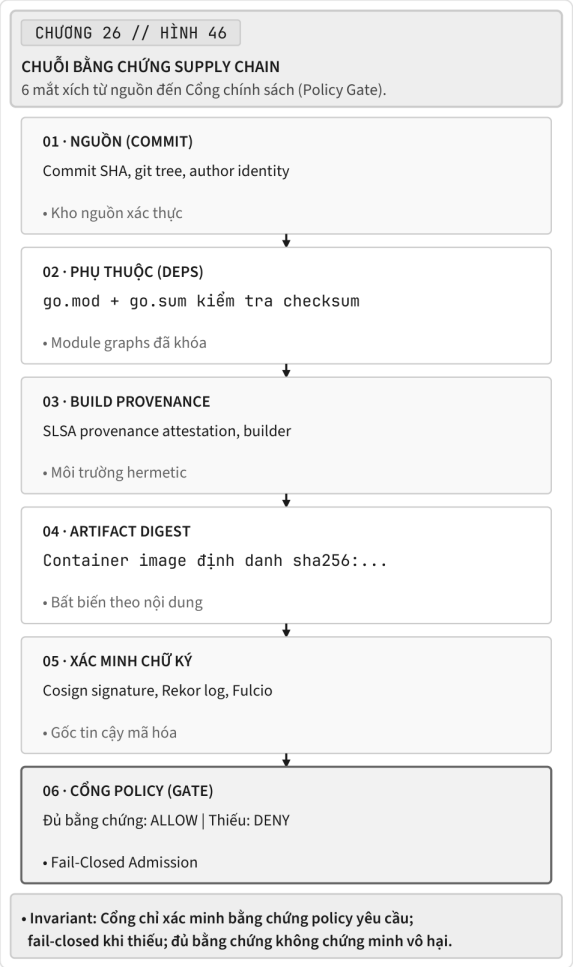
Ta cần kiểm tra image nào đã được xét, bằng chứng nào gắn nó với builder/source được phép, và finding nào phải chặn theo policy. Chữ ký và scan không chứng minh artifact không có lỗ hổng hay mã độc; từng verifier chỉ đưa ra kết luận trong phạm vi dữ liệu và trust root của nó.

Chương này hướng dẫn bạn tư duy và xây dựng một **Cổng kiểm soát chuỗi cung ứng phần mềm (Supply Chain Verification Gate)** bằng Go theo nguyên tắc đóng kín (fail-closed), tích hợp kiểm chứng digest OCI, chữ ký số mật mã ECDSA, chứng thực nguồn gốc SLSA và phân tích khả năng vươn tới của lỗ hổng (`govulncheck`).

---

1. Mental Model: Chuỗi bảo chứng từ Mã nguồn đến Triển khai

Mô hình dưới đây nối sáu chặng của một policy delivery. Mỗi mũi tên đòi bằng chứng riêng; đây không phải chuỗi tự bảo chứng:



Hình 46 — Cổng chỉ xác minh các bằng chứng mà policy yêu cầu. Digest, chữ ký và provenance có vai trò khác nhau; đủ bằng chứng theo policy không chứng minh artifact không có mọi lỗi hoặc lỗ hổng.

Mỗi mắt xích cần bằng chứng phù hợp, nhưng không một file hay label nào tự chứng minh toàn bộ chuỗi. `go.sum` ghi checksum module mà toolchain đã biết và kiểm tra, chứ không khóa hoàn toàn graph như lockfile. Provenance chỉ hữu ích sau khi verifier kiểm tra subject, identity và expectation của policy. Digest nhận diện nội dung; Cosign keyless verification còn kiểm tra trust root, certificate identity/issuer và claim digest. Lab dưới đây mô hình hóa policy fail-closed, không thực hiện các bước Cosign/SLSA đó.

---

2. Toàn vẹn Module: `go.sum` và Go Checksum Database

Trước khi container image được build, chuỗi cung ứng bắt đầu ngay tại thời điểm Go tải các gói phụ thuộc về máy.

Bản chất kỹ thuật của `go.sum`

Một ngộ nhận kinh điển trong cộng đồng là coi `go.sum` như một tệp khóa phiên bản tương tự `package-lock.json` hay `yarn.lock`. Về mặt bản chất, tệp `go.mod` mới là nơi quyết định phiên bản module thông qua giải thuật lựa chọn phiên bản tối thiểu (Minimal Version Selection - MVS). Ngược lại, `go.sum` là cơ sở dữ liệu xác thực nội dung mật mã (Content Validation Database). Nó lưu trữ các mã băm SHA-256 của toàn bộ mã nguồn module và tệp `go.mod` tương ứng nhằm phát hiện mọi hành vi can thiệp trái phép hoặc thay đổi nội dung sau thời điểm phát hành chính thức.

Mỗi bản ghi trong `go.sum` được cấu trúc thành hai dòng thông tin bổ trợ:

```
github.com/gin-gonic/gin v1.9.1 h1:4A06lVSJ...
github.com/gin-gonic/gin v1.9.1/go.mod h1:h1hp...
```

Dòng định dạng `h1:<base64-hash>` chứa chuỗi băm chuẩn tắc của cây thư mục module (theo giải thuật `dirhash.Hash1`): danh sách đường dẫn tệp được sắp xếp thứ tự, băm SHA-256 từng tệp, ghép thành danh sách định dạng chuẩn tắc rồi băm SHA-256 tổng thể và mã hóa base64. Dòng có hậu tố `/go.mod` lưu trữ mã băm chỉ tính riêng trên nội dung tệp khai báo phụ thuộc của module đó, cho phép công cụ Go kiểm tra cây phụ thuộc một cách nhanh chóng mà không bắt buộc phải tải toàn bộ mã nguồn của các module gián tiếp về máy.

Go Checksum Database (`sum.golang.org`)

Nếu một kẻ xấu xâm nhập được máy chủ Git của một thư viện bên thứ ba và tráo đổi nội dung của release tag `v1.2.0`, điều gì sẽ xảy ra?

Nếu máy bạn chưa từng tải bản release đó, làm sao Go biết mã băm `h1:` nào là chuẩn?

Với module thuộc phạm vi kiểm tra checksum database và cấu hình mặc định phù hợp, Go toolchain đối chiếu checksum đã tải với transparency log `sum.golang.org`. Nội dung không khớp gây lỗi checksum. Cơ chế này kiểm tra tính nhất quán của nội dung, không chứng minh nội dung không độc hại; `GOPRIVATE`, `GONOSUMDB`, `GOSUMDB=off` và source thay bằng `replace` còn tạo các boundary khác. Phải đọc cấu hình build trước khi suy ra module đã qua cùng đường kiểm tra.

> **Khuyến nghị:** Commit `go.sum` cùng `go.mod` khi module có dependency tạo file ấy. Không bỏ checksum đã ghi nhận chỉ để tránh merge conflict. Checksum database vẫn có thể kiểm tra module đủ điều kiện ngay cả khi chưa có entry local; vì vậy thiếu `go.sum` không tự chứng minh đã xảy ra tráo source, nhưng làm mất bằng chứng checksum đã được project lưu.

---

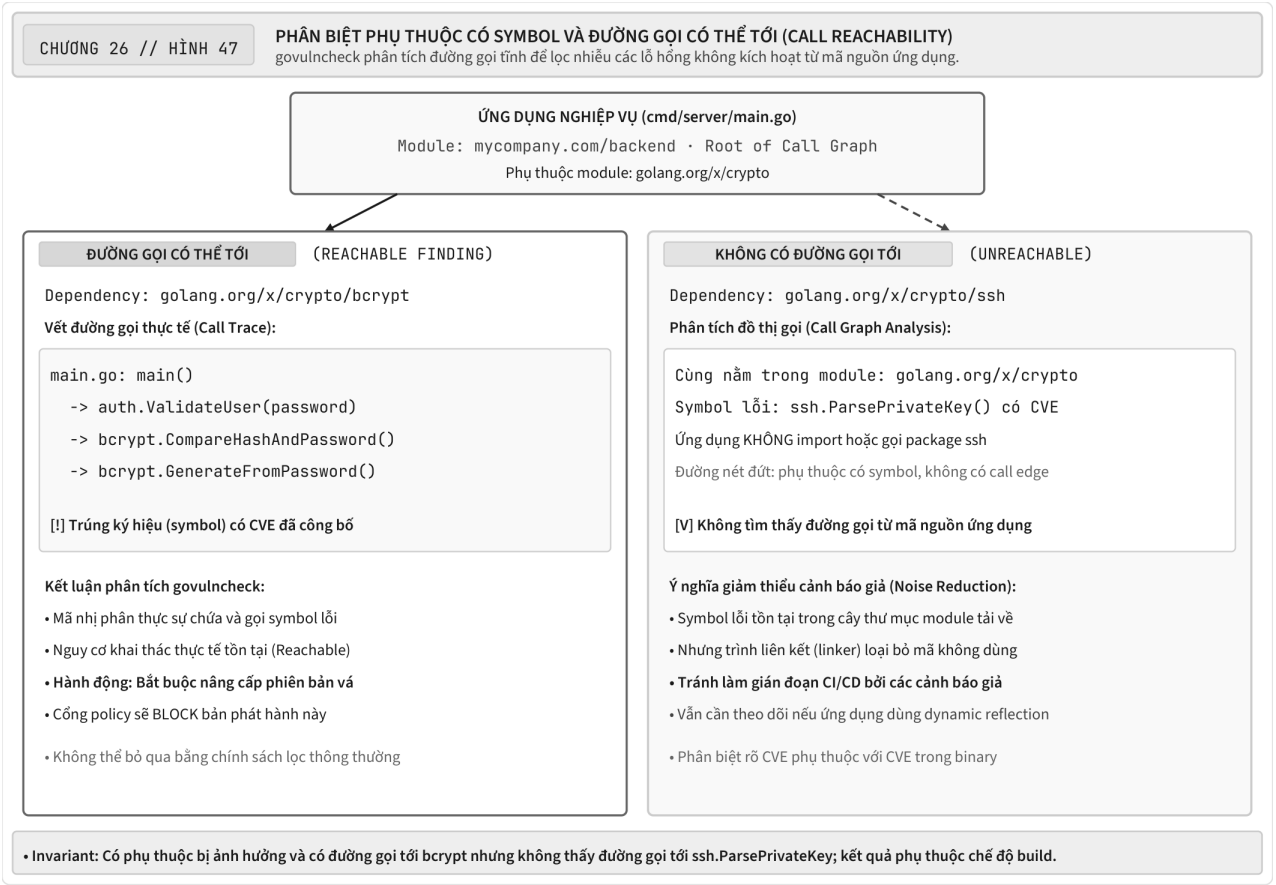
3. Quét lỗ hổng tĩnh vs Khả năng vươn tới ký hiệu (`govulncheck`)

Trong các pipeline CI/CD truyền thống, các công cụ quét container (như Trivy, Grype, Snyk) thường đối chiếu danh sách gói phần mềm với cơ sở dữ liệu CVE. Cách tiếp cận này tạo ra một vấn nạn nghiêm trọng trong vận hành: **Hội chứng mệt mỏi vì cảnh báo (Alert Fatigue)**.

Xét một scenario: project dùng `golang.org/x/crypto`, và version đang dùng có finding ở `ssh.ParsePrivateKey`, nhưng code ứng dụng chỉ gọi `bcrypt.GenerateFromPassword`. Một scanner chỉ xét inventory module có thể báo finding dù symbol không nằm trong đường gọi đã phân tích. Báo finding và chặn phát hành là hai quyết định khác nhau; policy của pipeline mới chọn điều kiện chặn. Đây không phải một CVE thật được gán cho version tùy ý.

Cơ chế phân tích đồ thị cuộc gọi của `govulncheck`

Ở chế độ source, `govulncheck` còn phân tích khả năng vươn tới của ký hiệu, thay vì chỉ báo module có finding. Phải đọc chế độ scan và coverage của kết quả đang dùng.



Hình 47 — Trong ví dụ phân tích này, ứng dụng có đường gọi tới bcrypt nhưng không thấy đường gọi tới ssh.ParsePrivateKey. Kết quả phải được hiểu trong giới hạn của bộ phân tích và build đang xét.

JSON của govulncheck là stream message theo schema của version dùng, có OSV và finding với trace; không phải một object cố định chứa `Modules` và `Traces` viết hoa. Trong lab, `Severity` và `Reachable` là input fixture đã chuẩn hóa, không được parse từ output scanner. Finding có trace tới symbol chỉ là kết quả phân tích tĩnh về đường gọi có thể tới, không chứng minh đường ấy chạy hoặc khai thác được. Adapter thật phải ghi version/schema, chế độ source hay binary, scanner coverage và cách suy ra policy từ finding.

Giới hạn phân tích cần lưu ý

Cần lưu ý rằng khả năng phân tích tĩnh của `govulncheck` tập trung vào mã nguồn Go. Kỹ thuật xấp xỉ đồ thị cuộc gọi tĩnh có giới hạn trước reflection, hàm assembly hay điều phối động, không phân tích các phụ thuộc liên kết động qua CGO runtime, không lần vết trong plugin tải động (`plugin.Open`), và không quét các thư viện hệ thống nhị phân của base image container (như OpenSSL hay glibc — những thành phần này cần công cụ quét container chuyên dụng).

Thiết kế chính sách thông minh

`Reachable`, severity, scanner coverage và acceptance threshold đều là input policy. Một tổ chức có thể deny finding reachable high/critical, tổ chức khác có exception có hạn hay deny cả finding chưa chứng minh reachability. Vì vậy `Reachable=false` không có nghĩa "an toàn"; trong lab nó chỉ dẫn tới `WARN` theo policy minh họa và cần được review trong policy thật.

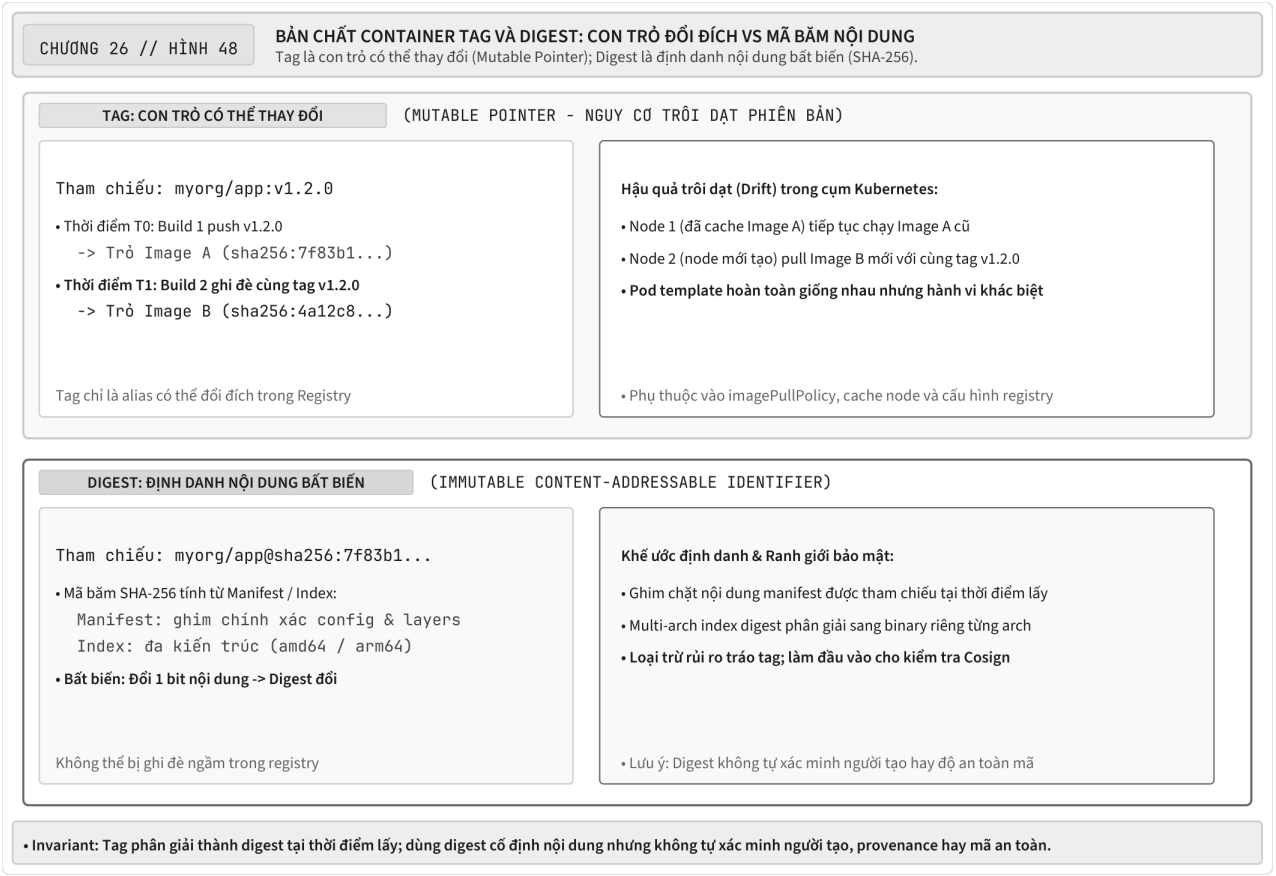
---

4. Bất biến OCI: Mã băm Digest vs Nhãn Tag có thể biến đổi

Khi container image được đóng gói và đẩy lên OCI Registry (Docker Hub, AWS ECR, GitHub Packages), chúng ta thường gắn thẻ tag như `:latest` hoặc `:v1.2.3`.

Tuy nhiên, trong thế giới container:

> **Tag là con trỏ có thể thay đổi (Mutable Pointer), chỉ có Digest mới là Nguồn chân lý bất biến (Immutable Truth).**



Hình 48 — Tag được phân giải thành digest tại một thời điểm. Dùng digest cố định nội dung được tham chiếu, không tự xác minh người tạo hay độ an toàn của nội dung ấy.

Nếu tag được trỏ sang image khác, một lần pull tiếp theo có thể nhận nội dung khác dù Pod template vẫn cùng chuỗi tag. Kết quả còn phụ thuộc imagePullPolicy, cache node và policy registry. Ghim digest giúp ràng buộc nội dung đã xét, không tự xác thực builder hay chứng minh artifact vô hại.

Policy của lab chọn quy tắc đầu tiên: **Từ chối mọi image không được định danh tường minh bằng mã băm SHA-256 dạng** `sha256:<64_hex_chars>`.

---

5. Chữ ký số Sigstore/Cosign và Attestation SLSA

Làm sao chúng ta biết một container image có mã băm `sha256:abc...` thực sự được tạo ra bởi quy trình CI chính thức của tổ chức chứ không phải do hacker tự biên dịch rồi đẩy lên?

Sigstore/Cosign là một lựa chọn cho việc ký và xác minh artifact:

Một là, Ký không cần khóa tĩnh (Keyless Signing): Thay vì quản lý private key dài hạn trên máy chủ CI, runner tạo cặp khóa tạm thời (ephemeral keys) cho mỗi lần ký và dùng OpenID Connect (OIDC) token để xin chứng chỉ ngắn hạn từ Fulcio ràng buộc danh tính workflow với public key.

Hai là, chứng chỉ ngắn hạn: Fulcio ràng buộc public key với identity OIDC theo policy của CA.
`https://token.actions.githubusercontent.com` là issuer, không phải danh tính riêng của workflow. Verifier phải xét cả certificate identity được phép và issuer, validity cùng evidence của đường verify đang dùng.

Ba là, Ký và xác minh claim digest: Cosign ký payload có claim về digest image; verifier mặc định kiểm tra claim đó cùng danh tính/certificate theo policy. Đây không tương đương với việc gọi `ecdsa.VerifyASN1` trên chuỗi digest trong lab.

Bốn là, SLSA Provenance Attestation: Ngoài chữ ký, CI còn tạo ra bản chứng thực nguồn gốc (Provenance) ghi rõ: Commit SHA nào, quy trình workflow nào (Builder ID) đã tạo ra artifact.

---

6. Xây dựng Cổng chính sách Fail-Closed trong Go

Bây giờ, chúng ta sẽ hiện thực hóa toàn bộ các nguyên lý trên vào một module Go theo **Mô hình chính sách kiểm chứng sư phạm (Pedagogical Verification Policy Model).**

Mô hình này không nhằm mục đích thay thế hay bao bọc toàn bộ mã nguồn của Cosign CLI hay SLSA verifier bên ngoài. Thay vào đó, nó tách bạch rõ ràng 3 khế ước giao tiếp (interfaces) cốt lõi của một hệ thống kiểm định chuỗi cung ứng hiện đại:

Khế ước giao tiếp	Nhiệm vụ kỹ thuật
`SignatureVerifier`	Chịu trách nhiệm xác thực chữ ký số mật mã của image (Cosign / Notary).
`ProvenanceVerifier`	Chịu trách nhiệm kiểm chứng xuất xứ bản build (SLSA Provenance / in-toto).
`VulnerabilityProvider`	Chịu trách nhiệm cung cấp dữ liệu lỗ hổng (govulncheck / scanner).

Khai báo các Interface và Mô hình dữ liệu chính sách

```
type Decision string

const (
    DecisionAllow Decision = "ALLOW"
    DecisionDeny  Decision = "DENY"
)

// 3 Interface cốt lõi của Cổng kiểm định chuỗi cung ứng:
type SignatureVerifier interface {
    VerifySignature(
        digest string, sig *SignatureVerification,
    ) error
}

type ProvenanceVerifier interface {
    VerifyProvenance(
        digest string, att *Attestation,
    ) error
}

type VulnerabilityProvider interface {
    GetVulnerabilities(
        digest string,
    ) ([]Vulnerability, error)
}

// Vulnerability mô phỏng phát hiện với ngữ nghĩa
// reachability của govulncheck và mức độ từ OSV.
type Vulnerability struct {
    ID        string `json:"id"`
    Package   string `json:"package"`
    Symbol    string `json:"symbol"`
    Severity  string `json:"severity"` // Từ OSV
    Reachable bool   `json:"reachable"` // Từ call-graph
}
```

Metadata Xuất xứ, Chữ ký số và Bộ điều phối PolicyEngine

```
// Attestation đại diện cho metadata xuất xứ SLSA.
type Attestation struct {
    BuilderID     string `json:"builderId"`
    SubjectDigest string `json:"subjectDigest"`
}

// SignatureVerification chứa thông tin chữ ký ECDSA
// và định danh OIDC Issuer.
type SignatureVerification struct {
    PublicKey *ecdsa.PublicKey
    Signature []byte
    Issuer    string
}

type EvaluationResult struct {
    Decision   Decision `json:"decision"`
    Violations []string `json:"violations,omitempty"`
    Warnings   []string `json:"warnings,omitempty"`
}

// PolicyEngine điều phối việc kiểm định fail-closed
```

```
type PolicyEngine struct {
    TrustedBuilders []string
    TrustedIssuers  []string
    SigVerifier     SignatureVerifier
    ProvVerifier    ProvenanceVerifier
}

func NewPolicyEngine(builders, issuers []string) *PolicyEngine {
    sigV := &DefaultSignatureVerifier{
        TrustedIssuers: issuers,
    }
    provV := &DefaultProvenanceVerifier{
        TrustedBuilders: builders,
    }
    return &PolicyEngine{
        TrustedBuilders: builders,
        TrustedIssuers:  issuers,
        SigVerifier:     sigV,
        ProvVerifier:    provV,
    }
}
```

Kiểm tra tính bất biến của OCI Digest

```
// VerifyDigest xác thực artifact dùng mã băm bất biến.
func VerifyDigest(digest string) error {
    if !digestRegex.MatchString(digest) {
        return errors.New(
            "invalid OCI digest: must be sha256:64hex",
        )
    }
    return nil
}
```

Xác thực chữ ký mật mã ECDSA và OIDC Issuer

Đọc đoạn sau như một model dùng input fixture đã tin cậy, không phải verifier identity. Public key do caller cung cấp; `Issuer` chỉ là chuỗi caller khai báo, không được ràng buộc với key bằng certificate. `Attestation` cũng là struct chưa xác thực. Một người tự tạo key, ký digest rồi chép issuer/builder được allowlist vẫn được model chấp nhận. `TestModelAllowsSelfAssertedIdentity` giữ phản ví dụ này để ngăn test ALLOW bị đọc thành bằng chứng GitHub OIDC hay SLSA. Khi dùng gate thật, phải thay các verifier bằng đường xác minh trust root, identity và provenance phù hợp; thêm một phép so chuỗi không đóng được gap này.

```
func (e *PolicyEngine) VerifySignature(
    digest string,
    sig *SignatureVerification,
) error {
    if sig == nil || sig.PublicKey == nil {
        return errors.New(
            "missing cryptographic signature or public key",
        )
    }

    // 1. Kiểm tra OIDC Issuer có nằm trong danh sách tin cậy
    trustedIssuer := false
    for _, ti := range e.TrustedIssuers {
        if sig.Issuer == ti {
            trustedIssuer = true
            break
        }
    }
    if !trustedIssuer {
        return fmt.Errorf(
            "untrusted OIDC signature issuer: %s",
```

```
            sig.Issuer,
        )
    }

    // 2. Kiểm tra chữ ký toán học ECDSA P-256
    hash := sha256.Sum256([]byte(digest))
    if len(sig.Signature) == 0 {
        return errors.New("empty signature payload")
    }

    valid := ecdsa.VerifyASN1(
        sig.PublicKey, hash[:], sig.Signature,
    )
    if !valid {
        return errors.New(
            "signature verification failed: bad signature",
        )
    }
    return nil
}
```

Thực thi đánh giá toàn diện (Fail-Closed Evaluation)

Khâu thẩm định được chia thành các chốt chặn nối tiếp nhau. Đầu tiên là kiểm tra tính toàn vẹn của mã băm và chữ ký số:

```
func (e *PolicyEngine) evaluateIntegrity(
    digest string,
    sig *SignatureVerification,
    res *EvaluationResult,
) bool {
    // Cửa 1: Khóa cứng tính bất biến OCI Digest
    if err := VerifyDigest(digest); err != nil {
        res.Decision = DecisionDeny
        res.Violations = append(res.Violations, err.Error())
        return false // Dừng ngay lập tức (fail closed)
    }

    // Cửa 2: Xác thực chữ ký số Cosign
    if err := e.VerifySignature(digest, sig); err != nil {
        res.Decision = DecisionDeny
        res.Violations = append(res.Violations, err.Error())
    }
    return true
}
```

Tiếp theo, mô hình policy kiểm tra các field provenance đã được cung cấp. Trong hệ thật, verifier phải xác thực bằng chứng trước khi truyền chúng vào gate:

```
func (e *PolicyEngine) evaluateProvenance(
    digest string,
    att *Attestation,
    res *EvaluationResult,
) {
    // Cửa 3: Kiểm chứng xuất xứ SLSA và Builder ID
    if att == nil {
        res.Decision = DecisionDeny
        res.Violations = append(
            res.Violations,
            "missing SLSA provenance attestation",
        )
        return
    }

    if att.SubjectDigest != digest {
        res.Decision = DecisionDeny
        res.Violations = append(
            res.Violations,
            "provenance subject digest mismatch",
        )
```

```
    }

    builderOK := false
    for _, tb := range e.TrustedBuilders {
        if att.BuilderID == tb {
            builderOK = true
            break
        }
    }
    if !builderOK {
        res.Decision = DecisionDeny
        res.Violations = append(
            res.Violations,
            fmt.Sprintf(
                "untrusted builder: %s",
                att.BuilderID,
            ),
        )
    }
}
```

Cuối cùng là chốt chặn kiểm tra lỗ hổng bảo mật với ngữ nghĩa reachability của `govulncheck`:

```
func (e *PolicyEngine) evaluateVulnerabilities(
    vulns []Vulnerability,
    res *EvaluationResult,
) {
    // Cửa 4: Kiểm soát lỗ hổng govulncheck
    for _, v := range vulns {
        isCrit := strings.EqualFold(v.Severity, "CRITICAL")
        isHigh := strings.EqualFold(v.Severity, "HIGH")

        if isCrit || isHigh {
            if v.Reachable {
                // Hàm lỗi trên đường thực thi: TỪ CHỐI
                res.Decision = DecisionDeny
                res.Violations = append(
                    res.Violations,
                    fmt.Sprintf(
                        "reachable %s vuln %s (sym: %s)",
                        v.Severity, v.ID, v.Symbol,
                    ),
                )
            } else {
                // Hàm lỗi không được gọi: CẢNH BÁO
                res.Warnings = append(
                    res.Warnings,
                    fmt.Sprintf(
                        "uncalled %s vuln %s: tolerated",
                        v.Severity, v.ID,
                    ),
                )
            }
        }
    }
}
```

Hàm `Evaluate` chính thức chỉ việc điều phối tuần tự 3 chốt chặn:

```
func (e *PolicyEngine) Evaluate(
    digest string,
    sig *SignatureVerification,
    att *Attestation,
    vulns []Vulnerability,
) EvaluationResult {
    res := EvaluationResult{Decision: DecisionAllow}
    if !e.evaluateIntegrity(digest, sig, &res) {
        return res
    }
    e.evaluateProvenance(digest, att, &res)
    e.evaluateVulnerabilities(vulns, &res)
    return res
}
```

---

7. Kiểm chứng Lab thực tế (`labs/part26-supply-chain-gate`)

Mã nguồn hoàn chỉnh của bài lab nằm tại thư mục `labs/part26-supply-chain-gate` (phân loại mức kiểm chứng: `UNIT_TESTED` / `MODEL_ONLY` đối với logic chính sách cổng fail-closed và chữ ký ECDSA P-256 nội bộ, chứng minh khế ước an ninh của 7 kịch bản kiểm định mà không phụ thuộc vào hạ tầng mạng Sigstore bên ngoài). Khi chạy kiểm thử với cờ kiểm tra xung đột dữ liệu:

```
go test -v -race ./...
```

Bộ kiểm thử thực hiện xác minh toàn bộ 7 kịch bản:

```
=== RUN   TestPolicyGateAllowedWithValidSignature
--- PASS: TestPolicyGateAllowedWithValidSignature (0.00s)
=== RUN   TestModelAllowsSelfAssertedIdentity
--- PASS: TestModelAllowsSelfAssertedIdentity (0.00s)
=== RUN   TestPolicyGateDenyUnsignedImage
--- PASS: TestPolicyGateDenyUnsignedImage (0.00s)
=== RUN   TestPolicyGateDenyReachableVulnerability
--- PASS: TestPolicyGateDenyReachableVulnerability (0.00s)
=== RUN   TestPolicyGateAllowUncalledVulnerability
--- PASS: TestPolicyGateAllowUncalledVulnerability (0.00s)
=== RUN   TestPolicyGateFailClosedOnMutableTag
--- PASS: TestPolicyGateFailClosedOnMutableTag (0.00s)
=== RUN   TestPolicyGateStopsOnSignatureFailure
--- PASS: TestPolicyGateStopsOnSignatureFailure (0.00s)
PASS
ok      part26-supply-chain-gate   2.128s
```

Phân tích các kịch bản kiểm thử

Kịch bản fixture	Điều kiểm tra trong model	Quyết định (`Decision`)
Input được model chấp nhận	ECDSA tự tạo trong test; chuỗi issuer/builder khớp; không truyền finding. Không gọi GitHub OIDC hay scanner.	**ALLOW** theo policy model
Phản ví dụ tự khai danh tính (`TestModel...`)	Model chấp nhận key tự tạo và issuer/builder tự khai; chứng minh model chưa xác thực OIDC	`ALLOW` (nhấn mạnh ranh giới model)
Image không có chữ ký	Bị chặn đứng ngay tại Cửa 2	`DENY`
Lỗ hổng nghiêm trọng có thể vươn tới (`Reachable = true`)	Ký hiệu `ssh.ParsePrivateKey` được gọi trong ứng dụng	`DENY`
Finding không được fixture đánh dấu reachable (`Reachable = false`)	`http2.Server.ServeConn` là tên symbol minh họa, không phải bằng chứng quan sát mọi đường chạy	`ALLOW` (kèm cảnh báo trong `Warnings`, theo policy lab)
Cố tình dùng tag thay vì digest	Truyền vào chuỗi `my-registry.io/app:v1.2.0`	Bị chặn ngay từ Cửa 1 → `DENY`
Dừng trước provenance sau lỗi chữ ký	Khi chữ ký sai, cổng dừng ngay tại Cửa 2 và không kiểm tra tiếp Cửa 3	`DENY` (fail-closed tuần tự)

8. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Triển khai bằng Docker tag (`:latest` hoặc `:v1.0.0`) thay vì sha256 digest.	Bị tấn công tráo đổi container image khi registry bị thỏa hiệp; pod scale up chạy phiên bản khác nhau.	Ghim (pin) mã băm bất biến `sha256:<hex>` trong Kubernetes Pod Spec.
Nhận public key và issuer do caller tự khai mà coi là identity đã xác thực.	Caller tự ký rồi khai issuer được phép; model vẫn có thể chấp nhận.	Dùng verifier thật kiểm tra trust roots, certificate identity/issuer và claim digest; so chuỗi không đủ.
Thiết kế Cổng dạng Fail-Open: Bỏ qua kiểm tra khi mạng tới OIDC/Registry bị timeout.	Khi mạng gặp sự cố, hệ thống tự động cho phép mọi image chưa được kiểm chứng đi thẳng vào production.	Luôn áp dụng nguyên tắc Fail-Closed: Bất kỳ lỗi mạng hay timeout nào cũng phải quy về `DENY`.
Bỏ go.sum vì xem nó là file rác.	Mất checksum đã ghi nhận trong project; không tự đồng nghĩa checksum database bị tắt.	Commit checksum khi có dependency; hiểu GOSUMDB/GOPRIVATE/replace. go mod verify kiểm tra cache, không quét mã độc.

9. Bài tập thực hành thiết kế Cổng bảo vệ

Thử thách 1: Ghim Base Image chuẩn (Golden Image Pinning)

Yêu cầu: Trong tệp Dockerfile nhiều tầng (multi-stage build), tầng thực thi cuối cùng thường bắt đầu bằng một base image (ví dụ `gcr.io/distroless/static-debian12`). Hãy viết một hàm Go nhận vào danh sách base images tin cậy (dưới dạng ánh xạ giữa tên image và mã băm sha256 cho phép) và kiểm tra xem một dòng `FROM <image>` trong Dockerfile có tuân thủ đúng mã băm đã được phê duyệt hay không.

Thử thách 2: Cơ chế Timeout Fail-Closed (Network Timeout Fallback)

Yêu cầu: Khi cổng kiểm tra gọi sang máy chủ Rekor hoặc Fulcio qua mạng, nếu request bị timeout (quá 2 giây), hãy thiết kế một cấu trúc hàm sao cho: thay vì trả về lỗi chung chung khiến người vận hành bối rối, hàm phải trả về quyết định `DENY` kèm thông điệp vi phạm giải thích rõ ràng rằng: Cổng đã chủ động đóng lại theo nguyên tắc Fail-Closed để bảo vệ an toàn cho cụm máy chủ.

---

10. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Xác thực Golden Base Image

```
type GoldenCatalog map[string]string

func VerifyBaseImage(
    imageRef string,
    catalog GoldenCatalog,
) (bool, error) {
    parts := strings.Split(imageRef, "@")
    if len(parts) != 2 {
        return false, fmt.Errorf(
            "image must be pinned with digest: repo@sha256:hex",
        )
    }

    repo := parts[0]
    digest := parts[1]

    expectedDigest, exists := catalog[repo]
    if !exists {
        return false, fmt.Errorf(
            "untrusted base image repository: %s", repo,
        )
    }

    if digest != expectedDigest {
        return false, fmt.Errorf(
            "base image digest mismatch: expected %s, got %s",
            expectedDigest, digest,
        )
    }

    return true, nil
}
```

Lời giải Thử thách 2: Xử lý Timeout theo nguyên tắc Fail-Closed

```
func SafeRemoteVerify(
    ctx context.Context,
    verifyFunc func(context.Context) error,
) EvaluationResult {
    // Giới hạn thời gian xác thực tối đa 2 giây
    timeoutCtx, cancel := context.WithTimeout(
        ctx, 2*time.Second,
    )
    defer cancel()

    err := verifyFunc(timeoutCtx)
    if err != nil {
        // Nguyên tắc Fail-Closed: Bất kỳ sự cố mạng nào
        // cũng dẫn đến quyết định TỪ CHỐI (DENY)
        return EvaluationResult{
            Decision: DecisionDeny,
            Violations: []string{
                fmt.Sprintf("fail-closed: aborted: %v", err),
            },
        }
    }

    return EvaluationResult{Decision: DecisionAllow}
}
```

Fail-closed từ chối artifact khi thiếu bằng chứng mà policy yêu cầu, kể cả lúc verifier gặp outage. Nó không bảo đảm mọi artifact đã chấp nhận đều vô hại: trust root, signer được phép, nội dung và chính policy vẫn có thể sai hoặc bị xâm phạm. Safety và availability của gate là hai câu hỏi cần kiểm chứng riêng.

Quy trình kiểm soát chuỗi cung ứng phần mềm giúp đối chiếu tính toàn vẹn của mã nguồn, xuất xứ build và chữ ký số artifact container với policy đã chọn. Bằng chứng ấy không chứng minh file nhị phân vô hại: mã nguồn, signer hay trust root vẫn có thể bị xâm phạm.

Khi container thực sự chạy, cần thêm bằng chứng về hành vi: tiến trình nào mở kết nối mạng, đọc tệp cấu hình hay thực hiện thao tác đáng ngờ? Trong

Chương 27, chúng ta dùng **eBPF (Extended Berkeley Packet Filter) kết hợp với Go** để quan sát các sự kiện tại những hook được chọn trong kernel Linux. Đây không phải khả năng bắt trọn mọi hành vi hay quan sát miễn phí. Phạm vi hook, cửa sổ thu thập, sự kiện bị mất và chi phí xử lý đều giới hạn kết luận; phải đo overhead với workload thực tế trước khi triển khai trên production.

Chương 27 — Quan sát Linux từ kernel bằng eBPF và Go

Cho đến thời điểm này của cuốn sách, chúng ta đã xây dựng các công cụ giám sát dựa trên ba trụ cột Observability truyền thống: Metrics (Prometheus), Logs và Tracing (OpenTelemetry).

Instrumentation đặt trong application cần code ứng dụng tham gia để tạo metric và span có ngữ nghĩa nghiệp vụ. Nhưng metrics, logs và tracing không chỉ có nguồn ấy: OS, proxy và các tầng hạ tầng cũng có thể cung cấp tín hiệu độc lập. Câu hỏi ở đây là điểm quan sát nào còn thấy được hành vi khi application không chủ động báo cáo.

Xét một scenario: tiến trình web bị khai thác RCE, mở `/bin/sh`, tải payload vào `/tmp` rồi thực thi mà không tạo span hay metric trong application. Instrumentation của chính application có thể thiếu dấu vết đó; log OS, proxy hoặc audit vẫn có thể hữu ích. Ta bổ sung một điểm quan sát kernel cho hook đã chọn, không tuyên bố eBPF là nguồn nhìn thấy mọi hành vi.

eBPF cho phép đặt điểm quan sát gần kernel cho những hook mà policy và kernel hỗ trợ; nó không nhìn thấy "mọi hành vi" và không thay thế audit log hay kiểm soát truy cập. Trước đây, can thiệp vào kernel thường dùng Linux Kernel Module (LKM) bằng C, có blast radius lớn khi lỗi. eBPF kết hợp `cilium/ebpf` cho phép nạp bytecode qua verifier, nhưng khả năng attach, quyền hạn và overhead vẫn phải được kiểm chứng trên kernel/workload thật.

---

1. Mental Model: Chuỗi luân chuyển sự kiện Kernel - Go Userspace

Mô hình trung tâm cần tách hai thời điểm. Verifier kiểm tra chương trình khi nạp, không chạy lại như một bước trong đường đi của từng sự kiện:

Thời điểm	Đường xử lý của lab	Ranh giới
Nạp và gắn chương trình	Loader nạp object; kernel verifier kiểm tra bytecode; loader gắn chương trình vào tracepoint.	Chấp nhận chương trình không chứng minh policy quan sát đầy đủ hoặc overhead phù hợp.
Một lời gọi `execve` đi qua hook	Chương trình lấy trường sự kiện, reserve/submit record vào ring buffer; Go đọc record, giải mã ABI rồi áp dụng heuristic.	Record có thể mất; sự kiện đầu vào syscall không chứng minh exec thành công.

Với hook `sys_enter_execve` của lab, record chứa PID, UID, GID, tên tiến trình gọi và đường dẫn được đọc từ argument. Reader của `cilium/ebpf` ở version ghim có thể đọc dữ liệu đã sẵn sàng trước khi cần chờ qua `epoll`. Little-endian là lựa chọn ABI của object `bpfel` đang build, không phải quy tắc cho mọi kernel hay chương trình eBPF. Go chỉ phân tích những record nhận được; không có record không đồng nghĩa không có hành vi.

---

2. Kernel Verifier: Kiểm tra trước khi nạp

Tại sao Linux Kernel lại cho phép mã do người dùng viết chạy trực tiếp bên trong không gian bộ nhớ của nhân?

Câu trả lời nằm ở **Bộ kiểm định nhân (Kernel Verifier)**. Trước khi bất kỳ chương trình eBPF nào được nạp vào kernel thông qua lời gọi hệ thống `SYS_BPF`, Verifier sẽ phân tích tĩnh toàn bộ mã bytecode và thực thi các quy tắc kiểm tra nghiêm ngặt:

Tiêu chuẩn an toàn Verifier	Cơ chế kiểm tra	Mục đích bảo vệ Kernel
Kiểm tra đường thực thi hữu hạn	Phân tích CFG, state và các loop mà kernel/version cho phép chứng minh an toàn.	Hạn chế đường chạy không an toàn; không biến mọi program được nạp thành không có overhead.
Kiểm soát truy cập bộ nhớ	Bắt buộc kiểm tra biên; truy cập userspace qua `bpf_probe_read_user_str()`.	Chống rò rỉ hoặc ghi đè trái phép lên không gian nhớ kernel.
Giới hạn stack của mô hình BPF đang xét	Tài liệu verifier mô tả frame 512 byte; call chain, program type và tính năng kernel còn có kiểm tra riêng.	Không dùng số này làm tổng stack budget cho mọi call chain hay version.
Theo dõi thanh ghi BPF và FP	`R10` là frame pointer chỉ đọc; verifier theo dõi kiểu và trạng thái thanh ghi BPF.	Đây là thanh ghi của máy BPF, không phải lời hứa giữ nguyên mọi thanh ghi vật lý của CPU.

> **Tình huống minh họa suy luận: Ranh giới giữa trình biên dịch C và Kernel Verifier.**

> Xét một đoạn mã C eBPF minh họa việc tra cứu và cập nhật bộ đếm trong map:

> ~~~c

> struct event *val = bpf_map_lookup_elem(&counters, &key);

> val->count++;

> ~~~

> Về mặt cú pháp C thuần túy, trình biên dịch Clang có thể coi việc truy cập `val->count` là hợp lệ và phát sinh mã bytecode ELF. Tuy nhiên, khi chương trình Go nạp bytecode này vào kernel, bộ kiểm định Verifier sẽ từ chối chương trình tại thời điểm load. Vì sao Verifier lại nghiêm ngặt hơn trình biên dịch ngôn ngữ, và người viết eBPF phải thay đổi cách viết như thế nào?

Đáp án — chỉ đọc sau khi đã tự làm

Trình biên dịch C chỉ chịu trách nhiệm kiểm tra cú pháp và hệ thống kiểu ở tầng ngôn ngữ; nó không thể biết trước liệu tại thời điểm thực thi trong nhân, phần tử ứng với `key` có thực sự tồn tại trong bộ nhớ hay không. Hàm trợ giúp `bpf_map_lookup_elem` luôn có khả năng trả về con trỏ NULL nếu tra cứu thất bại. Trong không gian nhân Linux, việc giải tham chiếu con trỏ NULL tiềm ẩn nguy cơ nghiêm trọng đối với tính toàn vẹn của hệ thống: nó có thể gây ra kernel oops, làm đổ vỡ tiến trình liên quan, làm hỏng trạng thái nhân hoặc kích hoạt kernel panic tùy thuộc vào ngữ cảnh thực thi và cấu hình nhân.

Do đó, Kernel Verifier thực hiện phân tích đường đi trừu tượng và theo dõi trạng thái thanh ghi BPF chứa kết quả trả về dưới dạng con trỏ có thể mang giá trị NULL. Khi thanh ghi chưa được chứng minh an toàn, mọi chỉ thị đọc hoặc ghi bộ nhớ thông qua thanh ghi đó đều bị Verifier chặn đứng và từ chối nạp vì lỗi truy cập bộ nhớ không hợp lệ. Để vượt qua rào cản này, lập trình viên bắt buộc phải chèn một nhánh kiểm tra điều kiện tường minh ngay sau khi tra cứu:

```
struct event *val = bpf_map_lookup_elem(&counters, &key);
if (!val) {
    return 0; // Thoát an toàn nếu không tìm thấy key
}
val->count++;
```

Nhánh kiểm tra `if (!val)` cung cấp cho Verifier bằng chứng cần thiết để chứng minh rằng trên mọi nhánh thực thi đi tới câu lệnh `val->count++`,

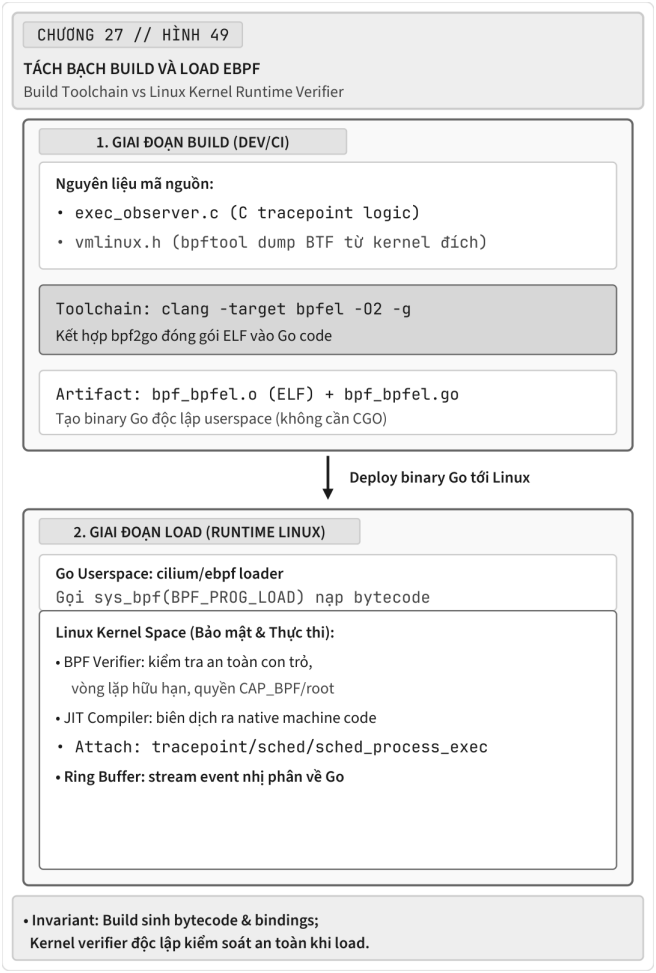
con trỏ trong thanh ghi chắc chắn khác NULL. Tuy nhiên, việc vượt qua kiểm tra con trỏ NULL chỉ giải quyết riêng lỗi truy cập con trỏ chưa kiểm chứng; nó hoàn toàn không bảo đảm chương trình sẽ vượt qua tất cả các tiêu chuẩn kiểm định khác của Verifier, chẳng hạn như giới hạn kích thước stack, tính hữu hạn của vòng lặp hay tính hợp lệ của các vùng đệm bộ nhớ truyền vào hàm trợ giúp.

---

3. Kiến trúc CGO-Free và Trình sinh mã `bpf2go`

Khi lập trình eBPF với Go, nhiều người lầm tưởng rằng bắt buộc phải cài đặt trình biên dịch Clang/LLVM cồng kềnh trên máy chủ sản xuất hoặc phải kích hoạt CGO.

Thư viện `github.com/cilium/ebpf` cung cấp đường nạp và quản lý object eBPF từ Go không phụ thuộc cgo ở phía userspace:



Hình 49 — Clang và bpf2go thuộc toolchain build. Binary Go mang object BPF đến runtime; kernel vẫn kiểm tra program, quyền và khả năng hỗ trợ trước khi load hoặc attach thành công.

Đường build dưới đây yêu cầu Linux có BTF, `bpftool`, Clang hỗ trợ target BPF và header libbpf. Chạy từ thư mục lab; `vmlinux.h` cung cấp kiểu tracepoint mà header `linux/bpf.h` không khai báo. Header được sinh từ kernel mục tiêu, không phải source Go viết tay:

```
bpftool btf dump file /sys/kernel/btf/vmlinux \
    format c > bpf/vmlinux.h
```

Chỉ thị trong `gen.go` dùng `bpf2go` của module đã ghim:

```
go run github.com/cilium/ebpf/cmd/bpf2go \
    -target bpfel -cc clang \
    bpf bpf/exec_observer.c -- \
    -I./bpf -O2 -g
```

Một là, giai đoạn phát triển: Kỹ sư viết mã C eBPF, sau đó chạy `go generate` với `bpf2go` và Clang trên máy phát triển để biên dịch mã nguồn C thành bytecode eBPF (định dạng ELF) cùng các tệp Go bindings tương ứng.

Hai là, tự động nhúng Bytecode: Công cụ `bpf2go` tự động sinh ra tệp Go chứa mã bytecode ELF đã được nhúng thẳng vào binary thông qua tính năng `//go:embed`.

Ba là, phía Go có thể nạp object eBPF đã build sẵn qua syscall Linux mà không cần cgo. Điều đó không xóa dependency của môi trường: kernel phải hỗ trợ tính năng và hook cần dùng, caller phải có quyền phù hợp, và BTF hay các asset build-time cần khớp cách ứng dụng được đóng gói. Không cần C runtime cho loader thuần Go không đồng nghĩa không cần điều kiện nào trên host.

Bốn là, test Go dùng nguồn đọc giả lập (`RecordReader`) để kiểm tra giải mã và điều phối channel mà không cần quyền nạp BPF. Lab chưa cung cấp một ứng dụng load/attach hoàn chỉnh; object thật, adapter ring buffer và quyền trên host phải được nối và kiểm chứng riêng. `CollectionSpec` dựng trong test không thay cho ELF được compiler tạo.

---

4. Kênh truyền dữ liệu tốc độ cao: BPF Ring Buffer

Để đưa dữ liệu từ Kernel lên Go Userspace, eBPF cung cấp cấu trúc dữ liệu **BPF Ring Buffer** (`BPF_MAP_TYPE_RINGBUF`). Khác với perf event array thường dùng buffer theo CPU, Ring Buffer dùng vùng nhớ chung và có thể giữ thứ tự reservation giữa producer. Đây hữu ích cho event tuần tự như fork/exec/exit; nó không biến record thành đồng hồ toàn cục chính xác, cũng không cho phép suy ra toàn bộ causal order của event song song trên nhiều CPU.

Reader ở userspace ánh xạ ring buffer bằng `mmap` để lấy record; không cần một syscall riêng cho mỗi record, nhưng vẫn có chi phí đồng bộ, polling/wakeup và decode. Lab reserve bộ nhớ trong ring rồi submit.

Reserve có thể thất bại vì đầy hoặc điều kiện đường chạy; khi đó chương trình bỏ record. Cần đếm loss nếu policy cần biết tín hiệu thiếu, không coi bỏ record là bảo đảm hệ thống không bị ảnh hưởng.

---

5. Viết mã Kernel: Bắt sự kiện `sys_enter_execve`

Dưới đây là phần khai báo cấu trúc sự kiện và bản đồ Ring Buffer trong C eBPF (`labs/part27-ebpf-observer/bpf/exec_observer.c`):

```
// +build ignore
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>

char __license[] SEC("license") = "Dual MIT/GPL";

struct exec_event {
    __u32 pid;
    __u32 uid;
    __u32 gid;
    char  comm[16];
    char  filename[128];
};

_Static_assert(sizeof(struct exec_event) == 156,
               "event ABI mismatch");

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 256 * 1024); // 256KB buffer
} events SEC(".maps");
```

Tiếp theo là hàm móc vào tracepoint `sys_enter_execve` để bắt thông tin mỗi khi có lời gọi thực thi nhị phân:

```
SEC("tracepoint/syscalls/sys_enter_execve")
int trace_execve(struct trace_event_raw_sys_enter *ctx) {
    struct exec_event *event;

    // Cấp phát trực tiếp trong ringbuffer
    event = bpf_ringbuf_reserve(
        &events, sizeof(struct exec_event), 0
    );
    if (!event) {
        return 0; // Buffer đầy hoặc bộ nhớ bận
    }

    // Xóa toàn record, kể cả phần đuôi chuỗi chưa được ghi
    __builtin_memset(event, 0, sizeof(*event));
    __u64 pid_tgid = bpf_get_current_pid_tgid();
    event->pid = (__u32)(pid_tgid >> 32);

    __u64 uid_gid = bpf_get_current_uid_gid();
    event->uid = (__u32)(uid_gid);
    event->gid = (__u32)(uid_gid >> 32);
```

```
    // Đọc tên của tiến trình gọi (calling task)
    if (bpf_get_current_comm(
        &event->comm, sizeof(event->comm)
    ) < 0) {
        bpf_ringbuf_discard(event, 0);
        return 0;
    }

    // Đọc đường dẫn file nhị phân đích từ tham số syscall
    const char *fn = (const char *)ctx->args[0];
    if (bpf_probe_read_user_str(
        &event->filename, sizeof(event->filename), fn
    ) < 0) {
        bpf_ringbuf_discard(event, 0);
        return 0;
    }

    // Gửi sự kiện lên Userspace
    bpf_ringbuf_submit(event, 0);
    return 0;
}
```

Đặc điểm kỹ thuật trong mã nguồn C eBPF

Hook `sys_enter_execve` biểu thị lời gọi bắt đầu, không chứng minh exec thành công. Lỗi `ENOENT` hay `EACCES` có thể xuất hiện sau đó. Chương trình tại hook có cơ hội tạo record, nhưng reserve, đọc argument và việc truyền record vẫn có thể thất bại; không hứa mỗi attempt đều được ghi nhận đầy đủ.

Record được khởi tạo toàn bộ rồi mới điền trường. Nếu helper đọc tên hoặc đường dẫn lỗi, chương trình discard thay vì publish dữ liệu thiếu; nếu buffer chuỗi quá ngắn, helper có thể trả chuỗi bị cắt. Lab chưa có bộ đếm loss hoặc cờ truncation. PID/TGID ở đây là định danh mà helper kernel trả về, không hứa trùng PID nhìn thấy từ mọi container namespace. `Timestamp` của Go là giờ decode, không phải thời điểm kernel bắt đầu syscall.

Bên cạnh đó, hàm `bpf_get_current_comm` tại thời điểm này phản ánh tên của tiến trình đang phát lệnh gọi (calling task như `bash`, `python` hoặc `containerd`), trong khi đường dẫn tệp nhị phân đích phải được trích xuất từ tham số `ctx->args[0]`. Để phân giải PID tương thích với không gian người dùng, mã nguồn dịch bit phải 32 bit từ giá trị 64-bit của `bpf_get_current_pid_tgid()`, bởi trong nhân Linux định danh Thread Group ID (TGID) mới tương ứng với PID của tiến trình. Cuối cùng, việc đọc đường dẫn chuỗi người dùng bắt buộc phải thông qua hàm trợ giúp `bpf_probe_read_user_str` nhằm ngăn ngừa lỗi vi phạm trang nhớ (page fault) khi con trỏ trỏ tới vùng địa chỉ chưa hợp lệ.

---

6. Xây dựng Trình giải mã nhị phân và Thu thập trong Go

Khi sự kiện từ kernel được đẩy vào Ring Buffer, ứng dụng Go nhận được một mảng byte nhị phân thô (`[]byte`). Nhiệm vụ của chúng ta là giải mã mảng byte này thành struct Go với hiệu năng tối đa.

Mô hình dữ liệu và giải mã ABI (`event.go`)

```
package ebpfobserver

import (
    "bytes"
    "encoding/binary"
    "errors"
    "fmt"
    "time"
)

// EventPayloadSize đại diện cho kích thước struct C (156B).
const EventPayloadSize = 156

type ExecEvent struct {
    PID       uint32    `json:"pid"`
    UID       uint32    `json:"uid"`
    GID       uint32    `json:"gid"`
    Comm      string    `json:"comm"`
    Filename  string    `json:"filename"`
    Timestamp time.Time `json:"timestamp"`
}
```

Hàm giải mã bóc tách từng trường dữ liệu theo chuẩn little-endian:

```
// DecodeExecEvent giải mã mảng byte thô từ Ring Buffer.
func DecodeExecEvent(data []byte) (*ExecEvent, error) {
    if len(data) < EventPayloadSize {
        return nil, fmt.Errorf(
            "truncated event: want %d bytes, got %d",
            EventPayloadSize, len(data),
        )
    }

    pid := binary.LittleEndian.Uint32(data[0:4])
    uid := binary.LittleEndian.Uint32(data[4:8])
    gid := binary.LittleEndian.Uint32(data[8:12])

    commBytes := data[12:28]
    fileBytes := data[28:156]

    return &ExecEvent{
        PID:       pid,
        UID:       uid,
        GID:       gid,
        Comm:      parseCString(commBytes),
        Filename:  parseCString(fileBytes),
        Timestamp: time.Now().UTC(),
    }, nil
}

// parseCString cắt bỏ các byte null kết thúc chuỗi của C.
func parseCString(b []byte) string {
    idx := bytes.IndexByte(b, 0)
    if idx >= 0 {
        return string(b[:idx])
    }
    return string(b)
}
```

---

7. Điều phối luồng sự kiện và Phát hiện Bất thường An ninh (`observer.go`)

Để kiểm thử được trên mọi nền tảng mà không phụ thuộc vào quyền root Linux khi chạy unit test, chúng ta trừu tượng hóa nguồn đọc bằng interface `RecordReader`:

```
// RecordReader trừu tượng hóa nguồn đọc từ BPF Ring Buffer.
type RecordReader interface {
    Read() ([]byte, error)
    Close() error
}

type Observer struct {
    reader    RecordReader
    events    chan *ExecEvent
    errs      chan error
    done      chan struct{}
    closeOnce sync.Once
}

func NewObserver(
    reader RecordReader, bufferSize int,
) *Observer {
    if bufferSize <= 0 {
        bufferSize = 128
    }
    return &Observer{
        reader: reader,
        events: make(chan *ExecEvent, bufferSize),
        errs:   make(chan error, 16),
        done:   make(chan struct{}),
    }
}
```

Đoạn rút gọn dưới đây cho thấy goroutine xử lý; `readLoop`, `sendErr` và `sendEvent` biểu diễn các nhánh tương ứng trong implementation lab, không phải các method đã được khai báo trong source. Khi thử chương trình, dùng `observer.go` đầy đủ, gồm cả goroutine gọi `Close` khi context bị hủy. `Start` chỉ được gọi một lần cho một observer:

```
func (o *Observer) Start(
    ctx context.Context,
) (<-chan *ExecEvent, <-chan error) {
    go func() {
        defer close(o.events)
        defer close(o.errs)
        o.readLoop(ctx)
    }()
    return o.events, o.errs
}
```

Vòng lặp đọc dữ liệu từ Ring Buffer và đẩy vào channel với khả năng hủy qua context:

```
func (o *Observer) readLoop(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            return
        case <-o.done:
            return
        default:
            data, err := o.reader.Read()
            if err != nil {
                if errors.Is(err, io.EOF) ||
                    errors.Is(err, ErrObserverClosed) {
                    return
                }
                o.sendErr(ctx, err)
                continue
            }

            event, decodeErr := DecodeExecEvent(data)
            if decodeErr != nil {
                o.sendErr(ctx, decodeErr)
                continue
            }
            o.sendEvent(ctx, event)
        }
    }
}
```

`context` chỉ mở đường thoát ở `select`; nó không chui vào interface `RecordReader.Read()`. Vì vậy lab đăng ký một nhánh cancellation gọi `reader.Close()`. Contract cần có là `Close` làm một `Read` đang block trở về; fake reader mới trong test block thật rồi được `Close` giải phóng. Nếu source reader khác không có contract đó, phải bọc hoặc đổi API thay vì hứa cancellation tức thì.

Động cơ phát hiện bất thường an ninh (Heuristic Security Rules)

Khi nhận được sự kiện, chúng ta đối chiếu với các quy tắc an ninh phỏng đoán. Cần lưu ý: Đây là các **quy tắc phỏng đoán (Heuristics)** hỗ trợ giám sát, không phải lá chắn bảo mật tuyệt đối, bởi kẻ tấn công nâng cao có thể lẩn tránh bằng cách gọi `execveat`, nạp nhị phân từ RAM qua `memfd_create`, hoặc dùng symlink:

```
type SecurityAlert struct {
    Event  *ExecEvent
    Rule   string
    Reason string
}

func checkBasicRules(e *ExecEvent) *SecurityAlert {
    fn := strings.ToLower(e.Filename)
    comm := strings.ToLower(e.Comm)

    // Luật 1: Thực thi từ thư mục tạm (/tmp, /dev/shm)
    if strings.HasPrefix(fn, "/tmp/") ||
        strings.HasPrefix(fn, "/dev/shm/") {
        return &SecurityAlert{
            Event:  e,
            Rule:   "SEC01_EXEC_FROM_TEMP_DIRECTORY",
            Reason: "process executed from writable temp path",
        }
    }

    // Luật 2: Daemon web tự ý mở shell tương tác (RCE)
    isShell := fn == "/bin/sh" || fn == "/bin/bash"
    isWeb := comm == "nginx" || comm == "node" ||
        comm == "python"
    if isShell && isWeb {
        return &SecurityAlert{
            Event:  e,
            Rule:   "SEC02_SHELL_FROM_WEBSERVER",
            Reason: "interactive shell spawned from web daemon",
        }
    }
    return nil
}
```

Tiếp theo là quy tắc phát hiện các tiện ích mạng chạy bằng quyền root:

```
func DetectSecurityAnomalies(e *ExecEvent) *SecurityAlert {
    if e == nil {
        return nil
    }
    if alert := checkBasicRules(e); alert != nil {
        return alert
    }

    // Luật 3: Tiện ích dò quét mạng chạy với quyền root (UID 0)
    fn := strings.ToLower(e.Filename)
    isNet := strings.HasSuffix(fn, "/nc") ||
        strings.HasSuffix(fn, "/ncat")
    if isNet && e.UID == 0 {
        return &SecurityAlert{
            Event:  e,
            Rule:   "SEC03_ROOT_NETWORK_RECON",
            Reason: "network recon tool executed as root",
        }
    }
    return nil
}
```

---

8. Kiểm chứng Lab thực tế (`labs/part27-ebpf-observer`)

Bộ kiểm thử tại `labs/part27-ebpf-observer/observer_test.go` vận hành độc lập (phân loại kiểm chứng: `UNIT_TESTED` / `MODEL_ONLY` đối với mô hình streaming channel, ABI decoding 156 bytes, heuristic anomaly detection và kiểm tra cấu trúc BPF Spec mà không đòi hỏi quyền root của Linux). Chạy bộ kiểm thử hoàn chỉnh với cờ kiểm tra tương tranh dữ liệu:

```
go test -v -race ./...
```

Một transcript lịch sử của sáu test dưới đây chỉ minh họa tên test; thời gian không phải số đo cho lần chạy hiện tại hay cho kernel thật:

```
=== RUN   TestEncodeDecodeExecEvent
--- PASS: TestEncodeDecodeExecEvent (0.00s)
=== RUN   TestDecodeExecEventTruncated
--- PASS: TestDecodeExecEventTruncated (0.00s)
=== RUN   TestObserverStreaming
--- PASS: TestObserverStreaming (0.00s)
=== RUN   TestObserverContextCancellation
--- PASS: TestObserverContextCancellation (0.00s)
=== RUN   TestDetectSecurityAnomalies
--- PASS: TestDetectSecurityAnomalies (0.00s)
=== RUN   TestModeledCollectionSpec
--- PASS: TestModeledCollectionSpec (0.00s)
PASS
ok      part27-ebpf-observer   0.742s
```

Phân tích kết quả kiểm thử và Ranh giới kiểm chứng:

Bộ kiểm thử được phân loại ở cấp độ `UNIT_TESTED` kết hợp `MODEL_ONLY`. Biên dịch object ELF là bước build, cần compiler và các header hoặc type asset mà mã C sử dụng; nó không tự đòi hỏi quyền nạp vào kernel. Load/attach tracepoint cần kernel Linux hỗ trợ hook và quyền phù hợp với kernel, loại program cùng chính sách bảo mật. Lab này chưa chạy bước ấy trên Linux (`SKIPPED_WITH_REASON`); thiếu kernel headers không phải lý do chung khiến object đã build không thể được nạp.

Sáu kịch bản kiểm thử kiểm tra các contract cục bộ: giải mã little-endian theo layout 156 byte của lab; từ chối dữ liệu bị cắt ngắn (`Truncated`); điều phối channel; ngắt luồng đọc qua Context; áp dụng heuristic cảnh báo; và dựng `CollectionSpec` từ `cilium/ebpf` với Map `RingBuf`, Program `TracePoint`. Việc không phát hiện race hay goroutine bị giữ lại trên những đường chạy được kiểm tra không phải bằng chứng cho mọi lịch thực thi. Bộ test cũng không chứng minh load/attach thành công, thu đủ sự kiện hoặc overhead chấp nhận được trên kernel production; những điều đó cần kiểm chứng riêng trong môi trường Linux mục tiêu.

---

9. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Dùng stack vượt giới hạn của kernel/program type.	Verifier có thể từ chối khi compiler hiện thực dữ liệu trên stack quá lớn; một khai báo bị tối ưu bỏ không tự tạo stack use.	Kiểm tra bytecode và verifier log; với event ring buffer của lab, reserve/submit tránh đặt toàn bộ event trên stack.
Lấy nhầm 32 bit thấp của `bpf_get_current_pid_tgid()`.	Thu được Thread ID (TID) thay vì Process ID (PID), khiến log hiển thị PID sai lệch hoàn toàn.	Luôn dịch bit phải 32 bit: `(__u32)(pid_tgid >> 32)` để lấy đúng PID của tiến trình.
Xử lý sự kiện userspace quá chậm trong vòng lặp đọc.	Tràn bộ đệm Ring Buffer trong kernel, khiến các sự kiện quan trọng bị âm thầm đánh rơi (dropped).	Sử dụng buffered channel và mô hình Worker Pool ở tầng Go để tiêu thụ sự kiện với tốc độ cao.
Chạy ứng dụng thiếu quyền Linux capabilities.	Load/attach có thể trả `EPERM`.	Quyền cần thiết phụ thuộc kernel version, cấu hình LSM và loại program; kiểm tra policy của node. Không copy một bộ capability như công thức chung.

---

10. Bài tập thực hành thiết kế Observer eBPF

Thử thách 1: Bộ đếm tần suất thực thi tiến trình (Exec Rate Limiter)

Yêu cầu: Với fixture này, đếm exec attempt theo tên `Comm` trong cửa sổ cố định khoảng 1 giây, cảnh báo khi vượt 50. Đây là nhóm theo tên, không phải identity của một process; nhiều task có thể cùng Comm. Muốn đếm theo process cần PID/TGID cùng lifecycle và namespace phù hợp. Test phải phân biệt cửa sổ cố định với trượt; ngưỡng 50 chỉ là cấu hình bài tập.

Thử thách 2: Bóc tách tham số dòng lệnh từ Syscall (Argv Extraction)

Yêu cầu: `ctx->args[1]` trỏ tới mảng pointer `argv`. Thiết kế việc đọc có giới hạn từng pointer/chuỗi ở phía kernel và wire format với delimiter NUL. Ở Go, decode record thành `Args []string` mà không split hay trim khoảng trắng. Hook hiện tại chưa thu argv; đây là bài tập thiết kế thêm, không phải khả năng đã có.

11. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Bộ phát hiện tần suất thực thi

```
type ForkBombDetector struct {
    mu       sync.Mutex
    counts   map[string]int
    lastTick time.Time
}

func NewForkBombDetector() *ForkBombDetector {
    return &ForkBombDetector{
        counts:   make(map[string]int),
        lastTick: time.Now(),
    }
}

func (d *ForkBombDetector) Check(
    e *ExecEvent, threshold int,
) bool {
    d.mu.Lock()
    defer d.mu.Unlock()

    now := time.Now()
    if now.Sub(d.lastTick) >= time.Second {
        // Reset bộ đếm mỗi giây
        d.counts = make(map[string]int)
        d.lastTick = now
    }

    d.counts[e.Comm]++
    return d.counts[e.Comm] > threshold
}
```

Lời giải Thử thách 2: Bóc tách đối số dòng lệnh

```
type ExecEventExtended struct {
    ExecEvent
    Args []string `json:"args"`
}

func ParseNULSeparatedArgs(raw []byte) []string {
    if len(raw) == 0 {
        return nil
    }
    // Payload đã được validate, không gồm padding cuối buffer.
    if raw[len(raw)-1] == 0 {
        raw = raw[:len(raw)-1]
    }
    tokens := bytes.Split(raw, []byte{0})
    args := make([]string, len(tokens))
    for i, tok := range tokens {
        args[i] = string(tok) // Giữ cả argument rỗng.
    }
    return args
}
```

`argv` là mảng con trỏ tới các chuỗi C; wire format phải ghi số argument, độ dài payload đã dùng và cờ truncation, bên cạnh giới hạn capture. Decoder trên chỉ nhận payload hợp lệ: mỗi argument kết thúc bằng NUL, không truyền toàn bộ buffer còn padding. `a\x00\x00` phải thành hai argument `"a"`, `""`; bỏ mọi token rỗng sẽ làm mất đối số hợp lệ. Khoảng trắng cũng là dữ liệu, không dùng để split hay trim. Hook hiện tại chỉ thu filename; đây là thiết kế mở rộng, không phải khả năng capture argv đã có.

---

eBPF bổ sung một điểm quan sát mạnh, có giới hạn và cần được đo đạc; heuristic của lab chỉ tạo tín hiệu để điều tra, không phải cơ chế ngăn chặn hay bằng chứng xâm nhập. Trước khi dùng trên node thật, hãy xác minh kernel, capability, hook, tỷ lệ drop và policy dữ liệu của chính môi trường đó.

Chương 28 chuyển từ điểm quan sát kernel sang boundary công cụ Agent: ai được gọi, target nào được phép và bằng chứng nào cho phép một thay đổi.

MCP chuẩn hóa giao tiếp, không thay thế authorization hay trách nhiệm vận hành. Chương 29 tiếp tục xét bằng chứng và giới hạn của tự động hóa.

Chương 28 — MCP và AIOps bằng Go: trao công cụ cho Agent mà không trao toàn quyền

Xét một Agent có tool đọc health, xem log và đề xuất restart service. Đọc dữ liệu và thay đổi hệ thống có tác động khác nhau; nối chúng qua một protocol không tự định nghĩa ai được phép làm gì. Chương này bắt đầu từ contract quyền hành động, không từ độ tự tin của câu trả lời.

Khi tool nối tới production, cần hỏi cụ thể:

> **Nếu trao cho Agent quyền truy cập vào một công cụ, làm sao bạn bảo đảm tác tử không bị tấn công tiêm lời nhắc (Prompt Injection) để vô tình hoặc cố ý phá hủy cơ sở dữ liệu, rò rỉ token truy cập đám mây, hay làm sập toàn bộ hạ tầng?**

MCP chuẩn hóa ranh giới giao tiếp, nhưng không tự cấp quyền hay làm tool an toàn. Chương này dùng thư viện chính thức `github.com/modelcontextprotocol/go-sdk/mcp` (v1.8.0) để dựng một server sư phạm có đặc quyền tối thiểu, target registry và audit trail. Policy authorization, trust của target registry và actuator thật vẫn là trách nhiệm của deployment.

---

1. Mental Model: Ranh giới phòng thủ đa tầng từ Agent đến Hạ tầng

Mô hình trung tâm của lab là mỗi lời gọi phải đi qua các kiểm tra riêng. Danh tính do transport xác thực, không do argument tự khai; schema hợp lệ chưa đủ để cấp quyền thực thi:

Kiểm tra	Đối tượng thật của lab	Không suy ra
Giao thức/ transport	MCP SDK; stdio khi chạy server, transport trong memory khi test.	In-memory fixture không phải chứng minh authentication của HTTP deployment.
Schema	Struct và constraint của argument.	Chuỗi đúng schema chưa là target được phép hay instruction đáng tin.
Authorization	Identity trong session context, role và action.	Caller tự ghi `operator` vào argument không tạo quyền.
Target và side effect	Target registry được tin, handler và actuator đã cấu hình.	Allowlist tên không bảo vệ nếu chính registry hay credential bị đổi trái phép.
Audit	Ghi cả quyết định cho phép/từ chối ở boundary tương ứng.	Log không chỉ xảy ra sau hành động thành công; không có bảo đảm bền vững từ interface logger.

Stdio tránh mở listener HTTP của chính server nhưng không bảo vệ trước process local có quyền thích hợp. Schema kiểm tra constraint đã khai báo; quyền dựa trên identity tin cậy và action/target cụ thể. Registry target của lab là cấu hình tin cậy, không phải bất kỳ URL Agent đưa vào. Audit chỉ có ích khi recorder, đường lưu và phản ứng khi mất log có contract riêng.

---

2. Model Context Protocol: Thư viện Go SDK Chính thức

MCP là giao thức mở cho client trao đổi với server cung cấp tool và dữ liệu. Nó định nghĩa thông điệp và lifecycle kết nối; mức phổ biến hay nhãn "tiêu chuẩn công nghiệp" không chứng minh policy của một deployment đúng.

Về bản chất kỹ thuật, MCP vận hành trên nền giao thức **JSON-RPC 2.0**. Máy chủ Go công bố danh sách các công cụ khả dụng kèm mô tả chức năng và JSON Schema qua phương thức `tools/list`. Khi tác tử quyết định sử dụng một công cụ, nó phát thông điệp `tools/call` chứa tên công cụ và các tham số tương ứng.

Dự án thực hành tại `labs/part28-mcp-ops-tools/` ghim SDK v1.8.0. Theo compatibility matrix của chính SDK, bản này hỗ trợ và thương lượng revision mới nhất `2026-07-28` với peer hỗ trợ nó, đồng thời còn tương thích các revision cũ. Vì protocol revision được thương lượng theo kết nối, server không được tự suy ra wire behavior chỉ từ version của dependency.

SDK vẫn cho phép typed handler, JSON Schema từ struct tags và `ReceivingMiddleware` cho các policy cục bộ.

Giao vận Stdio và Kết nối Cục bộ

Lab có đường Stdio để client khởi chạy server và trao đổi qua stdin/stdout, cùng `NewInMemoryTransports` cho test. Stdio không cần endpoint HTTP hay chứng chỉ TLS cho pipe cục bộ, nhưng vẫn cần kiểm soát executable, quyền process và cấu hình client. Không gán một latency hoặc ưu tiên transport chung cho mọi hệ DevOps.

---

3. Cạm bẫy thiết kế: SSRF qua URL tùy ý và Giải pháp Target Allowlist

Một sai lầm rất phổ biến của các kỹ sư khi mới thiết kế MCP Server cho AIOps là cho phép Agent truyền trực tiếp một URL tùy ý:

```
{
  "name": "query_service_health",
  "arguments": {"target_url": "https://..."}
}
```

Đây là một khiếm khuyết an ninh nghiêm trọng. Các bộ xác thực JSON Schema chỉ kiểm tra định dạng hình thức của chuỗi ký tự mà không hiểu ngữ nghĩa của địa chỉ đích. Khi tác tử bị tấn công tiêm lời nhắc hoặc gặp hiện tượng ảo giác, nó có thể gửi yêu cầu nhắm tới địa chỉ siêu dữ liệu nhạy cảm của điện toán đám mây (`http://169.254.169.254/latest/meta-data/...`), cổng quản trị nội bộ trên `localhost:9090`, hoặc các máy chủ cơ sở dữ liệu chưa mã hóa trong mạng riêng (VPC).

Để giảm bề mặt SSRF do **caller** chọn, agent không được truyền URL trực tiếp mà chỉ truyền `target_id` đã được duyệt trước. Server ánh xạ ID sang URL từ target registry và từ chối ID lạ. Đây không tự chứng minh URL trong registry là an toàn: registry bị ghi sai hoặc bị xâm phạm vẫn là một trust

boundary riêng, cần review cấu hình, DNS/egress policy và quyền sửa registry.

---

4. Cấu hình Target Allowlist và Đăng ký Công cụ Typed Go

Dưới đây là cấu trúc định danh mục tiêu và mô hình tham số đầu vào được trích xuất từ `labs/part28-mcp-ops-tools/server.go`:

```
type HealthTarget struct {
    ID          string `json:"id"`
    ServiceName string `json:"serviceName"`
    URL         string `json:"url"`
}

// QueryHealthInput định nghĩa schema đầu vào
type QueryHealthInput struct {
    TargetID string `json:"target_id"`
}

// RestartServiceInput định nghĩa tham số mutating tool
type RestartServiceInput struct {
    ServiceName  string `json:"service_name"`
    ChangeTicket string `json:"change_ticket"`
}
```

Các trường struct có thể được bổ sung thêm tag `jsonschema` để cung cấp phần mô tả chi tiết cho từng tham số khi máy chủ MCP công bố danh sách công cụ tới tác tử qua lệnh `tools/list`.

---

5. Phân quyền vai trò theo Ngữ cảnh Phiên và Rào chắn Đột biến

Policy phải lấy quyền của caller từ nguồn tin cậy:

> **Vai trò của Agent phải được xác định qua Ngữ cảnh Phiên (Session Context), không để Agent tự khai báo vai trò trong tham số JSON của công cụ.**

Nếu để Agent gửi `{"role": "admin"}` trong JSON arguments, việc prompt injection có thể dẫn tới việc Agent mạo danh quản trị viên để chiếm quyền điều khiển hệ thống.

Gắn danh tính vào context.Context

```
type Role string

const (
    RoleObserver Role = "observer" // Chỉ đọc
    RoleOperator Role = "operator" // Đột biến có phiếu
    RoleAdmin    Role = "admin"    // Toàn quyền
)

type contextKey string
const roleContextKey contextKey = "mcp_caller_role"

func WithCallerRole(
    ctx context.Context, role Role,
) context.Context {
    return context.WithValue(ctx, roleContextKey, role)
}

func CallerRoleFromContext(ctx context.Context) Role {
    if r, ok := ctx.Value(roleContextKey).(Role); ok {
        return r
    }
    return RoleObserver // Mặc định đặc quyền tối thiểu
}
```

Một danh tính thật không tự sinh ra quyền tool

Test session của chương giữ nguyên mức `MOCK_VERIFIED` cho nguồn identity. Muốn nhìn thấy boundary mạng thật mà không kéo MCP vào mọi phép thử, chạy lab TLS `part11-request-path/identity` trước: reader có verified certificate vẫn bị từ chối mutation; header tự khai admin không đổi quyết định. Đó là bằng chứng handshake và policy HTTP cục bộ, chưa phải kiểm định authorization của Streamable HTTP MCP.

Khi deployment dùng TLS termination ở proxy, handler có thể không còn `r.TLS.VerifiedChains` của client gốc. Không copy identity header từ

Internet vào session context; phải xác định proxy nào được tin, bảo vệ đường proxy–service và kiểm tra cơ chế truyền identity của deployment. Stdio, OAuth hay mTLS có cách thiết lập caller khác nhau, nhưng cùng cần ánh xạ caller đã xác thực sang action/target được phép. Lab certificate không thay đổi policy MCP và không tuyên bố mọi MCP server cần mTLS.

Tách bạch hai nhóm công cụ và Rào chắn Đột biến

Policy của lab cho `observer` gọi `query_service_health` trên target đã đăng ký và cấm `restart_service`. Đó không phải quyền "tự do truy vấn" chung: hệ thống thật cần giới hạn dữ liệu, target, budget và identity. Vai trò `operator` vẫn phải qua `ChangeAuthorizer` rồi `ServiceActuator`; tên role không tự đủ để cho phép thay đổi.

```
type ServiceActuator interface {
    RestartService(
        ctx context.Context, serviceName string,
    ) error
}

type ChangeAuthorizer interface {
    AuthorizeChange(
        ctx context.Context, role Role, svc, ticket string,
    ) error
}
```

---

6. Xây dựng OpsServer trên nền tảng Official MCP Go SDK

Dưới đây là các phần chính của `OpsServer` trong `labs/part28-mcp-ops-tools/server.go`. Snippet cho thấy các boundary của lab; các phần wiring còn lại nằm trong source:

6.1. Khởi tạo server và ranh giới danh tính

Trong kiến trúc của MCP Go SDK, `AddReceivingMiddleware` cho phép can thiệp vào mọi thông điệp RPC gửi đến trước khi điều phối tới handler cụ thể. Ta sử dụng middleware này để bóc tách định danh caller từ phiên làm việc (`Session Context`) và tiêm vai trò bảo mật vào `context.Context`. Cần lưu ý rằng `context.Context` không tự tạo ra authenticated identity; middleware trong bài lab lấy role từ ánh xạ phiên giả lập để phục vụ kiểm thử mô hình. Trên môi trường production, tầng transport hoặc identity provider phải xác thực danh tính caller trước khi tiêm role vào context:

```
type OpsServer struct {
    mcpServer   *mcp.Server
    httpClient  *http.Client
    actuator    ServiceActuator
    authorizer  ChangeAuthorizer
    targets     map[string]HealthTarget

    mu          sync.RWMutex
    sessionRole map[string]Role
    auditLog    []AuditRecord
}

func NewOpsServer(
    client *http.Client,
    actuator ServiceActuator,
    authorizer ChangeAuthorizer,
    targets []HealthTarget,
) *OpsServer {
    if client == nil {
        client = &http.Client{Timeout: 5 * time.Second}
    }
    targetMap := make(map[string]HealthTarget, len(targets))
    for _, t := range targets {
        targetMap[t.ID] = t
    }
```

```
    server := mcp.NewServer(
        &mcp.Implementation{
            Name: "ops-mcp-server", Version: "v1.8.0",
        },
        &mcp.ServerOptions{
            Instructions: "AIOps server allowlist",
        },
    )

    ops := &OpsServer{
        mcpServer:   server,
        httpClient:  client,
        actuator:    actuator,
        authorizer:  authorizer,
        targets:     targetMap,
        sessionRole: make(map[string]Role),
    }

    // Test mapping only; not production authentication.
    // Real transport authenticates before dispatch.
    server.AddReceivingMiddleware(
        func(next mcp.MethodHandler) mcp.MethodHandler {
            return func(
```

```
                ctx context.Context,
                method string,
                req mcp.Request,
            ) (mcp.Result, error) {
                role := CallerRoleFromContext(ctx)
                sess := req.GetSession()
                if role == RoleObserver && sess != nil {
                    ops.mu.RLock()
                    id := sess.ID()
                    sessRole, found := ops.sessionRole[id]
                    ops.mu.RUnlock()
                    if found {
                        role = sessRole
                    }
                }
                ctx = WithCallerRole(ctx, role)
                return next(ctx, method, req)
            }
        },
    )

    ops.registerTools()
    return ops
}
```

6.2. Đăng ký Công cụ Kiểm tra Sức khỏe và Rào chắn SSRF

Công cụ `query_service_health` được đăng ký qua `mcp.AddTool`. SDK tự động giải mã tham số vào `QueryHealthInput` và chuyển giao quyền kiểm soát cho closure handler:

```
mcp.AddTool(s.mcpServer, &mcp.Tool{
    Name:        "query_service_health",
    Description: "Ktra sức khỏe qua target_id allowlist",
}, func(
    ctx context.Context,
    req *mcp.CallToolRequest,
    input QueryHealthInput,
) (*mcp.CallToolResult, any, error) {
    role := CallerRoleFromContext(ctx)
    target, found := s.targets[input.TargetID]
    if !found {
        errStr := fmt.Sprintf(
            "target %q blocked by policy", input.TargetID,
        )
        s.RecordAudit(
            role, "query_service_health",
            map[string]any{"target_id": input.TargetID},
            "DENY", errStr,
        )
        return &mcp.CallToolResult{
            IsError: true,
            Content: []mcp.Content{
                &mcp.TextContent{Text: errStr},
            },
        }, nil, nil
    }

    start := time.Now()
```

```
    httpReq, err := http.NewRequestWithContext(
        ctx, http.MethodGet, target.URL, nil,
    )
    if err != nil {
        errStr := fmt.Sprintf("req err: %v", err)
        s.RecordAudit(
            role, "query_service_health",
            map[string]any{"target_id": input.TargetID},
            "DENY", errStr,
        )
        return &mcp.CallToolResult{
            IsError: true,
            Content: []mcp.Content{
                &mcp.TextContent{Text: errStr},
            },
        }, nil, nil
    }

    resp, err := s.httpClient.Do(httpReq)
    if err != nil {
        errStr := fmt.Sprintf("health check failed: %v", err)
        s.RecordAudit(
            role, "query_service_health",
            map[string]any{"target_id": input.TargetID},
            "DENY", errStr,
        )
        return &mcp.CallToolResult{
            IsError: true,
```

```
            Content: []mcp.Content{
                &mcp.TextContent{Text: errStr},
            },
        }, nil, nil
    }
    defer resp.Body.Close()

    duration := time.Since(start).Round(time.Millisecond)
    health := "unhealthy"
    if resp.StatusCode >= http.StatusOK &&
        resp.StatusCode < http.StatusMultipleChoices {
        health = "healthy"
    }
    resText := fmt.Sprintf(
        "Service %s (%s) %s: HTTP %d",
        target.ServiceName, target.ID, health, resp.StatusCode,
    )
    s.RecordAudit(
        role, "query_service_health",
        map[string]any{"target_id": input.TargetID},
        "ALLOW", resText,
    )
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: resText},
        },
    }, nil, nil
})
```

6.3. Công cụ gây đột biến: dependency bắt buộc, không có đường mặc định cho phép

Với công cụ `restart_service`, `ChangeAuthorizer` và `ServiceActuator` là dependency bắt buộc. Thiếu một trong hai phải trả `DENY`; không được coi `nil` là "không cần kiểm tra" rồi báo restart thành công. Khi cả hai có mặt, yêu cầu phải qua `AuthorizeChange` trước khi kích hoạt `RestartService`:

```
mcp.AddTool(s.mcpServer, &mcp.Tool{
    Name:        "restart_service",
    Description: "Restart dịch vụ có điều kiện, cần ticket",
}, func(
    ctx context.Context,
    req *mcp.CallToolRequest,
    input RestartServiceInput,
) (*mcp.CallToolResult, any, error) {
    role := CallerRoleFromContext(ctx)
    args := map[string]any{
        "service_name":  input.ServiceName,
        "change_ticket": input.ChangeTicket,
    }

    if s.authorizer == nil || s.actuator == nil {
        return deny(
            "required authorization or actuator missing",
        )
    }
    if err := s.authorizer.AuthorizeChange(
        ctx, role, input.ServiceName, input.ChangeTicket,
    ); err != nil {
            s.RecordAudit(
                role, "restart_service", args,
                "DENY", err.Error(),
            )
            return &mcp.CallToolResult{
                IsError: true,
                Content: []mcp.Content{
                    &mcp.TextContent{Text: err.Error()},
                },
```

```
            }, nil, nil
    }

    if err := s.actuator.RestartService(
        ctx, input.ServiceName,
    ); err != nil {
            errStr := fmt.Sprintf("restart failed: %v", err)
            s.RecordAudit(
                role, "restart_service", args, "DENY", errStr,
            )
            return &mcp.CallToolResult{
                IsError: true,
                Content: []mcp.Content{
                    &mcp.TextContent{Text: errStr},
                },
            }, nil, nil
    }

    resText := fmt.Sprintf(
        "Service %s restarted under ticket %s",
        input.ServiceName, input.ChangeTicket,
    )
    s.RecordAudit(
        role, "restart_service", args, "ALLOW", resText,
    )
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: resText},
        },
    }, nil, nil
})
```

6.4. Ghi Nhật ký Kiểm toán Có Cấu trúc

Để theo dõi mọi hành vi của tác tử, máy chủ lưu vết các quyết định vào cấu trúc `AuditRecord`:

```
type AuditRecord struct {
    Timestamp time.Time      `json:"timestamp"`
    Caller    Role           `json:"caller"`
    ToolName  string         `json:"toolName"`
    Arguments map[string]any `json:"arguments"`
    Decision  string         `json:"decision"` // ALLOW/DENY
    Result    string         `json:"result"`
}

func (s *OpsServer) RecordAudit(
    caller Role, tool string, args map[string]any,
    decision, result string,
) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.auditLog = append(s.auditLog, AuditRecord{
        Timestamp: time.Now().UTC(),
        Caller:    caller,
        ToolName:  tool,
        Arguments: args,
        Decision:  decision,
        Result:    result,
    })
}
```

`auditLog` trong RAM chỉ giữ record trong process lifetime. Storage ngoài process có thể tăng độ bền, nhưng Kafka, Elasticsearch hay S3 Object Lock không tự bảo đảm toàn vẹn. Cần xác định ai được ghi/xóa, retention, cấu hình chống sửa và cách phát hiện mất record. Gửi bất đồng bộ còn có cửa sổ crash trước khi record được xác nhận lưu; policy phải quyết định có cho mutation khi audit sink lỗi hay không, cùng cơ chế đối soát outcome.

---

7. Kiểm chứng Lab thực tế (`labs/part28-mcp-ops-tools`)

Mã nguồn hoàn chỉnh nằm tại `labs/part28-mcp-ops-tools` (phân loại kiểm chứng: `UNIT_TESTED` / `MOCK_VERIFIED` cho giao thức MCP chính thức, phân quyền theo session context, rào chắn SSRF qua target allowlist, thực thi HTTP thật tới test server, và kích hoạt actuator). Chạy kiểm thử:

```
go test -v -race ./...
```

Bộ kiểm thử xác thực 6 kịch bản an ninh thực chiến trên nền giao vận in-memory:

```
=== RUN   TestMCPListTools
--- PASS: TestMCPListTools (0.00s)
=== RUN   TestMCPQueryHealthRealHTTP
--- PASS: TestMCPQueryHealthRealHTTP (0.00s)
=== RUN   TestMCPSSRFDeniedForUnapprovedTarget
--- PASS: TestMCPSSRFDeniedForUnapprovedTarget (0.00s)
=== RUN   TestMCPOperatorRestartSuccess
--- PASS: TestMCPOperatorRestartSuccess (0.00s)
=== RUN   TestMCPObserverMutatingActionDenied
--- PASS: TestMCPObserverMutatingActionDenied (0.00s)
=== RUN   TestMCPMissingOrInvalidTicketDenied
--- PASS: TestMCPMissingOrInvalidTicketDenied (0.00s)
PASS
ok      part28-mcp-ops-tools   0.234s
```

Phân tích kết quả kiểm thử và Ranh giới xác thực:

Bộ kiểm thử khởi tạo client và server MCP qua kênh `mcp.NewInMemoryTransports()`. Kịch bản kiểm tra `tools/list` xác nhận schema JSON được sinh tự động từ struct tags `jsonschema`. Kịch bản kiểm tra sức khỏe gửi yêu cầu HTTP thực tế tới `httptest.Server`, đo thời gian phản hồi và ghi nhận trạng thái `ALLOW`.

Các test xác nhận unknown target bị từ chối, observer không gọi được restart, và ticket sai không kích hoạt actuator trong các case đã chạy. Denial record được kiểm tra trong log RAM. Đây không chứng minh chống mọi SSRF, prompt injection hay mất audit: registry, identity provider, transport và storage thật là những boundary chưa được fixture này kiểm chứng.

---

8. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Phó mặc an ninh cho JSON Schema của MCP.	Bị tấn công SSRF hoặc Prompt Injection đánh cắp token IAM đám mây.	Luôn coi tham số từ Agent là không tin cậy; kiểm tra ngữ nghĩa và IP trong Go handler.
Cho phép Agent gọi tool mutating mà không có mã phiếu phê duyệt.	Agent tự ý xóa hoặc khởi động lại các dịch vụ quan trọng khi hiểu nhầm bối cảnh.	Yêu cầu `change_ticket` và chỉ cấp quyền mutating cho vai trò Operator.
Mở HTTP không bảo vệ identity/kênh truyền.	Có thể bị nghe lén hoặc mạo danh.	Chọn Stdio khi phù hợp; với Streamable HTTP, dùng TLS và authorization theo spec/deployment. mTLS là một lựa chọn, không yêu cầu chung cho mọi MCP transport.
Không ghi nhật ký kiểm toán (Audit Trail).	Thiếu dữ liệu ở boundary tool để xác định caller và hành động, dù có thể còn log từ các tầng khác.	Ghi structured audit log phù hợp policy lưu trữ và bảo vệ dữ liệu.

---

9. Bài tập thực hành thiết kế MCP Server an toàn

Thử thách 1: Điều tốc phiên làm việc của Agent (Agent Rate Limiting)

Yêu cầu: Thiết kế ReceivingMiddleware cho tools/call, gắn rate limit với caller/session đã xác thực. Bài tập chọn 10 request/giây, không phải ngưỡng an toàn production. Test cả caller dùng nhiều session và cleanup state limiter; SDK không có API `ExecuteToolCall` trong ví dụ này.

Thử thách 2: Cơ chế Phê duyệt Hai bước (Human-in-the-Loop Gate)

Yêu cầu: Đối với các hành động phá hủy nghiêm trọng (như xóa cơ sở dữ liệu hay thu hồi hạ tầng), hãy thiết kế cơ chế phê duyệt hai bước cho công cụ MCP: hành động ban đầu trả về trạng thái chờ kèm mã xác nhận tạm thời. Thao tác chỉ được thực thi khi nhận được lệnh xác nhận chứa token hợp lệ trong thời hạn 5 phút. Cơ chế phải ràng buộc chặt chẽ token với hành động và mục tiêu đã phê duyệt, đồng thời đảm bảo token chỉ được dùng thành công đúng một lần.

---

10. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Bộ điều tốc theo caller đã xác thực

```
type ToolRateLimiter interface {
    Allow(key string, now time.Time) bool
}

// Giữ state theo session/caller đã xác thực.
// Không chia một lastCall toàn cục giữa các agent.
// Replicas need distributed limiting for a shared quota.
```

Lời giải Thử thách 2: Rào chắn Phê duyệt Hai bước

> **Dừng để dự đoán.** Giả sử Agent xin cấp token để khởi động lại dịch vụ cache staging, nhưng sau đó gửi lệnh xác nhận kèm theo hành động xóa database production. Nếu hệ thống chỉ kiểm tra thời hạn của token mà không so sánh hành động và mục tiêu, rủi ro an ninh nào sẽ phát sinh? Khi thiết kế thao tác xác nhận, ta nên xóa token ngay ở lần thử sai đầu tiên hay chỉ hủy token sau khi đã khớp thành công?

Đáp án — chỉ đọc sau khi đã tự làm Nếu mã xác nhận không gắn chặt với hành động và mục tiêu (`Action` và `Target`), hệ thống sẽ vấp phải lỗ hổng tráo đổi mục tiêu (action substitution). Một lời nhắc độc hại từ bên ngoài có thể lợi dụng sự phê duyệt của con người cho một thao tác vô hại để thực thi lệnh phá hủy trên hệ thống trọng yếu. Vì vậy, bản ghi phê duyệt phải đóng băng bộ đôi hành động và tài nguyên ngay từ thời điểm tạo token.

Về chính sách tiêu thụ token, ta lựa chọn chính sách vô hiệu hóa khi khớp thành công (single-use successful claim). Khi nhận đúng token, đúng hành động, đúng mục tiêu và còn trong thời hạn 5 phút, hàm sẽ xóa token khỏi bộ nhớ để chống tấn công phát lại (replay attack). Ngược lại, nếu chọn chính sách xóa token ngay cả khi tham số truyền vào không khớp, kẻ tấn công hoặc một tiến trình thử lại nhầm lẫn có thể cố tình gửi sai tham số để hủy hoại phiên phê duyệt hợp lệ, tạo ra nguy cơ từ chối dịch vụ (denial-of-service). Việc người dùng gửi một token không tồn tại hoàn toàn

không gây ảnh hưởng đến các token hợp lệ khác. Nguy cơ dò đoán tham số khi giữ token được kiểm soát bằng độ hỗn loạn cao của chuỗi sinh ngẫu nhiên từ `crypto/rand` và thời gian hết hạn ngắn.

Cài đặt an toàn của `ApprovalManager` dưới mô hình bộ nhớ cục bộ:

```
type PendingApproval struct {
    Action    string
    Target    string
    ExpiresAt time.Time
}

type ApprovalManager struct {
    mu       sync.Mutex
    pendings map[string]PendingApproval
}

func (m *ApprovalManager) RequestApproval(
    action, target string,
) string {
    m.mu.Lock()
    defer m.mu.Unlock()

    token := randomTokenFromCryptoRand()
    m.pendings[token] = PendingApproval{
        Action:    action,
        Target:    target,
```

```
        ExpiresAt: time.Now().Add(5 * time.Minute),
    }
    return token
}

func (m *ApprovalManager) Confirm(
    token, action, target string,
) bool {
    m.mu.Lock()
    defer m.mu.Unlock()

    item, exists := m.pendings[token]
    if !exists || time.Now().After(item.ExpiresAt) {
        return false
    }
    if item.Action != action || item.Target != target {
        return false
    }
    delete(m.pendings, token)
    return true
}
```

Toàn bộ thao tác kiểm tra và tiêu thụ token phải diễn ra nguyên tử dưới cùng một khóa `m.mu.Lock()` nhằm ngăn chặn hai goroutine chạy đua cùng lúc với một token và bảo toàn liên kết chặt chẽ với hành động cùng mục tiêu (`Action` và `Target`). Trong mô hình này, việc chỉ xóa token khi xác nhận thành công giúp duy trì tính hợp lệ của phiên phê duyệt trước các lần gửi sai tham số hoặc thử lại nhầm lẫn, đồng thời bảo đảm mỗi token chỉ được sử dụng đúng một lần.

Tuy nhiên, việc không xóa token khi kiểm tra thất bại đặt ra yêu cầu bắt buộc về chính sách dọn dẹp: những token đã hết hạn mà không bao giờ được xác nhận sẽ tích tụ vĩnh viễn trong bản đồ nếu không có cơ chế thu gom. Một hệ thống hoàn chỉnh đòi hỏi tiến trình chạy nền dọn dẹp định kỳ (periodic cleanup worker) hoặc cơ chế loại bỏ lười (lazy eviction) để quét và xóa các mục có `time.Now().After(item.ExpiresAt)`. Cần nhấn mạnh rằng `ApprovalManager` trên đây là mô hình kiến trúc minh họa trong bộ nhớ cục bộ của một tiến trình, chưa phải giải pháp an toàn cho môi trường sản xuất. Môi trường thực tế đòi hỏi kho dữ liệu bền vững chịu lỗi qua các

lần khởi động lại, xác thực danh tính con người độc lập khỏi AI agent, và nhật ký kiểm toán không thể sửa đổi.

Một HTTP request hoàn thành không đồng nghĩa service khỏe. Lab dùng quy ước hẹp: chỉ `2xx` là `healthy`; `4xx`/`5xx` được trả về như trạng thái `unhealthy`, còn lỗi transport mới là lỗi gọi tool. Hệ thống thật cần contract health riêng (ví dụ readiness, body schema và timeout) thay vì suy ra sức khỏe chỉ từ mã HTTP.

---

11. Điểm dừng: một tool an toàn là một boundary có bằng chứng

Chương 28 nối các boundary công cụ đã học. Bảng dưới là bản đồ tra cứu, không phải chứng nhận đã làm chủ production; Chương 29 tiếp tục phân biệt đề xuất, bằng chứng, thẩm quyền và kết quả.

Chặng đường	Phạm vi kiến thức và kỹ năng	Thành tựu kỹ thuật cốt lõi
Phần I–III: Nền tảng	Cú pháp, value, pointer, slice, interface và error.	Dự đoán copy/aliasing và đọc contract lỗi.
Phần IV–V: Đồng thời & Vận hành	Memory model, worker pool, runtime, GC và pprof.	Tái lập race, thiết kế đường thoát và đo bottleneck.
Phần VI–VII: Mạng & Dịch vụ	HTTP/gRPC, TLS, SQL và lifecycle server.	Đặt timeout, ownership tài nguyên và transaction boundary.
Phần VIII: Quan sát & Điều phối	Metrics/trace, container, Kubernetes và opsprobe.	Phân biệt tín hiệu, desired state và observation.
Phần IX: Hạ tầng & Agent	Controller, operator, AWS, Git, supply chain, eBPF và MCP.	Xét retry, identity, quyền và evidence của từng boundary.

Khi đưa những ý tưởng này vào production, hãy quay lại threat model, trust boundary, workload và incident policy của hệ thống cụ thể. Các lab chứng minh contract hẹp của chúng; chúng không thay thế xác thực danh tính thật, egress policy, durable audit hay review vận hành.

Chương 29 — Kỹ sư và bằng chứng trong kỷ nguyên Agent

Một Agent vừa đề xuất đổi timeout, tăng số replica và restart một service. Nó đưa ra một đoạn giải thích mạch lạc, một patch hợp lệ và dòng "tests passed". Nếu anh bấm đồng ý, điều gì đã thực sự được chứng minh? Có thể chỉ là test của một checkout khác đã xanh; có thể patch đang dùng một config khác production; cũng có thể quyền restart chưa từng được cấp cho caller này. Chất lượng câu trả lời và quyền tác động vào hệ thống không phải cùng một đại lượng.

Chương 28 đã dạy cách trao một công cụ có giới hạn. Chương này đổi góc nhìn từ tool sang công việc của kỹ sư: **đề xuất → bằng chứng → quyền → hành động giới hạn → hậu điều kiện**. Agent có thể hỗ trợ nhiều bước, nhưng một bước không tự thay thế bước kế tiếp. Một hash đúng không chứng minh code đúng; test đúng không cấp quyền deploy; deploy thành công không chứng minh người dùng đã được phục vụ tốt hơn. Đây là mô hình để thiết kế quy trình và để đọc những lời hứa về nghề nghiệp, không phải một dự báo rằng chức danh nào sẽ tồn tại mãi.

Đọc số liệu theo câu hỏi nó thực sự trả lời

Ngày đối chiếu lại nguồn của chương là 08-10-2026. "AI giúp lập trình nhanh hơn" thiếu ít nhất ba đối tượng: ai làm, làm việc gì, và nhanh hơn theo phép đo nào. Dữ liệu sau không dùng chung mẫu hay cùng loại outcome, nên không được lấy trung bình các phần trăm để tạo một con số về "năng suất AI".

Nghiên cứu của Peng và cộng sự, công bố tháng 2-2023 trên arXiv, phân nhóm ngẫu nhiên 95 lập trình viên tuyển qua Upwork trong 15-05 đến 20-06-2022. Công việc là viết HTTP server JavaScript. Có 70 người hoàn thành task và survey, 35 ở mỗi nhóm; phân tích thời gian trên nhóm hoàn thành cho mức giảm 55,8% khi được dùng GitHub Copilot, khoảng tin cậy 95% 21–89%. Mẫu phần lớn từ Ấn Độ và Pakistan. Kết quả thuộc nhiệm vụ chuẩn

hóa và công cụ năm 2022, không đại diện toàn vòng đời service hay incident production. Tác giả có liên hệ Microsoft/GitHub; nghiên cứu không đo đầy đủ chất lượng bảo trì dài hạn.

METR công bố ngày 10-07-2025 một RCT với 16 contributor có kinh nghiệm, hoàn thành 246 task trên repository mã nguồn mở quen thuộc. Mỗi task được phân ngẫu nhiên vào điều kiện cho phép hoặc không cho phép AI; công cụ thuộc giai đoạn tháng 2–6-2025, chủ yếu Cursor Pro với Claude 3.5/3.7 Sonnet. Ước lượng có điều chỉnh của nghiên cứu là thời gian tăng 19%, khoảng tin cậy 95% 2–39%; không phải chỉ lấy tỷ số hai thời gian trung bình thô. Đối tượng đo là contributor và task được tuyển, không đại diện lao động quốc gia. Kết quả không chứng minh AI vô dụng ở mọi bối cảnh.

Bản cập nhật METR ngày 24-02-2026 nêu vấn đề chọn người, chọn task và đo thời gian khi người tham gia dùng công cụ mới hoặc làm nhiều việc song song. Tác giả coi các ước lượng mới là không đáng tin để định lượng hiệu ứng hiện tại. Vì vậy cũng không được dùng con số chậm 19% của năm 2025 như định luật cho Agent năm 2026. “Chưa đo chắc” là một kết luận có ích: nó ngăn cả quảng cáo lẫn phản đối AI dựa trên một thí nghiệm đã đổi bối cảnh.

Go Developer Survey 2025 do Go team công bố 21-01-2026, thu thập 09–30-09-2025, dùng 5.379 phản hồi sau làm sạch. Mời qua kênh công khai và lời mời ngẫu nhiên trong VS Code/GoLand; đây không phải mẫu ngẫu nhiên của toàn thị trường lao động. Phản hồi chủ yếu từ Bắc Mỹ và châu Âu. CLI và API service là nhóm use case nổi bật, và có người dùng Go cho hạ tầng cloud. Điều đó cho bối cảnh ecosystem, không cho số việc làm Go ở Việt Nam. Nhận xét về AI là tự báo cáo, không phải phép đo tốc độ có nhóm đối chứng.

Ở lớp thị trường, BLS cập nhật Occupational Outlook Handbook ngày 27-08-2026, dự báo việc làm tại Mỹ của nhóm software developers, quality assurance analysts và testers tăng 10% trong 2025–2035. Đây là dự báo nghề nghiệp từ chương trình Employment Projections, không phải quan sát rằng việc làm đã tăng từng ấy, không tách Go, DevOps hay SRE và không mô tả Việt Nam. Openings còn gồm nhu cầu thay người rời nghề, không chỉ vị trí

mới. Sách không có dữ liệu đủ mạnh để suy ra lương hay nhu cầu Go tại Việt Nam từ nhóm nghề rộng này.

DORA 2025 đặt AI trong bối cảnh năng lực tổ chức: một công cụ có thể khuếch đại cả điểm mạnh lẫn điểm yếu đang có. Trang công bố thuộc chương trình nghiên cứu do Google Cloud dẫn dắt, có đối tác ngành. Ta dùng framing này như một gợi ý kiểm tra quy trình, không lấy nó làm bằng chứng nhân quả rằng mua công cụ sẽ làm SLO tốt hơn. Khi cần một con số từ survey, phải đọc mẫu, câu hỏi và phương pháp của chính số đó; chương không đưa phần trăm từ một báo cáo chỉ đọc phần giới thiệu.

Tự động hóa task không đồng nghĩa xóa một nghề

Một chức danh là bó công việc, quyền hạn và trách nhiệm trong một tổ chức. Tạo YAML, tra syntax API, viết boilerplate test, phân loại log, điều tra nguyên nhân và quyết định chấp nhận rủi ro là những task khác nhau. Tự động hóa được một task có thể giảm công gõ, tăng số thay đổi có thể đề xuất, hoặc chuyển thời gian sang review. Nó không tự nói tổ chức sẽ tăng hay giảm headcount; còn có giá công cụ, nhu cầu sản phẩm, chi phí phối hợp, quy định và cách doanh nghiệp tổ chức công việc. Phần này là suy luận về thiết kế công việc, không phải thống kê tuyển dụng.

Sự thật khó chịu là thuộc nhiều CLI và YAML không tạo một hàng rào bền nếu task chỉ là biến một yêu cầu rõ thành cú pháp quen thuộc. Khi tool làm tốt phần ấy, kỹ sư phải cạnh tranh ở chỗ khác: đặt contract, nhận diện thiếu dữ liệu, giới hạn blast radius, hiểu vì sao một counterexample bác bỏ lời giải và kiểm tra thay đổi trong môi trường thật. Không phải ai cũng tự động chuyển sang phần việc khó hơn; học các năng lực ấy cần thực hành độc lập, không chỉ biết viết prompt.

Cơ hội cũng không chỉ là "viết code nhanh". Một internal developer platform có thể giảm các thao tác lặp lại bằng API chuẩn, deployment contract và policy có thể kiểm tra. Reliability engineering có thể dùng tự động hóa để thu thập trace, so config và khoanh vùng failure. AI infrastructure cần người hiểu network, resource budget, identity, chi phí và đường đi của dữ liệu. Security và incident response cần phân biệt đề xuất

hữu ích với hành động làm mất bằng chứng hoặc mở thêm quyền. Những hướng ấy có giá trị khi tổ chức thực sự có bài toán, không phải bảo đảm mỗi người biết Go đều được tuyển.

Ở tầng entry-level, ta có thể đặt một scenario: nếu phần lớn task đầu vào từng là boilerplate, người mới có ít cơ hội học qua việc tự viết chúng. Đây là rủi ro đào tạo cần thiết kế lại, không phải một claim rằng mọi vị trí junior đã biến mất. Một cách đáp ứng là giữ bài tập solo nhỏ, yêu cầu dự đoán trước khi chạy, review code sinh ra bằng counterexample và tự điều tra một failure chưa có đáp án. Một portfolio tốt nên cho thấy anh biết bác bỏ lời giải sai, không chỉ có nhiều repository được Agent tạo.

Go nằm ở đâu trong bó công việc ấy?

Các chương trước đã cho ví dụ kiểm chứng được: CLI gom dữ liệu vận hành, HTTP service, controller Kubernetes, cloud SDK, GitHub automation và tool server MCP. Go có static types, build/test tooling thống nhất và standard library phù hợp với nhiều bài toán I/O. Chúng giúp tạo vòng phản hồi cho cả người lẫn Agent: compiler bắt một lớp lỗi, test bắt những contract đã được mô tả, race detector tìm access racy trên đường chạy được thực thi. Không công cụ nào thay bài toán chọn contract.

Đây là lựa chọn theo workload. Một CLI thuần Go có thể giảm phụ thuộc runtime khi phân phối, nhưng cgo, chứng chỉ, timezone, file config và quyền OS vẫn có thể tạo dependency ngoài binary. Service có GC cần budget memory và latency. Interface nhỏ giúp thay implementation, nhưng không diễn đạt tự động ownership, deadline hoặc retry semantics. Simplicity làm một số bước review dễ hơn; nó cũng để lại công việc thiết kế bằng convention và test thay vì một hệ kiểu kiểm tra mọi invariant.

Nếu task chủ yếu là khám phá dữ liệu và notebook với library ML sẵn có, Python có thể hợp lý hơn. Nếu cần một subsystem có yêu cầu ownership hoặc kiểm soát memory rất chặt, Rust hay C/C++ có thể đáng cân nhắc tùy hệ thống và năng lực team. Browser UI có ecosystem khác. Một hệ thống có thể dùng Go ở control plane và ngôn ngữ khác ở data plane; boundary protocol và lifecycle quan trọng hơn tranh luận một ngôn ngữ phải làm tất

cả. Chương 14 đã cho một ví dụ nhỏ về chi phí interop, không một bảng xếp hạng ngôn ngữ.

Khi nút thắt chuyển từ viết sang kiểm chứng

Phần này là suy luận thiết kế của tác giả, không phải một kết quả đo thêm từ các nghiên cứu trên. Xét scenario Agent tạo nhiều patch hơn trong cùng một buổi nhưng người review vẫn phải kiểm tra contract, dependency, test thiếu, quyền deploy và tác động tới người dùng. Nếu hàng chờ review tăng, thời gian gõ giảm chưa chắc làm thời gian từ yêu cầu đến thay đổi được chấp nhận giảm. Đếm patch hoặc dòng code ở đầu vào sẽ bỏ sót nút thắt ấy.

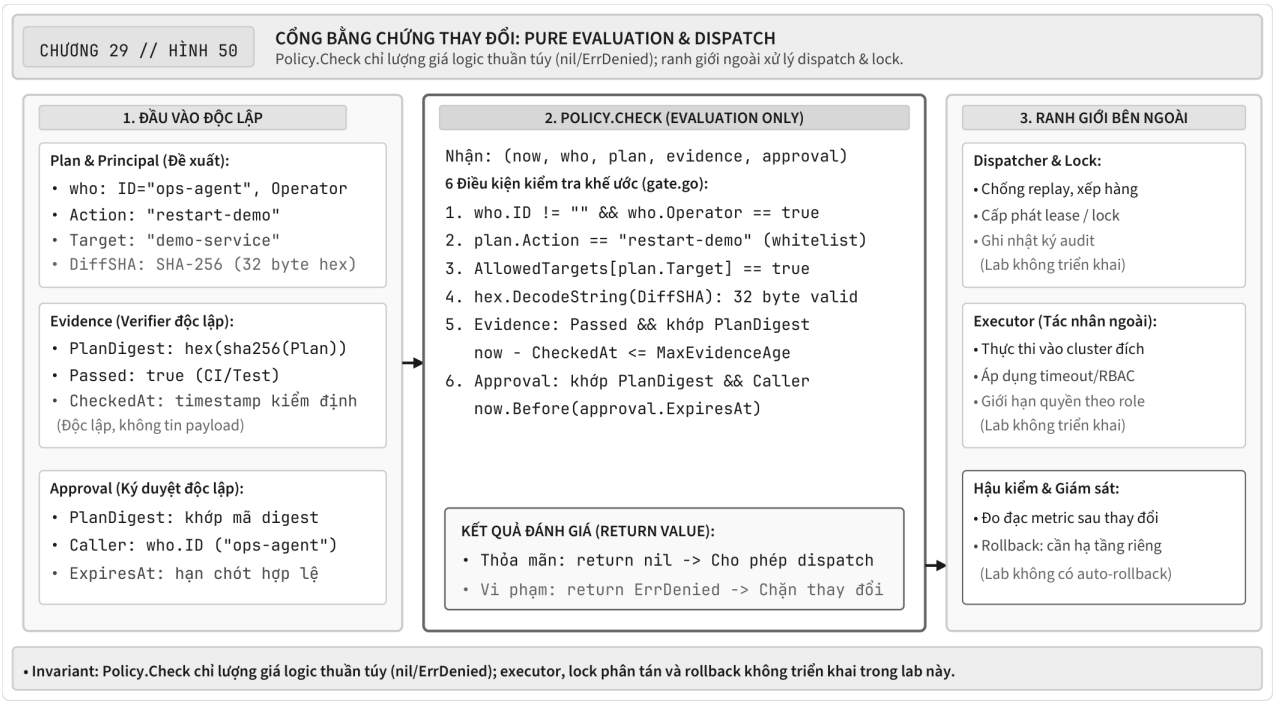
Muốn kiểm chứng lợi ích, hãy định nghĩa điểm kết thúc trước: thay đổi đã qua review của đúng artifact, chạy trong target được phép và đạt hậu điều kiện. Đo riêng thời gian chờ review, thời gian kiểm tra, rework sau reject, chi phí tool và sự cố phát sinh; so task có độ khó và yêu cầu chất lượng tương đương. Không dùng cờ "tests passed" do cùng bên sinh patch tự khai làm phép đo độc lập. Một đề xuất sai bị bác bỏ sớm có thể là kết quả có ích, không phải lỗi năng suất cần giấu.

Deployment xanh cũng chưa phải outcome người dùng. Ví dụ rollout không crash nhưng làm request chậm hơn hoặc đổi kết quả nghiệp vụ sai: hậu điều kiện phải nhìn cả điều mà người dùng cần, không chỉ process có sống. Ch16 đã phân biệt các phép chiếu observability; Ch18 ràng buộc artifact với promotion. Ở đây ta thêm câu hỏi về chi phí và điểm kết thúc của công việc, không lặp lại một checklist phát hành.

Bài review ngắn. Một team báo "Agent tăng gấp đôi số PR mỗi tuần" nhưng không có số PR được chấp nhận, thời gian review hay lỗi sau rollout. Hãy viết ba dữ liệu cần thu trước khi kết luận năng suất tăng. Không cần phủ nhận con số PR; cần chỉ ra nó đo đầu vào nào và chưa đo outcome nào.

Một cuộc điều tra trước khi cho phép restart

Xét scenario local sau, không phải sự cố production đã quan sát. Agent đọc log "timeout" rồi đề xuất restart `local-demo`. Nó viện dẫn test xanh từ một phút trước. Trong phút ấy, diff thay đổi từ A sang B. Chính sách chỉ kiểm tra `testsPassed` và `approved` sẽ cho B đi qua bằng chứng của A. Bug không nằm ở model viết sai câu; nó nằm ở việc hệ thống không gắn bằng chứng với đúng object đang được chấp nhận.



Hình 50 — Đề xuất không có quyền thực thi. Mỗi bước kiểm tra phải gắn với đúng artifact, target và caller; mũi tên diễn tả policy của scenario, không phải một protocol đảm bảo an toàn sẵn có.

Hãy review policy lỗi này trước khi đọc lab:

```
if evidence.Passed && approved {
    return nil
}
return ErrDenied
```

Nó thiếu identity của thay đổi, tuổi bằng chứng, caller, target và action. Một `Passed` do Agent tự khai còn không phải bằng chứng độc lập. Thêm hash vào cùng payload chưa chữa trust: bên không tin cậy có thể bịa cả hash lẫn cờ xanh. Verifier và nơi lưu approval phải nằm ngoài quyền ghi của Agent. Chương 26 đã tách artifact identity khỏi provenance; Chương 28 đã tách

authenticated identity khỏi argument của tool. Ta ghép hai boundary ấy, không tạo framework mới.

Lab: viết rào chắn từ contract, không từ lời giải

Mở `labs/part29-change-evidence/gate_test.go` trước `gate.go`. Trong một bản sao thử nghiệm, bỏ implementation tham chiếu rồi dựng API theo test. Một request hợp lệ cần operator đã xác thực, action duy nhất `restart-demo`, target local trong allowlist, SHA-256 hợp lệ của diff, evidence mới và xanh gắn với digest của plan, approval chưa hết hạn gắn với cùng caller và plan. Thiếu một phần phải từ chối. Đây là policy dạy học: giới hạn tuổi một phút là dữ liệu của fixture, không phải một tiêu chuẩn bảo mật cho production.

Digest bao gồm version của encoding, action, target và diff hash. Version giữ nghĩa của encoding khi format đổi; SHA-256 chỉ giúp so identity, không kiểm tra tác dụng của diff. Không lấy một JSON có thứ tự tùy ý rồi giả định mọi producer đều hash ra cùng giá trị; lab dùng một struct với encoding cố định trong Go và giữ contract ấy trong test. Nếu hệ thống có nhiều ngôn ngữ, cần định nghĩa encoding canonical như một protocol riêng.

Chạy từ thư mục lab:

```
go test -v ./...
go vet ./...
go test -race ./...
```

Test phải từ chối diff đổi, target khác, action khác, caller khác, viewer, thiếu evidence, test lỗi, evidence cũ hoặc ở tương lai và approval hết hạn. Test deterministic kiểm tra cùng plan cho cùng digest và target đổi thì digest đổi. Nó không chứng minh thuật toán hash không có collision; đó không phải điều một unit test nhỏ đo được.

Reference implementation không chạy shell, không restart một service và không nhận credential production. Evidence và approval là input tin cậy của fixture; trong hệ thật, nguồn của chúng cần authentication và storage

policy. `Check` trả `nil` cũng chưa ngăn approval bị dùng hai lần, chưa khóa artifact khỏi đổi giữa check và execute, chưa xác nhận target state hay rollback. Giới hạn này là chủ đích: lab chỉ chứng minh contract kiểm tra local. Không gọi nó là một authorization service production-ready.

Bài thiết kế tiếp theo. Approval chỉ được dùng một lần. Hai executor có thể cùng đọc approval hợp lệ. Hãy viết failing test cho hai consumer trước khi chọn mutex hay transaction; chỉ một consumer được claim quyền dùng, consumer còn lại bị từ chối. Nếu executor crash sau claim nhưng trước action, policy có cho retry không, và action có idempotent không? Câu trả lời phải dựa trên hành động thật, không trên việc đã có một lock trong code.

Hướng phân tích — chỉ đọc sau khi đã viết contract. Một cờ đã dùng trong memory không giải quyết hai process. Check và claim cần một atomic boundary trong storage đáng tin; action ngoài storage có thể vẫn tạo khoảng crash. Có thể cần state machine pending/claimed/observed, idempotency key hoặc điều tra trước retry. Mỗi lựa chọn đổi failure mode. Đây là cùng trách nhiệm của transaction ở Chương 13 và reconciliation ở Chương 21, nay áp dụng cho tự động hóa có Agent.

Năng lực nào cần luyện nếu không muốn chỉ làm người bấm duyệt?

Sự khác biệt giữa review và đọc lướt là anh có thể tạo một phép thử bác bỏ. Với code concurrency, vẽ access và cạnh happens-before thay vì tin nhãn thread-safe. Với network, lần timeout qua DNS, connect, TLS, response body; một status code không giải thích whole request path. Với Linux, phân biệt process, descriptor, namespace, quota và memory được kế toán. Với distributed systems, hỏi duplicate, stale observation, retry và side effect xảy ra ở boundary nào. Đây là những kỹ năng chuyển được giữa tool, không phụ thuộc một model cụ thể.

Observability cũng không phải chỉ cài dashboard. Một trace có đủ correlation để phân biệt queue wait với dependency wait không? Một metric đổi vì traffic mix hay vì code? Agent có thể viết query và tóm tắt log; kỹ sư vẫn cần chọn measurement contract và bảo vệ dữ liệu nhạy cảm. Trong incident, tự động hóa thu thập read-only thường có rủi ro khác tự động hóa

xóa Pod hoặc đổi firewall. Hành động "sửa nhanh" có thể làm mất hiện trạng cần điều tra. Quyền và bằng chứng phải tăng theo tác động, không theo mức tự tin của văn bản.

Một bài học độc lập có thể kết thúc bằng một hồ sơ nhỏ: requirement anh viết, phản ví dụ anh tìm, artifact đúng version, test anh chạy, phép đo có giới hạn, quyết định quyền và hậu điều kiện anh quan sát. Đừng chỉ lưu transcript dài. Hồ sơ tốt cho người khác tái kiểm tra lý do quyết định, kể cả khi Agent hay library đã đổi. Đó là năng lực verification có thể luyện được, không phải lời hứa miễn nhiễm với thay đổi thị trường.

Kết thúc ở một quyết định có thể bảo vệ

Không có kết luận chắc rằng "AI sẽ lấy nghề" hay "kỹ sư giỏi sẽ không bị ảnh hưởng" từ các nguồn đã đọc. Có bằng chứng về công cụ giúp một số task, về task khác chậm hơn trong một bối cảnh cũ, và về khó khăn khi đo công cụ đang thay đổi. Có dự báo nghề rộng tại Mỹ, không có một dự báo riêng đủ mạnh cho Go/SRE ở Việt Nam. Từ đó, lời khuyên của sách là một khuyến nghị có điều kiện: học xây và kiểm chứng hệ thống, dùng AI để mở rộng khả năng làm việc nhưng giữ quyền hạn, artifact và outcome thành những đối tượng có thể audit.

Khi người khác hỏi vì sao anh chấp nhận một thay đổi, câu trả lời tốt không phải "Agent nói thế" hoặc "test đều xanh". Nó là: contract nào, bằng chứng của artifact nào, dưới điều kiện nào, do ai cấp quyền, và hệ thống sau hành động đã được quan sát ra sao. Bách khoa thực dụng không kết thúc ở việc nhớ thêm một package. Nó kết thúc ở khả năng biết mình đang chứng minh điều gì — và điều gì vẫn chưa được chứng minh.

1. Peng, S.; Kalliamvakou, E.; Cihon, P.; Demirer, M. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590, 2023. arxiv.org/abs/2302.06590
2. Becker, J.; Rush, N.; Barnes, B.; Rein, D. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. METR, 10-07-2025. metr.org/Early_2025_AI_Experienced_OS_Devs_Study-paper.pdf
3. METR. We are Changing our Developer Productivity Experiment Design. 24-02-2026. metr.org/blog/2026-02-24-uplift-update/
4. Go Team. Results from the 2025 Go Developer Survey. 21-01-2026, mục Survey methodology. go.dev/blog/survey2025
5. U.S. Bureau of Labor Statistics. Occupational Outlook Handbook: Software Developers, Quality Assurance Analysts, and Testers. Cập nhật 27-08-2026; Mỹ, dự báo 2025–2035.

bls.gov/ooh/computer-and-information-technology/software-developers.htm

6. DORA. State of AI-assisted Software Development 2025, trang công bố và ranh giới framing. dora.dev/research/2025/dora-report/

BACK MATTER — ATLAS MÃ NGUỒN 50 THƯ VIỆN GO DEVOPS & CLOUD

Điểm neo để đọc API và implementation đã ghim

Phụ bản này đứng sau Chương 29 và ngay trước Atlas Lỗi Go. Các nhận xét về implementation chỉ áp dụng cho version và commit ghi ở mỗi mục, không phải cam kết API cho mọi version. Khóa source chứng minh identity của checkout, không tự chứng minh mọi giải thích cơ chế; khi dùng một chi tiết để quyết định production, đọc code và test tại commit đó.

Phụ bản chọn 50 thư viện Go trong các lĩnh vực DevOps, cloud, networking, observability, IaC và hạ tầng Agent. Mỗi mục ghi version/commit được ghim để người đọc có thể đối chiếu. Mô tả API hoặc hợp đồng công khai không đồng nghĩa đã kiểm chứng mọi chi tiết implementation; câu nào nói đến cơ chế nội bộ phải có điểm neo source tương ứng ở bản đã ghim.

> Quy tắc kiểm chứng: không đoán version, tag, module path hay tên hàm. Một claim về implementation phải có source đã xác minh; cơ chế được neo vào type, struct hoặc file cụ thể. Mỗi mục phân tích cơ chế và ý nghĩa kỹ thuật, không dùng lời quảng bá thay cho bằng chứng.

PHẦN 1: NHÓM NỀN TẢNG TRỌNG YẾU (TIER S: 01–12)

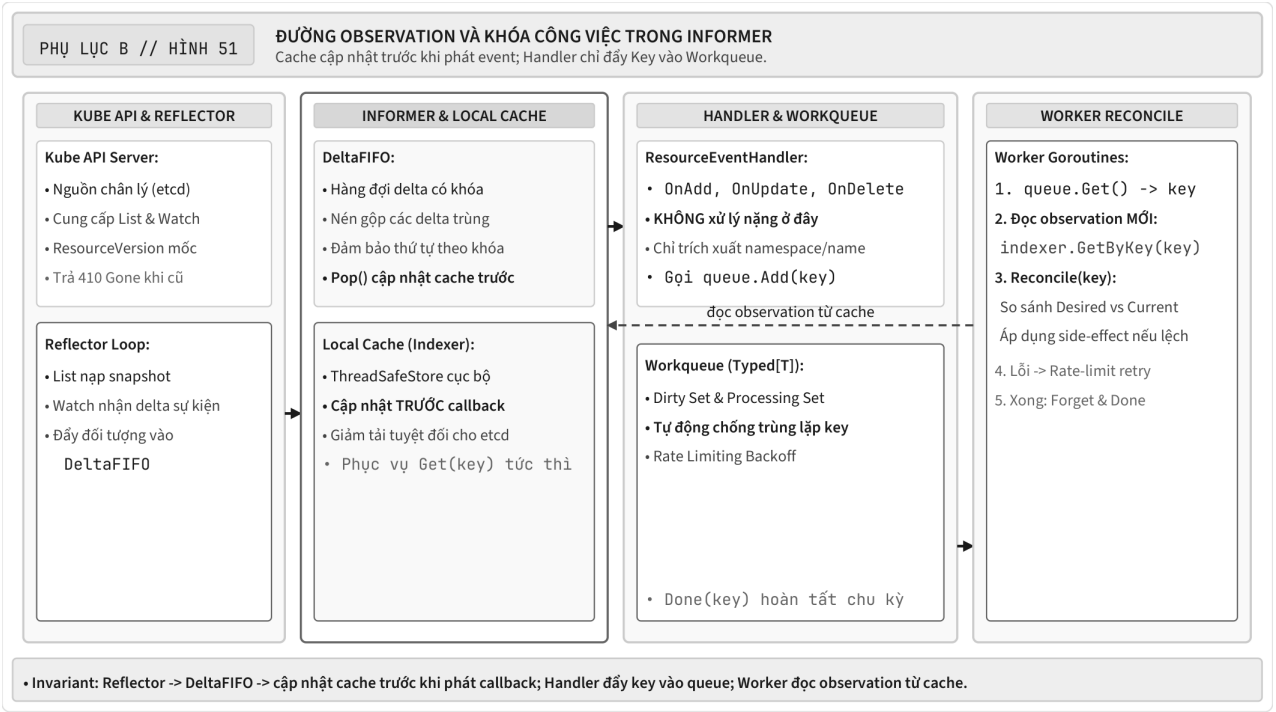
01. `k8s.io/client-go` **(v0.37.0 — 28076445)**

Nếu controller lặp List toàn bộ Pod mỗi giây, nó tạo công việc tuần tự hóa và truyền snapshot lặp lại. Mức tải phụ thuộc số object, kích thước và API server; không suy ra CPU 100% hay cluster sập chỉ từ 500 Pod. Informer hỗ trợ duy trì observation local từ list/watch để nhiều consumer không phải tự polling toàn bộ tài nguyên theo cùng nhịp.

Reflector duy trì observation bằng list/watch và xử lý thay đổi resourceVersion cùng lỗi transport. Đường khởi tạo cụ thể có thể dùng List

hoặc watch-list tùy version, capability và cấu hình. Watch là stream API event, không phải trực tiếp theo dõi mọi thay đổi etcd; HTTP/2 cũng không dùng HTTP/1.1 chunked transfer encoding. Khi lịch sử không còn đủ hoặc stream lỗi, Reflector xử lý recovery theo source đã pin; không phải mỗi lần đứt stream đều bắt buộc List lại.

Dữ liệu nhận về không được chuyển ngay cho người dùng mà đẩy vào một hàng đợi gom cụm đột biến mang tên `DeltaFIFO` (`tools/cache/delta_fifo.go`). Khi một Pod bị sửa đổi dồn dập 10 lần trong nửa giây, `DeltaFIFO` lưu trữ chuỗi delta theo khóa định danh `namespace/name`. Từ đây, vòng lặp xử lý nội bộ của Informer (`controller.processLoop`) rút delta ra, cập nhật đối tượng vào một bộ nhớ đệm luồng an toàn cục bộ gọi là `Indexer` (`tools/cache/index.go`), được bảo vệ bởi `sync.RWMutex`.



Hình 51 — Informer cập nhật cache rồi phát event tới handler; handler của ứng dụng đưa khóa vào workqueue. Worker đọc observation hiện có khi điều hòa, không coi payload event là trạng thái cuối cùng.

Điểm neo kỹ thuật đáng giá nhất để học trong `client-go` nằm ở `util/workqueue/queue.go` (`type Typed[T]`). `TypedInterface[T]` là giao diện public; `TypedQueueConfig[T]` là cấu hình. Triển khai `Typed[T]` phối hợp hàng đợi có thể thay thế và hai tập khóa:

```
type Typed[T comparable] struct {
    queue      Queue[T]
    dirty      sets.Set[T]
    processing sets.Set[T]
    cond       *sync.Cond
    // Các trường lifecycle và metrics khác được lược bỏ.
}
```

Hãy hình dung: Worker đang bận xử lý key `"default/my-pod"` trong tập `processing`. Cùng lúc đó, 5 sự kiện cập nhật liên tiếp của Pod này được Informer gửi tới. `Add(item)` chỉ đánh dấu key trong `dirty` khi nó đang được xử lý, không đẩy thêm vào hàng đợi. Khi worker gọi `Done(item)`, nếu key vẫn `dirty`, nó được đẩy lại đúng một lần. Cơ chế khử trùng lặp này giới hạn việc xếp trùng một key; thứ tự và cấu trúc lưu trữ phụ thuộc triển khai `Queue[T]`. Worker thường lấy key rồi đọc snapshot hiện tại từ Indexer — snapshot đó có thể stale so với API server. Reconciliation lặp lại mới giúp hội tụ về trạng thái mong muốn.

Khi worker không chạy, kiểm tra lifecycle, error của Reflector và cache sync. Thiếu quyền list có thể ngăn initial population; thiếu quyền watch ảnh hưởng nhận cập nhật. Không suy ra HasSynced luôn false chỉ từ một lỗi watch, vì trạng thái sync còn phụ thuộc đường khởi tạo. Phải có cancellation/deadline và quan sát error thay vì chờ vô hạn.

---

02. `sigs.k8s.io/controller-runtime` (**v0.25.1** — `67b72c25`)

`controller-runtime` tổ chức cache, controller và lifecycle dùng chung, giảm phần wiring phải tự làm với `client-go`. `Reconcile(ctx, Request)` là contract điều hòa, không xóa trách nhiệm thiết kế retry, quyền, cleanup hay shutdown của ứng dụng. Không có một số dòng boilerplate cố định cho mọi controller.

Ở `pkg/manager/internal.go`, `controllerManager.Start` khởi động HTTP servers và webhooks trước caches, rồi mới các nhóm runnable phụ thuộc cache và leader election. Tín hiệu chỉ cancel context nếu caller đã thiết kế signal propagation. Shutdown đi qua stop procedure với timeout cấu hình; không hứa mọi worker luôn hoàn tất mọi side effect trước khi process dừng.

Ở commit này, đọc `pkg/client/client.go`, `Options.Cache` và `CacheOptions`, không tìm một `delegatingClient` trong file `split.go` của version cũ. Cấu hình rút gọn sau minh họa boundary đọc:

```
type CacheOptions struct {
    Reader       Reader
    DisableFor   []Object
    Unstructured bool
}
```

Client của manager thường dùng cache cho `Get`/`List` của type được cache. Nhưng `client.New` không có `Options.Cache.Reader` thì đọc API server; `DisableFor` và cấu hình unstructured còn có thể chọn đường trực tiếp. Lazy discovery/informer initialization cũng có thể phát sinh HTTP. Không suy ra một `Get` bất kỳ là "zero HTTP". Write đi tới API server; response thành công còn phải diễn giải theo operation/options, chẳng hạn dry-run không lưu thay đổi. Watch propagation và trạng thái workload thực tế là các boundary riêng, không được xác nhận chỉ bằng một write response.

`reconcile.Request` mang key thay vì snapshot object tại lúc phát event. Khi worker xử lý, object có thể đã đổi. Reconciler chọn reader phù hợp và đọc lại theo key; request không bắt buộc dùng cache và cũng không cung cấp freshness guarantee. Ví dụ dưới đây chọn `r.Get` qua client của reconciler:

```
var pod corev1.Pod
if err := r.Get(ctx, req.NamespacedName, &pod); err != nil {
    return ctrl.Result{}, client.IgnoreNotFound(err)
}
```

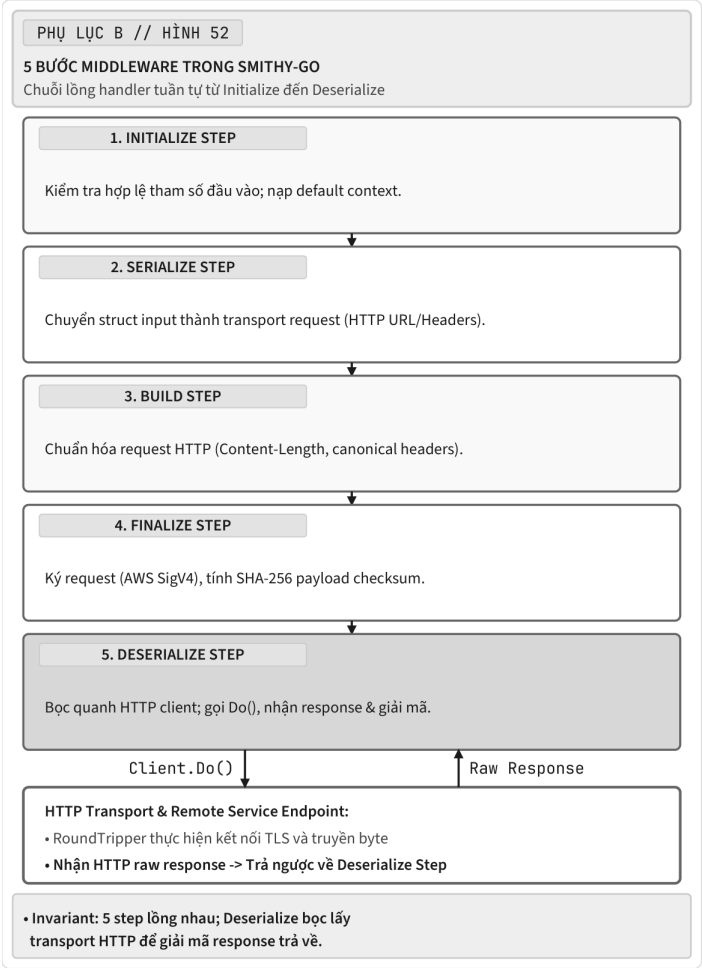
`client.IgnoreNotFound(err)` chỉ chuyển lỗi not-found thành `nil`; nó không làm các side effect trở nên lũy đẳng. Request theo key giúp reconciler đọc trạng thái khi xử lý, nhưng cache có thể stale. Muốn hội tụ an toàn cần thiết kế từng thao tác: kiểm tra ownership, dùng precondition khi ghi và quan sát lại sau lỗi hoặc retry. Khi cần độ mới của API server, chọn client đọc trực tiếp thay vì suy ra từ cache.

---

03. `github.com/aws/aws-sdk-go-v2` **(v1.47.0 —** `b189f382`**)**

Một lời gọi S3 được SDK serialize, resolve endpoint, ký khi operation yêu cầu và gửi qua HTTP. SigV4 dùng canonical request và credential; không phải mọi đường S3 đều băm toàn bộ body, vì có chế độ unsigned payload. Đọc `aws/signer/v4/middleware.go` và operation source để biết đường ký cụ thể.

SDK dùng `middleware.Stack` của module `github.com/aws/smithy-go`, không phải type khai báo trong `aws/middleware/stack.go`. Năm stage tổ chức request và response path như sau:



Hình 52 — Năm step tổ chức đường gọi vào bên trong. Deserialize gọi handler HTTP trước khi giải mã response; sơ đồ lược bỏ đường response đi ngược ra ngoài các step.

1. **Initialize:** Kiểm tra hợp lệ các tham số đầu vào và nạp giá trị mặc định vào ngữ cảnh.
2. **Serialize:** Đưa input operation vào request của transport; đường HTTP dùng `transport/http.Request` của smithy-go cho URL, method, headers và body.
3. **Build:** Chuẩn bị request, chẳng hạn content length và metadata phù hợp operation.
4. **Finalize:** Đường ký và retry được đặt ở đây trước transport; thứ tự middleware cụ thể phải đọc theo operation và cấu hình.
5. **Deserialize:** Đọc mã phản hồi HTTP, giải mã XML/JSON body thành struct kết quả, hoặc ánh xạ mã lỗi AWS (như `NoSuchKey`, `AccessDenied`) thành các struct lỗi cụ thể trong Go.

Xét scenario nhiều client gặp lỗi gần cùng lúc: backoff cố định theo cấp số nhân có thể làm các lần retry tiếp tục dồn thành đợt. `aws/retry/standard.go` ở version ghim dùng Full Jitter để phân tán thời điểm retry trong khoảng backoff. Jitter giảm đồng bộ thời gian, không tạo thêm capacity hay bảo đảm hạ tầng hồi phục; vẫn cần attempt budget, deadline và policy cho side effect.

Ở phiên bản được ghim `github.com/aws/aws-sdk-go-v2` v1.47.0, `retry.Standard` mặc định dùng rate limiter dạng quota với capacity 500 token. Khi `AWS_NEW_RETRIES_2026` không bằng `true`, cấu hình cũ tính `RetryCost` là 5 token cho retry thông thường và `RetryTimeoutCost` là 10 cho retry do timeout. Khi `AWS_NEW_RETRIES_2026=true`, `RetryCost` mặc định là 14 cho lỗi nhất thời, `ThrottlingRetryCost` là 5 cho lỗi throttling, còn `RetryTimeoutCost` không được dùng. Mỗi attempt thành công cộng `NoRetryIncrement` 1 token; retry thành công còn hoàn lại token đã lấy theo đường retry. Đây là hai bộ mặc định của cùng Standard retryer, không phải hai retry mode của SDK. Các cost, capacity và rate limiter đều có thể được cấu hình lại; khi quota không đủ, retry bị chặn. Vì thế không thể gán một chi phí token cố định cho mọi lỗi mạng hoặc mọi cấu hình AWS.

---

04. `github.com/prometheus/client_golang` (**v1.24.1** — `d6087ee4`)

Nhiều goroutine cập nhật cùng counter có thể tranh chấp lock và cache line chứa state chia sẻ. Chi phí phụ thuộc workload, số writer, kiến trúc và đường đồng bộ; không hứa nó tăng theo một quy luật chung khi thêm core. Cần profile/benchmark trước khi chọn một primitive khác.

Trong `prometheus/counter.go` của version ghim, `type counter` giữ hai phần state và có hai đường cập nhật, không chỉ một vòng CAS cho mọi thao tác:

```
type counter struct {
    // valBits lưu giá trị float64 bằng bit representation
    valBits uint64
    valInt  uint64
    // ...
}
```

Bộ đếm tách giá trị ra làm hai thành phần: số nguyên (`valInt`) và số thực thập phân (`valBits`). Khi bạn gọi hàm thông dụng nhất là `Inc()` (tương đương `Add(1)`), hàm nhận thấy giá trị tăng là số nguyên dương. Nó lập tức kích hoạt đường dẫn nhanh (fast path):

```
atomic.AddUint64(&c.valInt, 1)
```

`Inc` dùng `atomic.AddUint64`. Trong `Add`, nhánh integer kiểm tra `float64(uint64(v)) == v`; nếu không thỏa, chẳng hạn `0.15`, code cập nhật `valBits` bằng vòng CAS cho phép cộng float. Không gọi mọi giá trị nguyên là nhánh integer: khả năng biểu diễn còn quan trọng. Chi phí atomic vẫn phụ thuộc target, contention và cache coherence; không có số nanosecond cố định hay ranking "nhanh/chậm" cho mọi môi trường từ hai nhánh này.

Độ phức tạp tiếp theo nằm ở metric có nhãn động: `MetricVec` (`prometheus/vec.go`) dùng cơ chế tra cứu theo bộ giá trị nhãn và tái dùng metric đã tạo. Điều này không chứng minh mỗi lần tra cứu đều không cấp phát; chi phí thực tế cần đo với tập nhãn và đường gọi cụ thể.

Ở scrape path, `Gather()` thu thập metric families, còn handler chọn encoder theo format negotiation và ghi response. Gathering và encoding là hai bước khác nhau; chi phí cấp phát phụ thuộc số metric, tần suất scrape và encoder, không chỉ việc không dùng JSON.

---

05. `go.opentelemetry.io/otel` (v1.46.0 — 58db4c89)

Telemetry có overhead và cần budget. Export đồng bộ thêm thời gian chờ vào request path; queue không giới hạn có thể giữ quá nhiều memory khi backend lỗi. Không có tỷ lệ latency tăng gấp đôi phổ quát. Batch processor chọn queue, timeout, drop/block policy và shutdown behavior; đo overhead và loss theo workload thay vì coi instrumentation là miễn phí.

Ở version ghim, `batchSpanProcessor` trong `sdk/trace/batch_span_processor.go` dùng buffered channel có dung lượng giới hạn để chuyển span sang đường export. Đây là chi tiết implementation hữu ích để hiểu đầy queue và mất span, không phải lợi thế mặc định của channel so với mọi ring buffer:

```
type batchSpanProcessor struct {
    e              SpanExporter
    o              BatchSpanProcessorOptions
    queue          chan ReadOnlySpan
    dropped        atomic.Uint32
    // ...
}
```

Đoạn rút gọn dưới đây giữ nhánh enqueue/drop sau kiểm tra sampled span; source thật còn có context và instrumentation, không nên chép fragment này như một hàm nguyên vẹn của thư viện:

```
func (bsp *batchSpanProcessor) enqueueDrop(
    sd ReadOnlySpan,
) bool {
    select {
    case bsp.queue <- sd:
        return true
    default:
        bsp.dropped.Add(1)
        return false
    }
}
```

Trong đường không bật blocking, `select` có nhánh `default` làm enqueue không chờ khi queue đầy. Span có thể bị bỏ và bộ đếm `bsp.dropped` tăng. Nhánh này chỉ giới hạn việc chờ dung lượng queue, không chứng minh cả lời gọi không có chi phí hoặc không tranh chấp. Bật blocking đổi trade-off; shutdown, exporter và timeout còn có contract riêng.

Ở đầu ra, một goroutine chạy nền (`bsp.processQueue`) lắng nghe hai tín hiệu: một timer chu kỳ (`batchTimeout`, mặc định 5 giây) hoặc kích thước lô gom tụ (`maxExportBatchSize`, mặc định 512 spans). Khi một trong hai điều kiện thỏa mãn, lô spans được rút ra và chuyển cho `Exporter.ExportSpans(ctx, batch)`.

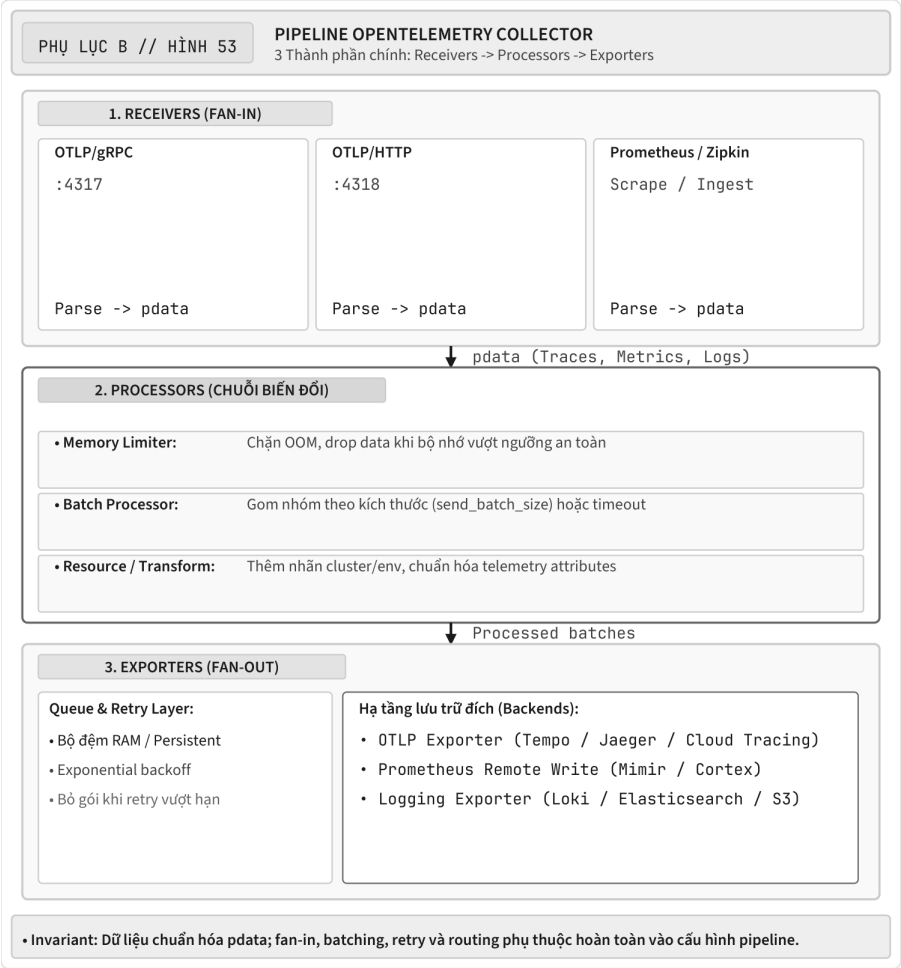
`TraceContext` (`propagation/trace_context.go`) extract/inject context theo W3C, trong đó có header `traceparent`. Consumer phải cấu hình và dùng propagator ở boundary thích hợp; đây không phải bảo đảm trace hoàn chỉnh. Khi tạo child span, TraceID nối cùng trace còn SpanID mới phân biệt span với parent. Truyền `context.Context` giúp mang quan hệ ấy qua các hàm, nhưng không tự tạo span, sample hoặc export dữ liệu.

---

06. `go.opentelemetry.io/collector` (v0.161.0 — `0bf928af`)

Nếu SDK đo telemetry trong tiến trình, Collector là đường tiếp nhận, xử lý và chuyển tiếp dữ liệu ở cấp hạ tầng. Một bản phân phối Collector có thể cấu hình receiver, processor và exporter để nối nhiều nguồn/đích; danh sách OTLP, Prometheus, Jaeger, Zipkin, Datadog, Elasticsearch hay S3 phụ thuộc component thực sự được đóng gói và cấu hình, không mặc định có trong core module.

Hợp đồng cấu hình Collector nối ba loại component trong pipeline:



Hình 53 — Pipeline nối receiver, các processor được cấu hình và exporter. Fan-in, fan-out, queue và retry phụ thuộc cách nối component cùng cấu hình cụ thể.

1. **Receiver:** Lắng nghe trên cổng mạng (gRPC/HTTP), chuyển đổi payload của các giao thức khác nhau về mô hình dữ liệu nội bộ chuẩn hóa mang tên `pdata` (`go.opentelemetry.io/collector/pdata`).
2. **Processor:** Áp dụng các quy tắc biến đổi: gom lô (`batch`), lọc dữ liệu (`filter`), hoặc lấy mẫu (`probabilistic_sampler`).
3. **Exporter:** Dịch `pdata` ngược lại định dạng của hệ thống đích và đẩy qua mạng.

`consumer.Capabilities` cho phép component khai báo nó có thể sửa dữ liệu. Đó là thông tin đầu vào cho việc xử lý chia sẻ dữ liệu ở fan-out, không phải một bảo đảm toàn pipeline không copy; khi thiết kế processor, caller phải tôn trọng hợp đồng ownership của component đang dùng.

Exporter helper có queue/retry policy tùy exporter và cấu hình. Persistent queue dùng storage extension khi được hỗ trợ và cấu hình; không phải mặc

định mọi exporter đều có queue, cũng không phải RAM queue đầy sẽ tự spill xuống disk. Phải kiểm tra capacity, unit, overflow policy, retry timeout và storage của exporter đang dùng.

---

07. `github.com/moby/moby` (v28.5.2 — `89c5e8fd`)

Trong đường Linux container đang xét, runtime tổ chức process với namespace, filesystem và cgroup theo cấu hình; nó dùng kernel của host, không tự có kernel riêng như VM. Không phải mọi container bật đủ cùng một tập namespace, và cgroup chỉ giới hạn những resource đã cấu hình. Moby điều phối các thành phần này; Docker trên host khác có boundary triển khai khác.

Khi mổ xẻ `daemon/daemon.go` (`type Daemon`), bạn sẽ nhận ra Docker daemon không trực tiếp thực thi container. Nhiệm vụ chính của nó là quản lý trạng thái và phối hợp tài nguyên. Máy trạng thái vòng đời của một container được kiểm soát nghiêm ngặt tại `container/state.go` (`type State`):

```
type State struct {
    sync.Mutex
    Running           bool
    Paused            bool
    Restarting        bool
    OOMKilled         bool
    Dead              bool
    Pid               int
    ExitCodeValue     int
    ErrorMsg          string
    StartedAt         time.Time
    FinishedAt        time.Time
    // ...
}
```

Mutex của `State` được dùng làm lock chung với `Container`. Các cờ không luôn loại trừ nhau: paused process vẫn có thể Running; `Dead` không chỉ là tên khác của một process vừa nhận SIGKILL. Lock bảo vệ các state

transition theo implementation, không tự chứng minh toàn bộ lifecycle bên ngoài không có race.

Khía cạnh kỹ thuật ngoạn mục thứ hai của Moby là hệ thống tập tin phân lớp (Layered Filesystem) thông qua đồ thị driver lưu trữ `daemon/graphdriver/overlay2`. Khi bạn khởi chạy một container từ image Ubuntu:

- Các layer của image được mount dưới dạng các thư mục chỉ đọc (`lowerdir`).
- Container được cấp một thư mục ghi duy nhất (`upperdir`).
- Kernel kết hợp chúng lại thành một thư mục ảo duy nhất (`merged`) bằng hệ thống tập tin `overlayfs`.

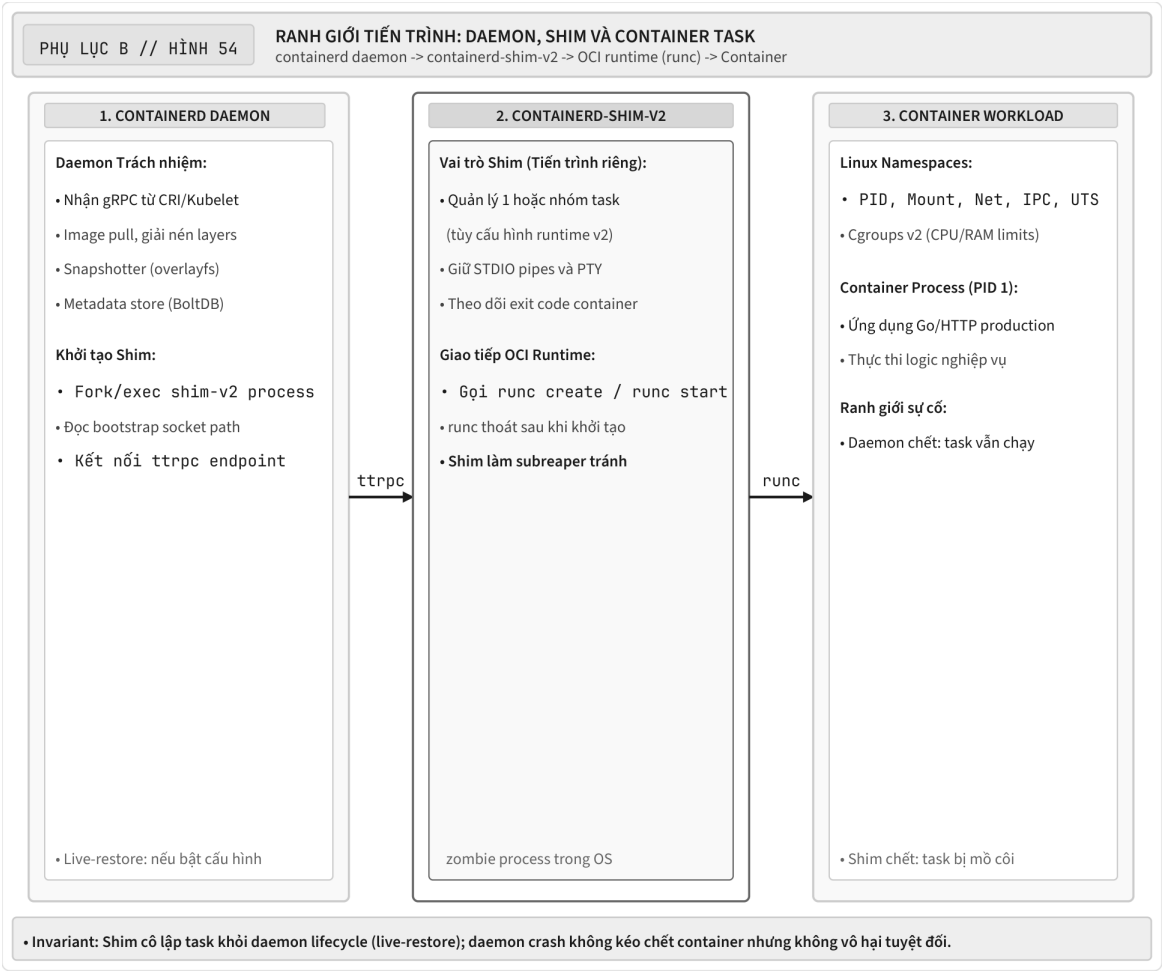
Với một file thường nằm trong layer chỉ đọc, OverlayFS có thể copy-up sang writable layer trước khi sửa, tùy operation và cấu hình filesystem. Không dùng `/etc/hosts` làm ví dụ mặc định vì runtime thường quản lý nó bằng mount riêng. Shared volume và bind mount có ownership khác image layer. Copy-on-write không chứng minh container isolation tuyệt đối hay một thời gian start cố định.

---

08. `github.com/containerd/containerd/v2` (v2.4.0 — `a7fe631d`)

Nếu daemon quản lý container gặp sự cố, vòng đời của task đang chạy có bắt buộc chấm dứt theo không? Câu trả lời phụ thuộc ranh giới giữa daemon và runtime shim, không phải một cam kết restart luôn êm cho mọi workload.

Câu trả lời nằm ở ranh giới daemon–shim của runtime v2; phía daemon được triển khai trong `core/runtime/v2/shim.go`. `containerd-shim-runc-v2` là một shim cụ thể, không đại diện mọi runtime.



Hình 54 — Daemon giao tiếp với shim qua endpoint; shim quản lý task qua runtime được chọn. Ranh giới tiến trình này không bảo đảm mọi failure của daemon đều vô hại với workload.

Daemon dùng shim process riêng để quản lý task. Ở commit được pin, `core/runtime/v2/shim.go` cho thấy containerd nạp shim, đọc bootstrap result và nối tới endpoint của shim. File này không chứng minh shim nào cũng trở thành subreaper, dùng đúng syscall `wait4` hay trực tiếp giữ mọi luồng I/O; muốn khẳng định các chi tiết ấy phải đọc implementation của shim được chọn.

Boundary giữa daemon và shim dùng endpoint có thể chọn ttrpc hoặc gRPC theo bootstrap result trong source đã pin; không có một transport hay mức tiết kiệm memory cố định cho mọi shim. Đường I/O, reaping và recovery phụ thuộc shim/runtime cụ thể.

Shim tách lifecycle task khỏi daemon containerd nên một số trường hợp daemon restart không buộc task dừng. Đây không phải bảo đảm workload không bị ảnh hưởng trong mọi failure; reattach, state storage và hoạt động

đang cần daemon có thể lỗi. Thời gian recovery cần đo trên version và cấu hình thật, không gán vài chục millisecond cho mọi host.

---

09. `github.com/hashicorp/terraform-plugin-framework` (v1.19.0 — `c7ac25e8`)

String Go có nhiều giá trị, trong đó chuỗi rỗng vẫn là giá trị hợp lệ; pointer có nil và các giá trị không nil. IaC cần biểu diễn thêm việc giá trị chưa biết tại plan time và null theo schema, thay vì dùng chuỗi rỗng cho tất cả trạng thái thiếu dữ liệu.

Hãy xem xét kịch bản sau: Bạn viết một file cấu hình Terraform để tạo một mạng VPC mới, và trong cùng một kế hoạch đó, bạn tạo một Subnet sử dụng thuộc tính `vpc_id = aws_vpc.main.id`. Khi bạn chạy lệnh `terraform plan`, tài nguyên VPC chưa hề được tạo trên AWS, do đó `vpc_id` chưa hề tồn tại. Nếu Terraform gán giá trị đó bằng chuỗi rỗng `""` hoặc `nil`, các bước kiểm tra hợp lệ logic (validation) sẽ báo lỗi cú pháp sai, hoặc tệ hơn, provider sẽ gửi một request vô nghĩa lên cloud provider làm hỏng toàn bộ kế hoạch. Ngược lại, nếu coi nó là đã có giá trị, hệ thống sẽ hành xử sai.

Ở boundary dữ liệu Terraform, `terraform-plugin-framework` dùng các value biểu diễn được known, null và unknown, thay vì chỉ dùng một giá trị primitive Go không mang đủ trạng thái. Điều này không có nghĩa implementation không còn dùng kiểu Go bên trong. Các interface và kiểu value được xem tại `attr/value.go` và các kiểu liên quan:

```
type StringValue struct {
    state attr.ValueState
    value string
}
```

`types/basetypes/string_value.go` dùng `state` để phân biệt ba trạng thái:

- **Known (Đã biết):** Giá trị đã được xác định cụ thể (ví dụ `"us-east-1"`).

- **Null:** Giá trị vắng mặt theo type model; có thể đến từ cấu hình bỏ trống, biểu thức trả null hoặc logic provider. Không suy ra ý định người dùng chỉ từ `IsNull()`.
- **Unknown:** Giá trị chưa biết trong phase đang xét, chẳng hạn thuộc tính computed ở plan. Nó không chỉ áp dụng cho tài nguyên cloud hay luôn được giải quyết bởi một tài nguyên phụ thuộc.

`internal/fwserver/server.go` tổ chức các operation của framework; tầng server adapter cung cấp provider protocol tương ứng. Khi đọc `IsUnknown()` và `IsNull()`, câu hỏi cần trả lời là provider sẽ trì hoãn, giữ null hay tính một giá trị nào ở phase này. Gộp hai trạng thái thành chuỗi rỗng sẽ làm mất thông tin mà Terraform cần khi plan và apply.

---

10. `helm.sh/helm/v3` **(v3.22.0 —** `144ca65f`**)**

Một thay đổi kiến trúc của Helm 3 là bỏ daemon Tiller của Helm 2. Quyền Kubernetes của Tiller phụ thuộc ServiceAccount và RBAC được cấu hình; cấu hình quá rộng tạo rủi ro leo quyền cho người gửi lệnh tới Tiller.

Helm 3 bỏ Tiller và thực hiện action từ client. Quyền Kubernetes phụ thuộc cấu hình REST client và credential thực tế, có thể từ kubeconfig hoặc môi trường chạy service. Đây là thay đổi trust boundary, không phải lời hứa giải quyết mọi rủi ro quyền hạn; chart, hook và quyền client vẫn phải được review.

Không có Tiller thì release history được lưu ở đâu? Đây là đường cần lần theo khi điều tra rollback:

Một storage driver của Helm nằm ở `pkg/storage/driver/secrets.go` (`type Secrets`): khi chọn driver Secrets, release history được lưu trong Kubernetes Secret ở namespace triển khai, với nhãn nhận diện. Đây là đường mặc định phổ biến, không phải storage backend duy nhất Helm hỗ trợ:

```
name: sh.helm.release.v1.my-app.v1
labels:
  name: my-app
  owner: helm
  status: deployed
  version: "1"
```

Toàn bộ thông tin của bản phát hành — bao gồm source chart, file `values.yaml` đã merge, và toàn bộ chuỗi manifest YAML kết quả sinh ra từ `pkg/engine/engine.go` — được gom lại thành một struct `Release`, sau đó được tuần tự hóa JSON, nén bằng thuật toán gzip, mã hóa base64 và nhét trọn vẹn vào trường `data["release"]` của Secret.

Khi rollback, Helm đọc release history từ storage đã cấu hình và dùng action rollback để tạo revision mới, rồi cập nhật tài nguyên qua Kubernetes API. Đây không đơn thuần là giải nén một Secret, tính diff và Patch mọi tài nguyên. Helm 3 bỏ Tiller; quyền thực tế phụ thuộc credentials và RBAC của client đang gọi.

---

11. `github.com/go-git/go-git/v5` **(v5.19.2 — `3eeb238d`)**

Một image `scratch` không tự mang shell, git hay C runtime; các biến thể distroless có thành phần khác nhau và có thể mang library native. Nếu image không có executable git, gọi `exec.Command` không thể dùng nó. go-git là một lựa chọn library in-process; lựa chọn khác là đóng gói git phù hợp. Phải kiểm tra đúng image digest và dependency cần dùng, không suy ra từ nhãn “tối giản”.

`go-git` tách các thao tác object/packfile ở tầng plumbing khỏi các thao tác repository ở tầng porcelain. Phân tầng này giúp đọc đường đi của dữ liệu mà không cần gọi Git CLI:

Tầng plumbing (`plumbing/format/packfile/parser.go`, `packfile.go`) đọc packfile chứa các Git object, trong đó object delta tham chiếu một base object bằng offset hoặc object ID. Base không nhất thiết là "file của commit cũ"; đối tượng có thể là blob, tree, commit hoặc tag. Đường giải mã cần tìm base và áp dụng chuỗi delta để tái tạo object. Không suy ra toàn bộ object graph luôn được giữ trong RAM.

`plumbing/storer/storer.go` khai báo `EncodedObjectStorer`, tách thao tác với Git object khỏi backend lưu trữ. Hai lựa chọn cho ví dụ là:

- `storage/filesystem`: Ghi các object và reflog xuống thư mục `.git` trên ổ đĩa vật lý như Git truyền thống.
- `storage/memory`: Lưu trữ toàn bộ các object Git bên trong một mảng băm trên RAM (`storage/memory.Storage`).

```
r, err := git.Clone(memory.NewStorage(), nil, &git.CloneOptions{
    URL: "https://github.com/my-org/my-repo",
})
```

Đoạn code trên minh chứng cho tính linh hoạt của kiến trúc: Một controller GitOps có thể clone một repository, đọc nội dung file cấu hình YAML tại commit HEAD, tính toán diff, và kết thúc vòng lặp mà không lưu object storage của Git xuống ổ đĩa. `memory.Storage` tránh được chi phí filesystem I/O cho lưu trữ Git objects, nhưng vẫn tốn CPU cho hashing và giải nén delta, cùng với áp lực allocation/GC tương ứng với kích thước repository.

---

12. `golang.org/x/crypto/ssh` (v0.57.0 — `3f62bf11`)

Một `ssh.Client` có thể multiplex shell, SFTP và port-forwarding trên cùng một connection nếu các consumer dùng chung phiên ấy. Mở nhiều tab terminal độc lập thường tạo nhiều connection nếu không cấu hình chia sẻ. Channel của protocol cho phép nhiều luồng logic, không bảo đảm mọi ứng dụng SSH mặc định chỉ dùng một TCP socket.

Làm thế nào SSH có thể điều phối nhiều luồng dữ liệu khác nhau trên một socket mà không bị xáo trộn hoặc làm nghẽn lẫn nhau?

Mã nguồn tại `ssh/mux.go` (`type mux`) và `ssh/channel.go` (`type channel`) chứa đựng câu trả lời. Giao thức SSHv2 phân chia kết nối TCP duy nhất thành nhiều **kênh logic (logical channels)**. Mỗi khi bạn gọi `client.NewSession()`, một gói tin `msgChannelOpen` được gửi qua mạng để đàm phán một kênh logic mới với ID riêng biệt. Bộ phân kênh `mux` lắng nghe luồng socket vật lý, đọc header của từng gói tin SSH và định tuyến các payload dữ liệu vào channel tương ứng.

Xét scenario một channel liên tục xuất dữ liệu trong khi consumer đọc chậm, còn channel khác dùng port forwarding. Cần phân biệt cửa sổ nhận riêng của channel với tài nguyên TCP dùng chung; không thể suy ra thông lượng hay thời gian chặn từ tên thao tác `cat /dev/urandom`.

SSH có flow control theo channel để receiver giới hạn lượng dữ liệu sender được gửi cho channel đó. Nó không loại bỏ contention của connection chung, TCP head-of-line blocking hay giới hạn memory của application; vẫn cần budget và xử lý consumer chậm.

- Mỗi kênh logic sở hữu một biến kích thước cửa sổ nhận dữ liệu độc lập (`myWindow uint32`).
- Bên gửi chỉ được phép truyền một lượng byte tối đa bằng đúng kích thước cửa sổ mà bên nhận đã cấp phép.
- Khi ứng dụng của bạn gọi `session.StdoutPipe().Read(buf)` và đọc bớt dữ liệu ra khỏi bộ đệm, mã nguồn trong `channel.go` mới gửi một gói

tin kiểm soát đặc biệt mang tên `msgChannelWindowAdjust` để cấp thêm hạn ngạch nhận byte cho bên gửi.

Nếu consumer ngừng đọc và cửa sổ nhận của kênh đó hết, sender không thể tiếp tục gửi data cho riêng kênh ấy cho tới khi có điều chỉnh cửa sổ. Các kênh khác có cửa sổ riêng, nhưng vẫn chia sẻ cùng TCP connection và tài nguyên process; vì vậy không có bảo đảm chúng luôn "mượt mà" khi một consumer chậm.

PHẦN 2: NHÓM HỆ THỐNG CHUYÊN TRÁCH (TIER A: 13–30)

13. `github.com/open-policy-agent/opa` (**v1.20.2** — `b2c26708`)

Một gateway cần budget cho authorization theo SLO và concurrency của chính nó. Không suy ra budget một millisecond chỉ từ request rate; đo policy, input, contention và end-to-end latency trước khi chọn cách evaluate.

Tại sao OPA có thể đánh giá các tập luật phức tạp viết bằng ngôn ngữ Rego nhanh hơn đáng kể so với phân tích cú pháp lại từ đầu?

OPA parse và compile policy/query thành các representation nội bộ rồi evaluate với input/data. Không mô tả mọi JSON data là một trie, cũng không khẳng định evaluation không thể chạy regex: policy có thể gọi builtin regex. Đọc đường evaluator và builtin của source đã pin khi cần hiểu chi phí thực tế.

Với policy/query ổn định được dùng lặp lại, cân nhắc `PrepareForEval` để tái dùng phần chuẩn bị. Tạo query mới cho mỗi request có thể lặp lại công việc parse/compile, nhưng lựa chọn phụ thuộc lifecycle và yêu cầu policy động. Prepared query không tự chứng minh mọi input đã được authorize đúng:

```
query, err := rego.New(
    rego.Query("data.authz.allow"),
    rego.Compiler(compiler),
).PrepareForEval(ctx)
```

`PrepareForEval(ctx)` tạo prepared query dùng lại cho các lần `Eval(ctx, rego.EvalInput(input))`. Evaluation vẫn thực hiện logic policy và builtin, không chỉ đối soát con trỏ. Đo phần chuẩn bị riêng với evaluation khi benchmark; không gán một speedup chung.

---

14. `github.com/sigstore/cosign/v2` **(v2.6.5 — `3e82f50a`)**

Khi bạn kéo một container image `registry.internal/app:v1.2.0` về triển khai lên cụm Kubernetes sản xuất, làm sao bạn có thể chứng minh với hệ thống kiểm toán rằng image này thực sự được sinh ra từ pipeline CI/CD chính thức của công ty chứ không phải do một hacker nội bộ sửa đổi đè lên registry?

Cosign hỗ trợ chữ ký tách rời gắn với image digest; ký số nói chung không bắt buộc sửa image. Cách lưu và tìm signature phụ thuộc artifact format và registry support của version được dùng.

Một đường lưu chữ ký kiểu tag dùng tên suy ra từ digest, chẳng hạn:

```
registry.internal/app:sha256-abc....sig
```

Chữ ký gắn với identity của image theo digest, không sửa các layer của image. Đường lưu và truy vấn còn tùy artifact format, OCI referrers và hỗ trợ

registry ở bản Cosign đang dùng; ví dụ tag trên không phải định dạng bắt buộc cho mọi chữ ký.

Tính năng nổi bật của Cosign là chế độ **Ký Không Cần Quản Lý Khóa (Keyless Signing)**: Thay vì lưu trữ private key trên máy chủ CI/CD (nơi rất dễ bị lộ lọt), Cosign tích hợp với hai dịch vụ của Sigstore:

1. **Fulcio:** Cấp chứng chỉ ngắn hạn theo policy của CA và identity OIDC; đọc validity thực tế thay vì coi mọi chứng chỉ luôn sống đúng mười phút.
2. **Rekor:** Ghi nhận chữ ký vào một sổ cái nhật ký minh bạch bất biến (Transparency Log).

Trong keyless verification, verifier vẫn cần trust roots và policy identity/issuer, cùng các kiểm tra signature, certificate và transparency evidence của đường verify đang dùng. Một chữ ký hợp lệ không chứng minh image an toàn hay đến từ workflow được phép nếu identity chưa được đối chiếu.

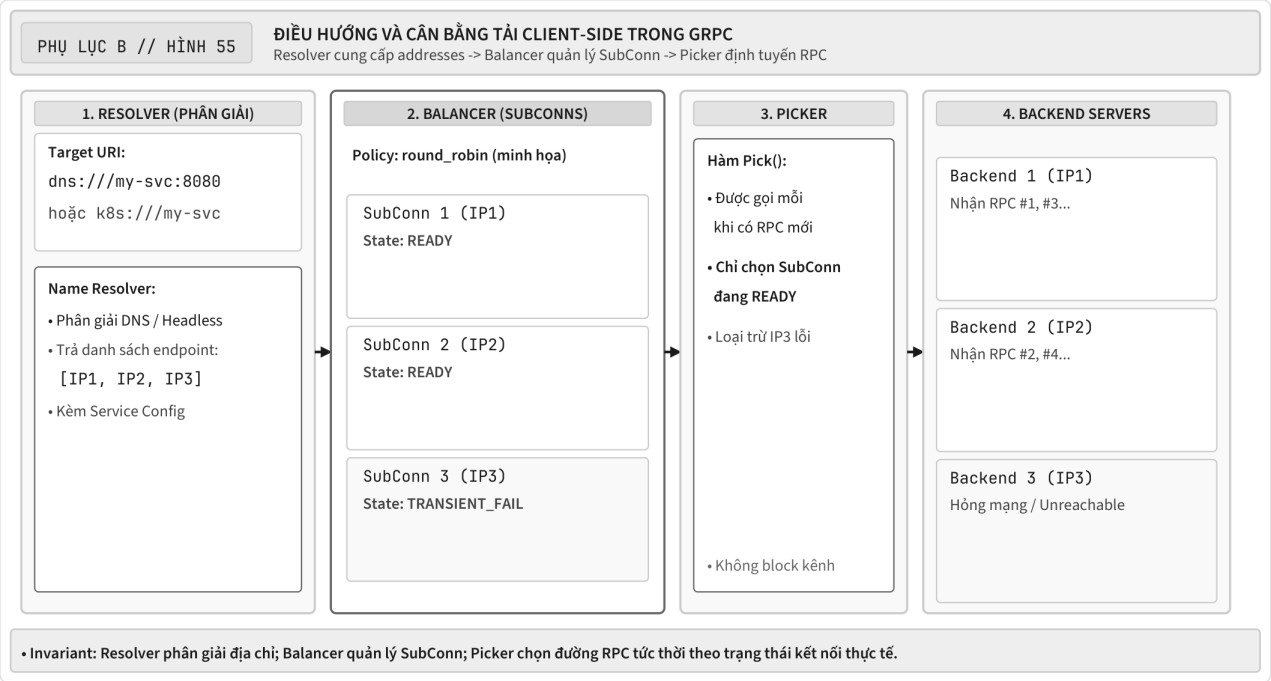
---

15. `google.golang.org/grpc` (v1.84.0 — `e84aa5ab`)

Xét scenario có một gRPC client duy trì connection lâu tới Service có nhiều backend. Nếu connection đó được route vào một Pod, nhiều RPC trên nó có thể cùng tới Pod ấy, dù còn backend khác. Đây là ví dụ về granularity cân bằng tải, không phép đo CPU hay cam kết rằng backend chắc chắn sập.

Ranh giới là connection so với request: cân bằng L4 không tự phân phối từng RPC trên một connection tới backend khác. HTTP/1.1 cũng hỗ trợ keep-alive và connection reuse; vấn đề không riêng REST hay gRPC. Muốn phân phối RPC cần resolver biết các endpoint thích hợp và balancer policy phù hợp; resolver chỉ thấy một ClusterIP không tự nhìn thấy toàn bộ Pod.

Để cân bằng tải thực sự, `grpc-go` triển khai kiến trúc **Cân Bằng Tải Phía Máy Khách (Client-Side Load Balancing)** tại `clientconn.go` (`type ClientConn`) và `balancer_conn_wrappers.go`.



Hình 55 — Resolver cung cấp endpoint, balancer quản lý SubConn và picker chọn đường cho RPC. Ba backend cùng policy round_robin là cấu hình minh họa, không phải mặc định của mọi ClientConn.

Resolver cung cấp endpoint theo scheme và implementation. Với DNS headless service phù hợp, endpoint có thể là Pod IP; không hứa danh sách luôn mới hoặc đầy đủ. Policy `round_robin` có thể tạo SubConn tới endpoint và chọn đường ready cho RPC; nó không phải mặc định duy nhất. SubConn là abstraction kết nối của balancer, không tên một type trong file transport cũ.

---

16. `google.golang.org/protobuf` (v1.36.12 — `cdd4c5f7`)

Protobuf dùng field number và wire type thay cho lặp tên field trên wire. Dung lượng và tốc độ so với JSON phải đo trên schema, value, encoder và workload; không có tỷ lệ 3–10 lần chung.

Để trả lời, hãy nhìn vào cách các byte được mã hóa trong `encoding/protowire/wire.go`. Protobuf không lặp tên trường như `"user_id":` hay `"is_active":` trên wire. Thay vào đó, mỗi trường dùng một tag nhị phân:

```
Tag = (Field Number << 3) | Wire Type
```

Ba bit cuối cùng xác định kiểu dây (`WireType`: Varint, 64-bit, Length-delimited, hoặc 32-bit), còn các bit phía trên chứa số thứ tự trường của struct.

Phép thuật tối ưu dung lượng của Protobuf nằm ở kỹ thuật mã hóa **Varint (Variable-length Quantity)**:

- Mỗi byte chỉ sử dụng 7 bit để lưu trữ dữ liệu số nguyên.
- Bit cao nhất (Most Significant Bit - MSB) là cờ báo hiệu: nếu MSB bằng 1, nghĩa là giá trị số còn kéo dài sang byte tiếp theo; nếu MSB bằng 0, đây là byte kết thúc của số đó.
- Value 5 dùng một byte varint, chưa tính tag. ZigZag áp dụng cho `sint32`/`sint64`; `int32`/`int64` âm không tự dùng ZigZag và có thể cần mười byte varint.

Runtime protobuf có các đường coder và metadata cho generated message, trong đó có thể dùng unsafe access ở implementation phù hợp. Reflection API vẫn tồn tại; không suy ra mọi decode chỉ là đọc memory hoặc đạt một giới hạn tốc độ phần cứng từ tên file `internal/impl/message.go`.

---

17. `github.com/google/go-containerregistry` (v0.22.1 — 8a72a424)

Xét scenario chỉ cần đọc metadata hoặc tìm một file trong image lớn. `docker pull` tải những layer cần mà local store chưa có; kích thước image đã giải nén không phải số byte phải truyền. Nếu công việc chưa cần layer content, một client đọc manifest/config riêng có thể tránh tải dư. Tìm file còn cần xét layer, whiteout và filesystem view, không chỉ thấy một path trong một tar bất kỳ.

`pkg/v1/image.go` định nghĩa interface `Image`; implementation `remote.Image` có thể trì hoãn việc đọc layer. Interface tự nó không bảo đảm mọi backend đều lazy:

```
type Image interface {
    // Rút gọn từ pkg/v1/image.go; còn các method khác.
    Manifest() (*Manifest, error)
    ConfigName() (Hash, error)
    RawConfigFile() ([]byte, error)
    Layers() ([]Layer, error)
    LayerByDigest(Hash) (Layer, error)
}
```

`remote.Image` cung cấp image handle với layer access trì hoãn; đọc metadata không đồng nghĩa tải toàn bộ layer. Số request có thể gồm auth, manifest/index và config theo operation; không khẳng định chỉ một GET hoặc một kích thước metadata cố định cho mọi image.

Layer cung cấp reader cho nội dung compressed hoặc uncompressed. Muốn dùng `tar.NewReader`, dùng luồng tar đã giải nén, không đưa gzip byte trực tiếp cho parser tar. Có thể dừng sớm và đóng reader, nhưng bandwidth tiết kiệm phụ thuộc vị trí file, compression và buffering; không có một tỷ lệ từ 20GB xuống vài trăm KB chung cho mọi scan.

---

18. `oras.land/oras-go/v2` (**v2.6.2** — `105715ee`)

OCI Distribution quy định API trao đổi manifest và blob, trong đó digest hỗ trợ định danh nội dung. Authentication, authorization, backup và phân phối nhiều vùng là khả năng hay policy của registry cụ thể, không phải tất cả đều có sẵn vì nó tuân theo OCI.

Tại sao chúng ta phải xây dựng các hệ thống lưu trữ riêng biệt cho Helm charts, tệp nhị phân WebAssembly (Wasm), tệp cấu hình Terraform, file chữ ký điện tử hay tài liệu thành phần phần mềm (SBOM - Software Bill of Materials), trong khi có thể lưu trữ toàn bộ chúng trực tiếp trên OCI Registry có sẵn của doanh nghiệp?

ORAS (OCI Registry As Storage) hiện thực hóa tầm nhìn đó thông qua mã nguồn tại `registry/remote/repository.go` và `content/oci/store.go`. Trọng tâm kiến trúc của ORAS là interface `Target`:

```
type Target interface {
    content.Storage
    content.TagResolver
}
```

ORAS hỗ trợ đóng gói artifact với mediaType mô tả nội dung và liên kết qua `subject`. Ví dụ Wasm hay SBOM cần được thử với registry đích: hỗ trợ mediaType, manifest, referrers và policy có thể khác nhau. API cho phép diễn đạt một artifact không bảo đảm mọi registry chấp nhận nó.

`subject` và referrers hỗ trợ liên kết artifact. Lifecycle xóa/garbage collection phụ thuộc registry và policy; ORAS không tự bảo đảm xóa một image sẽ đồng bộ xóa mọi signature/SBOM liên quan.

---

19. `github.com/containernetworking/cni` **(v1.3.1 — `3f51e880`)**

Trong đường CNI thông thường của Pod không dùng hostNetwork, runtime chuẩn bị network namespace rồi plugin thiết lập network theo cấu hình. Không coi mọi Pod đều có namespace mới không interface: loopback, hostNetwork và plugin implementation tạo các trường hợp khác. CNI là contract giữa runtime và plugin, không một topology veth duy nhất.

Kubelet gắn Pod vào mạng lưới cluster bằng cách nào?

Đọc `pkg/skel/skel.go` với `PluginMainWithError`: CNI mô tả contract thực thi plugin process, không phải RPC qua gRPC. Cần phân biệt input điều khiển, cấu hình stdin và output/error mà runtime nhận.

Khi cần cấp phát mạng, container runtime gọi trực tiếp file thực thi của plugin CNI (ví dụ `/opt/cni/bin/bridge` hoặc `calico`) thông qua

`pkg/invoke/raw_exec.go`. Mọi tham số điều khiển được truyền qua đúng hai kênh:

1. Các biến môi trường:

- `CNI_COMMAND`: Hành động cần làm (`ADD`, `DEL`, `CHECK`, hoặc `VERSION`). - `CNI_CONTAINERID`: Định danh của container. - `CNI_NETNS`: Đường dẫn tới file namespace mạng của Pod trong hệ thống (ví dụ `/proc/12345/ns/net`). - `CNI_IFNAME`: Tên interface mà runtime yêu cầu plugin tạo trong namespace; không phải một hằng số bắt buộc `eth0`.

2. Dữ liệu cấu hình mạng dạng JSON được đẩy trực tiếp qua luồng nhập chuẩn: `os.Stdin`.

Với bridge plugin, một đường triển khai có thể tạo veth pair, nối một đầu vào bridge host, đưa đầu kia vào namespace và đặt tên theo `CNI_IFNAME`, rồi dùng IPAM/configure routes. Plugin khác có thể dùng topology khác. Kết quả JSON và lifecycle command phải theo CNI spec; ví dụ veth không định nghĩa toàn bộ chuẩn.

---

20. `github.com/cilium/ebpf` **(v0.22.0 —** `e55144e1`**)**

Để quan sát syscall hay xử lý packet trong Linux, đã có nhiều cơ chế như audit, ptrace, ftrace, packet capture và kernel module, với điểm quan sát và chi phí khác nhau. eBPF bổ sung cách nạp chương trình vào hook được kernel hỗ trợ. Chọn nó theo dữ liệu cần thu, quyền, kernel và workload, không từ một so sánh chỉ có hai lựa chọn cực đoan.

Với object eBPF đã được biên dịch trước, `cilium/ebpf` cho phép Go loader nạp và gắn chương trình mà không cần cgo hoặc Clang trên máy đích. Máy đích vẫn cần Linux kernel, feature, quyền và resource limit tương thích; đường build object cần toolchain riêng.

`prog.go` (`type ProgramSpec`) và `map.go` (`type Map`) mô tả chương trình và map; các tầng nội bộ gọi Linux `bpf` syscall khi nạp hoặc thao tác tài nguyên. Không đồng nhất hai file API này với vị trí của lời gọi syscall. Sau

khi vượt qua kernel verifier, chương trình có thể được gắn vào hook phù hợp như XDP, TC hoặc kprobe.

Đường kernel → Go có thể dùng BPF ring buffer. `ringbuf/reader.go` (`type Reader`) tạo poller cho file descriptor của map; backend không-Windows tại `ringbuf/reader_other.go` dùng epoll để chờ dữ liệu. `Read` lấy từng record theo API, không phải lời hứa "batch read" cho mọi caller. Throughput và mức tiêu thụ tài nguyên phụ thuộc hook, kích thước event và workload; không suy ra speedup từ riêng cơ chế chờ.

---

21. `github.com/vishvananda/netlink` **(v1.3.1 — `17daef60`)**

Gọi ip bằng subprocess là một dependency vào executable, quoting/arguments và lifecycle process. Một CNI plugin có thể dùng library netlink để bỏ các lượt spawn đó. Chi phí phải đo theo số operation và workload; không tự suy ra bảng process bị quá tải chỉ vì code dùng CLI.

Với các thao tác link của ví dụ, `netlink` gửi message tới kernel Linux qua Netlink thay vì khởi chạy executable `ip`. Đọc `netlink_linux.go` và `link_linux.go`; không suy ra cả ứng dụng không còn subprocess hay mọi thao tác mạng đều là Netlink.

Ở tầng transport Netlink bên dưới, việc mở socket có dạng khái niệm:

```
fd, err := unix.Socket(
    unix.AF_NETLINK, unix.SOCK_RAW, unix.NETLINK_ROUTE,
)
```

Khi gọi `netlink.LinkAdd(&netlink.Veth{...})`, thư viện tạo Netlink request và thực thi qua `NETLINK_ROUTE`, không chạy shell. `link_linux.go` cho thấy `RTM_NEWLINK` và `req.Execute`; các chi tiết mở socket và syscall gửi nằm ở tầng transport bên dưới, không phải đoạn code chép nguyên từ `LinkAdd`.

Kernel trả kết quả hoặc lỗi theo Netlink protocol. Bỏ subprocess thay đổi đường đi và dependency, nhưng thời gian tạo veth và speedup so với CLI cần

benchmark theo host, namespace và operation; không có con số vài phần mười millisecond hay hàng trăm lần chung.

---

22. `github.com/crossplane/crossplane-runtime` (**v1.20.11** — `84fc49a3`)

Kubernetes vốn được thiết kế để điều phối container trên một cụm máy chủ cục bộ. Nhưng triết lý điều hòa (Reconciliation loop) của Kubernetes xuất sắc đến mức người ta muốn dùng nó để quản lý toàn bộ thế giới điện toán đám mây: tạo database AWS RDS, cấp phát Google Cloud Storage, hay cấu hình Azure Virtual Network.

Làm thế nào để biến một API REST bất đồng bộ của AWS thành một đối tượng điều hòa tuần hoàn chuẩn mực của Kubernetes?

`crossplane-runtime` đặt hợp đồng với tài nguyên bên ngoài tại `pkg/reconciler/managed/reconciler.go`. `ExternalClient` là alias của `TypedExternalClient[resource.Managed]`; đoạn sau rút gọn interface generic ở commit đã pin:

```
type TypedExternalClient[T resource.Managed] interface {
    Observe(
        ctx context.Context, mg T,
    ) (ExternalObservation, error)
    Create(
        ctx context.Context, mg T,
    ) (ExternalCreation, error)
    Update(
        ctx context.Context, mg T,
    ) (ExternalUpdate, error)
    Delete(
        ctx context.Context, mg T,
    ) (ExternalDelete, error)
    Disconnect(ctx context.Context) error
}
```

Đọc bốn trách nhiệm sau trong reconciler cùng management policy; không coi đây là bốn lời gọi luôn chạy tuần tự ở mọi reconcile:

1. **Observe (Quan sát):** Reconciler định kỳ gọi `Observe()` để thăm dò trạng thái thực tế của tài nguyên trên cloud provider.
2. **Late initialization:** Provider có thể đưa giá trị quan sát vào field spec chưa được đặt, theo logic resource và management policy. Phải kiểm tra field nào thuộc quyền provider; không tự suy rằng mọi default luôn giữ nguyên ý định người dùng.
3. **Reconcile State (Điều hòa sai lệch):** Nếu tài nguyên chưa tồn tại, nó gọi `Create()`. Nếu tài nguyên đã tồn tại nhưng cấu hình bị sai lệch (drift) so với file YAML khai báo, nó gọi `Update()` để kéo trạng thái cloud về đúng mong muốn.
4. **Delete & Finalizer:** Khi managed resource được yêu cầu xóa, policy và finalizer điều phối cleanup tài nguyên ngoài cụm. Xóa file YAML local không tự là API delete, và managed object không phải CRD definition. Management policy có thể chọn orphan thay vì xóa; đọc đường `ShouldDelete` cùng observe/delete để hiểu lúc finalizer được gỡ.

Crossplane có thể dùng Kubernetes làm control plane cho các tài nguyên mà provider và policy hỗ trợ. Không phải mọi dịch vụ cloud hay mọi field đều được quản lý theo cùng semantics; ownership, late initialization và deletion policy cần đọc theo managed resource.

---

23. `github.com/fluxcd/pkg/runtime` (**runtime/v0.114.0** — `a1797f9a`)

Khi một hệ thống GitOps tự động hóa triển khai phần mềm cho hàng trăm microservices, tình huống sự cố tồi tệ nhất là: Một commit cấu hình bị sai cú pháp, việc đồng bộ thất bại, nhưng người vận hành không hề hay biết và phải bỏ ra hàng giờ đồng hồ đào bới qua hàng chục nghìn dòng log của pod controller để tìm nguyên nhân.

Helpers trong `runtime/conditions/setter.go` giúp controller cập nhật condition theo cách nhất quán. Condition là thông tin controller báo về quá trình xử lý, không biến mọi tài nguyên thành một state machine có quan sát đầy đủ. Vẫn cần định nghĩa trạng thái nào có ý nghĩa và ai chịu trách nhiệm ghi status.

Helper condition của Flux dùng các kiểu condition cho những resource triển khai contract tương ứng; nó không áp đặt `.status.conditions` lên mọi Custom Resource trong Kubernetes. Ví dụ:

```
conditions.MarkTrue(
    obj, meta.ReadyCondition,
    "ReconciliationSucceeded",
    "Applied revision: %s", revision,
)
```

Nếu việc đồng bộ gặp sự cố (ví dụ lỗi xác thực SSH với GitHub), controller không chỉ ghi log ra màn hình console, mà gọi:

```
conditions.MarkFalse(
    obj, meta.ReadyCondition,
    "AuthenticationFailed",
    "Invalid SSH private key",
)
```

Helper cập nhật condition trên object trong memory. Controller còn phải ghi status qua Kubernetes API; helper không tự ghi etcd. Cột hiển thị của `kubectl get` phụ thuộc CRD printer columns; muốn xem Reason/Message có thể cần đọc status chi tiết.

`ObservedGeneration` ghi generation của spec mà writer status đã xử lý. So sánh với metadata.generation giúp phát hiện status chưa theo kịp spec; nó không chứng minh cache không stale hoặc toàn bộ reconciliation đã thành công.

---

24. `github.com/google/go-github/v92` (**v92.0.0** — `5149b4d7`)

Khi viết một con bot tự động hóa GitHub Actions hoặc công cụ dọn dẹp các pull request cũ trong một tổ chức doanh nghiệp có hàng nghìn repositories, bạn sẽ phải đối mặt với bài toán phân trang (pagination) và giới hạn tần suất gọi API (Rate Limiting).

Với REST endpoint phân trang theo page trong ví dụ, GitHub chỉ đường tiếp theo bằng HTTP header `Link`, chẳng hạn `<https://api.github.com/...page=2>; rel="next"`. Không lấy loop theo page này áp cho mọi endpoint hay cursor API.

`go-github` có đường offset pagination trong `github/github.go`. Nhiều hàm list nhận `ListOptions` và trả `*github.Response`; một số API dùng cursor hoặc dạng phân trang khác, nên ví dụ sau chỉ dành cho endpoint dùng page number:

```
opt := &github.PullRequestListOptions{
    ListOptions: github.ListOptions{PerPage: 100},
}
for {
    prs, resp, err := client.PullRequests.List(
        ctx, "my-org", "my-repo", opt,
    )
    if err != nil {
        return err
    }
    // Xử lý prs...
    if resp.NextPage == 0 {
        break // Đã duyệt hết tất cả các trang
    }
    opt.Page = resp.NextPage
}
```

Trong `github.go`, `Response.populatePageValues` đọc quan hệ của header `Link`. Loop theo page dưới đây dùng `resp.NextPage`, dừng khi không có page-number tiếp theo (`0`). Response còn có thông tin cursor cho các kiểu pagination khác. SDK parse header không thay việc kiểm tra error, quota, deadline hoặc sự thay đổi dữ liệu giữa các trang.

Đồng thời, struct `github.Response` công khai các trường `Rate: Rate{Limit, Remaining, Reset}`, cho phép các worker tự động điều chỉnh tốc độ gọi hoặc chủ động ngủ đông (`time.Sleep`) cho đến thời điểm `Reset` khi nhận thấy hạn ngạch gọi API của GitHub sắp cạn kiệt.

---

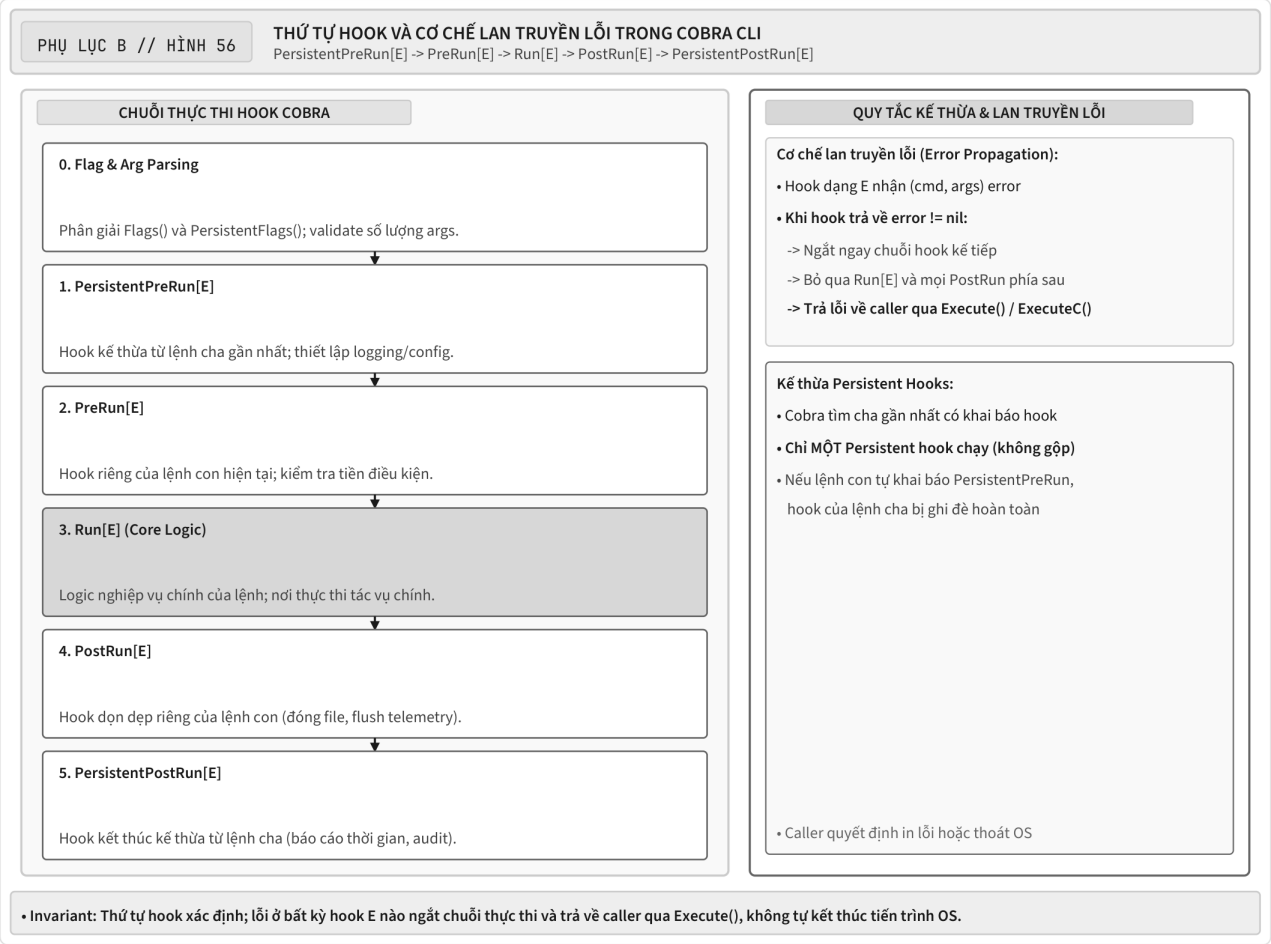
25. `github.com/spf13/cobra` (v1.10.2 — `88b30ab8`)

Một số CLI Go như `kubectl`, `helm`, `gh` và `hugo` dùng Cobra để tổ chức lệnh phân cấp, flags và help. Chúng có các quy ước tương tự, không phải mọi CLI Go đều dùng cùng framework hay có hành vi giống nhau:

`cobra` trong `command.go` tổ chức command, flag và các điểm hook. Hãy đọc đường Execute thay vì suy từ trải nghiệm CLI rằng mọi lỗi đã được framework xử lý.

Cobra tổ chức toàn bộ ứng dụng CLI dưới dạng một **Cây Lệnh Phân Cấp (Hierarchical Command Tree)**. Mỗi lệnh là một nút trên cây sở hữu con trỏ tới lệnh cha (`parent *Command`) và danh sách các lệnh con (`commands []*Command`).

Trong đường command execution của version ghim, các hook được chọn có thứ tự sau; hook có thể vắng, bản `E` có thể trả lỗi làm dừng đường chạy:



Hình 56 — Chỉ các hook được chọn mới chạy; hook có thể vắng. Lỗi từ hook E hoặc RunE ngắt đường chạy trước các hook phía sau.

Cobra phân biệt `Flags()` của lệnh với `PersistentFlags()` có thể kế thừa xuống cây lệnh. Lệnh con có thể khai báo flag trùng tên hoặc đổi behavior; đọc resolution thực tế thay vì mặc định một flag luôn có cùng nghĩa ở mọi cấp.

`PersistentPreRun` có thể đặt tác vụ khởi tạo chung ở lệnh cha. Cần kiểm tra cách Cobra chọn hook theo cây lệnh ở version đang dùng: hook của cha không phải cam kết sẽ luôn chạy nếu lệnh con khai báo hook tương ứng.

---

26. `github.com/spf13/viper` **(v1.21.0 — `394040ca`)**

12-Factor App khuyến nghị tách cấu hình thay đổi theo deployment khỏi code; đó là hướng dẫn thiết kế, không phải guarantee của Viper. Trong ví dụ, các nguồn cấu hình có thứ tự ưu tiên cần được kiểm tra:

- Khi chạy trên máy tính cá nhân (local dev): Đọc cấu hình từ file `config.yaml`.
- Khi đóng gói vào Kubernetes: Nhận cấu hình ghi đè từ các biến môi trường (Environment Variables) hoặc cờ dòng lệnh CLI.
- Khi không có cấu hình ưu tiên cao hơn, Viper có thể dùng giá trị từ `SetDefault`. Giá trị mặc định vẫn cần validation theo contract ứng dụng; framework không làm nó tự trở nên an toàn.

Làm thế nào để kết hợp tất cả các nguồn cấu hình này mà không biến mã nguồn thành một mớ hỗn độn các câu lệnh `if-else`?

Viper ở version đã pin có thứ tự ưu tiên gồm explicit `Set`, flag đã bind, environment, config file, remote key/value store và default. Đây là quy tắc tìm value, không thay validation miền giá trị hay policy secret:



Hình 57 — Với một key, Viper ưu tiên nguồn phía trên khi nguồn ấy có giá trị hợp lệ theo cấu hình. Đây là thứ tự ưu tiên khi đọc, không phải lịch nạp file.

`GetInt` đọc theo precedence đó. `DATABASE_PORT` chỉ là key môi trường tương ứng khi đã cấu hình binding hoặc key replacer phù hợp; dấu chấm trong config key không tự luôn đổi thành gạch dưới. Nếu `Set` đã override key, nó đứng trên cả flag. Sau khi tìm và chuyển giá trị, application vẫn cần kiểm tra range và báo lỗi config phù hợp.

Cơ chế này mang lại sự linh hoạt tối đa cho các kỹ sư DevOps: Ứng dụng của bạn có thể được triển khai ở nhiều môi trường mà không cần thay đổi code, chỉ điều chỉnh nguồn cấu hình theo thứ tự ưu tiên.

---

27. `github.com/fsnotify/fsnotify` (**v1.10.1** — `76b01a6e`)

Hot reload là một policy của ứng dụng, không phải tác dụng tự động của việc sửa ConfigMap. Với volume ConfigMap, kubelet có thể cập nhật file sau một khoảng trễ; biến môi trường và mount `subPath` có semantics khác. Ứng dụng còn phải nhận ra thay đổi, parse, validate và công bố config mới an toàn. Không giả định một callback fsnotify là đủ cho mọi cách cập nhật file.

`fsnotify` hiện thực hóa tính năng này bằng cách trừu tượng hóa các cơ chế thông báo sự kiện tập tin cấp thấp của từng hệ điều hành:

- Trên Linux: Sử dụng các hàm hệ thống `inotify_init1` và `inotify_add_watch` (`backend_inotify.go`).
- Trên Windows: Sử dụng hàm Win32 API `ReadDirectoryChangesW` (`backend_windows.go`).
- Trên macOS: backend kqueue ở version được pin; README vẫn ghi FSEvents chưa được hỗ trợ.

Tuy nhiên, có một cái bẫy chết người mà rất nhiều kỹ sư Go mắc phải khi triển khai hot reloading trên Kubernetes: **Cái bẫy Symlink của ConfigMap**.

Kubernetes không ghi đè trực tiếp nội dung vào file cấu hình đang mở. Thay vào đó, nó tạo một thư mục mới có tên là một chuỗi timestamp, mount dữ liệu vào đó, rồi thực hiện hoán đổi một liên kết mềm (symlink) mang tên `..data` trỏ sang thư mục mới, sau đó xóa thư mục cũ.

Nếu code Go của bạn chỉ lắng nghe sự kiện `fsnotify.Write` trên file cấu hình:

```
if event.Op&fsnotify.Write == fsnotify.Write { ... }
```

Watch một file có thể mất hiệu lực khi file được thay atomically thay vì ghi tại chỗ. Với projected volume Kubernetes, cần hiểu việc đổi symlink và sự kiện của filesystem/OS. Theo dõi directory và xử lý rename/remove, đăng ký lại khi cần, rồi parse/validate config mới trước publication là một policy có

thể kiểm tra. Không hứa hot reload tức thời hoặc không gián đoạn chỉ từ một callback fsnotify.

---

28. `go.uber.org/zap` (**v1.28.0** — `5b81b37b`)

Nếu ứng dụng của bạn xử lý tải cao, việc ghi lại log trên mỗi request có thể trở thành nguồn áp lực cấp phát đáng kể. Nếu sử dụng thư viện log thông thường dựa trên `fmt.Printf` hoặc các thư viện dùng `interface{}`:

```
log.Printf(
    "User %d transferred %f to user %d",
    fromID, amount, toID,
)
```

Đưa value vào interface không tự bảo đảm heap allocation. Escape analysis, call path và formatter quyết định allocation thực tế. Nếu logging là đường nóng, kiểm tra escape output và benchmark đúng field, formatter và destination trước khi kết luận GC pressure.

`zap` hướng tới logging có ít cấp phát ở các đường nóng; kết quả còn phụ thuộc encoder, field, output và phiên bản thư viện.

Cơ chế cốt lõi nằm ở cấu trúc `zap.Field` (`zapcore/field.go`):

```
type Field struct {
    Key       string
    Type      zapcore.FieldType
    Integer   int64
    String    string
    Interface any
}
```

Khi bạn ghi log: `logger.Info("transfer", zap.Int64("from", fromID), zap.Float64("amount", amount))`, `zap.Int64` không hề đưa số nguyên vào `interface{}`! Nó gán thẳng giá trị vào trường `Integer int64` bên trong struct `Field`.

`zapcore/json_encoder.go` dùng bộ đệm tái sử dụng qua `sync.Pool` và ghi trực tiếp nhiều trường vào buffer. Điều đó giảm allocation trong những đường đi phù hợp, nhưng không cho phép suy ra một ngưỡng nano-giây hay `0 allocs/op` cho ứng dụng khác. Muốn so sánh với logger khác, cần benchmark cùng Go version, encoder, destination và tập field.

---

29. `go.uber.org/automaxprocs` (**v1.6.0** — `1ea14c35`)

`automaxprocs` quan trọng nhất trong bối cảnh lịch sử của các binary Go cũ chạy trong container có CPU limit thấp. Trước Go 1.25, mặc định `GOMAXPROCS` không xét cgroup CPU bandwidth limit, nên một process có thể chọn số lượng song song gần số CPU host thay vì giới hạn CPU của container.

Edition dùng toolchain Go 1.27.1; các module lab có directive riêng cần kiểm tra khi build. Với Go từ 1.25 trên Linux, mặc định `GOMAXPROCS` có thể xét CPU logic, affinity và throughput limit của cgroup, rồi cập nhật định kỳ. Main module hoặc workspace khai báo Go 1.24 trở xuống có thể giữ mặc định tương thích cũ. `GODEBUG=containermaxprocs=0` tắt xét quota; `updatemaxprocs=0` tắt cập nhật. Đặt `GOMAXPROCS` thủ công bằng biến môi trường hoặc API cũng vô hiệu cập nhật tự động. CPU request của Kubernetes không phải quota này; `GOMAXPROCS` không thay việc kernel áp quota.

Vì vậy không thể kết luận rằng đặt `GOMAXPROCS` bằng quota sẽ loại bỏ throttling. Nó có thể giảm các đỉnh song song bất lợi cho một workload, nhưng GC, syscall, loại workload, quota phân số và chính sách scheduler đều còn ảnh hưởng đến độ trễ. Runtime Go 1.27 cũng có các quy tắc làm tròn và không hạ mặc định xuống dưới hai chỉ vì cgroup quota.

Một cách dùng lịch sử của `automaxprocs` là import side effect tại `maxprocs/maxprocs.go`:

```
import _ "go.uber.org/automaxprocs"
```

Khi được nạp, thư viện đọc cấu hình cgroup và gọi `runtime.GOMAXPROCS` theo chính sách của nó. Đây vẫn có thể hữu ích cho Go cũ, cho chính sách cấu hình rõ ràng, hoặc các cạnh môi trường cần kiểm chứng. Với Go 1.27 chạy mặc định, không nên xem nó là phụ thuộc bắt buộc chỉ để runtime hiểu Kubernetes CPU limit.

```
Quota = cfs_quota_us / cfs_period_us
```

Biểu thức này mô tả quota throughput của cgroup v1, không phải CPU request và cũng không phải cam kết throughput thực tế của ứng dụng. Hãy đo latency, CPU throttling và `runtime.GOMAXPROCS(0)` trên đúng workload trước khi quyết định giữ một override.

---

30. `github.com/hashicorp/go-retryablehttp` (v0.7.8 — `e1f5485f`)

Xét scenario một client retry sau response 503 rồi dần gặp lỗi thiếu file descriptor. Retry count không đủ để kết luận nguyên nhân; kiểm tra body có được đóng, số connection, goroutine và descriptor đang giữ. Đây là tình huống để luyện điều tra, không phải kết quả đo hay tần suất sự cố của thư viện.

Một nguyên nhân cần điều tra là vòng đời `response.Body` trong `net/http.Transport`. Caller phải luôn đóng `response.Body`; việc đọc đến EOF có thể là điều kiện hữu ích cho một số đường tái sử dụng kết nối, nhưng không phải bảo đảm tái sử dụng. Transport, giao thức, kích thước body, server và kết nối hiện có đều ảnh hưởng kết quả.

Nếu bạn chỉ đơn thuần viết:

```
resp, err := client.Do(req)
if err != nil {
    return err
}
if resp.StatusCode >= 500 {
    resp.Body.Close() // Có thể không reuse, không tự là leak.
    // Thực hiện retry...
}
```

Kết nối có thể không đủ điều kiện để tái sử dụng khi body bị bỏ dở. Tùy tình trạng kết nối và Transport, client có thể đóng kết nối thay vì giữ nó trong idle pool. Dưới tải cao, một pattern retry sai hoặc giới hạn connection không phù hợp có thể làm số file descriptor tăng; cần đo `httptrace`, connection metrics và lỗi thực tế trước khi kết luận nguyên nhân.

Ở `client.go`, thư viện dùng helper bounded drain trước khi đóng body. Mã minh họa policy, không phải signature chép nguyên từ source:

```
func (c *Client) drainBody(body io.ReadCloser) {
    defer body.Close()
    _, _ = io.Copy(
        io.Discard, io.LimitReader(body, respReadLimit),
    )
}
```

Trước mỗi lần thử lại, thư viện có thể dùng `io.LimitReader` để giới hạn lượng dữ liệu bỏ đi trước khi đóng body. Đây là trade-off giữa lượng đọc thêm, bộ nhớ và khả năng giữ kết nối; nó không chứng minh TCP đã được "thanh tẩy" hoặc connection chắc chắn quay lại pool.

PHẦN 3: NHÓM MỞ RỘNG CHUYÊN SÂU (TIER B: 31–43)

31. `golang.org/x/sync` **(v0.23.0 —** `f75267d8`**)**

Xét scenario nhiều request cùng thấy cache miss cho một key rồi cùng truy vấn database. Các lời gọi chồng thời gian có thể tạo tải dư thừa. Số request, capacity và ảnh hưởng cần đo trên workload thật; tên cache stampede không cho một ngưỡng gây sập chung.

`singleflight.Group` trong `singleflight/singleflight.go` cho phép gộp các lời gọi cùng key đang diễn ra:

```
var g singleflight.Group
v, err, shared := g.Do("product_123", func() (any, error) {
    return fetchProductFromDB("product_123")
})
```

`singleflight.Group.Do` gộp những lời gọi cùng key đang chồng thời gian: một call chạy function, các caller trùng chờ và nhận cùng kết quả. Implementation Do dùng WaitGroup; DoChan có đường channel riêng. Sau khi call hoàn tất, call mới có thể chạy function lại; đây không phải cache và không bảo đảm 10.000 request bất kỳ chỉ tạo một query. Scope, cancellation, error sharing và Forget cần policy riêng.

`errgroup.WithContext` tạo group và derived context bị cancel khi hàm đầu tiên trả lỗi hoặc `Wait` kết thúc. Một zero `errgroup.Group` không tự có context để cancel. `semaphore.Weighted` hỗ trợ giới hạn concurrency theo trọng số; caller còn phải release phần đã acquire.

---

32. `golang.org/x/time` (**v0.16.0** — `fb013b3d`)

Một implementation token bucket minh họa có thể dùng goroutine với ticker để nạp token vào channel. Khoảng tick là lựa chọn của ví dụ, không phải thành phần bắt buộc của thuật toán.

Nếu tạo một goroutine và timer cho mỗi limiter, tài nguyên nền tăng theo số limiter ngay cả khi nhiều limiter không có request. Cần đo chi phí ấy thay vì gắn một số người dùng với kết luận hết tài nguyên. `rate.Limiter` chọn cách tính lượng token theo thời gian đã trôi qua khi có thao tác.

Mã nguồn của `rate.Limiter` tại `rate/rate.go` (`type Limiter`) triển khai **Thuật Toán Token Bucket Không Cần Goroutine Chạy Nền**.

Bên trong struct `Limiter` hoàn toàn không có goroutine hay timer nào cả. Các trường trạng thái quan trọng của implementation hiện tại bao gồm:

`sync.Mutex` bảo vệ toàn bộ struct, `limit Limit` (tốc độ nạp token), `burst int` (dung lượng tối đa), `tokens float64` (số token hiện có), `last time.Time` (thời điểm cập nhật cuối), và `lastEvent time.Time` (dùng cho Reserve). Đây là implementation detail của version hiện tại, không phải API contract.

Khi một request gọi vào hàm `AllowN(now, n)`:

1. Nó lấy thời điểm hiện tại `now`.
2. Nó tính khoảng thời gian trôi qua kể từ lần gọi cuối: `Δt = now - last`.
3. Nó tính lượng token khả dụng từ elapsed time và rate, có giới hạn burst.
4. Đường reserve quyết định có chấp nhận không; không phải mọi lần `AllowN` thất bại đều ghi lại `last` và trừ token. `Wait`/reservation còn có semantics và timer riêng, khác đường `Allow` không chờ.

Implementation tính token theo khoảng thời gian khi gọi thao tác, không có goroutine/timer nền riêng cho từng `Limiter`. Nhưng `Wait` có thể tạo timer cho caller cần chờ; operation vẫn tốn CPU, lock và state. Số instance phù hợp phải theo workload và ngân sách, không suy ra khả năng scale không có chi phí.

---

33. `github.com/hashicorp/go-plugin` (**v1.8.0** — `155dcddc`)

Package `plugin` có giới hạn tương thích và platform được tài liệu hóa. Host và plugin cần toolchain/build configuration cùng các dependency chung tương thích; khác biệt có thể gây lỗi nạp hoặc runtime failure. Không phải cứ lệch một flag là chắc chắn panic ngay. Out-of-process plugin đổi boundary tương thích sang protocol và lifecycle IPC, với chi phí và failure mode riêng.

`go-plugin` tổ chức host và plugin thành các process giao tiếp qua IPC. Cần phân biệt handshake nhận diện executable/protocol với authentication; cookie không bảo vệ trước một plugin độc hại.

Với đường plugin dùng `go-plugin`, host giao tiếp với **tiến trình plugin riêng** qua RPC. Không suy rộng cơ chế này thành mọi loại plugin và extension của Terraform.

Quá trình bắt tay diễn ra như sau:

1. Host khởi chạy process plugin bằng API tạo process của OS, với environment có magic cookie và cấu hình handshake. Không mô tả đường Windows là fork.
2. Tiến trình plugin khởi động, mở một điểm lắng nghe cục bộ (Unix Domain Socket hoặc port TCP ngẫu nhiên tùy cấu hình), và in ra stdout dòng thông báo sẵn sàng kèm theo địa chỉ kết nối.
3. Plugin kiểm tra magic cookie để tránh chạy nhầm ngoài host; host parse dòng địa chỉ và protocol trên stdout để kết nối. Cookie không xác thực peer. Transport `net/rpc` hay gRPC và cấu hình TLS có trust boundary riêng.

Lợi ích của thiết kế này bao gồm: Plugin có thể được viết bằng ngôn ngữ khác nhau và biên dịch độc lập, miễn là phù hợp với transport và protocol được cấu hình (gRPC path hỗ trợ cross-language tốt hơn net/rpc path). Về **cách ly sự cố (Fault Isolation)**: panic xảy ra trong tiến trình con plugin không trực tiếp panic tiến trình mẹ — đây là lợi ích thực sự của kiến trúc out-of-process. Tuy nhiên plugin vẫn có thể làm host treo (hung RPC), tiêu hao CPU/RAM/I/O của máy chủ, hoặc làm hỏng tài nguyên chia sẻ bên ngoài; không nên hiểu là cách ly hoàn toàn.

---

34. `github.com/hashicorp/hcl/v2` (v2.25.0 — `00057cf0`)

Tại sao HashiCorp không dùng YAML hay JSON để viết cấu hình cho Terraform mà lại kỳ công sáng tạo ra một ngôn ngữ riêng mang tên HCL (HashiCorp Configuration Language)?

HCL native syntax hỗ trợ block, attribute và expression; HCL còn có JSON syntax. Việc dùng HCL cho Terraform cho phép tách parse cấu trúc khỏi evaluate biểu thức theo context. YAML hay JSON là format dữ liệu khác; khả

năng biểu thức còn phụ thuộc application dùng chúng, không phải một tuyên bố rằng YAML không thể mô hình hóa cấu hình phức tạp.

Mã nguồn của `hcl/v2` được cấu trúc thành hai tầng tách bạch rõ rệt: Tầng Cú Pháp Cấu Trúc (`hclsyntax/parser.go`) và Tầng Đánh Giá Ngữ Cảnh (`eval_context.go`).

HCL có thể parse cấu hình trước khi có giá trị cho các biến trong biểu thức. `EvalContext` cung cấp biến và hàm khi caller quyết định evaluate biểu thức; HCL không tự xây hay lập lịch đồ thị phụ thuộc của Terraform. Terraform là application dùng HCL và quyết định lúc nào các giá trị tham chiếu sẵn sàng.

---

35. `github.com/hashicorp/terraform-plugin-go` **(v0.31.0 —** `09a1181b`**)**

Nếu `terraform-plugin-framework` (thư viện số 09) là giao diện cấp cao thân thiện dành cho lập trình viên, thì `terraform-plugin-go` là tầng nền móng cấp thấp (Low-Level RPC Protocol) giao tiếp trực tiếp với Terraform Core.

Package `tfprotov6` định nghĩa hợp đồng RPC của Terraform Provider Protocol v6. Các giá trị động có thể mang dữ liệu MessagePack hoặc JSON theo protocol. Không quy toàn bộ dữ liệu trạng thái thành một định dạng nhị phân duy nhất khi đọc contract này.

`tftypes.Value` biểu diễn type, null và unknown khi trao đổi dữ liệu với Terraform. Caller cần giữ schema và trạng thái ấy qua conversion; có type model không tự bảo đảm mọi adapter của application đều không làm mất thông tin.

---

36. `github.com/prometheus/common` **(v0.71.0 —** `9a4aff03`**)**

Một scrape workload nhiều target có thể tốn CPU ở parser. Chi phí phụ thuộc số sample, nhãn, format và input; không suy ra trần CPU từ một số target giả định. Đọc parser của expfmt để hiểu grammar, rồi benchmark đúng dữ liệu nếu lựa chọn parser là một quyết định performance.

`expfmt/text_parse.go` triển khai parser trạng thái cho Prometheus text exposition format.

Bộ giải mã đọc luồng đầu vào và nhận diện tên metric, bộ nhãn, giá trị cùng timestamp theo grammar. Không suy ra số cấp phát, lợi thế performance hoặc mức phổ biến của format từ riêng cấu trúc parser; nếu throughput là tiêu chí chọn format, đo trên dữ liệu thật.

---

37. `modernc.org/sqlite` (v1.59.0 — `c96a4e6c`)

Driver dùng cgo cần C toolchain phù hợp khi build và có thêm allocator/lifetime boundary. Cross-build cần native target support. Chi phí crossing phụ thuộc operation và môi trường; không có số nanosecond chung nếu không có benchmark cùng hardware, payload, Go version và cấu hình.

Driver `modernc.org/sqlite` tại version đã pin không dùng cgo ở đường driver. Đây là lựa chọn triển khai; cần kiểm tra target được hỗ trợ và các dependency của chính version được build.

Đây là driver SQLite không dùng cgo trong đường build được tài liệu hóa, không phải lời hứa tương thích và hiệu năng cho mọi target. Cơ chế chuyển dịch C sang Go, biểu diễn con trỏ và allocator là chi tiết implementation cần đọc đúng source ở version được ghim; không dùng suy đoán về bố trí bộ nhớ để chọn driver.

Tránh cgo có thể đơn giản hóa build trên target mà driver hỗ trợ. Không kết luận driver Go luôn dùng nhiều memory hơn hay luôn chậm hơn bản C: so sánh cần cùng SQLite version, query, concurrency, database, hardware và cấu hình. Chọn driver theo compatibility, lifecycle và chi phí vận hành trước; nếu performance quyết định lựa chọn, benchmark đúng workload.

---

38. `go.opentelemetry.io/contrib` (**v1.46.0** — `c4c6248e`)

OpenTelemetry Go cốt lõi (mục 05) cung cấp API/SDK. Các module contrib cung cấp integration cho thư viện chuẩn như `net/http` và cho thư viện khác như client gRPC. Cần chọn module và version tương thích với SDK; đây không phải một bộ wrapper tự bao phủ mọi dependency của ứng dụng.

`otelhttp` cung cấp wrapper cho `http.Handler` trong `instrumentation/net/http/otelhttp/handler.go`.

Khi bọc một `http.Handler` tiêu chuẩn bằng `otelhttp.NewHandler(handler, "my-operation")`, middleware thực hiện một chu trình hoàn chỉnh:

1. Trích xuất context từ request header theo propagator đã cấu hình. Đọc `traceparent` cần propagator hỗ trợ W3C Trace Context; không tự suy ra chỉ từ wrapper.
2. Khởi tạo một Server Span mới và nhúng vào `r.Context()`.
3. Dùng `request.NewRespWriterWrapper` cùng `httpsnoop.Wrap` để ghi nhận status code và số byte phản hồi, đồng thời giữ các interface tùy chọn của writer gốc. `http.ResponseWriter` chuẩn không có hàm đọc lại status code sau khi ghi.
4. Khi handler kết thúc, middleware đặt span status theo semantic convention, ghi response attributes và kết thúc span. Cách diễn giải mã 5xx thuộc semantic convention được dùng, không phải một lệnh `RecordError` vô điều kiện trong handler.

Wrapper tạo điểm instrumentation cho đường request đã bọc. Trace propagation, sampling, exporter và middleware khác vẫn quyết định dữ liệu cuối; một dòng bọc handler không chứng minh trace xuyên service đã đầy đủ.

---

39. `github.com/aquasecurity/trivy` (**v0.74.0** — `e1fd17a0`)

Trivy phát hiện package/version từ artifact rồi đối chiếu nguồn vulnerability phù hợp. Scan latency phụ thuộc image, cache, scanner được bật, database và I/O; không có cam kết vài giây cho mọi image.

Mã nguồn tại `pkg/fanal/artifact/artifact.go` hé lộ quy trình bóc tách từng lớp (layer by layer) của Trivy.

Artifact analysis xử lý filesystem/layer và các analyzer đã chọn. Các file metadata thường có ích gồm:

- Hệ điều hành Linux: `/var/lib/dpkg/status` (Debian/Ubuntu), `/lib/apk/db/installed` (Alpine), hoặc cơ sở dữ liệu RPM.
- Ứng dụng ngôn ngữ: `go.sum`, `package-lock.json`, `pom.xml`, `requirements.txt`.

Kết quả phụ thuộc analyzer và dữ liệu package có trong artifact; không bảo đảm suy ra mọi version chính xác từ mọi file. Trivy còn có các scanner khác như secret và misconfiguration, nên không mô tả mọi scan là chỉ đọc manifest trong RAM. Vấn đề cache, database freshness và scanner coverage cần ghi trong scan contract.

---

40. `github.com/in-toto/in-toto-golang` (**v0.11.0** — `36d782ff`)

Trong một quy trình CI/CD hiện đại, mã nguồn trải qua nhiều bước: Lập trình viên commit -> CI checkout -> Biên dịch nhị phân -> Chạy unit test -> Đóng gói container image. Kẻ tấn công có thể không tấn công được vào Git, nhưng có thể can thiệp vào máy chủ build để hoán đổi file nhị phân ngay sau khi biên dịch xong và trước khi đóng gói.

`in-toto` cung cấp model metadata và verifier để kiểm tra các bước mà layout yêu cầu; đọc `in_toto/model.go` (`type Link`) để biết dữ liệu nào thực sự được ghi.

Mỗi bước trong pipeline CI/CD được ghi nhận lại bằng một tài liệu bằng chứng gọi là **Link Metadata**:

- **Materials:** Tên và digest của những đầu vào được recorder thu thập theo cấu hình; SHA-256 là một thuật toán thường dùng, không phải mọi file đều mặc nhiên được ghi.
- **Products:** Tên và digest của đầu ra được thu thập. Artifact ngoài tập này không được bảo vệ chỉ vì bước có một Link.
- Metadata có thể được ký bởi recorder với key được layout tin. Chữ ký chỉ ràng buộc dữ liệu đã ghi với key, không chứng minh recorder không bị can thiệp.

Verifier đối chiếu signatures và rules của layout với materials/products đã được ghi. Coverage phụ thuộc những file và bước được policy yêu cầu, cùng trust của key và recorder; không hứa phát hiện mọi tráo đổi nếu một artifact hoặc boundary chưa được ghi nhận.

---

41. `github.com/theupdateframework/go-tuf/v2` (v2.4.2 — `f5edbde3`)

Khi máy chủ tự động tải các bản cập nhật phần mềm hoặc chữ ký container từ xa, nó phải đối mặt với nhiều hình thức tấn công tinh vi: Kẻ tấn công có thể giả mạo máy chủ cập nhật, hoặc nguy hiểm hơn, thực hiện cuộc tấn công đóng băng thời gian (Freeze/Replay Attack): liên tục gửi lại một bản cập nhật cũ đã có lỗ hổng bảo mật nhưng chữ ký vẫn còn hợp lệ.

TUF tách trách nhiệm metadata giữa các role để giới hạn trust và kiểm tra update theo version, expiration và signature:

1. **Root Role:** Xác định các key và threshold được tin cậy cho metadata. Giữ root key offline là khuyến nghị bảo vệ quan trọng; không phải bằng chứng rằng mọi deployment thực tế đều làm vậy.
2. **Targets Role:** Ký xác nhận danh sách các tệp tin cập nhật cùng mã hash và dung lượng chính xác.
3. **Snapshot Role:** Ghi version và có thể ghi hash/length của metadata targets và delegated roles cần dùng, nhằm kiểm tra một tập metadata nhất quán. Nó không kiểm kê mọi file của hệ thống.

4. **Timestamp Role:** Tham chiếu snapshot metadata và có expiration để giúp client phát hiện freeze. Thời hạn là policy triển khai; timestamp không phải chứng thực liên tục giờ hệ thống của server.

TUF client kiểm tra metadata version, expiration, role signatures và các quan hệ theo policy/spec; expiration hỗ trợ chống freeze, không thay mọi cơ chế chống rollback. Không suy ra mọi hệ ký artifact như Sigstore đều dùng cùng update protocol TUF ở từng operation.

---

42. `cloud.google.com/go` **(v0.123.0 —** `4e837358`**)**

Với download lớn trên WAN, retry từ byte đầu có thể lặp nhiều công việc. Range read/resume có thể hữu ích, nhưng phải kiểm tra generation/identity của object và contract API; không ghép các byte từ hai version khác nhau thành một file rồi gọi đó là download thành công.

Thư viện Cloud Storage của Google giải quyết bài toán phục hồi dữ liệu tại `storage/reader.go` (`type Reader`).

`Reader` có API `io.ReadCloser`. Trong đường HTTP của `storage/http_client.go`, `httpReader` giữ số byte đã đọc và callback reopen; lỗi retryable có thể mở lại range theo offset và generation. Không phải mọi lỗi hoặc transport đều cùng behavior. Đường decompressive transcoding còn có trường hợp phải đọc bỏ byte đã thấy; kiểm tra source và generation condition thay vì hứa mọi lần resume đều không lặp dữ liệu truyền.

Metadata server là một nguồn credential trong môi trường Google Cloud được cấu hình phù hợp. Việc ứng dụng nhận identity nào phụ thuộc môi trường và cấu hình xác thực, kể cả đường Workload Identity; không suy ra từ riêng `storage.Reader` hay `compute/metadata` rằng mọi container đều có token đúng quyền.

---

43. `github.com/Azure/azure-sdk-for-go/sdk/azcore` **(sdk/azcore/v1.23.1 —** `d86ae78b`**)**

Azure SDK for Go dùng policy pipeline để tổ chức các bước như authentication, retry và transport. Với client ARM đang xét, xem `sdk/azcore/runtime/pipeline.go`; không suy ra rằng mọi lời gọi HTTP trong một ứng dụng đều đi qua pipeline này.

Một pipeline của Azure là một chuỗi các `Policy` được thực thi tuần tự:

- **Authentication Policy:** Dùng `TokenCredential` mà caller cấp; Managed Identity chỉ là một cách cung cấp credential, không mặc định cho mọi client.
- **Telemetry Policy:** Gắn metadata nhận diện SDK/request theo cấu hình; không khẳng định Azure portal trực tiếp dùng trường đó cho mọi request.
- **Retry Policy:** Áp dụng retry theo policy cho response/error phù hợp, có xét thông tin server như `Retry-After` khi đường thực thi hỗ trợ.

Client có thể thêm policy theo các điểm mở rộng mà SDK hỗ trợ. Thứ tự, per-call/per-retry scope và dịch vụ dùng client ấy phải được kiểm tra; không suy ra một cấu hình client thay đổi mọi dịch vụ Azure trong process.

PHẦN 4: HẠ TẦNG AI AGENT & ĐIỀU PHỐI HIỆN ĐẠI (FRONTIER: 44–50)

44. `github.com/modelcontextprotocol/go-sdk` **(v1.8.0 —** `3f3b699b`**)**

MCP định nghĩa cách client và server trao đổi tool/resource qua protocol; ứng dụng vẫn phải xác định server nào được tin, dữ liệu nào được phép đọc và hành động nào caller được phép yêu cầu.

Go SDK tổ chức server trong `mcp/server.go` và transports trong package `mcp`. Giao thức dùng JSON-RPC với stdio hoặc HTTP transport phù hợp version; Streamable HTTP không đồng nghĩa transport SSE cũ. SDK không tự biến một tool thành hành động được phép.

JSON Schema là kiểm tra cấu trúc, không phải bộ phát hiện prompt injection. Một argument có thể đúng schema nhưng chứa yêu cầu không được phép; chính sách handler phải kiểm tra nghĩa và quyền.

JSON Schema kiểm tra cấu trúc và constraint đã khai báo, không phân biệt mọi nội dung chuỗi an toàn với instruction độc hại. Handler vẫn cần semantic validation, authenticated identity, authorization và giới hạn egress/side effect. Process sandbox là một lớp có trade-off riêng; CNI chỉ thiết lập network, không tự là security sandbox cho Agent.

---

45. `github.com/google/adk-go` (**HEAD-main** — `f9ce16ef`)

ADK tổ chức model call, tool và session thành một vòng thực thi nhiều bước. Đọc đường runner ở commit ghim để biết state nào được giữ và lúc nào một tool được gọi; không suy ra quyền tự trị từ tên framework.

Ở ADK, đọc package `agent` và `tool/functiontool`. Wrapper function tool dùng config name/description và input/output types hoặc schema được cung cấp; không giả định nó tự parse mọi docstring Go thành quyền hay tool semantics.

Session, runner và tool call tạo các boundary lifecycle khác nhau. Khi triển khai, phải kiểm tra budget cho model calls, token, thời gian và side effect bằng API thực có ở version được pin; không dựa vào một field `MaxSteps` chưa được đối chiếu. Framework không tự cấp quyền production hay bảo đảm task hoàn tất.

---

46. `github.com/microsoft/agent-framework-go` (**HEAD-main** — `5fea5266`)

Ở commit đã ghim, hãy đọc feature và giới hạn của Go implementation trước khi chọn nó cho ứng dụng; nhãn enterprise không thay thế test compatibility và failure recovery.

README của commit đã pin mô tả các package `agent`, `provider`, `tool` và `workflow`. Đây là Go implementation đang public preview, không có feature parity mặc định với .NET/Python. Middleware, workflow và

checkpoint là các abstraction để đọc tiếp; không quy một `runtime.go` tưởng tượng thành bảo đảm tự planning, authorization hay Redis/CosmosDB session isolation.

---

47. `github.com/cloudwego/eino` **(v0.9.21 —** `ba04fde8`**)**

`eino` của CloudWeGo cung cấp các component và orchestration cho ứng dụng LLM trong Go. So sánh overhead với framework Python cần workload và phép đo tương đương, không suy ra từ ngôn ngữ implementation.

`compose/graph.go` mô hình hóa graph và mode chạy. Directed graph không mặc nhiên là DAG: source phân biệt Pregel mode có thể chứa cycle với DAG mode và validation tương ứng.

Mỗi thành phần trong pipeline — bộ thu hồi dữ liệu RAG (`Retriever`), mẫu câu lệnh (`PromptTemplate`), mô hình ngôn ngữ (`ChatModel`), và công cụ (`Tool`) — được coi là một Node trên đồ thị, kết nối với nhau bằng các cạnh (Edges) truyền dữ liệu kiểu mạnh.

Một stream abstraction có thể phục vụ nhiều consumer, nhưng fan-out cần policy backpressure, cancellation và bộ đệm. Nếu một nhánh hiển thị token cho người dùng trước khi nhánh moderation kết luận, đó là kiểm tra sau phát hành, không phải bảo đảm chặn nội dung trước khi lộ ra. Với yêu cầu chặn trước, cần buffer/gate theo đơn vị nội dung đã chọn và chấp nhận độ trễ tương ứng; không coi streaming song song là safety boundary tự có.

---

48. `trpc.group/trpc-go/trpc-agent-go` **(v1.11.2 —** `5a0030b6`**)**

Một Agent phụ thuộc provider có thể gặp throttling, timeout hoặc lỗi transport. Tần suất và latency cần đo ở provider, model và thời điểm thật; không có con số ba mươi giây chung cho mọi API.

Khi dùng `trpc-agent-go`, kiểm tra các package agent/tool và middleware của version thực tế. Hai yêu cầu thiết kế sau là policy cần triển khai và test, không claim rằng framework tự bật sẵn chúng:

- **Circuit Breaker (scenario):** Chọn threshold và cửa sổ quan sát để chuyển sang Open, giới hạn request thử lại và định nghĩa half-open. Fallback cần model/semantics phù hợp; không có ngưỡng 20% hay local model dự phòng tự động cho mọi hệ thống.
- **Metrics & Tracing:** Thiết kế correlation, đo token và thời gian theo provider response, rồi kiểm tra export và privacy. Có span không chứng minh billing được đo chính xác.

---

49. `github.com/kagent-dev/kagent` (HEAD-main — `375fe73a`)

Nếu bạn muốn trao quyền cho một AI Agent tự động điều tra nguyên nhân sự cố trong cụm máy chủ Kubernetes, làm thế nào để ngăn chặn con AI đó vô tình thực hiện một lệnh tai hại như xóa nhầm namespace `production`?

README ở commit này mô tả `Agent`, `ModelConfig`, `ToolServer` và controller/engine/UI/CLI. Kubernetes-native resource là cách tổ chức lifecycle, không bảo đảm Agent không thể xóa namespace.

Không suy ra từ chữ task rằng mọi invocation là một CRD `Task` hoặc luôn tạo một sandbox Pod mới. Nếu ứng dụng chọn sandbox-per-task, đó là deployment policy phải đối chiếu controller source và cấu hình, cùng identity/egress của tool server thực sự chạy hành động.

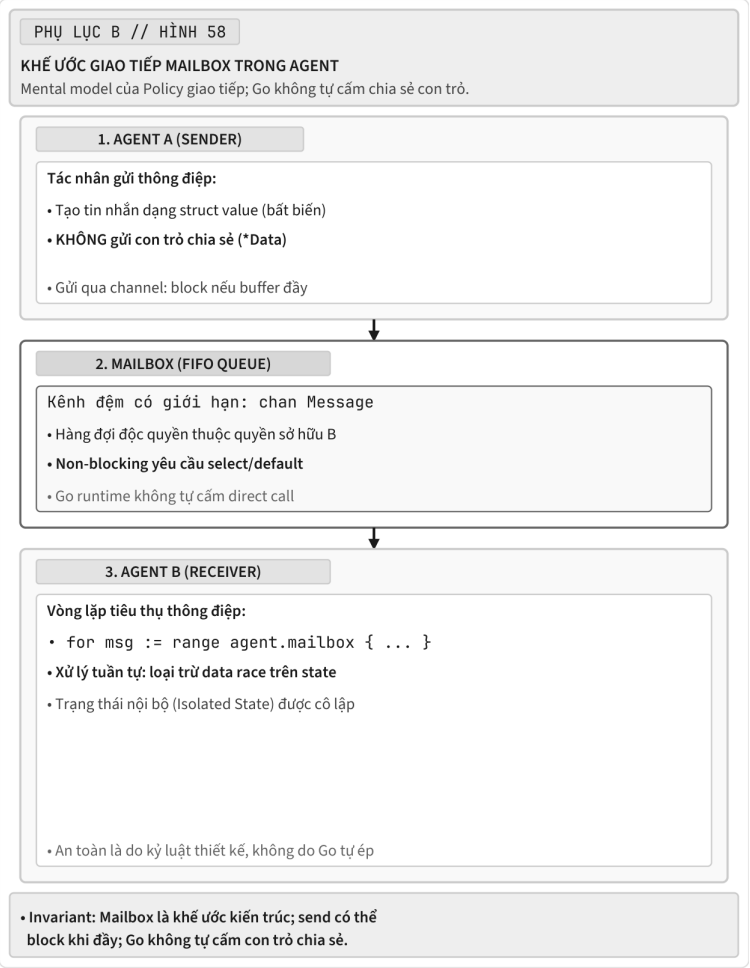
Trong scenario này, sandbox chỉ được cấp quyền đọc cần thiết và có owner dọn tài nguyên. Nhưng ServiceAccount không đại diện toàn bộ sandbox: token mount, network egress, host mounts, security context và quyền bên ngoài Kubernetes vẫn phải được kiểm tra. Pod không có quyền sửa qua Kubernetes API vẫn có thể tạo side effect trên một endpoint khác nếu egress và credential cho phép.

---

50. `github.com/agentscope-ai/agentscope-go` (HEAD-main — `8f82bd22`)

Khi nhiều Agent cùng sửa state có pointer dùng chung, chương trình vẫn cần synchronization và ownership như các chương concurrency đã dạy. Race hoặc deadlock không tất yếu chỉ vì có nhiều Agent; chúng phụ thuộc access và giao thức phối hợp.

`agentscope-go` là một lựa chọn tổ chức Agent, message và lifecycle. Khi đọc source đã pin, phải phân biệt abstraction Actor với những mechanism implementation thực sự có; tên framework không chứng minh isolation hoặc mọi failure đã được giải quyết.



Hình 58 — Mailbox là mental model của policy giao tiếp. Go và framework không tự cấm direct call hay chia sẻ pointer giữa các agent.

Một mô hình actor có thể đặt state và mailbox riêng cho từng Agent, nhưng Go không tự cấm direct call hay chia sẻ pointer. Thiết kế ứng dụng phải quy định giao tiếp và ownership, rồi kiểm tra các access thực tế. Không suy ra an toàn chỉ từ tên pattern.

README mô tả agent, tools, workflow, contexts và channels. Diagram mailbox phía trên là mental model thiết kế ứng dụng, không khẳng định mỗi Agent trong framework luôn là một actor process cô lập. Budget vẫn cần xét model calls, memory, network và lifecycle của từng implementation.

---

5. TỔNG KẾT & QUY TẮC BẢO TOÀN KIẾN TRÚC

Sau các chương, sách đặt Atlas mã nguồn 50 thư viện Go DevOps và cloud để nối kiến thức với implementation thực tế. Phụ lục A — Atlas Lỗi Go — nằm sau Atlas mã nguồn và luôn là nội dung cuối cùng của sách.

Atlas này không tồn tại độc lập mà là điểm tựa thực tế cho các chương trước đó trong cuốn sách:

- Informer và controller-runtime liên hệ **Chương 17, 21–23**; Chương 16 dạy observability, không phải Kubernetes client API.
- Kỹ thuật `sync/atomic` trong `client_golang` và `zap` minh họa cho **Chương 08** (Một race bắt đầu từ đâu) và **Chương 10** (Khi chương trình chậm hoặc phình).
- Vòng lặp xử lý `io.ReadCloser` và drain body trong `aws-sdk-go-v2` và `go-retryablehttp` neo chặt vào **Chương 11** (Lần theo một request HTTP) và **Chương 20** (OpsProbe Capstone).
- Tinh chỉnh CFS quota của `automaxprocs` và netlink socket giải thích tường tận các ranh giới hệ điều hành được trình bày tại **Chương 15** (Từ incident đến công cụ).

`library_sources/lock.json` khóa identity của 50 nguồn, không bảo đảm một binary hay cấu hình khác có cùng behavior. Đọc API contract, implementation và test như ba lớp bằng chứng khác nhau. Các scenario/policy là yêu cầu thiết kế của bài học; những chi tiết source chỉ có phạm vi commit ở heading, không phải một lời hứa về mọi version tương lai.

PHỤ LỤC A — ATLAS LỖI GO

Đọc lỗi từ triệu chứng đến nguyên nhân

Phụ lục này là tài liệu tra cứu kỹ thuật và phản xạ chẩn đoán nhanh, đóng vai trò điểm hội tụ cuối cùng cho các lỗi Go và lỗi vận hành xuất hiện xuyên suốt cuốn sách theo nguyên tắc Grayscale-first. Mỗi mục gồm: mã định danh chẩn đoán (bao gồm chuỗi lỗi thực tế `EXACT`, giá trị sentinel chuẩn `SENTINEL`, trạng thái nền tảng `STATUS`, hoặc nhóm hiện tượng chẩn đoán `FAMILY`); bản chất cơ chế trong 1 câu; điểm cảnh báo hiểu lầm (`!`) nếu có; hành động kiểm tra đầu tiên (→); cùng tham chiếu chương (`[ChX,Y]`).

Bảng phân nhóm Taxonomy

`EXACT` là diagnostic hoặc phần diagnostic thực có, có thể thay tên và số bằng placeholder; không bao gồm prefix đường dẫn/vị trí do tool thêm. Các heading lược thuật ngữ, ghép nhiều chuỗi hay ghi tên tool để định vị được phân loại `FAMILY`, không giả làm byte output để copy so khớp. Wording còn phụ thuộc toolchain, OS, driver và version; kiểm tra output thật trước khi kết luận nguyên nhân.

Nhóm	Tên nhóm	Phạm vi chẩn đoán	Mã lỗi
A	Compiler & Type System	Lỗi biên dịch, hệ thống kiểu, interface, scope và syntax	A01–A14
B	Runtime & Panic	Crash process, nil pointer dereference, bounds, panic	B01–B11
C	Error Values & I/O	Sentinel errors, stream truncated, encoding/decoding	C01–C08
D	Context & Cancellation	Hủy tác vụ, timeout, deadline cascade, rò rỉ context	D01–D04
E	Filesystem & Process	File I/O, quyền hạn Linux, binary PATH, signals OS	E01–E07
F	Network / HTTP / TLS	DNS resolution, TCP handshake, reset socket, TLS/x509	F01–F11
G	Database	Connection pool, locking SQLite, transaction lifecycle	G01–G06
H	Concurrency	Deadlock toàn cục, data race warning, goroutine leak	H01–H05
I	Modules / Test / Toolchain	go.mod resolution, go vet static analysis, go test timeout	I01–I08
J	Container / Kubernetes / CI-CD	OOMKilled, CrashLoopBackOff, probe failure, gate reject	J01–J11

Bản đồ phản xạ 30 giây

Triệu chứng nhận diện	Hướng tra	Triệu chứng nhận diện	Hướng tra
Không biên dịch được mã nguồn?	**Nhóm A**	Sự cố DNS / TCP / TLS / HTTP?	**Nhóm F**
Biên dịch được nhưng panic / crash?	**Nhóm B**	Database nghẽn / lỗi transaction?	**Nhóm G**
Hàm trả về `err != nil` / I/O stream?	**Nhóm C**	Data race / deadlock / leak goroutine?	**Nhóm H**
Tác vụ bị timeout / cancel?	**Nhóm D**	Lỗi go build / test / vet / module?	**Nhóm I**
Lỗi file / permission / process?	**Nhóm E**	Pod crash / OOM / probe fail / gate?	**Nhóm J**

A — COMPILER & TYPE SYSTEM

A01 `undefined: x`

Không tìm thấy identifier trong scope hiện tại hoặc identifier chưa được export từ package ngoài.

→ Kiểm tra lỗi chính tả tên biến/hàm, scope khai báo, hoặc viết hoa chữ cái đầu nếu gọi từ package khác. **Ch1,3,5**

A02 `no new variables on left side of :=`

Tất cả các biến ở vế trái toán tử `:=` đều đã được khai báo trước đó trong cùng một scope.

→ Đổi toán tử sang phép gán thường =, hoặc bổ sung ít nhất một biến mới vào vế trái. **Ch1,2**

A03 `"fmt" imported and not used`

Diagnostic này được compiler Go 1.27.1 phát cho import `fmt` không được dùng; tên package khác làm chuỗi thay đổi. Dot import cũng cần dùng identifier của package, còn blank import khai báo chủ đích chỉ chạy initialization.

→ Xóa dòng import thừa, dùng blank identifier `_ "pkg"` nếu cần chạy hàm `init()`, hoặc chạy `goimports`. **Ch1,5**

A04 `declared and not used: x`

Biến cục bộ được khai báo nhưng không có bất kỳ dòng code nào tiếp theo đọc hay sử dụng giá trị của nó.

→ Sử dụng biến vào logic, gán vào blank identifier `_ = x` nếu đang debug dở, hoặc loại bỏ khai báo thừa. **Ch1,2**

A05 `cannot use x (variable of type T1) as T2 value in argument`

Sai khác kiểu dữ liệu trong phép gán hoặc truyền đối số; Go không hỗ trợ ép kiểu ngầm định (implicit coercion).

→ Thực hiện ép kiểu tường minh `T2(x)` nếu hai kiểu tương thích, hoặc sửa signature của hàm nhận. **Ch2,3,14**

A06 `assignment mismatch`

- `1 variable but f returns 2 values`
- `2 variables but f returns 1 value`

Số lượng biến nhận ở vế trái không khớp chính xác với số lượng giá trị trả về của hàm ở vế phải.

→ Kiểm tra signature của hàm gọi và hứng đủ tất cả các giá trị trả về (dùng _ để bỏ qua nếu cần). **Ch1,4**

A07 `missing return`

Hàm có khai báo kiểu dữ liệu trả về nhưng tồn tại nhánh rẽ control-flow đi đến cuối block mà không có lệnh `return`.

→ Rà soát các nhánh `if/else`, `switch/case` để bảo đảm mọi luồng thực thi đều trả về giá trị hoặc panic. **Ch1,4**

A08 `invalid operation: operator not defined on struct`

Sử dụng toán tử so sánh (==, !=) trên struct chứa ít nhất một field không thể so sánh (như slice, map, func).

→ So sánh từng field có thể so sánh, hoặc tự viết hàm helper so sánh nghiệp vụ chuyên biệt. **Ch2,3**

A09 `cannot assign to struct field in map`

Không thể gán trực tiếp `m["k"].Field` vì map index expression không addressable theo Go spec; đừng suy ra quy tắc này từ một địa chỉ vật lý của implementation.

→ Lấy struct ra biến tạm, cập nhật field rồi gán ngược lại vào map, hoặc dùng map chứa pointer `map[K]*V`. **Ch2,3**

A10 `cannot take the address of x`

Toán tử lấy địa chỉ & được áp dụng lên unaddressable value (giá trị trả về của hàm, hằng số, map index expression).

→ Gán giá trị đó vào một biến tạm cục bộ trước khi thực hiện lấy địa chỉ `&temp`. **Ch2,14**

A11 `constant overflows int`

Giá trị hằng số vượt quá giới hạn biểu diễn bit của kiểu số nguyên đích trong kiến trúc hiện tại.

→ Chỉ định kiểu số nguyên có độ rộng bit lớn hơn (`int64`, `uint64`), hoặc sử dụng package `math/big`. **Ch1,2**

A12 `syntax error: unexpected newline, expecting comma or }`

Thiếu dấu phẩy `,` ở phần tử cuối cùng của composite literal hoặc danh sách tham số nhiều dòng (multi-line).

→ Bổ sung dấu phẩy `,` vào cuối dòng ngay trước dấu đóng ngoặc nhọn `}` hoặc ngoặc đơn `)`. **Ch1,3**

A13 `invalid type assertion: x.(T)`

Thực hiện cú pháp type assertion trên một biến không phải kiểu interface (`non-interface type`).

→ Chỉ áp dụng type assertion trên biến kiểu interface; kiểm tra lại kiểu dữ liệu thực tế của biến `x`. **Ch3,14,19**

A14 `x does not implement I (missing method M)`

- `method M has pointer receiver`

Type cụ thể không thỏa mãn interface vì thiếu method hoặc sai khác receiver (pointer receiver vs value receiver).

→ Kiểm tra receiver của method `M`: nếu là pointer receiver `*T`, đối tượng truyền vào phải là địa chỉ `&val`. **Ch3,5,19**

B — RUNTIME & PANIC

B01 `panic: runtime error: invalid memory address or nil pointer dereference`

Dereference con trỏ nil hoặc gọi method qua nil interface có thể gây panic. Method có pointer receiver vẫn có thể nhận nil và xử lý nó; chỉ lời gọi trên receiver nil chưa đủ kết luận.

> ! Nil dereference có thể làm process crash; đừng dùng recover để che lỗi mà không xác định contract và recovery boundary.

→ Thêm kiểm tra `if ptr == nil` trước khi truy cập, hoặc rà soát constructor/hàm khởi tạo chưa gán con trỏ. **Ch2,3,12**

B02 `panic: runtime error: index out of range [x] with length y`

Truy cập phần tử của slice hoặc mảng tại chỉ số âm hoặc lớn hơn hay bằng độ dài hiện tại `len`.

→ Kiểm tra `len(s)` trước khi truy cập trực tiếp qua index, hoặc ưu tiên duyệt bằng vòng lặp `for range`. **Ch2,7**

B03 `panic: runtime error: slice bounds out of range [:x] with capacity y`

Biểu thức cắt slice `s[low:high]` có cận âm, cận trên vượt `cap(s)` hoặc cận dưới lớn hơn cận trên.

→ Với slice hai cận, kiểm tra `0 <= low <= high <= cap(s)`; string dùng `len` làm cận trên, không có `cap`. **Ch2,7**

B04 `panic: runtime error: makeslice: len out of range`

Diagnostic này chỉ đường cấp phát slice có length không hợp lệ hoặc vượt giới hạn implementation. Capacity không hợp lệ và `len > cap` có thể phát `makeslice: cap out of range`; allocation thất bại vì OOM có đường lỗi khác.

→ Kiểm tra kích thước buffer không âm và nằm trong ngân sách; buffer độ dài 0 có thể hợp lệ theo contract. **Ch2,7**

B05 `panic: assignment to entry in nil map`

Thực hiện thao tác gán giá trị vào một key của map chưa được khởi tạo (`nil map`).

→ Khởi tạo map bằng hàm `make(map[K]V)` hoặc map literal `{}` trước khi ghi dữ liệu. **Ch2,3**

B06 `panic: send on closed channel`

Thực hiện hành vi gửi dữ liệu vào một channel đã bị gọi lệnh `close()`.

> ! Bên đóng phải biết mọi sender đã ngừng gửi. `sync.Once` chỉ tránh đóng hai lần; nó không chặn việc gửi sau khi đóng.

→ Gom quyền đóng về goroutine điều phối, chờ các sender kết thúc rồi mới `close`. Producer duy nhất có thể tự đóng sau lần gửi cuối. **Ch8,9,12**

B07 `panic: close of closed channel`

Gọi lệnh `close()` lần thứ hai trên cùng một channel đã được đóng trước đó.

→ Sử dụng `sync.Once` để bảo đảm đóng channel duy nhất một lần, hoặc kiểm soát lifecycle đóng tập trung. **Ch8,9**

B08 `panic: interface conversion: interface {} is T1, not T2`

Ép kiểu dạng assertion `v := x.(T2)` thất bại trong runtime khi giá trị thực tế bên trong là `T1` hoặc `nil`.

→ Dùng `v, ok := x.(T2)` khi type động chưa được bảo đảm bởi contract. Nếu dùng assertion một giá trị, cần invariant rõ và hiểu rằng vi phạm sẽ panic. **Ch3,14,19**

B09 `fatal error: concurrent map read and map write`

Nhiều goroutine cùng truy cập đọc và ghi đồng thời vào một biến kiểu `map` chuẩn mà không có cơ chế khóa đồng bộ.

> ! Đây là lỗi fatal từ kiểm tra nội tại của runtime, không thể chặn bằng `recover`; khác với `-race`, theo dõi truy cập không đồng bộ trên những đường chạy được thực thi. Cả hai không bảo đảm phát hiện mọi race trong mọi lịch chạy.

→ Bảo vệ map bằng `sync.RWMutex`, dùng `sync.Map` khi phù hợp, hoặc tuần tự hóa qua channel; kiểm thử với `go test -race`. **Ch8,9,20**

B10 `fatal error: stack overflow`

Hàm gọi đệ quy vô tận hoặc chuỗi lồng hàm quá sâu khiến kích thước stack của goroutine tăng trưởng liên tục vượt quá giới hạn tối đa do Go runtime quy định (`maxstacksize`).

→ Bổ sung điều kiện dừng (base case) chuẩn xác cho hàm đệ quy, hoặc chuyển đổi giải thuật sang dạng lặp (iterative). **Ch10**

B11 `fatal error: sync: unlock of unlocked mutex`

Go 1.27.1 phát lỗi fatal khi gọi `Unlock()` trên `sync.Mutex` hiện không được khóa, kể cả khi mutex đã được khóa rồi mở trước đó. `recover` không cứu được lỗi này; `sync.RWMutex` có diagnostic riêng.

→ Kiểm tra cấu trúc đặt `defer mu.Unlock()` ngay sau `mu.Lock()`, tránh gọi unlock thủ công rải rác nhiều nhánh return. **Ch8,12**

C — ERROR VALUES & I/O

C01 `io.EOF`

Tín hiệu hoàn tất stream đọc dữ liệu (đã đọc hết toàn bộ byte sẵn có, không còn byte nào phía sau).

! `io.EOF` thường đánh dấu hết stream; EOF xuất hiện sớm vẫn có thể vi phạm contract của format hoặc operation.

→ Xử lý phần `n > 0` trước khi xét error; chỉ coi EOF là kết thúc thành công nếu operation đã nhận đủ dữ liệu theo contract. **Ch7,11,20**

C02 `io.ErrUnexpectedEOF`

Stream dữ liệu bị ngắt đột ngột trước khi đọc đủ số byte dự kiến (ví dụ trong `io.ReadFull` hoặc fixed-size header).

→ Kiểm tra kết nối mạng có bị ngắt giữa chừng hoặc payload truyền tải có bị cắt cụt (truncated) hay không. **Ch7,11,20**

C03 `io.ErrShortWrite`

Hàm `Write()` ghi được ít byte hơn buffer cung cấp nhưng không trả về lỗi cụ thể từ hệ thống bên dưới.

→ Kiểm tra `n` và `err`; không retry vô hạn khi writer không tiến triển. Nếu policy cho phép ghi tiếp, cần budget và bảo toàn phần chưa ghi. **Ch7**

C04 `target document exceeds byte limit`

Dữ liệu stream đi vào vượt quá ngưỡng kích thước tối đa cho phép theo chính sách an toàn (giá trị sentinel `ErrDocumentTooLarge`).

→ Bảo vệ bộ nhớ khỏi cạn kiệt (OOM); từ chối xử lý tiếp và trả về lỗi payload vượt giới hạn cho caller. **Ch7,20**

C05 `json: cannot unmarshal string into Go struct field of type int`

Kiểu dữ liệu trong JSON payload không tương thích với định nghĩa kiểu của struct field đích trong Go.

→ Đối chiếu schema JSON nguồn, điều chỉnh kiểu dữ liệu struct field hoặc triển khai `json.Unmarshaler` tùy biến. **Ch7,14,20**

C06 `unexpected end of JSON input`

Chuỗi JSON đưa vào `json.Unmarshal` bị rỗng hoặc bị ngắt cụt giữa chừng cú pháp.

! Phân biệt rõ: `json.Unmarshal` trả về lỗi cú pháp này khi buffer rỗng; trong khi `json.Decoder.Decode()` trên stream rỗng trả về `io.EOF`, và chỉ trả về unexpected end hoặc `io.ErrUnexpectedEOF` khi stream bị cắt cụt khi đang parse dở token.

→ Kiểm tra dữ liệu thực đọc được và lỗi decode; không dùng riêng `Content-Length` làm bằng chứng body đầy đủ. Giới hạn byte và xử lý stream rỗng theo contract API. **Ch7,11,20**

C07 `sql: no rows in result set`

Truy vấn `QueryRowContext` không tìm thấy bất kỳ bản ghi nào khớp với điều kiện lọc trong cơ sở dữ liệu (`sql.ErrNoRows`).

→ Dùng `errors.Is(err, sql.ErrNoRows)` để phân biệt trường hợp không có dữ liệu với lỗi truy vấn database thực sự. **Ch13,20**

C08 `sql: transaction has already been committed or rolled back`

Gọi lệnh `Commit()` hoặc `Rollback()` trên một transaction cơ sở dữ liệu (`*sql.Tx`) đã kết thúc trước đó (`sql.ErrTxDone`).

→ Rà soát pattern `defer tx.Rollback()`; nếu `Commit()` đã thành công thì rollback sau đó trả về lỗi này và an toàn để bỏ qua. **Ch13**

D — CONTEXT & CANCELLATION

D01 `context canceled`

Context bị hủy bởi `cancel()`, context cha bị hủy hoặc lifecycle của operation kết thúc.

! Operation return không tự cancel child do nó tạo. Kiểm tra ai sở hữu CancelFunc; parent sống lâu và timer chưa hết có thể giữ child ngoài nhu cầu. `go vet` sạch không chứng minh mọi đường bàn giao cancel đều đúng. [Ch12]

! `http.Server.Shutdown` không tự hủy context của active request. Muốn signal dừng process hủy handler phải truyền application context có chủ đích; policy ấy có thể làm gián đoạn graceful drain.

→ Cho operation quan sát cancellation, dọn tài nguyên rồi trả kết quả theo contract caller. `OutcomeCancel` là phân loại của opsprobe; HTTP 499 là quy ước phi chuẩn của một số hệ thống, không phải status bắt buộc của Go server. **Ch4,11,12,20**

D02 `context deadline exceeded`

Tác vụ không hoàn thành trong khoảng thời gian timeout hoặc trước mốc thời gian deadline đã ấn định (`context.DeadlineExceeded`).

> ! Deadline có thể giải phóng quan hệ context/timer, nhưng không giết goroutine không quan sát context. Đối chiếu tín hiệu hoàn tất với stack chờ, không chỉ `ctx.Err()`. [Ch9,10,12]

> ! Client bị timeout không có nghĩa là downstream service đã ngừng xử lý; kết nối có thể vẫn đang chạy ngầm gây nghẽn.

→ Xác định nơi khởi tạo deadline (`WithTimeout`), đo lường độ trễ từng phân đoạn và điều chỉnh timeout hợp lý. **Ch4,11,12,20**

D03 Client Disconnect During Body Read

Client ngắt kết nối khi handler đang đọc `req.Body`. Lỗi đọc tùy đường kết nối và trạng thái body; không bảo đảm luôn là `context.Canceled`. Incoming request context bị hủy khi kết nối đóng, request bị hủy trên HTTP/2 hoặc `ServeHTTP` kết thúc.

> ! Hủy incoming context không chứng minh công việc dùng context tách rời như `Background` đã kết thúc. Tìm đường tạo child và owner của công việc chạy nền. [Ch12]

→ Kiểm tra `r.Context().Done()` hoặc `errors.Is(err, context.Canceled)` trong các tác vụ đọc stream để kịp thời giải phóng CPU và buffer. **Ch11,12,20**

D04 Server Graceful Shutdown Timeout

Phương thức `server.Shutdown(ctx)` chạm mốc timeout của context truyền vào trước khi toàn bộ các kết nối HTTP đang hoạt động được đóng mềm mại, trả về `context.DeadlineExceeded`.

> ! Khi drain hết ngân sách, mở goroutine dump tìm handler/downstream chưa trả về và đường join của owner. Gọi cancel không đồng nghĩa đã chờ công việc dừng. [Ch9,10,12]

→ Điều tra request còn chạy và ngân sách drain. Context truyền cho `Shutdown` giới hạn thời gian chờ, không tự trở thành context của handler; kiểm tra policy hủy request riêng. **Ch12,20**

E — FILESYSTEM & PROCESS

E01 `no such file or directory`

Đường dẫn tập tin hoặc thư mục mục tiêu không tồn tại trên filesystem của hệ điều hành (`os.ErrNotExist`).

→ Kiểm tra đường dẫn tuyệt đối/tương đối, working directory thực tế của process, hoặc mount path trong container. **Ch7,15,17**

E02 `permission denied`

Process không có đủ quyền đọc, ghi, hoặc thực thi tập tin/thư mục mục tiêu (`os.ErrPermission`).

→ Kiểm tra file mode permission (`chmod`), UID/GID của container user (non-root), hoặc chính sách SELinux/AppArmor. **Ch15,17**

E03 `file already exists`

Cố gắng tạo mới tập tin với cờ độc quyền (`os.O_EXCL`) hoặc tạo thư mục (`os.Mkdir`) khi đường dẫn đã tồn tại (`os.ErrExist`).

→ Sử dụng `os.IsExist(err)` để xử lý phân nhánh ghi đè hoặc bỏ qua nếu tập tin đã sẵn sàng. **Ch7,15**

E04 `read-only file system`

Cố gắng ghi dữ liệu vào một filesystem được mount ở chế độ chỉ đọc (lỗi hệ thống `EROFS`, ví dụ trong distroless hoặc container bảo mật cao).

→ Chuyển đường dẫn ghi file tạm sang thư mục được mount volume riêng biệt (như `/tmp` kiểu `emptyDir`). **Ch17,18**

E05 `executable file not found in $PATH`

Hàm `exec.LookPath` hoặc `exec.Command` không tìm thấy file thực thi tương ứng trong các thư mục của biến môi trường `$PATH` (`exec.ErrNotFound`).

→ Kiểm tra working directory, mount và base image thực tế; image distroless thường không có shell/ls, nhưng biến thể debug khác. Dùng đường dẫn phù hợp hoặc công cụ chẩn đoán riêng. **Ch15,17**

E06 `signal: terminated / signal: killed`

Process bị dừng đột ngột bởi tín hiệu hệ điều hành: SIGTERM khi dừng dịch vụ có phối hợp, hoặc SIGKILL khi bị cưỡng chế tiêu diệt (kill -9, hết hạn grace period, hoặc do Linux kernel OOM Killer).

> ! SIGKILL có thể đến từ nhiều nguyên nhân (admin kill, timeout của bộ điều phối, cgroup limit); không tự động suy diễn mọi SIGKILL đều là OOM.

→ Kiểm tra dmesg/journalctl hoặc Kubernetes termination reason để xác định process bị kill bởi OOM Killer hay do lệnh quản trị bên ngoài. **Ch10,12,17**

E07 `failed to load bpf program: operation not permitted / permission denied (eBPF Verifier Error)`

- `failed to load bpf program: operation not permitted`
- `failed to load bpf program: permission denied: ... (verifier log)`

Load bị từ chối có thể do quyền, chính sách host hoặc bytecode không đạt kiểm tra verifier. Capability cần thiết tùy kernel và loại program; không suy ra nguyên nhân chỉ từ một chuỗi permission denied.

! Đọc error và verifier log nếu có. Verifier phân tích program trước khi cho chạy; không phải mọi lỗi permission đều có log verifier.

→ Kiểm tra feature, hook, policy và quyền tối thiểu trước khi cấp thêm quyền. Sửa bytecode theo log cụ thể; không có quy tắc rằng mọi pointer đều phải đi qua `bpf_probe_read_*`. **Ch27**

F — NETWORK / HTTP / TLS

F01 `dial tcp: lookup host: no such host`

Hệ thống DNS resolver không thể phân giải tên miền mục tiêu thành địa chỉ IP hợp lệ (`net.DNSError`).

→ Kiểm tra cấu hình DNS cục bộ (`/etc/resolv.conf`), service discovery của cụm Kubernetes (CoreDNS), hoặc lỗi chính tả host. **Ch11,16,20**

F02 `dial tcp: connect: connection refused`

Connect bị từ chối; thường do không có listener ở đích, nhưng firewall hay thiết bị trung gian cũng có thể chủ động reject. Lỗi này không chứng minh host đích đang hoạt động bình thường.

→ Xác nhận service mục tiêu đã khởi chạy, bind đúng địa chỉ (`0.0.0.0` thay vì `127.0.0.1`), và port đích chính xác. **Ch4,11,16,20**

F03 `i/o timeout / dial tcp: i/o timeout`

Dial không hoàn tất trong ngân sách kết nối, hoặc đọc/ghi vượt deadline tương ứng. `net.Dialer.Timeout` giới hạn dial, không đặt timeout chung cho mọi lần đọc/ghi sau đó.

→ Kiểm tra tường lửa/security group có đang chặn gói tin SYN không, routing mạng, hoặc server mục tiêu bị treo cứng. **Ch11,16,20**

F04 `read: connection reset by peer`

TCP stack báo connection reset. RST có thể đến từ peer hoặc thiết bị trung gian; riêng error string không xác định nguồn gây reset.

! Process restart, policy timeout hoặc middlebox là các hướng điều tra, không phải nguyên nhân đã được chứng minh. Crash cũng không luôn tạo RST.

→ Kiểm tra log của server phía đối tác xem có sự cố panic, OOM, hay timeout của load balancer trung gian không. **Ch11,12,20**

F05 `write: broken pipe`

Lời gọi ghi bị báo EPIPE ở đường socket đang xét. TCP cho phép half-close; nhận FIN chỉ báo đóng chiều gửi của peer, không tự chứng minh mọi lần ghi tiếp phải lỗi.

→ Bắt lỗi ghi socket, ngừng gửi dữ liệu lên kết nối đã chết và dọn dẹp các tài nguyên liên đới của handler. **Ch11,12,20**

F06 `http: server closed idle connection`

Transport phát hiện server đóng một HTTP/1.x connection mà nó đang coi là idle. Idle timeout là một khả năng, không phải nguyên nhân duy nhất.

→ Kiểm tra policy server/proxy và thời điểm reuse. Retry chỉ khi request body và side effect cho phép; đặt `IdleConnTimeout` thấp hơn server có thể giảm một race, không loại bỏ mọi close. **Ch11,20**

F07 `x509: certificate signed by unknown authority`

Bắt tay TLS thất bại vì chứng chỉ số của máy chủ không được ký bởi Certificate Authority (CA) có trong trust store của client.

! `InsecureSkipVerify` bỏ kiểm tra certificate chain và hostname mặc định; không dùng để chữa lỗi production. Custom verification cần contract riêng; TLS encryption không vì cờ này mà tự biến mất.

→ Thêm Root CA nội bộ vào trust store của hệ điều hành/container hoặc nạp CA tùy biến qua `tls.Config.RootCAs`. **Ch11,17,20**

F08 `x509: certificate has expired or is not yet valid`

Chứng chỉ số TLS của máy chủ đã quá hạn sử dụng, hoặc đồng hồ hệ thống trên máy chạy Go bị lệch thời gian (time drift).

→ Kiểm tra thời hạn chứng chỉ bằng `openssl x509`, gia hạn chứng chỉ hoặc đồng bộ lại đồng hồ hệ điều hành qua NTP. **Ch11,17**

F09 `x509: certificate is valid for X, not Y`

Tên miền truy cập không khớp với danh sách Subject Alternative Names (SAN) được cấp trong chứng chỉ số TLS.

→ Cấp lại chứng chỉ với trường SAN bao quát đúng tên miền hoặc địa chỉ IP đang sử dụng để kết nối. **Ch11,17**

F10 Unread Response Body (Lost Connection Reuse)

Với HTTP/1.x, đóng body trước EOF có thể làm mất khả năng tái sử dụng kết nối. HTTP/2 có lifecycle stream riêng; không suy ra cứ đóng body sớm là toàn bộ socket phải đóng.

> ! EOF của `io.LimitReader` có thể chỉ báo hết ngân sách, không phải hết body gốc. `ReuseEligible` là cờ của helper trong lab, không phải cam kết từ Transport.

→ Đóng body; nếu chọn drain, giới hạn byte và thời gian. EOF hỗ trợ một số đường HTTP/1.x reuse, không bảo đảm reuse ở mọi Transport/protocol. **Ch11,20**

F11 `RequestTimeTooSkewed`

- `RequestTimeTooSkewed: The difference between the request time and the current time is too large.`

Diagnostic này có thể gặp từ Amazon S3 khi thời gian request lệch ngoài cửa sổ được service chấp nhận. Không áp một ngưỡng phút chung cho mọi service hoặc mọi kiểu ký AWS; đọc response và contract API thực tế.

> ! Thường gặp trong container hoặc máy ảo khi tiến trình đồng bộ thời gian NTP bị lỗi hoặc đóng băng sau khi suspend.

→ Đồng bộ lại đồng hồ hệ điều hành thông qua dịch vụ chrony hoặc NTP daemon (`chronyc makestep`). **Ch24**

G — DATABASE

G01 `sql: database is closed`

Thực hiện thao tác trên `*sql.DB` đã đóng. Đây không phải `sql.ErrConnDone`: sentinel ấy dành cho `*sql.Conn` đã trả lại pool hoặc đóng.

→ Xác định owner của pool và ai gọi Close quá sớm. Tái dùng pool lâu dài khi phù hợp; một process có thể có nhiều DB hoặc thay pool có phối hợp, không cần quy tắc duy nhất một lần suốt đời process. **Ch13,20**

G02 `driver: bad connection`

Driver báo connection không dùng được (`driver.ErrBadConn`); `database/sql` có thể retry theo contract. Đây khác với chờ pool đạt `SetMaxOpenConns`.

> ! Chờ pool có thể kết thúc khi connection được trả lại, context bị hủy/hết hạn hoặc DB đóng. Không có error string chuẩn mang tên "connection pool exhausted".

→ Điều tra connection lỗi và thống kê pool riêng. Chỉ đổi `SetMaxOpenConns` sau khi xét capacity database; context budget giới hạn chờ khi driver hỗ trợ, không sửa connection hỏng. **Ch13,20**

G03 Unclosed sql.Rows (Connection Pool Starvation)

Không đóng `Rows` khi dừng duyệt sớm có thể giữ connection ngoài pool. Đọc tới EOF có thể tự đóng Rows theo contract, nhưng caller vẫn nên đóng tường minh và kiểm tra `rows.Err()`.

> ! Không có diagnostic chuẩn mang nghĩa "quên close". Connection bị giữ có thể làm các query sau chờ pool; xem `DB.Stats`, cancellation và lifecycle thực tế, không suy ra mọi truy vấn đều treo tới timeout.

→ Sau khi `QueryContext` thành công, đặt `defer rows.Close()` gần acquisition khi cùng hàm sở hữu Rows. Nếu trả Rows cho caller, phải chuyển rõ quyền đóng. **Ch13,20**

G04 `sqlite: database is locked / busy`

SQLite gặp xung đột ghi đồng thời từ nhiều transaction hoặc goroutine trên cùng một tập tin cơ sở dữ liệu (`busy / locked`).

→ Cân nhắc WAL, busy timeout hoặc tuần tự hóa ghi theo workload. Cú pháp DSN/pragma tùy driver; các tên `_journal_mode` và `_busy_timeout` không phải contract chung của `database/sql`. **Ch13,20**

G05 `UNIQUE constraint failed`

Cố gắng chèn hoặc cập nhật bản ghi có giá trị trùng lặp trên cột được đánh chỉ mục duy nhất (PRIMARY KEY hoặc UNIQUE).

→ Giữ UNIQUE constraint làm rào chắn nguyên tử và xử lý conflict theo nghiệp vụ, có thể dùng upsert phù hợp. Check tồn tại rồi ghi vẫn có race; không thay constraint bằng check-then-act. **Ch13,20**

G06 `FOREIGN KEY constraint failed`

Thao tác insert hoặc delete vi phạm tính toàn vẹn quan hệ khóa ngoại giữa bảng cha và bảng con.

→ Kiểm tra foreign key, transaction, deferred constraint và cascade của database thật; không áp một thứ tự insert/delete duy nhất cho mọi schema. **Ch13,20**

H — CONCURRENCY

H01 `fatal error: all goroutines are asleep - deadlock!`

Runtime phát hiện không còn công việc có thể tiến triển theo các điều kiện kiểm tra nội tại của version đang chạy.

> ! Goroutine chờ I/O hoặc timer không tự đồng nghĩa deadlock. Runtime không phát hiện mọi vòng chờ cục bộ; goroutine khác còn hoạt động có thể che một nhóm đang kẹt.

→ Thiết kế đồ thị phụ thuộc của lock/channel; luôn tuân thủ thứ tự acquisition lock nhất quán và tránh chờ đợi vòng tròn. **Ch8,9,12**

H02 `WARNING: DATA RACE`

Go race detector (`go test -race` / `go run -race`) phát hiện các truy cập xung đột tới cùng vùng nhớ, có ít nhất một lần ghi và thiếu quan hệ đồng bộ cần thiết trên đường chạy được quan sát.

> ! Không bỏ qua data race. Kết quả có thể không nhất quán hoặc gây crash; một lần chạy không crash không chứng minh chương trình đúng.

→ Bảo vệ vùng nhớ dùng chung bằng `sync.Mutex`, chuyển sang toán tử `sync/atomic`, hoặc tái cấu trúc truyền dữ liệu qua channel. **Ch8,9,20**

H03 Goroutine Leak

Goroutine không kết thúc vì không còn đường thoát khỏi chờ channel, lock hoặc I/O. Context chỉ hữu ích nếu operation thực sự quan sát cancellation.

> ! Loop nền vẫn chạy định kỳ cũng có thể vượt lifecycle của owner. `Ticker.Stop` không đóng channel hoặc kết thúc reader; với cơ chế timer mới từ Go 1.23, ticker không còn tham chiếu có thể được GC thu hồi, không phải mọi lần thiếu Stop đều là leak. [Ch9]

→ So số goroutine qua các chu kỳ với dump/profile, xác nhận receive/loop/callback thiếu đường thoát trong source rồi chờ `done` sau sửa. Heap ổn không loại trừ lỗi vòng đời. **Ch9,10,12**

> ! Runtime không tự thu hồi một goroutine chỉ vì nó bị block; những reference còn sống có thể giữ dữ liệu. Go 1.27 có profile `goroutineleak` cho một lớp chờ không thể được đánh thức, không phải bộ phát hiện mọi leak hay cơ chế GC goroutine.

→ Xác định owner và đường thoát; truyền context/deadline tới API có hỗ trợ. Buffer chỉ đổi thời điểm block, không tự sửa leak. **Ch8,9,12,20**

H04 `panic: sync: negative WaitGroup counter`

Phương thức `wg.Add()` nhận giá trị âm làm counter nhỏ hơn 0, hoặc phương thức `wg.Done()` bị gọi nhiều lần hơn số lần `Add()`.

→ Kiểm tra số lần `Done()` và tăng counter trước khi khởi chạy worker trong mẫu đang học. `Add` dương khi counter bằng 0 phải xảy ra trước `Wait`; không để worker mới tự tăng sau khi caller đã có thể `Wait`. **Ch8,9**

H05 `panic: sync: WaitGroup misuse: Add called concurrently with Wait`

Diagnostic này không có nghĩa mọi `Add` đồng thời với `Wait` đều sai. Theo contract, `Add` dương khi counter đang bằng 0 phải xảy ra trước `Wait`; khi counter đã dương, quy tắc khác áp dụng.

→ Trong mẫu worker đang học, đăng ký task trước khi khởi chạy; nếu tái sử dụng WaitGroup, chỉ thêm lứa mới sau khi mọi `Wait` của lứa cũ đã trả về. **Ch8,9**

I — MODULES / TEST / TOOLCHAIN

I01 `go: cannot find main module`

Lệnh `go build` hoặc `go test` được thực thi tại một thư mục không chứa file `go.mod` và không thuộc không gian làm việc `go.work`.

→ Di chuyển terminal vào đúng thư mục con chứa module cần thao tác (`working-directory`), hoặc khởi tạo module bằng `go mod init`. **Ch1,5,18**

I02 `go: module X: git fetch: authentication required`

Go module resolver không thể tải mã nguồn của module private do thiếu thông tin xác thực Git qua SSH hoặc Personal Access Token.

→ Cấu hình biến môi trường `GOPRIVATE=github.com/myorg/*` và thiết lập Git credential helper hoặc SSH key tương ứng. **Ch5,18**

I03 `go vet: composite literal uses unkeyed fields`

Khai báo struct literal không chỉ định tên field tường minh (ví dụ viết `Point{1, 2}` thay vì `Point{X: 1, Y: 2}`).

→ Bổ sung tên field vào struct literal để tránh lỗi vỡ mã nguồn khi struct được thêm trường mới trong tương lai. **Ch3,6**

I04 `go vet: unreachable code`

Đoạn mã nằm ở vị trí không bao giờ có thể được thực thi tới (ngay sau lệnh `return`, `panic`, hoặc vòng lặp vô tận không có break).

→ Loại bỏ đoạn mã chết (dead code) không cần thiết hoặc điều chỉnh lại cấu trúc điều hướng control-flow. **Ch1,4**

I05 `go test: no Go files in directory`

Thư mục chỉ định trong lệnh test không chứa bất kỳ tập tin mã nguồn Go (`*.go`) hợp lệ nào thuộc package cần kiểm thử.

→ Kiểm tra lại đường dẫn package hoặc bổ sung tập tin kiểm thử `*_test.go` tương ứng vào thư mục. **Ch6,18**

I06 `go test: build constraints exclude all Go files`

Tất cả các file mã nguồn Go trong thư mục đều bị loại trừ bởi build tags chỉ định ở đầu file (ví dụ `//go:build integration`).

→ Truyền cờ build tags tương ứng khi thực thi lệnh kiểm thử, ví dụ: `go test -tags=integration ./...`. **Ch6,18**

I07 `go test: timed out after Xm`

Toàn bộ suite kiểm thử hoặc một test cụ thể chạy vượt quá giới hạn thời gian cho phép (mặc định 10 phút hoặc cờ `-timeout`).

→ Điều tra các test bị treo do deadlock, rò rỉ goroutine hoặc sleep quá lâu; chạy `go test -v -timeout=30s` để cô lập test lỗi. **Ch6,8,20**

I08 `API rate limit exceeded`

- `API rate limit exceeded for user ID <id>`

GitHub REST API có thể trả HTTP 403 hoặc 429 khi vượt primary hay secondary rate limit. Theo tài liệu ngày 29-09-2026, mức cơ bản là 5.000 request/giờ cho user được xác thực và 60 cho IP không xác thực; loại token, App, endpoint và Enterprise có ngoại lệ. Dùng header thực tế, không gán mức ấy cho mọi token.

! Thường gặp khi các bot tự động hóa hoặc CI/CD pipeline gửi yêu cầu liên tục trong vòng lặp mà không kiểm tra hạn mức còn lại.

→ Bóc tách header `X-RateLimit-Reset` hoặc `Retry-After` để tạm dừng tiến trình (sleep with jitter) cho đến khi quota được khôi phục trước khi gửi yêu cầu tiếp theo. **Ch25**

J — CONTAINER / KUBERNETES / CI-CD

J01 CrashLoopBackOff

Container liên tục kết thúc rồi được restart theo policy, và kubelet đang áp backoff. Exit 0 dưới `restartPolicy: Always`, failure hay liveness kill đều có thể dẫn tới vòng restart; không chỉ panic hoặc exit code dương.

! Đây là waiting reason của container được phản ánh trong trạng thái Pod, không phải chuỗi lỗi của Go runtime hay một Pod phase riêng.

→ Kiểm tra log của container trước khi crash bằng lệnh `kubectl logs <pod> --previous`, rà soát biến môi trường và config bị thiếu. **Ch17,18,21**

J02 OOMKilled (Exit Code 137)

Container runtime/kubelet báo container bị OOM kill. Có thể liên quan cgroup memory limit hoặc áp lực OOM của host; cần đọc reason và sự kiện, không suy từ RSS của một process đơn lẻ.

! Exit code 137 thường biểu thị SIGKILL và không riêng OOM. SIGKILL không thể bị handler chặn, nên process không chạy defer để dọn tài nguyên.

→ Kiểm tra rò rỉ bộ nhớ qua heap profile (`pprof`), điều chỉnh thuật toán caching hoặc nâng hạn mức `limits.memory` cho pod. **Ch10,17,21**

J03 ImagePullBackOff / ErrImagePull

Waiting reason của container, được phản ánh trong Pod status, khi kéo image lỗi hoặc đang backoff. Sai tag, thiếu credential và lỗi mạng là các hướng điều tra, không phải nguyên nhân đã được xác nhận chỉ từ reason.

! Kubelet trên node phối hợp với container runtime để kéo image; trạng thái được báo về API. Lỗi này xảy ra trước khi container Go khởi chạy.

→ Kiểm tra tên image và tag, cấu hình `imagePullSecrets` cho service account, hoặc kiểm tra kết nối mạng egress của cụm. **Ch17,18**

J04 CreateContainerConfigError

Trạng thái lỗi cấu hình (`Waiting Reason`) khi Kubernetes không tìm thấy ConfigMap hoặc Secret được tham chiếu trong cấu hình Pod.

! Kubelet gặp lỗi khi chuẩn bị cấu hình container; không phải diagnostic của Go hay riêng thao tác chọn node của scheduler.

→ Chạy `kubectl describe pod <pod>` để xác định chính xác tên ConfigMap/Secret đang bị thiếu và tạo bổ sung vào namespace. **Ch17,21**

J05 Readiness probe failed

- `HTTP probe failed with statuscode: 503`

Probe readiness thất bại theo cấu hình; khi đạt failure threshold, Pod Ready có thể thành false. EndpointSlice có thể vẫn giữ endpoint với `ready=false`; behavior Service còn phụ thuộc `publishNotReadyAddresses`.

! Readiness failure không tự restart container và cũng không chứng minh process đang chạy bình thường; nó chỉ phản ánh kết quả của probe đã chọn.

→ Kiểm tra tình trạng kết nối đến các dependency hạ tầng (database, cache) và rà soát logic kiểm tra sẵn sàng của handler. **Ch12,17,20,21**

J06 Liveness probe failed

Liveness probe thất bại; đạt failure threshold dẫn tới xử lý restart container theo lifecycle và restart policy. Một lần probe lỗi chưa đủ suy ra container bị restart ngay.

> ! Tránh dùng lỗi database xa làm liveness failure nếu restart không chữa được nó; policy sai có thể gây cascading restart.

→ Chọn probe mà restart có khả năng chữa failure. Thường tránh đưa dependency xa vào liveness nếu nó gây restart dây chuyền; đây là khuyến nghị thiết kế, không phải giới hạn API chỉ cho kiểm tra nội tại. **Ch12,17,21**

J07 Admission Gate Rejected

- `unsigned artifact or digest mismatch`

Trạng thái từ chối (`Admission / Policy Status`) từ Kubernetes Validating Admission Webhook hoặc Promotion Gate của CI/CD khi artifact container image không có chữ ký cosign hợp lệ hoặc digest SHA-256 không khớp với bản ghi phát hành.

> ! Đây là rào chắn chính sách ở tầng CI/CD delivery hoặc Kubernetes admission webhook, không phải lỗi từ runtime của ứng dụng.

→ Bảo đảm image được build và ký số thông qua pipeline CI chính thức có attestation/provenance hợp lệ trước khi promote. **Ch18**

J08 `Operation cannot be fulfilled: the object has been modified`

- `Operation cannot be fulfilled on <resource>: the object has been modified; please apply your changes to the latest version and try again`

API server trả HTTP 409 Conflict khi update gửi `metadata.resourceVersion` không còn khớp version hiện hành. Đối chiếu API semantics; không cần giả định client thấy hay thao tác trực tiếp với bản ghi etcd.

> ! Đây là cơ chế kiểm soát đồng thời lạc quan (Optimistic Concurrency Control) của Kubernetes nhằm ngăn chặn ghi đè mất dữ liệu giữa các client cạnh tranh.

→ Sử dụng `k8s.io/client-go/util/retry.RetryOnConflict` để đọc lại snapshot mới nhất từ API Server và áp dụng thay đổi trước khi thử lại. **Ch22**

J09 `no kind is registered for the type in scheme`

- `no kind is registered for the type <Type> in scheme <Scheme>`

Lỗi cấu hình runtime (`Kubernetes Scheme Error`) khi một Go struct được truyền vào Controller Client hoặc Reconciler nhưng kiểu dữ liệu này chưa được đăng ký vào bảng `runtime.Scheme` thông qua hàm `AddToScheme`.

> ! Xảy ra phổ biến khi khởi tạo Operator với Custom Resource Definition (CRD) nhưng quên nạp `v1alpha1.AddToScheme(mgr.GetScheme())`.

→ Đăng ký SchemeBuilder của CRD vào Manager Scheme trước khi khởi động Controller. **Ch23**

J10 Denied by Supply Chain Policy: reachable vulnerability or untrusted builder

- `reachable <SEVERITY> vulnerability <ID> in <package> (symbol: <symbol>)`
- `untrusted builder identity: <builder>`

Trạng thái chặn theo policy khi công cụ phân tích tìm đường gọi có thể tới ký hiệu liên quan lỗ hổng, hoặc identity của builder không được chấp nhận.

> ! Reachability tĩnh không chứng minh đường ấy đã chạy, khai thác thành công hay có mã độc. Phân biệt kết quả phân tích với trace thực nghiệm.

→ Cập nhật phiên bản dependency đã vá lỗi, loại bỏ việc gọi ký hiệu chứa lỗ hổng, hoặc kiểm tra lại builder ID trong CI workflow. **Ch26**

J11 MCP Tool Execution Denied: unauthorized mutation or SSRF boundary violation

- `mcp: tool authorization denied for role <role> on tool <tool>`
- `mcp: ssrf blocked: target ip <ip> belongs to private/metadata range`

Trạng thái từ chối (`MCP Security Boundary Status`) khi Agent AI cố gắng kích hoạt công cụ làm thay đổi trạng thái hạ tầng (Mutating Tool) mà không có quyền hạn, hoặc truyền tham số URL vi phạm ranh giới phòng vệ SSRF (hướng tới AWS Metadata Service `169.254.169.254` hoặc Loopback `127.0.0.1`).

> ! Schema kiểm tra cấu trúc và constraint đã khai báo, không thay authorization, kiểm soát destination hay phòng prompt injection. Handler, gateway hoặc lớp mạng phải thực thi policy phù hợp; không phải yêu cầu bắt buộc dùng Go.

→ Kiểm tra role của Agent trong phiên MCP, cấu hình whitelist URL cho công cụ mạng, hoặc cấp token ủy quyền (Change Request ID) trước khi gọi các tool gây đột biến. **Ch28**